SQL Handbook — Vol 2 (Ch 12–21)

Queries · joins · window functions · data modeling · normalization · indexes · transactions · scaling · 40+ interview problems — PostgreSQL first

10 parts · one place to study: DesignGurus courses + classic books + the web, combined, beginner to intermediate

what it is · problem · how it works · trade-offs · real world · interview · self-check · TL;DR

Personal study notes. Course diagrams belong to DesignGurus. Not for redistribution.

Contents

1. **Chapter 12 — Indexes, Storage and Query Performance**
   - 12.1 How a database stores data
   - 12.2 Data compression and encoding
   - 12.3 File organization
   - 12.4 What an index is
   - 12.5 Single-level indexing, with numbers
   - 12.6 Tree-based indexes: B-trees and B+ trees
   - 12.7 Hash-based indexing
   - 12.8 Bloom filters
   - 12.9 PostgreSQL's other index types: GIN, GiST, BRIN
   - 12.10 Reading EXPLAIN: the four ways to read a table
   - 12.11 Composite indexes and column order
   - 12.12 Covering, partial and expression indexes
   - 12.13 Anti-patterns that disable indexes
   - 12.14 OFFSET vs keyset pagination
   - 12.15 Join algorithms
   - 12.16 Statistics: how the planner estimates
   - 12.17 A tuning workflow
   - Chapter TL;DR
2. **Chapter 13 — Transactions and Concurrency**
   - 13.1 What a transaction is
   - 13.2 Atomicity
   - 13.3 Consistency
   - 13.4 Isolation — anomalies and isolation levels
   - 13.5 Durability
   - 13.6 Watching isolation anomalies on a real server
   - 13.7 Lost updates and how to prevent them
   - 13.8 Write skew and Serializable isolation
   - 13.9 Concurrency control — the big picture
   - 13.10 Lock-based concurrency control
   - 13.11 Deadlocks
   - 13.12 Row locks for work queues: `FOR UPDATE`, `NOWAIT`, `SKIP LOCKED`
   - 13.13 Timestamp-based concurrency control
   - 13.14 Multiversion concurrency control (MVCC)
   - 13.15 Putting it together
   - Chapter TL;DR
3. **Chapter 14 — Advanced SQL and Using SQL from Code**
   - 14.1 User-defined functions
   - 14.2 Stored procedures
   - 14.3 Triggers
   - 14.4 JSON and JSONB
   - 14.5 Arrays
   - 14.6 Full-text search
   - 14.7 Pivot and unpivot
   - 14.8 Security: roles, privileges and row-level security
   - 14.9 SQL injection and parameterized queries
   - 14.10 Using SQL from Python
   - 14.11 Using SQL from Java (JDBC)
   - 14.12 Connection pooling
   - 14.13 ORMs vs raw SQL
   - Chapter TL;DR
4. **Chapter 15 — Scaling Databases: Partitioning, Sharding, Replication and Recovery**
   - 15.1 Distributed databases: the big picture
   - 15.2 Partitioning
   - 15.3 Sharding
   - 15.4 Distributed query processing
   - 15.5 Replication
   - 15.6 Consistency models
   - 15.7 The CAP theorem (and PACELC)
   - 15.8 Consensus and leader election
   - 15.9 Fault tolerance and recovery
   - 15.10 High availability: replication plus failover
   - 15.11 Data resilience: redundancy, mirroring, backups
   - Chapter 15 TL;DR
5. **Chapter 16 — Beyond Relational: NoSQL, NewSQL and Specialised Databases**
   - 16.1 Why NoSQL?
   - 16.2 Key-value stores
   - 16.3 Document databases
   - 16.4 Column-family and wide-column stores
   - 16.5 Graph databases
   - 16.6 In-memory databases
   - 16.7 NewSQL / distributed SQL
   - 16.8 Time-series and spatial databases
   - 16.9 Big data and distributed processing frameworks
   - 16.10 Choosing a database
   - 16.11 Inside two workhorses: Elasticsearch (and the ELK stack) and Redis
   - Chapter 16 TL;DR
6. **Chapter 17 — Schema Design Case Studies**
   - 17.0 The template
   - 17.1 Hospital management system
   - 17.2 Hotel reservation system
   - 17.3 Banking system
   - 17.4 Instagram-style photo sharing (runnable)
   - 17.5 E-commerce store (runnable)
   - 17.6 Food delivery platform
   - 17.7 Library management system (runnable)
   - 17.8 Ticket booking system (runnable, with seat-hold concurrency)
   - 17.9 Warehouse inventory system
   - 17.10 Quora (questions and answers)
   - 17.11 TikTok (short-video platform)
   - 17.12 Airbnb (short-term rentals)
   - 17.13 Snapchat (ephemeral messaging)
   - 17.14 Recurring modeling patterns

- Chapter 17 TL;DR

7. **Chapter 18 — SQL Interview Problems I: Method and 42 Warm-Up to Medium Problems**
   - 18.1 How to approach a SQL interview question
   - 18.2 Filtering problems (P1–P8)
   - 18.3 Aggregation problems (P9–P16)
   - 18.4 Join problems (P17–P22)
   - 18.5 Anti-join problems (P23–P26)
   - 18.6 Conditional aggregation (P27–P28)
   - 18.7 Date-logic problems (P29–P33)
   - 18.8 Ratios and percentages (P34–P38)
   - 18.9 Window and ranking basics (P39–P42)
   - 18.10 Pattern table
   - Chapter 18 TL;DR
8. **Chapter 19 — SQL Interview Problems 2: Medium and Hard Patterns**
   - 19.1 How to attack a medium or hard problem
   - 19.2 Warm-ups: joins, counts and grids
   - 19.3 Nth highest, top-N per group and "first row per group"
   - 19.4 Running totals and row-to-row comparisons
   - 19.5 Gaps and islands: consecutive rows and streaks
   - 19.6 Pivot and unpivot (static and dynamic)
   - 19.7 Self-joins, "both directions" and graph-like questions
   - 19.8 Date bucketing and retention
   - 19.9 Ratios and cancellation rates
   - 19.10 String building and expression evaluation
   - 19.11 Pattern catalogue
   - Chapter 19 TL;DR
9. **Chapter 20 — Cheat Sheets, Interview Q&A and Glossary**
   - 20.1 The logical query-processing order
   - 20.2 SELECT, filtering, NULL, sorting and paging
   - 20.3 Functions and expressions
   - 20.4 Aggregation
   - 20.5 Joins
   - 20.6 Subqueries and set operations
   - 20.7 CTEs, recursion and views
   - 20.8 Window functions and frames
   - 20.9 DDL, DML, constraints and upsert
   - 20.10 Data modeling and normalization quick rules
   - 20.11 Indexes and reading EXPLAIN
   - 20.12 Transactions, isolation and concurrency
   - 20.13 Advanced PostgreSQL: PL/pgSQL, JSONB, full-text search, security
   - 20.14 Scaling: partitioning, sharding, replication
   - 20.15 Which database?
   - 20.16 Dialect differences
   - 20.17 Verified behaviours
   - 20.18 Interview pattern catalogue
   - 20.19 Interview questions and answers
   - 20.20 Glossary (A–Z)
   - Chapter 20 TL;DR
10. **Chapter 21 — Interview Question Bank**
    - How to use this bank
    - Part A — Concept questions
    - Part B — Coding problems
    - Sources of the question lists

Chapter 12 — Indexes, Storage and Query Performance

This chapter asks *how fast* a query runs, and why. The answer always comes down to: **how many pages of data does the database have to touch?** That depends on how rows sit on disk, what an index is, how the query planner (the part of the database that decides *how* to run a query) chooses between reading everything and jumping to the rows it needs, and which query habits defeat an index. Every claim is measured on PostgreSQL 14 with a one-million-row table.

What you'll learn: storage (hierarchy, 8 KB slotted pages, heap files, TOAST, row vs column layout, compression) · file organizations · index kinds (primary/clustering/secondary, dense/sparse, multilevel arithmetic) · B+ trees, hashing (static, extendible, linear), Bloom filters, GIN/GiST/BRIN/bloom · reading `EXPLAIN (ANALYZE, BUFFERS)` · composite, covering, partial and expression indexes · anti-patterns (functions on columns, leading wildcards and `pg_trgm`, `OR`, OFFSET vs keyset, N+1) · join algorithms and planner statistics.

How to run the examples. The runner builds a fresh database per file in `code/`, loading `00_schema.sql` first (~5 s; it generates one million orders):

```
bash tools_run_sql.sh sql '12-*'                                  # all files of this chapter
bash tools_pg.sh psql -d ch_12 -f sql/chapters/12-indexes-storage-and-query-performance/code/02_scans.sql
cd sql/chapters/12-indexes-storage-and-query-performance/code
python3 bloom_filter_sim.py; python3 compression_demo.py
../../../../.venv-books/bin/python n_plus_one.py                  # needs psycopg
```

> **About the numbers.** Timings depend on the machine and cache; **buffer counts** (pages touched) and **plan shapes** are what to compare. Long plan lines are trimmed with `...`.

The data is a small shop; every value is computed from the row number, so every run builds the same tables:

```
CREATE TABLE customers (id int PRIMARY KEY, name text, email text, country text, signup_on date); -- 100,000 rows
CREATE TABLE orders (id bigint PRIMARY KEY, customer_id int REFERENCES customers(id),
                     status text, amount numeric(10,2), created_at timestamp, note text);
INSERT INTO orders
SELECT g, (g::bigint * 7919) % 100000 + 1,               -- exactly 10 orders per customer, scattered
       CASE WHEN g % 100 < 2 THEN 'pending' WHEN g % 100 < 5 THEN 'refunded'
            ELSE 'shipped' END,                          -- 2 % / 3 % / 95 %
       ((g::bigint * 37) % 50000) / 100.0 + 1,
       TIMESTAMP '2024-01-01' + g * INTERVAL '31 seconds', -- grows with id
       CASE WHEN g % 10 = 0 THEN 'gift wrap please' END   -- 90 % NULL
FROM generate_series(1, 1000000) AS g;
VACUUM ANALYZE orders;
```

`VACUUM ANALYZE` cleans the table and gathers planner statistics (12.16). The schema file also defines `plan_stats(query)`, returning just execution time and pages touched, for side-by-side tables.

12.1 How a database stores data

What it is

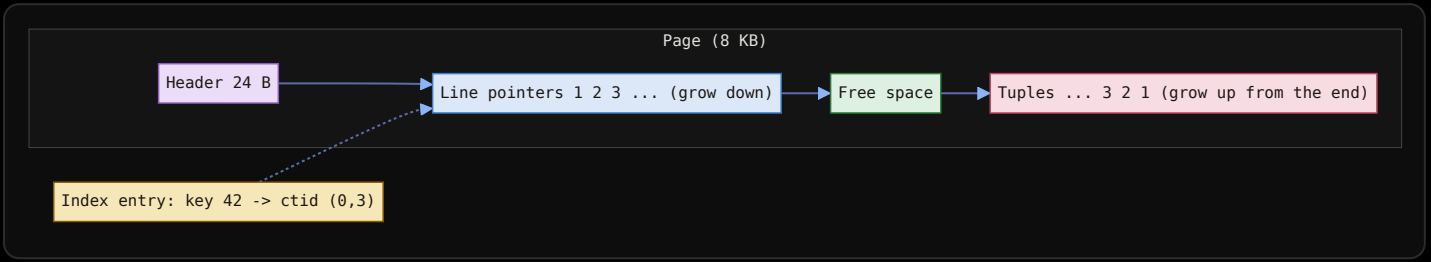
A database keeps data on **persistent storage** and works on it in **memory**; the layers differ by a factor of thousands in speed:

Level	Examples	Survives power loss?	Rough access time	Role in a DBMS
CPU cache / RAM (main memory; primary storage)	L1–L3 cache, DRAM	No (volatile)	~1–100 ns	Buffer pool: cached pages, sort and hash memory
SSD / NVMe (secondary storage)	flash drives	Yes	~20–100 µs	Data files, indexes, write-ahead log (the change log written before the data files)
Hard disk (secondary)	spinning disks	Yes	~5–10 ms per random seek	Same, on older or cheaper servers
Tertiary / archive	tape, object storage (S3 Glacier)	Yes	seconds to hours	Backups, cold archives

So a DBMS never fetches single rows: it moves fixed-size **blocks** (**pages**) — 8 KB in PostgreSQL and SQL Server, 16 KB in MySQL InnoDB, 4 KB by default in SQLite. RAM holding recently used pages is the **buffer pool** (PostgreSQL: `shared_buffers`). A read finding its page there is a *hit*; one going to the operating system is a *read* — the two words in `EXPLAIN (BUFFERS)` output.

How a table is laid out

1. Each table is one or more **files** (PostgreSQL: one per 1 GB segment under `base/<database-oid>/`, *oid* = internal object number) — a sequence of pages 0, 1, 2, …
2. Each page is a **slotted page**: header, an array of **line pointers** (slots) growing from the front, row data (**tuples**) growing from the back, free space in the middle.
3. A row's address is (page, slot) — PostgreSQL's hidden column `ctid`, which indexes store. Because it points at a *slot*, not a byte offset, the page can shuffle tuples internally without breaking any index.
4. **TOAST** (The Oversized-Attribute Storage Technique): when a row exceeds about 2 KB, big values are compressed and, if still too big, chunked into a separate TOAST table.



Seeing it on the real server

```
SELECT relname, pg_size_pretty(pg_relation_size(oid)) AS main_fork, ... FROM pg_class WHERE ...;
SELECT ctid, id, status, amount FROM orders WHERE id IN (1, 2, 3, 124, 125) ORDER BY id;
SELECT lower, upper, special, pagesize FROM page_header(get_raw_page('orders', 0));
SELECT lp, lp_off, lp_len, t_ctid FROM heap_page_items(get_raw_page('orders', 0)) LIMIT 4;
```

Output (`01_storage_pages.sql`, using the `pageinspect` extension for raw pages):

```
   relname   | main_fork | pages | est_rows
-------------+-----------+-------+----------
 customers   | 7344 kB   |   918 |   100000
 orders      | 68 MB     |  8662 |  1000000
 orders_pkey | 21 MB     |  2745 |  1000000

 ctid  | id  |  status  | amount
-------+-----+----------+--------
 (0,1) |   1 | pending  |   1.37
 (0,2) |   2 | refunded |   1.74
 (0,3) |   3 | refunded |   2.11
 (1,8) | 124 | shipped  |  46.88
 (1,9) | 125 | shipped  |  47.25

 lower | upper | special | pagesize
-------+-------+---------+----------
   488 |   504 |    8192 |     8192

 lp | lp_off | lp_len | t_ctid
----+--------+--------+--------
  1 |   8128 |     64 | (0,1)
  2 |   8064 |     64 | (0,2)
  3 |   8000 |     64 | (0,3)
  4 |   7936 |     64 | (0,4)
```

- One million orders occupy **8,662 pages**; a full scan touches all of them — the yardstick for everything that follows.
- Page 0 holds 116 rows (row 124 is the 8th row of page 1). Each tuple is 64 bytes: a 23-byte header (transaction ids used by MVCC — *multi-version concurrency control*, keeping old row versions so readers and writers don't block each other, chapter 13) plus aligned column values.
- `lower = 488` (slot array end) and `upper = 504` (tuple data start) leave 16 bytes: the page is full. Tuple 1 sits at the very end (8128 + 64 = 8192); later tuples go in front of it.

TOAST in action (`pg_column_size` reports bytes actually stored; a value stays uncompressed when compression would not save enough):

```
 id | logical_bytes | stored_bytes
----+---------------+--------------
  1 |            30 |           31      -- tiny: stored inline, 1-byte length header
  2 |         40000 |          477      -- 'abcdefgh' x 5000: compressed ~84x
  3 |         48000 |        48000      -- hex digests: incompressible, moved out of line as is
```

(Hex digests — hash outputs in hexadecimal — are effectively random, so they don't compress.)

From memory to bytes: serialization, endianness, alignment

- **Serialization** turns an in-memory value into bytes for a page or the network; **deserialization** rebuilds it. Formats differ per engine: PostgreSQL's tuple layout, MongoDB's BSON (binary JSON), Parquet's (a columnar file format) column chunks. Compact formats mean fewer pages.
- **Endianness** is the byte order of multi-byte numbers: *big-endian* most significant byte first, *little-endian* least first (x86 and ARM are little-endian). 258 is `01 02` big-endian, `02 01` little-endian. Formats must fix one order — why PostgreSQL's binary files are not portable across architectures while `pg_dump` output is.
- **Alignment**: CPUs read a 4- or 8-byte value fastest at an address that is a multiple of its size, so engines pad. In PostgreSQL column order affects row size: `(bool, bigint, bool, bigint)` pads each `bool` to 8 bytes, `(bigint, bigint, bool, bool)` does not. Index pages are aligned for the same reason.

Row stores vs column stores

Layout	How a page is filled	Good at	Examples
Row-oriented	All columns of a row together	OLTP: fetch or change whole rows by key	PostgreSQL, MySQL InnoDB, SQL Server (rowstore), SQLite
Column-oriented	All values of *one column* together	Analytics: scan a few columns of billions of rows; compresses very well	ClickHouse, Amazon Redshift, Snowflake, DuckDB, Parquet files, SQL Server columnstore

Summing `amount` in a row store reads every whole row; a column store reads only the `amount` column. More in DDIA (*Designing Data-Intensive Applications*): storage and retrieval.

Self-check

1. Why read whole pages, not single rows? → Storage transfers blocks anyway, and a random access costs far more than the extra bytes; grouping rows per page spreads the cost over many rows.
2. What is a `ctid`? → The (page, slot) address of a row version; indexes store it.

12.2 Data compression and encoding

What it is

Compression stores the same information in fewer bytes: **lossless** (original rebuilt exactly — always the case for table data) or **lossy** (detail thrown away, e.g. JPEG, MP3 — never used for table values). **Encoding** is compression exploiting a column's *type and shape* rather than treating it as a byte stream. Benefit: fewer pages to read, more rows in the buffer pool, cheaper storage and transfer. Price: CPU to compress/decompress, sometimes slower in-place updates.

Technique	Idea	Works best on	Used by
Run-length encoding (RLE)	Store (value, how many times in a row)	Sorted or slowly changing columns	Column stores, Parquet, bitmap indexes
Dictionary encoding	Each distinct value once; rows hold a small integer code	Few distinct values (country, status)	Parquet, ClickHouse `LowCardinality`, Redshift
Delta encoding	First value, then differences	Timestamps, ids, sorted numbers	Time-series DBs (Gorilla/Prometheus), Parquet
Bit-packing	Only as many bits as the largest value needs	Small integers (codes, deltas)	Combined with the two above
Lempel–Ziv (LZ77/LZ78 family)	Replace a repeated byte sequence with a reference ("copy 8 bytes from 40 bytes back")	Text, repeated strings; no type knowledge	PostgreSQL TOAST (`pglz`, `lz4` since PG 14), zlib/zstd page compression
Huffman coding	Short bit codes for frequent symbols, long for rare	Skewed symbol frequencies	Inside zlib/DEFLATE and zstd

`compression_demo.py` applies each to a 100,000-value column:

```
status  = ["shipped"] * 60_000 + ["pending"] * 25_000 + ["refunded"] * 15_000   # sorted
country = [["IN", "US", "DE", "BR", "JP"][(i * 7) % 5] for i in range(N)]      # 5 distinct
ts      = [1_700_000_000 + i * 31 + (i % 7) for i in range(N)]                  # increasing
# rle(), dictionary(), delta() are ~6 lines each; see the file
```

```
status   raw text               :   715,000 bytes
status   RLE (3 runs)           :        34 bytes  -> [['shipped', 60000], ['pending', 25000], ['refunded',
15000]]
country  raw text               :   200,000 bytes
country  dictionary + 3-bit codes:    37,510 bytes  (dictionary ['IN', 'DE', 'JP', 'US', 'BR'], codes [0, 1, 2, 3,
4, 0]...)
ts       raw 8-byte ints        :   800,000 bytes
ts       delta + 6-bit packing  :    75,007 bytes  (first 1700000000, deltas [32, 32, 32, 32, 32, 32]...)
ts       zlib on the text       :   273,868 bytes  (no knowledge of the data type)
```

Type-aware encodings beat the general-purpose compressor by far and can often be queried without decompressing (counting `'pending'` rows = reading one run length) — why column stores sort data and pick an encoding per column. **When not to use it:** for hot, small OLTP rows the CPU cost usually isn't worth it (PostgreSQL compresses only rows over ~2 KB).

12.3 File organization

File organization decides *where* in the data file a new record goes, and therefore how records are found later.

Organization	Where records go	Fast	Slow	Real systems
Heap (unordered)	Any page with free space (usually the end)	Inserts; full scans	Finding one record without an index	PostgreSQL tables, SQL Server heaps, Oracle default
Sequential (sorted)	In order of a key	Ranges and ordered output on that key; binary search (repeatedly halving the range)	Inserts/deletes in the middle (shift records or overflow pages, reorganize periodically)	Old ISAM files, sorted SSTables in LSM engines
Hashed	Page = hash(key) mod number of buckets	Equality on the key: ~1 page read	Ranges and ordering; growing beyond the bucket count	Hash clusters (Oracle), hash-partitioned stores
ISAM (indexed sequential access method, IBM's classic)	Sorted data file plus a static multilevel index built at load time; later inserts go to overflow pages	Reads and ranges on a stable file	Overflow chains grow until reorganized	IBM mainframe ISAM files
B+ tree file organization	Records in the leaves of a B+ tree on the key (12.6), which rebalances on insert	Point and range access that stays fast under inserts	Free space in every leaf; splits	Same as the next row
Clustered / index-organized	The table *is* a B+ tree ordered by a key; leaves hold the rows	Primary-key lookups and ranges	Random-order inserts cause page splits; secondary indexes must store the key	MySQL InnoDB, SQL Server clustered index, Oracle IOT (index-organized table), SQLite rowid / `WITHOUT ROWID` tables

Key terms: a **hash function** turns a key into a seemingly random number; `mod N` then picks one of N **buckets**. An **LSM** (log-structured merge) engine buffers writes in memory and flushes them as

> immutable sorted files (**SSTables**, sorted string tables), merged in the background — fast writes, reads check several files.

A clustered file keyed by department stores department 7's employees in the same few blocks, so "list department 7" reads only those. PostgreSQL has only heap tables; `CLUSTER` sorts a heap **once** by an index, but new rows are not kept in order.

Why physical order matters — measured

Each customer's 10 orders are scattered (the generator jumps 7,919 customers per row). Summing 1,000 customers' orders (10,000 rows) before and after `CLUSTER` (`09_clustering.sql`):

```
SELECT correlation FROM pg_stats WHERE tablename = 'orders' AND attname = 'customer_id';
EXPLAIN (ANALYZE, BUFFERS, COSTS OFF)
SELECT sum(amount) FROM orders WHERE customer_id BETWEEN 1000 AND 1999;
CLUSTER orders USING orders_customer_idx;  ANALYZE orders;
-- ... same two queries again
```

```
correlation = 0.0034                                        -- before: random order (sampled; varies per run)
Aggregate (actual time=5.814..5.815 rows=1 loops=1)
  Buffers: shared hit=8454 read=14
  ->  Bitmap Heap Scan on orders (... rows=10000 loops=1)
        Heap Blocks: exact=8454                             -- 10,000 rows on 8,454 pages!
...
correlation = 1                                             -- after CLUSTER
Aggregate (actual time=0.895..0.895 rows=1 loops=1)
  Buffers: shared hit=2 read=100
  ->  Index Scan using orders_customer_idx on orders (... rows=10000 loops=1)
Execution Time: 0.901 ms
```

Same rows, same index: **8,468 → 102 pages**. The statistic **correlation** (−1 … 1) tells the planner how well a column's order matches physical order. **Fillfactor** sets how full pages are packed on insert, leaving room so updated row versions stay on the same page (PostgreSQL's heap-only-tuple — HOT — updates, which skip index maintenance, chapter 13): the same 100,000 rows take 443 pages at the default `fillfactor = 100`, 633 at `fillfactor = 70`.

Trade-offs: only **one** physical order per table — choose the key of your most important ranges (often time or tenant id, the customer organisation a row belongs to in a multi-tenant app). In InnoDB and SQL Server the clustered key is copied into every secondary index, so a wide or random key (a random UUID) bloats every index and causes page splits; prefer a short, increasing key.

12.4 What an index is

What it is

An **index** is a separate, smaller structure mapping **search-key values to row addresses**, kept sorted (or hashed) so the database finds matching rows without reading the whole table — like a textbook's index: look up "B+ tree", jump to page 412, never read pages 1–411.

- **Search key**: the indexed column(s) (not necessarily a primary or unique key).
- **Index entry**: (key, pointer) — a `ctid` in PostgreSQL, the primary-key value in InnoDB secondary indexes, a row id in Oracle.

Without an index, `WHERE customer_id = 4242` checks every row; with one, a handful of index pages:

```
-- no index
 Seq Scan on orders (actual time=2.944..30.041 rows=10 loops=1)
   Filter: (customer_id = 4242)
   Rows Removed by Filter: 999990
   Buffers: shared hit=8662
 Execution Time: 30.068 ms
-- CREATE INDEX orders_customer_idx ON orders (customer_id);
 Bitmap Heap Scan on orders (actual time=0.015..0.023 rows=10 loops=1)
   Buffers: shared hit=10 read=3
   ->  Bitmap Index Scan on orders_customer_idx (... rows=10 loops=1)
 Execution Time: 0.036 ms
```

8,662 pages → 13 pages; ~30 ms → ~0.04 ms. That is the whole reason indexes exist.

Kinds of index (classic terminology)

Term	Meaning
Primary index	On the ordering key of a *sorted* file, usually the PK; one entry per data block suffices (sparse).
Clustering index	On a *non-unique* sorting field (file sorted by department); one entry per distinct value, pointing to its first block.
Secondary index	On any other column; the file is not sorted by it, so one entry per record (dense). A table can have many.
Dense index	An entry for **every** search-key value (every record).
Sparse index	Entries for only **some** values (typically each block's first key); works only if the file is sorted on that key.
Clustered vs non-clustered (vendor usage)	*Clustered*: rows stored in index order (at most one per table — SQL Server, InnoDB PK). *Non-clustered*: separate structure pointing to rows. All PostgreSQL indexes are non-clustered.
Unique index	Rejects duplicate keys; how `PRIMARY KEY` and `UNIQUE` are enforced.

```
CREATE INDEX orders_customer_idx ON orders (customer_id);             -- B-tree by default
CREATE UNIQUE INDEX customers_email_uq ON customers (email);          -- enforces uniqueness
CREATE INDEX CONCURRENTLY orders_note_idx ON orders (note);           -- PG: build without blocking writes
DROP INDEX orders_customer_idx;                          -- MySQL / SQL Server: DROP INDEX name ON table
ALTER TABLE t ADD CONSTRAINT t_pk PRIMARY KEY (id);      -- creates a unique index implicitly
```

SQL Server adds `CREATE [CLUSTERED|NONCLUSTERED] INDEX`. `CONCURRENTLY` (PostgreSQL), `ONLINE = ON` (SQL Server Enterprise) and `ALGORITHM=INPLACE, LOCK=NONE` (MySQL) avoid long write locks on big tables.

Trade-offs, when not to index, mistakes

Benefit	Cost
Fast lookups, ranges, joins, `ORDER BY`, `MIN/MAX`	Every `INSERT`, `DELETE` and many `UPDATE`s must change every index
Enforces uniqueness	Disk and buffer-pool space (our 1M-row PK index is 21 MB)
Enables index-only scans	Too many indexes slow writes and complicate planning

- **Don't index:** small tables (a seq scan is cheap); low-selectivity columns (`status = 'shipped'` matches 95 % — 12.10); write-heavy tables rarely searched; columns only used inside functions (index the expression — 12.12).
- **Common mistakes:** "index every column" — unused indexes are pure overhead (find them with `pg_stat_user_indexes.idx_scan = 0`); forgetting that **foreign-key columns are not indexed automatically** in PostgreSQL (InnoDB does create one) — joins on the FK and `ON DELETE` checks then scan the child table.

In the interview

"An index is a sorted (or hashed) side structure mapping key values to row locations, so a lookup reads a few pages instead of the whole table; it speeds up filters, joins and sorts at the cost of slower writes and extra space. I index selective predicates (`WHERE` conditions that match few rows) and join keys and confirm with EXPLAIN."

- *Why not index everything?* → Write amplification (one logical change causing many physical writes), space, cache pressure; low-selectivity indexes are ignored anyway.
- *Clustered vs non-clustered?* → Clustered stores the rows themselves in key order (one per table); non-clustered is separate, with pointers.

12.5 Single-level indexing, with numbers

An index file is much smaller than the data file (entries hold only key + pointer); binary search over it costs about $\log_2$(index blocks) reads, plus one data block. The arithmetic (from *Database System Concepts* and *Fundamentals of Database Systems*):

- **Blocking factor** `bfr = ⌊block size / record size⌋` — records per block (`⌊x⌋` rounds down, `⌈x⌉` up).
- **Blocks** `b = ⌈records / bfr⌉`; **binary search** on `b` sorted blocks: `⌈log₂ b⌉` reads.

Worked example: r = **1,000,000** records of **200 bytes**, block **4,096 bytes**, index entries 8-byte key + 8-byte pointer = **16 bytes**.

Access path	Calculation	Block reads
Data file	bfr = ⌊4096/200⌋ = 20; b = 1,000,000/20 = **50,000 blocks**	—
Linear scan, unordered file	average b/2, worst b	25,000 avg / 50,000 worst
Binary search, file sorted on the key	⌈$\log_2$ 50,000⌉ = 16	**16**
Primary index (sparse): one entry per data block	50,000 entries; index bfr = ⌊4096/16⌋ = 256; 50,000/256 → **196 blocks**; ⌈$\log_2$ 196⌉ = 8, +1 data block	**9**
Secondary index (dense) on an unsorted column	1,000,000/256 → **3,907 blocks**; ⌈$\log_2$ 3907⌉ = 12, +1	**13**
Multilevel (index on the index, fan-out — entries per index block — 256)	3,907 blocks → ⌈3907/256⌉ = 16 → 1 block; 3 levels + 1 data block	**4**

A sparse index is much smaller than a dense one but requires a sorted file; indexing **the index** — each level 256× smaller — turns a $\log_2$ search into $\log_{256}$. The top level stays in RAM, so only lower levels and the data block cost disk reads. Made dynamic, that idea is the B+ tree. (Courses call primary, clustering and secondary indexes *single-level* and B-/B+ trees *multi-level*.)

Trade-offs: sparse — small, but only on the sort key, and inserts must keep the file sorted; dense — any column, but large; both **static** (inserting shifts entries) — multilevel trees with free space per node fix that.

Self-check

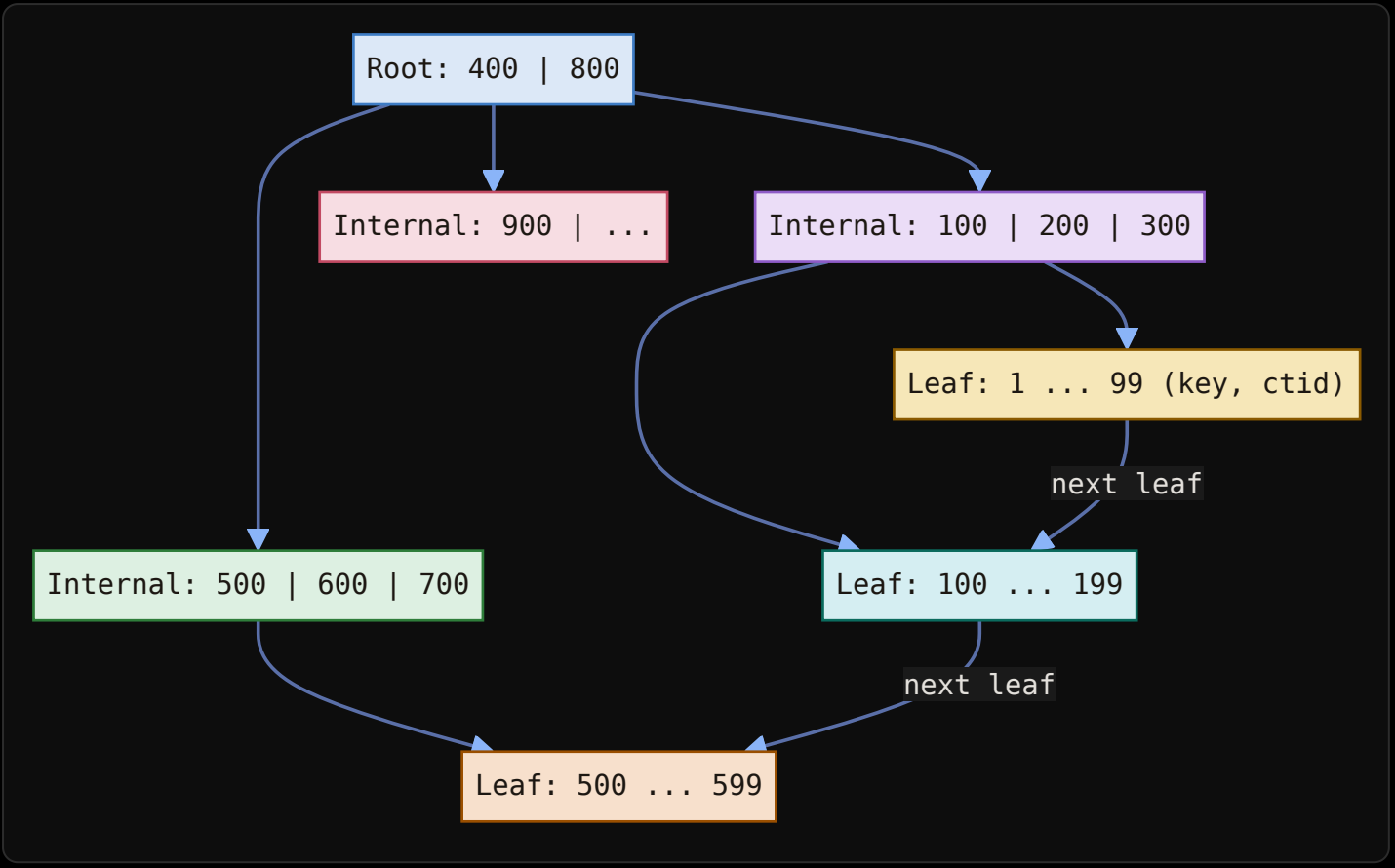
1. Why can a secondary index not be sparse? → The file isn't sorted on that column, so a block's first key says nothing about the others.
2. Fan-out 256, 16 million entries — how many levels? → $256^3 \approx 16.7$ million, so 3.

12.6 Tree-based indexes: B-trees and B+ trees

What it is

A **B-tree** (Bayer & McCreight, 1972) is a balanced search tree whose nodes are pages holding hundreds of sorted keys; every leaf is at the same depth, so every lookup costs the same number of page reads. It keeps keys sorted *and* absorbs inserts locally: O(log n) search, insert and delete, where a sorted array needs O(n) shifting per insert and an unsorted one O(n) per search. The **B+ tree** — what nearly every database uses (PostgreSQL "btree", InnoDB, SQL Server, SQLite, Oracle) — adds two rules:

1. **All data pointers live in the leaves**; internal nodes hold only separator keys and child pointers, so more fit per page (higher **fan-out**).
2. **Leaves are linked** left-to-right (PostgreSQL: both ways), so a range scan finds the first key and walks sideways.



How it works

- **Search**: from the root, pick the child whose range contains the key, repeat to a leaf, then follow the pointer to the heap (the table's unordered data file). Cost = height + 1. Finding 650: root (400 < 650 < 800 → middle) → internal (600 < 650 < 700) → leaf 600–699.
- **Range** (`BETWEEN 1000 AND 1999`): search for 1000, then read leaves rightwards until a key > 1999.
- **Insert**: into the leaf in sorted position; if full, **split** into two half-full leaves and push the middle key into the parent (which may split too). Only a root split makes the tree taller — why it stays balanced.
- **Delete**: remove; if a node becomes too empty, borrow from a sibling or merge (PostgreSQL doesn't merge eagerly; `VACUUM` reclaims empty pages).
- **Order** `n`: each node holds up to `n − 1` keys and `n` children; every non-root node stays at least half full.

```
SELECT level AS root_level FROM bt_metap('orders_pkey');
SELECT type, live_items, avg_item_size, free_size FROM bt_page_stats('orders_pkey', 1);
```

```
root_level = 2                  -- levels 2 (root), 1 (internal), 0 (leaves): height 3
type | live_items | avg_item_size | free_size
-----+------------+---------------+-----------
l    |        367 |            16 |       808
```

A leaf holds 367 16-byte entries and keeps ~10 % free (B-tree default fillfactor 90). One million keys need ~2,700 leaves and one internal level — why the PK lookup in 12.10 touched only **7 buffers** (three levels, the heap page, some index metadata) vs 8,662 for a full scan. A billion keys would need one more level.

Structure	Data pointers in	Range scans	Used by
B-tree (classic)	Every node	In-order tree walk	Rare today; some file systems
B+ tree	Leaves only; leaves linked	Walk the leaf chain	PostgreSQL, MySQL, SQL Server, Oracle, SQLite
LSM tree	Sorted immutable files merged in the background	Merge several files	RocksDB, Cassandra, ScyllaDB, LevelDB (ch. 16)
R-tree	Bounding boxes	Spatial "overlaps/near"	PostGIS (PostgreSQL's geometry extension, via GiST), SQLite R*Tree

Trade-offs: random-key inserts (random UUIDs) hit random leaves — more splits and cache misses than an increasing key. PostgreSQL 13+ **deduplicates** repeated keys in leaves, so a low-uniqueness B-tree is much smaller (our `customer_id` index: 9 MB vs 21 MB for unique `created_at`).

In the interview

> **30-second answer:** "Databases use B+ trees: balanced trees whose nodes are pages with hundreds of keys, so even a billion rows need only 3–4 levels. Values sit in linked leaves, so range scans walk the leaves. Inserts split full nodes and push keys up, keeping every leaf at the same depth."

- *B-tree vs LSM?* → B+ trees update pages in place (read-optimized); LSM trees append and merge (write-optimized, need Bloom filters for reads).
- *Why no row pointers in internal nodes?* → More separator keys per page: higher fan-out, lower height.
- *When does a B+ tree get taller?* → Only when the root splits.

12.7 Hash-based indexing

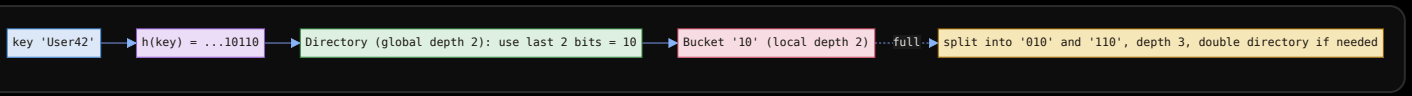
What it is

A **hash index** computes `hash(key)` to pick a **bucket** (a page or chain of pages) holding that key's entries; a lookup reads one bucket — typically one or two pages — however large the table.

How it works

1. **Hash function**: maps keys evenly across buckets (`bucket = h(key) mod N`), scattering similar keys (`User1`, `User2`) far apart.
2. **Collisions**: different keys in one bucket share its page; when full, an **overflow page** is chained on. (In-memory hash tables also use *open addressing*: probe the next slots.)
3. **Static hashing** fixes `N`. With `h(k) = k mod 7`, keys 15, 23, 40 go to buckets 1, 2, 5; inserting 29 (29 mod 7 = 1) collides with 15 and is **chained** in bucket 1, so a search for 29 walks that short chain. Growth makes chains long; shrinking wastes space; rehashing everything is expensive.
4. **Dynamic hashing** grows gradually:

- **Extendible hashing**: a **directory** of `2^d` pointers indexed by the last `d` hash bits (**global depth**); each bucket records the bits it uses (**local depth**). An overflowing bucket splits using one more bit; only if its local depth equals the global depth does the directory double (pointers copied, no data moved).
- **Linear hashing**: no directory; buckets split one at a time in round-robin order (a "next to split" pointer), using `h mod 2N` for already-split buckets — smooth growth, temporary overflow chains. PostgreSQL's hash index (based on Seltzer and Yigit's 1991 package) likewise has no directory: it splits one bucket at a time and allocates bucket pages in power-of-2 groups located by computation from the bucket number.




```
CREATE INDEX o_cust_hash ON orders USING hash (customer_id);
EXPLAIN (COSTS OFF) SELECT * FROM orders WHERE customer_id = 42;   -- Bitmap Index Scan on o_cust_hash
EXPLAIN (COSTS OFF) SELECT * FROM orders WHERE customer_id < 42;   -- Seq Scan: hash can't do ranges
```

```
   index_name    | type  |  size
-----------------+-------+---------
 o_cust_hash     | hash  | 32 MB
 o_created_btree | btree | 21 MB
 o_cust_btree    | btree | 9256 kB
 o_created_brin  | brin  | 24 kB
```

Trade-offs

	Hash index	B-tree
`=` lookups	O(1) buckets	O(log n) pages (3–4 in practice)
Ranges, `ORDER BY`, prefix `LIKE`	No (hashing destroys order: neighbouring keys land in unrelated buckets)	Yes
Unique constraints	No (PostgreSQL)	Yes
Size	Can be larger (no deduplication — 32 MB vs 9 MB above)	Compact with dedup

The B-tree's extra 2–3 page reads are usually cached, so hash indexes win only for very large, equality-only keys (long strings, URLs). PostgreSQL hash indexes are crash-safe only since version 10 — old "avoid them" advice predates that. InnoDB has no user-created hash indexes (its *adaptive hash index* is built automatically in memory); the MEMORY engine does; SQL Server has them on memory-optimized tables; SQLite has none. Hashing also drives **hash joins** (12.15) and **hash partitioning** (ch. 15).

12.8 Bloom filters

What it is

A **Bloom filter** (Burton Bloom, 1970) is a compact, probabilistic set answering "is x in the set?" with **"definitely not"** or **"probably yes"**: never a **false negative** ("absent" for a present key), but **false positives** ("maybe" for an absent key) at a rate you choose by spending bits. Checking it in RAM costs nanoseconds vs micro- to milliseconds for disk, so when most lookups are for absent keys it skips almost all disk work.

How it works

1. An array of **m bits**, all 0, and **k** independent hash functions.
2. **Add(x)**: set the k hashed bits to 1.

3. **Query(x)**: if **any** of the k bits is 0, x was never added; if all are 1, x is *probably* present (other keys may have set them). Work per operation depends on k, not m.
4. With n keys, false-positive probability **p ≈ (1 − e^(−kn/m))^k**, minimized at **k = (m/n)·ln 2**, needing about **1.44·log₂(1/p) bits per key** (≈ 9.6 bits for 1 %).
5. No deletes (clearing a bit could erase other keys). **Counting Bloom filters** keep a small counter per position to allow deletes; **scalable Bloom filters** add larger filters as n grows.

Tiny example (10 bits, k = 3; made-up hash outputs): `"sql"` → bits 2, 5, 8; `"java"` → 1, 5, 9. Bits: `0110010011`. Query `"css"` → 2, 6, 9: bit 6 is 0 → **definitely absent**. Query `"go"` → 1, 2, 8: all 1, never added → **false positive**.

Simulation vs theory

`bloom_filter_sim.py` builds real filters (SHA-256 with double hashing, `h_i = h1 + i*h2 mod m`), inserts 10,000 keys and probes 50,000 never-added keys:

```
bits/key   m bits   k  measured FP  theory FP    memory
       4    40000   3     14.6340%   14.6892%     4.9KB
       8    80000   6      2.1700%    2.1577%     9.8KB
      10   100000   7      0.8720%    0.8194%    12.2KB
      16   160000  11      0.0520%    0.0459%    19.5KB

Effect of k at a fixed 8 bits/key:
  k= 1  measured 11.604%   theory 11.750%
  k= 2  measured  4.770%   theory  4.893%
  k= 4  measured  2.324%   theory  2.397%
  k= 6  measured  2.170%   theory  2.158%
  k= 8  measured  2.654%   theory  2.549%
  k=12  measured  4.780%   theory  4.833%
```

Measured rates track the formula; ~10 bits per key buys < 1 % false positives regardless of key length; doubling m (8 → 16 bits/key) drops p from ~2 % to ~0.05 %; too many hashes hurts (k = 12 fills the array with ones).

Real-world examples

- **LSM-tree databases** (RocksDB, Cassandra, HBase, LevelDB): one filter per sorted file lets a read skip files that cannot hold the key.
- **"Username taken?"** and duplicate-submission checks: an in-memory filter answers most "no" cases without the database.
- **Web caches and CDNs** (content delivery networks — servers worldwide caching content near users, System Design Fundamentals) avoid caching one-hit objects; routers and distributed caches summarize which keys a peer holds.
- **Join filters**: engines build a filter from the smaller join side and push it to the larger side's scan (Spark, SQL Server bitmap filters, Oracle).
- **PostgreSQL `bloom` extension**: a per-row signature over many columns, so one index answers equality on *any* subset:

```
CREATE INDEX wide_bloom ON wide USING bloom (a, b, c, d, e) WITH (length = 80);
EXPLAIN (ANALYZE, COSTS OFF, TIMING OFF, SUMMARY OFF) SELECT * FROM wide WHERE c = 5 AND e = 9;
```

```
 bloom_index_size | table_size
------------------+------------
 4720 kB          | 15 MB
 Bitmap Heap Scan on wide (actual rows=50 loops=1)
   Recheck Cond: ((c = 5) AND (e = 9))
   Rows Removed by Index Recheck: 97          <- the false positives, removed by rechecking
   ->  Bitmap Index Scan on wide_bloom (actual rows=147 loops=1)
```

TL;DR

Bit array + k hashes; "no" is certain, "yes" is probable; ~10 bits/key ≈ 1 % false positives; no deletes (use counting filters); used to skip disk reads in LSM stores and joins.

12.9 PostgreSQL's other index types: GIN, GiST, BRIN

B-trees answer "equal / less / greater". *Does this array contain X, do these ranges overlap, which rows fall in this month of a huge log* need other structures (`10_index_types.sql`):

Type	Structure in one line	Answers	Typical use
B-tree (default)	Balanced sorted tree (12.6)	`= < > BETWEEN`, prefix `LIKE`, `ORDER BY`	Almost everything
Hash	Buckets by hash (12.7)	`=` only	Long keys, equality only
GIN (Generalized Inverted Index)	For each *element* (array item, JSON key/value, word, trigram — a 3-character chunk of text) a list of rows containing it	`@>` contains, `?` key exists, full-text `@@`, `LIKE '%x%'` with `pg_trgm`	Tags, JSONB documents, search
GiST (Generalized Search Tree)	Balanced tree of possibly overlapping "bounding" predicates (boxes, ranges)	Overlap `&&`, containment, nearest-neighbour `<->`	Geometry (PostGIS), ranges, exclusion constraints
SP-GiST	Space-partitioned trees (quadtrees, tries)	Points, IP ranges, text prefixes	Unevenly distributed data
BRIN (Block Range INdex)	min/max summary per 128-page range	Ranges on columns following physical order	Append-only logs, time-series
bloom (extension)	Bloom signature per row (12.8)	`=` on any subset of many columns	Wide tables with ad-hoc filters

BRIN — `created_at` rises with insertion order (correlation 1), so a 24 kB BRIN replaces a 21 MB B-tree:

```
Aggregate (actual time=0.716..0.717 rows=1 loops=1)
  Buffers: shared hit=130 read=3
  ->  Bitmap Heap Scan on orders (actual time=0.222..0.656 rows=2787 loops=1)
        Rows Removed by Index Recheck: 11992
        Heap Blocks: lossy=128                <- one 128-page block range read
        ->  Bitmap Index Scan on o_created_brin (... rows=1280 loops=1)
```

It is *lossy*: it knows only which ranges *might* match, reads the whole range and rechecks rows. On a randomly ordered column every range spans everything and BRIN is useless.

GIN on arrays and JSONB:

```
CREATE INDEX posts_tags_gin ON posts USING gin (tags);
CREATE INDEX posts_meta_gin ON posts USING gin (meta jsonb_path_ops);
SELECT count(*) FROM posts WHERE tags @> ARRAY['go', 'advanced'];     -- 20000, via Bitmap Index Scan on posts_tags_gin
EXPLAIN (COSTS OFF) SELECT count(*) FROM posts WHERE meta @> '{"lang": "hi", "likes": 7}';
--   ->  Bitmap Index Scan on posts_meta_gin
```

GIN is fast to read but slower to update (one row touches many posting lists — the per-element row lists; PostgreSQL buffers new entries in a "pending list").

GiST enforcing "no double booking" with an **exclusion constraint** (no two rows may conflict under the given operators; `tsrange` is a timestamp range, `&&` "overlaps"; `btree_gist` is needed for `room WITH =`):

```
CREATE TABLE bookings (room int, during tsrange, EXCLUDE USING gist (room WITH =, during WITH &&));
INSERT INTO bookings VALUES (101, '[2024-05-01 14:00, 2024-05-03 11:00)');
INSERT INTO bookings VALUES (102, '[2024-05-01 14:00, 2024-05-03 11:00)');  -- other room: fine
INSERT INTO bookings VALUES (101, '[2024-05-02 14:00, 2024-05-04 11:00)');  -- overlaps!
```

```
ERROR:  conflicting key value violates exclusion constraint "bookings_room_during_excl"
DETAIL:  Key (room, during)=(101, ["2024-05-02 14:00:00","2024-05-04 11:00:00")) conflicts with existing key
(room, during)=(101, ["2024-05-01 14:00:00","2024-05-03 11:00:00")).
```

Elsewhere: MySQL has B-tree, full-text, spatial and multi-valued JSON indexes (8.0.17+); SQL Server rowstore, columnstore, full-text and spatial; SQLite B-tree plus FTS5 and R*Tree.

12.10 Reading EXPLAIN: the four ways to read a table

What it is

`EXPLAIN` shows the **plan** the optimizer (planner) chose: a tree of operators, read from the innermost (most indented) node outwards. `EXPLAIN ANALYZE` also **runs** the query and adds real rows and times; `BUFFERS` adds pages hit in cache / read from the OS.

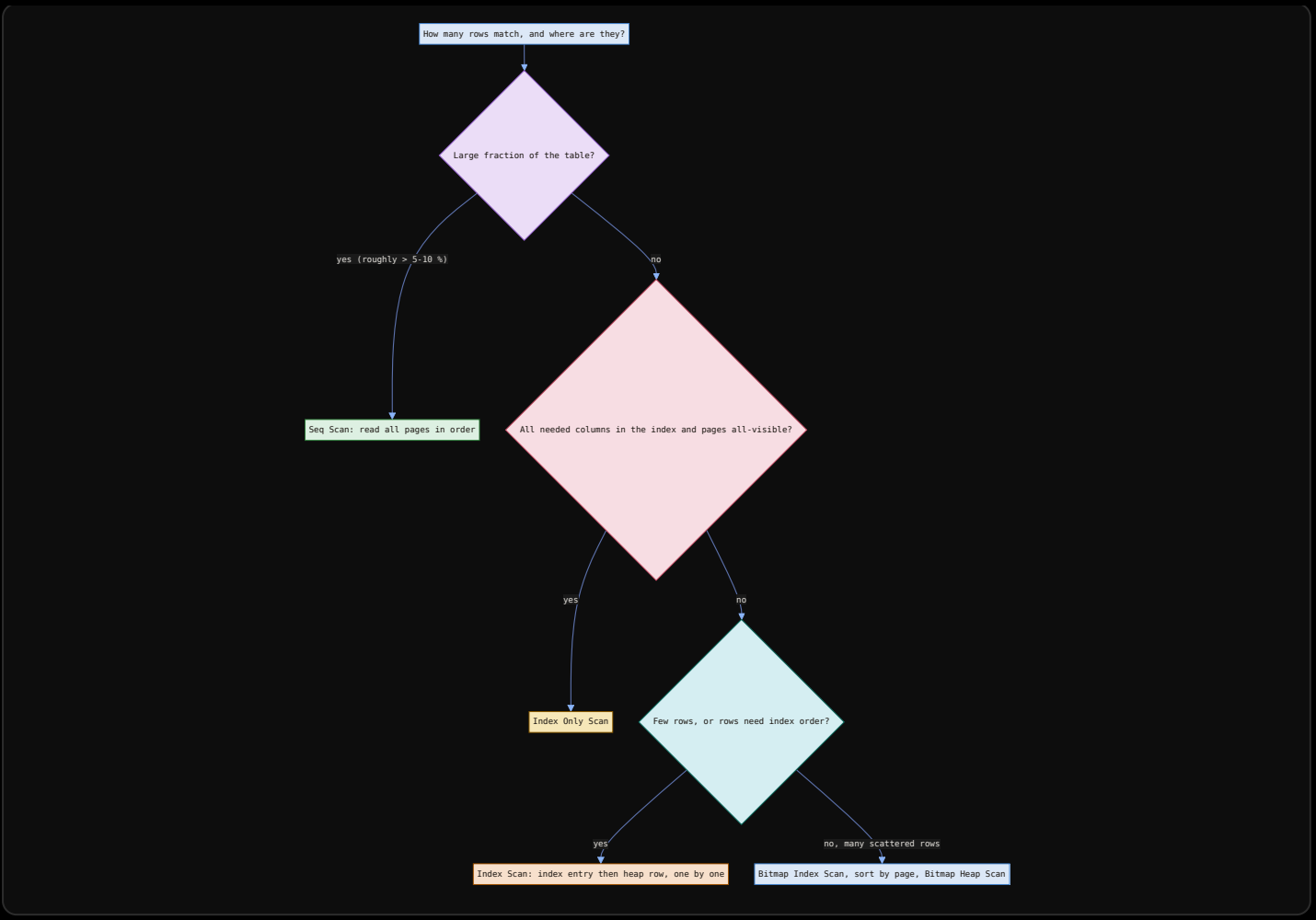
Field	Meaning
`cost=0.42..8.44`	Estimate in abstract units: startup .. total (sequential page read = 1.0, random = `random_page_cost` = 4.0)
`rows=` (estimate) vs `actual ... rows=`	A big gap means bad statistics (12.16)
`loops=`	Times the node ran (inner side of a nested loop); times and rows are per loop
`Buffers: shared hit=/read=`	Pages found in the buffer pool / fetched from the OS
`Filter` + `Rows Removed by Filter`	Rows read then thrown away — wasted work
`Index Cond`	Condition applied *inside* the index — the useful work

`EXPLAIN ANALYZE` executes the statement — wrap `INSERT/UPDATE/DELETE` in `BEGIN … ROLLBACK`. MySQL: `EXPLAIN ANALYZE` (8.0.18+); SQL Server: `SET SHOWPLAN_XML ON` / "actual execution plan"; SQLite: `EXPLAIN QUERY PLAN`.

The four access methods, measured

From `02_scans.sql` (parallel query — one query split across worker processes — off to keep plans small):

Query	Plan	Pages touched	Time
`customer_id = 4242`, no index	**Seq Scan** — read every page, filter	8,662	~30 ms
same, with `orders_customer_idx`	**Bitmap Index + Bitmap Heap Scan**	13	~0.04 ms
`id = 500000`	**Index Scan** on `orders_pkey`	7	~0.01 ms
`count(*) … customer_id BETWEEN 1000 AND 1999`	**Index Only Scan**, `Heap Fetches: 0`	15	~0.5 ms
`sum(amount) … status = 'shipped'` (95 %)	**Seq Scan** despite the index	8,662	~76 ms
`sum(amount) … customer_id BETWEEN 1000 AND 1999`	**Bitmap Heap Scan**, `exact=8454`	8,468	~5 ms
`status = 'refunded' AND customer_id < 2000`	**BitmapAnd** of two indexes	—	—



1. **Seq Scan** reads pages in physical order (storage read-ahead makes this fast) — right for a predicate keeping 95 % of rows.
2. **Index Scan** walks the B-tree, then fetches each heap row; best for few rows; rows come out **in index order**, so `ORDER BY … LIMIT` can stop early.
3. **Index Only Scan** answers from the index alone, visiting the heap only for pages not marked all-visible in the **visibility map** (maintained by `VACUUM`; shown as `Heap Fetches`, ch. 13).
4. **Bitmap scan** collects matching `ctid`s into a bitmap (one bit per row or page), sorts by page and reads each needed page once; can combine indexes (`BitmapAnd/Or`). Short of memory, the bitmap turns *lossy* (whole pages) and rows are rechecked.

The 1 % case: 10,000 rows lived on **8,454 of 8,662 pages** — scattered rows make an index save almost nothing, which is why the planner cares about correlation (12.3) and why "selective" means *few pages*, not just few rows.

Self-check

1. Why did `status = 'shipped'` ignore its index? → 95 % match; index + nearly every heap page randomly costs more than one sequential pass.
2. `Heap Fetches: 0`? → Every page was all-visible; the table was never touched.
3. `Filter` vs `Index Cond`? → `Index Cond` narrows the search inside the index; `Filter` discards rows after reading.

12.11 Composite indexes and column order

A **composite** (multi-column) index sorts by the first column, then by the second within equal first values — like a phone book (last name, then first name). `03_composite_indexes.sql`, `CREATE INDEX orders_cust_created_idx ON orders (customer_id, created_at)`:

Query	Uses the index?	Why
`WHERE customer_id = 777`	Yes (3 index pages)	Leading column
`WHERE customer_id = 777 ORDER BY created_at DESC LIMIT 3`	Yes, **Index Scan Backward**, no Sort node, 6 pages	Within one customer, entries are already in time order
`WHERE created_at >= '2024-06-01' AND created_at < '2024-06-01 00:10'`	**No** → Seq Scan, 8,662 pages, ~30 ms	Second column alone: the range is spread across every customer's section
same, after `CREATE INDEX … (created_at)`	Index Only Scan, 4 pages, ~0.04 ms	Now the range is contiguous

```
Limit (actual time=0.002..0.003 rows=3 loops=1)
  Buffers: shared hit=6
  ->  Index Scan Backward using orders_cust_created_idx on orders (... rows=3 loops=1)
        Index Cond: (customer_id = 777)
```

Leftmost-prefix rule: an index on `(a, b, c)` serves filters on `a`, `a,b` and `a,b,c`, not `b` or `c` alone. (Oracle, MySQL 8.0.13+ and PostgreSQL 18+ can sometimes "skip scan" when the leading column has few distinct values; don't design around it.)

Equality first, range last

`customer_id = 777 AND created_at >= '2024-06-01'`, two column orders:

```
-- (customer_id, created_at)
   ->  Bitmap Index Scan on orders_cust_created_idx (... rows=5 loops=1)
         Buffers: shared hit=3
-- (created_at, customer_id)
 Index Scan using orders_created_cust_idx on orders (actual time=1.541..8.275 rows=5 loops=1)
   Index Cond: ((created_at >= '2024-06-01 ...') AND (customer_id = 777))
   Buffers: shared hit=5 read=2211
```

8 pages (3 index) vs 2,216. Equality first puts all matches in one narrow slice; range first walks every entry after June 1 (≈ 55 % of the index) testing `customer_id` on each. Rule: **equality columns first, then the range or sort column; among equality columns, the most-used-alone first.**

> **Common mistakes:** separate `(a)`, `(a, b)`, `(a, b, c)` indexes (the last covers the others); a low-selectivity column first "because it's in every query" when queries also filter a range; assuming `(a, b)` helps `ORDER BY b` (only when `a` is fixed by equality).

12.12 Covering, partial and expression indexes

From `04_covering_partial_expression.sql`.

Covering index (`INCLUDE`, PostgreSQL 11+, SQL Server). Extra *payload* columns in the leaf entries (not part of the key: not sorted, not unique-checked) so the query never visits the table:

```
CREATE INDEX orders_cust_incl_idx ON orders (customer_id) INCLUDE (amount);
SELECT sum(amount) FROM orders WHERE customer_id BETWEEN 5000 AND 5999;
```

Index	Plan	Pages	Time
`(customer_id)`	Bitmap Heap Scan, 8,450 heap pages	8,464	~5.3 ms
`(customer_id) INCLUDE (amount)`	**Index Only Scan**, Heap Fetches: 0	42	~0.75 ms

MySQL: add the column to the key (`(customer_id, amount)`); InnoDB secondary indexes always contain the PK.

Partial index (PostgreSQL, SQLite, SQL Server "filtered index"; not MySQL). Index only the rows a query cares about:

```
CREATE INDEX orders_pending_idx ON orders (created_at) WHERE status = 'pending';
SELECT id, created_at FROM orders WHERE status = 'pending' ORDER BY created_at LIMIT 5;
```

```
       index_name         |  size
--------------------------+--------
 orders_pending_idx        | 456 kB       -- 2 % of rows
 orders_status_created_idx | 30 MB        -- full (status, created_at) rival
 Limit (actual time=0.009..0.011 rows=5 loops=1)
   ->  Index Scan using orders_pending_idx on orders (... rows=5 loops=1)
         Buffers: shared hit=2 read=2
```

No `Index Cond` at all: every entry is already pending. Ideal for work queues ("oldest unprocessed job"), soft deletes (`WHERE deleted_at IS NULL`) and **conditional uniqueness** — `CREATE UNIQUE INDEX ON orders (customer_id) WHERE status = 'pending'` allows one pending order per customer. Used only when the query's `WHERE` provably implies the index predicate (`status = 'refunded'` gets Seq Scan + Sort).

Expression (functional) index. Index an expression's result; queries must use the *same* expression:

```
CREATE INDEX customers_lower_email_idx ON customers (lower(email));
SELECT id FROM customers WHERE lower(email) = 'user4242@mail.com';   -- Seq Scan 918 pages -> Index Scan 4 pages
SELECT id FROM customers WHERE upper(email) = 'USER4242@MAIL.COM';   -- Seq Scan: different expression
```

Only `IMMUTABLE` functions (same input, same output forever). MySQL 8.0.13+ has functional key parts `((lower(email)))`; SQL Server indexes a computed column; alternatives are `citext` (case-insensitive text) or a case-insensitive collation.

12.13 Anti-patterns that disable indexes

From `05_antipatterns.sql`. An index helps only when the **bare column** is compared with something computable up front — **sargable** (Search ARGument ABLE).

Anti-pattern	Plan	Fix	Plan after fix
`WHERE created_at::date = DATE '2024-06-01'`	Seq Scan, 8,662 pages, ~39 ms	`created_at >= '2024-06-01' AND created_at < '2024-06-02'`	Index Only Scan, 15 pages, ~0.2 ms
`WHERE id + 1 = 500001`	Seq Scan	`WHERE id = 500001 - 1`	Index Scan on `orders_pkey`
`WHERE email LIKE '%4242@corp%'` (leading wildcard)	Seq Scan, 918 pages	trigram GIN index	Bitmap scan, 74 pages
`WHERE email = … OR name = …` (name unindexed)	Seq Scan	index both sides (`BitmapOr`) or rewrite as `UNION`	BitmapOr
`WHERE lower(email) = …` without an expression index	Seq Scan	expression index (12.12)	Index Scan

Also: comparing to a different type (MySQL `WHERE varchar_col = 123` casts the column and skips the index), `NOT IN` / `<>` (rarely selective), `SELECT *` (rules out index-only scans).

Prefix vs leading wildcard. A B-tree is sorted by the string's *start*, so `LIKE 'User4242%'` becomes a range (`email >= 'User4242' AND email < 'User4243'` in the Index Cond) — under the C collation used here; otherwise build the index with `text_pattern_ops`. For `'%…%'`, **`pg_trgm`** splits text into 3-character pieces and a GIN index finds rows sharing the pattern's trigrams:

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;
SELECT show_trgm('User42');     -- {"  u","  us","42 ",er4,r42,ser,use}
CREATE INDEX customers_email_trgm_idx ON customers USING gin (email gin_trgm_ops);
```

```
Bitmap Heap Scan on customers (actual time=0.563..0.565 rows=3 loops=1)
  Recheck Cond: (email ~~ '%4242@corp%'::text)
  ->  Bitmap Index Scan on customers_email_trgm_idx (... rows=3 loops=1)
        Buffers: shared hit=71
```

For words in prose use full-text search (understands word forms, ranks matches: `tsvector` + GIN, ch. 14).

The N+1 query problem

The most common performance bug in application code: load a list (1 query), then one more query **per item** (N). An **ORM** (*object-relational mapper*: a library that exposes rows as objects, `customer.orders`, and writes the SQL) does this silently when a **lazy** relationship (loaded on first touch) is touched inside a loop; **eager loading** fetches related rows up front. `n_plus_one.py` totals 500 customers' orders three ways:

```
for cid, _name in customers:                                    # N+1
    conn.execute("SELECT coalesce(sum(amount), 0) FROM orders WHERE customer_id = %s", (cid,))
conn.execute("SELECT customer_id, sum(amount) FROM orders WHERE customer_id = ANY(%s) "
             "GROUP BY customer_id", (ids,))                     # batched: 2 queries
conn.execute("SELECT c.id, coalesce(sum(o.amount), 0) FROM (...) c "
             "LEFT JOIN orders o ON o.customer_id = c.id GROUP BY c.id")   # 1 query
```

```
n_plus_one       queries= 501  local time=   14.5 ms  + network @1ms RTT ~    501 ms
batched_in_list  queries=   2  local time=    1.4 ms  + network @1ms RTT ~      2 ms
single_join      queries=   1  local time=    3.1 ms  + network @1ms RTT ~      1 ms
all three return the same totals: True
```

Over a local Unix socket the loop costs 10×; over a real network each round trip (RTT, round-trip time) adds latency, so 501 queries at 1 ms is half a second. Fixes: a join, an `IN` / `= ANY` batch, or eager loading (`select_related` / `prefetch_related` in Django, `JOIN FETCH` / `@EntityGraph` in JPA, `includes` in Rails).

12.14 OFFSET vs keyset pagination

`LIMIT 20 OFFSET 999960` must produce and **discard** all earlier rows. **Keyset** (seek) pagination remembers the last row shown and asks for rows *after* it, which an index answers directly. `06_pagination.sql`, index `(created_at, id)`, newest first, 20 per page:

```
-- OFFSET: page N
SELECT id, created_at, amount FROM orders
ORDER BY created_at DESC, id DESC LIMIT 20 OFFSET (N - 1) * 20;
-- Keyset: pass the last (created_at, id) of the previous page
SELECT id, created_at, amount FROM orders
WHERE (created_at, id) < (:'last_ts', :last_id)
ORDER BY created_at DESC, id DESC LIMIT 20;
```

```
      method       | exec_ms | buffers
-------------------+---------+---------
 OFFSET, page 1    |    0.02 |       4
 OFFSET, page 100  |    0.15 |      39
 OFFSET, page 10000 |   15.47 |    3425
 OFFSET, page 49999 |   79.05 |   17114
 keyset, page 49999 |    0.02 |       4
 rows_that_differ = 0
 Limit (actual time=0.004..0.007 rows=20 loops=1)
   ->  Index Scan Backward using orders_created_id_idx on orders (... rows=20 loops=1)
         Index Cond: (ROW(created_at, id) < ROW('2024-01-01 00:21:11'::timestamp ..., 41))
```

OFFSET cost grows linearly with the page; keyset stays at 4 pages. Rules: make the sort key **unique** with the PK as tie-breaker (else equal timestamps are skipped or repeated); `(a, b) < (x, y)` matches a

composite index in PostgreSQL (elsewhere `a < x OR (a = x AND b < y)`); keyset cannot jump to "page 537" but never skips or duplicates rows when data changes between requests. Web APIs hide the last key in an opaque **cursor** token the client passes back (Stripe, Slack, GitHub GraphQL).

12.15 Join algorithms

The logical join (ch. 5) says *what* to match; the planner picks *how* (`07_join_algorithms.sql`):

Algorithm	How it works	Cost	Best when
Nested loop	For each outer row, look up matching inner rows (ideally via an index)	outer rows × one lookup	Outer side small, inner side indexed on the join key
Hash join	Build an in-memory hash table on the smaller input's join key, stream the larger input and probe it	read both once; needs memory (`work_mem`, the per-operation memory budget)	Large unsorted inputs, equality joins only
Merge join	Both inputs sorted on the key (index or Sort); walk them together like merging two sorted lists	read both once (+ sorts)	Inputs already sorted, or output needs that order; also range-ish joins

Three customers (30 orders):

```
Nested Loop (actual time=0.015..0.047 rows=30 loops=1)
  ->  Index Scan using customers_pkey on customers c (... rows=3 loops=1)
  ->  Bitmap Heap Scan on orders o (... rows=10 loops=3)     <- inner side runs 3 times
        ->  Bitmap Index Scan on orders_customer_idx (... loops=3)
```

All customers × all orders, grouped by country:

```
HashAggregate (actual time=362.870..362.906 rows=10 loops=1)
  ->  Hash Join (actual time=11.526..288.224 rows=1000000 loops=1)
        Hash Cond: (o.customer_id = c.id)
        ->  Seq Scan on orders o (... rows=1000000 loops=1)
        ->  Hash (actual time=11.401..11.402 rows=100000 loops=1)
              Buckets: 131072  Batches: 2  Memory Usage: 2976kB
```

`Batches: 2`: the 100,000-row hash table did not fit in `work_mem` (4 MB), so it was split and partly written to temporary files (`temp read/written`); raising `work_mem` would make it one batch. With `ORDER BY c.id LIMIT 5` the planner instead walks `customers_pkey` in a **Nested Loop**, looks up orders by index, and stops after five rows. A **Merge Join** appears here only when forced. Forcing each with `enable_*` settings (experiments only):

```
     join_size        |  forced  |  join_node  | exec_ms
----------------------+----------+-------------+---------
small: 3 customers    | nestloop | Nested Loop |     0.1
small: 3 customers    | hash     | Hash Join   |    33.9
small: 3 customers    | merge    | Merge Join  |     0.2
large: all 1M orders | nestloop | Nested Loop |   525.8
large: all 1M orders | hash     | Hash Join   |   236.7
large: all 1M orders | merge    | Merge Join  |   487.5
```

No algorithm always wins: hash join is 300× slower than a nested loop for the small join (it scans all 1M orders) and 2× faster for the large one. The planner chooses by **row estimates** — why statistics matter. MySQL had only nested-loop variants until 8.0.18 added hash joins; SQL Server and Oracle have all three.

12.16 Statistics: how the planner estimates

What it is

The planner **estimates** rows per step from statistics that `ANALYZE` collects from a random sample (30,000 rows at the default `default_statistics_target = 100`) into `pg_statistic` (readable via `pg_stats`). Autovacuum (the background maintenance process) runs `ANALYZE` after enough rows change. Sampling is random, so frequencies vary slightly per run (`08_statistics.sql`):

```
   attname   | null_frac | n_distinct | correlation
-------------+-----------+------------+-------------
 amount      |         0 |      50066 |       0.002
 created_at  |         0 |         -1 |       1.000
 customer_id |         0 |  -0.100437 |       0.003
 note        |    0.9009 |          1 |       1.000
 status      |         0 |          3 |       0.907
      most_common_vals      |        most_common_freqs
----------------------------+---------------------------------
 {shipped,refunded,pending} | {0.95103335,0.0293,0.019666666}
```

Statistic	Meaning	Used for
`null_frac`	Fraction of NULLs	`IS NULL` estimates
`n_distinct`	Distinct values; negative = fraction of rows (−1 = unique, −0.1 = 10 rows per value)	`=` on non-common values, `GROUP BY` sizes
`most_common_vals/freqs`	Most frequent values and their shares	`status = 'refunded'` → 0.0293 × 1M ≈ 29,300 rows
`histogram_bounds`	Equal-population buckets over the other values	Range predicates
`correlation`	Physical order vs column order	Index-scan cost (12.3)

With fresh statistics, `rows=29300` estimated vs `actual rows=30000`.

When statistics go wrong

Stale statistics. `'error'` was 0.1 % of rows when analyzed; then 200,000 error rows arrive unanalyzed:

```
Index Scan using events_kind_idx on events  (cost=0.42..24.78 rows=440 ...) (actual rows=200200 loops=1)
-- after ANALYZE:
Index Scan using events_kind_idx on events  (cost=0.42..5249.09 rows=200000 ...) (actual rows=200200 loops=1)
```

Off by ~450× (varies with the sample). The plan survived here (the new rows are stored together), but in a join such an error makes the planner pick a nested loop running 200,000 times instead of one hash join. Run `ANALYZE` after bulk loads; don't wait for autovacuum.

Correlated columns. The planner multiplies selectivities as if columns were independent. `city = 42 AND state = 4` (city determines state): estimated ~100 (106 in this run), actual 1,000. **Extended statistics** teach the dependency:

```
CREATE STATISTICS addr_city_state (dependencies) ON city, state FROM addresses;
ANALYZE addresses;      -- estimate becomes about 1,000 (1040 in this run), actual 1000
```

Other levers: `ALTER TABLE … ALTER COLUMN … SET STATISTICS 1000` for a finer histogram on a skewed column; `CREATE STATISTICS … (ndistinct, mcv)` for multi-column `GROUP BY` and value combinations.

In the interview

30-second answer: "The optimizer is cost-based: it estimates rows per step from sampled statistics — null fraction, distinct counts, most common values, histograms, correlation — and picks the cheapest plan. When a query is slow, I compare estimated and actual rows in EXPLAIN ANALYZE; a big

gap points to stale or missing statistics, fixed with ANALYZE, a higher statistics target or extended statistics."

- *Why would the planner ignore my index?* → Low selectivity, poor correlation, tiny table, stale stats, a function or type mismatch on the column, or the query needs most rows anyway.

12.17 A tuning workflow

1. **Find** expensive queries: `pg_stat_statements` (per-query timing statistics), slow-query logs (`log_min_duration_statement`, MySQL slow log), `auto_explain`.
2. **Read** `EXPLAIN (ANALYZE, BUFFERS)`: big `Rows Removed by Filter`, estimate/actual gaps, `Batches > 1` or on-disk sorts, huge `loops=`.
3. **Fix the query first** (sargable predicates, fewer columns, keyset, no N+1), **then add indexes** (equality → range/sort, `INCLUDE`, partial, expression; `CONCURRENTLY` in production).
4. **Verify** with `EXPLAIN`; drop indexes whose `idx_scan` stays 0; keep statistics fresh.
5. **Still too big?** Partitioning, replicas, caching, sharding — chapter 15 and System Design Fundamentals.

Dialect differences for storage, covering/partial/expression/hash indexes and plan output are in the dialect table.

Chapter TL;DR

- Performance = **pages touched**. A table is a file of 8 KB slotted pages; 1M orders = 8,662 pages, and a Seq Scan reads them all (~30 ms here).
- Rows are addressed by `ctid` (page, slot); large values are compressed/moved by TOAST. Column stores + RLE/dictionary/delta encoding make analytics cheap.
- File organizations: heap, sorted, hashed, clustered. Physical order (correlation) decides whether an index read touches 100 pages or 8,400.
- Indexes: primary/clustering/secondary, dense/sparse; multilevel indexes → **B+ trees** (3 levels for 1M keys). Hash indexes: equality only. Bloom filters: "definitely not / probably yes", ~10 bits/key ≈ 1 % false positives.
- GIN for contains/JSON/trigrams, GiST for overlaps and exclusion constraints, BRIN for huge append-only time columns.
- Scans: Seq (many rows) · Index (few rows, ordered) · Index Only (all columns in the index, vacuumed) · Bitmap (many scattered rows, combinable).
- Composite indexes: **leftmost prefix**; **equality first, range last** (8 vs 2,216 pages). `INCLUDE` for index-only scans; partial indexes for hot subsets; expression indexes for `lower(email)`.
- Keep predicates **sargable**; no functions on indexed columns; `pg_trgm` for `'%x%'`; kill N+1 loops; **keyset** instead of deep OFFSET (79 ms → 0.02 ms).
- Joins: nested loop (small + index), hash (big, equality), merge (sorted). The planner chooses from **statistics** — compare estimated vs actual rows and `ANALYZE` when they diverge.
- Next: chapter 13 (MVCC, vacuum — why dead rows and visibility maps affect all of the above) and chapter 15 (partitioning and scaling beyond one server).

Chapter 13 — Transactions and Concurrency

A shop's stock table takes hundreds of checkouts per second while reports read it. Two things can go wrong: a failure stops a change **halfway**, or two users working **at the same time** trip over each other. The tools against them are **transactions** and **concurrency control**; this chapter shows how PostgreSQL really behaves with live sessions (separate open connections) running side by side.

What you'll learn: transactions, their life cycle and `BEGIN` / `COMMIT` / `ROLLBACK` / `SAVEPOINT` · ACID and how each letter is implemented · every classic anomaly (dirty read/write, non-repeatable read, phantom, lost update, write skew) **reproduced on a real server** with two interleaved sessions · isolation levels in PostgreSQL vs MySQL, SQL Server, Oracle, SQLite · locks and two-phase locking, optimistic validation, timestamp ordering, MVCC (`xmin` / `xmax`) · deadlocks, `FOR UPDATE`, `NOWAIT`, the `SKIP LOCKED` job queue.

How to run the examples. Single-session files in `code/` run on the book's PostgreSQL 14 server (port 5544):

```
bash tools_run_sql.sh sql                                  # runs every chapter's .sql files
bash tools_pg.sh psql -d <db> -f sql/chapters/13-transactions-and-concurrency/code/01_commit_and_rollback.sql
```

Two-session demos are in `code/concurrency_demo.py`: connections **A** and **B** send statements in a fixed order; a statement expected to block runs in a background thread while a third *monitor* connection checks `pg_stat_activity` (the view of every connection and what it is doing) to prove it is waiting on a lock:

```
cd sql/chapters/13-transactions-and-concurrency/code
../../../../.venv-books/bin/python concurrency_demo.py            # all scenarios (~2 s)
../../../../.venv-books/bin/python concurrency_demo.py writeskew  # just one group
```

In the output `A| …` is a statement from session A, `->` the server's answer, `..` a monitor observation; transaction IDs differ per run. Each scenario starts from fresh data (`00_schema.sql`):

```
CREATE TABLE accounts (id int PRIMARY KEY, owner text NOT NULL,
                       balance numeric(10,2) NOT NULL CHECK (balance >= 0));
INSERT INTO accounts VALUES (1, 'asha', 500), (2, 'ben', 300), (3, 'chen', 50);

CREATE TABLE products (id int PRIMARY KEY, name text NOT NULL,
                       stock int NOT NULL CHECK (stock >= 0), version int NOT NULL DEFAULT 1);
INSERT INTO products VALUES (1, 'concert ticket', 10, 1), (2, 'poster', 3, 1);

CREATE TABLE on_call (doctor text PRIMARY KEY, shift text NOT NULL, on_duty boolean NOT NULL);
INSERT INTO on_call VALUES ('dr_rao', 'night', true), ('dr_kim', 'night', true), ('dr_lee', 'day', true);

CREATE TABLE jobs (id int PRIMARY KEY, payload text NOT NULL,
                   status text NOT NULL DEFAULT 'queued', claimed_by text);
INSERT INTO jobs (id, payload) SELECT g, 'email #' || g FROM generate_series(1, 6) g;
```

13.1 What a transaction is

What it is

A **transaction** is a group of operations treated as **one logical unit of work**: every operation takes effect, or none does. It ends with a **commit** (changes permanent and visible to others) or an **abort**/rollback (every change undone).

Sending ₹100 from Asha to Ben needs two writes; a crash or broken rule **between** them loses 100. A transaction makes the pair all-or-nothing, which also makes **retrying safe**: after an abort nothing is half-applied, so the application simply runs it again.

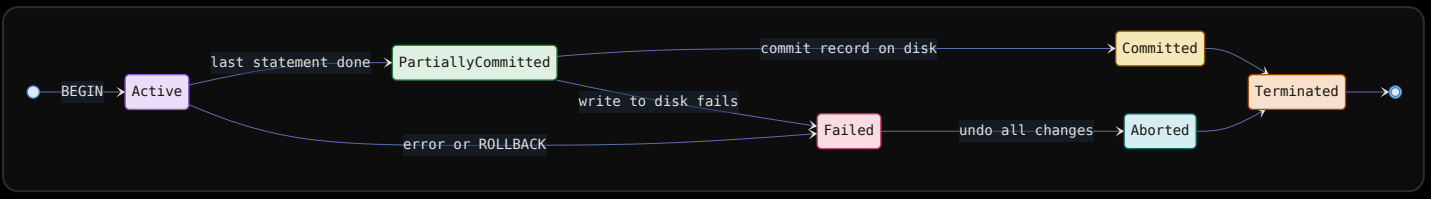
ACID in one table

The acronym **ACID** (Härder and Reuter, 1983); each letter gets a section below.

Letter	Guarantee	Main mechanism
Atomicity	All or nothing — no half-done transfers	Undo information / WAL (write-ahead log, §13.5), shadow copies (§13.2)
Consistency	Valid state → valid state (constraints and invariants — rules that must always hold — true after commit)	Constraints + correct application logic
Isolation	Concurrent transactions don't see each other's in-progress work	Locks, MVCC (multi-version concurrency control, §13.14), SSI (serializable snapshot isolation, §13.8)
Durability	Committed means kept, even after a crash or power loss	WAL flushed to disk, replication (live copies on other servers)

Life cycle of a transaction

State	What is happening
Active	Statements running; changes not final.
Partially committed	Last statement done; results not yet safely on permanent storage.
Committed	Commit record durable; changes permanent.
Failed	An error (constraint, crash, deadlock §13.11, cancel) stops normal progress.
Aborted	All changes rolled back.
Terminated	Finished; locks and snapshot (the frozen view of data it was reading) released.



After an abort the application **restarts** (temporary problems such as a deadlock) or **gives up** and reports (logical problems such as "insufficient funds").

How to write one in SQL

Statement	Effect
`BEGIN` (or `START TRANSACTION`)	Start a transaction block.
`COMMIT`	Make all changes permanent.
`ROLLBACK`	Undo all changes since `BEGIN`.
`SAVEPOINT name`	Mark a point inside the transaction.
`ROLLBACK TO SAVEPOINT name`	Undo only the work after that mark; the transaction stays open.
`RELEASE SAVEPOINT name`	Forget the mark (keep the work).

Autocommit. Without `BEGIN`, PostgreSQL (and by default psql, JDBC — Java's standard database API — and most drivers) wraps **each statement** in its own transaction and commits it. A lone `UPDATE` is already atomic; you need `BEGIN … COMMIT` only when several statements must succeed or fail together. (Python's `psycopg` is the odd one out: it opens a transaction and waits for `conn.commit()`.)

`code/01_commit_and_rollback.sql`:

```sql
-- 1) Money transfer that commits: both rows change together.
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE id = 1;   -- debit asha
UPDATE accounts SET balance = balance + 100 WHERE id = 2;   -- credit ben
COMMIT;
SELECT id, owner, balance, (SELECT sum(balance) FROM accounts) AS bank_total
FROM accounts ORDER BY id;

-- 2) A transfer we change our mind about: ROLLBACK restores the old state.
BEGIN;
UPDATE accounts SET balance = balance - 200 WHERE id = 1;
SELECT 'inside txn' AS moment, balance FROM accounts WHERE id = 1;
ROLLBACK;
SELECT 'after rollback' AS moment, balance FROM accounts WHERE id = 1;

-- 3) Savepoints: undo only part of a transaction.
BEGIN;
UPDATE accounts SET balance = balance + 10 WHERE id = 3;    -- keep this
SAVEPOINT bonus;
UPDATE accounts SET balance = balance + 999 WHERE id = 3;   -- oops
ROLLBACK TO SAVEPOINT bonus;                                -- undo just the oops
COMMIT;
SELECT id, owner, balance FROM accounts WHERE id = 3;
```

Output:

```
BEGIN
UPDATE 1
UPDATE 1
COMMIT
 id | owner | balance | bank_total
----+-------+---------+------------
  1 | asha  |  400.00 |     850.00
  2 | ben   |  400.00 |     850.00
  3 | chen  |   50.00 |     850.00
(3 rows)

BEGIN
UPDATE 1
   moment   | balance
------------+---------
 inside txn |  200.00
(1 row)

ROLLBACK
     moment     | balance
----------------+---------
 after rollback |  400.00
(1 row)

BEGIN
UPDATE 1
SAVEPOINT
UPDATE 1
ROLLBACK
COMMIT
 id | owner | balance
----+-------+---------
  3 | chen  |   60.00
(1 row)
```

The bank total stays 850; the session saw its own uncommitted 200, but after `ROLLBACK` the balance is 400 again; only the +10 survived the savepoint.

	PostgreSQL	MySQL (InnoDB)	SQL Server	SQLite
Start	`BEGIN` / `START TRANSACTION`	`START TRANSACTION` / `BEGIN`	`BEGIN TRAN`	`BEGIN [DEFERRED|IMMEDIATE|EXCLUSIVE]`
DDL (`CREATE TABLE` …) inside a transaction	Transactional (can be rolled back)	Implicit commit (silently commits the open transaction)	Mostly transactional	Transactional
After an error mid-transaction	Whole transaction "aborted" until `ROLLBACK`	Only the failing statement undone; transaction continues	Depends on `XACT_ABORT`	Only the failing statement undone

Weaker guarantees some systems offer

Many NoSQL and caching systems trade multi-statement transactions for speed or scale:

Guarantee	What it means	Example
Single-object atomicity	A write to *one* record is all-or-nothing; nothing spans two records	A MongoDB document update; a Cassandra row write
Atomic increment	Server does read-add-write in one indivisible step, so concurrent `+1`s never lose each other	Redis (in-memory key-value store) `INCR`; SQL `UPDATE … SET n = n + 1`
Compare-and-set (CAS)	"Write the new value only if the current value is still what I read"; else fail and retry	Redis `WATCH`/`MULTI`, DynamoDB conditional writes, SQL `UPDATE … WHERE version = 7` (§13.7)

They protect **one** object; keeping several in step (a transfer, an order plus its stock decrement) needs a real transaction — why MongoDB (4.0+) and NewSQL systems (distributed databases keeping SQL and full transactions) added multi-document transactions.

In the interview

> **30-second answer:** "A transaction is a unit of work that fully commits or fully rolls back, with the ACID guarantees. In SQL: `BEGIN … COMMIT/ROLLBACK`, savepoints for partial rollback; without `BEGIN` each statement autocommits."

- *Why not undo step 1 myself on error?* The crash may kill your program too; the database's log handles it even after a power failure.
- *Is a single `UPDATE` atomic?* Yes — autocommit makes it a transaction, and a multi-row `UPDATE` changes all matching rows or none.

Self-check: the six states? → active, partially committed, committed, failed, aborted, terminated. `ROLLBACK TO SAVEPOINT s` keeps? → everything before `s`; the transaction stays open.

13.2 Atomicity

What it is

Atomicity: a transaction is **indivisible** — all changes applied (commit) or none (rollback); no visible half-done state. Transfer 10 from X = 30 to Y = 100: the debit succeeds (X = 20), the credit fails. Without atomicity ten units vanish (X = 20, Y = 100); with it the debit is rolled back too (X = 30, Y = 100).

How it works — three implementations

1. Shadow copy (early systems, some editors):

1. At begin, copy the whole database file — the **shadow**.
2. Make every change in the shadow only.
3. Commit = atomically switch a **db-pointer** (a small file naming the current database file) to the shadow, then delete the old file.
4. Abort = delete the shadow; the pointer never moved.

The pointer switch is the only commit point: a crash after all writes but before the switch (or midway) leaves the untouched original current. Impractical for real databases: copying the **entire** database per transaction is slow and wasteful, and it allows only **one** transaction at a time. (Its descendant *shadow paging* — copy only modified pages — was used in System R and lives on in copy-on-write stores such as LMDB, where changes always go to fresh copies.)

2. Logging (PostgreSQL, MySQL, SQL Server, Oracle). Changes go into in-memory pages and are described in the **write-ahead log (WAL)** (§13.5). On abort or crash before commit the engine undoes them from the log — or, in PostgreSQL, simply marks the transaction ID *aborted* so its row versions are ignored (§13.14).

3. Across machines. Two-phase commit (2PC): a coordinator asks every participant to *prepare* (make the change durable but not final) and, only if all agree, tells them to *commit* (chapter 15; PostgreSQL's `PREPARE TRANSACTION`). Optimistic concurrency control (§13.9) also applies writes all at once at commit, only if no conflict was found.

Code: an error aborts the whole transaction

`code/02_atomicity_on_error.sql` — Chen has 50 and tries to send 80. The credit to Asha succeeds, then the debit breaks `CHECK (balance >= 0)` (`\set ON_ERROR_STOP off` lets psql continue after an error):

```
\set ON_ERROR_STOP off
BEGIN;
UPDATE accounts SET balance = balance + 80 WHERE id = 1;   -- credit asha: OK
UPDATE accounts SET balance = balance - 80 WHERE id = 3;   -- debit chen: violates CHECK
SELECT balance FROM accounts WHERE id = 1;                 -- refused: txn is aborted
COMMIT;                                                    -- turns into ROLLBACK
SELECT id, owner, balance FROM accounts ORDER BY id;       -- nothing changed

BEGIN;
UPDATE accounts SET balance = balance + 5 WHERE id = 2;
SAVEPOINT before_risky;
INSERT INTO accounts VALUES (1, 'dup', 1);                 -- duplicate primary key
ROLLBACK TO SAVEPOINT before_risky;
COMMIT;
SELECT id, owner, balance FROM accounts WHERE id = 2;
```

Output (psql file path shortened):

```
BEGIN
UPDATE 1
psql:02_atomicity_on_error.sql:10: ERROR:  new row for relation "accounts" violates check constraint
"accounts_balance_check"
psql:02_atomicity_on_error.sql:11: ERROR:  current transaction is aborted, commands ignored until end of
transaction block
ROLLBACK
 id | owner | balance
----+-------+---------
  1 | asha  |  500.00
  2 | ben   |  300.00
  3 | chen  |   50.00
(3 rows)

BEGIN
UPDATE 1
SAVEPOINT
psql:02_atomicity_on_error.sql:19: ERROR:  duplicate key value violates unique constraint "accounts_pkey"
ROLLBACK
COMMIT
 id | owner | balance
----+-------+---------
  2 | ben   |  305.00
(1 row)
```

After an error PostgreSQL marks the transaction **aborted** and refuses further commands; `COMMIT` is answered with `ROLLBACK`, undoing Asha's +80 too. Rolling back to a savepoint clears the error and keeps Ben's +5.

Common mistakes

- The two halves of a transfer as two autocommitted statements.
- Catching the error in code and carrying on to `COMMIT` — PostgreSQL silently rolls back; MySQL may commit the half that worked.
- Expecting rollback to undo an email sent or card charged — it undoes only **database** changes; write the action to an *outbox* table (pending messages, written in the same transaction, sent by a separate process after commit).

In the interview

> **30-second answer:** "All-or-nothing. Shadow copy (work on a copy, flip a pointer at commit) explains the idea but copies too much and serialises everything; real engines log changes and roll back from the log; across databases you need two-phase commit."

- *Does atomicity say anything about concurrency?* No — ACID's "atomic" is about failures; other users are isolation's job (unlike "atomic" in multithreaded code).

Self-check: PostgreSQL's answer to `COMMIT` in an aborted transaction? → `ROLLBACK`.

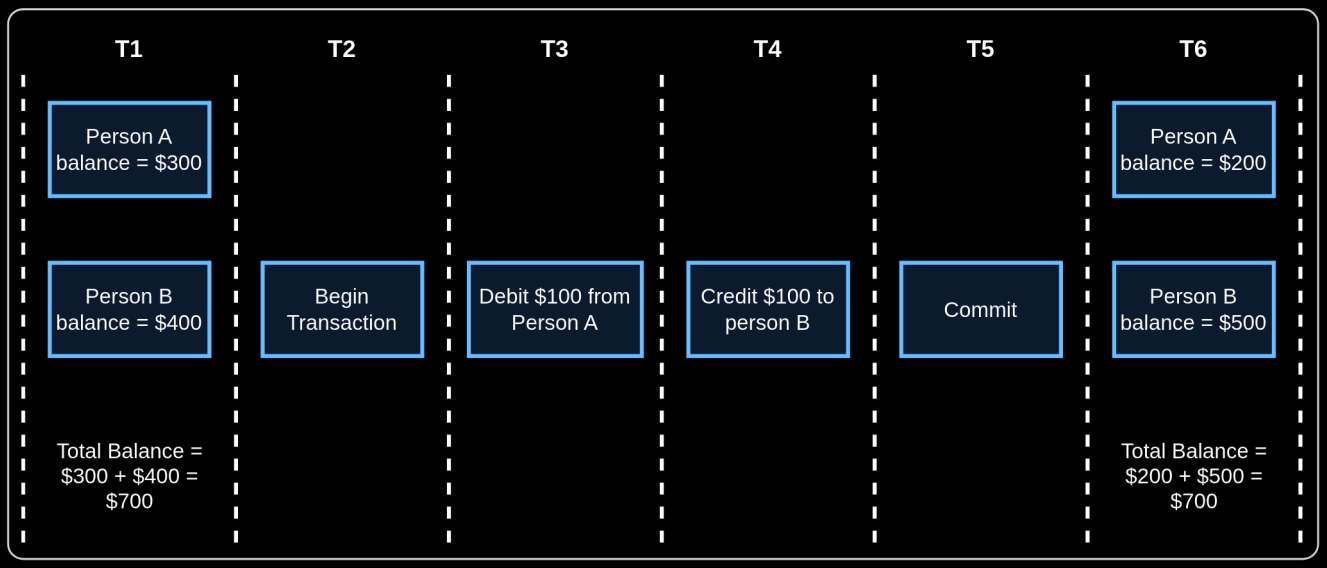
13.3 Consistency

What it is

Consistency (ACID's C): every committed transaction leaves the database **valid if it started valid**. "Valid" means the rules hold — primary/unique/foreign keys, `CHECK`, `NOT NULL`, and business **invariants** such as "a transfer neither creates nor destroys money". A rule-breaking transaction must fail and leave the previous state; rules may be briefly false only *inside* a transaction.

Asha 300 + Ben 400 = **700**; transfer 100:

Moment	Step	Asha	Ben	Total	Valid?
t1	initial state / `BEGIN`	300	400	700	yes
t2	debit Asha 100	200	400	**600**	no (intermediate)
t3	credit Ben 100	200	500	700	yes
t4	`COMMIT` / final state	200	500	700	yes



The temporary 600 is harmless: nobody else can see it (isolation) and it can never be final (atomicity).

How it is achieved — and how it breaks

1. **Declarative constraints**: `PRIMARY KEY`, `UNIQUE`, `FOREIGN KEY`, `CHECK (balance >= 0)`, `NOT NULL` (chapter 9); §13.2's `CHECK` rejected an overdraft.
2. **Pre-transaction checks**, best inside the statement: `UPDATE … SET balance = balance - 100 WHERE id = 1 AND balance >= 100`.
3. **Post-transaction verification**: deferred constraints (`DEFERRABLE INITIALLY DEFERRED` checks a foreign key only at commit) or audits (the ledger still sums to the same total).
4. **Atomicity** and **isolation** stop failures and concurrency from leaving a broken state.

Cause	Example	Who stops it
Constraint violation	Duplicate primary key	The database rejects the statement
Crash mid-transaction	Crash after debit, before credit	Atomicity + recovery
Logic bug	Code transfers more than the balance	Only a `CHECK` or correct code
Concurrency without enough isolation	Two withdrawals both see enough money	Isolation level / locking (§13.4–13.8)

Trade-offs: The database enforces only rules it knows. Cross-row invariants ("at least one doctor on call") can't be a simple `CHECK`; the application enforces them with strong enough isolation (§13.8) — hence Martin Kleppmann (DDIA, *Designing Data-Intensive Applications*) calls C an *application* property, with A, I and D as the database's tools. **ACID consistency is not CAP consistency:** in CAP (the

distributed-systems trade-off theorem, chapter 15) "consistency" means *linearizability* — every read sees the latest write across replicas.

In the interview

> **30-second answer:** "Every committed state obeys the rules — keys, checks, business invariants. The database enforces declared constraints, the application the rest; atomicity and isolation keep failures and concurrency from breaking them."

- *An invariant a CHECK can't express?* "At least one doctor on call per shift" — spans rows; needs SERIALIZABLE or explicit locking.

Self-check: a temporarily wrong total inside a transaction is a violation? → No, only committed states must be valid. Why can concurrency break consistency? → Two transactions each valid alone can together break an invariant under weak isolation.

13.4 Isolation — anomalies and isolation levels

What it is

Isolation controls what one running transaction can see of another's in-progress work. Ideally each behaves **as if alone**, but true one-at-a-time execution is too slow, so databases offer **isolation levels**: weaker levels allow more overlap but permit **anomalies** (wrong results caused by interleaving); stronger ones forbid more at the cost of blocking or aborts.

The anomalies

T1 and T2 are concurrent; the account starts at 500.

1. Dirty read — reading another transaction's **uncommitted** write.

Time	T1	T2
1	`BEGIN`; balance = 500 − 100 → **400** (uncommitted)	
2		reads balance → **400**
3	`ROLLBACK` → balance is 500 again	
4		has acted on 400, a value that never officially existed

2. Dirty write — overwriting another transaction's **uncommitted** write. T1 adds 100 (→ 600, uncommitted); T2 adds 100 (→ 700); T1 aborts. Restoring 500 erases T2's deposit, keeping 700 keeps T1's cancelled one; the correct 600 can't be reconstructed. Practically every database at every level prevents it by making the second writer **wait**.

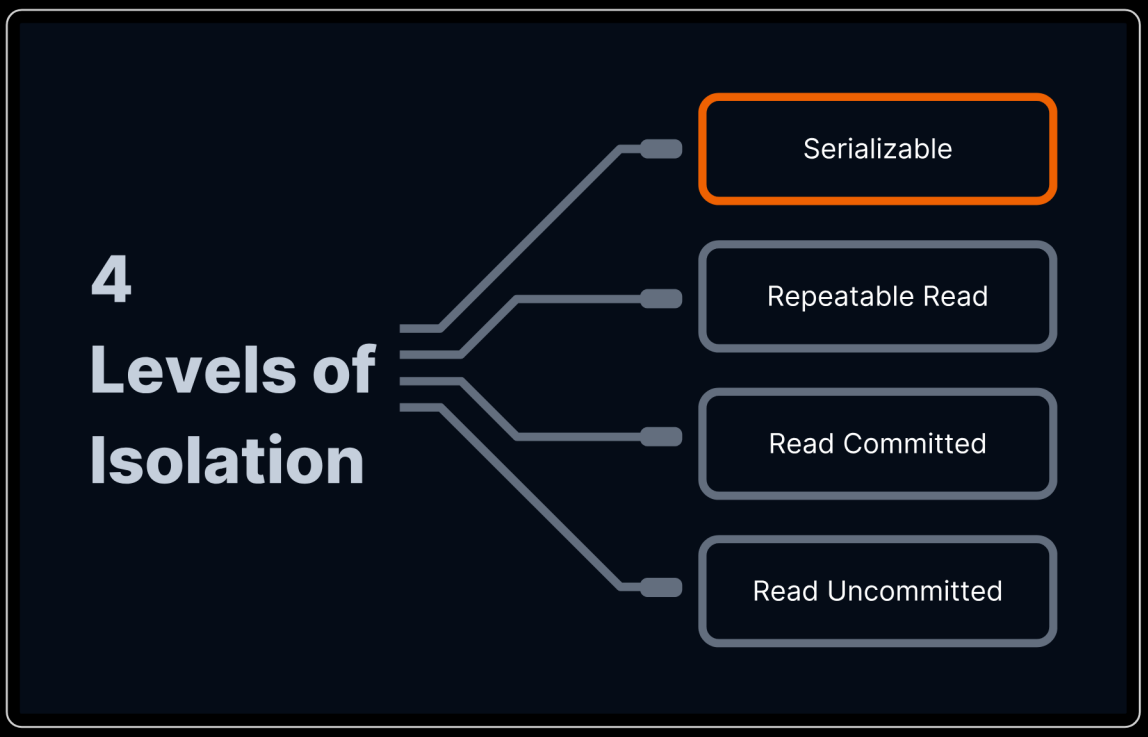
3. Non-repeatable read (read skew) — the same row read twice gives **different committed values** (T1 reads 500; T2 sets 400 and commits; T1 reads 400 — live in §13.6). "Read skew" broadly means seeing *different moments in time* in one transaction — e.g. account 1 before a transfer and account 2 after it, so the total looks wrong (§13.6).

4. Phantom read — the same **query** returns a **different set of rows** because another transaction inserted or deleted matching rows (a count of 2 becomes 3 — §13.6).

5. Lost update — two read-compute-write cycles; the second write silently overwrites the first (both read stock 10 and write 9: two sold, stock fell by one — §13.7).

6. Write skew — two transactions read the **same rows**, decide, and each writes **a different row**; neither overwrote the other, yet together they break a rule both checked (two on-call doctors each see "2 on call", both go off duty, §13.8). A lost update is the special case where both write the *same* row.

The isolation levels



The SQL standard defines four levels by which read anomalies they must prevent:

Level	Dirty read	Non-repeatable read	Phantom
Read Uncommitted	possible	possible	possible
Read Committed	prevented	possible	possible
Repeatable Read	prevented	prevented	possible
Serializable	prevented	prevented	prevented

That is a *minimum*; the standard ignores lost updates and write skew. What **PostgreSQL** actually provides (verified in §13.6–13.8):

Level in PostgreSQL	Dirty read	Dirty write	Non-repeatable / read skew	Phantom	Lost update	Write skew	How it's done
Read Uncommitted	**no** (treated as Read Committed)	no	yes	yes	yes	yes	new snapshot per statement
Read Committed (default)	no	no	yes	yes	yes (read-then-write in app)	yes	new snapshot per statement + row locks for writes
Repeatable Read	no	no	no	**no** (stronger than the standard)	no — aborts with a serialization error	**yes**	one snapshot per transaction (snapshot isolation)
Serializable	no	no	no	no	no	no — aborts one	snapshot isolation + SSI conflict tracking

At **Read Committed**, MVCC engines give readers the last committed version so they never wait, while lock-based engines (SQL Server unless `READ_COMMITTED_SNAPSHOT` is on) make readers wait for the writer.

Serializable (outcome equals *some* one-at-a-time order) is built with **two-phase locking** (take locks as you go, release none until the end, §13.10; MySQL, SQL Server) or **Serializable Snapshot Isolation (SSI)**, which tracks read/write dependencies and aborts a transaction when needed (PostgreSQL 9.1+) — so be ready to retry. Read Uncommitted (SQL Server's `NOLOCK` hint) is almost never used.

Engine	Default level	Notes
PostgreSQL	Read Committed	RR = snapshot isolation (no phantoms); Serializable = SSI.
MySQL InnoDB	Repeatable Read	Plain `SELECT` reads a snapshot; locking reads (`FOR UPDATE`, `UPDATE`) read the *latest* row with next-key locks (index entries plus the gaps between them, §13.10) against phantoms. Does **not** abort lost updates at RR — use `FOR UPDATE` or atomic updates.
SQL Server	Read Committed (locking)	Optional `READ_COMMITTED_SNAPSHOT` and `SNAPSHOT` isolation keep row versions in tempdb (its shared scratch database).
Oracle	Read Committed	Its "SERIALIZABLE" is snapshot isolation (write skew possible).
SQLite	Serializable	One writer at a time for the whole file; WAL mode lets readers run during a write.

How to choose and set a level

```
BEGIN ISOLATION LEVEL REPEATABLE READ;                -- for one transaction
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;         -- first statement inside BEGIN
SET default_transaction_isolation = 'serializable';   -- for the session
SHOW transaction_isolation;
```

If you need…	Use
Normal OLTP, single-statement updates (`SET x = x + 1`)	Read Committed
A consistent report or export across many queries	Repeatable Read (often `READ ONLY`)
Read-then-write logic on the same row	`SELECT … FOR UPDATE` at Read Committed, or Repeatable Read + retry
Invariants that span rows (no double booking, at least one on call)	Serializable + retry, or explicit locks

In the interview

30-second answer: "Isolation levels trade correctness for concurrency; the standard defines four by which of dirty reads, non-repeatable reads and phantoms they allow. Postgres never allows dirty reads, its Repeatable Read is snapshot isolation (no phantoms or lost updates, but write skew), and Serializable uses SSI to abort one transaction of a write skew."

- *Non-repeatable read vs phantom?* Non-repeatable: the *same row* changed value. Phantom: the *set of rows* matching a condition changed (insert/delete).
- *Why isn't everyone on Serializable?* More aborts (or blocking with 2PL) and mandatory retries; most workloads are fine at Read Committed with a few targeted locks.

Self-check: default level in PostgreSQL? MySQL? → Read Committed; Repeatable Read.

13.5 Durability

What it is

Durability: once the database says "committed", the change **stays** — even if power fails a millisecond later. People act on "committed" (ship the order), so recovery must restore every committed transaction

and nothing uncommitted.

How it works

1. **Write-ahead logging (WAL).** A record describing each change must reach the log before the changed data page may be written. At `COMMIT` the database appends a commit record and **flushes** the log (`fsync` — the OS call forcing data out of caches onto the physical disk); only then does it answer "COMMIT". Data pages are written later; after a crash recovery **replays** the log, redoing committed changes that never reached the data files. (One sequential append is much cheaper than updating scattered pages, and the log record alone suffices to redo the change.)
2. **Checkpoints.** Periodically all dirty pages (changed in memory, not yet in the data files) are written and "everything up to log position X is on disk" is recorded, so recovery replays only the log after the latest checkpoint (PostgreSQL: `checkpoint_timeout`, default 5 min, and `max_wal_size`).
3. **Synchronous disk writes.** A commit is acknowledged only once the log is on stable storage, not just in RAM or the OS cache: `synchronous_commit = on` and `fsync = on` (both defaults); `full_page_writes` protects against *torn pages* (a page only partly written at power loss).
4. **Replication.** Disks die, so the WAL is shipped to other machines (streaming replication: WAL records sent continuously); with *synchronous* replication a commit also waits for a replica (chapter 15).

Code: watching the WAL grow

`code/05_durability_wal.sql` inserts 1,000 rows and measures how far the WAL position (**LSN**, log sequence number) moved:

```
SELECT name, setting FROM pg_settings
WHERE name IN ('fsync', 'synchronous_commit', 'wal_level', 'full_page_writes', 'checkpoint_timeout')
ORDER BY name;

CREATE TEMP TABLE lsn_marks (label text, lsn pg_lsn);
INSERT INTO lsn_marks VALUES ('before', pg_current_wal_lsn());
INSERT INTO accounts SELECT g, 'user' || g, 100 FROM generate_series(10, 1009) g;   -- 1,000 rows
INSERT INTO lsn_marks VALUES ('after', pg_current_wal_lsn());

SELECT pg_size_pretty(pg_wal_lsn_diff(
        (SELECT lsn FROM lsn_marks WHERE label = 'after'),
        (SELECT lsn FROM lsn_marks WHERE label = 'before'))) AS wal_written_for_1000_rows;

BEGIN;
SET LOCAL synchronous_commit = off;     -- faster commit; a crash may lose this txn, never corrupts
UPDATE accounts SET balance = balance + 1 WHERE id = 10;
COMMIT;
```

Output:

```
       name        | setting
-------------------+---------
 checkpoint_timeout | 300
 fsync              | on
 full_page_writes   | on
 synchronous_commit | on
 wal_level          | replica
(5 rows)
...
 wal_written_for_1000_rows
---------------------------
 135 kB
(1 row)
...
```

About 135 kB of WAL for 1,000 rows (varies slightly by version and settings). The `synchronous_commit = off` transaction returns before its WAL is flushed: a crash within about three times `wal_writer_delay` could lose that one commit but never corrupt the database — fine for click logging, never for payments.

Common mistakes: turning off `fsync` in production (a crash can **corrupt** the database, not just lose commits); assuming an *asynchronous* replica makes a commit durable (the primary — the server accepting writes — can die before shipping it); disks or cloud volumes that lie about flushes (a drive write cache without battery backup).

In the interview

30-second answer: "Committed data survives crashes: the WAL record is fsynced before the commit is acknowledged, data pages are written later, and restart replays the log from the last checkpoint. Replication protects against losing the machine."

- *Why not write the data pages at commit?* Scattered page writes cost far more than one sequential log append, and the log record alone can redo the change.

Self-check: what must be on disk before `COMMIT` returns? → The transaction's WAL including the commit record. Does asynchronous replication make the latest commit more durable? → No; only synchronous replication does.

13.6 Watching isolation anomalies on a real server

The §13.4 anomalies on PostgreSQL 14 with two live sessions, via `code/concurrency_demo.py`. Its core: a `Session` class opens an autocommit connection (the script sends `BEGIN` / `COMMIT` itself) with `run(sql)` to send and print, and `start(sql)` to run a statement expected to **block** in a thread; a `Monitor` polls `pg_stat_activity` (`wait_event_type, wait_event, pg_blocking_pids(pid)`) until that session shows `wait_event_type = 'Lock'`. Results print as rows, a command tag (`UPDATE 1`), or `ERROR <sqlstate>: <message>` (SQLSTATE = the standard five-character error code, e.g. `40001`); process IDs in deadlock messages are replaced by `<A>` / `<B>`.

Dirty read and dirty write — impossible in PostgreSQL

A sets Asha's balance to 0 without committing; B even asks for `READ UNCOMMITTED`:

```
=== Dirty read and dirty write: impossible in PostgreSQL ===
  A| BEGIN  -> BEGIN
  A| UPDATE accounts SET balance = 0 WHERE id = 1  -> UPDATE 1
  B| BEGIN ISOLATION LEVEL READ UNCOMMITTED  -> BEGIN
  B| SHOW transaction_isolation  -> read uncommitted
  B| SELECT balance FROM accounts WHERE id = 1  -> 500.00
  B| UPDATE accounts SET balance = balance + 1 WHERE id = 1
     .. B is WAITING (wait_event=transactionid, blocked by A)
  A| ROLLBACK  -> ROLLBACK
     -> (B resumes) UPDATE 1
  B| COMMIT  -> COMMIT
  M| SELECT balance FROM accounts WHERE id = 1  -> 501.00
```

- **No dirty read:** at `read uncommitted` B still reads the committed **500** — PostgreSQL implements Read Uncommitted as Read Committed — and without blocking (MVCC hands it the older version).
- **No dirty write:** B's `UPDATE` **waits** (`transactionid`: for A's transaction to end); after A's rollback it turns the committed 500 into 501.

Non-repeatable read / read skew: Read Committed vs Repeatable Read

A reads Asha's balance; B transfers 100 from account 1 to 2 and commits; A reads both:

```
=== Non-repeatable read / read skew: READ COMMITTED ===
  A| BEGIN ISOLATION LEVEL READ COMMITTED  -> BEGIN
  A| SELECT balance FROM accounts WHERE id = 1  -> 500.00
  B| BEGIN  -> BEGIN
  B| UPDATE accounts SET balance = balance - 100 WHERE id = 1  -> UPDATE 1
  B| UPDATE accounts SET balance = balance + 100 WHERE id = 2  -> UPDATE 1
  B| COMMIT  -> COMMIT
  A| SELECT balance FROM accounts WHERE id = 1  -> 400.00
  A| SELECT balance FROM accounts WHERE id = 2  -> 400.00
  A| COMMIT  -> COMMIT
```

The same row gave **500 then 400** — non-repeatable. An auditor adding its first read of account 1 (500) to account 2 (400) reports **900**, though the true total was 800 at every instant: **read skew**.

```
=== Non-repeatable read / read skew: REPEATABLE READ ===
  A| BEGIN ISOLATION LEVEL REPEATABLE READ  -> BEGIN
  A| SELECT balance FROM accounts WHERE id = 1  -> 500.00
  ... (B runs the same transfer as above and commits)
  A| SELECT balance FROM accounts WHERE id = 1  -> 500.00
  A| SELECT balance FROM accounts WHERE id = 2  -> 300.00
  A| COMMIT  -> COMMIT
```

At Repeatable Read A sees the world as of its **first statement** (not `BEGIN`): 500 + 300 = 800 — consistent, slightly old.

Phantom read: Read Committed vs Repeatable Read

```
=== Phantom read: READ COMMITTED ===
  A| BEGIN ISOLATION LEVEL READ COMMITTED  -> BEGIN
  A| SELECT count(*) FROM accounts WHERE balance >= 100  -> 2
  B| INSERT INTO accounts VALUES (4, 'dina', 900)  -> INSERT 0 1
  A| SELECT count(*) FROM accounts WHERE balance >= 100  -> 3
  A| COMMIT  -> COMMIT

=== Phantom read: REPEATABLE READ (PostgreSQL prevents it) ===
  A| BEGIN ISOLATION LEVEL REPEATABLE READ  -> BEGIN
  A| SELECT count(*) FROM accounts WHERE balance >= 100  -> 2
  B| INSERT INTO accounts VALUES (4, 'dina', 900)  -> INSERT 0 1
  A| SELECT count(*) FROM accounts WHERE balance >= 100  -> 2
  A| COMMIT  -> COMMIT
```

Read Committed shows a phantom (2 → 3); at Repeatable Read the snapshot simply lacks Dina's row, though the standard would allow a phantom.

> **Common mistakes:** believing Repeatable Read shows "the latest data" (a long RR transaction sees an increasingly stale world and holds back cleanup, §13.14); running a multi-query report at Read Committed and getting totals that never existed — use `BEGIN ISOLATION LEVEL REPEATABLE READ READ ONLY`.

13.7 Lost updates and how to prevent them

A **lost update** comes from *read–modify–write*: the application reads a value, computes a new one in its own code, and writes it back; overlapping copies wipe each other out. It is the most common concurrency bug in web apps (stock counters, likes, wallet balances, wiki edits). Two buyers, each app instance subtracting one in Python/Java:

```
=== Lost update 1: read-modify-write in the application (READ COMMITTED) ===
  A| BEGIN  -> BEGIN
  A| SELECT stock FROM products WHERE id = 1  -> 10
  B| BEGIN  -> BEGIN
  B| SELECT stock FROM products WHERE id = 1  -> 10
  A| UPDATE products SET stock = 9 WHERE id = 1  -> UPDATE 1
  A| COMMIT  -> COMMIT
  B| UPDATE products SET stock = 9 WHERE id = 1  -> UPDATE 1
  B| COMMIT  -> COMMIT
  M| SELECT stock FROM products WHERE id = 1  -> 9
```

Two tickets sold, stock **9**, no error. Four standard fixes make read and write one protected step.

Fix 1 — let the database do the arithmetic (atomic update)

```
=== Lost update fix 1: atomic UPDATE ... SET stock = stock - 1 ===
  A| BEGIN  -> BEGIN
  A| UPDATE products SET stock = stock - 1 WHERE id = 1 RETURNING stock  -> 9
  B| BEGIN  -> BEGIN
  B| UPDATE products SET stock = stock - 1 WHERE id = 1 RETURNING stock
     .. B is WAITING (wait_event=transactionid, blocked by A)
     .. pg_locks for B: (transactionid, ExclusiveLock, True), (tuple, ExclusiveLock, True), (transactionid,
ShareLock, False)
  A| COMMIT  -> COMMIT
     -> (B resumes) 8
  B| COMMIT  -> COMMIT
  M| SELECT stock FROM products WHERE id = 1  -> 8
```

B waits for A's row lock; when A commits, PostgreSQL (at Read Committed) **re-reads the newest row version** and applies `stock - 1`: 9 → 8. `pg_locks` (the view of held and awaited locks) shows the mechanism: B holds its own transaction-ID lock and a *tuple* lock (first in line for the row) and waits (`granted = False`) for a `ShareLock` on **A's transaction ID** — "wake me when A ends". The row lock itself lives in the row (its `xmax`), not a lock table, so locking a million rows needs no million lock entries. *Use when* the new value is computable in SQL (`n = n + 1`, `balance = balance - 100 WHERE balance >= 100`) — simplest and fastest.

Fix 2 — pessimistic lock: `SELECT … FOR UPDATE`

When the new value needs application logic, lock the row while reading it:

```
=== Lost update fix 2: pessimistic SELECT ... FOR UPDATE ===
  A| BEGIN  -> BEGIN
  A| SELECT stock FROM products WHERE id = 1 FOR UPDATE  -> 10
  B| BEGIN  -> BEGIN
  B| SELECT stock FROM products WHERE id = 1 FOR UPDATE
     .. B is WAITING (wait_event=transactionid, blocked by A)
  A| UPDATE products SET stock = 9 WHERE id = 1  -> UPDATE 1
  A| COMMIT  -> COMMIT
     -> (B resumes) 9
  B| UPDATE products SET stock = 8 WHERE id = 1  -> UPDATE 1
  B| COMMIT  -> COMMIT
  M| SELECT stock FROM products WHERE id = 1  -> 8
```

B's read waits, then returns **9**, so B computes 8. The lock lasts to the end of the transaction — keep it short and never wait for a human while holding one.

Fix 3 — Repeatable Read detects it (optimistic, by the database)

```
=== Lost update fix 3: REPEATABLE READ detects the conflict ===
  A| BEGIN ISOLATION LEVEL REPEATABLE READ  -> BEGIN
  A| SELECT stock FROM products WHERE id = 1  -> 10
  B| BEGIN ISOLATION LEVEL REPEATABLE READ  -> BEGIN
  B| SELECT stock FROM products WHERE id = 1  -> 10
  A| UPDATE products SET stock = 9 WHERE id = 1  -> UPDATE 1
  A| COMMIT  -> COMMIT
  B| UPDATE products SET stock = 9 WHERE id = 1  -> ERROR 40001: could not serialize access due to concurrent
update
  B| ROLLBACK  -> ROLLBACK
  (B retries the whole transaction)
  ... (B re-reads 9, writes 8, commits)
  M| SELECT stock FROM products WHERE id = 1  -> 8
```

B's snapshot says 10 but the row was changed by a transaction committed after it, so PostgreSQL refuses with **40001** (`serialization_failure`); the application **retries the whole transaction**. (MySQL InnoDB at Repeatable Read raises no such error — its `UPDATE` reads the latest version — so use Fix 1 or 2 there.)

Fix 4 — optimistic locking with a version column (compare-and-set)

CAS holds no lock, so it suits long gaps between read and write (a user editing a form for minutes, several HTTP requests each in its own transaction). Add a `version` column to the `WHERE`:

```
=== Lost update fix 4: optimistic locking with a version column (compare-and-set) ===
  A| SELECT stock, version FROM products WHERE id = 1  -> (10, 1)
  B| SELECT stock, version FROM products WHERE id = 1  -> (10, 1)
  A| UPDATE products SET stock = 9, version = version + 1 WHERE id = 1 AND version = 1  -> UPDATE 1
  B| UPDATE products SET stock = 9, version = version + 1 WHERE id = 1 AND version = 1  -> UPDATE 0
  (B sees UPDATE 0 -> someone changed the row; re-read and retry)
  B| SELECT stock, version FROM products WHERE id = 1  -> (9, 2)
  B| UPDATE products SET stock = 8, version = version + 1 WHERE id = 1 AND version = 2  -> UPDATE 1
  M| SELECT stock, version FROM products WHERE id = 1  -> (8, 3)
```

`UPDATE 0` is the conflict signal ("someone else won — reload and retry, or tell the user the record changed"); the check and write are one atomic step. JPA/Hibernate `@Version`, Rails `lock_version`, DynamoDB conditional writes and Redis `WATCH` / `MULTI` / `EXEC` are the same idea: no locks (so no deadlocks), great when conflicts are rare, wasteful retries when frequent.

Fix	Blocking?	Retry needed?	Best for
Atomic `UPDATE … SET x = x - 1`	Briefly (row lock)	No	Counters, balances — anything expressible in SQL
`SELECT … FOR UPDATE`	Yes, until commit	No	Complex logic between read and write, short transactions
Repeatable Read / Serializable	No (aborts instead)	Yes (40001)	Code that already has a retry loop
Version column (CAS)	No	Yes (0 rows)	Long "think time", web forms, APIs, distributed caches

In the interview

> **30-second answer:** "Two read-modify-write cycles interleave and one write overwrites the other. Fixes: atomic SQL arithmetic; SELECT FOR UPDATE; Repeatable Read, where Postgres aborts the second writer (40001); or a version column where zero updated rows means retry."

- *A "like" counter?* Atomic `UPDATE posts SET likes = likes + 1` (or a likes table and count).
- *A user editing a product page for minutes?* Version column — never hold a DB lock across think time.

- *Does Read Committed prevent lost updates?* Only if the write is one atomic statement; not for app-side read-modify-write.

13.8 Write skew and Serializable isolation

Hospital rule: **at least one doctor on call for the night shift**. Dr Rao and Dr Kim each click "go off duty"; the app checks "is anyone else on call?" and, if so, updates *its own row*. Different rows, no row-lock conflict — yet the rule can break.

Repeatable Read (snapshot isolation) allows it

```
=== Write skew: REPEATABLE READ lets both doctors go off duty ===
  A| BEGIN ISOLATION LEVEL REPEATABLE READ  -> BEGIN
  B| BEGIN ISOLATION LEVEL REPEATABLE READ  -> BEGIN
  A| SELECT count(*) FROM on_call WHERE shift = 'night' AND on_duty  -> 2
  B| SELECT count(*) FROM on_call WHERE shift = 'night' AND on_duty  -> 2
  A| UPDATE on_call SET on_duty = false WHERE doctor = 'dr_rao'  -> UPDATE 1
  B| UPDATE on_call SET on_duty = false WHERE doctor = 'dr_kim'  -> UPDATE 1
  A| COMMIT  -> COMMIT
  B| COMMIT  -> COMMIT
  M| SELECT count(*) FROM on_call WHERE shift = 'night' AND on_duty  -> 0
```

Nobody is on call. Each was correct against its snapshot; run serially, the second would have seen 1 and refused. Same shape: double-booking a room, claiming a username checked by `SELECT` first, overspending a shared budget.

Serializable (SSI) aborts one

```
=== Write skew: SERIALIZABLE aborts one of them ===
  A| BEGIN ISOLATION LEVEL SERIALIZABLE  -> BEGIN
  B| BEGIN ISOLATION LEVEL SERIALIZABLE  -> BEGIN
  A| SELECT count(*) FROM on_call WHERE shift = 'night' AND on_duty  -> 2
  B| SELECT count(*) FROM on_call WHERE shift = 'night' AND on_duty  -> 2
  A| UPDATE on_call SET on_duty = false WHERE doctor = 'dr_rao'  -> UPDATE 1
  B| UPDATE on_call SET on_duty = false WHERE doctor = 'dr_kim'  -> UPDATE 1
  A| COMMIT  -> COMMIT
  B| COMMIT  -> ERROR 40001: could not serialize access due to read/write dependencies among transactions
     DETAIL: Reason code: Canceled on identification as a pivot, during commit attempt.
  M| SELECT count(*) FROM on_call WHERE shift = 'night' AND on_duty  -> 1
```

How **SSI** caught it:

1. Serializable transactions still read snapshots (readers and writers don't block).
2. Each records what it **read** as *SIREAD* "locks" — non-blocking notes such as "B read the night-shift rows".
3. Writing something a concurrent transaction read records a **read-write dependency** (rw-conflict): "A wrote what B read", "B wrote what A read".
4. Such a cycle means no serial order could produce the result; on the dangerous pattern (a "pivot" with a conflict in and out) PostgreSQL aborts one with **40001**. The retried B sees 1 doctor and refuses.

The error can come at any statement, including `COMMIT`, so **every Serializable transaction needs a retry loop**:

```python
for attempt in range(5):
    try:
        with conn.transaction():                    # BEGIN ... COMMIT
            conn.execute("SET TRANSACTION ISOLATION LEVEL SERIALIZABLE")
            go_off_duty(conn, doctor)
        break
    except psycopg.errors.SerializationFailure:     # SQLSTATE 40001
        continue                                    # rerun the whole transaction
```

SSI can also abort harmless transactions (false positives): its bookkeeping is approximate and coarsens from rows to pages or tables when a transaction reads a lot. Keep Serializable transactions short; mark read-only ones `READ ONLY`.

Fix without Serializable: lock what you based the decision on

Both at the default Read Committed:

```
=== Write skew fix without SERIALIZABLE: lock the rows you base the decision on ===
  A| BEGIN  -> BEGIN
  B| BEGIN  -> BEGIN
  A| SELECT doctor FROM on_call WHERE shift = 'night' AND on_duty FOR UPDATE  -> (dr_rao), (dr_kim)
  B| SELECT doctor FROM on_call WHERE shift = 'night' AND on_duty FOR UPDATE
     .. B is WAITING (wait_event=transactionid, blocked by A)
  A| UPDATE on_call SET on_duty = false WHERE doctor = 'dr_rao'  -> UPDATE 1
  A| COMMIT  -> COMMIT
     -> (B resumes) dr_kim
  (B's app sees only one doctor left, so it refuses to go off duty)
  B| ROLLBACK  -> ROLLBACK
  M| SELECT count(*) FROM on_call WHERE shift = 'night' AND on_duty  -> 1
```

After waiting, B's `FOR UPDATE` re-checks the rows and returns only Dr Kim. **Limitation:** `FOR UPDATE` locks only rows that **exist**; for a rule about *absent* rows ("no booking overlaps 10:00–11:00") there is nothing to lock. Then use Serializable; a constraint — `UNIQUE`, or an exclusion constraint forbidding two conflicting rows, `EXCLUDE USING gist (room WITH =, during WITH &&)` (needs `btree_gist` for `=`); or *materialize the conflict* by pre-creating lockable rows (one per room per time slot).

In the interview

30-second answer: "Two transactions read the same data, decide on it, and write different rows — no write conflict, but an invariant breaks. Snapshot isolation allows it. Fixes: Serializable (SSI aborts one with 40001 — retry), FOR UPDATE on the rows the decision depends on, or a unique/exclusion constraint."

- *Why doesn't FOR UPDATE fix double-booking of an empty slot?* The conflicting row doesn't exist yet (a phantom); use a constraint, Serializable, or materialized slot rows.

Self-check: why no conflict at Repeatable Read? → They updated different rows. SIREAD locks? → Non-blocking records of what a Serializable transaction read, used to find rw-dependencies.

13.9 Concurrency control — the big picture

What it is

Concurrency control decides how overlapping transactions may interleave so results stay correct (no §13.4 anomalies, each transaction isolated as its level promises) with as much parallelism as possible. The formal goal is **serializability**: a concurrent execution (a *schedule*) is correct if its effect equals *some* serial order. Two operations **conflict** if they are in different transactions, touch the same item, and at least one writes; a schedule is *conflict-serializable* if its conflict graph (arrow T1 → T2 when an operation of T1 conflicts with and precedes one of T2) has no cycle.

Technique	Core idea	Waits or aborts?	Used by
Pessimistic / lock-based (e.g. 2PL)	Lock before you touch; others wait	Waits (deadlocks possible)	SQL Server, MySQL locking reads, DB2; `FOR UPDATE` everywhere
Optimistic (OCC)	Work freely, validate at commit, abort on conflict	Aborts	App-level version columns, some in-memory and distributed DBs
Timestamp ordering	Operations must respect transaction timestamp order	Aborts	Mostly textbook; ideas used in distributed DBs
MVCC	Several versions; readers read an old version, never block writers	Readers never wait; writers wait for writers	PostgreSQL, Oracle, MySQL InnoDB, SQL Server snapshot modes

They combine in practice: PostgreSQL = MVCC for reads + row locks for writes + SSI (optimistic validation) for Serializable.

Optimistic vs pessimistic

Optimistic concurrency control (OCC) assumes conflicts are rare: (1) **read/execute** in a private workspace without locks; (2) **validate** at commit — did a transaction that committed meanwhile change anything it read? (3) **write**, or **abort and retry** (the version-column pattern, §13.7 Fix 4). **Pessimistic (PCC)** assumes conflicts are likely: **lock** on first access, make conflicting transactions **wait**, **release** at commit or rollback (§13.7 Fix 2).

Aspect	Optimistic (OCC)	Pessimistic (PCC)
Assumes	Conflicts are rare	Conflicts are common
Locks	None during execution	Taken on access, held to the end
Cost of a conflict	Abort and redo the work	Waiting
Overhead with no conflicts	Very low	Lock management on every access
Deadlocks	None (no waits)	Possible — need detection
Fits	Read-heavy, low contention, distributed systems, long user think time	Write-heavy, hot rows (a popular product's stock), critical updates where retrying is expensive — e.g. a flash sale on one product

Real world: **Git** and **Google Docs** are optimistic (conflicts surface at merge/sync); **banking** and **ticketing** lock hot rows so a seat is never sold twice; **e-commerce** locks (or atomically decrements) the stock row.

In the interview

> **30-second answer:** "Keep interleaved transactions serializable. Pessimistic: lock and wait (high contention; deadlocks possible). Optimistic: validate at commit and abort on conflict (rare conflicts). Timestamp ordering: follow start order. MVCC: versions so readers never block writers. Postgres combines MVCC, row locks and SSI."

- *Why does OCC have no deadlocks?* → Transactions never wait for each other's locks.

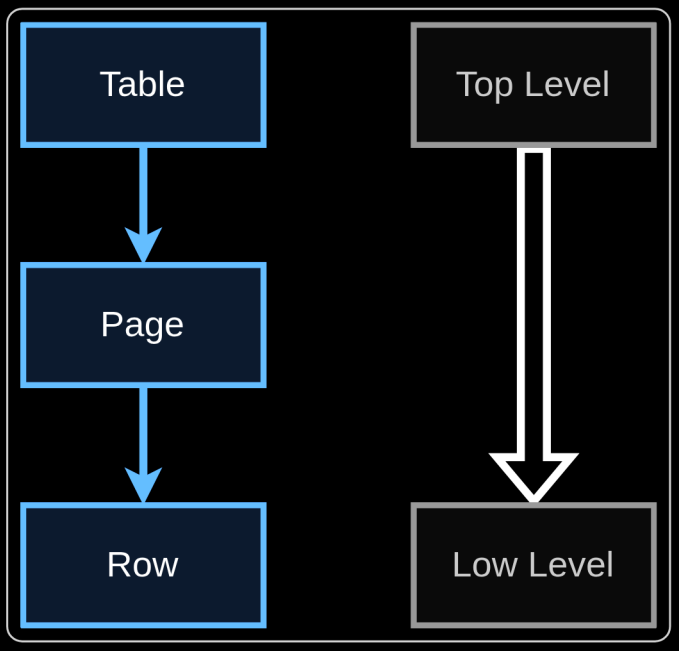
13.10 Lock-based concurrency control

What it is

A transaction must obtain a **lock** before using data; a conflicting request waits until release — so others can't read or overwrite in-progress changes.

Lock	Purpose	Others may…
Shared (S)	Reading	also take S; may **not** write
Exclusive (X)	Writing	do nothing with the item until released
Intention Shared (IS)	On a table: "I will take S locks on some rows inside"	—
Intention Exclusive (IX)	On a table: "I will take X locks on some rows inside"	—

Lock granularity



Granularity	Advantage	Disadvantage	Example
Row	Maximum concurrency	Many locks to manage (memory, CPU)	Update one customer's row
Page	Middle ground	Locks unrelated rows sharing the page	Older SQL Server, DB2
Table	Simple, very low overhead	Blocks everyone else	Batch update of all `orders`

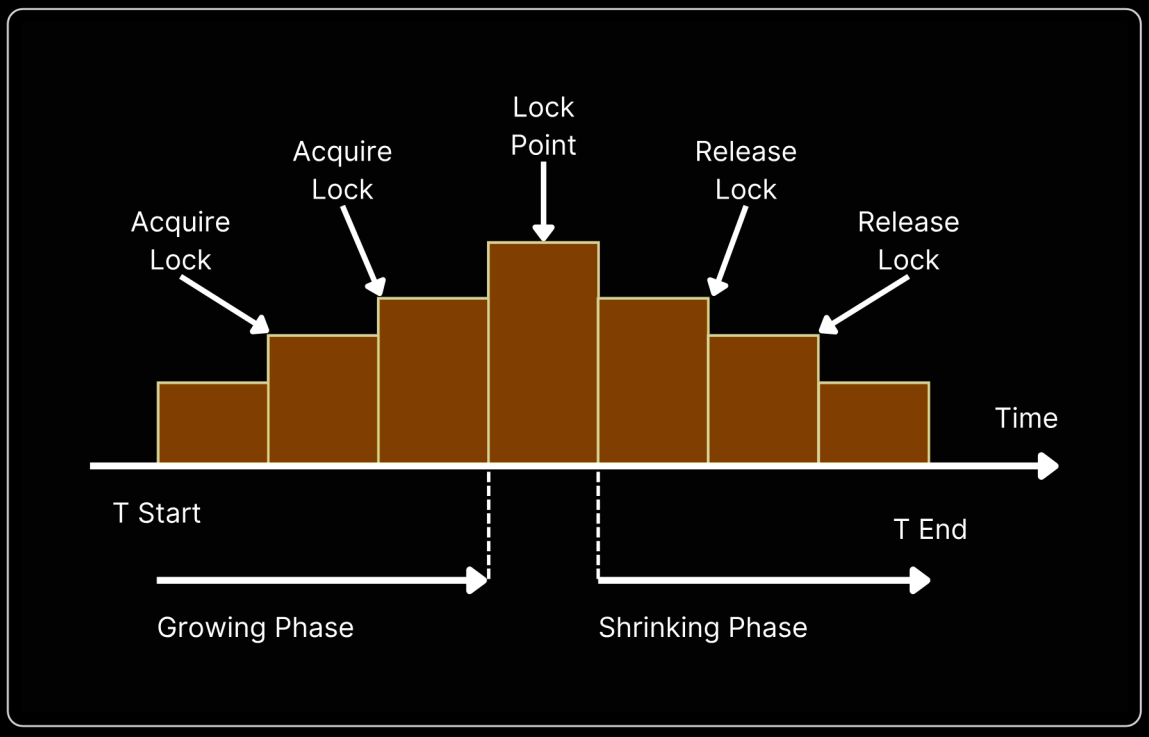
Some engines **escalate** thousands of row locks into one table lock (SQL Server). PostgreSQL never escalates because row locks live in the rows.

Why intention locks: T1 holds X on row 5 of `employees`; T2 wants S on the **whole table**. Without extra information the manager would scan every row lock. With hierarchical locking: (1) T1 places **IX** on the table, then X on row 5; (2) T2's table S conflicts with IX → T2 waits, no row scan; (3) T3 wanting X on row 9 places IX (compatible with IX) and runs in parallel.

Held → / Requested ↓	IS	IX	S	X
IS	yes	yes	yes	no
IX	yes	yes	no	no
S	yes	no	yes	no
X	no	no	no	no

Two-phase locking (2PL)

2PL guarantees conflict-serializability: (1) **growing (expanding) phase** — acquire, never release; (2) **shrinking phase** — after the first release, never acquire. The peak is the **lock point**; transactions serialize in lock-point order.



Variant	Rule	Why
Basic 2PL	grow, then shrink	Serializable, but one may read data from another that later aborts (*cascading aborts*)
Strict 2PL	hold all **X** locks until commit/abort	Nobody reads uncommitted data; what most lock-based databases do
Rigorous 2PL	hold **all** locks until commit/abort	Simplest to reason about; serial order = commit order
Conservative 2PL	take every lock before starting	No deadlocks, but the data set must be known in advance

Pros: guaranteed serializability. *Cons:* **deadlocks** (§13.11) and lower throughput under contention — a long reader blocks writers and vice versa. MySQL and SQL Server implement Serializable with (strict) 2PL; it suits strong-isolation domains such as finance.

Predicate locking and index-range locking

2PL on existing rows can't stop **phantoms**: a new row that *would* match didn't exist when you locked. **Predicate locking** locks the **condition** — every row, existing or future, satisfying `WHERE …`. `seats` holds 10 and 30; T runs `UPDATE seats SET held = true WHERE seat_no BETWEEN 10 AND 30`:

Seat	Exists now?	Locked by predicate lock?
10	yes	yes
20	no	yes — an insert of seat 20 must wait
30	yes	yes

It prevents phantoms and write skew, but checking every request against every active predicate is expensive and over-locks. **Index-range locking** (*next-key / gap* locking) approximates the predicate with index ranges: it locks matching entries *and the gaps between them*, so inserting seat 20 waits (locking only keys 10 and 30 would let it in). Cheaper because it reuses the index, sometimes locking a slightly wider range; without a suitable index the engine locks far more (up to the whole table). InnoDB's next-key locks at Repeatable Read work this way.

Aspect	2PL on rows	Predicate locking	Index-range (next-key) locking
What is locked	Rows actually touched	Every row matching the condition, even future ones	Matching index entries plus gaps
Phantoms prevented?	No	Yes	Yes (needs an index on the condition)
Seen in	All lock-based engines	Theory; PostgreSQL SSI's SIREAD locks are a non-blocking cousin	MySQL InnoDB, SQL Server key-range locks

PostgreSQL's locks in practice

Eight **table-level lock modes** (taken automatically by statements) and four **row-level** modes. `code/04_lock_modes.sql`:

```
BEGIN;
SELECT * FROM accounts WHERE id = 1;                          -- plain read
UPDATE products SET stock = stock - 1 WHERE id = 1;           -- write
SELECT * FROM on_call WHERE doctor = 'dr_rao' FOR SHARE;      -- shared row lock
LOCK TABLE jobs IN SHARE MODE;                                -- explicit table lock
SELECT c.relname, l.mode, l.granted
FROM pg_locks l JOIN pg_class c ON c.oid = l.relation
WHERE l.pid = pg_backend_pid() AND c.relnamespace = 'public'::regnamespace
ORDER BY c.relname, l.mode;
COMMIT;

SELECT pg_try_advisory_lock(42) AS got_it_first_time;
SELECT count(*) AS advisory_locks_held FROM pg_locks WHERE locktype = 'advisory' AND pid = pg_backend_pid();
SELECT pg_advisory_unlock(42) AS released;
```

Output (row echoes trimmed; the three advisory-lock results condensed onto one line):

```
    relname    |       mode       | granted
---------------+------------------+---------
 accounts      | AccessShareLock  | t
 accounts_pkey | AccessShareLock  | t
 jobs          | ShareLock        | t
 on_call       | RowShareLock     | t
 on_call_pkey  | RowShareLock     | t
 products      | RowExclusiveLock | t
 products_pkey | RowExclusiveLock | t
(7 rows)
...
 got_it_first_time → t     advisory_locks_held → 1     released → t
```

These table modes act as intention locks: `RowExclusiveLock` (from `UPDATE/INSERT/DELETE`) means "changing rows in here"; it doesn't conflict with other writers but does with `SHARE` and stronger:

Common statement	Table lock taken	Blocks
`SELECT`	ACCESS SHARE	only ACCESS EXCLUSIVE (`DROP` , most `ALTER TABLE`)
`SELECT … FOR UPDATE/SHARE`	ROW SHARE	EXCLUSIVE, ACCESS EXCLUSIVE
`INSERT/UPDATE/DELETE`	ROW EXCLUSIVE	SHARE and stronger
`CREATE INDEX`	SHARE	writers
`CREATE INDEX CONCURRENTLY` , `VACUUM`	SHARE UPDATE EXCLUSIVE	other schema changes / vacuums
most `ALTER TABLE` , `DROP` , `TRUNCATE` , `VACUUM FULL`	ACCESS EXCLUSIVE	everything, even `SELECT`

Row-level lock modes, strongest first: `FOR UPDATE` (`DELETE` or an update of key columns), `FOR NO KEY UPDATE` (a normal `UPDATE`), `FOR SHARE` (read, block changes), `FOR KEY SHARE` (foreign-key checks; blocks only key changes/deletes). **Advisory locks** (`pg_advisory_lock(n)`) lock a number only your application gives meaning to — e.g. "only one nightly job may run".

Common mistakes: an `ALTER TABLE` on a busy table waits for ACCESS EXCLUSIVE — and **every** later query queues behind it (use `SET lock_timeout = '2s'` for migrations); holding locks during slow work (HTTP calls, user input); relying on the absence of a row without a unique constraint or Serializable (phantoms).

In the interview

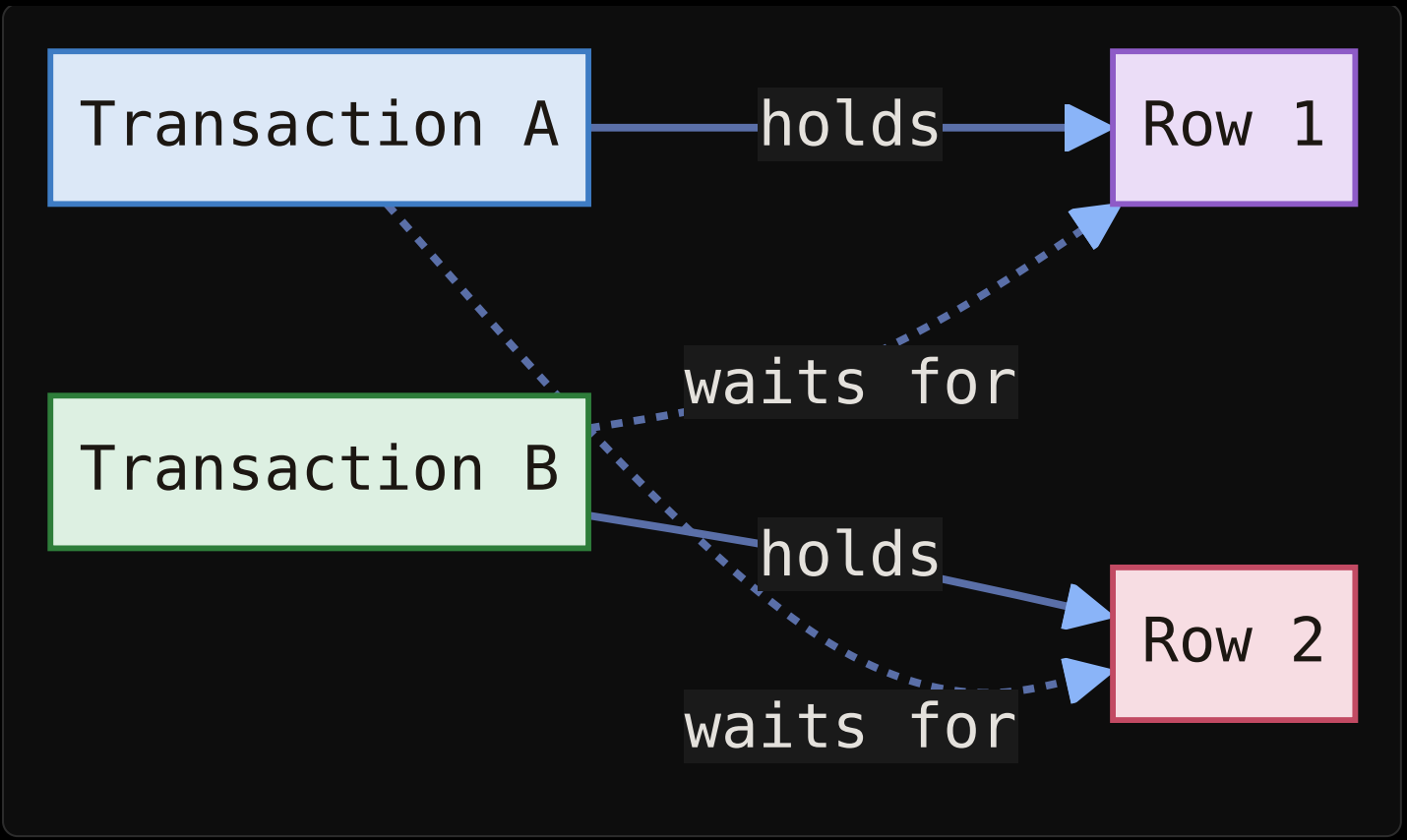
30-second answer: "S locks for reads, X for writes; only S is compatible with S. Finer granularity means more concurrency and more overhead; intention locks let the manager check table-versus-row conflicts without scanning rows. 2PL (grow then shrink) guarantees serializability, strict 2PL keeps write locks to commit, and next-key locks stop phantoms."

- *Why does strict 2PL hold X locks until commit?* So nobody reads uncommitted data and cascading aborts can't happen.
- *How does Postgres avoid lock-table explosion for row locks?* Row locks are stored in the tuple header (`xmax` + infomask status bits); waiters wait on the holder's transaction ID.

13.11 Deadlocks

What it is

A **deadlock** is a cycle of waiting: T1 holds resource 1 and waits for 2, T2 holds 2 and waits for 1; neither proceeds unless something intervenes.



Strategy	How	Trade-off
Detection	*Wait-for graph* (edge T1 → T2 when T1 waits for T2); a cycle is a deadlock; abort a victim	Some work lost; PostgreSQL, MySQL, SQL Server, Oracle
Timeout	Abort any transaction waiting too long	Simple, but may abort non-deadlocked waiters or react slowly (`lock_timeout`, InnoDB `innodb_lock_wait_timeout`)
Prevention by ordering	Always lock in one global order (e.g. lower `id` first)	Needs discipline in application code — the most practical fix
Wait-die / wound-wait	By start timestamp: *wait-die* — an older requester waits, a younger one aborts ("dies"); *wound-wait* — an older requester aborts ("wounds") the younger holder, a younger one waits	No cycles; more aborts; some distributed databases
Conservative 2PL	Take all locks up front	Must know everything in advance

Seeing PostgreSQL detect one

A moves 10 from account 1 to 2; B from 2 to 1; each locks its "from" row first:

```
=== Deadlock: PostgreSQL detects it and aborts one transaction ===
  A| SET deadlock_timeout = '10s'  -> SET
  B| SET deadlock_timeout = '200ms'  -> SET
  A| BEGIN  -> BEGIN
  B| BEGIN  -> BEGIN
  A| UPDATE accounts SET balance = balance - 10 WHERE id = 1  -> UPDATE 1
  B| UPDATE accounts SET balance = balance - 10 WHERE id = 2  -> UPDATE 1
  A| UPDATE accounts SET balance = balance + 10 WHERE id = 2
     .. A is WAITING (wait_event=transactionid, blocked by B)
  B| UPDATE accounts SET balance = balance + 10 WHERE id = 1  -> ERROR 40P01: deadlock detected
     DETAIL: Process <B> waits for ShareLock on transaction <xid>; blocked by process <A>.
             Process <A> waits for ShareLock on transaction <xid>; blocked by process <B>.
  B| ROLLBACK  -> ROLLBACK
     -> (A resumes) UPDATE 1
  A| COMMIT  -> COMMIT
  M| SELECT id, balance FROM accounts WHERE id IN (1, 2) ORDER BY id  -> (1, 490.00), (2, 310.00)
```

1. A waiting backend (the server process serving one connection) first sleeps `deadlock_timeout` (default **1 s**) — most waits end by themselves, and the check is costly.
2. Then it inspects the wait-for graph. B's timer fired first (200 ms vs 10 s, to make the victim predictable); it found A → B → A and aborted **B** with **40P01**.
3. B's locks are released, A completes; the application retries B.

The fix here is ordering: lock the lower id first (`SELECT … FROM accounts WHERE id IN (1, 2) ORDER BY id FOR UPDATE`), so both transfers queue for row 1 and no cycle forms. **Common mistakes:** not retrying on 40P01 (or 40001) — both mean "rerun the whole transaction"; locking rows in input order (an unsorted `for item in cart` loop) — sort by primary key first.

In the interview

30-second answer: "A cycle in the wait-for graph. Databases detect it (Postgres after `deadlock_timeout`, 1 s by default) and abort a victim, which the app retries. Prevent by locking in a consistent order and keeping transactions short; textbook alternatives are wait-die and wound-wait."

13.12 Row locks for work queues: `FOR UPDATE`, `NOWAIT`, `SKIP LOCKED`

A **job table** (emails to send, images to resize) served by several workers must give each worker jobs **no other worker** holds. With plain `FOR UPDATE` all workers lock the *same* first rows and queue behind each other.

Clause	If the row is locked by someone else…
`FOR UPDATE`	wait until they finish
`FOR UPDATE NOWAIT`	fail immediately with error 55P03
`FOR UPDATE SKIP LOCKED`	silently skip that row and take the next unlocked one

`SKIP LOCKED` exists in PostgreSQL (9.5+), MySQL (8.0+) and Oracle; SQL Server's `READPAST` hint does the same.

```
=== Job queue: FOR UPDATE SKIP LOCKED (and NOWAIT) ===
  A| BEGIN  -> BEGIN
  A| SELECT id, payload FROM jobs WHERE status = 'queued' ORDER BY id LIMIT 2 FOR UPDATE SKIP LOCKED  -> (1, email
#1), (2, email #2)
  B| BEGIN  -> BEGIN
  B| SELECT id, payload FROM jobs WHERE status = 'queued' ORDER BY id LIMIT 2 FOR UPDATE SKIP LOCKED  -> (3, email
#3), (4, email #4)
  B| SELECT id FROM jobs WHERE id = 1 FOR UPDATE NOWAIT  -> ERROR 55P03: could not obtain lock on row in relation
"jobs"
  B| ROLLBACK  -> ROLLBACK
  B| UPDATE jobs SET status = 'running', claimed_by = 'B' WHERE id IN (SELECT id FROM jobs WHERE status = 'queued'
ORDER BY id LIMIT 2 FOR UPDATE SKIP LOCKED) RETURNING id  -> (3), (4)
  A| UPDATE jobs SET status = 'done', claimed_by = 'A' WHERE id IN (1, 2)  -> UPDATE 2
  A| COMMIT  -> COMMIT
  M| SELECT id, status, claimed_by FROM jobs ORDER BY id  -> (1, done, A), (2, done, A), (3, running, B), (4,
running, B), (5, queued, None), (6, queued, None)
```

B, running the identical query, got jobs 3–4 **without waiting**; `NOWAIT` on job 1 failed at once. The usual form **claims and marks in one statement**:

```sql
UPDATE jobs SET status = 'running', claimed_by = $1
WHERE id IN (SELECT id FROM jobs
             WHERE status = 'queued'
             ORDER BY id
             LIMIT 10
             FOR UPDATE SKIP LOCKED)
RETURNING id, payload;
```

Two designs: (a) keep the transaction open while processing — a crashed worker's rollback releases its jobs; (b) mark `running` and commit at once, plus a `claimed_at` column and a sweeper job re-queuing stuck jobs. A partial index `CREATE INDEX ON jobs (id) WHERE status = 'queued'` keeps the claim fast (chapter 12).

When not to use it: `SKIP LOCKED` deliberately hides locked rows — never for reports or business-rule checks. For very high throughput, fan-out (one message to many consumers) or replay, use a message broker such as Kafka, RabbitMQ or SQS (System Design Fundamentals); a Postgres queue shines when jobs must be created in the same transaction as the business data. **Real-world examples:** GoodJob (Ruby), Oban (Elixir), graphile-worker (Node), Procrastinate (Python); ticketing systems handing concurrent buyers different free seats.

13.13 Timestamp-based concurrency control

What it is

Timestamp ordering (TO) gives each transaction a unique, increasing **timestamp** TS(T) at start (clock or counter). The result must equal running transactions **in timestamp order**; instead of waiting for locks, a transaction that would break the order is **aborted and restarted** with a new timestamp — always serializable.

Symbol	Kept for	Meaning
TS(T)	each transaction	its start timestamp — its place in the serial order
RTS(X)	each data item X	largest timestamp of any transaction that **read** X
WTS(X)	each data item X	largest timestamp of any transaction that **wrote** X

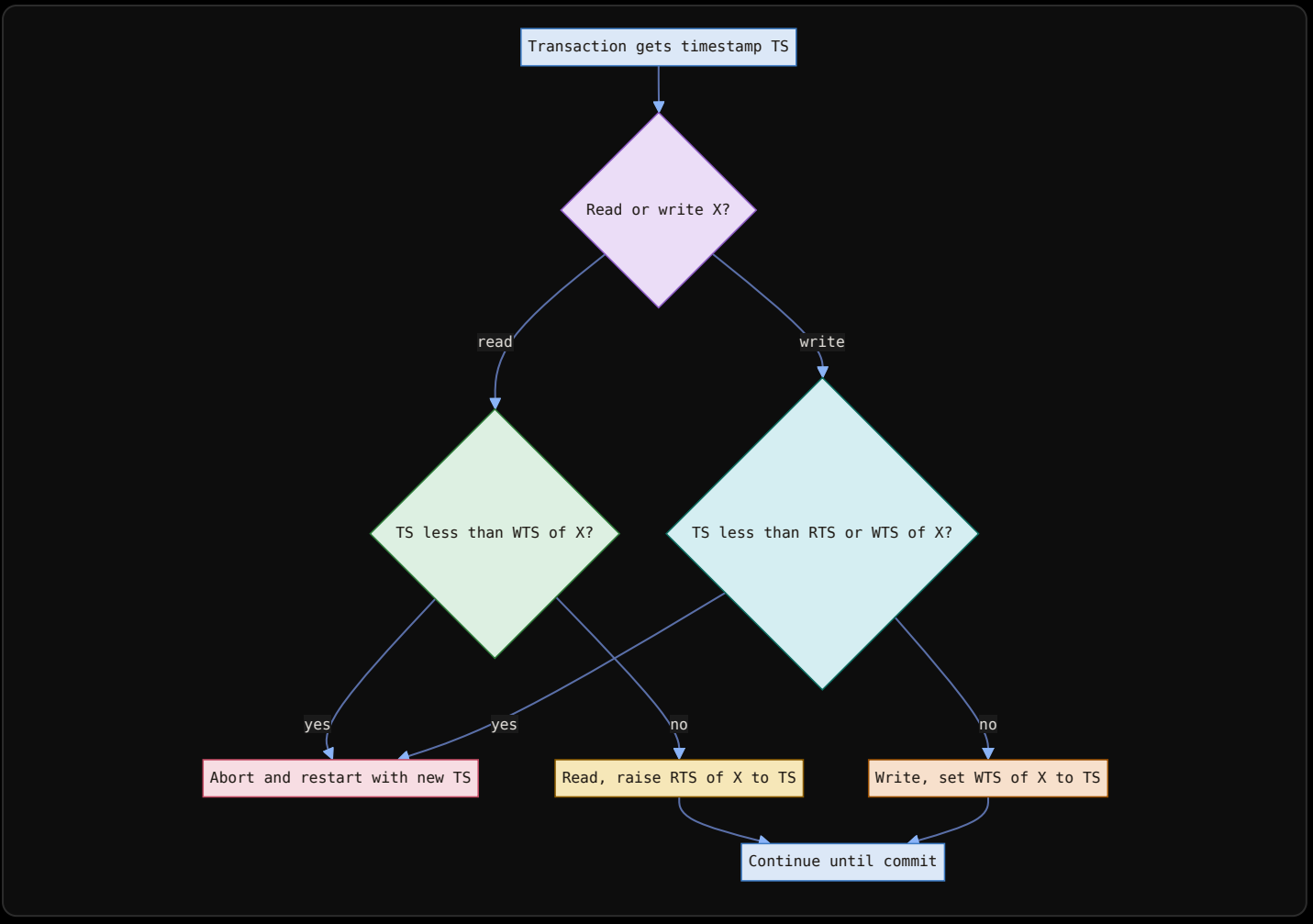
The rules

T reads X: if TS(T) < WTS(X), a *younger* transaction already overwrote X (T would read "from its future") → **abort T**. Otherwise read and set RTS(X) = max(RTS(X), TS(T)).

T writes X:

1. TS(T) < RTS(X): a younger transaction already read the old value → **abort T**.

2. TS(T) < WTS(X): a younger transaction already wrote X, so T's write is outdated → **abort T** — or, under the **Thomas write rule**, **skip** the obsolete write (timestamp order would have overwritten it anyway).
3. Otherwise write and set WTS(X) = TS(T).



Worked example — booking the last seats

Item **S** = seats left (RTS = WTS = 0). Priya's booking gets TS = 5; Omar's, a moment later, TS = 7.

Step	Operation	Check	Result	RTS(S)	WTS(S)
1	T5 reads S	5 ≥ WTS 0	allowed	5	0
2	T7 reads S	7 ≥ WTS 0	allowed	7	0
3	T7 writes S (books a seat)	7 ≥ RTS 7 and ≥ WTS 0	allowed	7	7
4	T5 writes S (books a seat)	5 < RTS 7	**T5 aborted**	7	7
5	T5 restarts as T9, reads S	9 ≥ WTS 7	allowed, sees Omar's booking	9	7

Priya's first attempt relied on a count a "later" transaction had already read and changed, so it was rolled back instead of double-selling.

Advantages	Disadvantages
No deadlocks — nobody waits	**Many rollbacks** under contention; work thrown away
Serializability by construction	**Starvation** — a long transaction may restart again and again as younger ones touch its data
Simple conflict rule: compare two numbers	**Overhead** — every read updates RTS (reads become metadata writes); extra care to avoid reading uncommitted values

Where used: pure TO is mostly textbook. Its descendant **multiversion timestamp ordering** (timestamped versions; each read picks the right one) underlies many MVCC systems; distributed databases rely on timestamps — Google Spanner's TrueTime (GPS- and atomic-clock-backed time with a known error bound), CockroachDB's hybrid logical clocks (physical time plus a counter) with MVCC.

In the interview

> **30-second answer:** "Transactions must behave as if run in start-timestamp order. Each item keeps its largest read and write timestamps; a read is rejected if a younger transaction already wrote the item, a write if a younger one already read or wrote it, and the loser restarts with a new timestamp. No deadlocks, but many aborts and possible starvation; the Thomas write rule skips obsolete writes."

Self-check: TS 4 wants to write X with RTS(X) = 6? → Abort (a younger transaction read X). Thomas write rule? → An outdated write (TS < WTS but ≥ RTS) is ignored instead of aborting.

13.14 Multiversion concurrency control (MVCC)

What it is

MVCC keeps **several versions** of each row. A write creates a **new version** and marks the old one replaced; each transaction reads a **snapshot** — the versions committed as of some moment. **Readers and writers never block each other** (writers still block writers on the same row), and everyone sees a consistent picture.

How it works

1. **Versions carry metadata:** the transaction that **created** it and the one that **deleted/replaced** it (empty if current).
2. **Snapshots:** when a transaction (Repeatable Read) or statement (Read Committed) starts, the database notes which transactions had committed.
3. **Visibility:** a version is visible if its creator committed before the snapshot and its deleter is empty, aborted, or uncommitted as of the snapshot.
4. **Writes** create versions; concurrent writers of a row serialize on a row lock (§13.7); at Repeatable Read/Serializable a concurrent committed change raises a serialization error.
5. **Cleanup:** versions no snapshot can see are garbage (PostgreSQL's `VACUUM`, InnoDB's purge thread).

Worked example. Account A = 500 (created by txn 1), B = 300; txn 2 adds 200 to A; txn 3 subtracts 100:

After	Versions of account A (created_by, deleted_by, balance)	Account B
txn 1	(1, –, **500**)	(1, –, 300)
txn 2	(1, **2**, 500) · (2, –, **700**)	unchanged
txn 3	(1, 2, 500) · (2, **3**, 700) · (3, –, **600**)	unchanged

A snapshot from before txn 2's commit sees 500, one between txn 2 and 3 sees 700, new ones 600 — simultaneously, without locks.

MVCC in PostgreSQL: `xmin` and `xmax` for real

Every row version carries hidden system columns `xmin` (inserting transaction ID) and `xmax` (deleting/locking transaction ID, 0 if none); `ctid` is its physical address (page, slot). `code/03_mvcc_xmin_xmax.sql` repeats the worked example (500 → +200 → −100) and looks inside the first 8 KB page with `pageinspect`:

```
CREATE EXTENSION IF NOT EXISTS pageinspect;
CREATE TABLE wallet (id int PRIMARY KEY, balance int);
INSERT INTO wallet VALUES (1, 500), (2, 300);
SELECT ctid, xmin, xmax, id, balance FROM wallet ORDER BY id;

UPDATE wallet SET balance = balance + 200 WHERE id = 1;
UPDATE wallet SET balance = balance - 100 WHERE id = 1;
SELECT ctid, xmin, xmax, id, balance FROM wallet ORDER BY id;

SELECT lp AS slot, t_xmin, t_xmax, t_ctid
FROM heap_page_items(get_raw_page('wallet', 0)) ORDER BY lp;

BEGIN;
UPDATE wallet SET balance = 0 WHERE id = 2;
SELECT txid_current() AS my_txid;
ROLLBACK;
SELECT ctid, xmin, xmax, id, balance FROM wallet WHERE id = 2;

VACUUM wallet;
SELECT lp AS slot, lp_flags, t_xmin, t_xmax, t_ctid
FROM heap_page_items(get_raw_page('wallet', 0)) ORDER BY lp;
```

Output (transaction IDs differ on every run; command tags trimmed):

```
 ctid  | xmin | xmax | id | balance
-------+------+------+----+---------
 (0,1) |  802 |    0 |  1 |     500
 (0,2) |  802 |    0 |  2 |     300
(2 rows)

UPDATE 1
UPDATE 1
 ctid  | xmin | xmax | id | balance
-------+------+------+----+---------
 (0,4) |  804 |    0 |  1 |     600
 (0,2) |  802 |    0 |  2 |     300
(2 rows)

 slot | t_xmin | t_xmax | t_ctid
------+--------+--------+--------
    1 |    802 |    803 | (0,3)
    2 |    802 |      0 | (0,2)
    3 |    803 |    804 | (0,4)
    4 |    804 |      0 | (0,4)
(4 rows)

BEGIN
UPDATE 1
 my_txid
---------
     805
(1 row)

ROLLBACK
 ctid  | xmin | xmax | id | balance
-------+------+------+----+---------
 (0,2) |  802 |  805 |  2 |     300
(1 row)

VACUUM
 slot | lp_flags | t_xmin | t_xmax | t_ctid
------+----------+--------+--------+--------
    1 |        2 |        |        |
    2 |        1 |    802 |    805 | (0,5)
    3 |        0 |        |        |
    4 |        1 |    804 |      0 | (0,4)
    5 |        0 |        |        |
(5 rows)
```

1. Transaction 802 inserted both rows (`xmax = 0`: not replaced).
2. After the updates, row 1's current version sits at a **new address** `(0,4)` with `xmin = 804`: an update = "insert a new version + mark the old one".
3. The raw page still holds **all three versions** of row 1 — slot 1 (500, 802 → 803), slot 3 (700, 803 → 804), slot 4 (600, current); each old version's `t_ctid` points to its successor.
4. The rolled-back update left `xmax = 805` on row 2, yet 300 is visible: 805 is marked **aborted** in the commit log (`pg_xact`), so its `xmax` is ignored — PostgreSQL rolls back instantly by marking, not undoing.
5. `VACUUM` removed dead versions: slots 3 and 5 are unused (`lp_flags = 0`) and slot 1 became a **redirect** (`lp_flags = 2`) to slot 4 — a *HOT* (heap-only tuple) chain, so the index can keep pointing at slot 1.

Two sessions — what *another* transaction sees during an uncommitted update:

```
=== MVCC: xmin / xmax seen from two sessions ===
  A| SELECT xmin, xmax, balance FROM accounts WHERE id = 1  -> (1697, 0, 500.00)
  A| BEGIN  -> BEGIN
  A| UPDATE accounts SET balance = 450 WHERE id = 1  -> UPDATE 1
  A| SELECT txid_current()  -> 1698
  A| SELECT xmin, xmax, balance FROM accounts WHERE id = 1  -> (1698, 0, 450.00)
  B| SELECT xmin, xmax, balance FROM accounts WHERE id = 1  -> (1697, 1698, 500.00)
  A| COMMIT  -> COMMIT
  B| SELECT xmin, xmax, balance FROM accounts WHERE id = 1  -> (1698, 0, 450.00)
```

While 1698 is open, **two versions** exist: A sees its new one (450); B sees the **old** one (500) with `xmax = 1698` — "being replaced by an uncommitted transaction, so still visible to me". After the commit B sees 450. No one waited.

Trade-offs

Advantages	Costs
Readers and writers don't block each other	Extra storage for old versions; tables and indexes **bloat** (dead-version waste)
Consistent snapshots for reports and backups (`pg_dump` uses one)	Needs garbage collection (autovacuum — the background process running `VACUUM` — must keep up)
Cheap rollback in PostgreSQL (mark aborted)	Long-running transactions keep old versions alive for everyone
Basis for snapshot isolation and SSI	Snapshot isolation alone still allows write skew (§13.8)

Two designs. PostgreSQL keeps every version **in the table** (vacuum later). InnoDB and Oracle update **in place** and rebuild older versions from an **undo log** — less bloat, more work for long readers ("snapshot too old" in Oracle).

Common mistakes: sessions "idle in transaction" for hours — vacuum can't remove versions newer than their snapshot, so tables bloat (set `idle_in_transaction_session_timeout`); disabling autovacuum — besides bloat, the 32-bit transaction IDs must be "frozen" (marked visible to everyone) before they wrap around (~2 billion transactions), or the server stops accepting writes; thinking MVCC means "no locks" — same-row writes still lock and wait. It is what lets shoppers browse while prices change and `pg_dump` take a coherent backup of a running database.

In the interview

30-second answer: "Keep multiple row versions tagged with the writing transaction; readers pick the version visible to their snapshot, so readers and writers don't block each other. In Postgres an update inserts a tuple and sets xmax on the old one; VACUUM removes dead tuples. Costs: bloat, garbage collection, and long transactions hurt."

- *Non-zero `xmax` on a visible row?* Its deleter/updater hasn't committed or aborted — or it's only a row lock.

- *Why did the updated row get a new `ctid`?* An UPDATE writes a new tuple version elsewhere in the page.

13.15 Putting it together

1. Keep transactions **short** — never open across user think time or slow network calls.
2. Prefer **atomic statements** (`SET n = n - 1 WHERE n > 0`) over read-then-write in code; otherwise `FOR UPDATE`, or a **version column** for long gaps.
3. Declare every invariant you can as a **constraint** (unique, check, foreign key, exclusion); for cross-row rules use **Serializable + retry** or lock the rows the decision depends on.
4. **Retry** the whole transaction on 40001 and 40P01, with backoff (waiting longer before each attempt) and a limit.
5. Lock rows in a **consistent order**; set `lock_timeout` for migrations and `idle_in_transaction_session_timeout` to protect vacuum.
6. For consistent multi-query reports use `REPEATABLE READ READ ONLY`.

Deeper theory: DDIA ch. 7 notes. Thread-level locking in application code: LLD (low-level design) handbook concurrency concepts and concurrency problems. Distributed transactions and replication: chapter 15; transactions from Python/Java: chapter 14; one-page summary: §20.12.

Chapter TL;DR

- A **transaction** is all-or-nothing; write it with `BEGIN … COMMIT/ROLLBACK`, using savepoints for partial undo.
- **ACID:** atomicity (logs/shadow copies), consistency (constraints + code), isolation (locks/MVCC/SSI), durability (WAL + fsync + checkpoints + replication).
- **Anomalies:** dirty read, dirty write, non-repeatable read/read skew, phantom, lost update, write skew. PostgreSQL never allows dirty reads/writes; Read Committed allows the next four; Repeatable Read (snapshot isolation) stops all but write skew; Serializable (SSI) stops everything by aborting with 40001.
- **Lost updates:** prefer atomic SQL updates; otherwise `FOR UPDATE`, Repeatable Read + retry, or a version column (CAS) for long think times.
- **Concurrency control:** pessimistic (S/X/intention locks, 2PL, predicate/next-key locks) vs optimistic (validate at commit) vs timestamp ordering vs **MVCC** (PostgreSQL's `xmin`/`xmax`, cleaned by VACUUM).
- **Deadlocks** are detected (40P01) and prevented by consistent lock ordering; **retry** 40001/40P01.
- `FOR UPDATE SKIP LOCKED` turns a table into a safe multi-worker job queue.

Next: Chapter 14 — Advanced SQL and using SQL from code, which runs these transactions from Python and Java.

Chapter 14 — Advanced SQL and Using SQL from Code

Real systems go beyond queries typed into `psql`: the database runs **code of its own** (functions, procedures, triggers), stores **semi-structured data** whose shape varies by row (JSON, arrays, searchable text), protects data **per user and per row**, and is driven by **application programs** in Python, Java and other languages. This chapter covers those "last mile" skills.

What you'll learn: functions and procedures in SQL and PL/pgSQL, and when logic belongs in the database · triggers (`updated_at`, validation, a generic audit log, statement-level) · JSON/JSONB and arrays: store, query, update, index · full-text search (`tsvector` / `tsquery`, ranking, highlighting) · pivot/unpivot (`FILTER`, `CASE`, `crosstab`, `LATERAL`) · roles, `GRANT` / `REVOKE`, column privileges, row-level security · SQL injection, seen succeeding then fixed with parameters · Python (`psycopg`, `sqlite3`) and Java (JDBC): parameters, transactions, batches, streaming · connection pooling, ORMs vs SQL.

How to run the examples. Everything in `code/` runs on the book's PostgreSQL 14 server (port 5544). The shared sample database from Chapter 1 — an office-gear company with `departments`, `employees`, `customers`, `products`, `orders`, `order_items` — loads first from `code/00_schema.sql`; each numbered file runs in a fresh database.

```
bash tools_run_sql.sh sql '14-*'                                    # all SQL files of this chapter
.venv-books/bin/python sql/chapters/14-advanced-sql-and-using-sql-from-code/code/psycopg_demo.py
python3 sql/chapters/14-advanced-sql-and-using-sql-from-code/code/sqlite3_demo.py
java --enable-native-access=ALL-UNNAMED -cp '.java-books/lib/*' \
     sql/chapters/14-advanced-sql-and-using-sql-from-code/code/JdbcDemo.java
```

14.1 User-defined functions

What it is

A **user-defined function** (UDF) is named logic stored *inside* the database that takes arguments and returns a value (or a set of rows), called from SQL like `upper()` or `round()`. PostgreSQL bodies can be written in several **procedural languages**:

Language	What the body looks like	Use it for
`sql`	One or more SQL statements; the last one's result is returned	Short calculations and reusable queries; the planner can often *inline* them (paste the body into the calling query)
`plpgsql`	PL/pgSQL: SQL plus variables, `IF`, loops, error handling	Branches, loops, several steps
`plpython3u`, `plperl`, …	Other languages (extensions, often "untrusted" = superuser-only)	Rare; needs the extension

Why it matters: one definition of a rule ("line total = qty × price × (1 − discount)") used by every report and app; fewer network round trips for multi-step operations; set-returning functions can be joined like tables.

How it works

1. `CREATE FUNCTION name(args) RETURNS type LANGUAGE … AS $$ body $$` stores it in the catalog (`pg_proc`).
2. `$$ … $$` is **dollar quoting**: a string literal whose inner single quotes need no escaping.
3. Arguments may have a `DEFAULT`; callers pass by position or by name (`p_limit => 1`).
4. A **volatility** label tells the planner how the function behaves, affecting optimisation and index use:

Label	Promise	Example
`IMMUTABLE`	Same inputs → same result forever; reads no tables	`line_total(2, 10, 0)`
`STABLE`	Same result within one statement; may read tables	"tier of customer 3"
`VOLATILE` (default)	Anything may change; may modify data	`random()`, a function that inserts rows

Only `IMMUTABLE` functions can appear in an index expression; marking a table-reading function `IMMUTABLE` gives wrong, cached-looking results.

Example — SQL and PL/pgSQL functions

From `code/01_functions.sql`:

```
-- (a) A plain SQL function, SQL-standard body (PostgreSQL 14+)
CREATE FUNCTION line_total(qty integer, price numeric, disc numeric DEFAULT 0)
RETURNS numeric
LANGUAGE sql IMMUTABLE
RETURN round(qty * price * (1 - disc), 2);

-- (b) PL/pgSQL: variables, IF, SELECT ... INTO, RAISE
CREATE FUNCTION customer_tier(p_customer_id integer)
RETURNS text
LANGUAGE plpgsql STABLE
AS $$
DECLARE
    v_spent numeric;
BEGIN
    IF NOT EXISTS (SELECT 1 FROM customers WHERE customer_id = p_customer_id) THEN
        RAISE EXCEPTION 'customer % does not exist', p_customer_id
              USING ERRCODE = 'no_data_found';
    END IF;

    SELECT coalesce(sum(line_total(oi.quantity, oi.unit_price, oi.discount)), 0)
    INTO   v_spent
    FROM   orders o JOIN order_items oi USING (order_id)
    WHERE  o.customer_id = p_customer_id AND o.status <> 'cancelled';

    RETURN CASE WHEN v_spent >= 1500 THEN 'gold'
                WHEN v_spent >= 300  THEN 'silver'
                ELSE 'bronze' END;
END;
$$;

SELECT customer_id, first_name, customer_tier(customer_id) AS tier
FROM   customers ORDER BY customer_id LIMIT 5;
```

```
 customer_id | first_name |  tier
-------------+------------+--------
           1 | Anita      | gold
           2 | Ben        | gold
           3 | Chen       | bronze
           4 | Divya      | bronze
           5 | Ethan      | silver
```

`DECLARE` lists local variables; `SELECT … INTO v_spent` stores a result; `RAISE EXCEPTION … USING ERRCODE` aborts with a chosen SQLSTATE (`%` is replaced by the next argument); `RAISE NOTICE` only prints.

Catching errors. `BEGIN … EXCEPTION WHEN no_data_found THEN … END` catches errors raised inside (`SQLERRM` holds the message): the file's `safe_tier(999)` returns `unknown (customer 999 does not exist)`. An `EXCEPTION` block silently creates a **savepoint** on entry, so it costs a little — don't wrap every statement.

Set-returning functions. `RETURNS TABLE (...)` makes a function usable in `FROM`, like a parameterised view:

```
CREATE FUNCTION top_products(p_limit integer DEFAULT 3)
RETURNS TABLE (product text, units bigint, revenue numeric)
LANGUAGE sql STABLE AS $$
    SELECT p.name, sum(oi.quantity), sum(line_total(oi.quantity, oi.unit_price, oi.discount))
    FROM   order_items oi JOIN orders o USING (order_id) JOIN products p USING (product_id)
    WHERE  o.status <> 'cancelled'
    GROUP  BY p.name ORDER BY 3 DESC LIMIT p_limit
$$;
SELECT * FROM top_products(p_limit => 1);  -- named argument; top_products() uses the default 3
```

```
    product    | units | revenue
---------------+-------+---------
 Laptop Pro 14 |     3 | 3782.05
```

PL/pgSQL can instead loop (`FOR r IN SELECT … LOOP`) and emit rows with `RETURN NEXT` — the file's `salary_bands(50000)` counts employees per 50,000 band. **One-off code:** `DO $$ … $$;` runs an anonymous block once (the file prints `NOTICE: pending orders: 2`), then inspects the catalog (`prokind` `f` = function, `provolatile` `i` / `s` / `v`).

	PostgreSQL	MySQL	SQL Server	SQLite
Stored functions	`CREATE FUNCTION`, many languages	`CREATE FUNCTION` (SQL/PSM style)	`CREATE FUNCTION` (T-SQL), scalar and table-valued	None in SQL; register from the host program (Python's `conn.create_function`)
Replace	`CREATE OR REPLACE FUNCTION`	`DROP` then `CREATE`	`CREATE OR ALTER`	—
Error raising	`RAISE EXCEPTION`	`SIGNAL SQLSTATE`	`THROW`	`RAISE()` only inside triggers

Common mistakes: wrong volatility (stale results, or blocked optimisations for a pure calculation left `VOLATILE`); hiding row-by-row loops in functions (a query-running function called per row of a million-row table runs a million queries — a set-based `JOIN` is far faster); name clashes between PL/pgSQL variables and columns (`WHERE customer_id = customer_id`) — prefix `p_` and `v_`.

In the interview

30-second answer: "A UDF packages logic in the database so it's reused and runs next to the data. In Postgres I'd use a `LANGUAGE sql` function for simple expressions — it can be inlined — and PL/pgSQL for branching or loops. I'd set the right volatility, keep heavy business rules in the application unless they guard data integrity, and prefer set-based SQL over per-row function calls."

- *Function vs view?* → A view has no parameters; a set-returning function does. For a fixed query a view is simpler and fully optimisable.
- *Index `lower(email)` with my own function?* → Only if it is `IMMUTABLE`.
- *Can a `STABLE` function update a table?* → It shouldn't (it promises not to); use `VOLATILE`.
- *Return several rows?* → `RETURNS TABLE(...)` / `RETURNS SETOF type`, with a query body or `RETURN NEXT` / `RETURN QUERY`.

TL;DR

Functions are named, reusable logic stored in the database: `sql` for simple, `plpgsql` for procedural. Label volatility honestly, prefix variable names, and don't replace set-based SQL with row-by-row calls.

14.2 Stored procedures

What it is

A **stored procedure** *does* something rather than *computes* something: run with `CALL`, it returns nothing (except through `INOUT` / `OUT` parameters) and — the key difference in PostgreSQL (procedures since version 11) — may **commit or roll back inside its body**.

	Function	Procedure
Invoked by	`SELECT f(...)`, inside any expression	`CALL p(...)` only
Returns	A value or a set of rows	Nothing; results via `INOUT` / `OUT` parameters
`COMMIT` / `ROLLBACK` inside	Not allowed — runs inside the caller's transaction	Allowed when called outside an explicit transaction block
Typical use	Calculations, reusable queries, trigger bodies	Batch jobs, maintenance, multi-step operations

Why it matters: long batch jobs (archive a year of orders, back-fill a column) should commit in **chunks** — one giant transaction holds locks long, keeps old row versions alive (blocking vacuum, Chapter 13), and loses everything if it fails at the end.

Example — archiving in committed batches

From `code/02_procedures.sql`:

```
CREATE TABLE orders_archive (LIKE orders INCLUDING ALL);

CREATE PROCEDURE archive_delivered(p_before date, p_batch int, INOUT moved int DEFAULT 0)
LANGUAGE plpgsql AS $$
DECLARE n int;
BEGIN
    LOOP
        WITH picked AS (
            SELECT order_id FROM orders
            WHERE  status = 'delivered' AND placed_at < p_before
              AND  order_id NOT IN (SELECT order_id FROM orders_archive)
            ORDER  BY order_id LIMIT p_batch
        )
        INSERT INTO orders_archive
        SELECT o.* FROM orders o JOIN picked USING (order_id);
        GET DIAGNOSTICS n = ROW_COUNT;     -- rows affected by the last statement
        EXIT WHEN n = 0;
        moved := moved + n;
        RAISE NOTICE 'batch of % committed', n;
        COMMIT;
    END LOOP;
END;
$$;

CALL archive_delivered('2025-04-01', 2);
```

```
NOTICE:  batch of 2 committed
NOTICE:  batch of 2 committed
NOTICE:  batch of 1 committed
 moved
-------
     5
```

The five delivered orders before April 2025 (101, 102, 103, 104, 106) were copied in three committed batches; the `INOUT` parameter came back as a one-row result.

The catch. Inside a caller's transaction (`BEGIN; CALL …`) the procedure cannot end it:

```
BEGIN;
DO $$ DECLARE m int := 0; BEGIN
    CALL archive_delivered('2025-12-31', 2, m);
EXCEPTION WHEN invalid_transaction_termination THEN
    RAISE NOTICE 'caught: %', SQLERRM;
END $$;
ROLLBACK;
```

```
NOTICE:  batch of 2 committed
NOTICE:  caught: invalid transaction termination
```

The first `NOTICE` prints before the `COMMIT` fails — misleading, which is why the file rolls everything back. Many drivers open a transaction automatically (14.10), so a committing procedure must be called in **autocommit** mode.

	Syntax	Transaction control inside
PostgreSQL 11+	`CREATE PROCEDURE … ; CALL p()`	Yes, if not called inside a transaction block
MySQL	`CREATE PROCEDURE` (the `mysql` client needs `DELIMITER //` to change the statement terminator from `;`); `CALL p()`	Yes
SQL Server	`CREATE PROCEDURE`; `EXEC p`	Yes; can also return result sets directly
SQLite	No stored procedures	—

When not to use it: procedures tie logic to one product, are harder to unit-test, version and deploy than application code, and scale only as far as the database server. Balanced rule: **integrity rules and data-heavy batch work in the database; business workflows in the application.**

- *Value back from a procedure?* → An `INOUT` (or, from PG 14, `OUT`) parameter.
- *Why does `COMMIT` in a procedure sometimes fail?* → The caller is already in a transaction block.
- *Why commit in batches?* → Shorter locks, less bloat, completed work survives a later failure.

TL;DR

Procedures are `CALL`ed, return through `INOUT` parameters, and can `COMMIT` inside — ideal for batch jobs, but only when called in autocommit mode.

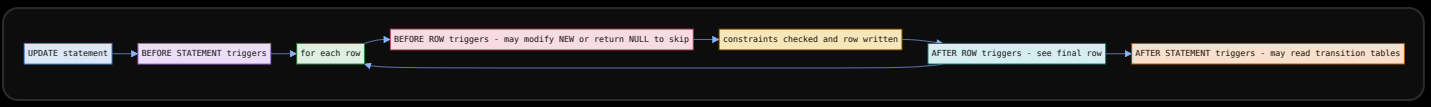
14.3 Triggers

What it is

A **trigger** says: "whenever *this event* happens on *this table*, run *this function* automatically". Events: `INSERT`, `UPDATE`, `DELETE`, `TRUNCATE`.

Choice	Options	Meaning
Timing	`BEFORE` / `AFTER` / `INSTEAD OF` (views only)	Before the write (can change or skip the row), after it (can react), or replacing the write on a view
Level	`FOR EACH ROW` / `FOR EACH STATEMENT`	Once per affected row, or once per statement (even for 0 rows)
Condition	`WHEN (…)`	Fire only if the condition on `OLD` / `NEW` is true
Column filter	`UPDATE OF col`	Fire only when those columns are named in the `UPDATE`

In a row trigger `NEW` is the new row (INSERT/UPDATE), `OLD` the previous one (UPDATE/DELETE); `TG_OP`, `TG_TABLE_NAME` and friends describe the event. The function is declared `RETURNS trigger`.



Why it matters: triggers enforce behaviour **whichever application, script or person** changes the data: an `updated_at` stamp, an audit trail, a denormalised counter kept in sync, a rule a `CHECK` can't express because it needs another table.

Example 1 — `updated_at`

From `code/03_triggers.sql`:

```
ALTER TABLE products ADD COLUMN updated_at timestamptz NOT NULL DEFAULT '2025-01-01 00:00+00';

CREATE FUNCTION set_updated_at() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
    NEW.updated_at := now();      -- change the row before it is written
    RETURN NEW;                   -- BEFORE row trigger: return the row to store (NULL = skip it)
END;
$$;

CREATE TRIGGER products_updated_at
BEFORE UPDATE ON products
FOR EACH ROW
WHEN (OLD.* IS DISTINCT FROM NEW.*)          -- skip no-op updates
EXECUTE FUNCTION set_updated_at();

UPDATE products SET unit_price = 22.99 WHERE product_id = 2;
UPDATE products SET unit_price = 35.00 WHERE product_id = 8;   -- same value as before
SELECT product_id, unit_price, updated_at > '2025-01-02' AS stamped_now
FROM   products WHERE product_id IN (2, 8) ORDER BY product_id;
```

```
 product_id | unit_price | stamped_now
------------+------------+-------------
          2 |      22.99 | t
          8 |      35.00 | f
```

`EXECUTE FUNCTION` is the PostgreSQL 11+ spelling (`EXECUTE PROCEDURE` still accepted).

Example 2 — a validation rule that needs another table

```
CREATE FUNCTION forbid_discontinued() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
    IF (SELECT discontinued FROM products WHERE product_id = NEW.product_id) THEN
        RAISE EXCEPTION 'product % is discontinued', NEW.product_id;
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER order_items_check_product
BEFORE INSERT OR UPDATE OF product_id ON order_items
FOR EACH ROW EXECUTE FUNCTION forbid_discontinued();
-- inserting the discontinued Desk Lamp (id 8):
-- NOTICE:  rejected: product 8 is discontinued
```

Example 3 — a generic audit log

One function serves any table, because `to_jsonb(OLD)` / `to_jsonb(NEW)` turn a whole row into JSON:

```
CREATE TABLE audit_log (
    id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    table_name  text        NOT NULL,
    op          text        NOT NULL,
    changed_by  text        NOT NULL DEFAULT current_user,
    changed_at  timestamptz NOT NULL DEFAULT now(),
    old_row     jsonb,
    new_row     jsonb
);

CREATE FUNCTION audit_row() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO audit_log (table_name, op, old_row, new_row)
    VALUES (TG_TABLE_NAME, TG_OP,
            CASE WHEN TG_OP IN ('UPDATE','DELETE') THEN to_jsonb(OLD) END,
            CASE WHEN TG_OP IN ('INSERT','UPDATE') THEN to_jsonb(NEW) END);
    RETURN NULL;                  -- the return value of an AFTER trigger is ignored
END;
$$;

CREATE TRIGGER customers_audit
AFTER INSERT OR UPDATE OR DELETE ON customers
FOR EACH ROW EXECUTE FUNCTION audit_row();

INSERT INTO customers VALUES (11, 'Jia', 'Wong', 'jia.wong@mail.example', 'Singapore', 'Singapore', '2025-06-20');
UPDATE customers SET city = 'Mumbai' WHERE customer_id = 8;
DELETE FROM customers WHERE customer_id = 11;

SELECT id, table_name, op, changed_by, old_row ->> 'city' AS old_city, new_row ->> 'city' AS new_city
FROM   audit_log ORDER BY id;
```

```
 id | table_name |   op   | changed_by | old_city  | new_city
----+------------+--------+------------+-----------+-----------
  1 | customers  | INSERT | postgres   |           | Singapore
  2 | customers  | UPDATE | postgres   | mumbai    | Mumbai
  3 | customers  | DELETE | postgres   | Singapore |
```

Which columns changed? Compare the JSON objects key by key:

```
SELECT n.key, o.value AS before, n.value AS after
FROM   audit_log a, jsonb_each(a.new_row) n
JOIN   jsonb_each(a.old_row) o USING (key)
WHERE  a.op = 'UPDATE' AND n.value IS DISTINCT FROM o.value;
```

```
 key  |  before  |  after
------+----------+----------
 city | "mumbai" | "Mumbai"
```

The audit insert runs **in the same transaction** as the change, so a rolled-back change leaves no audit row — the log never lies.

Example 4 — statement-level trigger with transition tables

A row trigger on a million-row `UPDATE` fires a million times. A **statement-level** trigger fires once and (PostgreSQL 10+) sees all changed rows through **transition tables**:

```
CREATE TRIGGER products_bulk_price
AFTER UPDATE ON products
REFERENCING OLD TABLE AS old_rows NEW TABLE AS new_rows
FOR EACH STATEMENT EXECUTE FUNCTION log_bulk_price_change();
-- the function reads new_rows / old_rows like tables (count, avg of price differences)

UPDATE products SET unit_price = unit_price * 1.10 WHERE category = 'Furniture';
-- NOTICE:  3 rows changed price, average change 29.47
```

	PostgreSQL	MySQL	SQL Server	SQLite
Row-level	Yes	Yes (only kind)	No — statement-level only, with `inserted` / `deleted` pseudo-tables	Yes (only kind)
Statement-level	Yes (+ transition tables)	No	Yes (the only kind)	No
`INSTEAD OF`	On views	No	On tables and views	On views
Body	A separate trigger function	Inline SQL body	Inline T-SQL	Inline SQL body

When not to use it, and mistakes

- **Business workflows** (send an email, call a payment API): triggers run inside the transaction and must be fast — write an *outbox* row and let a background worker act on it (the **transactional outbox** pattern for reliable event publishing).
- When a **constraint** can do the job (`CHECK`, `FOREIGN KEY`, `UNIQUE`, generated column) — declarative, faster, visible in the schema.
- When the logic surprises people — "invisible" side effects are the top complaint about triggers.
- **Common mistakes:** forgetting `RETURN NEW` in a `BEFORE` row trigger (`NULL` silently **skips** the row); a trigger updating its own table and re-firing in a loop (guard with `WHEN` or `pg_trigger_depth()`); expecting row-level triggers on `TRUNCATE` (only statement-level `ON TRUNCATE` fires).

Real uses: compliance audit trails (who changed a salary), `updated_at` columns read by sync jobs, the outbox.

In the interview

> **30-second answer:** "A trigger runs a function automatically on INSERT/UPDATE/DELETE, before or after, per row or per statement. I use them for cross-cutting data rules — audit logs, timestamps, integrity checks a constraint can't express — and avoid them for business workflows because they're invisible and slow down writes."

- *BEFORE vs AFTER?* → BEFORE can modify or reject the row; AFTER sees the final row (after constraints) and is for reactions like auditing.
- *Does a trigger's work roll back with the statement?* → Yes, same transaction.
- *Audit a bulk update efficiently?* → Statement-level trigger with transition tables.
- *Record available in a DELETE trigger?* → `OLD` only. *Why* *`WHEN (OLD.* IS DISTINCT FROM NEW.*)`*?* → Skips no-op updates, so `updated_at` changes only on real changes.

TL;DR

Triggers = automatic functions on data events. Great for audit logs, `updated_at` and cross-table checks; keep them small, prefer constraints when possible, and never put slow external work in them.

14.4 JSON and JSONB

What it is

JSON (JavaScript Object Notation) is the text format of web APIs: objects `{"key": value}`, arrays `[…]`, strings, numbers, booleans, `null`. PostgreSQL has two types:

	`json`	`jsonb` ("JSON binary")
Storage	The exact text you sent	Parsed into a binary tree
Duplicate keys, whitespace, key order	Kept as written	Last duplicate wins; whitespace removed; keys reordered
Speed	Fast to store, re-parsed on every read	Slightly slower to store, fast to query
Operators & indexing	Few	Containment `@>`, key-exists `?`, GIN (inverted) indexes, SQL/JSON path

Use `jsonb` unless you must keep the original text byte-for-byte.

```
SELECT '{"b":1, "a":2, "a":3}'::json AS as_json, '{"b":1, "a":2, "a":3}'::jsonb AS as_jsonb;
```

```
       as_json        |     as_jsonb
----------------------+------------------
{"b":1, "a":2, "a":3} | {"a": 3, "b": 1}
```

Why it matters: some data is genuinely **semi-structured** — specs differ by category (a laptop has RAM, a desk a height range), webhook payloads change shape, preferences grow new keys. A `jsonb` column holds that without a column per attribute, while stable, important fields stay typed, constrained columns. Whole-document databases: Chapter 16.

Operators

Operator / function	Returns	Example → result
`->` key or index	`jsonb`	`specs -> 'brand'` → `"Nimbus"`
`->>` key or index	`text`	`specs ->> 'brand'` → `Nimbus`
`#>` / `#>>` path	`jsonb` / `text`	`specs #>> '{screen,type}'` → `OLED`
`@>` contains	boolean	`specs @> '{"brand":"Nimbus"}'`
`?`, `?`	`, ?&`	boolean
`@?`, `@@`	boolean	SQL/JSON path exists / path predicate (PG 12+)
`		`, `-`, `#-`
`jsonb_set(doc, path, value)`	`jsonb`	replace or add a value at a path

Example — product specs

From `code/04_jsonb.sql` (adds `specs jsonb` for five products):

```
SELECT name,
       specs -> 'brand'               AS brand_json,
       specs ->> 'brand'              AS brand_text,
       specs #>> '{screen,type}'      AS screen_type,
       (specs ->> 'ram_gb')::int      AS ram_gb,
       specs -> 'ports' -> 0          AS first_port,
       jsonb_array_length(specs -> 'ports') AS n_ports
FROM   products WHERE specs ? 'ports' ORDER BY product_id;
```

```
     name      | brand_json | brand_text | screen_type | ram_gb | first_port | n_ports
---------------+------------+------------+-------------+--------+------------+---------
 Laptop Pro 14 | "Nimbus"   | Nimbus     | OLED        |     16 | "usb-c"    |       2
 Wireless Mouse| "Clicko"   | Clicko     |             |        | "usb-a"    |       1
 USB-C Hub     | "Nimbus"   | Nimbus     |             |        | "usb-c"    |       4
```

`->` gives JSON (`"Nimbus"`), `->>` plain text; missing keys give `NULL`, not an error; compare numbers after casting (`(specs ->> 'ram_gb')::int`).

```
SELECT name FROM products WHERE specs @> '{"brand":"Nimbus"}';          -- Laptop Pro 14, USB-C Hub
SELECT name FROM products WHERE specs -> 'ports' ? 'hdmi';              -- Laptop Pro 14, USB-C Hub
SELECT name, jsonb_path_query_first(specs, '$.height_cm.max') AS max_height
FROM   products WHERE specs @@ '$.height_cm.max > 100';                 -- Standing Desk | 120
```

Updating — `jsonb` values are immutable, so each operation builds a new document:

```
UPDATE products SET specs = jsonb_set(specs, '{ram_gb}', '32')  WHERE product_id = 1;
UPDATE products SET specs = specs || '{"color":"black"}'        WHERE product_id = 2;
UPDATE products SET specs = specs - 'wireless'                  WHERE product_id = 7;
UPDATE products SET specs = specs #- '{screen,type}'            WHERE product_id = 1;
```

```
product_id |                                          specs
-----------+-------------------------------------------------------------------------------------------
         1 | {"brand": "Nimbus", "ports": ["usb-c", "hdmi"], "ram_gb": 32, "screen": {"inches": 14}}
         2 | {"brand": "Clicko", "color": "black", "ports": ["usb-a"], "wireless": true}
         7 | {"brand": "Hushh", "battery_h": 30}
```

Rows ↔ JSON. `jsonb_build_object` + `jsonb_agg` build a nested API response in one query — for order 108: `{"items": [{"qty": 1, "product": "Laptop Pro 14"}, …], "status": "delivered", "order_id": 108}`. Back: `jsonb_to_recordset('[{"sku":"A1","qty":2},{"sku":"B7","qty":5}]') AS t(sku text, qty int)` gives rows `(A1, 2)`, `(B7, 5)`; `jsonb_each(doc)` gives `(key, value)` rows.

Indexing

A **GIN** index (each key/value → the rows containing it) serves `@>`, `?`, `?|`, `?&` and path operators; a B-tree **expression index** serves equality and ranges on one extracted key. (The file disables sequential scans only because the demo table is tiny, and runs `VACUUM products` first: an index built over rows still carrying old versions from recent `UPDATE`s may be ignored by transactions with older snapshots. Reading plans: Chapter 12.)

```
CREATE INDEX products_specs_gin ON products USING gin (specs);
EXPLAIN (COSTS OFF) SELECT name FROM products WHERE specs @> '{"brand":"Nimbus"}';
CREATE INDEX products_brand_idx ON products ((specs ->> 'brand'));
EXPLAIN (COSTS OFF) SELECT name FROM products WHERE specs ->> 'brand' = 'Nimbus';
```

```
Bitmap Heap Scan on products
  Recheck Cond: (specs @> '{"brand": "Nimbus"}'::jsonb)
  ->  Bitmap Index Scan on products_specs_gin
        Index Cond: (specs @> '{"brand": "Nimbus"}'::jsonb)

Index Scan using products_brand_idx on products
  Index Cond: ((specs ->> 'brand'::text) = 'Nimbus'::text)
```

The GIN index is *not* used for `specs ->> 'brand' = 'Nimbus'` — write `@>` or add the expression index.

	Type	Extract text	Notes
PostgreSQL	`json`, `jsonb`	`->>`, `#>>`, `jsonb_path_query`	GIN indexes; `JSON_TABLE` and SQL/JSON query functions in PG 17
MySQL 5.7+	`JSON` (binary)	`->>`, `JSON_EXTRACT`, `JSON_UNQUOTE`	Index via generated columns or multi-valued indexes (8.0.17+)
SQL Server	`nvarchar` + `JSON_VALUE` / `OPENJSON`; native `json` type in SQL Server 2025 and Azure SQL	`JSON_VALUE(doc, '$.brand')`	Index a computed column
SQLite	`TEXT` with built-in JSON functions; binary JSONB since 3.45	`->>` (3.38+), `json_extract`	Expression indexes

When not to use it: fields that are known, always present, filtered or joined on, or need constraints belong in **real columns**. One JSON blob for everything is the "EAV / schemaless" antipattern (EAV = entity–attribute–value, every attribute a generic key/value pair): no types, no constraints, poor statistics. **Common mistakes:** comparing `->` output with text (`specs -> 'brand' = 'Nimbus'` fails, jsonb vs text — use `->>` or `@>`); sorting numbers as text (`'9' > '10'`); forgetting that updating one key still rewrites the whole row.

In the interview

> **30-second answer:** "I use `jsonb` for genuinely variable attributes alongside normal columns. I query with `->>` / `@>`, index with GIN for containment or a B-tree expression index for one hot key, and promote any field that becomes important to a real column."

- *json vs jsonb?* → jsonb is parsed, indexable, fast to query; json keeps exact text.
- *`'{"a":1}'::jsonb -> 'a'` vs `->> 'a'`?* → jsonb `1` vs text `'1'`. *Append a key?* → `specs || '{"k":"v"}'` or `jsonb_set`.

TL;DR

`jsonb` + `->>`/`@>` + GIN = flexible attributes inside a relational table. Keep core fields as columns.

14.5 Arrays

An **array** column holds an ordered list of one type (`text[]`, `int[]`); arrays are **1-based**, and reading past the end returns `NULL` (`ARRAY[10,20][0]` → `NULL`). From `code/05_arrays.sql`:

```
ALTER TABLE products ADD COLUMN tags text[] NOT NULL DEFAULT '{}';
UPDATE products SET tags = CASE product_id
    WHEN 1 THEN ARRAY['laptop','work','premium']
    WHEN 2 THEN '{mouse,wireless,budget}'          -- text literal form
    WHEN 4 THEN ARRAY['keyboard','work']
    WHEN 5 THEN ARRAY['desk','work','premium']
    WHEN 7 THEN ARRAY['audio','wireless','premium']
    ELSE '{}' END;

SELECT name, tags, tags[1] AS first_tag, tags[2:3] AS slice, cardinality(tags) AS n, tags[9] AS no_such
FROM   products WHERE product_id IN (1, 2, 9) ORDER BY product_id;
```

```
     name      |          tags           | first_tag |       slice       | n | no_such
---------------+-------------------------+-----------+-------------------+---+---------
 Laptop Pro 14 | {laptop,work,premium}   | laptop    | {work,premium}    | 3 |
 Wireless Mouse | {mouse,wireless,budget} | mouse     | {wireless,budget} | 3 |
 Gift Card     | {}                      |           | {}                | 0 |
```

Operation	Syntax	Result in the demo
Element in array	`'premium' = ANY (tags)`	Laptop, Headphones, Desk
Contains all	`tags @> ARRAY['work','premium']`	Laptop, Desk
Overlaps (any in common)	`tags && ARRAY['wireless','keyboard']`	Keyboard, Headphones, Mouse
Append / remove	`array_append(a, x)`, `array_remove(a, x)`	`{a,b,c}`, `{b}`
Concatenate / find	`a		
Array ↔ rows	`unnest(a) WITH ORDINALITY`, `array_agg(x ORDER BY …)`	see below
Array ↔ string	`string_to_array(s, ',')`, `array_to_string(a, '	')`

`unnest` and `array_agg` are opposites; together they answer "which tags are popular?":

```
SELECT tag, count(*) AS products, array_agg(name ORDER BY name) AS which
FROM   products, unnest(tags) AS tag
GROUP  BY tag HAVING count(*) > 1 ORDER BY products DESC, tag;
```

```
   tag    | products |                              which
----------+----------+-----------------------------------------------------------------
premium   |        3 | {"Laptop Pro 14","Noise-Cancelling Headphones","Standing Desk"}
work      |        3 | {"Laptop Pro 14","Mechanical Keyboard","Standing Desk"}
wireless  |        2 | {"Noise-Cancelling Headphones","Wireless Mouse"}
```

Arrays as a parameter. For "rows whose id is in this list", don't build `IN (3, 5, 7)` as a string (injection risk, 14.9, and a new statement per list length) — pass **one array**: `WHERE product_id = ANY ($1)`. The file shows `PREPARE by_ids(int[])` / `EXECUTE by_ids('{3,5,7}')` (a server-side prepared statement: parsed once, run with different values) returning USB-C Hub, Standing Desk and Noise-Cancelling Headphones. A GIN index on `tags` serves `@>`, `&&` and `<@` (`Bitmap Index Scan on products_tags_gin`).

When not to use it: to **join** on elements, enforce **foreign keys** on them, or update single elements often, use a child table (`product_tags(product_id, tag)`) — what first normal form (§11.7) recommends. Arrays suit small, read-mostly lists like tags. MySQL, SQL Server and SQLite have no array type (JSON arrays or a child table). `x = ANY(arr)` tests one value; `arr @> other` tests every element of `other`.

TL;DR

Arrays: 1-based lists in one column; query with `ANY`, `@>`, `&&`, explode with `unnest`, build with `array_agg`, index with GIN; use a child table when elements need relationships.

14.6 Full-text search

What it is

Full-text search (FTS) matches *words* — ignoring case, endings and filler words — and ranks documents by relevance.

Term	Meaning
Lexeme	A normalised word: lower-cased and **stemmed** ("shipping", "shipped" → `ship`)
Stop word	A very common word ("the", "and", "was") that is dropped
`tsvector`	A document as sorted lexemes with positions: `'chair':2 'ship':4,8`
`tsquery`	A search: lexemes joined by `&` (and), `
`@@`	"Does this vector match this query?"
Text search configuration	Language rules, e.g. `'english'` (stemmer + stop-word list)

Why it matters: `LIKE '%word%'` has no stemming ("returning" misses "returned"), no ranking, and a leading `%` can't use a B-tree. FTS fixes all three and often postpones a separate search engine (Elasticsearch/OpenSearch).

How it works

1. `to_tsvector('english', text)` parses, drops stop words, stems the rest.
2. Build a `tsquery` from input: `to_tsquery` (operator syntax `'ship & chair'`; errors on bad syntax), `plainto_tsquery` (all words ANDed), `websearch_to_tsquery` (PG 11+, Google-like: quotes for phrases, `or`, `-` to exclude; never errors on user input).
3. `vector @@ query` tests a match; `ts_rank` scores; `ts_headline` highlights.
4. A GIN index on the `tsvector` makes matching fast.

From `code/06_full_text_search.sql`:

```
SELECT to_tsvector('english', 'The chairs were shipped quickly and the shipping was free') AS doc;
SELECT to_tsquery('english', 'shipping & chair')                        AS q1,
       plainto_tsquery('english', 'fast shipping chairs')               AS q2,
       websearch_to_tsquery('english', '"standing desk" -cheap or lamp') AS q3;
```

```
                   doc
-----------------------------------------
 'chair':2 'free':10 'quick':5 'ship':4,8

       q1        |            q2             |                  q3
-----------------+----------------------------+---------------------------------------
 'ship' & 'chair' | 'fast' & 'ship' & 'chair' | 'stand' <-> 'desk' & !'cheap' | 'lamp'
```

Example — a help-centre search

The search vector is a **generated column** (computed from other columns on every write; PostgreSQL 12+), so it never goes stale; `setweight` marks title words `A` (most important) and body words `B`, which `ts_rank` rewards:

```
CREATE TABLE help_articles (
    id     int PRIMARY KEY,
    title  text NOT NULL,
    body   text NOT NULL,
    search tsvector GENERATED ALWAYS AS (
             setweight(to_tsvector('english', title), 'A') ||
             setweight(to_tsvector('english', body),  'B')) STORED
);
-- 5 articles: 'Returning a chair', 'Standing desk assembly', 'Shipping times',
--             'Keyboard not connecting', 'Cheap desk lamps discontinued'

SELECT id, title, round(ts_rank(search, q)::numeric, 3) AS rank
FROM   help_articles, websearch_to_tsquery('english', 'chair shipping') AS q
WHERE  search @@ q
ORDER  BY rank DESC, id;
```

```
 id |       title       | rank
----+-------------------+-------
  1 | Returning a chair | 0.588
  3 | Shipping times    | 0.529
```

Query	Matches	Why
`to_tsquery('english', 'assem:*')`	2 Standing desk assembly	`:*` = prefix match (autocomplete)
`websearch_to_tsquery('english', '"standing desk"')`	2	Phrase: `stand <-> desk`
`websearch_to_tsquery('english', 'desk -lamp')`	2, 3	Article 5 excluded by `-lamp`

Highlighting shows *why* a result matched:

```
SELECT id, ts_headline('english', body, websearch_to_tsquery('english', 'desk chair'),
                       'StartSel=[, StopSel=], MaxWords=12, MinWords=5') AS snippet
FROM   help_articles WHERE search @@ websearch_to_tsquery('english', 'desk chair');
```

```
 id |             snippet
----+---------------------------------
  3 | [desk] or [chair] ships in a week
```

Stemming in action — no body contains "returning", but article 1 has "returned" and "returns":

```
like_hits | fts_hits          -- body ILIKE '%returning%'  vs  to_tsvector(body) @@ 'returning'
----------+---------
        0 |        1
```

With `CREATE INDEX … USING gin (search)` the plan becomes `Bitmap Index Scan on help_articles_search_gin`.

	PostgreSQL FTS	Dedicated engine (Elasticsearch / OpenSearch / Solr)
Setup	None — same database, same transaction	Separate cluster plus a sync pipeline
Freshness	Immediately consistent	Near-real-time, eventually consistent
Features	Stemming, ranking, phrases, prefixes, weights	Fuzzy/typo tolerance, facets (match counts per category, shown as filters), synonyms at scale, relevance tuning, distributed scoring
Best for	Up to millions of documents, app-level search	Very large corpora, search-centric products

Typo tolerance in Postgres needs `pg_trgm` (trigram similarity, §12.13), not installed on the book's server. Elsewhere: MySQL `FULLTEXT` with `MATCH … AGAINST`, SQL Server full-text catalogs with `CONTAINS` / `FREETEXT`, SQLite `FTS5` virtual tables.

Common mistakes: indexing with `to_tsvector('english', …)` but querying `to_tsvector(body)` (default config) — the index isn't used, so always name the configuration; raw user input to `to_tsquery` (a stray `&` or `(` is a syntax error — use `websearch_to_tsquery`); computing `to_tsvector` per query instead of storing and indexing it.

In the interview

30-second answer: "Postgres FTS turns text into a `tsvector` of stemmed lexemes and matches a `tsquery` with `@@`. I store the vector as a weighted generated column, index it with GIN, parse user input with `websearch_to_tsquery`, and rank with `ts_rank`. I'd move to Elasticsearch only when I need fuzzy matching, facets or scale beyond one database."

- *Why does "shipping" match "shipped"?* → Both stem to `ship`. *Titles count more?* → `setweight(…, 'A')`; `ts_rank` weights A highest.

TL;DR

FTS = `tsvector @@ tsquery` + GIN + `ts_rank` + `ts_headline`; beats `LIKE` on stemming, ranking and speed.

14.7 Pivot and unpivot

Pivoting turns *values in rows* into *columns* (one row per category, one column per month); **unpivoting** turns a wide table tall again. Databases prefer tall data; reports prefer wide. Standard SQL has no `PIVOT`, so pivots use conditional aggregation (§4.4).

From `code/07_pivot.sql`; a view supplies tall `category, month, revenue`:

```sql
SELECT category,
       round(sum(revenue) FILTER (WHERE month = '2025-01'), 2) AS jan,
       round(sum(revenue) FILTER (WHERE month = '2025-02'), 2) AS feb,
       round(sum(revenue) FILTER (WHERE month = '2025-03'), 2) AS mar,
       round(sum(revenue) FILTER (WHERE month = '2025-04'), 2) AS apr,
       round(sum(revenue), 2)                                  AS all_months
FROM   monthly_category_revenue
GROUP  BY category ORDER BY category;
```

```
  category   |   jan   |  feb   |  mar   |   apr   | all_months
-------------+---------+--------+--------+---------+------------
 (none)      |         |        | 100.00 |         |     100.00
 Accessories |  188.00 |        |        |  200.80 |     671.35
 Electronics | 1683.90 |        |  74.97 | 1448.95 |    4691.80
 Furniture   |         | 811.49 | 629.98 |  499.00 |    1940.47
```

(`all_months` also includes May and June.) The portable form puts `CASE` inside the aggregate — `sum(CASE WHEN month = '2025-01' THEN revenue ELSE 0 END) AS jan` — and with `ELSE 0` empty cells show `0.00` instead of `NULL`. The same technique counts statuses per customer (`count(*) FILTER (WHERE o.status = 'delivered') AS delivered`, …).

`crosstab(source_sql, category_sql)` (tablefunc extension) pivots a three-column query (row id, category, value). Not installed on the book's server (Homebrew's build ships without it), so not run:

```
-- (PostgreSQL with contrib — not run)
CREATE EXTENSION IF NOT EXISTS tablefunc;
SELECT *
FROM crosstab(
  $$ SELECT category, month, round(revenue, 2) FROM monthly_category_revenue ORDER BY 1, 2 $$,
  $$ VALUES ('2025-01'), ('2025-02'), ('2025-03'), ('2025-04') $$
) AS ct(category text, jan numeric, feb numeric, mar numeric, apr numeric);
```

Output columns and types must still be listed in `AS ct(...)`; the two-argument form places each value in the right column even when some categories are missing. `FILTER` / `CASE` does the same without an extension, so most teams use them.

Unpivot with `LATERAL (VALUES …)` (`LATERAL`, §6.7, lets `VALUES` read the current row):

```
CREATE TABLE quarterly_targets (region text PRIMARY KEY, q1 int, q2 int, q3 int, q4 int);
INSERT INTO quarterly_targets VALUES ('India', 120, 150, 170, 200), ('UK', 60, 65, NULL, 80);

SELECT t.region, v.quarter, v.target
FROM   quarterly_targets t
CROSS  JOIN LATERAL (VALUES ('Q1', t.q1), ('Q2', t.q2), ('Q3', t.q3), ('Q4', t.q4)) AS v(quarter, target)
WHERE  v.target IS NOT NULL
ORDER  BY t.region, v.quarter;
```

7 rows — four for India, three for the UK (its `NULL` Q3 filtered out). Portable alternative: `UNION ALL` of one `SELECT` per column.

Dynamic pivots. SQL needs the column list **at plan time**, so data-dependent columns (every month that exists) require building SQL in code (identifiers quoted, 14.9; full worked version in §19.6.2) or returning a JSON object per row:

```
SELECT category, jsonb_object_agg(month, round(revenue, 2) ORDER BY month) AS by_month
FROM   monthly_category_revenue GROUP BY category ORDER BY category;
```

```
  category   |                        by_month
-------------+-----------------------------------------------------------
 (none)      | {"2025-03": 100.00}
 Accessories | {"2025-01": 188.00, "2025-04": 200.80, "2025-06": 282.55}
 ...           (Electronics and Furniture rows similar)
```

	Pivot	Unpivot
PostgreSQL	`FILTER`, `CASE`, `crosstab` (tablefunc)	`LATERAL (VALUES …)`, `unnest`
SQL Server / Oracle	`PIVOT (SUM(x) FOR col IN (…))` operator	`UNPIVOT` operator (skips NULLs)
MySQL	`CASE` (or `SUM(cond)` since booleans are 0/1)	`UNION ALL`; `LATERAL` from 8.0.14
SQLite	`FILTER` (3.30+) or `CASE`	`UNION ALL`

In the interview

> **30-second answer:** "Pivot = group by the row key and write one conditional aggregate per output column — `SUM(x) FILTER (WHERE col = 'v')` or `SUM(CASE WHEN …)` . Unpivot = `CROSS JOIN LATERAL (VALUES …)` or `UNION ALL` . If the columns aren't known in advance, generate the SQL in code or return JSON."

- *`count(CASE WHEN … THEN 1 END)` vs `sum(CASE WHEN … THEN 1 ELSE 0 END)`?* → Same counts; `count` ignores the NULLs from the missing `ELSE` .

TL;DR

Pivot with conditional aggregates, unpivot with `LATERAL (VALUES …)`; dynamic column lists need generated SQL or JSON.

14.8 Security: roles, privileges and row-level security

What it is

- A **role** is PostgreSQL's single concept for *users* and *groups*: with `LOGIN` it can connect (a user); without, it is a group others belong to. Roles are **cluster-wide** (shared by all databases on a server).
- A **privilege** permits one action on one object: `SELECT` , `INSERT` , `UPDATE` , `DELETE` , `TRUNCATE` , `REFERENCES` , `TRIGGER` on tables; `USAGE` / `CREATE` on schemas; `EXECUTE` on functions; `CONNECT` on databases.
- `GRANT` gives, `REVOKE` takes away — the **DCL** statements (§1.9).
- **Row-level security (RLS)** adds *per-row* filters: two users read one table and see different rows.

Why — the principle of least privilege: every account gets only what it needs. A leaked reporting password can read reports, not delete orders; with an injection bug (14.9) a read-only role limits damage. RLS pushes tenant isolation ("customer A never sees customer B's data") into the database, so one forgotten `WHERE tenant_id = …` can't leak data.

Example — groups, grants, column privileges

From `code/08_security.sql` (roles prefixed `ch14_` and dropped at the end, since they are server-wide):

```
CREATE ROLE ch14_reporting NOLOGIN;                          -- a group
CREATE ROLE ch14_analyst  LOGIN IN ROLE ch14_reporting;      -- a user in that group
CREATE ROLE ch14_app      LOGIN;                             -- the application's account

GRANT SELECT ON orders, order_items TO ch14_reporting;
GRANT SELECT, INSERT, UPDATE ON orders TO ch14_app;
GRANT SELECT (customer_id, first_name, city, country) ON customers TO ch14_reporting;  -- column-level
```

`information_schema.role_table_grants` (`information_schema` = the SQL-standard catalog views) confirms: `ch14_app` has INSERT, SELECT, UPDATE on `orders` ; `ch14_reporting` SELECT on `orders` and `order_items` (column grants appear in `information_schema.column_privileges`). `SET ROLE` lets a superuser act as another role for testing, with the same checks:

```
SET ROLE ch14_analyst;
SELECT count(*) FROM orders;                                    -- 14
SELECT first_name, city FROM customers WHERE customer_id = 1;      -- Anita | Mumbai
PERFORM email FROM customers;             -- NOTICE: analyst: permission denied for table customers
DELETE FROM orders WHERE order_id = 114;  -- NOTICE: analyst: permission denied for table orders
RESET ROLE;
REVOKE SELECT ON order_items FROM ch14_reporting;   -- analyst now denied on order_items too
```

(Failing statements are wrapped in `DO` blocks so the script continues; `PERFORM` runs a query and discards its result.) The analyst had no direct grants — privileges were **inherited** from the group, so revoking from the group affected every member: grant to groups, not individuals.

Example — row-level security: each sales rep sees only their orders

```
CREATE ROLE ch14_priya LOGIN;      -- employee 7
CREATE ROLE ch14_arjun LOGIN;      -- employee 8
CREATE TABLE rep_logins (role_name text PRIMARY KEY, emp_id int NOT NULL REFERENCES employees);
INSERT INTO rep_logins VALUES ('ch14_priya', 7), ('ch14_arjun', 8);
GRANT SELECT ON rep_logins TO ch14_priya, ch14_arjun;
GRANT SELECT, UPDATE ON orders TO ch14_priya, ch14_arjun;

ALTER TABLE orders ENABLE ROW LEVEL SECURITY;          -- from now on: no matching policy = no rows

CREATE POLICY rep_sees_own_orders ON orders
    FOR ALL TO ch14_priya, ch14_arjun
    USING      (sales_rep_id = (SELECT emp_id FROM rep_logins WHERE role_name = current_user))
    WITH CHECK (sales_rep_id = (SELECT emp_id FROM rep_logins WHERE role_name = current_user));

CREATE POLICY reporting_reads_all ON orders FOR SELECT TO ch14_reporting USING (true);

SET ROLE ch14_priya;
SELECT current_user, order_id, sales_rep_id, status FROM orders ORDER BY order_id;
UPDATE orders SET status = 'shipped' WHERE order_id = 103;     -- Arjun's order
UPDATE orders SET sales_rep_id = 8 WHERE order_id = 101;       -- hand her order to Arjun (in a DO block)
```

```
 current_user | order_id | sales_rep_id |  status
--------------+----------+--------------+-----------
 ch14_priya   |      101 |            7 | delivered
 ch14_priya   |      105 |            7 | cancelled
 ch14_priya   |      108 |            7 | delivered
 ch14_priya   |      113 |            7 | pending

UPDATE 0
NOTICE:  priya: new row violates row-level security policy for table "orders"
```

Role	Policy that applies	count(*) from orders
ch14_priya	rep_sees_own_orders	4
ch14_arjun	rep_sees_own_orders	2
ch14_analyst (member of reporting)	reporting_reads_all	14
ch14_app	none — has SELECT but no policy (default deny)	0
postgres (superuser / owner)	bypasses RLS	14

- **USING** filters which *existing* rows are visible (to SELECT, UPDATE, DELETE) — so updating order 103 found 0 rows rather than failing.
- **WITH CHECK** validates rows being *written* (INSERT, UPDATE) — so Priya can't move a row out of her view.
- Policies are **permissive** by default (combined with OR); AS RESTRICTIVE ones combine with AND.

Multi-tenant isolation with a session setting

SaaS apps (one hosted app shared by many customer organisations) usually connect as **one** application role and tell the database the current request's tenant:

```
ALTER TABLE notes ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON notes
    USING (tenant = current_setting('app.tenant', true));    -- true: unset -> NULL, not an error

SET ROLE ch14_app;
SET app.tenant = 'acme';    SELECT * FROM notes;          -- only acme's 2 notes
SET app.tenant = 'globex';  SELECT * FROM notes;          -- only globex's note
RESET app.tenant;           SELECT count(*) FROM notes;   -- 0: no tenant, no rows
```

The app runs `SET LOCAL app.tenant = …` per transaction — `SET LOCAL` resets at commit, so it can't leak to the next request on a pooled connection (14.12).

> **Common mistakes:** testing RLS as a superuser or the table owner (both bypass it — test with `SET ROLE`; `ALTER TABLE … FORCE ROW LEVEL SECURITY` filters the owner too); letting the app connect as superuser or owner; forgetting new tables aren't covered by old grants (`ALTER DEFAULT PRIVILEGES … GRANT …`); leaving the `PUBLIC` pseudo-role (every user belongs to it) with unintended rights (PG ≤ 14 lets everyone `CREATE` in the `public` schema; PG 15 removed that).

	Roles / users	Row-level security
PostgreSQL	Roles (users = roles with `LOGIN`)	`CREATE POLICY` (9.5+)
MySQL 8.0	Users + roles	None built in — views or app logic
SQL Server 2016+	Logins, users, roles	Security policies with inline predicate functions
Oracle	Users, roles	Virtual Private Database (VPD)
SQLite	No users — file-system permissions only	None

In the interview

> **30-second answer:** "I apply least privilege: group roles that hold grants, login roles as members, the app gets only the DML it needs, reporting gets read-only, and nobody uses the superuser from code. For multi-tenant data I add row-level security keyed on a session variable, test it with `SET ROLE` — never as owner — and keep a tenant_id index so the policy filter is cheap."

- *USING vs WITH CHECK?* → Visibility of existing rows vs validity of new/changed rows.
- *Why `SET LOCAL` with pooled connections?* → It ends with the transaction, so it can't leak to the next request.

TL;DR

Roles + `GRANT`/`REVOKE` for what each account may do; RLS policies for which rows it may see or write; test as the real role, never as owner.

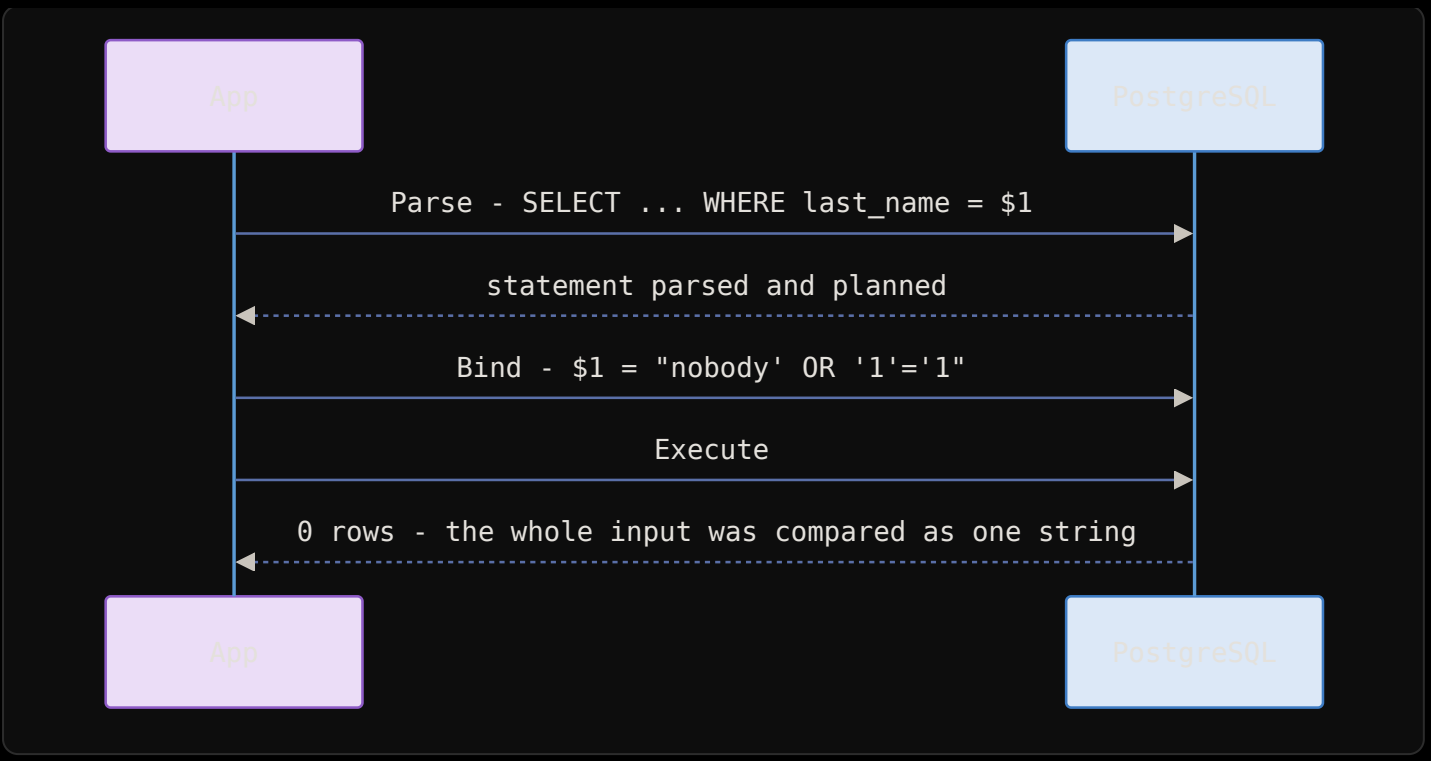
14.9 SQL injection and parameterized queries

What it is

SQL injection: user input pasted into the *text* of a SQL statement changes its meaning. It has been in the OWASP (Open Worldwide Application Security Project) Top 10 web risks since the list began and caused real breaches (the 2015 TalkTalk breach in the UK, the 2023 MOVEit Transfer attacks). If code builds `SELECT … FROM customers WHERE last_name = '<input>'`, the input `nobody' OR '1'='1` yields `… WHERE last_name = 'nobody' OR '1'='1'` — always true.

The fix — parameters

A **parameterized query** (*prepared statement*, *bind variables*) sends SQL with placeholders (`$1`, `?`, `%s`) and the values **separately**; the server parses first, then plugs values in as data, so input can never become syntax.



Example — injection inside the database (dynamic SQL)

The same bug appears in PL/pgSQL when `EXECUTE` runs a built string. From `code/09_sql_injection.sql`:

```
-- VULNERABLE
CREATE FUNCTION find_customers_bad(p_last_name text)
RETURNS TABLE (customer_id int, first_name text, email text)
LANGUAGE plpgsql AS $$
BEGIN
    RETURN QUERY EXECUTE
        'SELECT customer_id, first_name, email FROM customers WHERE last_name = '''
        || p_last_name || '''';
END;
$$;
SELECT * FROM find_customers_bad('Carter');                -- 1 row: Ben
SELECT * FROM find_customers_bad($$nobody' OR '1'='1$$);     -- 10 rows: every customer leaks

-- SAFE: the value is passed with USING, referenced as $1
CREATE FUNCTION find_customers_good(p_last_name text) ... AS $$
BEGIN
    RETURN QUERY EXECUTE
        'SELECT customer_id, first_name, email FROM customers WHERE last_name = $1'
        USING p_last_name;
END;
$$;
SELECT count(*) AS rows_leaked FROM find_customers_good($$nobody' OR '1'='1$$);   -- 0
```

Identifiers can't be parameters — placeholders stand for *values*, never table or column names. For a dynamic name, check an **allow-list** and quote with `format('%I')` (identifier; `%L` for a literal):

```
SELECT format('SELECT * FROM %I WHERE name = %L', 'Order Items', $$O'Brien$$);
--  SELECT * FROM "Order Items" WHERE name = 'O''Brien'
```

The file's `count_rows(p_table)` rejects any table outside `('orders','customers','products')` before building its query. SQL-level prepared statements work the same way: `PREPARE orders_of(int) AS … WHERE customer_id = $1; EXECUTE orders_of(2);`. The client-side versions (Python f-string vs `%s`, Java concatenation vs `PreparedStatement`, 14.10–14.11) all show **10 rows vs 0 rows**.

Defence in depth

Layer	Defence
Query	Parameters for every value; allow-list + `%I` / `quote_ident` for identifiers
Account	Least-privilege role (14.8) — injection can't drop tables the role can't touch
Data	RLS for tenant isolation
Code	Static analysis that flags string-built SQL; code review
Errors	Don't show raw database errors to users (they help map the schema)

Escaping by hand (doubling quotes yourself) is fragile — character encodings and edge cases have broken it repeatedly. Use parameters.

In the interview

> **30-second answer:** "Injection is user input interpreted as SQL because it was concatenated into the query. The fix is parameterized queries — values travel separately and can't change the parse tree. For dynamic identifiers use an allow-list and identifier quoting. Add least privilege so a bug has limited blast radius. ORMs (14.13) parameterize by default, but their raw-SQL escape hatches are still injectable."

- *Do prepared statements help performance too?* → Yes — the server can reuse the parsed/planned statement.
- *Is `LIKE ?` safe?* → From injection yes, but `%` and `_` in the input still act as wildcards; escape them if they should be literal.

TL;DR

Never build SQL from input strings; bind values as parameters, allow-list identifiers, and limit what the database account can do.

14.10 Using SQL from Python

Drivers and the DB-API

Python drivers follow **DB-API 2.0** (PEP 249, a Python design document): `connect()` → a **connection** (a session) → **cursors** (run statements, hold results) → `execute()`, `fetchone()`, `fetchall()`, `commit()`, `rollback()`.

	`psycopg` (version 3)	`sqlite3`
Database	PostgreSQL server	SQLite file or in-memory DB, in-process (no separate server)
Install	`pip install "psycopg[binary]"` (already in `.venv-books`)	Built into Python
Placeholder	`%s` or `%(name)s` (*never* Python `%` formatting)	`?` or `:name`
Transactions	Implicit `BEGIN` unless `autocommit=True`; `with conn.transaction():` blocks	Implicit transaction before DML; `with conn:` commits/rolls back

`sqlite3` basics: Python ch. 14.

Example — psycopg against the private server

Key parts of `code/psycopg_demo.py` (creates database `ch_14_code` and loads the shared schema):

```python
SERVER = "host=/tmp port=5544 user=postgres"       # Unix socket directory + port
DSN = SERVER + " dbname=ch_14_code"

# autocommit=True: each statement commits on its own, and conn.transaction() blocks are
# real BEGIN ... COMMIT/ROLLBACK units
with psycopg.connect(DSN, autocommit=True) as conn:
    # 1. parameters: values are sent separately from the SQL text
    cur = conn.execute(
        "SELECT first_name, city FROM customers WHERE country = %s AND signup_date >= %s ORDER BY customer_id",
        ("India", "2025-01-01"))
    for row in cur.fetchall():
        print("  ", row)
    with conn.cursor(row_factory=dict_row) as cur:              # rows as dicts
        cur.execute("SELECT name, unit_price FROM products WHERE unit_price > %(min)s ORDER BY unit_price DESC",
                    {"min": 300})

    # 2. injection: f-string vs parameter; identifiers via psycopg.sql
    evil = "nobody' OR '1'='1"
    conn.execute(f"SELECT count(*) FROM customers WHERE last_name = '{evil}'")      # NEVER
    conn.execute("SELECT count(*) FROM customers WHERE last_name = %s", (evil,))
    q = sql.SQL("SELECT {} FROM customers WHERE customer_id = %s").format(sql.Identifier("city"))

    # 3. a transaction: both updates or neither
    try:
        with conn.transaction():
            conn.execute("UPDATE products SET unit_price = unit_price * 0.9 WHERE category = 'Furniture'")
            conn.execute("UPDATE products SET unit_price = -1 WHERE product_id = 2")   # CHECK fails
    except psycopg.errors.CheckViolation as e:
        print("rolled back:", e.diag.message_primary)

    # 4. batch (executemany), bulk load (cur.copy("COPY ... FROM STDIN") + write_row),
    #    and a named = server-side cursor that streams rows in chunks
    with conn.transaction(), conn.cursor(name="big_scan") as cur:
        cur.itersize = 4                                        # rows per round trip
        cur.execute("SELECT g FROM generate_series(1, 10) g")
        print("streamed:", [r[0] for r in cur])
```

```
connected to ch_14_code on port 5544 | server 14

== 1. query, fetch, parameters ==
customers: 10
   ('Chen', 'Bengaluru')
   ('Divya', 'Delhi')
   ('Fatima', None)
   ('Anita', 'mumbai')
   ('Ivan', 'Delhi')
dict rows: [{'name': 'Laptop Pro 14', 'unit_price': Decimal('1299.00')}, ...]   (3 dicts, shortened here)

== 2. SQL injection: string building vs parameters ==
built SQL : SELECT count(*) FROM customers WHERE last_name = 'nobody' OR '1'='1'
f-string  : 10 rows
parameter : 0 rows
composed  : SELECT "city" FROM customers WHERE customer_id = %s -> Bengaluru

== 3. transactions ==
rolled back: new row for relation "products" violates check constraint "products_unit_price_check"
desk price after rollback: 499.00
desk price after commit: 479.00

== 4. batch insert, COPY, streaming ==
executemany rowcount: 3
customers now: 16
streamed: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

== 5. connection pooling ==
no pool : 20 requests used 20 server processes
pool(2) : 20 requests used 2 server processes
```

- **Type mapping**: `NULL` → `None`, `numeric` → `Decimal` (exact — never `float` for money), `date` → `datetime.date`.
- **Transactions**: the 10 % furniture discount rolled back with the failing update (desk still 499.00). Exceptions are typed by SQLSTATE (`CheckViolation`, `UniqueViolation`, …).
- **Bulk loading**, fastest last: one `execute` per row → `executemany` (psycopg 3 pipelines it, not waiting for each reply) → `COPY` (the bulk-load command).
- **Server-side cursor** (`conn.cursor(name=…)`): the result stays on the server and arrives in `itersize` chunks, so a 10-million-row export needs little memory.
- **Default transaction trap**: without `autocommit=True` psycopg sends `BEGIN` before your first statement and saves nothing until `conn.commit()` (or a normal exit from `with psycopg.connect(...)`). A forgotten commit, or a connection left "idle in transaction", is a classic bug.
- **Mistake:** `cur.execute("… = %s" % value)` is Python string formatting — injection again; pass values as the second argument.

Example — sqlite3

`code/sqlite3_demo.py` — same ideas, `?` placeholders, no server:

```
conn = sqlite3.connect(":memory:")
conn.row_factory = sqlite3.Row
conn.execute("PRAGMA foreign_keys = ON")    # SQLite leaves FK checks off unless asked
...
with conn:                                   # commit on success, rollback on exception
    conn.executemany("INSERT INTO products (name, price, specs) VALUES (?, ?, ?)", rows)
for r in conn.execute("SELECT product_id, name, price FROM products WHERE price < :max ORDER BY price", {"max":
40}):
    print(dict(r))
```

```
{'product_id': 1, 'name': 'Wireless Mouse', 'price': 24.99}
{'product_id': 3, 'name': 'Desk Lamp', 'price': 35.0}
rolled back: UNIQUE constraint failed: products.name
hub price: 49.5
f-string  rows: 3
parameter rows: 0
sqlite 3.53.4 | brands: ['Clicko', 'Nimbus']
```

(The version line shows the SQLite bundled with this Python.) `with conn:` in `sqlite3` manages the **transaction**, not closing — call `conn.close()` yourself.

14.11 Using SQL from Java (JDBC)

What it is

JDBC (Java Database Connectivity) is Java's standard database API (`java.sql`); a vendor **driver** jar implements it — for PostgreSQL `org.postgresql:postgresql` (42.7.13 here, from Maven Central).

Type	Role
`DriverManager` / `DataSource`	Hands out connections (`DataSource` is how pools plug in)
`Connection`	A session; controls autocommit, commit/rollback, isolation level
`PreparedStatement`	SQL with `?` placeholders, typed setters (`setString`, `setObject`, `setBigDecimal`)
`ResultSet`	A cursor over rows; `next()` + typed getters
`SQLException`	Errors, with `getSQLState()` (e.g. `23514` = check violation)

Resources close via **try-with-resources** so connections never leak. Java side: ch. 8 exceptions, ch. 18 tooling & JDBC.

Running it. The jar lives in `.java-books/lib/`. The book's server listens only on a Unix socket (`/tmp/.s.PGSQL.5544`), so the URL adds the `junixsocket` socket factory (two more jars in that folder); against a normal server it is just `jdbc:postgresql://localhost:5544/ch_14_jdbc`. The `java` launcher compiles and runs a single source file (source-file mode, Java 11+; the book targets Java 25); the file starts with `// tools_run: skip` so the Java book's runner skips it:

```
java --enable-native-access=ALL-UNNAMED -cp '.java-books/lib/*' \
     sql/chapters/14-advanced-sql-and-using-sql-from-code/code/JdbcDemo.java
```

Example — key parts of `code/JdbcDemo.java`

```
// PreparedStatement: '?' placeholders, typed setters and getters
String sql = """
        SELECT first_name, city, signup_date
        FROM customers WHERE country = ? AND signup_date >= ? ORDER BY customer_id""";
try (PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.setString(1, "India");
    ps.setObject(2, LocalDate.of(2025, 1, 1));              // java.time maps to DATE
    try (ResultSet rs = ps.executeQuery()) {
        while (rs.next()) {
            String city = rs.getString("city");              // SQL NULL -> Java null
            LocalDate d = rs.getObject("signup_date", LocalDate.class);
            ...
        }
    }
}

// Transaction: turn auto-commit off, commit or roll back as one unit
conn.setAutoCommit(false);
conn.setTransactionIsolation(Connection.TRANSACTION_REPEATABLE_READ);
try (PreparedStatement up = conn.prepareStatement(
        "UPDATE products SET unit_price = unit_price + ? WHERE product_id = ?")) {
    up.setBigDecimal(1, new BigDecimal("-10.00")); up.setInt(2, 5); up.executeUpdate();
    up.setBigDecimal(1, new BigDecimal("-99.00")); up.setInt(2, 2); up.executeUpdate(); // CHECK fails
    conn.commit();
} catch (SQLException e) {
    conn.rollback();                                         // undo BOTH updates
} finally {
    conn.setAutoCommit(true);
}

// Batch: setXxx(...) + ps.addBatch() per row, then int[] counts = ps.executeBatch(); conn.commit();
```

```
connected: PostgreSQL 14.15 (Homebrew) via PostgreSQL JDBC Driver 42.7.13

== PreparedStatement ==
  Chen   Bengaluru  2025-01-08
  Divya  Delhi      2025-01-20
  Fatima (null)     2025-02-14
  Anita  mumbai     2025-03-09
  Ivan   Delhi      2025-05-30

== SQL injection ==
concatenated : 10 rows
parameterized: 0 rows

== transaction ==
rolled back, SQLState 23514: ERROR: new row for relation "products" violates check constraint
"products_unit_price_check"
desk price unchanged: 499.00

== batch ==
batch update counts: [1, 1, 1]
  Monitor Arm -> 61.95
  Webcam HD -> 83.90
  Cable Tray -> 20.48
```

The last block used `UPDATE … RETURNING name, unit_price` with `executeQuery()` — `RETURNING` gives back computed values without a second query (`getGeneratedKeys()` is the portable alternative for new ids).

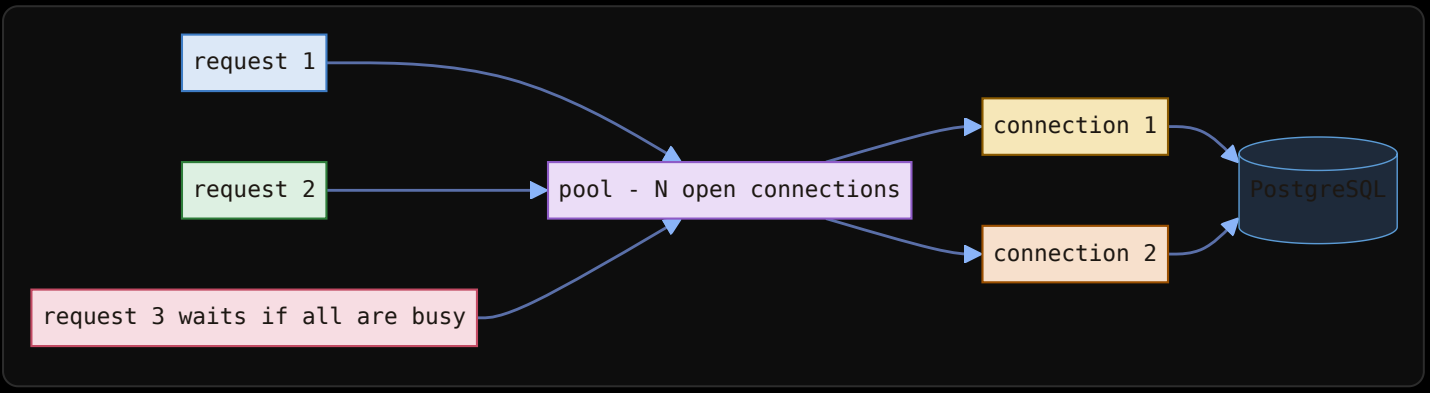
Common mistakes (Python and Java alike)

- **Leaking connections** — not closing them (`with` / try-with-resources); the server runs out of slots.
- `float` / `double` **for money** — use `Decimal` / `BigDecimal`.
- `getInt` **returns** `0` **for SQL** `NULL` — use `getObject(col, Integer.class)` or check `rs.wasNull()`.
- **Autocommit surprises** — JDBC starts with autocommit *on*; psycopg *off*. Know which you're in.
- **One query per row** in a loop (N+1, 14.13) instead of a join or `= ANY(?)` with an array (`conn.createArrayOf("int", ids)`).

14.12 Connection pooling

What it is

Opening a PostgreSQL connection is expensive: network handshake, TLS (encryption), authentication, and — PostgreSQL uses a **process per connection** — forking a new server process with its own memory. That costs milliseconds; a simple indexed query takes well under one. A **connection pool** opens a few connections once and **lends** them to requests.



`psycopg_demo.py`'s last section uses a 15-line `TinyPool` (a `queue.Queue` of open connections): 20 requests without a pool created **20 server processes**; with a pool of 2, **2**.

How it works

1. At start-up the pool opens *min* connections.
2. A request **borrows** one (`getConnection()` / `pool.connection()`), waiting up to a timeout if all are busy.
3. It **returns** it; the pool resets session state (rolls back an open transaction, may run a validation query) and keeps it open.
4. Connections idle too long or past a maximum lifetime are closed.

Where	Examples	Notes
In the application	HikariCP (Java; Spring Boot's default), `psycopg_pool`, SQLAlchemy's pool, node-postgres `Pool`	Per process; total = processes × pool size
External proxy between apps and database	PgBouncer, pgcat, Amazon RDS Proxy	Shares a few server connections among many app processes

PgBouncer's **transaction mode** lends a server connection only for one transaction — very efficient, but session features (plain `SET`, session advisory locks, `LISTEN` notifications) don't carry across transactions; use `SET LOCAL` (14.8).

Sizing. More connections is **not** faster: past a point processes compete for cores and disk, and each costs memory (`max_connections` defaults to 100). HikariCP's widely quoted start: *(CPU cores × 2) + effective disk spindles* (physical disks working in parallel), then measure. If the pool's **wait time** grows, look for slow queries before raising the size.

> **Common mistakes:** holding a connection during slow non-database work (an external API call mid-transaction); session state leaking between borrowers (`SET` without `LOCAL`, temp tables, open transactions); every microservice replica with a pool of 50 → thousands of connections (use an external pooler).

14.13 ORMs vs raw SQL

What it is

An **ORM** (object-relational mapper) maps tables to classes and rows to objects and generates SQL: Hibernate/JPA (Java), SQLAlchemy and the Django ORM (Python), Entity Framework (.NET), ActiveRecord (Ruby), Prisma (TypeScript). **Query builders** and **mappers** (jOOQ, MyBatis, Knex, SQLAlchemy Core) help write SQL without hiding it. The clash between object graphs and tables is the **object-relational impedance mismatch**.

	ORM	Raw SQL / query builder
CRUD code	Very little	More boilerplate
Safety	Parameterized by default	Parameterized if you do it (always do)
Complex queries (windows, CTEs, pivots, JSONB, FTS)	Awkward or impossible → raw-SQL escape hatch	Full power of the database
Performance visibility	SQL hidden; easy to generate N+1 or huge joins	You see and tune every query
Portability	Easier to switch databases	Tied to a dialect (usually fine)
Learning curve	ORM concepts (sessions, lazy loading, dirty checking — tracking which objects changed) *plus* SQL	SQL
Schema changes	Often paired with migrations (versioned schema-change scripts: Alembic, Flyway, Liquibase — usable without an ORM)	Same migration tools

The N+1 query problem

The classic ORM trap (measured in §12.13): load 100 orders (1 query), touch `order.customer` for each, and **lazy loading** (fetch related objects on first access) fires one query per order:

```sql
-- N+1: what lazy loading does
SELECT * FROM orders WHERE status = 'shipped';          -- 1 query
SELECT * FROM customers WHERE customer_id = 6;          -- + 1 per order ...
SELECT * FROM customers WHERE customer_id = 7;
-- fix: one join (ORM: eager loading, e.g. JOIN FETCH / joinedload / select_related)
SELECT o.*, c.first_name FROM orders o JOIN customers c USING (customer_id) WHERE o.status = 'shipped';
-- or one batched lookup: WHERE customer_id = ANY($1)   (ORM: batch fetching / selectinload / prefetch_related)
```

Spot it by logging SQL in development and counting queries per request.

When to use which: ORM for CRUD-heavy apps with many simple entities (still reading the SQL it generates); raw SQL / builder for reporting, analytics, bulk jobs, window functions, CTEs, JSONB, FTS and hot paths that need tuning. **Most real systems mix both**: ORM for everyday writes, hand-written SQL for the hard 10 %.

In the interview

> **30-second answer:** "ORMs speed up CRUD and parameterize by default, but hide the SQL, which invites N+1 queries and inefficient joins. I use an ORM for simple entity work, drop to SQL or a query builder for reporting and performance-critical paths, log generated SQL, and fix N+1 with eager or batched loading."

- *How do you detect N+1?* → Count queries per request in logs/APM (application performance monitoring); the same query shape 100 times in one request is the tell.
- *Are ORMs immune to injection?* → Their generated queries are; raw-SQL helpers with string concatenation are not.

Chapter TL;DR

Topic	Remember
Functions	`sql` for simple, `plpgsql` for procedural; honest volatility; set-based beats per-row calls
Procedures	`CALL`, `INOUT`, can `COMMIT` in batches — only from autocommit mode
Triggers	BEFORE modifies/rejects, AFTER reacts; audit with `to_jsonb(OLD/NEW)`; statement-level + transition tables for bulk; no slow external work
JSONB	`->` json, `->>` text, `@>` contains, GIN index; core fields stay columns
Arrays	1-based; `ANY`, `@>`, `&&`, `unnest`, `array_agg`; `= ANY($1)` for id lists from code
Full-text search	`tsvector @@ tsquery`, stemming, `websearch_to_tsquery`, `ts_rank`, weighted generated column + GIN
Pivot	`FILTER` / `CASE` aggregates; `crosstab` needs tablefunc; unpivot with `LATERAL (VALUES …)`
Security	Group roles, least privilege, `GRANT` / `REVOKE`; RLS `USING` (read) + `WITH CHECK` (write); owners/superusers bypass
Injection	Never concatenate input; bind parameters; allow-list + `%I` for identifiers
From code	psycopg `%s`, sqlite3 `?`, JDBC `?`; explicit transactions; `Decimal` / `BigDecimal`; batch/COPY; stream big results
Pooling	Reuse connections; small pools; `SET LOCAL`; PgBouncer for many clients
ORMs	Productive for CRUD, watch for N+1; raw SQL for complex/hot queries

Next: Chapter 15 — Scaling databases, which takes one server to many: partitioning, replication, sharding and failover.

Chapter 15 — Scaling Databases: Partitioning, Sharding, Replication and Recovery

One PostgreSQL server with plenty of RAM, fast SSDs and good indexes (Chapter 12) can serve thousands of queries per second. Sooner or later, though, the data outgrows one disk, the traffic outgrows one CPU, or the business can't accept that one broken machine takes the product offline. This chapter is about what databases do when **one box is no longer enough**. Two ideas carry most of the weight:

- **Partitioning / sharding**: split the data so each piece is smaller (more capacity, faster scans).
- **Replication**: copy the data so each piece exists more than once (availability, read scaling, disaster recovery).

Then come the problems they create: copies disagree (consistency models, CAP), machines must agree (consensus, leader election), and failures must not lose committed data (logging, checkpoints, two-phase commit, failover, backups).

Section	Topic	Runnable part
15.1	What a distributed database is, scale up vs scale out	—
15.2	Partitioning: horizontal/vertical, range/list/hash/composite/round-robin	Postgres declarative partitioning + pruning with `EXPLAIN`
15.3	Sharding, shard keys, consistent hashing	Python: keys moved when a shard is added
15.4	Distributed query processing	`postgres_fdw` : predicate/aggregate push-down
15.5	Replication: types, sync vs async, single/multi/leaderless	Python: quorum reads/writes
15.6	Consistency models	—
15.7	CAP and PACELC	—
15.8	Consensus (Paxos, Raft, PBFT, PoW) and leader election	Python: Bully vs Ring message counts
15.9	Fault tolerance: WAL, redo/undo, LSNs, checkpoints, 2PC/3PC	Postgres WAL/LSN demo, Python recovery + 2PC
15.10	High availability and failover	—
15.11	Data resilience: redundancy, mirroring, backups	—

> **Going deeper.** This chapter explains each idea once, at interview depth. Book-level depth: DDIA notes — Replication, Partitioning, Trouble with distributed systems, Consistency and consensus. Design-interview patterns: Storing data (primary-replica, sharding, consistent hashing, WAL), Consistency (2PC, saga, quorum, vector clocks), Scaling, Reliability.

SQL files in `code/` run on the book's PostgreSQL 14 server (`bash tools_run_sql.sh sql '15-*'`); `00_schema.sql` creates 12 customers in three regions (`NA` , `EU` , `APAC`) and 3,000 orders over 2022–2024. Python simulations: `.venv-books/bin/python code/<file>.py` .

15.1 Distributed databases: the big picture

What it is

A **distributed database** stores data on several machines (**nodes**), possibly in different **data centres**, but presents itself to applications as *one* database.

Term / characteristic	Meaning
Node	One server (physical or virtual) running a database process. **Cluster** = the group of nodes forming the database.
Transparency	The client needn't know *where* a row is (**location transparency**) or *how many copies* exist (**replication transparency**).
Fault tolerance	Keeps working when nodes fail, because data is copied elsewhere.
Concurrency	Many transactions on many nodes run at once without corrupting shared data (Chapter 13's concurrency control, across machines).
Scalability	Capacity grows by adding nodes.
Network partition	Some nodes can't talk to others because the network between them failed — not to be confused with *data* partitioning (15.2).

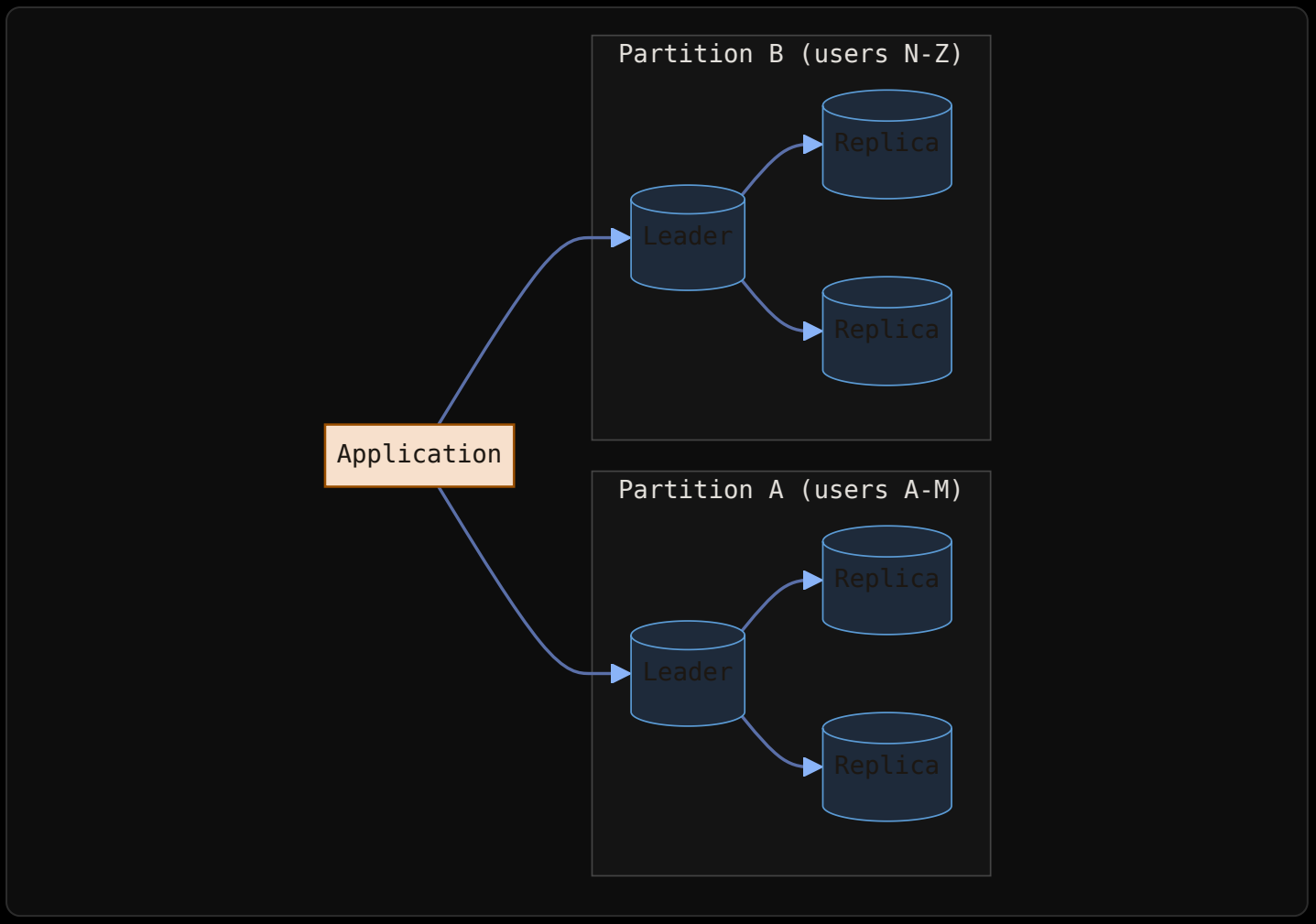
Scale up vs scale out

	Vertical scaling (scale up)	Horizontal scaling (scale out)
How	Bigger machine: more CPU, RAM, faster disks	More machines, each holding part of the data/load
Example	Move Postgres from 8 to 64 cores for Black Friday	Add nodes to a Cassandra or CockroachDB cluster as users grow
Limit	The biggest machine you can buy (and its price)	Coordination cost between nodes
Failure	Still one machine = one point of failure	Survives node loss if data is replicated
App changes	None	Often needed (shard keys, cross-node queries)

Distributed databases are built for **scale-out**, but scale-up is simpler and usually first: many companies run a large Postgres or MySQL **primary** (the one server accepting writes) plus **read replicas** (copies serving reads) for years before sharding (System Design Fundamentals). Scale-up's limits are cost and a **single point of failure** (one component whose failure takes everything down).

Benefits	Challenges
High availability — other copies serve if a node or region dies	**Consistency** of copies written in different places (15.5–15.7)
Lower latency — data near its users (Singapore reads a Singapore node)	**Network latency** — milliseconds per cross-region hop
Load spreading — no node takes every request	**Failure detection** — silent may mean dead or slow (15.8–15.10)
Capacity grows with node count	**Operational complexity** — harder debugging and upgrades

How scale-out works: partition *and* replicate



Partitioning gives each node a *subset* of the data (capacity, parallel work); **replication** stores each subset on *several* nodes (fault tolerance, read capacity); most real systems do both. *Example:* a food-delivery app in 20 countries partitions menus and orders by country (queries are local, and data-residency laws such as **GDPR**, the EU's personal-data regulation, are easier to meet), replicates each partition to three **availability zones** (separate data centres in one cloud region with independent power and networking), and adds nodes to the busiest cities on New Year's Eve.

15.2 Partitioning

What it is

Partitioning splits one large table into smaller pieces (**partitions**) stored and managed separately while queries still see one logical table. Goals: **faster queries** (March 2023 reads only the 2023 partition — **partition pruning**), **easier maintenance** (archive a year by detaching one partition), **scalability** (partitions on different nodes = *sharding*, 15.3), **fault tolerance** (each partition replicated independently).

Aspect	**Horizontal** (split rows)	**Vertical** (split columns)
Each piece has	All columns, some rows (2023 orders, 2024 orders)	Some columns, all rows; pieces share the primary key (1:1)
Speeds up	Queries that filter on the partition key	Queries that need only a few columns
Cost	Routing; queries without the key hit every piece	Joins to reassemble a full row
Typical use	Huge event/order/log tables, distributed DBs	Wide tables with big text/JSON/blob columns

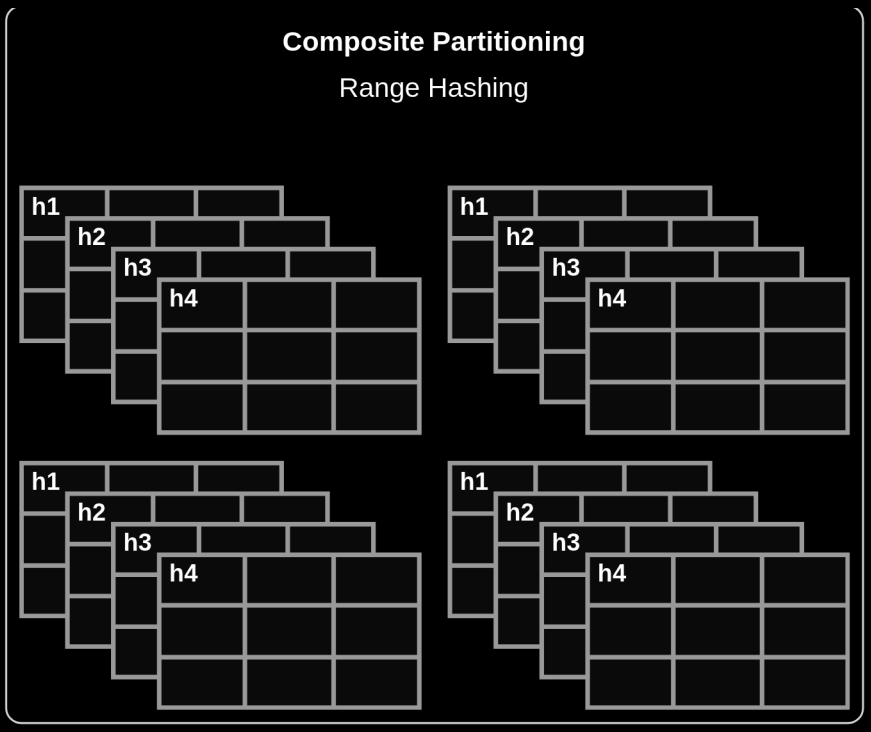
A vertical split (`code/05_vertical_partitioning.sql`) keeps listing-page columns (`product_id`, `name`, `price`) in `product_core` and the long `description` / `spec_sheet` in `product_details` (PK also referencing `product_core`). For 1,000 products, `pg_total_relation_size`:

```
 core_size | details_size
-----------+--------------
 128 kB    | 1672 kB
```

The listing page reads 128 kB instead of ~1.8 MB; a view (`product_full`) joins the halves back. Postgres already moves very large values out of line (**TOAST**, §12.1), so a manual split pays off most for medium-sized, rarely read columns.

Partitioning strategies

Strategy	Rule	Good for	Watch out for
Range	Key in an interval, e.g. `2023-01-01 ≤ date < 2024-01-01`	Time data, "last N days", archiving	**Hot partition**: all new writes hit the newest range
List	Key in a set of values, e.g. `region IN ('EU')`	Regions, tenants, data-residency rules	Uneven sizes; new values need a partition (or DEFAULT)
Hash	`hash(key) mod N`	Even spread with no natural range (`order_id`, `user_id`)	Range queries can't be pruned; changing N moves data
Composite	Two levels, e.g. range by month, then hash by server	High-volume time series	More partitions to manage
Round-robin	Row 1 → P1, row 2 → P2, row 3 → P3, row 4 → P1…	Perfectly even bulk loads and parallel scans	Can't find a row by key — every lookup scans all partitions



Postgres supports range, list and hash natively, composite by nesting; no round-robin (that's in warehouse systems, e.g. Azure Synapse `ROUND_ROBIN`). **To choose:** the key should be in the `WHERE` of your most common and most expensive queries (so pruning works), spread data reasonably evenly, and keep adding/dropping partitions simple.

How it works in PostgreSQL (declarative partitioning)

Since PostgreSQL 10 (hash partitioning, default partitions and PKs on partitioned tables in 11): a **parent** declared `PARTITION BY RANGE | LIST | HASH (columns)` stores no rows; each **partition** is `CREATE TABLE … PARTITION OF parent FOR VALUES …`. Inserts are routed (a row fitting none is rejected unless a `DEFAULT` partition exists), and a `WHERE` on the key lets the planner **prune** partitions that can't match.

Range partitioning, with pruning

`code/01_range_partitioning.sql`:

```
CREATE TABLE orders_by_year (
  order_id    int  NOT NULL,
  customer_id int  NOT NULL,
  order_date  date NOT NULL,
  amount      numeric(10,2) NOT NULL,
  PRIMARY KEY (order_id, order_date)          -- a PK must include the partition key
) PARTITION BY RANGE (order_date);

CREATE TABLE orders_2022 PARTITION OF orders_by_year FOR VALUES FROM ('2022-01-01') TO ('2023-01-01');
CREATE TABLE orders_2023 PARTITION OF orders_by_year FOR VALUES FROM ('2023-01-01') TO ('2024-01-01');
CREATE TABLE orders_2024 PARTITION OF orders_by_year FOR VALUES FROM ('2024-01-01') TO ('2025-01-01');
CREATE TABLE orders_other PARTITION OF orders_by_year DEFAULT;   -- anything outside the ranges

INSERT INTO orders_by_year SELECT * FROM orders;
INSERT INTO orders_by_year VALUES (9001, 1, '2025-03-15', 99);   -- no 2025 partition -> DEFAULT

SELECT tableoid::regclass AS partition, count(*) AS rows,
       min(order_date) AS first_day, max(order_date) AS last_day
FROM orders_by_year GROUP BY 1 ORDER BY 1;
```

```
  partition   | rows | first_day  |  last_day
--------------+------+------------+------------
 orders_2022  | 1016 | 2022-01-01 | 2022-12-31
 orders_2023  |  990 | 2023-01-01 | 2023-12-31
 orders_2024  |  994 | 2024-01-01 | 2024-12-31
 orders_other |    1 | 2025-03-15 | 2025-03-15
```

`FROM … TO …` has an **inclusive** lower and **exclusive** upper bound (so `2023-01-01` is in `orders_2023`); `tableoid::regclass` is a hidden column naming the physical table holding each row; the PK includes `order_date` because each partition enforces uniqueness only on its own rows. Pruning at plan time, versus a query not filtering on the key:

```
EXPLAIN (COSTS OFF) SELECT sum(amount) FROM orders_by_year
WHERE order_date >= '2023-03-01' AND order_date < '2023-04-01';
EXPLAIN (COSTS OFF) SELECT sum(amount) FROM orders_by_year WHERE customer_id = 7;
```

```
Aggregate
  ->  Seq Scan on orders_2023 orders_by_year
        Filter: ((order_date >= '2023-03-01'::date) AND (order_date < '2023-04-01'::date))

Aggregate
  ->  Append
        ->  Seq Scan on orders_2022 orders_by_year_1
              Filter: (customer_id = 7)
        ...   (same for orders_2023, orders_2024 and orders_other)
```

`Append` = "scan each child and concatenate" — all four read. **Partitioning only helps queries that filter on the partition key.** When the value is a prepared-statement parameter (known only at run time), Postgres 11+ prunes during execution:

```
SET plan_cache_mode = force_generic_plan;
PREPARE by_day(date) AS SELECT count(*) FROM orders_by_year WHERE order_date = $1;
EXPLAIN (COSTS OFF) EXECUTE by_day('2023-05-05');
```

```
Aggregate
  ->  Append
        Subplans Removed: 3
        ->  Seq Scan on orders_2023 orders_by_year_1
              Filter: (order_date = $1)
```

Maintenance is where range partitioning shines — archiving a year is a metadata change, not a giant `DELETE`:

```
ALTER TABLE orders_by_year DETACH PARTITION orders_2022;        -- instant; now a plain table
DELETE FROM orders_other WHERE order_date >= '2025-01-01' AND order_date < '2026-01-01';
CREATE TABLE orders_2025 PARTITION OF orders_by_year FOR VALUES FROM ('2025-01-01') TO ('2026-01-01');
INSERT INTO orders_by_year VALUES (9001, 1, '2025-03-15', 99);
```

Afterwards the parent holds `orders_2023` (990), `orders_2024` (994), `orders_2025` (1); the detached `orders_2022` keeps its 1,016 rows as an ordinary table. The `DELETE` is needed because Postgres refuses to create `orders_2025` while DEFAULT still holds 2025 rows — create partitions ahead of time (a scheduled job or the `pg_partman` extension). `DETACH PARTITION … CONCURRENTLY` (Postgres 14+) avoids blocking queries on the parent.

List, hash and composite

`code/02_list_partitioning.sql` splits customers by region (`FOR VALUES IN ('NA')`, `('EU')`, `('APAC')`):

```
   partition    |         customers
----------------+--------------------------
 customers_na   | Ben, Elena, Grace, Jack
 customers_eu   | Chloe, Dmitri, Ines, Liam
 customers_apac | Asha, Farid, Hiro, Kavya

 Seq Scan on customers_eu customers_by_region
   Filter: (region = 'EU'::text)

NOTICE:  rejected: no partition of relation "customers_by_region" found for row
```

The last line: a `'LATAM'` customer with no partition and no `DEFAULT` is rejected — a safety net against data landing in the wrong jurisdiction.

`code/03_hash_partitioning.sql` spreads orders over four partitions (`FOR VALUES WITH (MODULUS 4, REMAINDER 0..3)`): 730 / 752 / 774 / 744 rows. `WHERE order_id = 2024` prunes to one (`Seq Scan on orders_h2`), but `order_id BETWEEN 100 AND 110` scans all four — hashing scatters neighbours.

`code/04_composite_partitioning.sql` partitions server logs by month, each month `PARTITION BY HASH (server_id)` with two leaves (`MODULUS 2, REMAINDER 0/1`):

```
 leaf_partition  | rows | servers
-----------------+------+---------
 logs_2024_01_h0 |  403 | 1,2
 logs_2024_01_h1 |  804 | 3,4,5,6
 logs_2024_02_h0 |  265 | 1,2
 logs_2024_02_h1 |  529 | 3,4,5,6

 Aggregate
   ->  Seq Scan on logs_2024_02_h1 server_logs
         Filter: ((logged_at >= '2024-02-10 00:00:00+00'::timestamp with time zone) AND
                  (logged_at < '2024-02-11 00:00:00+00'::timestamp with time zone) AND (server_id = 3))
```

A filter on both keys prunes to **one leaf of four**. The 2-vs-4 server split shows that **hashing evens things out only with many distinct key values**. `pg_partition_tree('server_logs')` lists parent (level 0), months (1), leaves (2).

Trade-offs and dialects

Postgres limits: unique/PK constraints must include every partition-key column; planning time grows with partition count (hundreds fine, many thousands heavy); an index on the parent becomes a *local* index per partition — no global index; an `UPDATE` changing the key moves the row (PG 11+); `enable_partitionwise_join` / `_aggregate` (off by default) process matching partitions pairwise.

Other dialects *(not run)*: **MySQL** declares `PARTITION BY RANGE/LIST/HASH/KEY (...) (PARTITION p0 VALUES LESS THAN (...), ...)` in `CREATE TABLE`, every unique key including the partitioning columns; **SQL Server** a partition function plus scheme (range only); **Oracle** adds interval partitioning (auto-created ranges); **SQLite** none.

> **Common mistakes:** a key the hot queries don't filter on; partitioning tables under a few million rows (a good index is simpler); forgetting next month's partition (inserts fail or pile into DEFAULT); thousands of tiny partitions; expecting hash partitioning to speed up range scans.

In the interview

> **30-second answer:** "Partitioning splits a big table into physical pieces by a key: range for time, list for categories, hash for even spread, composite to combine. The win is pruning — the planner skips partitions the `WHERE` clause excludes — plus cheap maintenance like dropping a month instantly. It only helps queries that filter on the key, and in Postgres unique keys must include it."

- *Range vs hash for an `events` table?* Range by time if queries are "recent events" and old data is archived; hash by `user_id` if queries are per user and write load must be spread.
- *Hot-partition problem?* Range-by-time sends all current writes to the newest partition; mitigate with time + hash composite.
- *`Subplans Removed`?* Partitions pruned at execution time because the value was a parameter.

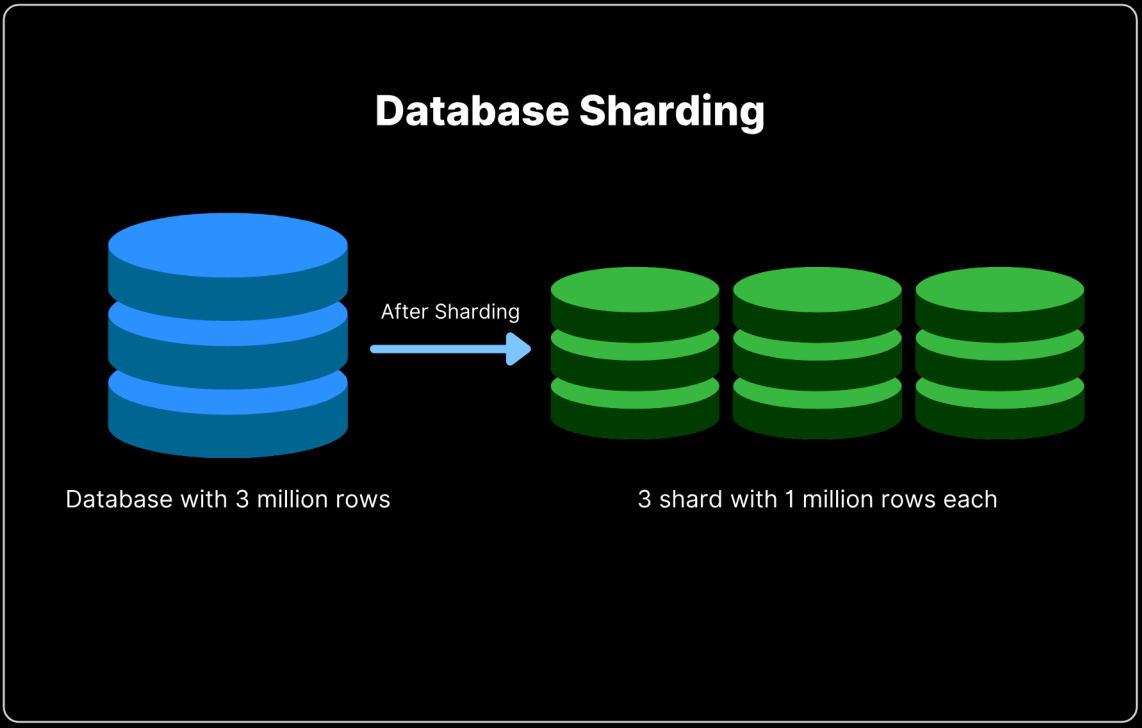
TL;DR

Partition by the column your big queries filter on: range for time, list for categories, hash for even spread, composite to combine. Check pruning with `EXPLAIN`; unique keys must include the partition key.

15.3 Sharding

What it is

Sharding is horizontal partitioning where each partition — a **shard** — lives on a **separate database server**, a complete, independent database holding a subset of rows. A **shard key** decides which shard owns a row; a **router** sends each query to the right shard(s). Example: `order_id` 1–1,000,000 on shard A, 1,000,001–2,000,000 on B; `WHERE order_id = 42` goes only to A.



How it works

1. Pick a **shard key** (`user_id`, `tenant_id`).
2. Choose a **mapping**: a range table, a hash, or a lookup directory.
3. A **router** applies it: the application, a **proxy** (a middle server forwarding client connections — Vitess for MySQL; pgcat/pgdog for Postgres), or the database itself (Citus, MongoDB `mongos`, CockroachDB).
4. Single-shard queries go to one shard; queries without the key are **scattered** to all and **gathered** — slower.
5. Each shard is **replicated** (15.5) — sharding alone doesn't remove single points of failure.

Choosing the shard key (the most important decision)

Property	Why	Example
Spreads data and load evenly	Avoids a **hotspot** — one overloaded shard sets the whole system's speed	`user_id` (millions of values), not `country`
Matches the main query pattern	Most queries hit exactly one shard	B2B SaaS sharded by `tenant_id`
Keeps related data together	Joins and transactions stay on one shard (**co-location**)	A user's orders on the user's shard
Rarely changes / minimises rebalancing	Changing the key or shard count means moving data	Immutable ids, not email addresses

Typical: `user_id` for social apps, `tenant_id` for SaaS, time ranges for time-series (but then current writes hit the newest shard). Versus partitioning in one server, sharding needs a router instead of the planner, turns cross-piece queries into network scatter-gather and cross-piece transactions into

distributed commits (2PC, 15.9), and lets one shard fail while others serve (**fault isolation**). Benefits: **write scaling**, **smaller working sets** (each shard's frequently touched data fits in RAM), fault isolation, more total capacity.

Rebalancing: why `hash % N` hurts and consistent hashing helps

With naive `hash(key) % N`, changing N from 4 to 5 remaps most keys. **Consistent hashing** puts shards and keys on a ring (0 to 2^{32}); each key belongs to the next shard clockwise, so a new shard takes keys only from its neighbour. **Virtual nodes** give each shard many small ring points to even out load (Cassandra, Riak, Dynamo-style systems). `code/consistent_hashing.py` places 100,000 keys, then adds a fifth shard; the ring is a sorted list of `(hash, shard)` points searched with `bisect`:

```
def lookup(self, key):
    i = bisect.bisect(self.hashes, h(key)) % len(self.points)  # next point clockwise
    return self.points[i][1]
```

```
hash % N                    moved  79.7%   max/min shard load 1.01
ring, 1 point per shard     moved   2.8%   max/min shard load 10.44
ring, 200 vnodes per shard  moved  17.1%   max/min shard load 1.28
ideal fraction to move when going 4 -> 5 shards: 1/5 = 20.0%
```

- `hash % N` balances perfectly but moves **80%** of keys — a huge migration.
- One point per shard moves little only because the new shard landed on a tiny arc; the biggest shard holds 10× the smallest (uneven arcs → uneven load).
- **200 vnodes** move ~17% (near the ideal 20%) and stay balanced — why real systems use them.

Alternative: a **fixed number of logical partitions** (say 1,024) mapped to servers by a directory; adding a server moves whole partitions, nothing re-hashed (roughly Citus shards, Vitess keyspace ranges, Kafka partitions). More: DDIA ch.6 — rebalancing.

Sharding Postgres in practice. Core Postgres has no built-in sharding: application-level routing, the **Citus** extension (`create_distributed_table('orders','customer_id')` co-locates tables sharing the key), partitions whose leaves are `postgres_fdw` foreign tables (15.4), or a Postgres-compatible distributed SQL database (CockroachDB, YugabyteDB — Chapter 16).

Common mistakes: monotonic key + range sharding (auto-increment id, timestamp — every new row hits the last shard); **low-cardinality keys** (country, status — too few shards, some huge); **sharding too early** (read replicas, caching — Caching patterns — and a bigger machine come first); **forgetting cross-shard needs** (global unique constraints, cross-shard joins, all-user reports become hard).

In the interview

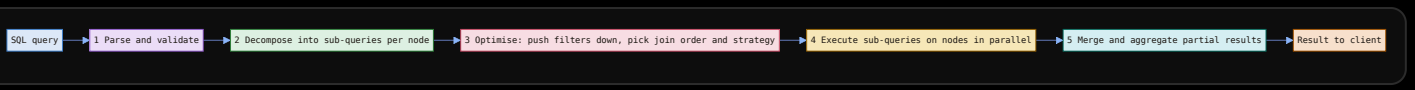
30-second answer: "Sharding puts horizontal partitions on separate servers so writes and storage scale out. The shard key should spread load evenly, match the main access pattern so most queries hit one shard, and keep related rows together. Queries without the key become scatter-gather and cross-shard transactions need 2PC. For rebalancing I'd use consistent hashing with virtual nodes, or fixed logical partitions mapped to nodes, never `hash mod N`."

- *A celebrity user overloads one shard?* Split that key (`user123#0..9`) to spread writes and merge on read, or cache their reads.
- *Unique ids across shards?* UUIDv7 (a 128-bit id generated anywhere without coordination; v7 starts with a timestamp so ids sort by time), Snowflake-style ids (timestamp + machine id + sequence), or per-shard sequences with disjoint ranges.

15.4 Distributed query processing

What it is

With rows spread over nodes, the database decides **which nodes to ask, what to ask each, and how to combine answers** — moving as little data as possible, in parallel, correctly.



Step 1 includes **data localisation** (which nodes hold each table); step 3 minimises network traffic and time given location, link speeds and node load; step 5 merges partial results (union, final sort, final sum).

Technique	Idea
Query fragmentation	Independent fragments run simultaneously on different nodes.
Predicate push-down	Apply `WHERE` on the node storing the data, so only matching rows travel.
Aggregate push-down	Each node computes partial `count` / `sum`; only small results travel.
Local (co-located) joins	Both tables sharded on the join key → each node joins its own pieces.
Distributed joins	Otherwise **broadcast** the small table, **repartition (shuffle)** both sides by the join key, or **semi-join**: send only join keys first, then fetch matching rows.
Smart sharding, caching	Keep common queries on one node; reuse frequent intermediate results.

Challenges: data localisation, network latency, a far larger plan space, nodes failing mid-query, and consistency when pieces come from replicas at slightly different moments.

Seeing it in Postgres with `postgres_fdw`

`postgres_fdw` (a **foreign data wrapper**) lets one Postgres query tables on another, which appear locally as **foreign tables**. `code/06_distributed_query_fdw.sql` points it at the same server: schema `shard_b` plays the "remote node", `remote.orders` is the foreign table:

```
CREATE EXTENSION IF NOT EXISTS postgres_fdw;
CREATE SERVER node_b FOREIGN DATA WRAPPER postgres_fdw
  OPTIONS (host '/tmp', port '5544', dbname 'ch_15');          -- in real life: another host
-- ... user mapping; shard_b.orders created as a copy of orders ...
IMPORT FOREIGN SCHEMA shard_b LIMIT TO (orders) FROM SERVER node_b INTO remote;

EXPLAIN (VERBOSE, COSTS OFF) SELECT order_id, amount FROM remote.orders WHERE amount > 500;
EXPLAIN (VERBOSE, COSTS OFF) SELECT customer_id, count(*), sum(amount) FROM remote.orders GROUP BY 1;
```

```
Foreign Scan on remote.orders
  Output: order_id, amount
  Remote SQL: SELECT order_id, amount FROM shard_b.orders WHERE ((amount > 500::numeric))

Foreign Scan
  Output: customer_id, (count(*)), (sum(amount))
  Relations: Aggregate on (remote.orders)
  Remote SQL: SELECT customer_id, count(*), sum(amount) FROM shard_b.orders GROUP BY 1
```

`Remote SQL` shows what was pushed down: the filter (only two columns and matching rows travel) and the whole `GROUP BY` (12 rows come back). A **cross-node join** — local USA `customers` with remote `orders` — is correct (Ben 250 orders / 64,750.00, Grace 250 / 65,000.00, Jack 250 / 64,750.00), but its plan shows a weakness:

```
Hash Join
  Hash Cond: (o.customer_id = c.customer_id)
  ->  Foreign Scan on remote.orders o
        Remote SQL: SELECT order_id, customer_id FROM shard_b.orders
  ->  Hash
        ->  Seq Scan on public.customers c
              Filter: (c.country = 'USA'::text)
```

The remote query has **no WHERE**: all 3,000 orders cross the network, 750 survive. The **semi-join** fix finds USA ids locally and ships only those:

```
EXPLAIN (VERBOSE, COSTS OFF)
SELECT o.order_id, o.customer_id FROM remote.orders o
WHERE o.customer_id = ANY (ARRAY(SELECT customer_id FROM customers WHERE country = 'USA'));
```

```
Foreign Scan on remote.orders o
  Output: o.order_id, o.customer_id
  Remote SQL: SELECT order_id, customer_id FROM shard_b.orders WHERE ((customer_id = ANY ($1::integer[])))
  InitPlan 1 (returns $0)
    ->  Seq Scan on public.customers
          Filter: (customers.country = 'USA'::text)
```

The `InitPlan` (a sub-query run once before the main query) runs locally; its three ids travel as `$1`, and only the 750 matching orders return. Distributed optimisers (Citus, CockroachDB, Spark, Trino) choose such strategies automatically from table sizes and location.

Common mistakes: wrapping remote columns in functions the remote side can't evaluate (blocks push-down — check `Remote SQL` in `EXPLAIN VERBOSE`); joining two large tables sharded on *different* keys in the hot path (co-locate or denormalise); `ORDER BY … LIMIT` across shards without per-shard limits (each shard should return its own top N first).

In the interview

30-second answer: "The **coordinator** (the node receiving the query and orchestrating the others) localises the data, splits the query into per-node fragments, pushes filters and partial aggregates to the data, runs fragments in parallel and merges results. Network transfer dominates, so the optimiser prefers co-located joins, broadcasts small tables and uses semi-joins to ship keys instead of rows."

- *Why can't `AVG` be pushed down as `AVG`?* An average of averages is wrong when groups differ in size; push down `SUM` and `COUNT`, divide at the end.

15.5 Replication

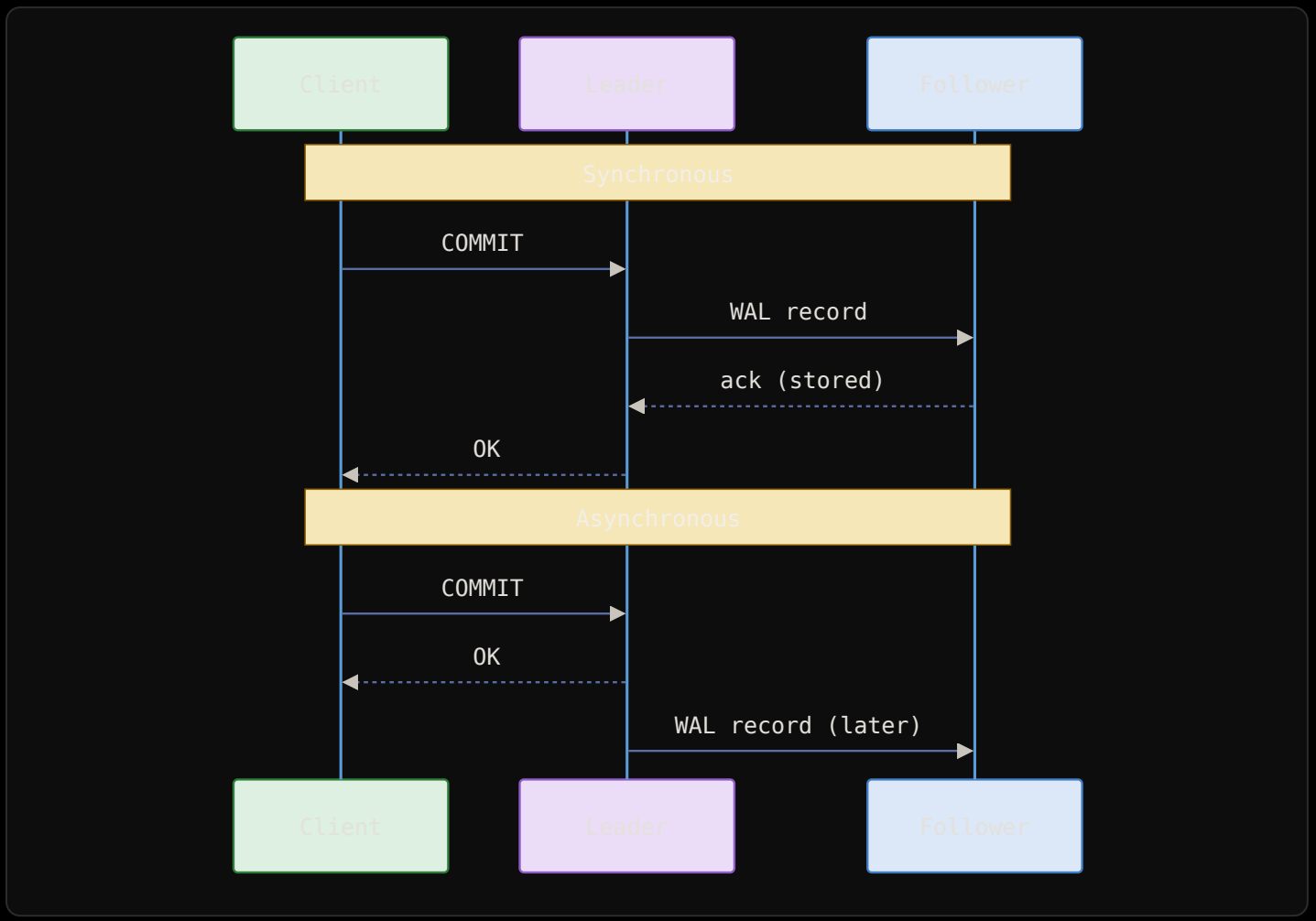
What it is

Replication keeps **copies of the same data on several nodes** (**replicas**). Benefits: **high availability** (a replica takes over when the primary dies), **read scaling**, **lower latency** (a Frankfurt replica serves Europe), **disaster recovery** (**DR** — a copy in another region for losing a whole site). Costs: keeping replicas in sync, extra storage and traffic, **conflicts** when several copies accept writes.

Type	How it works	Trade-off	Typical use
Full	Every replica holds the whole database	Any replica answers anything / storage × replicas	DR copies
Transactional	Each committed transaction streams to replicas in commit order, near real time	Close and ordered / continuous overhead	OLTP read replicas, Postgres logical replication
Snapshot	Periodically copy a full snapshot	Very simple / only as fresh as the last copy	Nightly reporting copies
Merge	Copies change independently (even offline), then merge by rules	Works disconnected / conflicts	Field and mobile apps
Key-based incremental	Copy rows whose `updated_at`/version is newer than the last sync	Moves little data / **misses hard deletes** (physically removed rows) unless tombstoned (marked deleted)	ETL into warehouses, IoT device sync

A retailer might mix them: orders and stock transactionally, the catalogue as nightly snapshots, a full replica in another region.

Synchronous vs asynchronous

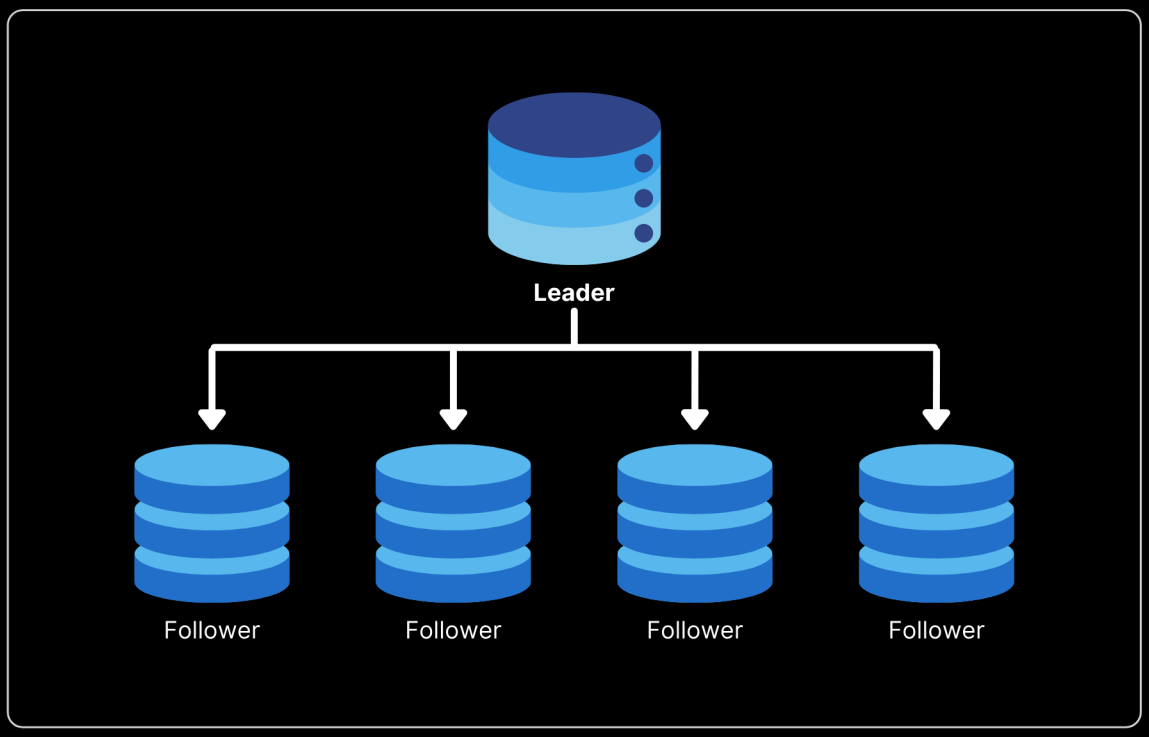


Synchronous: a commit returns only when the sync replica(s) have it — no acknowledged commit lost, but every write pays a network round trip and commits **block** if that replica is down (money, orders). **Asynchronous**: returns once the leader has it — fast, but the newest commits may be lost if the leader dies, and replicas lag (**replication lag**) — fine for read replicas, cross-region copies, analytics. Most systems are **semi-synchronous**: wait for *one* (or a quorum of) replicas, the rest async.

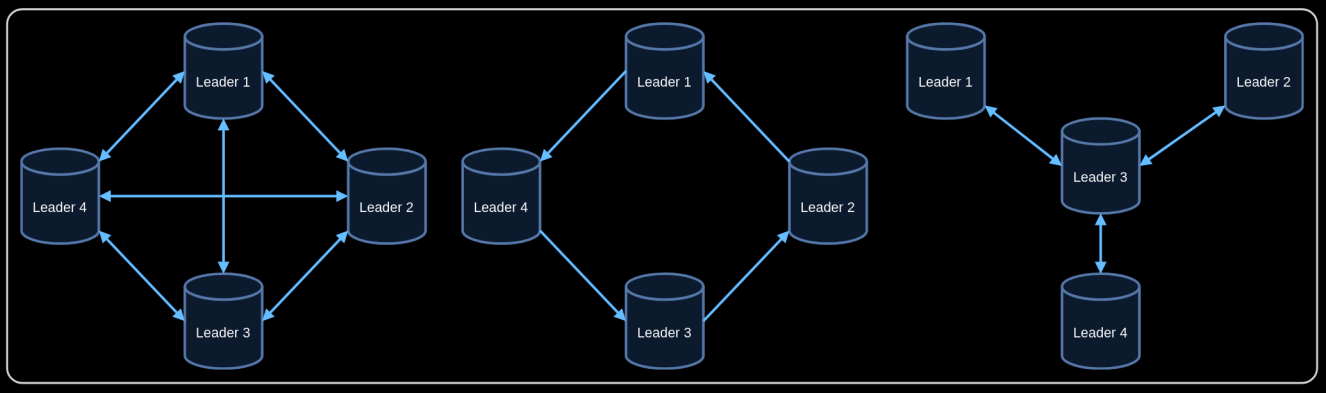
In PostgreSQL, physical **streaming replication** ships the WAL (§13.5) to **standbys** (replicas ready to take over), async by default; `synchronous_standby_names` (e.g. `'ANY 1 (s1, s2)'`) makes commits wait, and

`synchronous_commit` sets how far: `remote_write` (received), `on` (flushed), `remote_apply` (applied, so standby reads see it). **Logical replication** (`CREATE PUBLICATION` / `SUBSCRIPTION`, PG 10+) streams row changes per table.

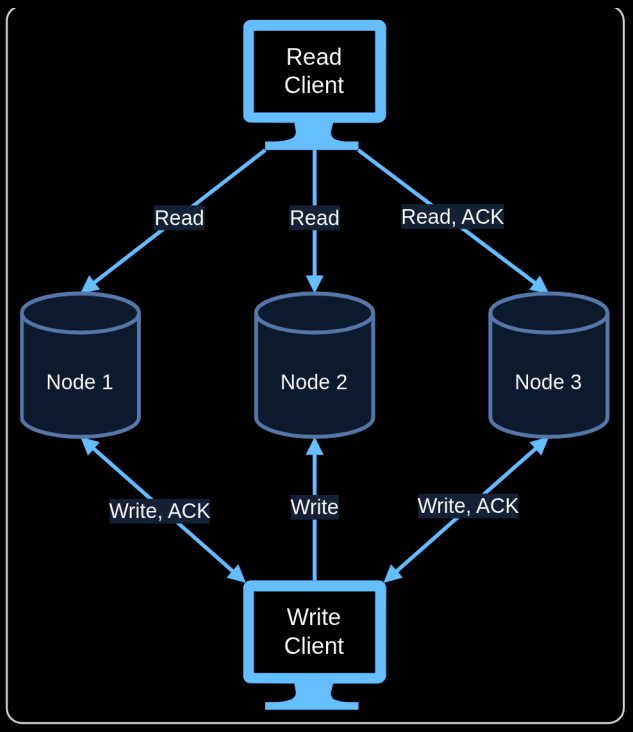
Replication topologies



1. Single-leader (primary/replica; formerly master/slave). All writes go to the leader, which ships its **replication log / WAL** to **followers** (sync or async); reads may use any node; if the leader fails, **failover** promotes a follower (15.10). Simple, no write conflicts, strong consistency possible; but write throughput is one machine's and async followers can be stale. PostgreSQL, MySQL, SQL Server Always On, MongoDB replica sets.



2. Multi-leader (active-active). Several nodes (often one per region) accept writes and replicate to each other. One row changed at two leaders at once is a **conflict**, resolved by **last-write-wins** (by timestamp — silently drops one write), custom merge code, or **CRDTs** (conflict-free replicated data types, which merge automatically). If a leader fails, clients switch. Used for multi-region writes, offline apps (each device a leader), collaborative editing. MySQL Group Replication (multi-primary), Postgres with BDR/pgEdge, CouchDB.



3. Leaderless (Dynamo-style). The client (or a coordinator) sends each write to all **N** replicas and succeeds once **W** acknowledge; reads ask **R** replicas and keep the newest version. If **R + W > N**, every read set overlaps every write set (the **quorum condition**). Stale replicas are fixed by **read repair** (the reader writes the newest value back), background **anti-entropy** (comparing **Merkle trees** — trees of hashes where each parent hashes its children, so replicas find differing data by comparing a few hashes) and **hinted handoff** (writes for a down node are held and delivered later). Concurrent writes are detected with **version vectors / vector clocks** (a per-node list of counters on each value; comparing two lists shows whether one write happened after the other or they were concurrent). Cassandra, ScyllaDB, Riak, Amazon's original Dynamo (the DynamoDB *service* uses a leader per partition). Leaderless systems use **quorums, not consensus** (no Paxos/Raft per write): available, but guaranteeing less.

`code/quorum.py` simulates N = 3: each trial a new version reaches W random replicas, a read asks R random replicas and keeps the highest version; stale if still old:

```
N = 3 replicas, 10,000 trials per setting
 W  R  R+W>N  stale reads
 1  1  False    66.4%
 1  2  False    34.0%
 1  3  True      0.0%
 2  1  False    33.8%
 2  2  True      0.0%
 ...  (every remaining row has R+W>N and 0.0%)

N=3, W=2, R=2: writes and reads still succeed with 1 replica down (2 of 3 answer); with 2 down they fail -> the system chooses consistency.
```

Every R + W > N setting had **zero** stale reads (for N=5, e.g. W=3, R=3). N=3, W=2, R=2 survives one failed replica — Cassandra's `QUORUM`; `ONE` is fast but possibly stale. Quorums still aren't **linearizability** (15.6): sloppy quorums (writes accepted by stand-in nodes), concurrent writes and clock-based last-write-wins can produce anomalies (DDIA ch.5).

Choosing a topology: correctness first → single-leader with a sync replica; writes in several regions → multi-leader with a conflict plan; must stay writable through failures and eventual is acceptable → leaderless with quorums.

Replication lag in practice. A user updates their profile, a lagging replica serves the next page, the old version appears. Fixes: read your own recent writes from the leader, pin a session to one replica, or wait until the replica's `pg_last_wal_replay_lsn()` reaches the write's LSN — the session guarantees of 15.6.

In the interview

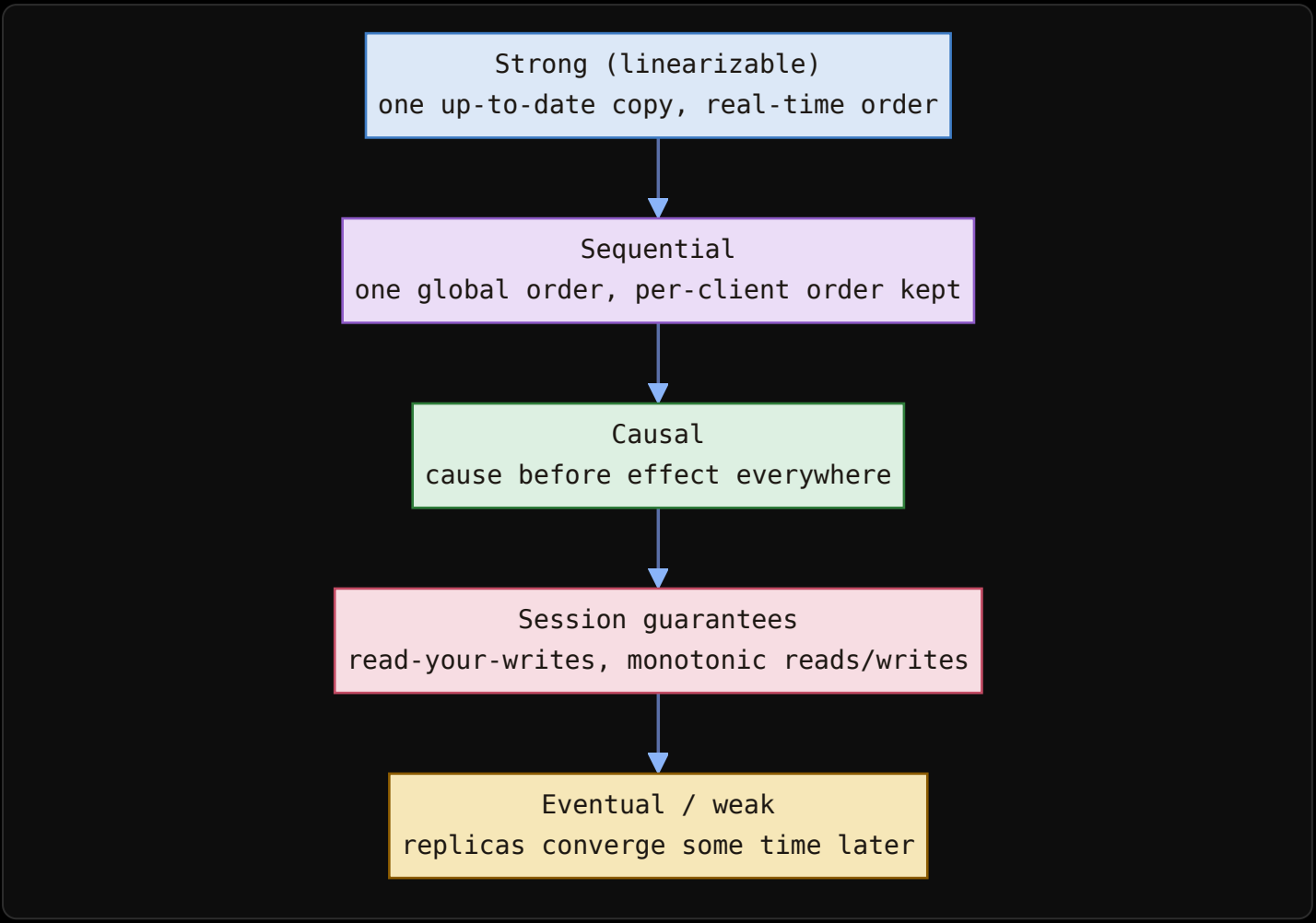
30-second answer: "Replication keeps copies for availability, read scaling and DR. Sync loses no acknowledged commits but slows writes; async is fast but can lose recent commits and serve stale reads, so most systems are semi-sync. Topologies: single-leader (simple, no conflicts), multi-leader (multi-region writes, conflicts), leaderless (quorums with R + W > N, read repair)."

- *A user doesn't see their own comment?* A lagging async replica served the read; use read-your-writes routing.
- *Last-write-wins downside?* Concurrent updates are silently lost, and **clock skew** (clocks disagreeing slightly) can discard the newer one.

15.6 Consistency models

What it is

With several copies, "what will a read return?" has more than one answer. A **consistency model** is the contract: which writes a read is guaranteed to see, and in what order clients observe operations. Stronger models are easier to program against but need more coordination (slower, less available); weaker ones are faster but push work onto the application. (Not ACID's C, which is about constraints — §13.3; these are about **replicas and ordering**.)



Model	Guarantee	How it's built	Pros / cons	Use
Strong (linearizable)	Every read sees the **latest completed write**; acts like one copy with operations in **real-time** order (if A finishes before B starts, everyone sees A first)	Synchronous replication or consensus (15.8), atomic operations, one global order	Simplest to reason about, no stale reads / coordination latency, scales less, may be **unavailable** during partitions	Balances, stock at checkout, "is this username taken?", locks. etcd, ZooKeeper writes, Spanner, CockroachDB, single-node Postgres
Sequential	All clients see **the same order**, each client's operations in program order — but not necessarily real-time order	A global sequencer	One agreed history / still needs coordination, no recency promise	CPU memory models, distributed shared memory; rare by name in databases
Causal	If A could have influenced B (same client did A then B, or B read A), **everyone sees A before B**; **concurrent** operations may appear in different orders	Track dependencies with **vector clocks**; delay an update until its dependencies are visible	Little coordination, works across regions, matches intuition / complex tracking, brief divergence	Replies after questions, chat, collaborative editing; MongoDB causally consistent sessions
Weak / eventual	Few promises; **eventual consistency**: if writes stop, replicas converge	Async propagation, anti-entropy, conflict resolution	Lowest latency, highest availability / stale and out-of-order reads, confusing UX (a like count that jumps)	Caches/CDNs, counters, analytics, DNS

Sequential example: Alice posts P1 then P2, Bob posts Q1; everyone sees one interleaving (say P1, Q1, P2), never P2 before P1 — but a reader just after Q1 finished may not see it yet (strong would).

Session (client-centric) guarantees

Weaker than the global models, but they fix the anomalies users notice, **within one session**:

Guarantee	Promise	Anomaly prevented	Example
Read-your-writes	You see your own earlier writes	"My new profile photo reverted"	Profile edits, cart
Monotonic reads	Never an older version after a newer one	A comment *disappears* on refresh	Feeds, notifications, config values
Monotonic writes	Your writes apply in the order issued	"price = 12" applied before the earlier "price = 10"	Version-control commits, debit then credit
Writes-follow-reads	A write after reading X is ordered after X	Reply stored before the post it answers	Comment threads

Session consistency bundles them (Azure Cosmos DB's default): track the last version/LSN each session saw and route its reads to replicas at least that fresh — coherent for the user and cheap, though sessions may see different states and routing loses some load-balancing freedom.

How to choose: strong for money and stock; session guarantees where users mostly need their own actions to stick; causal for cause-and-effect content; eventual for high-volume, tolerant data. Many systems let you **choose per operation** — Cassandra consistency levels, DynamoDB `ConsistentRead`, MongoDB read/write concerns, Cosmos DB's five levels.

In the interview

> **30-second answer:** "A consistency model says what reads can return under replication. Linearizable: one up-to-date copy in real-time order. Sequential: one global order without the real-time promise. Causal: cause before effect. Eventual: convergence only. Session guarantees — read-your-writes, monotonic reads — fix what users see. Stronger costs latency and availability, so I pick per feature."

- *Model for a comment thread?* Causal — replies must never precede their parent.
- *Linearizable vs serializable?* Linearizability is about single objects and real-time recency; serializability is about multi-object transactions behaving as if run one at a time. Both together = "strict serializability" (Spanner).
- *A refresh shows an older comment list — which guarantee broke?* Monotonic reads.

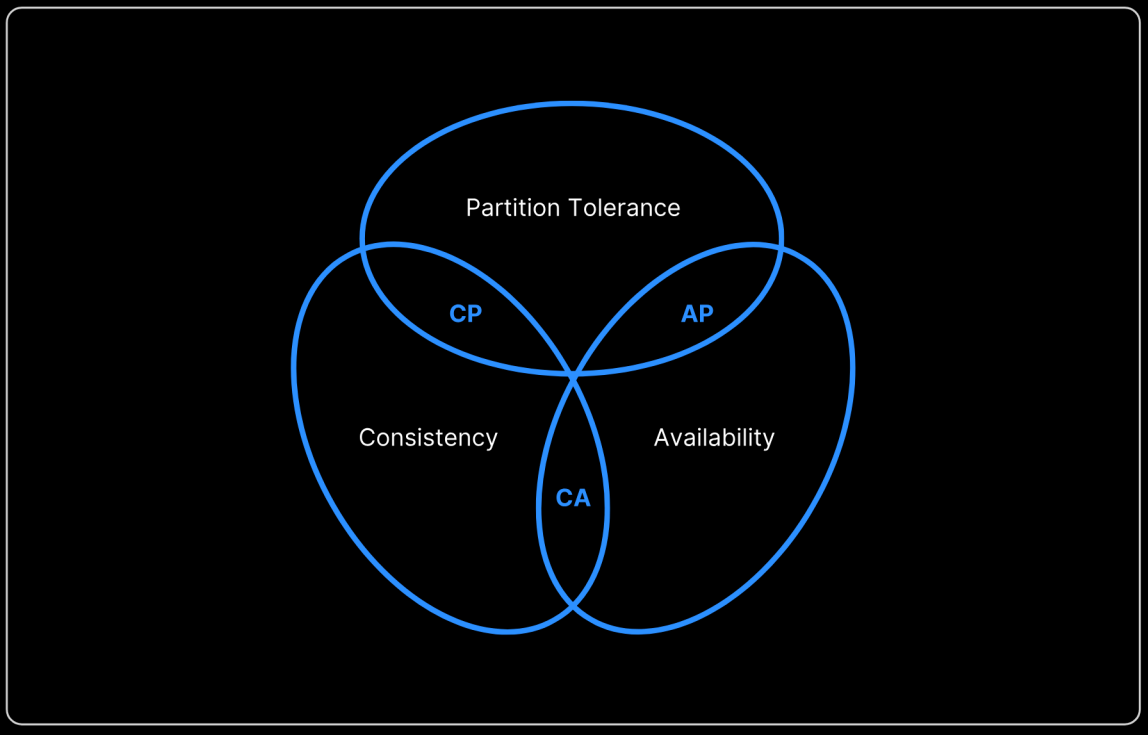
15.7 The CAP theorem (and PACELC)

What it is

The **CAP theorem** (Eric Brewer's conjecture, 2000; proved by Gilbert and Lynch, 2002) concerns three properties of a replicated store:

- **C — Consistency** (here *linearizability*; CAP in context: System Design Fundamentals): every read sees the most recent successful write — if Priya's balance drops from ₹500 to ₹300, no node may still report ₹500.
- **A — Availability**: every request to a **non-failed** node gets a non-error response, without waiting indefinitely.
- **P — Partition tolerance**: the system keeps working when the network drops or delays messages between groups of nodes.

The theorem: *during a network partition, a system must choose between consistency and availability.*



Why "pick two of three" is misleading

Across machines **partitions will happen** (switches fail, cables get cut), so P can't be given up; the real choice is what to do *during* one:

Choice	During a partition	Pros	Cons	Examples
CP	Refuse or delay some requests (usually the minority side) rather than return stale data	Correct, no divergence	Errors/timeouts for some users, coordination latency	etcd, ZooKeeper (small, strongly consistent coordination stores), HBase, Spanner, CockroachDB
AP	Keep answering on both sides, reconcile later	Always responsive	Temporarily inconsistent, needs conflict resolution	Cassandra/ScyllaDB (at `ONE`), Riak, CouchDB, DNS
CA	Only if partitions never happen — effectively one node	Simple and strong	No tolerance of network failure	A single Postgres server (not distributed, so CAP doesn't really apply)

Example: a subscriber counter in two data centres when the link breaks — **AP** keeps accepting clicks on both sides and merges later; **CP** makes the minority side refuse clicks until the link returns. Nuances: the choice is often **per operation** (CP for "place order and decrement stock", AP for "show reviews"); with **no** partition a system can be both C and A; CAP's A is strict — many "CP" systems are highly available in practice.

PACELC

Daniel Abadi's **PACELC** (2010): **if Partition, choose A or C; Else, choose Latency or Consistency** — even on a good day, synchronous coordination costs latency.

System	Partition	Else
Cassandra, Riak, DynamoDB (default reads)	PA	EL (fast, possibly stale)
Spanner, CockroachDB, etcd	PC	EC (pay latency for consistency)

In the interview

30-second answer: "CAP says that during a partition a replicated store gives up either linearizable consistency or availability. Partitions are unavoidable, so the real choice is CP or AP — banking and inventory lean CP, feeds and carts lean AP with reconciliation. PACELC adds that without partitions you still trade latency for consistency."

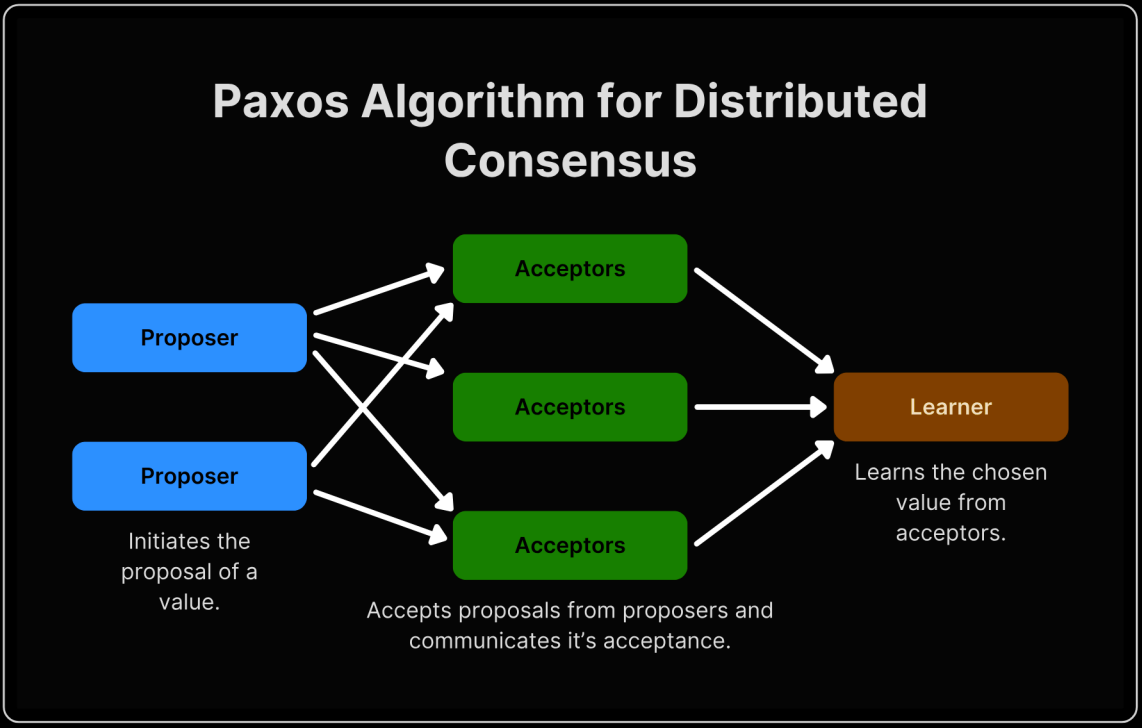
- *Is a single Postgres server CA?* It isn't distributed, so CAP isn't about it; with async replicas plus failover it's neither fully C (can lose recent commits) nor fully A.

15.8 Consensus and leader election

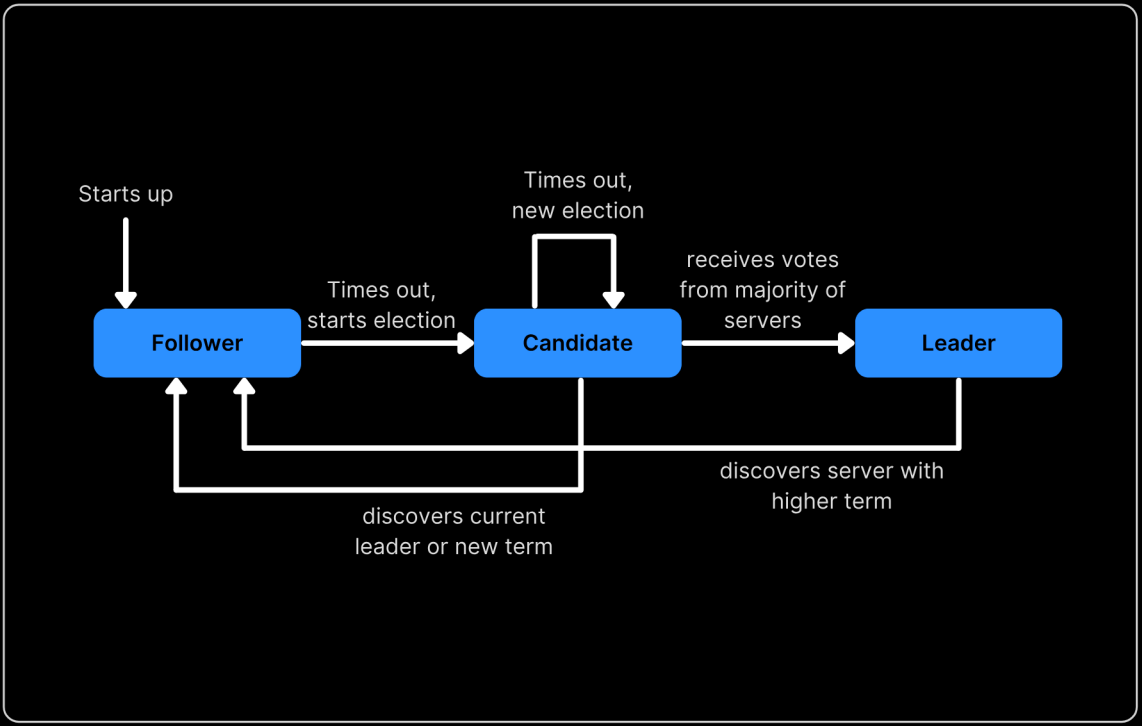
What consensus is

Consensus gets nodes to **agree on one value** (or one ordered log) even when some crash or messages are delayed. Everything needing "exactly one" rests on it — one leader, one commit decision, one log order, one lock owner. A correct protocol guarantees **agreement** (all non-faulty nodes decide the same value), **validity** (the decision was proposed by someone; if all propose v, it's v), **termination** (non-faulty nodes eventually decide) and **fault tolerance** (survives a minority of **crash faults**, or fewer than a third of **Byzantine faults** — nodes that lie). It's hard because nodes crash, networks partition and reorder messages, some nodes may be malicious, and every round trip adds latency. The **FLP result** (Fischer, Lynch, Paterson, 1985): no **deterministic** protocol can *guarantee* termination in a fully asynchronous network if even one node may crash — so real protocols are always *safe* and use timeouts to *usually* make progress.

The algorithms in brief



Paxos (Leslie Lamport; written 1989, published 1998) has **proposers**, **acceptors**, **learners**. (1) **Prepare/Promise:** a proposer sends *prepare(n)* with a unique, higher number to a majority; each acceptor promises to ignore anything below *n* and reports the highest-numbered proposal it already accepted. (2) **Accept:** with a majority of promises it sends *accept(n, v)*, where *v* must be the highest-numbered reported value (its own only if none). (3) Once a **majority accepts**, *v* is chosen and learners are told. Tolerates ⌊(N−1)/2⌋ crashes, needs no synchronised clocks for safety, formally proven — but hard to implement, two round trips per decision (**Multi-Paxos** keeps a stable leader to skip phase 1), and duelling proposers can stall it. Google's Chubby and Spanner.



Raft (Ongaro and Ousterhout, 2014) gives the same guarantees, designed to be **understandable**. (1) **Election:** time is split into numbered **terms**; a follower hearing no **heartbeat** (periodic "I'm alive") within its **randomised election timeout** (150–300 ms in the paper) becomes a **candidate**, increments the term and requests votes; each node votes once per term, only for a candidate whose log is at least as up to date; a majority makes a **leader**, and random timeouts make split votes rare. (2) **Log replication:**

the leader sends **AppendEntries** (also the heartbeat); an entry is **committed** once a majority stores it. (3) **Safety:** only a node holding every committed entry can win. Weaknesses: the leader caps throughput, unstable networks cause repeated elections, each commit waits for a majority. etcd (Kubernetes' store), Consul, CockroachDB, TiKV, YugabyteDB, Kafka's KRaft; MongoDB's replica-set protocol is Raft-inspired.

PBFT (Practical Byzantine Fault Tolerance; Castro and Liskov, 1999) tolerates **f lying nodes out of N ≥ 3f + 1**. In each **view** a **primary** runs *pre-prepare* (assign a sequence number) → *prepare* (replicas broadcast; prepared after 2f matching messages from others) → *commit* (after 2f + 1 matching commits) → *reply* (the client waits for f + 1 identical replies, so one is honest). **O(N²) messages**, so clusters stay at tens of nodes; **permissioned blockchains** (ledgers only approved participants may join; Hyperledger family) and critical control systems.

Proof of Work (Bitcoin) serves **open** networks, where fake identities make vote-counting useless: miners race to find a **nonce** that puts the block hash under a target, others verify cheaply, and the chain with the most accumulated work wins — no central authority, but heavy energy use, slow finality (a block every ~10 minutes) and exposure to a **51% attack** (most of the computing power can rewrite recent history). **Proof of Stake** (Ethereum since the 2022 "Merge") weights votes by locked-up currency.

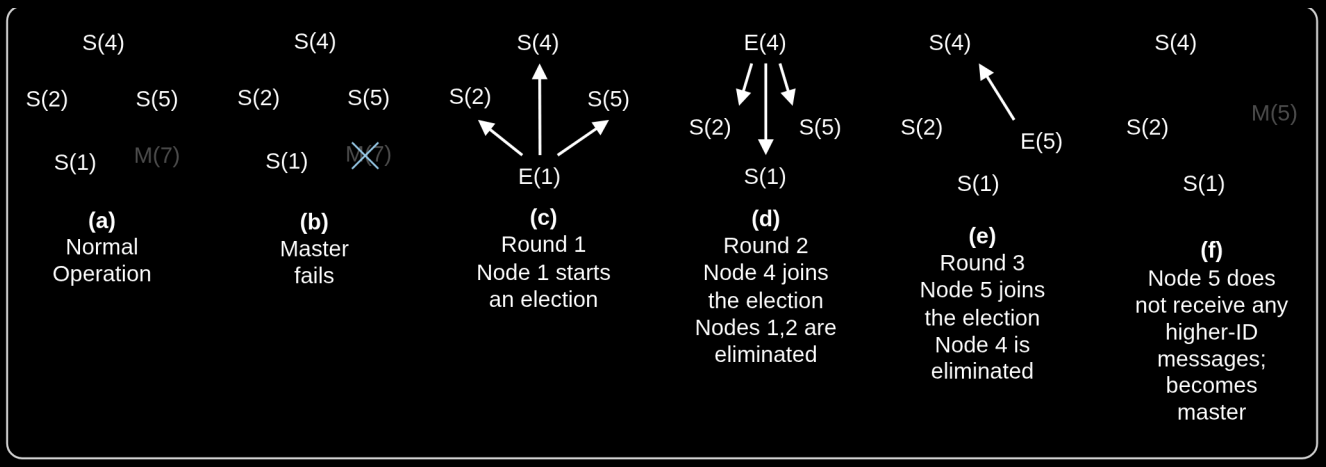
Algorithm	Fault model, tolerance	Cost	Fits
Paxos / Multi-Paxos	Crash, ⌊(N−1)/2⌋	Majority round trips; hard to implement	Databases, lock services
Raft	Crash, ⌊(N−1)/2⌋	Majority round trips, leader-bound	Config stores, distributed SQL
PBFT	Byzantine, ⌊(N−1)/3⌋	O(N²) messages	Permissioned ledgers
PoW / PoS	Byzantine, open membership (honest majority of hash power / stake)	Slow, energy-hungry / better throughput	Public blockchains

Known members that may crash → Raft or Paxos (what you'll meet inside databases); known members that may lie → PBFT family; unknown members → PoW/PoS. Full reasoning: DDIA ch.9.

Leader election

A **leader** is the single decider for writes, task assignment or a lock; election picks one and re-picks on failure. Hard parts: a slow leader looks dead, partitions can create **two leaders** (**split brain**), clocks differ.

Strategy	How it works	Strengths	Weaknesses
Bully	The detecting node sends ELECTION to all **higher** ids; if none answers it wins, else the higher node takes over; the winner broadcasts COORDINATOR	Simple, deterministic (highest live id)	Up to **O(n²)** messages; the top node always leads, even if weak
Ring	ELECTION travels a logical ring, each node substituting its id if higher; whoever gets its **own** id back wins and circulates COORDINATOR	About **2n** messages, equal participation	Slow on big rings; a dead node breaks the ring unless skipped
Randomised	Candidates wait a **random delay**, the first to announce wins, collisions retry	Simple, collisions rare	Non-deterministic (Raft adds majority votes)
ZooKeeper	Each candidate creates an **ephemeral sequential** znode (gone when its session dies); the **lowest** leads; each watches only the znode just **before** its own	Reliable, simple API; ZooKeeper runs the consensus protocol **ZAB**	Extra system to run (same pattern: etcd, Consul, Kubernetes **Lease**)



`code/leader_election.py` counts messages when the highest-id leader has crashed and the *lowest* id notices first (Bully's worst case):

nodes	bully leader	bully msgs	ring leader	ring msgs
5	4	19	4	8
10	9	89	9	18
50	49	2449	49	98

Same leader either way; Bully grows roughly with n^2, Ring with $2n$.

Fencing. A correctly elected leader can be *paused* (a long garbage-collection pause or a VM freeze), replaced, then wake still believing it leads. Give each new leader a higher term/epoch number (a **fencing token**) and have storage reject writes carrying an older one — Raft terms do exactly this.

In the interview

30-second answer: "Consensus lets nodes agree on a value or log despite failures. Raft elects a leader with randomised timeouts and majority votes and commits an entry once a majority stores it, tolerating $\lfloor (N-1)/2 \rfloor$ crashes; Paxos is the older equivalent. PBFT tolerates liars with $3f+1$ nodes at $O(N^2)$ messages. For leader election I'd use etcd or ZooKeeper and fence stale leaders with epoch numbers."

- *Why 3 or 5 nodes, not 4?* A majority of 4 is 3, so 4 nodes tolerate one failure — no better than 3 (5 tolerate 2).
- *What is split brain?* Two nodes both acting as leader after a partition; prevent with majority quorums and fencing.
- *PBFT with $f = 2$?* At least $3f + 1 = 7$ nodes.

TL;DR

Consensus = majority agreement that survives crashes (Raft/Paxos) or lies (PBFT, 3f+1). Leader election needs consensus underneath and fencing on top — use etcd, ZooKeeper or Consul rather than inventing your own.

15.9 Fault tolerance and recovery

What it is

Fault tolerance is **working correctly while parts fail**. A **fault** is one component going wrong (dead disk, crashed process, dropped packet); a **failure** is the whole system stopping or answering wrongly — fault tolerance stops the first becoming the second. Objectives: **continuity of service**, **data integrity**, **graceful degradation** (recommendations vanish but checkout works), **recovery** — in support of **availability**, **reliability**, **safety** (failures stay contained) and **maintainability**.

Handling a fault has five phases: **detection** (heartbeats, health checks, monitoring) → **diagnosis** (logs, error codes) → **evidence generation** (detailed logs, snapshots, audit trails) → **assessment** (impact analysis, dependency maps) → **recovery** (fail over, restart, reconfigure, restore). Techniques at every level: **hardware** (RAID, 15.11; hot-swappable drives, dual power supplies), **software** (exception handling, retries with backoff, timeouts, graceful degradation, **circuit breakers** that stop calling a failing service for a while — Reliability patterns) and **system** (checkpointing, mirroring, load balancing, replication and failover). The rest covers the database machinery.

Write-ahead logging, redo, undo and LSNs

The WAL rule — *a change's log record reaches disk before the changed data page*, with the log flushed (`fsync`) before COMMIT is acknowledged — is explained in §13.5. *Example:* Ravi moves ₹5,000 from A to B; "A −5,000", "B +5,000" and a commit record are flushed and he sees "Transfer complete"; if the server crashes a millisecond later, before the data pages were written, recovery replays those records. Recovery has two jobs:

- **Redo** makes **committed** work survive: re-apply logged changes (**after-images** — new values) that may not have reached the data files.
- **Undo** makes **uncommitted** work vanish: **before-images** (old values) restore any in-flight change that reached disk.
- Most databases do both. The classic design, **ARIES** (analysis → redo, "repeat history" → undo the losers), is used by DB2 and SQL Server; InnoDB uses the same redo + undo idea, its **undo logs** also serving MVCC snapshots.
- **PostgreSQL's WAL is redo-only.** Under MVCC (§13.14) an update writes a *new* row version; an aborted transaction is just marked not-committed in `pg_xact`, its versions are invisible, and VACUUM removes them later.

Every WAL record has a **log sequence number (LSN)** — its byte position in the log, always increasing. LSNs **order** redo/undo, tell recovery **what's already applied** (each data page stores the LSN of its last change, so older records are skipped), **mark checkpoints** (replay starts at the checkpoint's redo LSN) and **measure replication lag** (primary LSN vs replica replay LSN).

`code/07_wal_lsn_checkpoint.sql` first prints the defaults (`wal_level` = replica, `synchronous_commit` = on, `checkpoint_timeout` = 5min, `max_wal_size` = 1GB, `full_page_writes` = on), then measures WAL growth:

```
INSERT INTO lsn_marks VALUES ('before', pg_current_wal_insert_lsn());
INSERT INTO ledger SELECT i, 'acct-' || (i % 5), (i % 7) - 3 FROM generate_series(1, 5000) i;
INSERT INTO lsn_marks VALUES ('after 5,000 inserts', pg_current_wal_insert_lsn());

SELECT a.label AS from_mark, b.label AS to_mark,
       pg_wal_lsn_diff(b.lsn, a.lsn) > 0         AS wal_grew,
       pg_wal_lsn_diff(b.lsn, a.lsn) > 5000 * 50 AS more_than_50_bytes_per_row
FROM lsn_marks a, lsn_marks b WHERE a.label = 'before' AND b.label = 'after 5,000 inserts';

CHECKPOINT;
SELECT redo_lsn >= (SELECT lsn FROM lsn_marks WHERE label = 'after 5,000 inserts')
       AS redo_point_moved_past_our_inserts
FROM pg_control_checkpoint();
```

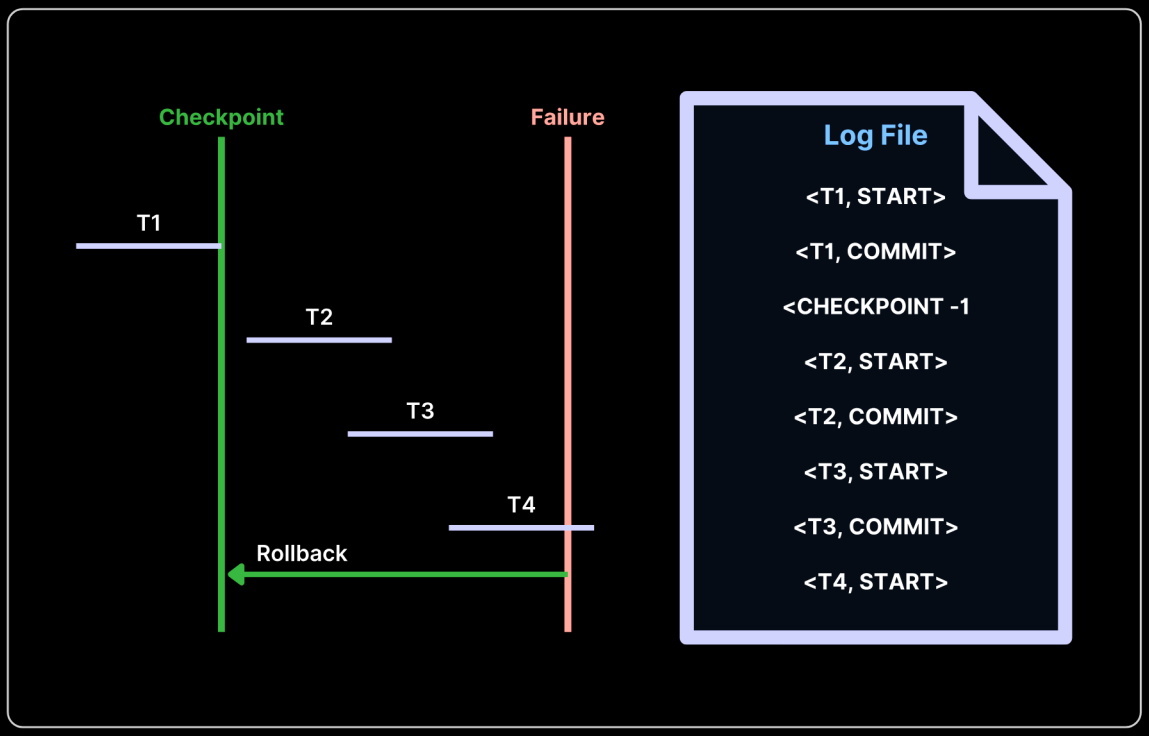
```
 from_mark |       to_mark       | wal_grew | more_than_50_bytes_per_row
-----------+---------------------+----------+---------------------------
 before    | after 5,000 inserts | t        | t

 redo_point_moved_past_our_inserts
-----------------------------------
 t
```

`pg_current_wal_insert_lsn()` returns the end of WAL (like `0/1A2B3C4`; booleans are printed because exact LSNs differ per run). Each row cost over 50 bytes of WAL (row data, index entries, headers). With `full_page_writes = on`, the first change to a page after a checkpoint logs the **whole page**, guarding against torn pages. After `CHECKPOINT` the **redo LSN** is past our inserts, so a crash now would replay from there.

Checkpointing

A **checkpoint** flushes all dirty pages and logs *"everything before LSN X is on disk"*, so recovery replays only from there and older WAL (not needed by replicas or archiving) can be recycled.



At the failure: **T1** finished before the checkpoint (already in the data files, ignored); **T2 and T3** committed after it and may exist only in the log (**redone**); **T4** never committed (**undone** — in Postgres, simply invisible). `code/wal_recovery.py` runs this on a toy log — T1 (A→B 300) commits before the checkpoint, T2 (C→D 50) after it, T3 (A→C 20) is interrupted after one page was flushed — with a forward **redo** pass after the checkpoint and a backward **undo** pass using before-images of uncommitted transactions:

```
on disk after crash : {'A': 580, 'B': 400, 'C': 500, 'D': 0}
redo  LSN 206 T2: C = 450
redo  LSN 207 T2: D = 50
redo  LSN 210 T3: A = 580
undo  LSN 210 T3: A = 600
recovered state     : {'A': 600, 'B': 400, 'C': 450, 'D': 50}
money conserved     : True
```

T1 needed nothing, T2 was replayed, T3's early-flushed change rolled back; the 1,500 total is intact.

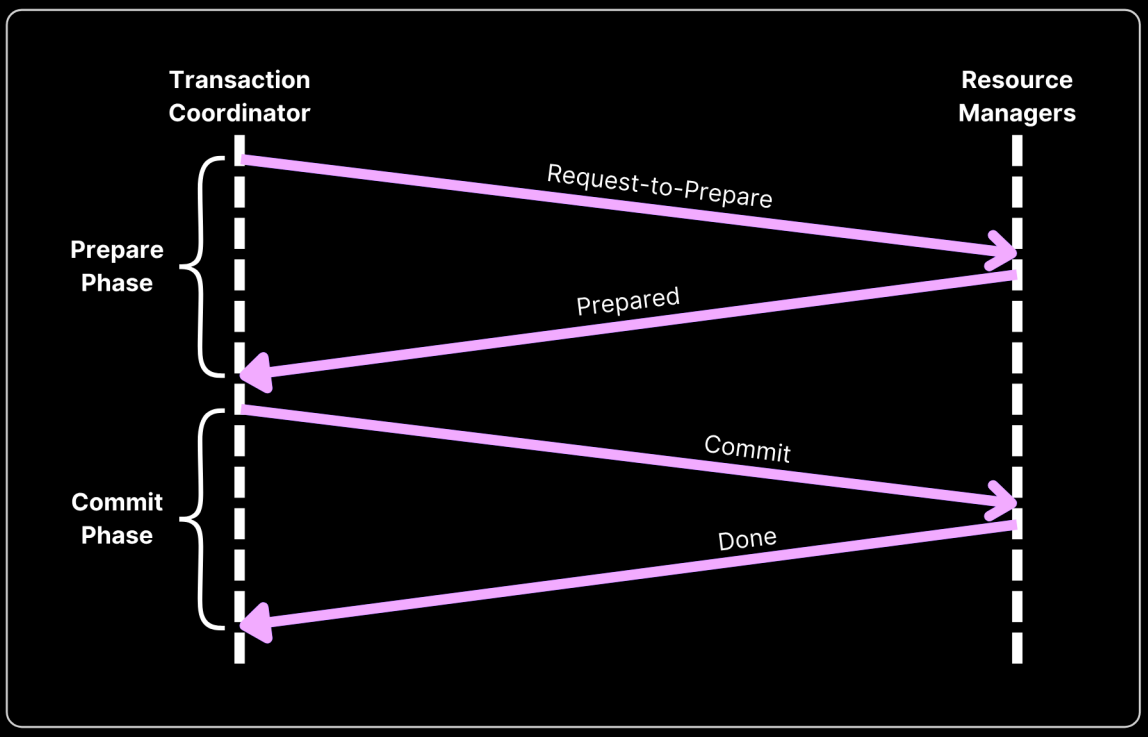
A **sharp (consistent)** checkpoint pauses new work, flushes every dirty page and resumes — simple, fully consistent, a visible stall. A **fuzzy** checkpoint logs a start point, flushes *while work continues*, then logs the end; recovery starts from the start point — no downtime, more bookkeeping. PostgreSQL is fuzzy: every `checkpoint_timeout` (5 min default) or when `max_wal_size` of WAL accumulates, spreading writes over `checkpoint_completion_target` (0.9 of the interval by default since PG 14).

Distributed checkpointing. Nodes' saved states must form a **consistent global state**: if node 1's checkpoint records *sending* a message, node 2's must record *receiving* it. **Coordinated** checkpointing (a coordinator, or markers along channels as in the Chandy–Lamport snapshot) is always consistent at the cost of a short pause; **uncoordinated** is cheaper but recovery must search for a consistent combination and can cascade rollbacks (the **domino effect**). *Example:* payment, stock and email nodes restore from one global checkpoint, so never "stock decremented but payment never recorded" (Apache Flink's barrier checkpoints work this way).

Coordinated commit: two-phase commit (2PC)

Distributed recovery is hard because networks partition, nodes fail and recover at different times, many transactions run concurrently, and every node must agree to commit or abort. When one transaction

changes data on **several nodes** (a transfer between accounts on nodes 1 and 2), atomicity must span machines. **2PC** uses a **coordinator**:



1. **Phase 1 — prepare (vote).** "Can you commit?" Each participant makes its changes durable, logs **PREPARED** and votes **YES** — from then on it may not decide alone — or **NO** (constraint failure, disk full).
2. **Phase 2 — decide.** All YES → the coordinator logs **COMMIT** and tells everyone; any NO or timeout → **ABORT**, everyone rolls back.

Recovery: a participant restarting without a PREPARED record aborts alone; one *with* PREPARED must **ask the coordinator**. If the coordinator crashes, participants that haven't voted can abort, but those that **voted YES must wait** — guessing could break atomicity. That wait, with locks held, is **2PC's blocking problem**; network delays and partitions also stall decisions. `code/two_phase_commit.py` shows all three cases:

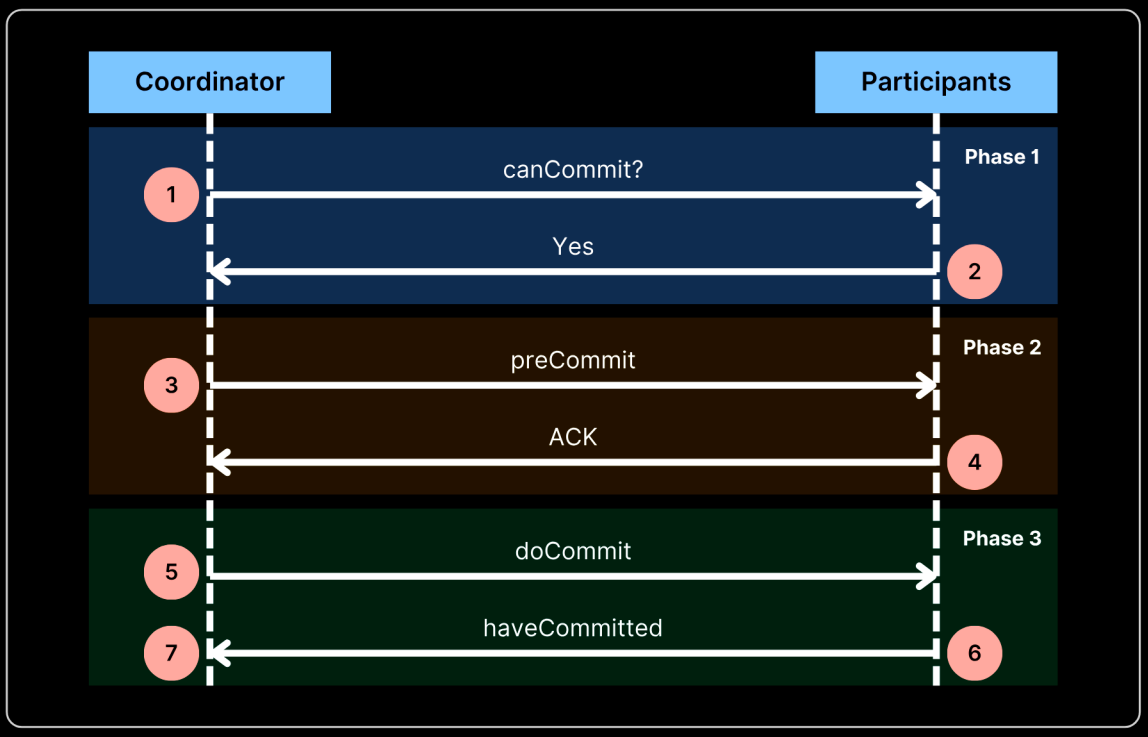
```
--- happy path: debit on node 1, credit on node 2
  ...
  phase 2 final states: {'node1': 'committed', 'node2': 'committed'}
--- one participant votes no (e.g. constraint check failed)
  ...
  phase 2 final states: {'node1': 'aborted', 'node2': 'aborted'}
--- coordinator failure after the votes
  phase 1 votes: {'node1': True, 'node2': True}
  coordinator crashes before sending a decision
  node1: prepared, holding locks, cannot commit or abort alone -> BLOCKED
  node2: prepared, holding locks, cannot commit or abort alone -> BLOCKED
```

2PC in PostgreSQL — the participant side, driven by an external transaction manager *(not run: needs `max_prepared_transactions > 0`; the book server keeps the default 0)*:

```
BEGIN;
UPDATE accounts SET balance = balance - 5000 WHERE id = 'A';
PREPARE TRANSACTION 'transfer-42';   -- phase 1: durable, survives a crash, locks kept
COMMIT PREPARED 'transfer-42';       -- later, on the coordinator's decision (or ROLLBACK PREPARED)
```

Java XA/JTA transaction managers (XA = the standard distributed-transaction interface; JTA = Java's API for it) and MySQL's `XA START … XA PREPARE … XA COMMIT` work the same way. Forgotten prepared transactions hold locks and block VACUUM — monitor `pg_prepared_xacts`.

Three-phase commit (3PC)



3PC inserts **PreCommit** between voting and committing: (1) **CanCommit** — vote as in 2PC; (2) **PreCommit** — if all voted YES, participants acknowledge and log that the outcome will be commit; (3) **DoCommit**. If the coordinator dies, participants that logged pre-commit know everyone voted YES and commit after a timeout; if no live participant saw pre-commit, nobody can have committed, so they abort. **Non-blocking** — *provided* message delays are bounded and nodes only crash; under a real **partition** both sides can time out and decide differently, so 3PC is rarely used. Modern systems make the coordinator fault-tolerant by **running the commit decision through consensus** (Spanner: 2PC over Paxos groups; CockroachDB: a transaction record replicated by Raft), and cross-service workflows use **sagas** (compensating steps) — 2PC and Saga patterns.

In the interview

30-second answer: "Databases survive crashes with write-ahead logging: the log is flushed before commit is acknowledged and data pages are written later. Recovery starts at the last checkpoint, redoes committed changes and undoes uncommitted ones — Postgres needs no undo thanks to MVCC. Across nodes, 2PC gives atomic commit, but prepared participants block if the coordinator dies, so real systems replicate the decision with Paxos/Raft or use sagas."

- *What does* `synchronous_commit = off` *trade?* Commits return before the WAL flush: a crash can lose the last fraction of a second of commits, never corrupt data.
- *A 2PC participant voted YES and lost the coordinator — can it abort?* No, it must wait.

TL;DR

Log first, write data later; checkpoints bound recovery; recovery = redo committed + undo uncommitted. 2PC blocks on coordinator failure and 3PC isn't partition-safe, so modern systems put the commit decision on a consensus group.

15.10 High availability: replication plus failover

What it is

High availability (HA) keeps the service up despite failures, measured in "nines": 99.9% ≈ 8.8 hours of downtime a year, 99.99% ≈ 53 minutes. Two targets: **RPO** (recovery point objective — how much *data* you

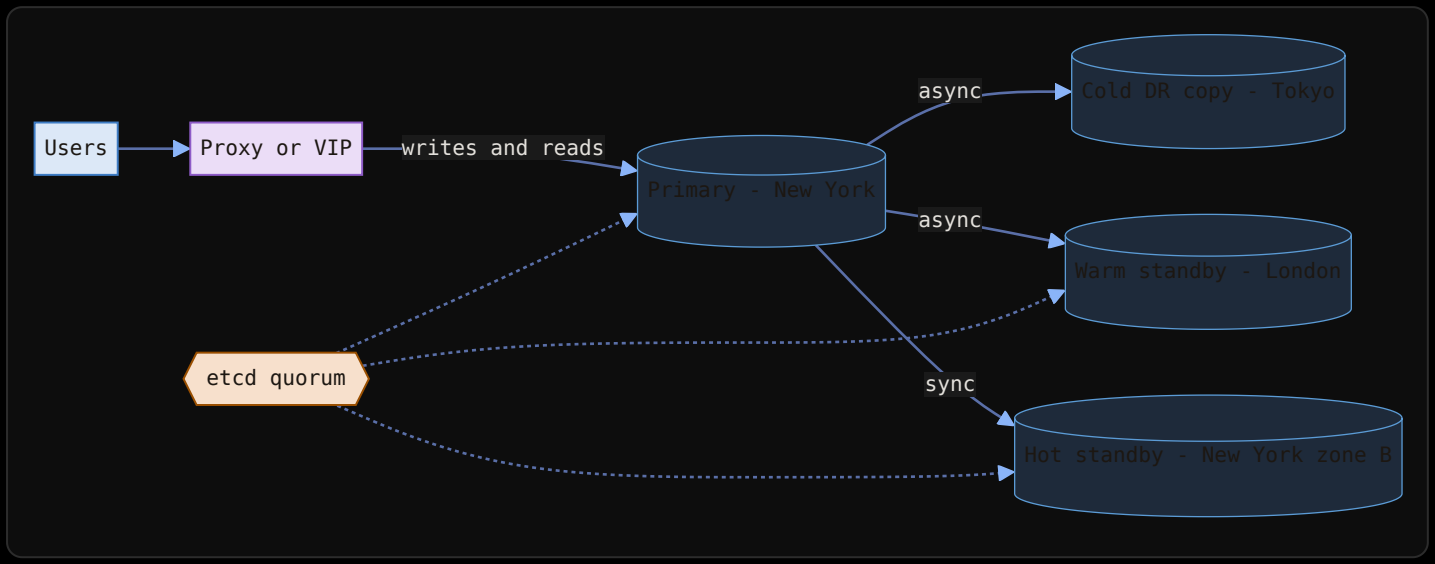
may lose; synchronous replication gives RPO = 0) and **RTO** (recovery time objective — how long you may be *down*; hot standbys and automatic failover make it short). HA = **replication** (15.5) + **failover**. A sync standby has every acknowledged commit (RPO 0, slower commits — local standbys in financial systems); an async one may miss the last commits (RPO > 0 — the lag at failure — fast commits; remote regions, read replicas).

Failover

Type	Standby state	Recovery time	Cost
Cold	Not running; must start and restore/catch up	Minutes to hours	Cheapest
Warm	Running and receiving updates, needs some catch-up or config	Seconds to minutes	Medium
Hot	Running, in sync, ready to serve immediately	Seconds	Highest

1. **Heartbeats** — several missed "I'm alive" signals raise suspicion.
2. **Decision** — a **quorum** (majority of nodes or witnesses) must agree the primary is dead, preventing **split brain**; **priority rules** pick the standby (highest replayed LSN, preferred zone).
3. **Promotion** — Postgres: `pg_ctl promote` or `SELECT pg_promote()` (PG 12+).
4. **Fencing** the old primary — STONITH ("shoot the other node in the head"), lease revocation, or blocking at the proxy.
5. **Redirecting clients** — DNS, a **virtual IP** (an address movable between machines), a proxy (HAProxy, PgBouncer) or **service discovery** (a registry answering "where is the primary right now?").

Core PostgreSQL has **no automatic failover** (without a human); use **Patroni** (leader state in etcd/Consul/ZooKeeper — consensus-backed), **repmgr** or **pg_auto_failover**, or a managed service (Amazon RDS/Aurora, Cloud SQL, Azure Database). **Best practices:** redundancy across failure domains (racks, zones, regions); load-balanced reads with writes routed to the current primary; monitoring (replication lag, heartbeats, disk); **backups too** (a bad `DELETE` replicates in milliseconds — HA is not a backup); **regular failover tests** (game days, **chaos experiments** — deliberately breaking things in a controlled way).



A zone failure in New York fails over to the sync hot standby in seconds with no data loss. Losing all of New York promotes London (RPO seconds, RTO about a minute; it also serves European reads). Tokyo is a cold copy for when both are gone. The etcd quorum spans three sites, so a partitioned minority can never promote itself.

In the interview

30-second answer: "HA = replicas plus automatic failover: heartbeats detect failure, a quorum confirms it to avoid split brain, the most up-to-date standby is promoted, the old primary is

fenced, clients are redirected. Sync standbys give zero data loss, async lower latency with some loss. I'd state RPO/RTO, choose hot/warm/cold to match, and test failover regularly."

15.11 Data resilience: redundancy, mirroring, backups

HA keeps the service **running**; **data resilience** makes the **data itself** survive disk crashes, corruption, ransomware, operator mistakes and site disasters.

Redundancy

Technique	How	Overhead	Survives
Replication	Full copies on other nodes (15.5)	N×	Node or site loss
RAID 1 / 10	Mirror each disk (10 = mirrors, striped; popular for databases)	2×	1 disk per mirror pair
RAID 5 / 6	Stripe data across disks with one / two parity blocks (computed from the data, used to rebuild a lost disk)	1 / 2 disks	Any 1 / any 2 disks
Erasure coding	k data + m parity fragments; any k rebuild the data	(k+m)/k: 6+3 → 1.5×	Any m fragments

Erasure coding (e.g. Reed–Solomon) gives replication-level safety at 1.5× instead of 3× storage, paid for with CPU and slower rebuilds — **object stores** (Amazon S3 and similar, storing whole files by key) and HDFS (Hadoop's distributed file system) use it for large, rarely read data.

Mirroring keeps an **exact, real-time copy**: every write goes to primary and mirror together, and the mirror takes over immediately — near-instant failover, always current, but double storage and extra write latency. *Example:* a bank mirrors its transaction database synchronously between two nearby data centres. (SQL Server literally called it "Database Mirroring"; Postgres's equivalent is synchronous streaming replication.)

Media failure recovery. A **media failure** is lost or damaged storage — **physical** (dead drive, bad sectors) or **logical** (corruption from software/firmware bugs or malware). Defences: **hot swapping** failed RAID drives while running; **health monitoring** (**SMART** drive telemetry, cloud volume metrics); **data scrubbing** (periodically read every block, verify **checksums** — small fingerprints of the data whose mismatch means corruption — and repair from redundancy: ZFS, Ceph; Postgres detects corrupted pages with `initdb --data-checksums`); **replication** to other nodes and regions.

Backup and restoration

A **backup** is an independent copy that can take you **back in time** — replicas can't, they follow the primary instantly (so a synchronous replica is not a backup). **Restoration** gets data back from it.

Type	Contains	Backup cost	Restore needs
Full	Everything	Slowest, largest	Just that backup (simplest restore)
Incremental	Changes since the **last backup of any kind**	Fastest, smallest	Last full + **every** incremental since, in order
Differential	Changes since the **last full**	Grows through the week	Last full + the **latest** differential

PostgreSQL (commands in §9.12): `pg_dump` / `pg_restore` make **logical** backups of one database, portable across versions. `pg_basebackup` plus **continuous WAL archiving** (`archive_mode`, `archive_command`) gives a **physical** backup replayable to *any moment* — **point-in-time recovery (PITR)** with `recovery_target_time`, the cure for "someone ran `DELETE` without `WHERE` at 14:02". PostgreSQL 17 added incremental base backups (`pg_basebackup --incremental`); pgBackRest and Barman manage all of this.

Best practices: backup frequency matching RPO (daily plus continuous WAL for critical data); **off-site** copies by the **3-2-1 rule** (3 copies, 2 media, 1 off-site); **test restores** regularly; **versioning and immutability** (object lock — copies can't be changed or deleted for a set period) so **ransomware** (malware that encrypts or deletes data for payment) can't destroy every copy. *Example:* a hospital takes a full backup every Sunday, incrementals Monday–Saturday and continuous WAL, stores copies locally and in another cloud region, and monthly restores "last Wednesday 15:00" into a test environment and checks row counts.

In the interview

> **30-second answer:** "Replication and RAID protect against hardware loss but not bad writes, which replicate too. So I add backups — full plus incremental or differential, off-site, versioned, test-restored. In Postgres that's base backups plus WAL archiving for PITR. Erasure coding gives durable storage at about 1.5× instead of 3× for large, cold data."

- *Accidental* `DROP TABLE` *at 14:02?* Restore the base backup into a separate instance, replay WAL to 14:01:59, copy the table back.
- *Which RAID level survives any two disk failures?* RAID 6.

TL;DR

Redundancy (replicas, RAID, erasure coding) survives hardware loss; backups (full/incremental/differential + WAL archiving) survive mistakes and disasters. Keep copies off-site and versioned, and test restores.

Chapter 15 TL;DR

Topic	Remember
Scale	Scale up first, then scale out with **partitioning + replication** together.
Partitioning	Range (time), list (category), hash (even), composite. Verify pruning with `EXPLAIN`. Unique keys include the partition key.
Sharding	Partitions on separate servers; a high-cardinality shard key matching the main query; consistent hashing with vnodes.
Distributed queries	Push filters and partial aggregates to the data, co-locate joins, semi-join to ship keys not rows.
Replication	Sync = no loss but slower; async = fast, lag, possible loss. Single-leader by default. Leaderless quorums need R + W > N.
Consistency	Strong > sequential > causal > session > eventual. Pick per feature.
CAP / PACELC	Under partition choose C or A; otherwise latency or consistency.
Consensus	Raft/Paxos: 2f+1 nodes survive f crashes. PBFT: 3f+1 for f liars. Elect leaders via etcd/ZooKeeper and fence stale ones.
Recovery	WAL first, checkpoints bound replay, redo committed + undo uncommitted (Postgres: redo-only + MVCC). 2PC blocks, 3PC isn't partition-safe; consensus-backed commit is the modern answer.
HA	Heartbeat → quorum decision → promote → fence → redirect. State RPO/RTO.
Resilience	Replicas are not backups. Full/incremental/differential + PITR, off-site, tested.

Next: Chapter 16 — Beyond relational looks at the NoSQL, NewSQL and specialised databases that make these trade-offs in different ways.

Chapter 16 — Beyond Relational: NoSQL, NewSQL and Specialised Databases

Relational databases are the right default for most applications: a flexible query language, joins, constraints and ACID transactions (Chapter 13). But some workloads fit tables awkwardly — a catalogue where every item has different attributes, a session cache needing sub-millisecond reads, billions of sensor readings a day, "friends of friends who like jazz", "every restaurant within 2 km". A family of databases grew up around those shapes of data. This chapter tours them, shows how much of each you can already do **inside PostgreSQL**, and ends with a decision table.

Section	Topic	Runnable part
16.1	Why NoSQL appeared, CAP and NoSQL, BASE vs ACID, the four families	—
16.2	Key-value stores (Redis, Riak)	KV + TTL + leaderboard in Postgres
16.3	Document databases (MongoDB, CouchDB)	**JSONB as a document store** with GIN indexes
16.4	Column-family and wide-column stores (Cassandra, HBase)	Wide-row time-series table
16.5	Graph databases (Neo4j, Neptune)	Friends-of-friends and shortest path with recursive CTEs
16.6	In-memory databases (SAP HANA, SingleStore, Redis)	—
16.7	NewSQL / distributed SQL (Spanner, CockroachDB)	—
16.8	Time-series and spatial databases	Partitions, BRIN, `date_bin`, geometric KNN with GiST
16.9	Big data frameworks: Hadoop, Spark, SQL-on-Hadoop, Kafka, Flink	Python MapReduce in miniature
16.10	Choosing a database	Decision table
16.11	Inside Elasticsearch/ELK (inverted index, analyzers, shards, refresh) and Redis (event loop, encodings, RDB/AOF, Cluster slots, eviction)	**Python inverted index + BM25 over two shards**

Prerequisites: Chapter 15 (partitioning, replication, CAP, consensus) and Chapter 12 (indexes). Theory: DDIA notes ch.2 — data models, ch.3 — storage engines (LSM trees, column storage), ch.10 — batch, ch.11 — stream processing. Patterns: Caching, Batch and streaming, Storing data.

All SQL runs on the book's PostgreSQL 14 (`bash tools_run_sql.sh sql`); the two Python toys (MapReduce, inverted index) with `bash tools_run_python.sh sql`. Snippets for other products (MongoDB, Redis, Cassandra, Neo4j, PostGIS) are marked **(not run)**.

16.1 Why NoSQL?

What it is

NoSQL ("Not only SQL", a term that caught on around 2009) covers databases whose primary model **isn't the relational table**. They trade some relational features — usually joins, a fixed **schema** (declared tables, columns, types) and sometimes multi-row transactions — for **flexible data models, easy horizontal scaling, and speed on specific access patterns**.

The problem it solves

Pain with relational (as typically deployed in the 2000s)	NoSQL answer
Rigid schema: every row has the same columns; changes need **migrations** (scripted schema changes on a live database)	**Flexible/dynamic schema**: each record carries its own fields (JSON or BSON documents, dynamic columns)
Scale-up only: one bigger server, expensive, with a ceiling	**Scale-out built in**: automatic sharding and replication across cheap nodes
Joins and transactions get slow at huge volume	Data **stored the way it's read** (denormalised or pre-aggregated), so no joins in the **hot path** (the code run on every request)

Example. A marketplace sells laptops (RAM, CPU, warranty), T-shirts (size, material) and books (authors, pages). Relationally that means **sparse columns** (mostly empty), an **entity-attribute-value** table (one row per `(item, attribute, value)`), or a table per category; in a document store each product is simply a document with the fields it has (16.3). Modern relational systems closed much of the gap — Postgres has JSONB, partitioning, logical replication and **Citus** (an extension spreading tables across servers) — hence this chapter shows the Postgres equivalent of each model.

CAP and NoSQL

During a network partition a distributed store chooses consistency or availability (§15.7):

- **CP** stores refuse or delay requests on the minority side: **HBase**, MongoDB (with majority write/read concern — how many replicas must confirm a write or agree on a read), etcd.
- **AP** stores keep answering and reconcile later: **Cassandra**, Riak, CouchDB, DynamoDB reads (eventual unless a strongly consistent read is requested).

Many make this **tunable per request** (Cassandra `ONE` vs `QUORUM`).

BASE: the other end of the spectrum from ACID

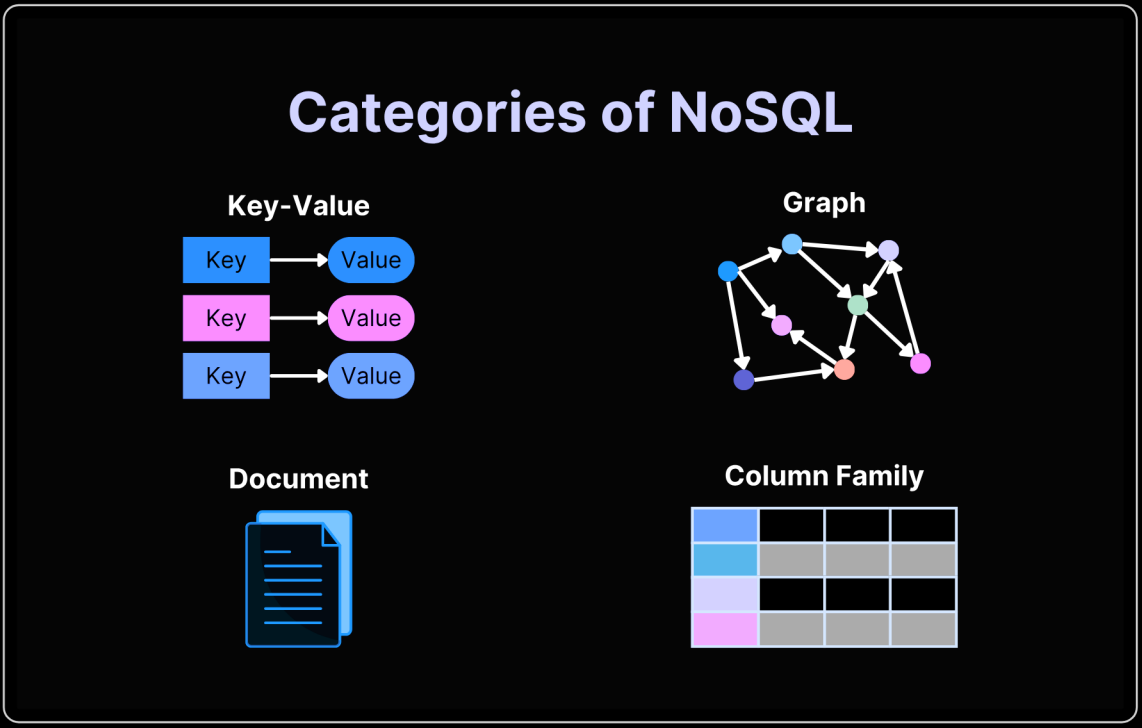
Many AP-leaning stores describe their guarantees as **BASE** — essentially CAP's **AP** choice as a design philosophy, suiting feeds, activity streams and metrics where being up beats every reader seeing the latest value:

- **Basically Available**: answers almost every request during partial failures, maybe not the newest value (a Cassandra read at `ONE` succeeds while one replica is up).
- **Soft state**: replicas may be temporarily out of sync, and state can change without new input as updates propagate (a lagging secondary in a MongoDB **replica set** — one **primary** taking writes, **secondaries** copying it).
- **Eventual consistency**: if writes stop, replicas converge (§15.6) — e.g. via hinted handoff, read repair and anti-entropy (§15.5).

Property	ACID (typical relational)	BASE (typical AP NoSQL)
Atomicity	All-or-nothing, including multi-row transactions	Often per single record/document only
Consistency	Constraints hold after every commit, and reads see committed data	Replicas may briefly disagree and converge later
Isolation	Concurrent transactions don't interfere (per isolation level)	Weaker; concurrent writes may conflict and need resolution
Durability	Committed = on disk (WAL)	Configurable (write concern, replication factor = number of copies, fsync policy)
Availability focus	Consistency first during failures	Availability first
Scaling	Traditionally vertical plus read replicas	Horizontal by design

Myth correction: the line has blurred. MongoDB has multi-document ACID transactions since 4.0 (2018), DynamoDB `TransactWriteItems` since 2018, Cassandra lightweight transactions (Paxos-based compare-and-set), while Postgres stores schemaless JSON. ACID vs BASE is a **spectrum of defaults**.

The four NoSQL families



Family	Data model	Great at	Examples
Key-value	Opaque value (not interpreted by the database) looked up by a unique key	Caching, sessions, counters, leaderboards	Redis, Riak KV, Memcached, DynamoDB (also a document store)
Document	JSON-like documents grouped into collections	Catalogues, CMS content, user profiles	MongoDB, CouchDB, Firestore
Column-family / wide-column	Rows with a partition key, columns grouped into families, sorted within a partition	Huge write volumes, time-series, logs	Cassandra, HBase, ScyllaDB, Bigtable
Graph	Nodes and edges, both with properties	Relationships: social, fraud rings, recommendations	Neo4j, Amazon Neptune, ArangoDB

Shared features: **dynamic schema**, **horizontal scaling**, **performance tuned to one access pattern**, **fault tolerance** through built-in replication.

In the interview

> **30-second answer:** "NoSQL databases drop the relational model to get flexible schemas, easy scale-out and speed on specific access patterns. There are four families: key-value, document, wide-column and graph. Many lean AP in CAP terms and follow BASE (basically available, soft state, eventually consistent) instead of strict ACID, though modern ones offer transactions and tunable consistency. You model data around queries, not around normalisation."

- *Is MongoDB ACID?* Single-document operations always were; multi-document transactions since 4.0 (replica sets) and 4.2 (sharded clusters).
- *When is Postgres JSONB enough?* When you need flexible attributes *plus* joins, constraints and transactions, at a scale one Postgres (or Citus) can handle.
- *A CP and an AP store?* HBase; Cassandra (by default). *Family for "friends' friends"?* Graph.

TL;DR

NoSQL = flexible models + scale-out + access-pattern-specific speed, often with BASE semantics. The ACID/BASE divide is now a spectrum.

16.2 Key-value stores

What it is

The simplest database: a giant, distributed **dictionary**. Each **key** is unique; the **value** is whatever you store (text, number, JSON, a binary **blob**), uninterpreted. You `GET`, `SET`, `DELETE` — no joins, usually no queries by value.

```
session:9f2c      -> {"user_id": 42, "cart": ["LAP-13"], "expires": "..."}
rate:10.0.0.7     -> 17
weather:paris     -> {"temp_c": 18}
```

Designing keys is the whole data model (prefixes like `user:42:profile`, because the key is the only way in). Fits when you need very fast access by a known id, relationships are minimal or handled in the app, and pairs are independent.

Popular systems

Redis ("Remote Dictionary Server") keeps everything **in memory**, answering in well under a millisecond. Values are rich **data structures**: strings, hashes (field→value maps), lists, sets, **sorted sets** (members ordered by a numeric score), streams, bitmaps and HyperLogLogs (a tiny structure estimating distinct counts). It also offers **pub/sub** messaging (every subscriber to a channel receives each published message), **Lua** scripting run inside the server, key expiry (**TTL**, time to live), **eviction policies** such as **LRU** (least recently used) / **LFU** (least frequently used) when memory is full, optional **persistence** (RDB snapshots and/or an append-only file, AOF), and Redis Cluster's 16,384 hash slots — internals in 16.11.

```
# Redis (not run)
SET session:42 '{"user":42}' EX 1800     # expires in 30 min
GET session:42
INCR page:home:views                      # atomic counter
ZADD steps 12050 ana 8300 bo 15010 cy     # sorted set = leaderboard
ZREVRANGE steps 0 2 WITHSCORES            # top 3
```

Riak KV is a Dynamo-style (after Amazon's Dynamo paper) **distributed** key-value store: leaderless replication, consistent hashing, tunable N/R/W quorums (§15.5), **eventual consistency** with sibling

values for conflicts; built for high availability, survives node failures without failover. Its company (Basho) closed in 2017; the code lives on as open source.

Use	Example	Why KV fits
Caching	A weather app caches the latest forecast per city; a shop caches product pages	Read by key, expiry built in
Session storage	Logged-in user state	Tiny, per-user, high-churn data
Real-time data	Live stock prices or ride locations updated many times a second	In-memory speed
Counters and rate limits	"100 requests per minute per API key"	Atomic `INCR` with a TTL
Leaderboards	A fitness app ranks users by daily steps	Sorted sets keep order as scores change

Strengths	Limits
O(1) lookups by key, very high throughput	**No ad-hoc queries**: filtering, joins and secondary lookups must be built in the app or with extra keys (e.g. "all sessions of user 42" needs an app-maintained `user:42:sessions`)
Easy to shard (hash the key) and replicate	**Poor fit for relationships**
Values can be anything	In-memory stores are **bounded by RAM** (expensive)

The same pattern in Postgres

`code/02_key_value_style.sql` builds a KV table with TTL, an UPSERT "put", and a leaderboard:

```
CREATE UNLOGGED TABLE kv (            -- UNLOGGED: no WAL, fast, emptied after a crash (cache-like)
  k          text PRIMARY KEY,
  v          jsonb NOT NULL,
  expires_at timestamptz
);
-- PUT = insert or overwrite
INSERT INTO kv VALUES ('session:42', '{"user":42,"cart":["LAP-13","TSH-01"]}', now() + interval '30 minutes')
ON CONFLICT (k) DO UPDATE SET v = EXCLUDED.v, expires_at = EXCLUDED.expires_at;
INSERT INTO kv VALUES ('weather:paris', '{"temp_c":18}', now() - interval '1 minute');  -- already expired
-- GET only unexpired keys
SELECT k, v FROM kv WHERE k = 'session:42' AND (expires_at IS NULL OR expires_at > now());
```

```
     k      |                    v
------------+-------------------------------------------
 session:42 | {"cart": ["LAP-13", "TSH-01"], "user": 42}

 weather_hits
--------------
            0
```

The expired key isn't returned. The leaderboard is a window function (Chapter 8); ties share a rank:

```
 rank | user_name | steps
------+-----------+-------
    1 | cy        | 15010
    2 | ana       | 12050
    3 | bo        |  8300
    3 | di        |  8300
```

Cleanup is `DELETE FROM kv WHERE expires_at <= now() RETURNING k` (evicts `weather:paris`; Redis expires automatically), and `pg_class.relpersistence = 'u'` confirms the table is unlogged — speed (no WAL) traded

for durability: truncated after a crash, not replicated. Fine for moderate loads; Redis still wins on latency, built-in expiry and data structures.

> **Common mistakes:** a cache as the **source of truth** (the authoritative copy) without persistence, losing data on restart; huge values or "hot keys" (one key with a large share of traffic) that one shard serves alone; no TTLs, so memory fills and eviction drops useful keys.

In the interview

> **30-second answer:** "A key-value store is a distributed hash map: O(1) get/put by key, easy to shard and replicate, no queries by value. Redis adds in-memory speed and data structures like sorted sets. I use it for caching, sessions, counters, rate limiting and leaderboards, with a TTL on keys, and keep the durable truth in a primary database."

- *Rate limiter?* `INCR key` with `EXPIRE` for a fixed window, or a sorted set of timestamps for a sliding window.
- *Cache invalidation?* **Cache-aside** (check the cache; on a miss read the database and fill the cache) with TTLs, and delete or refresh the key on writes (caching patterns).
- *What does a sorted set give a leaderboard?* Members ordered by score, O(log n) updates, fast range queries.

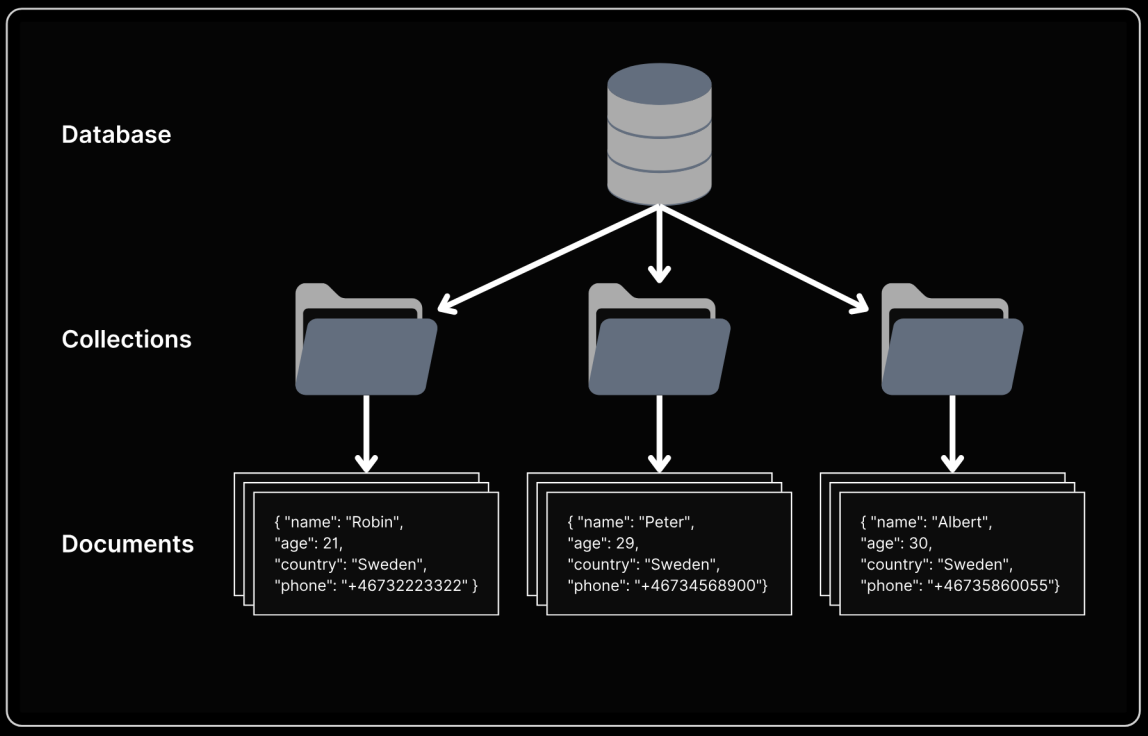
TL;DR

Key-value = dictionary at scale. It's the fastest way to get by key, with nothing beyond that. Perfect for caches, sessions, counters and leaderboards.

16.3 Document databases

What it is

A **document database** stores self-contained **documents**, usually JSON or a binary form (MongoDB's **BSON**), grouped into **collections** (≈ tables) inside a **database**. Every document has a unique id (`_id` in MongoDB); documents in one collection may have **different fields** and **nest** objects and arrays.



```
{ "_id": 1, "name": "Meera", "city": "Pune",  "phone": "+91 98xxxxxx01" }
{ "_id": 2, "name": "Tomás", "city": "Porto", "languages": ["pt", "en"] }
```

Key features: (1) **flexible schema** — electronics with `warranty` and `brand` next to clothing with `size` and `material`, no migrations; (2) **nesting** — a customer document **embeds** addresses and recent orders, so one read serves a page; (3) **no joins needed** — data read together is stored together; (4) **horizontal scaling** — built-in sharding by a shard key (MongoDB).

Popular systems

- **MongoDB**: BSON, a rich query language, **secondary indexes** (on fields other than the id), **aggregation pipelines** (`$match` → `$group` → `$sort`, like `WHERE`/`GROUP BY`), replica sets (single-leader), sharding, multi-document transactions since 4.0.
- **CouchDB**: JSON over a **RESTful HTTP API** (resources addressed by URLs, manipulated with GET, PUT, DELETE), multi-leader **replication and sync** for occasionally connected devices (PouchDB in the browser syncs with it), **MVCC** document revisions for conflict detection.

```
// MongoDB shell (not run)
db.products.insertOne({ sku: "LAP-15", type: "laptop", price: 1399, specs: { ram_gb: 32 } })
db.products.find({ type: "laptop", "specs.ram_gb": { $gte: 16 } }).sort({ price: 1 })
db.products.aggregate([{ $group: { _id: "$type", avgPrice: { $avg: "$price" } } }])
db.products.createIndex({ type: 1, price: 1 })
```

Use cases: **content management** (posts with varying metadata), **product catalogues** (per-category attributes in one collection), **user profiles and settings** (optional, evolving fields), **event logging** (a game records player actions for later analysis).

Modelling: embed or reference?

Embed (nest inside the parent)	Reference (store the id, look it up separately)
Data read together and owned by the parent (a customer's addresses)	Data shared by many parents (a product referenced by many orders)
Bounded size (a few addresses)	Unbounded growth (all of a user's comments ever)
One read, atomic single-document update	Second query or `$lookup` (the document world's join)

Strengths	Limits
Schema flexibility for evolving data	**Many-to-many relationships** and cross-document constraints are awkward
Fast reads when documents match the page	**Duplicated (denormalised) data** must be kept in sync by the app
Built-in scale-out	Some configurations favour availability over strong consistency (e.g. reading from secondaries)

A document store inside Postgres: JSONB

`jsonb` (operators, indexing and update functions taught in §14.4) plus **SQL/JSON path** queries (a mini-language like `$.authors[*]`) make Postgres a document store. `code/01_jsonb_document_store.sql`:

```
CREATE TABLE products (
  id  bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  doc jsonb NOT NULL
);
INSERT INTO products (doc) VALUES
 ('{"sku":"LAP-13","type":"laptop","name":"Aero 13","price":999,
    "brand":"Nimbus","warranty_years":2,"specs":{"ram_gb":16,"cpu":"8-core"}}'),
 ('{"sku":"TSH-01","type":"tshirt","name":"Plain Tee","price":15,
    "sizes":["S","M","L"],"material":"cotton"}'),
 ('{"sku":"LAP-15","type":"laptop","name":"Aero 15","price":1399,
    "brand":"Nimbus","warranty_years":3,"specs":{"ram_gb":32,"cpu":"12-core"}}'),
 ('{"sku":"BK-042","type":"book","name":"Data at Scale","price":45,
    "authors":["R. Rao","M. Chen"],"pages":412}');

SELECT doc->>'sku' AS sku, (SELECT string_agg(k, ',' ORDER BY k) FROM jsonb_object_keys(doc) k) AS keys
FROM products ORDER BY id;
```

```
  sku   |                      keys
--------+-------------------------------------------------
 LAP-13 | brand,name,price,sku,specs,type,warranty_years
 TSH-01 | material,name,price,sizes,sku,type
 LAP-15 | brand,name,price,sku,specs,type,warranty_years
 BK-042 | authors,name,pages,price,sku,type
```

Every row has its own shape, like documents in a collection. Querying (`->` JSON, `->>` text, `#>>` path):

```
SELECT doc->>'name' AS name, (doc->>'price')::numeric AS price, doc#>>'{specs,ram_gb}' AS ram_gb
FROM products WHERE doc->>'type' = 'laptop' ORDER BY price;
```

```
  name   | price | ram_gb
---------+-------+--------
 Aero 13 |   999 | 16
 Aero 15 |  1399 | 32
```

Containment `@>` ("does the document contain this sub-document?"; arrays match if they contain the listed elements):

```
SELECT doc->>'name' AS name FROM products WHERE doc @> '{"sizes":["M"]}';                        -- Plain Tee
SELECT doc->>'name' AS name FROM products WHERE doc @> '{"brand":"Nimbus","specs":{"ram_gb":32}}'; -- Aero 15
SELECT doc->>'name' AS name FROM products WHERE doc ? 'authors';                                 -- Data at Scale
SELECT jsonb_path_query(doc, '$.authors[*]') AS author FROM products WHERE doc->>'sku' = 'BK-042';
```

```
  author
-----------
 "R. Rao"
 "M. Chen"
```

Partial updates with `jsonb_set`, `||` (merge) and `-` (remove key):

```
UPDATE products SET doc = jsonb_set(doc, '{price}', '1299') WHERE doc->>'sku' = 'LAP-15';
UPDATE products SET doc = doc || '{"on_sale":true}'         WHERE doc->>'sku' = 'TSH-01';
UPDATE products SET doc = doc - 'warranty_years'            WHERE doc->>'sku' = 'LAP-13';
```

```
  sku   | price | on_sale | warranty
--------+-------+---------+----------
 LAP-13 | 999   |         |
 TSH-01 | 15    | true    |
 LAP-15 | 1299  |         | 3
 BK-042 | 45    |         |
```

(Postgres rewrites the whole row version — JSONB isn't updated in place, so huge, frequently updated documents are a poor fit.)

Indexing, after adding 20,000 generated products:

```
CREATE INDEX products_doc_gin ON products USING gin (doc jsonb_path_ops);   -- serves @> on ANY key
EXPLAIN (COSTS OFF) SELECT count(*) FROM products WHERE doc @> '{"sku":"GEN-777"}';

CREATE INDEX products_price_idx ON products (((doc->>'price')::numeric));  -- B-tree on ONE field
EXPLAIN (COSTS OFF) SELECT doc->>'sku' FROM products WHERE (doc->>'price')::numeric > 503;
```

```
Aggregate
  ->  Bitmap Heap Scan on products
        Recheck Cond: (doc @> '{"sku": "GEN-777"}'::jsonb)
        ->  Bitmap Index Scan on products_doc_gin
              Index Cond: (doc @> '{"sku": "GEN-777"}'::jsonb)

Bitmap Heap Scan on products
  Recheck Cond: (((doc ->> 'price'::text))::numeric > '503'::numeric)
  ->  Bitmap Index Scan on products_price_idx
        Index Cond: (((doc ->> 'price'::text))::numeric > '503'::numeric)
```

The GIN index covers containment on any key; `jsonb_path_ops` is smaller and faster for `@>` but doesn't support `?`, while the default `jsonb_ops` supports both. An expression B-tree serves range and sort queries on one field.

Guard rails document databases often lack — a `CHECK` can enforce required keys and types:

```
ALTER TABLE products ADD CONSTRAINT doc_has_sku_and_price
  CHECK (doc ? 'sku' AND jsonb_typeof(doc->'price') = 'number');
-- inserting '{"name":"No SKU","price":"cheap"}' now fails:
-- NOTICE:  rejected: new row for relation "products" violates check constraint "doc_has_sku_and_price"
```

A common hybrid: **regular columns for stable, queried fields** (sku, price, type) plus a **JSONB column for variable attributes** — integrity where it matters, flexibility where it helps.

In the interview

30-second answer: "A document store keeps self-contained JSON documents in collections, with flexible schemas and nesting, so a page can be served by one read. I embed data that's owned and read together, and reference data that's shared or unbounded. It's weaker at many-to-many relationships and cross-document constraints. If I also need joins and transactions, Postgres JSONB with a GIN index gives most of the benefit."

- *Index a JSONB field for range queries?* A B-tree expression index on `((doc->>'field')::type)`. (`json` vs `jsonb`: §14.4.)
- *Reference instead of embed when?* The child data is shared or grows without bound.

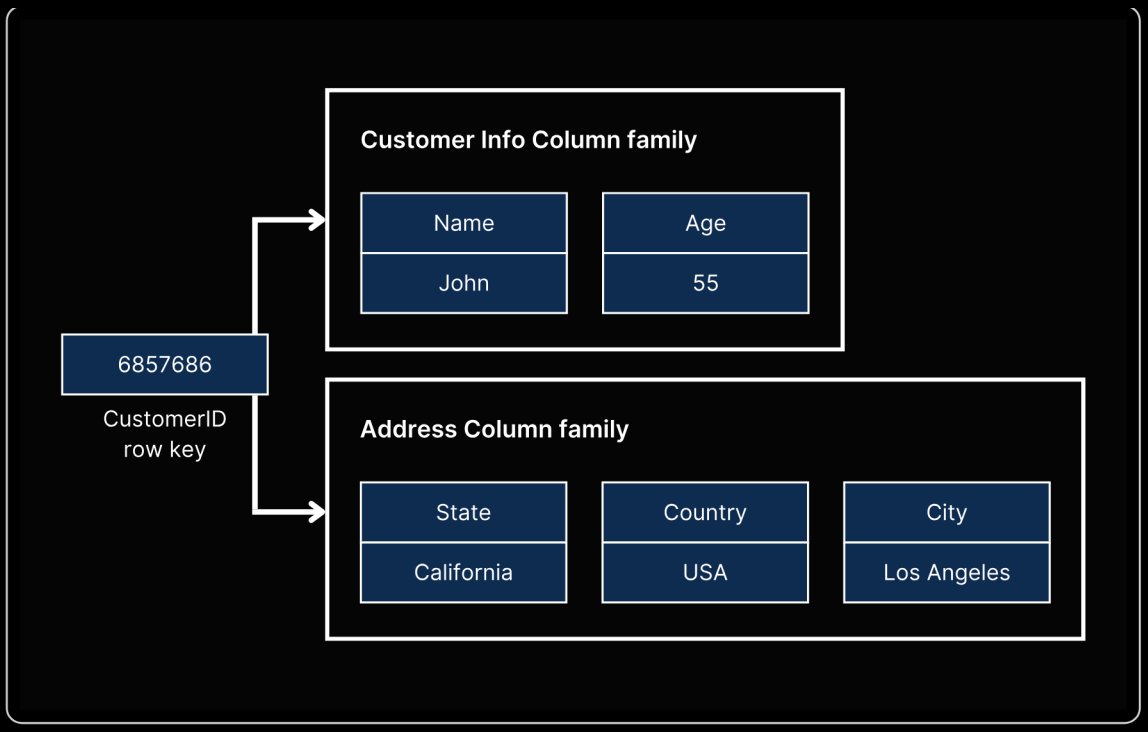
TL;DR

Documents = flexible, nested records read in one go. Model around your queries. In Postgres, `jsonb` + GIN + CHECK constraints gives you a document store with SQL around it.

16.4 Column-family and wide-column stores

What it is

A **column-family** database stores rows identified by a **row key**, with columns grouped into **column families** (named groups stored together on disk). Rows needn't share columns, and a row can have thousands — hence **"wide"**.



Row key `cust#6857686` might have family **info** (`name = "John"`, `age = 55`) and family **address** (`state = "California"`, `country = "US"`, `city = "..."`); an address query reads only the `address` family's storage. This is Google Bigtable's (2006) **column-family model**, copied by HBase. The **wide-column model** (Cassandra, ScyllaDB) adds distribution:

- A **partition key** decides which node stores the row (consistent hashing, §15.3); rows sharing it live together.
- **Clustering columns** sort rows *within* a partition, making in-partition range scans cheap.
- Replication is typically **eventually consistent** with **tunable** consistency.

```
-- Cassandra CQL (not run): one partition per server per day, rows sorted by time
CREATE TABLE metrics (
  server_id text, day date, ts timestamp, cpu_pct float, mem_mb int,
  PRIMARY KEY ((server_id, day), ts)          -- (partition key), clustering column
) WITH CLUSTERING ORDER BY (ts DESC);
SELECT ts, cpu_pct FROM metrics WHERE server_id = 'S1' AND day = '2024-12-06' LIMIT 60;
```

> **Don't confuse** *wide-column* stores (Cassandra, HBase) with **columnar** analytics databases (ClickHouse, Redshift, BigQuery, Parquet files). Columnar stores keep each *column* contiguous so `SUM(amount)` over billions of rows reads one column (§12.1); wide-column stores keep each *row partition* together by key (DDIA ch.3).

Why they fit time-series and write-heavy data

- **Sequential writes**: most use **LSM trees** — writes go to an in-memory table plus a commit log, then flush as **immutable** sorted files (**SSTables**) merged in the background (**compaction**). Writes are appends, so throughput is very high.
- **Partitioning by time range**: a partition key such as `(sensor, day)` spreads load and keeps each day together.
- **Flexible columns**: a new metric is a new column, no migration.

server_id	ts	cpu_pct	mem_mb
web-1	2024-12-06 10:00	41	1980
web-1	2024-12-06 10:01	44	2012
db-1	2024-12-06 10:00	63	1705

A monitoring feed partitioned by server, clustered by time: "last hour of web-1" is one sequential read.

	Cassandra (wide-column)	**HBase (column-family)**
Origin	Facebook (2008), combining Dynamo's distribution with Bigtable's data model	Apache clone of Google Bigtable, on Hadoop
Architecture	Leaderless ring, every node equal	Master + RegionServers, data in **HDFS** (16.9)
Scaling unit	Token ranges on the ring	**Regions** (key ranges) that split automatically as they grow
Consistency	**Tunable** per query (`ONE` , `QUORUM` , `ALL`), AP by default	**Strong** per-row consistency (CP)
Query	CQL (Cassandra Query Language: SQL-like, no joins)	Java API, get/put/scan by row key, or SQL through Apache Phoenix
Typical use	User activity feeds, messaging, IoT at global scale	Big-data random read/write next to Hadoop and Spark jobs, e.g. transaction histories for fraud analysis

Use cases: **time-series and IoT** (millions of smart-home sensors), **logs**, **activity and recommendation data** (every view and purchase per user), **real-time analytics** (counters and recent windows for dashboards and alerts). **Limits:** tables designed **per query** (the same data often written to several tables), no joins, limited secondary indexes and ad-hoc queries, and deletes create **tombstones** (deletion markers, since immutable files can't be edited) that slow reads until compaction.

The same idea in Postgres

`code/04_time_series_and_wide_rows.sql`: composite PK `(server_id, ts)` acts as partition key + clustering column, daily range partitions provide retention:

```
CREATE TABLE metrics (
  server_id text        NOT NULL,       -- like a wide-column "partition key"
  ts        timestamptz NOT NULL,       -- like a "clustering key": rows sorted by time
  cpu_pct   real,
  mem_mb    int,
  PRIMARY KEY (server_id, ts)
) PARTITION BY RANGE (ts);
-- one partition per day; 3 servers x 2 days x 1,440 minutes = 8,640 readings
EXPLAIN (COSTS OFF)
SELECT ts, cpu_pct FROM metrics
WHERE server_id = 'web-1' AND ts >= '2024-12-06 23:30' AND ts < '2024-12-07 00:00';
```

```
 Bitmap Heap Scan on metrics_2024_12_06 metrics
   Recheck Cond: ((server_id = 'web-1'::text) AND (ts >= '2024-12-06 23:30:00+00'::timestamp with time zone) AND
...)
   ->  Bitmap Index Scan on metrics_2024_12_06_pkey
         Index Cond: ((server_id = 'web-1'::text) AND (ts >= '2024-12-06 23:30:00+00'::timestamp with time zone)
AND ...)
```

One partition scanned, then a narrow `(server_id, ts)` range — exactly the wide-column access pattern (more in 16.8).

In the interview

30-second answer: "Wide-column stores like Cassandra distribute rows by a partition key and sort them within the partition by clustering columns. Writes go to LSM trees, so they're append-fast. You design one table per query and live with no joins. Consistency is tunable per request. They're great for time-series, logs and activity feeds at huge write rates. They aren't the same as columnar analytics stores."

- *"Latest 50 messages per chat" in Cassandra?* Partition key `chat_id` (plus a time bucket if chats are huge), clustering column `sent_at DESC`, `LIMIT 50`.
- *Why bucket the partition key by day?* To keep partitions bounded (Cassandra's guidance: roughly 100 MB or less each) and spread hot writes.
- *HBase vs Cassandra consistency?* HBase strong per row; Cassandra tunable, eventual by default.

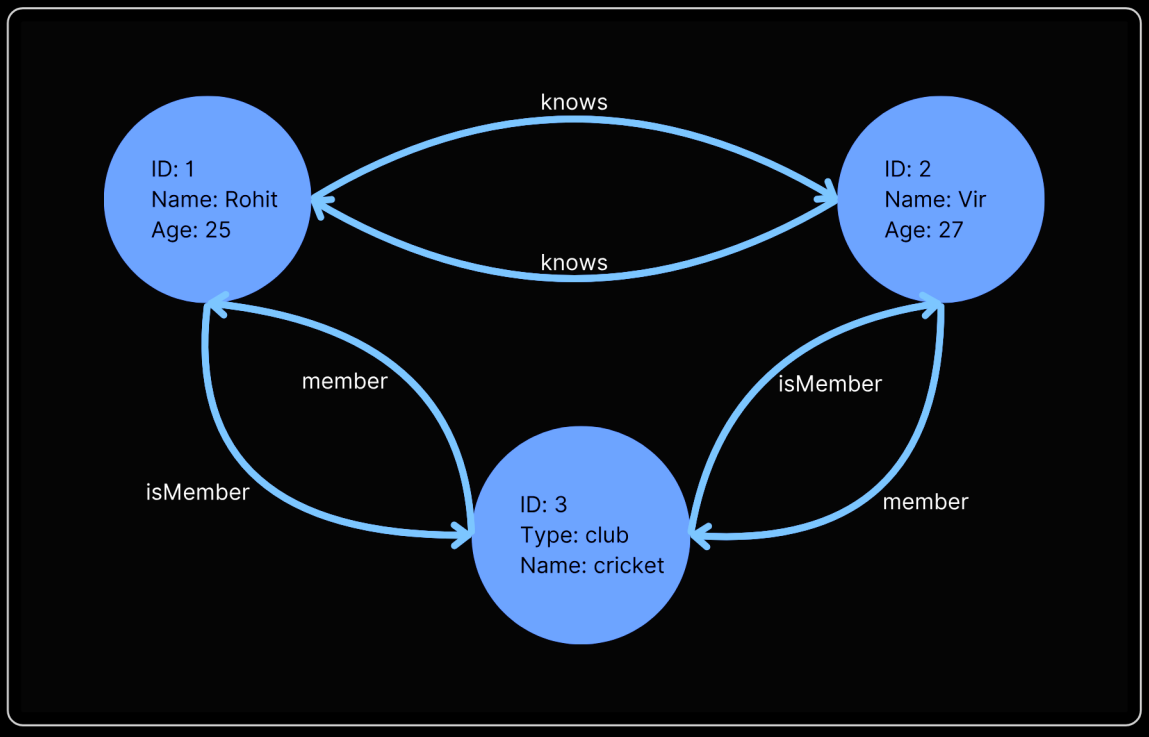
TL;DR

Wide-column = partition key for placement + clustering key for order + LSM for fast writes. One table per query. Built for massive write-heavy, time-ordered data.

16.5 Graph databases

What it is

A **graph database** stores **nodes** (people, products, places), **edges** (`KNOWS`, `BOUGHT`, `MEMBER_OF`) and **properties** on both (`name`, `since`, `weight`). Edges are first-class stored data, not computed from foreign keys at query time. Choose one when **relationships matter as much as the records**: social connections, recommendations, fraud rings, network topology, knowledge graphs, access-control hierarchies.



Relational vs graph for the same data

Relational: `person` plus a link table `knows(src, dst)`, friendships stored in both directions (so one `src = ?` lookup finds all friends):

src	dst
1 (Ava)	2 (Bilal)
1 (Ava)	3 (Chen)
2 (Bilal)	4 (Dana)
…	…

Each extra hop ("friends of friends of friends") adds **another join** — O(log n) per edge with B-tree lookups — and intermediate results can explode. **Graph:** each node stores direct pointers to its edges

(Neo4j's **index-free adjacency**), so traversal follows edges with no index lookup; cost depends on edges traversed, **not database size**, so deep, variable-length traversals stay fast.

```
// Neo4j Cypher (not run)
CREATE (a:Person {name:'Ava'})-[:KNOWS]->(b:Person {name:'Bilal'});
MATCH (me:Person {name:'Ava'})-[:KNOWS]->(f)-[:KNOWS]->(fof)
WHERE NOT (me)-[:KNOWS]->(fof) AND fof <> me
RETURN DISTINCT fof.name;                                  // friend suggestions
MATCH p = shortestPath((:Person {name:'Ava'})-[:KNOWS*]-(:Person {name:'Goran'}))
RETURN [n IN nodes(p) | n.name];
```

Advantages: relationship queries are **simpler** (patterns, not join chains), **fast at depth**, and the model is **flexible** (new node/edge types freely). Languages: **Cypher/openCypher** (Neo4j, Neptune, Memgraph), **Gremlin** (Apache TinkerPop), **SPARQL** (RDF triple stores), the ISO standard **GQL** (ISO/IEC 39075, 2024); SQL:2023 added **SQL/PGQ** for property-graph queries over relational tables.

Use cases: (1) **social networks** — mutual friends, degrees of separation, "people you may know"; (2) **recommendations** — "customers who bought this also bought…" is a two-hop traversal over users, products, categories; (3) **fraud detection** — accounts sharing devices, addresses or cards form **connected components** (groups all reachable from one another); (4) **IT dependency maps and knowledge graphs**.

Scaling limits: horizontal sharding is hard (traversals cross the network on almost every hop, so many keep the graph on one machine and scale reads with replicas); **bulk updates** by property are slower than in column or document stores; **aggregations over everything** belong in a warehouse.

Graph queries in Postgres

`code/03_graph_in_sql.sql` models seven people with symmetric `knows` edges and walks the graph with recursive CTEs (Chapter 7):

```
-- 2 hops: friend-of-friend suggestions for Ava
SELECT DISTINCT p.name AS suggestion
FROM knows k1 JOIN knows k2 ON k2.src = k1.dst JOIN person p ON p.id = k2.dst
WHERE k1.src = 1 AND k2.dst <> 1
  AND k2.dst NOT IN (SELECT dst FROM knows WHERE src = 1);

-- N hops: degrees of separation (path array prevents cycles)
WITH RECURSIVE walk(id, depth, path) AS (
  SELECT 1, 0, ARRAY[1]
  UNION ALL
  SELECT k.dst, w.depth + 1, w.path || k.dst
  FROM walk w JOIN knows k ON k.src = w.id
  WHERE k.dst <> ALL (w.path) AND w.depth < 6
)
SELECT p.name, min(depth) AS degrees
FROM walk JOIN person p USING (id) GROUP BY p.name ORDER BY degrees, p.name;
```

Output (Ava's friends are Bilal and Chen; the suggestion, degrees, and the shortest path to Goran):

```
 suggestion
------------
 Dana

  name  | degrees
--------+---------
 Ava    |       0
 Bilal  |       1
 Chen   |       1
 Dana   |       2
 Eli    |       3
 Fatima |       4
 Goran  |       5

                shortest_path
---------------------------------------------
 Ava -> Bilal -> Dana -> Eli -> Fatima -> Goran
```

Fine for small graphs and shallow depths; recursive CTEs enumerate **every path**, so on large, dense graphs work grows exponentially with depth — where a graph engine (or the Apache AGE extension, adding Cypher to Postgres) pays off.

In the interview

> **30-second answer:** "A graph database stores nodes, edges and properties, and follows edges directly, so multi-hop queries cost in proportion to the edges touched rather than the table sizes. It's ideal for social graphs, recommendations and fraud rings. It's harder to shard and weaker at bulk aggregates. For a few hops on modest data, SQL with a link table and recursive CTEs is fine."

- *Why are deep joins slow in SQL?* Each hop is another join over the whole link table (through an index), and intermediate results multiply.
- *Detect a fraud ring?* Link accounts to shared devices, cards and addresses, then find connected components or short cycles.

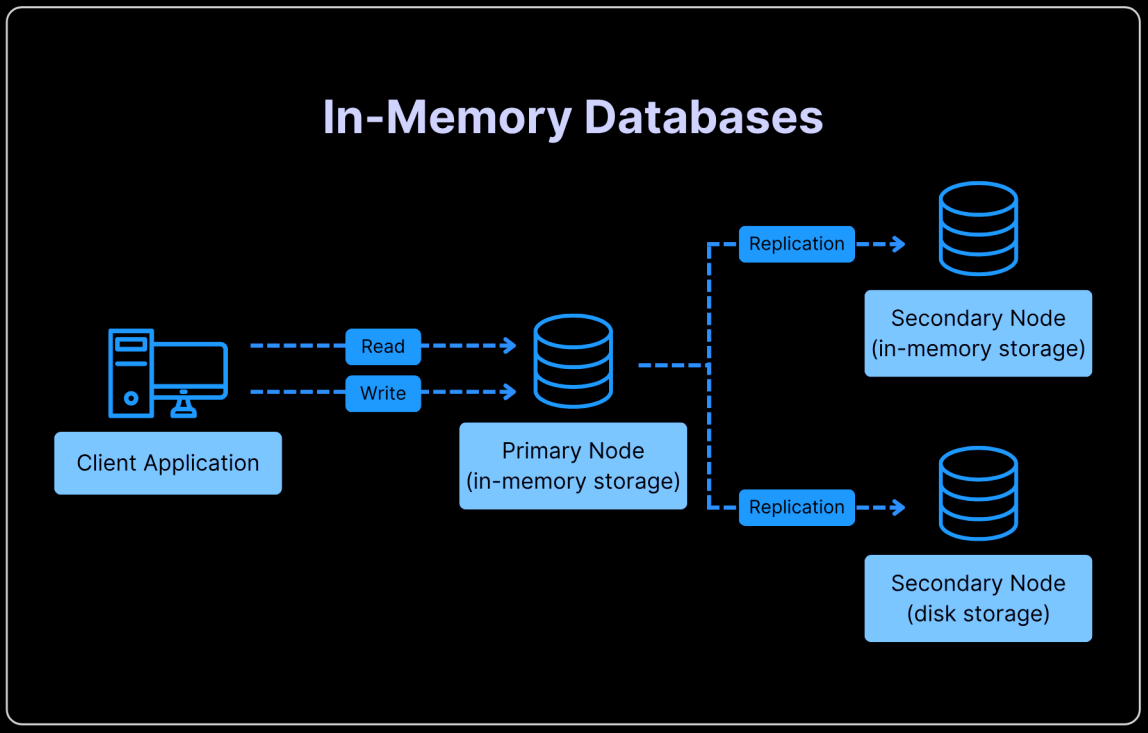
TL;DR

Graphs make relationships first-class. Use one when the questions are "how is X connected to Y". Recursive CTEs cover the small cases.

16.6 In-memory databases

What it is

An **in-memory database (IMDB)** keeps its primary copy in **RAM**: ~100 nanoseconds per access vs tens to hundreds of microseconds for an SSD read. The engine uses memory-native structures (**hash tables**; **skip lists** — sorted linked lists with "express lanes"; compact column vectors) instead of disk pages and buffer management.



Architecture: (1) a **primary node** holds working data in memory and serves reads and writes; (2) changes **replicate** to secondaries (in memory or disk-backed); (3) because RAM is **volatile**, **persistence** comes from **snapshots** and/or an **append-only / write-ahead log**, reloaded and replayed on restart (the WAL idea of §15.9).

Benefit	Example
Speed	A user's recent purchase history in well under a millisecond
Real-time processing	Live game leaderboards, fraud checks during card authorisation, IoT alerts
Memory-optimised structures	Redis's native hashes and sorted sets
High concurrency	Hundreds of thousands of operations per second per node

Limitations: volatility (no persistence = data lost on restart; persistence = extra I/O and recovery time); **cost** per GB far above disk; **scaling** means more RAM on more nodes, data bounded by total cluster memory.

Real systems:

- **SAP HANA**: in-memory, **column-oriented**, running **OLTP and OLAP on the same data** (HTAP, hybrid transactional/analytical processing) — a retailer analyses today's sales in real time and adjusts stock orders the same afternoon.
- **SingleStore** (**MemSQL** until 2020): distributed SQL with in-memory row storage plus disk-based columnstore tables, MySQL-wire-compatible (MySQL drivers work) ANSI SQL — a bank scores millions of card transactions a day for fraud in real time.
- **Redis** (16.2, 16.11), **Memcached** (cache only, no persistence), **VoltDB** (in-memory NewSQL).

Postgres note: not an IMDB, but with enough `shared_buffers` (its page cache) and OS cache its hot data is served from memory anyway; `UNLOGGED` tables (16.2) skip the WAL for cache-like data; the on-disk WAL lets it survive a crash.

In the interview

30-second answer: "In-memory databases keep data in RAM for microsecond access and use snapshots plus an append-only log, plus replicas, for durability. They're ideal for caches, sessions, leaderboards and real-time analytics. The trade-offs are cost per GB and dataset size limited by memory."

- *What's HTAP?* Transactional and analytical workloads on the same live data (SAP HANA, SingleStore). (Redis restart survival: 16.11.)

TL;DR

RAM-first storage for speed. Durability comes from logs, snapshots and replicas. The price is cost and a size ceiling.

16.7 NewSQL / distributed SQL

What it is

NewSQL (today **distributed SQL**) keeps the **relational model, SQL and ACID transactions** but **scales out** like NoSQL, with automatic sharding, replication and failover. It fills the gap between **traditional SQL** (strong consistency, but one primary; scaling writes means scaling up or manual sharding with cross-shard transactions your problem) and **NoSQL** (scale-out, but historically eventual consistency and limited transactions — risky for balances, inventory, bookings).

How it works

1. **Automatic sharding** into ranges (CockroachDB *ranges*, TiDB *regions*, Spanner *splits*) that move between nodes as they grow or get hot — a ride-hailing app's trips spread so one busy city doesn't slow the others.

2. **Consensus-based replication**: each shard is a **Raft** (CockroachDB, TiDB, YugabyteDB) or **Paxos** (Spanner) group of usually 3 or 5 replicas; a write commits once a majority stores it, so losing a minority loses nothing and needs no manual failover (§15.8).
3. **Distributed transactions** via **2PC whose coordinator state is replicated by consensus**, avoiding 2PC's blocking problem (§15.9); **serializable** isolation is the default in CockroachDB and Spanner.
4. **Clocks for global consistency**: **Google Spanner**'s **TrueTime** (GPS- and atomic-clock-backed time with a known uncertainty bound) waits out the uncertainty before acknowledging a commit ("commit wait"), giving externally consistent (**linearizable**) transactions worldwide; CockroachDB uses **hybrid logical clocks** on ordinary servers, with an uncertainty interval and retries.
5. **SQL front end**, often wire-compatible (CockroachDB and YugabyteDB speak PostgreSQL's protocol, TiDB MySQL's).

System	Notes
Google Spanner (paper 2012, Cloud Spanner service 2017)	Global, Paxos + TrueTime, strict serializability (serializable + linearizable). *Use:* a global shop keeping stock counts correct across regions during a flash sale.
CockroachDB	Postgres-compatible, Raft per range, serializable by default, survives zone or region loss. *Use:* a fintech processing payments across several data centres.
TiDB	MySQL-compatible, TiKV storage (Raft), optional columnar replica (TiFlash) for HTAP.
YugabyteDB	Reuses PostgreSQL's query layer on a Raft-replicated distributed store.
VoltDB	In-memory, partitioned, serial execution per partition.

Fits global apps needing correct transactions (payments, inventory, bookings), multi-region active-active with strong consistency, real-time queries over distributed data. Costs: higher write latency (consensus round trips, cross-region for global writes), more complex operations, a compatible-but-not-identical SQL dialect.

Feature	Traditional SQL	NoSQL	NewSQL
Data model	Relational tables	Key-value, document, wide-column, graph	Relational tables
Schema	Defined up front (JSON columns add flexibility)	Dynamic	Defined up front, with JSON support
Scaling	Vertical + read replicas, manual sharding	Horizontal, built in	Horizontal, built in
Consistency	Strong, ACID	Often eventual/tunable (BASE)	Strong, ACID (serializable)
Typical use	ERP (finance, stock, HR systems), banking, most business apps	Caches, feeds, IoT, catalogues	Global transactional apps, high-scale OLTP
Examples	PostgreSQL, MySQL, SQL Server	Redis, MongoDB, Cassandra, Neo4j	Spanner, CockroachDB, TiDB, YugabyteDB

In the interview

> **30-second answer:** "NewSQL keeps SQL and ACID but scales out. Data is auto-sharded into ranges, each replicated with Raft or Paxos, and cross-shard transactions use 2PC on top of those consensus groups, so there's no single blocking coordinator. Spanner uses TrueTime for global external consistency, and CockroachDB uses hybrid logical clocks. The cost is write latency from consensus."

- *Why is a global write slower in Spanner/CockroachDB?* A majority of replicas, possibly in other regions, must acknowledge it, and Spanner adds the commit-wait.

- *What does TrueTime solve?* Ordering transactions globally without a central clock, by waiting out a bounded clock uncertainty.

TL;DR

NewSQL = relational + ACID + automatic scale-out through consensus-replicated shards. You pay for it with latency.

16.8 Time-series and spatial databases

Time-series databases

What it is

A **time-series database (TSDB)** is optimised for **timestamped measurements** (metrics, sensor readings, prices): data arrives in time order, is mostly appended, rarely updated, and is queried by **time range**, often **downsampled** (averages per minute or hour).

- **Time-partitioned storage**: chunks per hour, day or week, so "last 7 days" touches 7 chunks and dropping old data drops whole chunks.
- **B-trees and LSM trees**: B-trees serve range scans; **LSM trees** make high-rate appends cheap (InfluxDB's engine, Cassandra; 16.4).
- **Compression**: consecutive values are similar, so **delta**, delta-of-delta and XOR-float encodings (Facebook's **Gorilla** paper, 2015) shrink data ~10×; columnar layouts help too (§12.2).

Feature	Meaning
Write optimisation	Millions of points per second
Retention policies	Delete (or downsample) data older than N days automatically
Continuous aggregates / downsampling	Per-minute data for a week, per-hour for a year
Time functions	Buckets, gap filling, interpolation, "last value"

Applications: **IoT** (home and factory sensors), **server monitoring** (**Prometheus**, its own TSDB, and the Datadog platform), **finance** (tick-by-tick prices). Products: **InfluxDB**, **TimescaleDB** (a PostgreSQL extension with automatic time partitions called *hypertables*, compression and continuous aggregates), **Prometheus**, **QuestDB**, kdb+.

Time-series in plain Postgres

The rest of `code/04_time_series_and_wide_rows.sql` uses built-in features:

```
-- Downsample: 6-hour buckets per server (date_bin: PostgreSQL 14+)
SELECT server_id, date_bin('6 hours', ts, timestamptz '2024-12-06') AS bucket,
       round(avg(cpu_pct)::numeric, 1) AS avg_cpu, max(mem_mb) AS peak_mem
FROM metrics WHERE ts >= '2024-12-06'
GROUP BY 1, 2 ORDER BY 1, 2 LIMIT 6;
```

```
 server_id |         bucket         | avg_cpu | peak_mem
-----------+------------------------+---------+----------
 db-1      | 2024-12-06 00:00:00+00 |    42.0 |     1923
 db-1      | 2024-12-06 06:00:00+00 |    54.0 |     1915
 db-1      | 2024-12-06 12:00:00+00 |    66.0 |     1922
 db-1      | 2024-12-06 18:00:00+00 |    78.0 |     1923
 web-1     | 2024-12-06 00:00:00+00 |    22.0 |     1923
 web-1     | 2024-12-06 06:00:00+00 |    34.0 |     1917
```

```
CREATE INDEX metrics_ts_brin ON metrics USING brin (ts);
-- index sizes on one day's partition (4,320 rows)
```

```
          index           |  size
--------------------------+--------
 metrics_2024_12_06_pkey   | 152 kB
 metrics_2024_12_06_ts_idx | 24 kB
```

BRIN (§12.9) stores min/max per block range — tiny and selective when rows arrive in time order (24 kB is its minimum footprint; on millions of rows it stays small while the B-tree grows with the rows).

```
-- Retention policy: dropping a day is instant (no giant DELETE, no vacuum debt).
DROP TABLE metrics_2024_12_05;
SELECT min(ts)::date AS oldest_day_kept, count(*) AS readings FROM metrics;
```

```
 oldest_day_kept | readings
-----------------+----------
 2024-12-06      |     4320
```

Spatial databases

What it is

A **spatial database** stores and queries **geometry and geography**: where things are and how shapes relate — **points** (a shop's GPS coordinates), **lines / LineStrings** (a road, a delivery route), **polygons** (a city boundary, a delivery zone).

B-trees order one dimension and can't answer "near this point" in 2-D. Spatial indexes can:

- **R-tree**: nested **bounding rectangles**; a query descends only into rectangles overlapping the search area ("stores within 10 km" checks a few boxes). PostgreSQL's **GiST** implements R-tree-style indexing for geometric types and PostGIS.
- **Quadtree**: recursively splits space into **four quadrants** until each cell holds few points — good for uneven density (Postgres SP-GiST).
- **Geohash**: latitude/longitude as a short **string** whose shared prefix means nearby, so lookups run on a plain key-value store or B-tree (`u09tv` covers central Paris); points near a cell border can be close yet in different cells, so neighbouring cells are checked too. Related grids: Uber's **H3** (hexagons), Google's **S2**.

Queries: **proximity** ("restaurants within 5 km", "3 nearest drivers" — k-nearest-neighbour, KNN); **intersection / containment** ("which zones overlap?", "is this address in the service area?"); **routing** (pgRouting over road networks). Applications: location-based services, smart cities and logistics (traffic, bus and rubbish-collection routes), geo-analytics (where customers cluster). Products: **PostGIS** (the standard PostgreSQL extension), **BigQuery GIS**, Oracle Spatial, SQL Server spatial types, MongoDB 2dsphere indexes, Redis GEO commands.

Spatial queries with Postgres built-ins

PostGIS isn't installed on the book server, so `code/05_spatial_builtin.sql` uses **built-in geometric types** (flat x/y plane, units km) with a **GiST** index:

```
CREATE TABLE store (id int PRIMARY KEY, name text, loc point NOT NULL);
-- 6 named stores + 5,000 kiosks scattered on the grid
CREATE INDEX store_loc_gist ON store USING gist (loc);    -- R-tree-like index

SELECT name, round((loc <-> point(2, 2))::numeric, 2) AS km          -- <-> = distance
FROM store WHERE id < 100 ORDER BY loc <-> point(2, 2) LIMIT 3;
SELECT name FROM store WHERE id < 100 AND loc <@ circle(point(0, 0), 5) ORDER BY name;   -- radius
SELECT name FROM store WHERE id < 100 AND loc <@ box(point(2, 3), point(7, 9)) ORDER BY name; -- zone
EXPLAIN (COSTS OFF) SELECT id FROM store ORDER BY loc <-> point(2, 2) LIMIT 3;
```

```
  name    |  km
----------+------
 Riverside | 2.24
 Central   | 2.83
 Mall      | 3.16

  name       (within 5 km of (0,0))
-----------
 Central
 Mall
 Old Town
 Riverside

  name       (inside the delivery-zone box)
-----------
 Harbour
 Riverside

 Limit
   ->  Index Scan using store_loc_gist on store
         Order By: (loc <-> '(2,2)'::point)
```

The last plan is **KNN search through GiST**: the index returns points in distance order, so `LIMIT 3` stops after three, with no full sort. For real latitude/longitude on the curved Earth use PostGIS (`4326` = **WGS 84**, the coordinate system GPS uses):

```sql
-- PostGIS (not run)
CREATE EXTENSION postgis;
CREATE TABLE shop (id int PRIMARY KEY, name text, geom geography(Point, 4326));  -- WGS 84 lon/lat
CREATE INDEX ON shop USING gist (geom);
SELECT name FROM shop
WHERE ST_DWithin(geom, ST_MakePoint(2.3522, 48.8566)::geography, 2000);        -- within 2 km of Paris centre
SELECT z.name FROM zone z JOIN shop s ON ST_Intersects(z.area, s.geom);
```

Feature	Time-series	Spatial
Data focus	Timestamped measurements	Coordinates, lines, areas
Indexing	Time partitions/chunks, B-tree/LSM, BRIN, compression	R-tree (GiST), quadtree, geohash/H3/S2
Typical query	"Average CPU per hour, last 24 h"	"Shops within 10 km", "which zone contains this point"
Applications	IoT, monitoring, finance	Maps, logistics, smart cities

In the interview

30-second answer: "Time-series stores partition by time, append with LSM or B-trees, compress heavily and apply retention and downsampling. In Postgres that's time partitions, BRIN, `date_bin`, or TimescaleDB. Spatial stores use R-trees, quadtrees or geohashes for near/within/intersects queries. In Postgres that's GiST, with PostGIS for real geography."

- *Design "drivers near me" at scale?* Driver locations in a geohash/H3 cell index (Redis GEO or an in-memory grid) updated every few seconds; query neighbouring cells, then rank by exact distance.
- *What does an R-tree store in internal nodes?* Bounding rectangles of its children.

TL;DR

Time-series: partition by time, compress, retain, downsample. Spatial: R-tree/GiST, quadtrees and geohashes for near/within/intersects. PostGIS and TimescaleDB bring both to Postgres.

16.9 Big data and distributed processing frameworks

What "big data" means

Big data is too large, too fast or too varied for one traditional server to store and process in reasonable time — the **3 Vs**: **volume** (terabytes to petabytes; 1 PB = 1,000 TB), **velocity** (continuous fast arrival — clickstreams, sensors), **variety** (tables, semi-structured JSON and logs, unstructured text, images, video); veracity and value are often added. The challenges are storage, processing time and scaling; the answer, as in Chapter 15: **spread storage and computation over many machines, in parallel, moving computation to the data**.

Hadoop: HDFS + MapReduce

- **HDFS (Hadoop Distributed File System)** splits files into large **blocks** (128 MB default), each **replicated** (3 copies default) across nodes.
- **MapReduce**: **map** (each node turns its local blocks into key/value pairs) → **shuffle** (pairs grouped by key across the network, so all values for a key reach the same reducer) → **reduce** (each group combined). Intermediate results go to disk between stages — robust but slow for multi-step jobs.

Example: a retailer totals years of HDFS sales files per product per month. `code/mapreduce_word_count.py` runs the phases over three "nodes" of order-event logs, with a **combiner** pre-aggregating on each node before the shuffle:

```
def map_phase(lines):                # runs locally on each node
    for line in lines:
        action, item = line.split()
        yield (f"{action}:{item}", 1)
# ... combine per node, then send each key to reducer  hash(key) % 2 ...
```

```
map+combine on node-1: [('buy:laptop', 1), ('view:laptop', 2), ('view:tshirt', 1)]
map+combine on node-2: [('buy:book', 1), ('view:book', 1), ('view:laptop', 1)]
map+combine on node-3: [('buy:laptop', 1), ('view:laptop', 1), ('view:tshirt', 1)]
reducer 0: {'buy:laptop': 2, 'view:book': 1}
reducer 1: {'buy:book': 1, 'view:laptop': 4, 'view:tshirt': 2}
```

Each key reaches exactly one reducer, so totals are complete — `SELECT action, item, count(*) … GROUP BY action, item`, across machines.

Apache Spark

Spark fixes MapReduce's slowness: **in-memory processing** between steps (originally **RDDs**, resilient distributed datasets recomputable from their **lineage** — the recorded chain of steps — after a node failure; today mostly **DataFrames**, distributed tables with named columns), so iterative jobs (machine learning, multi-stage queries) are much faster; **many workloads in one engine** (batch, **Structured Streaming**, MLlib, GraphX); **APIs** in Python (PySpark), Scala, Java, R and SQL. *Example:* a social platform turns behaviour logs into recommendation features.

SQL-on-Hadoop

Tool	What it is
Apache Hive	A data warehouse over HDFS; HiveQL compiles to distributed jobs (originally MapReduce, later Tez or Spark). Large batch queries on structured files.
Presto / Trino	Distributed SQL engines (Presto from Facebook, 2012; Trino its fork) for **fast, interactive** queries that **federate** many sources as one: HDFS/S3 files, MySQL, Postgres, Kafka…
Spark SQL	SQL and DataFrames on Spark, sharing one optimiser with the other Spark APIs

Example: a retailer uses Trino to join in-store sales (Postgres) with online clickstream data (Parquet on S3) in one query.

Real-time: Kafka and Flink

- **Apache Kafka**: a distributed, replicated, append-only **log** used as a message broker. **Producers** append events to **topics** (named streams split into partitions); **consumers** read at their own pace and can replay. *Example:* GPS pings from delivery vans stream to live dashboards.
- **Apache Flink**: a **stream processor** for low-latency, **stateful** computations (windows, joins, pattern detection) with **event-time** handling (when an event happened, not when it arrived) and **exactly-once** state (each event affects results once, even after retries) via distributed checkpoints (§15.9). *Example:* flagging unusual trades within milliseconds.

Common architecture: events into Kafka → Flink or Spark Streaming for real-time views → a **data lake** (cheap raw files in any format, usually on object storage) or warehouse for **batch** analysis with Spark, Hive or Trino. **Change data capture (CDC)** — publishing every insert/update/delete from the database's change log as an event, e.g. Debezium from Postgres into Kafka — feeds the same pipeline from your OLTP database (Batch and streaming patterns).

Area	Example
IoT / sensors	Smart factories analyse machine telemetry with Spark to predict failures
Recommendations	Streaming services update suggestions from live viewing events with Flink
Fraud detection	Banks score transactions from Kafka streams with Spark/Flink in real time
Data integration	Retailers combine store and web sales with Presto/Trino

In the interview

> **30-second answer:** "Big data is volume, velocity and variety beyond one server. Hadoop stores blocks replicated in HDFS and processes them with MapReduce (map, shuffle, reduce, via disk). Spark keeps data in memory and covers batch, streaming, ML and SQL. Hive, Trino and Spark SQL put SQL on top. For real time, Kafka is the durable event log and Flink does stateful, exactly-once stream processing."

- *Why is Spark faster than MapReduce for iterative jobs?* It keeps intermediate data in memory instead of writing it to HDFS after every step.
- *What is a combiner?* A local reduce before the shuffle that shrinks network traffic; only for associative operations like sum and count.
- *Kafka vs Flink?* Kafka stores and transports event streams; Flink computes on them.

TL;DR

Split data into replicated blocks, move code to data, map → shuffle → reduce. Spark does it in memory. SQL engines sit on top. Kafka + Flink handle real time.

16.10 Choosing a database

Start from the **access patterns and guarantees you need**, not fashion. For most new applications the first answer is still **PostgreSQL** (plus JSONB, partitioning, PostGIS or TimescaleDB as needed); add a specialised store when a requirement clearly exceeds it.

If you need…	Consider	Why	Postgres can go far with
General business data, joins, constraints, transactions	**PostgreSQL / MySQL**	ACID, SQL, mature tooling	— (this is its home)
Sub-millisecond cache, sessions, counters, rate limits, leaderboards	**Redis** (Memcached for a pure cache)	In memory, TTL, data structures	`UNLOGGED` tables for moderate load
Flexible, nested, per-item attributes, read as whole objects	**MongoDB**, Firestore, CouchDB (offline sync)	Document model, easy scale-out	`jsonb` + GIN + CHECK
Massive write throughput, time-ordered per key, multi-region always-on	**Cassandra / ScyllaDB**, HBase/Bigtable	LSM writes, partitioned wide rows, tunable consistency	Partitioning + BRIN up to a point
Deep relationship queries (paths, communities, fraud rings)	**Neo4j**, Neptune	Traversals cost per edge touched	Recursive CTEs for shallow/small graphs
Metrics and IoT with retention and downsampling	**TimescaleDB**, InfluxDB, Prometheus	Time chunks, compression, continuous aggregates	Range partitions + `date_bin` + BRIN
Maps, "near me", zones	**PostGIS**, BigQuery GIS, Redis GEO	Spatial types + R-tree indexes	Built-in `point` / `box` + GiST (planar only)
SQL + ACID beyond one node, multi-region strong consistency	**CockroachDB, Spanner, YugabyteDB, TiDB**	Consensus-replicated shards	Citus for scale-out Postgres
Ad-hoc analytics over billions of rows	**Columnar warehouse**: BigQuery, Snowflake, Redshift, ClickHouse	Column storage, vectorised scans (batches of values per CPU instruction)	Read replicas and summary tables
Petabyte batch and stream processing	**Spark, Trino, Kafka + Flink**	Distributed compute over a data lake	CDC out of Postgres into these
Full-text search with relevance ranking and facets	**Elasticsearch / OpenSearch**	Inverted indexes, scoring	`tsvector` + GIN full-text search (Chapter 14)

Questions to ask: (1) top queries — joins, ranges, aggregations or traversals? (2) consistency per piece of data (§15.6) — money strong, like counts eventual; (3) read/write ratio, peak throughput, size in 3 years; (4) single or multi-region, RPO/RTO (§15.10); (5) what the team can operate — each extra database adds backups, monitoring, upgrades and on-call load.

Polyglot persistence. Large systems use several stores, each for what it does best — Postgres for orders and payments, Redis for sessions and caching, Elasticsearch for search, Kafka + a warehouse for analytics — all fed by CDC from the primary so there's **one source of truth**. Case studies: Chapter 17.

In the interview

> **30-second answer:** "I start from access patterns and consistency needs. Relational Postgres is the default for transactional data. I add Redis for caching and sessions, a document store if data is truly schemaless and read as whole objects, Cassandra for huge write-heavy time-ordered data, a graph DB for deep relationship queries, a TSDB for metrics, NewSQL if I need SQL + ACID across regions, and a columnar warehouse for analytics. Every extra store is operational cost, so each must earn its place."

- *Like counts vs balances?* Counts can be eventually consistent; balances need strong consistency and ACID.

16.11 Inside two workhorses: Elasticsearch (and the ELK stack) and Redis

Two non-relational systems sit next to almost every production relational database: a **search engine** for text search and log analytics, and **Redis** for caching and fast data structures. §16.2 and §16.10 said *when* to use them; this section explains *how* they work and where they fail.

Search engines: what it is

Elasticsearch (and its open-source **fork OpenSearch**, created by AWS in 2021 after Elastic changed its licence) is a distributed search and analytics engine built on **Apache Lucene**. You send JSON documents; every field becomes searchable within about a second, with relevance-ranked full-text queries, filters and aggregations over billions of documents. Solr is an older Lucene-based alternative.

The problem it solves

`LIKE '%running shoe%'` can't use a B-tree, scans every row, misses "shoes" and "Running", and has no useful order. Users expect typo tolerance, stemming, **relevance ranking** and **facets** ("Brand: Nike (42)"). PostgreSQL's `tsvector` + GIN (§14.6, which compares the two) covers the basics; a dedicated engine takes over when search is core or the data is huge.

How it works — the inverted index

A B-tree maps *row → values*; an **inverted index** maps each **term → the documents containing it** (the **posting list**).

1. **Analysis.** At index time an **analyzer** turns text into terms: *character filters* (strip HTML), a *tokenizer* (split into **tokens**, i.e. words), *token filters* (lowercase, drop stop words, stem "running" → "run", add synonyms). The same analyzer runs on the query, so "Running SHOES" and "running shoe" meet on the same terms. A `text` field is analysed; a `keyword` field is stored whole for exact filters, sorting and aggregations.
2. **Matching.** `run AND trail` intersects two sorted posting lists — fast even for millions of entries.
3. **Scoring. BM25** (default since Elasticsearch 5): a term counts more if frequent in *this* document (term frequency, with diminishing returns), rare across *all* documents (inverse document frequency), and the document is short (length normalisation).

`code/inverted_index.py` builds all three in miniature, splits five products over two shards and merges per-shard results:

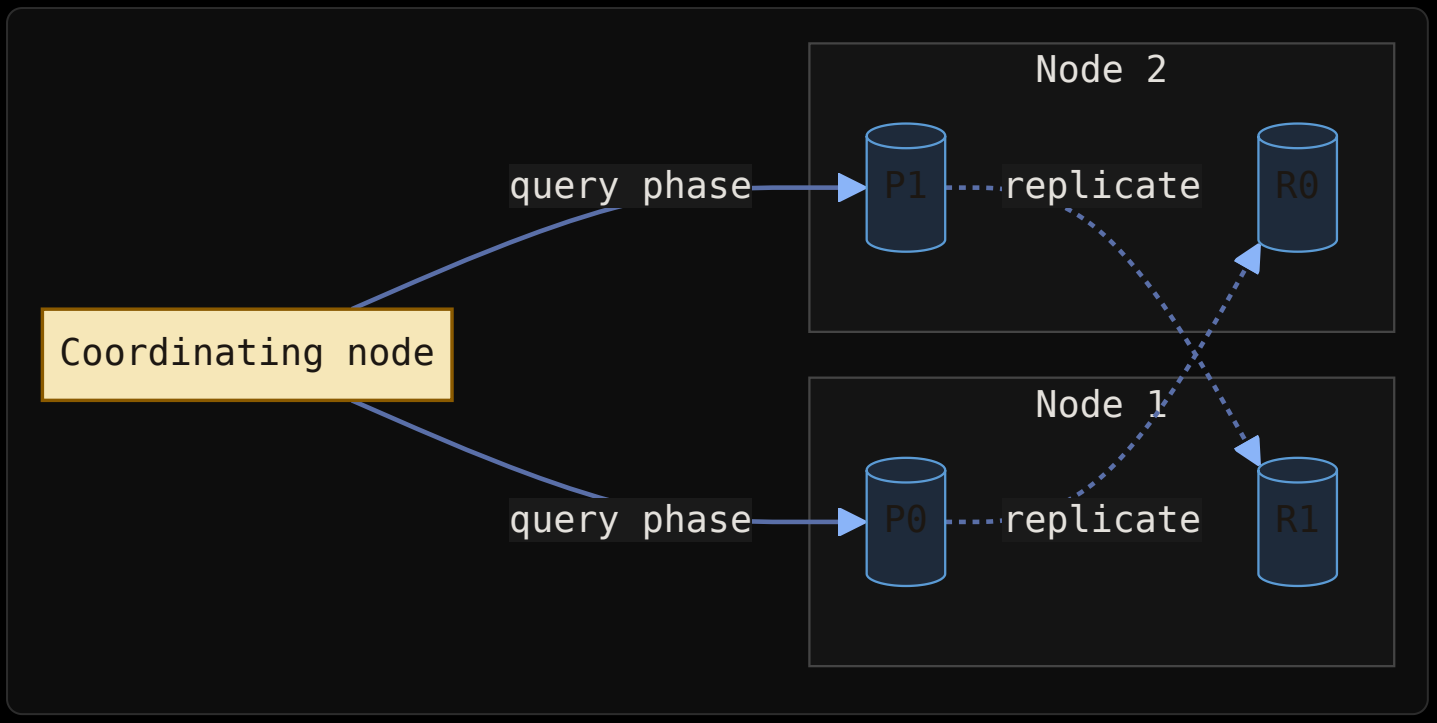
```
analyze('Running shoes for the TRAIL') -> ['run', 'shoe', 'trail']
shard 0: docs [2, 4]
shard 1: docs [1, 3, 5]
postings['run'         ] -> {1: 2, 3: 1, 5: 1}
postings['trail'       ] -> {1: 1, 2: 1, 4: 1}
postings['shoe'        ] -> {1: 1, 3: 1}
postings['hik'         ] -> {2: 1, 5: 1}
postings['lightweight' ] -> {1: 1, 5: 1}
docs matching run AND trail: [1]
query terms: ['trail', 'run', 'shoe']
shard 0 top hits: [(2, 0.578), (4, 0.53)]
shard 1 top hits: [(1, 2.123), (3, 1.391), (5, 0.53)]
merged top 3: [(1, 2.123), (3, 1.391), (2, 0.578)]
  doc 1: Running shoes for trail running, lightweight
  doc 3: Road running shoes with extra cushioning
  doc 2: Waterproof hiking boots for the trail
```

The toy stemmer turns "hiking" into `hik`; real stemmers (Porter, Snowball) and dictionary lemmatisers do better. Doc 4 ("Trail mix snack") matches `trail` yet loses to documents matching more, rarer terms.

How it works — shards, replicas and near-real-time search

- An **index** (≈ table) is split into **primary shards**, each a complete Lucene index. A document goes to `shard = hash(_routing or _id) % number_of_primary_shards`, so the primary shard count is fixed at creation (changing it means `_split`, `_shrink` or reindexing).

- Each primary has **replica shards** on other nodes for failover and read capacity; writes go to the primary, which forwards them to in-sync replicas.
- A search is **scatter-gather**: the coordinating node queries one copy of every shard (*query phase*: each returns top-N ids and scores), merges, then fetches the winners (*fetch phase*). **Deep pagination** (e.g. from hit 10,000) is expensive because every shard returns 10,000+ hits; use `search_after`.
- Lucene writes into small, **immutable segments**. A **refresh** (every 1 s by default) opens the latest segment for search — hence **near-real-time** (searchable about a second after indexing). A **translog** (write-ahead log) makes acknowledged writes durable before the next Lucene commit (**flush**); background **merges** combine segments and purge deleted documents.

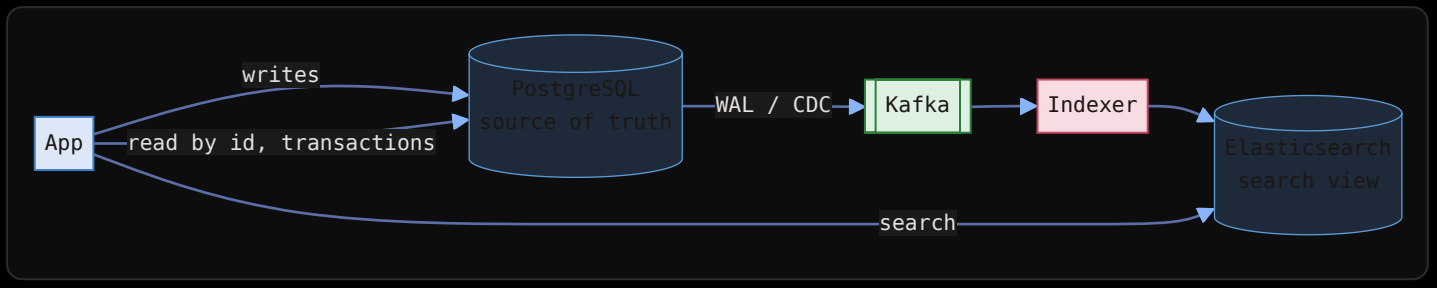


The ELK stack

ELK = **Elasticsearch** (store and search) + **Logstash** (ingest: parse, enrich, route) + **Kibana** (dashboards, search UI); lightweight shippers (**Beats**, now largely **Elastic Agent**) were added, and Elastic calls the set the **Elastic Stack**. Typical log pipeline: JSON logs → an agent ships them (often via Kafka as a buffer) → ingest parsing → time-based indices (`logs-2026.09.28`) with **index lifecycle management** moving old indices to cheaper nodes and deleting them after retention. OpenSearch offers the same shape (OpenSearch Dashboards instead of Kibana); Loki and ClickHouse are popular cheaper log alternatives.

When to pair it with a database — and when not

Keep the **relational database as the source of truth** and the search index as a derived, rebuildable view, fed by **CDC** (e.g. Debezium reading the Postgres WAL → Kafka → an indexer) or an **outbox table** (§14.3) — not by **dual writes** from the application, which drift apart when one write fails.



Do **not** make Elasticsearch the primary store for money or orders: no multi-document transactions, no joins, eventual visibility (refresh), mapping changes need reindexing. Stay with Postgres full-text search while data fits one database and relevance needs are simple.

Redis internals: what it is

§16.2 introduced **Redis**. (Redis moved to **source-available** licences — readable code with usage restrictions — in 2024 and added the AGPLv3 open-source option in 2025; the Linux Foundation's **Valkey** fork keeps the BSD licence. The internals below apply to both.)

How it works — one thread, an event loop

Redis executes all commands on **one main thread**, using **I/O multiplexing** (`epoll` on Linux, `kqueue` on macOS/BSD — OS calls reporting which of many connections have data ready) to watch thousands of client sockets and run each ready command to completion, one at a time.

- **Why it is fast anyway:** data in RAM, O(1) or O(log n) commands on compact structures, no locks or context switches (the CPU pausing one thread for another). One instance handles on the order of 100,000+ simple operations per second; the bottleneck is usually the network.
- **Why every command is atomic:** nothing else runs in the middle of `INCR` or a Lua script / `MULTI`–`EXEC` block.
- **The catch:** one slow command blocks *everyone* — `KEYS *`, `HGETALL` on a hash with millions of fields, `DEL` of a huge key, a long Lua script. Use `SCAN` (iterates in small batches), `UNLINK` (frees memory in a background thread) and keep keys small. Since Redis 6, optional **I/O threads** parallelise socket reads/writes, but command execution stays single-threaded.
- **Expiry** is lazy (checked when a key is touched) plus an active background sampler deleting a portion of expired keys each cycle.

How it works — data structures and encodings

Each type switches internal encoding by size: small hashes, lists and sorted sets use a compact **listpack** (one contiguous block), then convert to a **hash table**, **quicklist** (linked list of listpacks) or **skip list + hash table** as they grow. The sorted set's skip list gives O(log n) insert, rank and range — the classic leaderboard structure. Sets of small integers use an **intset**. `OBJECT ENCODING key` shows a key's encoding.

How it works — persistence: RDB and AOF

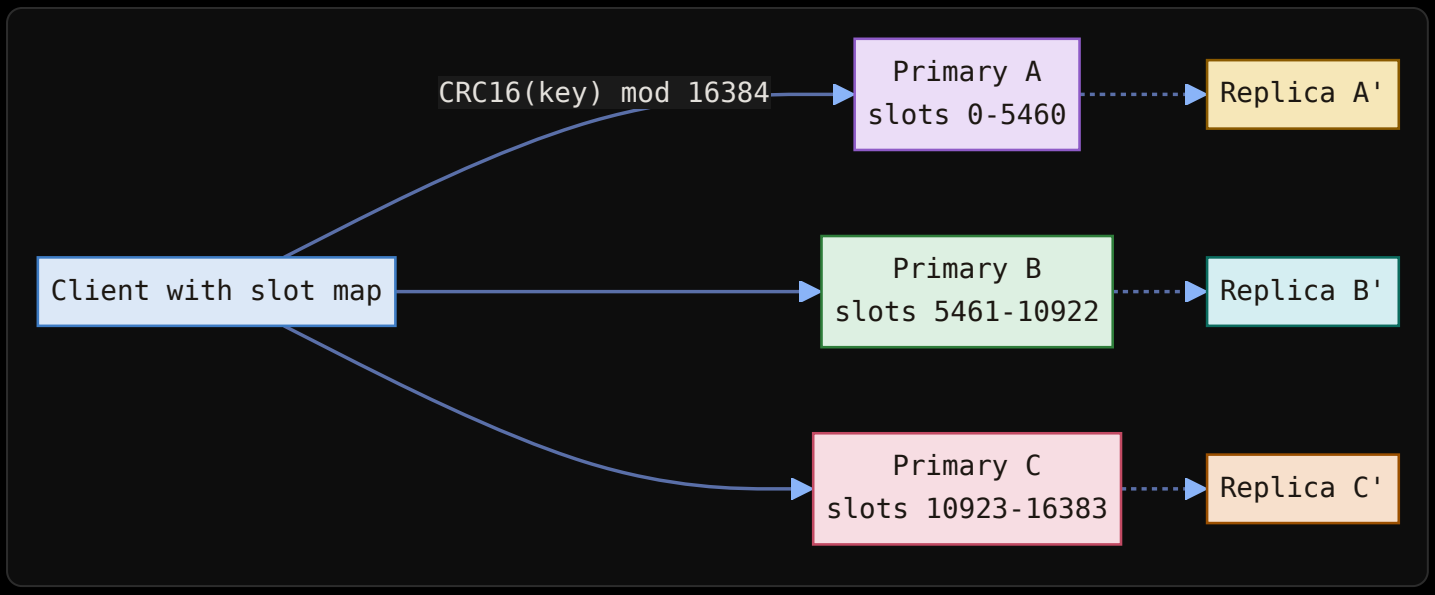
	RDB snapshot	**AOF (append-only file)**
What	point-in-time binary dump of the whole dataset	log of every write command
How	`fork()` (the OS call cloning a running process); the child writes the snapshot while the parent serves; the OS copies memory pages only when they change (**copy-on-write**)	appended on each write; `fsync` policy `always`, `everysec` (default) or `no`; rewritten in the background to stay compact
Data loss on crash	everything since the last snapshot (minutes)	about 1 second with `everysec`
Restart speed	fast load	slower replay (hybrid AOF with an RDB preamble helps)

Most deployments enable both, or run Redis as a pure cache with persistence off. Watch memory during `fork()`: a write-heavy instance can briefly need much more RAM as pages are copied.

How it works — replication, Sentinel and Cluster

- **Replication** is asynchronous primary → replicas: one full sync (RDB transfer), then streamed commands; after a short disconnect a replica resumes from the primary's **replication backlog** (`PSYNC`). An acknowledged write can be lost if the primary dies before replicating it (`WAIT` reduces, not removes, that window).
- **Sentinel** monitors primaries and promotes a replica on failure (HA without sharding).
- **Redis Cluster** shards over **16,384 hash slots**: `slot = CRC16(key) mod 16384` (CRC16 = a fast 16-bit checksum used as a hash); each primary owns a slot range. Clients cache the slot map; a wrong-node request gets `MOVED` (or `ASK` during slot migration). Multi-key commands work only within one slot —

force that with a **hash tag**: `{user:42}:cart` and `{user:42}:profile` hash only the braced part. Nodes **gossip** (periodically swap status with a few random peers) to detect failures and promote replicas.



How it works — eviction when memory is full

With `maxmemory` set, `maxmemory-policy` decides what to drop: `noeviction` (default: writes fail), `allkeys-lru` / `allkeys-lfu` (the usual cache choice), `volatile-lru` / `volatile-lfu` / `volatile-ttl` (only keys with a TTL), or `allkeys-random` / `volatile-random`. LRU and LFU are **approximated** by sampling a few keys (`maxmemory-samples`, default 5) and evicting the best candidate, avoiding the memory of a full LRU list. Memcached, by contrast, is multi-threaded, strings-only, without persistence or replication, using a **slab allocator** (memory pre-split into fixed-size chunks by size class) with per-slab LRU — simpler, good for a pure cache, without Redis's data structures.

Trade-offs

	Elasticsearch / OpenSearch	Redis
Best at	relevance search, filters + facets, log analytics	sub-millisecond reads/writes, counters, sessions, leaderboards, rate limits
Consistency	near-real-time (≈1 s refresh), no multi-doc transactions	atomic single commands/scripts; async replication can lose recent writes
Scaling	shards (fixed count per index) + replicas	Cluster hash slots + replicas; data bounded by RAM
Role	derived view fed from the source of truth	cache or fast secondary store; primary only if losing ~1 s of writes is acceptable

Common mistakes

- **Dual writes** to Postgres and Elasticsearch from the app → use CDC or an outbox so the index can always be rebuilt.
- **Too many small shards** (thousands of tiny indices) → each costs JVM heap and file handles; aim for roughly 10–50 GB per shard.
- **Expecting read-your-writes from search** → a new document appears after the next refresh; read it back by id from the database, or use `refresh=wait_for` sparingly.
- `KEYS *` **or giant keys in production Redis** → blocks the single thread; use `SCAN`, split big keys, `UNLINK`.
- **No** `maxmemory` **/** `noeviction` **on a cache** → writes fail when memory fills; pick an eviction policy.
- **Multi-key operations across Cluster slots** → `CROSSSLOT` error; design keys with hash tags.

In the interview

> **30-second answer:** "Elasticsearch stores an inverted index per shard: analysers turn text into terms, posting lists map terms to documents, and BM25 ranks matches. An index is split into a fixed number of primary shards with replicas; queries scatter to every shard and the results are merged. New documents become searchable after a one-second refresh, so it is near-real-time. I keep Postgres as the source of truth and feed Elasticsearch via CDC. Redis runs commands on a single thread with an epoll event loop, so each command is atomic but a slow one blocks everyone; it persists with RDB snapshots and/or an AOF, replicates asynchronously, shards over 16,384 hash slots in Cluster mode, and evicts with approximated LRU or LFU when it hits `maxmemory`."

- *Why is Redis single-threaded yet fast?* → In-memory data, no locks, cheap commands, I/O multiplexing; the network is usually the bottleneck.
- *RDB vs AOF?* → Snapshots are compact and fast to load but lose minutes; AOF with `everysec` loses about one second but replays slower. Many use both.
- *Why can't you change Elasticsearch's primary shard count in place?* → Routing is `hash(id) % shards`; changing the divisor moves most documents, so you split, shrink or reindex.
- *Two keys in one `MULTI` on Redis Cluster?* → Same hash tag, so they share a slot.

TL;DR

Search engines = analysers + inverted index + BM25, split into fixed primary shards with replicas, near-real-time via 1 s refresh; ELK = Elasticsearch + Logstash + Kibana for logs; pair with a relational source of truth through CDC. Redis = single-threaded event loop (atomic, but beware slow commands), size-adaptive encodings, RDB + AOF persistence, async replication, 16,384-slot Cluster with hash tags, approximated LRU/LFU eviction.

Sources: Elasticsearch Guide (near real-time search, shards and replicas, text analysis, similarity/BM25, index lifecycle management), Apache Lucene documentation, Robertson & Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond" (2009), Redis documentation (persistence, replication, cluster specification, eviction, `OBJECT ENCODING`, I/O threads), Memcached wiki, Debezium documentation.

Chapter 16 TL;DR

Model	One-line summary	Examples	In Postgres
Key-value	Get/put by key, fastest possible	Redis, Riak	PK table, `UNLOGGED`, UPSERT
Document	Flexible nested JSON per record	MongoDB, CouchDB	`jsonb` + GIN
Wide-column	Partition key + clustering order + LSM writes	Cassandra, HBase	Composite PK + partitions
Graph	Nodes and edges, fast multi-hop traversal	Neo4j, Neptune	Link table + recursive CTE
In-memory	RAM-first, persisted by snapshot + log	SAP HANA, SingleStore, Redis	Buffer cache, unlogged tables
NewSQL	SQL + ACID + auto-sharding via Raft/Paxos	Spanner, CockroachDB, TiDB	Citus (scale-out)
Time-series	Time chunks, compression, retention	TimescaleDB, InfluxDB	Range partitions + BRIN + `date_bin`
Spatial	Geometry types + R-tree style indexes	PostGIS	`point` / `box` + GiST, PostGIS
Big data	Distributed batch and stream processing	Hadoop, Spark, Trino, Kafka, Flink	CDC out to them
Search engine	Analyzer + inverted index + BM25; shards + replicas; ~1 s refresh	Elasticsearch, OpenSearch (ELK for logs)	`tsvector` + GIN

Redis internals (16.11): one command thread + epoll event loop, RDB/AOF persistence, async replication, 16,384 Cluster slots, approximated LRU/LFU eviction. BASE vs ACID is a spectrum. Choose by access pattern and consistency need, default to Postgres, and add specialised stores only when they clearly earn their operational cost.

Next: Chapter 17 — Schema design case studies.

Chapter 17 — Schema Design Case Studies

Chapters 10–16 gave you the tools one at a time. Interviews and real projects say "design the database for a hotel booking site" and expect you to go from a vague brief to tables, keys, the queries that matter, and a view on what breaks at scale. This chapter does that thirteen times with the same template, so the method becomes a habit.

What you'll learn: (1) a repeatable eight-step template plus the mapping rules and judgement calls every case reuses; (2) nine classic business systems — hospital, hotel, bank, Instagram-style photo sharing, e-commerce, food delivery, library, ticket booking, warehouse; (3) four large-scale products — Quora, TikTok, Airbnb, Snapchat — where the relational core meets caches, **object storage** (services such as Amazon S3 storing whole files by key) and NoSQL (Chapter 16); (4) four designs built for real on PostgreSQL (e-commerce, ticket booking with seat-hold concurrency, library, Instagram feed) with real output; (5) the recurring patterns — junction tables, status enums, soft deletes, audit columns, polymorphic associations, denormalized counters — compared in one table at the end (§17.14).

How to run the examples. `00_schema.sql` creates four schemas — `shop`, `tix`, `lib`, `insta` — with small invented data; files `01`–`05` in `code/` each run on a fresh copy:

```
bash tools_run_sql.sh sql '17-*'                          # run this chapter's files
bash tools_pg.sh psql -d ch_17 -f sql/chapters/17-schema-design-case-studies/code/02_ticket_booking.sql
```

Every "Output:" block is real psql output (command tags like `UPDATE 2` trimmed). This chapter **applies** earlier material: modeling and ER notation (Chapter 10), normal forms (Chapter 11), indexes and `EXPLAIN` (Chapter 12), locking and isolation (Chapter 13), partitioning/sharding/replication (Chapter 15), NoSQL families (Chapter 16).

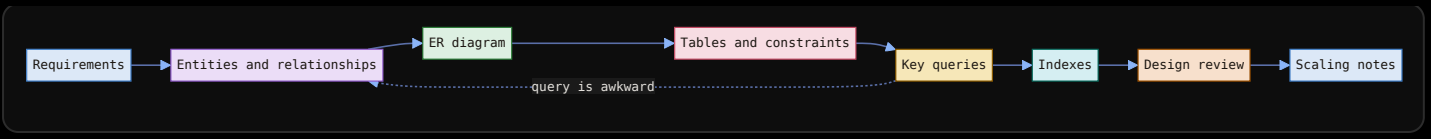
17.0 The template

What it is

A **schema-design case study** is a short, structured answer to "what tables would you build for X?" — the modeling process of §10.3 compressed into interview-sized steps.

How it works

1. **Requirements.** What users do (functional) and the numbers and guarantees that matter (non-functional: reads vs writes, consistency, **retention** — how long data must be kept). **Write down the 5–6 questions the system must answer** ("which rooms are free between two dates?") — they drive the tables *and* the indexes. Ask; don't assume.
2. **Entities and relationships.** Nouns you store several facts about, with several instances, become entities; verbs become relationships. Decide **cardinality** (1:1, 1:N, M:N) **and participation** (total = every row must take part; partial = optional) for each, and say it aloud as a sentence.
3. **ER diagram.** Draw it (here `erDiagram` **crow's-foot notation**: a three-pronged "foot" means "many", a bar "one") — where a reviewer spots a missing entity fastest.
4. **Key tables (DDL).** Primary keys, foreign keys, `NOT NULL`, `UNIQUE`, `CHECK`; an M:N relationship becomes a **junction table** (PK = the pair of FKs). Aim for **3NF**, then **denormalize** on purpose.
5. **Key queries.** The 3–5 queries the product runs most. They validate the schema: if a core question needs six joins and a `DISTINCT`, the model is probably wrong.
6. **Indexes.** One per hot access path, usually a **composite index** matching the query's `WHERE` + `ORDER BY` (Chapter 12).
7. **Design review.** Check 1NF → BCNF (Chapter 11), name every deliberate redundancy, and list what the design *cannot* do yet (no history, an unenforced rule). Volunteering limits reads as judgement.
8. **Scaling notes.** Which table grows fastest, what to **partition** (inside one database) or **shard** (across servers) it by, what to cache, what to move to another store, where strict consistency is required.



Rules every case reuses

Mapping rules (§10.15) and judgement calls used by all thirteen designs, so the cases only point at them:

Situation	Rule
Total participation (every order has a customer)	The foreign key is `NOT NULL`; partial participation stays nullable
1:1 relationship	Foreign key **plus** `UNIQUE` (or a shared primary key) — without it the relationship is silently 1:N
M:N relationship	Junction table; an attribute of the *pairing* (rate agreed, price of a seat at a show, dosage per medicine) lives there
Composite attribute (name = first + last)	Split into columns
Multivalued attribute (several phone numbers)	Its own table keyed by (owner, value)
Money	`numeric` / `DECIMAL`, never `float` (0.10 has no exact binary form) or `int` (no cents)
Times that order events	`timestamptz`, not `date` — two postings on one day still need an order
Codes with letters or leading zeros (policy, passport, room "204B")	`text` / `varchar`, never `int`
Table named `user`, `order`, `group`, `like`, `transaction`, `show`	Reserved (or keyword) in some dialects — rename (`app_user`, `customer_order`, `screening`) rather than quote forever
Passwords	Store a slow salted hash (`password_hash`: a one-way scramble of the password plus a random per-user value), never the password
One entity doing two jobs	Split **type** from **instance**: `titles` / `copies`, `room_types` / `rooms`, `seats` / `show_seats`
Only current state stored	Keep history rows (loans, bookings, status events, ledger) — "when did…?" needs them
A value that must not change later	Snapshot it (price paid, address shipped to, fine charged)
Designing for Instagram scale on day one	State the single-node design first, then say what changes at 100×

In the interview

> **30-second answer:** "I'll clarify the main use cases and scale, pick the entities and relationships, sketch the ER diagram, turn it into tables with keys and constraints, check it against the top queries, add indexes for those queries, and finish with what I'd partition, cache or move to another store as it grows."

- *How do you know the design is right?* — Every core query is simple and index-friendly, every never-break business rule is a **constraint** or runs inside one transaction, and no fact is stored twice without a reason.
- *When do you denormalize?* — For a measured hot read (a feed, a counter), with a plan to keep the copy correct (same transaction, a trigger, or a periodic **reconciliation** job that recomputes the value and flags differences).

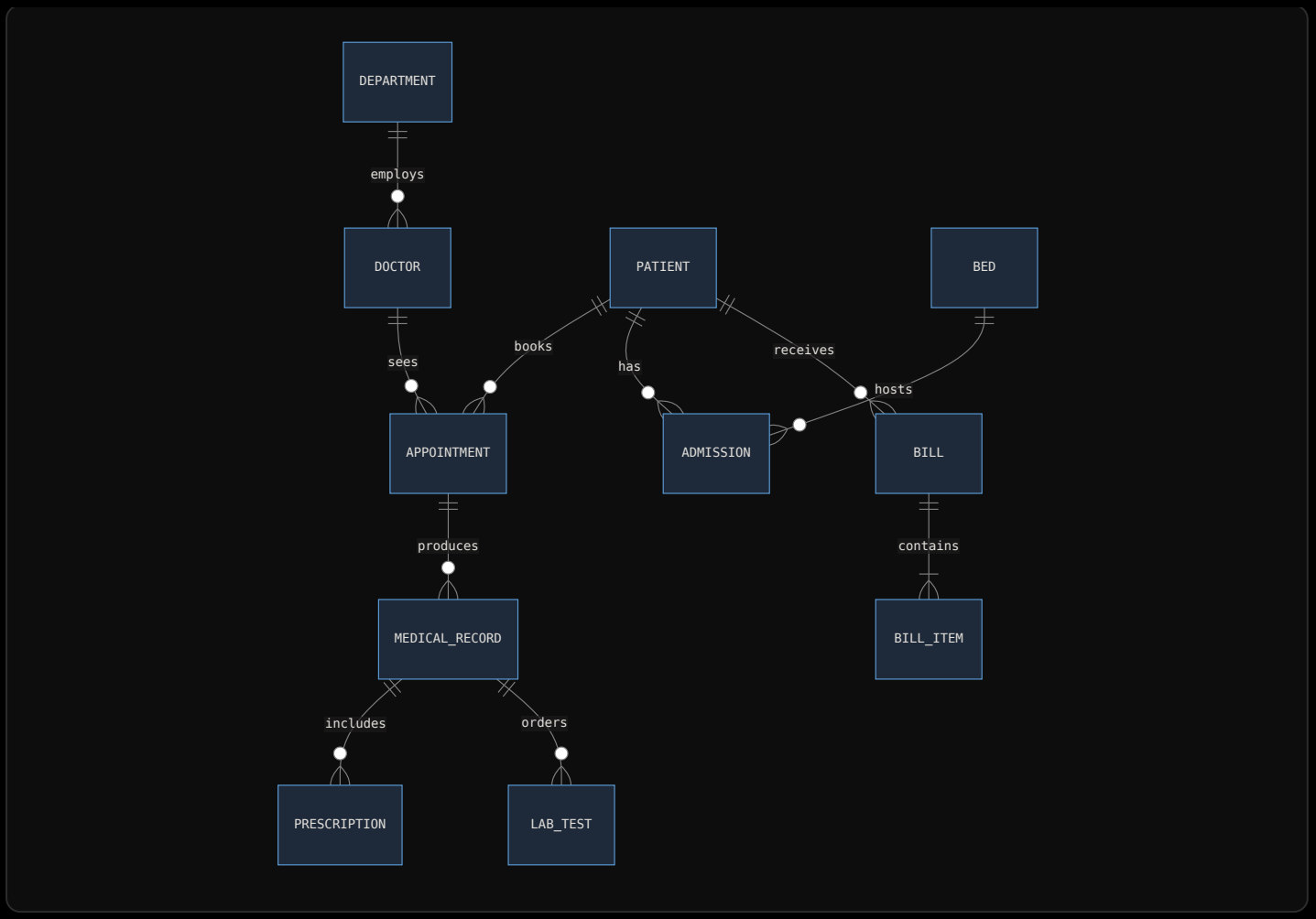
17.1 Hospital management system

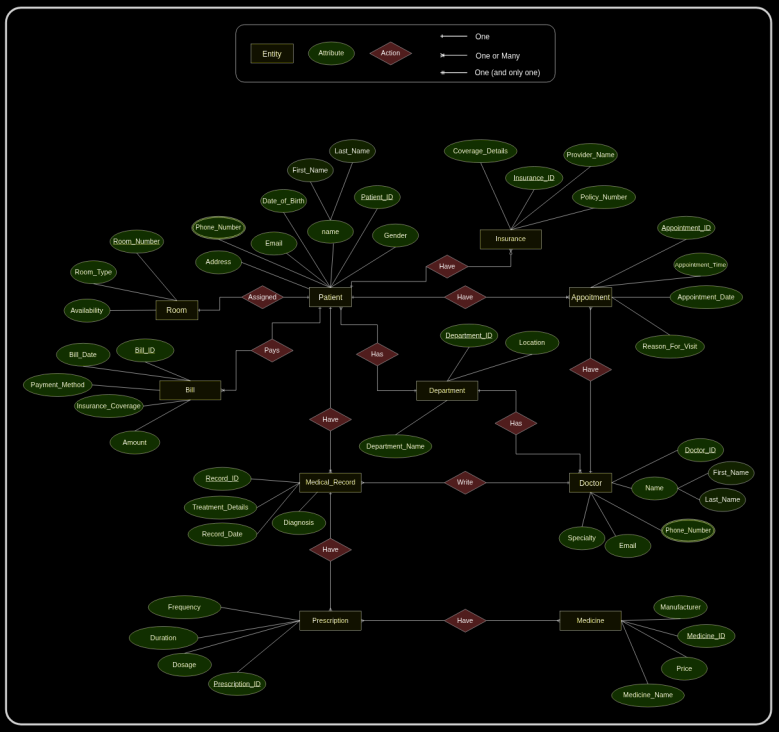
Requirements

- Register **patients** (name, birth date, contact, one or more phone numbers, insurance policies) and **doctors** (specialty) grouped into **departments**.
- Schedule **appointments** (one patient, one doctor, a time slot); a doctor is never in two overlapping slots.
- Record **admissions** to a **room/bed** with admit and discharge dates.
- Keep **medical records** (diagnosis, treatment, by a doctor), **prescriptions** listing **medicines** from a catalogue, and **lab tests** with results.
- Produce **bills** per visit/admission with line items and payments.
- Non-functional: legal retention for years, **auditable** (every change traceable to who and when), strictly **role-based** access (doctor vs clerk).
- Questions: a doctor's appointments tomorrow, a patient's full history, free rooms now, what a patient still owes, the medicines on a prescription.

Entities and relationships

Relationship	Cardinality	Note
Department — Doctor	1:N	a doctor belongs to one department
Patient — Appointment — Doctor	M:N via `appointments`	the appointment is the junction row with its own attributes (time, status)
Patient — Admission — Bed	1:N and N:1	a bed has many admissions over time, one at a time
Appointment / Admission — Medical record	1:N	a record belongs to one encounter
Doctor — Medical record	1:N	the author; total participation
Record — Prescription, Record — Lab test	1:N	
Prescription — Medicine	M:N	junction `prescription_items`; dose and frequency per medicine live there
Patient — Insurance policy	1:N	partial: a patient may have none; `(provider, policy_no)` is `UNIQUE`
Patient — Phone	1:N	multivalued attribute → `patient_phones (patient_id, phone)`
Patient — Bill — Bill items	1:N, 1:N	


DEPARTMENT
employs
DOCTOR
PATIENT
BED
sees
books
has
receives
hosts
APPOINTMENT
ADMISSION
BILL
produces
contains
MEDICAL_RECORD
BILL_ITEM
includes
orders
PRESCRIPTION
LAB_TEST


Entity
Attribute
Action
One
One or Many
One (and only one)
Last_Name
First_Name
Date_of_Birth
Patient_ID
Phone_Number
name
Gender
Email
Address
Room_Number
Room_Type
Availability
Room
Assigned
Patient
Coverage_Details
Provider_Name
Insurance_ID
Policy_Number
Insurance
Have
Appointment_ID
Appointment_Time
Appointment_Date
Appointment
Reason_For_Visit
Department_ID
Location
Department
Department_Name
Has
Pays
Bill_Date
Bill_ID
Payment_Method
Insurance_Coverage
Amount
Bill
Record_ID
Treatment_Details
Record_Date
Diagnosis
Medical_Record
Write
Doctor_ID
First_Name
Last_Name
Name
Specialty
Email
Doctor
Frequency
Duration
Dosage
Prescription_ID
Prescription
Manufacturer
Medicine_ID
Price
Medicine_Name
Medicine

Key tables

```
CREATE TABLE appointments (
  appointment_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  patient_id bigint NOT NULL REFERENCES patients,
  doctor_id  bigint NOT NULL REFERENCES doctors,
  slot       tstzrange NOT NULL,                    -- [start, end)
  status     text NOT NULL CHECK (status IN ('booked','checked_in','completed','cancelled','no_show')),
  -- no two active appointments of the same doctor may overlap (needs: CREATE EXTENSION btree_gist)
  EXCLUDE USING gist (doctor_id WITH =, slot WITH &&) WHERE (status <> 'cancelled')
);
CREATE TABLE admissions (
  admission_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  patient_id bigint NOT NULL REFERENCES patients,
  bed_id     int    NOT NULL REFERENCES beds,
  admitted_at   timestamptz NOT NULL,
  discharged_at timestamptz CHECK (discharged_at > admitted_at)
);
CREATE UNIQUE INDEX one_patient_per_bed ON admissions (bed_id) WHERE discharged_at IS NULL;
```

`tstzrange` holds a start and end timestamp in one value; `&&` means "overlaps". The **exclusion constraint** says "no two rows may have overlapping ranges for the same doctor" — a rule `UNIQUE` cannot express. MySQL and SQL Server have no equivalent: there you lock the doctor's row (`SELECT … FOR UPDATE`) and check for overlaps inside the booking transaction (Chapter 13).

Key queries and indexes

1. A doctor's schedule for a day: `WHERE doctor_id = $1 AND slot && tstzrange($day, $day + interval '1 day')`.
2. Free beds per ward: beds with no open admission (`NOT EXISTS … discharged_at IS NULL`).
3. A patient's full history: records, prescriptions and lab results across all encounters, newest first.
4. Unpaid bills older than 30 days.

Indexes: the exclusion constraint's **GiST** index serves schedule lookups; `admissions (patient_id)`, `medical_records (patient_id, created_at DESC)`, and the partial unique index for bed occupancy.

Design review

- **Normal forms.** Phone and junction tables have no non-key columns (2NF holds); `department_name` stays in `departments` (copying it onto patients would be a transitive dependency, breaking 3NF); the extra candidate keys (`email`, `(provider, policy_no)`) keep BCNF.
- **Room occupancy.** `rooms.patient_id UNIQUE` (a 1:1 "occupies") knows only who is in the room *now*; `admissions` with a partial unique index keeps the same rule *and* the history.
- **Dose per medicine.** On `prescriptions`, every medicine on one prescription would share it; it belongs to the pairing.
- **Still missing:** who cancelled an appointment and when (status history); a patient's department over time.

Scaling notes

Volume is modest — one PostgreSQL primary with a replica for reports. The hard requirements are **integrity and audit**: append-only record tables (corrections are new rows referencing the old one), `created_by` / `created_at` on every clinical row, **row-level security** for role-based access (§14.8), encrypted backups with point-in-time recovery (§15.11). Large files (scans, X-rays) go to object storage; the table keeps only the key.

17.2 Hotel reservation system

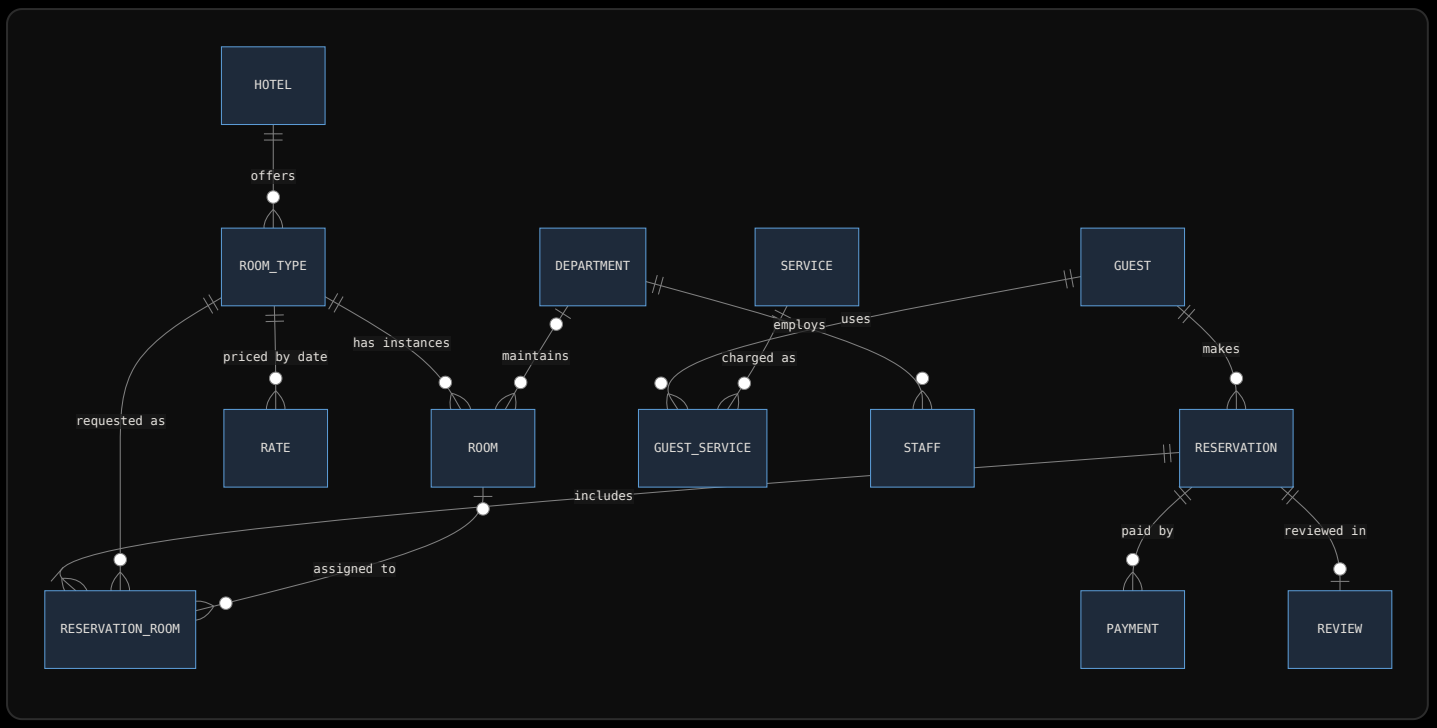
Requirements

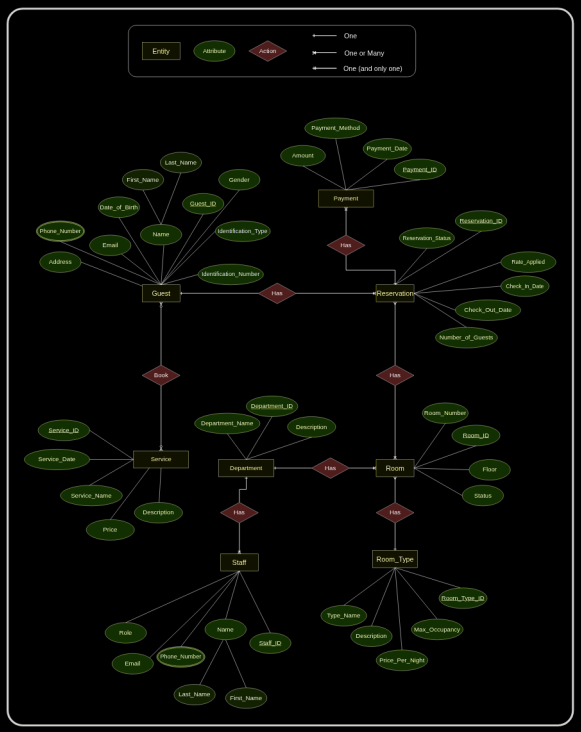
- **Hotels** have **room types** (single, double, suite) with a base price, and physical **rooms** of each type.

- **Guests** search availability by city and dates, then **reserve** one or more rooms; no room may be double-booked.
- Life cycle: pending → confirmed → checked-in → checked-out, or cancelled. **Payments** and **reviews** attach to reservations. Prices vary by date (seasonal rates).
- Guests are identified by a document (passport, national ID), may have several phone numbers, and use **extra services** (dining, spa, laundry) charged to the stay.
- **Staff** belong to **departments** (housekeeping, front desk); a department looks after rooms.
- Questions: free rooms between two dates, what a guest paid and owes, today's check-ins, services a guest used, revenue per room type last month, who works in housekeeping.

Entities and relationships

The central decision is **type versus instance**: guests book a *room type* for a date range; a specific *room* is assigned at or before check-in. Only rooms makes search slow; only types makes check-in impossible.





Two details from the full model: price and maximum occupancy belong to the **room type** (ten identical doubles share one price); service usage is a junction row `guest_services (guest_services_id, guest_id, service_id, reservation_id, used_on, quantity, price_charged)` with **its own surrogate key**, because a guest orders dinner on several nights — a `(guest_id, service_id)` key would allow one use ever. `guests` has `UNIQUE (id_type, id_number)` so one identity document never appears on two guest rows.

Key tables

```
-- reservations (guest_id NOT NULL, hotel_id, check_in, check_out CHECK (check_out > check_in), status CHECK
(...))
CREATE TABLE reservation_rooms (
  reservation_id bigint REFERENCES reservations ON DELETE CASCADE,
  line_no        smallint,
  room_type_id   int NOT NULL REFERENCES room_types,
  room_id        int REFERENCES rooms,            -- NULL until assigned
  stay           daterange NOT NULL,              -- copied from the reservation for the constraint
  nightly_price  numeric(10,2) NOT NULL,          -- snapshot
  PRIMARY KEY (reservation_id, line_no),
  EXCLUDE USING gist (room_id WITH =, stay WITH &&)   -- a room is never in two overlapping stays
);
-- Inventory per type per night: the row that search reads and booking decrements.
CREATE TABLE room_inventory (
  room_type_id int REFERENCES room_types,
  night        date,
  total        smallint NOT NULL,
  reserved     smallint NOT NULL DEFAULT 0 CHECK (reserved <= total),
  PRIMARY KEY (room_type_id, night)
);
```

The exclusion constraint needs `CREATE EXTENSION btree_gist` (for `=` on an integer inside GiST); rows with `room_id` still `NULL` never conflict, and cancelling must clear or delete the lines to release the room. `room_inventory` is a deliberate **denormalization**: counting overlapping reservations per search is too slow, so each night keeps a counter. Booking increments `reserved` for every night of the stay in one transaction; the `CHECK` makes overbooking fail (hotels allowing a little overbooking use `reserved <= total * 1.1`).

Key queries and indexes

```
-- Room types in a hotel with at least one room free on every night of 10–13 June
SELECT rt.room_type_id, rt.name
FROM room_types rt JOIN room_inventory ri USING (room_type_id)
WHERE rt.hotel_id = $1 AND ri.night >= '2026-06-10' AND ri.night < '2026-06-13'
GROUP BY rt.room_type_id
HAVING count(*) = 3 AND min(ri.total - ri.reserved) > 0;
```

Other hot queries: today's arrivals and departures per hotel, a guest's upcoming stays, average rating per hotel. Indexes: `reservations (hotel_id, check_in)`, `reservations (guest_id, check_in DESC)`, the inventory PK, the GiST index behind the exclusion constraint.

Design review

- **Normal forms.** Price on `room_types`, not `rooms` (3NF); `nightly_price` on the booking line is a *snapshot*, a different fact from today's list price.
- **Double booking.** `UNIQUE` only detects identical values, never overlapping ranges — hence the exclusion constraint (or a check inside the booking transaction elsewhere).
- **A room status column** (free/occupied) repeats what reservations say and can disagree; keep it only for what bookings can't express ("under maintenance").
- **Still missing:** mid-stay room changes (the line's own `stay` range supports them), who cancelled and when.

Scaling notes

Search traffic is 100–1,000× booking traffic. Serve search from replicas or a search index (Elasticsearch/OpenSearch) plus an inventory cache, but make the **booking** transaction hit the primary

and re-check the inventory row under lock — search may be seconds stale, bookings may not. Partition `reservations` by `check_in` month; shard by `hotel_id` if needed, since nearly every transaction touches one hotel.

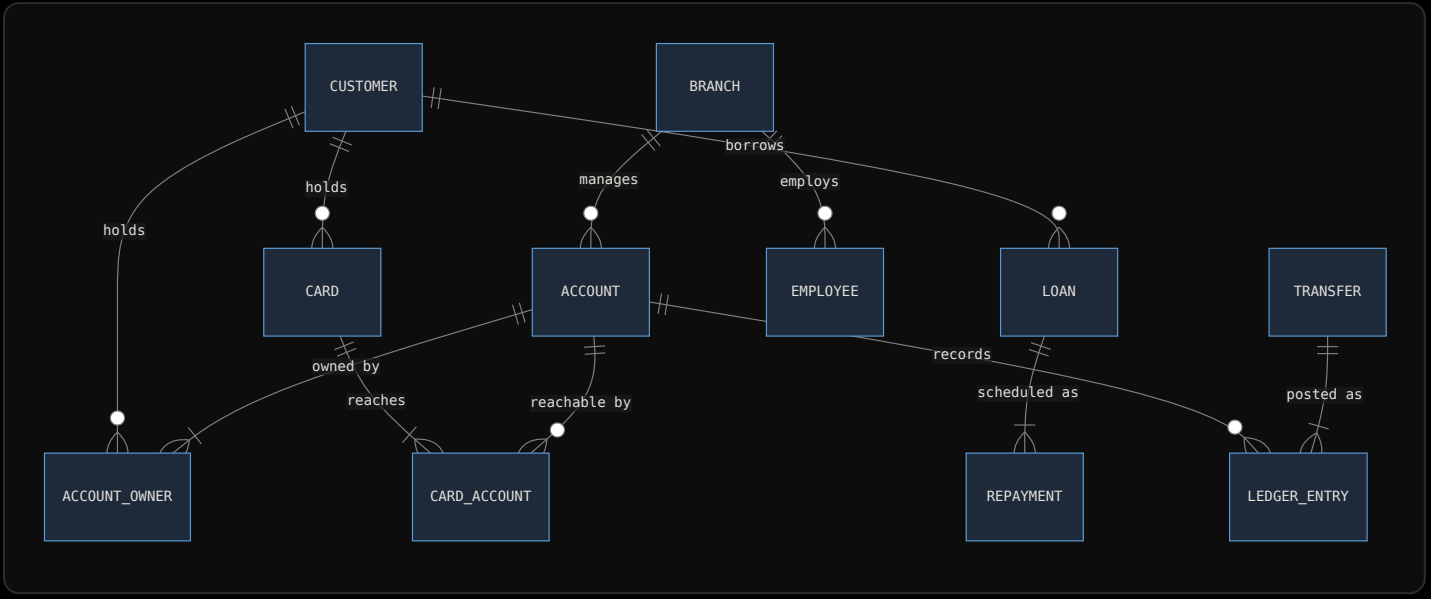
17.3 Banking system

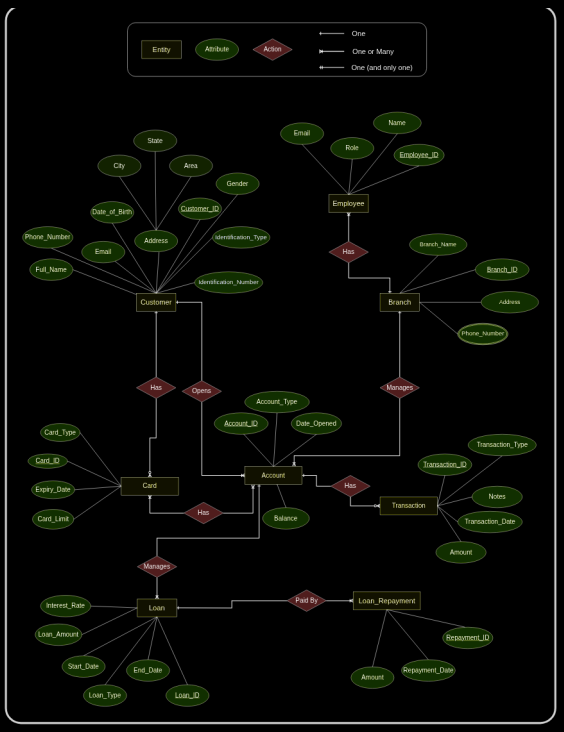
Requirements

- **Customers** own **accounts** (checking, savings, loan); accounts may have joint owners.
- Deposits, withdrawals and **transfers**; balances never below an allowed limit.
- A full, immutable history of every money movement; monthly statements.
- **Branches** (several phone numbers each) with **employees**; **cards**, one card reaching several accounts; **loans** with repayments.
- Questions: balance now, last month's activity, a customer's accounts, what is owed on a loan, which accounts a card reaches, who works at a branch.
- Non-functional: strict consistency (ACID), auditability, regulatory retention.

Entities and relationships

The key idea is **double-entry bookkeeping**: money is never "updated", it is *moved*. A **transfer** (the business event) has two or more **entries** (ledger lines), each a signed amount on one account, summing to zero.





Key tables

```
CREATE TABLE accounts (
  account_id  bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  iban        text NOT NULL UNIQUE,
  kind        text NOT NULL CHECK (kind IN ('checking','savings','loan')),
  currency    char(3) NOT NULL,
  balance     numeric(18,2) NOT NULL DEFAULT 0,   -- cached; the ledger is the truth
  overdraft_limit numeric(18,2) NOT NULL DEFAULT 0,
  CHECK (balance >= -overdraft_limit),
  status      text NOT NULL DEFAULT 'open' CHECK (status IN ('open','frozen','closed'))
);
CREATE TABLE transfers (
  transfer_id     bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  idempotency_key uuid NOT NULL UNIQUE,           -- a retried request cannot post twice
  kind            text NOT NULL,
  created_at      timestamptz NOT NULL DEFAULT now()
);
CREATE TABLE ledger_entries (
  entry_id    bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  transfer_id bigint NOT NULL REFERENCES transfers,
  account_id  bigint NOT NULL REFERENCES accounts,
  amount      numeric(18,2) NOT NULL CHECK (amount <> 0),   -- + credit, - debit
  balance_after numeric(18,2) NOT NULL
);
```

`idempotency_key` makes the transfer **idempotent** — repeating the same request (e.g. after a network timeout) has the same effect as sending it once (the client generates a random 128-bit `uuid`). The ledger is **append-only**: revoke `UPDATE` and `DELETE`; fix mistakes with a reversing transfer.

Key queries

A transfer is the textbook transaction (§13.1):

```
BEGIN;
INSERT INTO transfers (idempotency_key, kind) VALUES ($key, 'transfer') RETURNING transfer_id;
-- lock both rows in a fixed order (lower id first) so two opposite transfers cannot deadlock
SELECT account_id FROM accounts WHERE account_id IN ($from, $to) ORDER BY account_id FOR UPDATE;
UPDATE accounts SET balance = balance - $amt WHERE account_id = $from;   -- CHECK stops overdraft
UPDATE accounts SET balance = balance + $amt WHERE account_id = $to;
INSERT INTO ledger_entries (transfer_id, account_id, amount, balance_after) VALUES (...), (...);
COMMIT;
```

Others: a monthly statement (`WHERE account_id = $1 AND created_at` in the month, ordered), a reconciliation check that every transfer's entries sum to zero (`GROUP BY transfer_id HAVING sum(amount) <> 0` must return nothing), "balance = sum of entries" per account.

Indexes

`ledger_entries (account_id, entry_id DESC)` for statements, `account_owners (customer_id)`, the unique `idempotency_key`. Junction `card_accounts` has PK `(card_id, account_id)`, answering "which accounts can this card reach"; the reverse needs an index on `account_id`, because an index is usable only from its leading column.

Design review

- **Transfers need a pairing key.** One `transactions` row per account (type = deposit/withdrawal/transfer) cannot name both sides of a transfer; the shared `transfer_id` fixes that. Name the table `ledger_entries` or `account_transactions` — `TRANSACTION` is reserved in SQL Server and a keyword elsewhere.
- **Cached balance.** `accounts.balance` stores the sum of entries. Normal forms don't forbid it (it isn't a dependency *inside* one row), but it can drift, so a nightly job recomputes and compares.
- **Rules a foreign key cannot express:** a card only reaches accounts its owner holds; each transfer's entries sum to zero — check in the transaction or with a periodic query.
- **Still missing:** which employee opened an account or approved a loan (audit columns), card number and status (blocked ≠ expired), and `numeric` width chosen for the largest expected value (widening later rewrites the table). Customer `city` with `state` is a small 3NF compromise most teams accept knowingly.

Scaling notes

Correctness beats throughput: **synchronous replication** to a standby so failover loses no committed money (§15.5), `SERIALIZABLE` or explicit row locks for balance checks, monthly range partitions of the ledger for archiving. Sharded by account (or customer), a cross-shard transfer needs a **saga** (local steps, each with a compensating "undo") or two-phase commit (§15.9) — why many banks keep the core ledger on one very large vertically scaled database or a distributed SQL system (Spanner, CockroachDB).

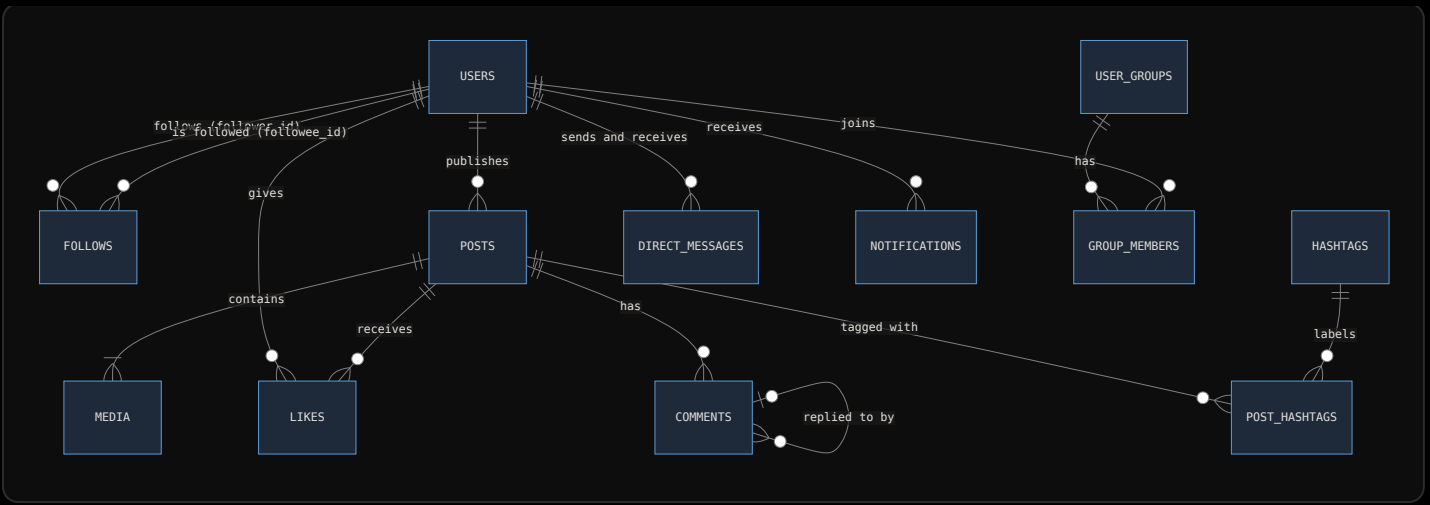
17.4 Instagram-style photo sharing (runnable)

Requirements

- **Users** have a profile (public or private) and **follow** others (one-directional; "friends" = mutual follows).
- **Posts** have a caption and one or more ordered **media** items (a carousel), tagged with **hashtags**.
- Users **like** posts (at most once) and **comment**, including replies.
- **Direct messages** to one other user, **notifications** (likes, comments, follows), **groups**.
- The **home feed** shows recent posts from followed accounts, newest first, paged; profiles show like and follower counts instantly.
- Non-functional: extremely read-heavy, hundreds of millions of posts, media bytes far larger than the rows.

Entities and relationships

`follows` is a **self-referencing M:N** relationship on `users`; `likes` and `post_hashtags` are junction tables; `comments.parent_id` forms reply threads.

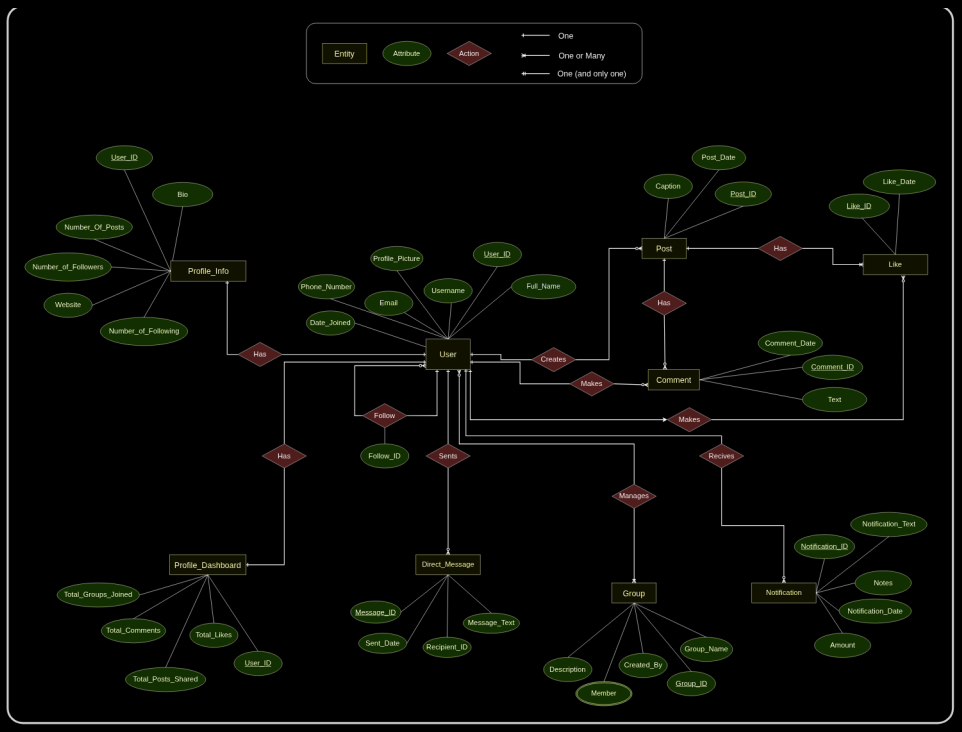


Key tables

From `00_schema.sql` (schema `insta`):

```
CREATE TABLE follows (
  follower_id bigint REFERENCES users,
  followee_id bigint REFERENCES users,
  created_at  timestamptz NOT NULL,
  PRIMARY KEY (follower_id, followee_id),       -- "whom do I follow?" + no duplicate follows
  CHECK (follower_id <> followee_id)
);
CREATE INDEX ON follows (followee_id);          -- "who follows me?"
CREATE TABLE posts (
  post_id bigint PRIMARY KEY, user_id bigint NOT NULL REFERENCES users,
  caption text, created_at timestamptz NOT NULL,
  like_count int NOT NULL DEFAULT 0, comment_count int NOT NULL DEFAULT 0
);
CREATE INDEX ON posts (user_id, created_at DESC, post_id DESC);
-- media (post_id, position, url = object-store key, kind): carousel order kept by position
CREATE TABLE likes (
  user_id bigint REFERENCES users, post_id bigint REFERENCES posts ON DELETE CASCADE,
  created_at timestamptz NOT NULL,
  PRIMARY KEY (post_id, user_id)                  -- one like per user per post
);
-- comments (comment_id, post_id, user_id, parent_id REFERENCES comments, body, created_at)
```

The file also creates `direct_messages` (FKs to sender and recipient), `notifications` (with `actor_id` and `post_id`, so tapping one opens the post), and `user_groups` + `group_members` (a group's multivalued "members" becomes a junction table that can record `joined_at`). `user`, `group` and `like` are reserved in PostgreSQL or MySQL, hence the plural or prefixed names.



The course keeps profile numbers (posts, followers, following) in a **1:1 table** whose PK is also the FK to the user — a *shared primary key*, the simplest way to guarantee one row per user. Here those counters sit on `users`; either way they are a speed decision the normal forms cannot see, because the redundancy spans two tables.

Key queries

The home feed joins "whom I follow" to their posts. **Fan-out on read** computes it at request time:

```
SELECT p.post_id, u.username, p.caption, p.created_at, p.like_count
FROM follows f
JOIN posts p ON p.user_id = f.followee_id
JOIN users u ON u.user_id = p.user_id
WHERE f.follower_id = 1
ORDER BY p.created_at DESC, p.post_id DESC
LIMIT 2;
-- next page: add  AND (p.created_at, p.post_id) < ('2026-05-02 07:30+00', 1003)
```

Output:

```
 post_id | username |           caption           |       created_at       | like_count
---------+----------+-----------------------------+------------------------+------------
    1005 | cy       | Sourdough attempt 2 #baking | 2026-05-03 10:15:00+00 |          1
    1003 | bo       | Trail views #running #hills | 2026-05-02 07:30:00+00 |          2

 post_id | username |       caption        |       created_at       (page 2)
---------+----------+----------------------+------------------------
    1002 | cy       | Fresh bread #baking  | 2026-05-01 08:00:00+00
    1001 | bo       | Sunrise run #running | 2026-05-01 06:10:00+00
```

Page 2 uses **keyset pagination** (§12.14): fast on page 500, never repeating or skipping a post when new ones arrive; `post_id` breaks timestamp ties. A like must insert the row **and** bump the counter together, and a double tap must not count twice:

```
BEGIN;
WITH ins AS (
  INSERT INTO likes (user_id, post_id, created_at) VALUES (4, 1005, '2026-05-03 13:00+00')
  ON CONFLICT DO NOTHING RETURNING post_id
)
UPDATE posts SET like_count = like_count + 1 WHERE post_id IN (SELECT post_id FROM ins)
RETURNING post_id, like_count;
-- ...the same statement again a minute later...
COMMIT;
```

Output:

```
 post_id | like_count
---------+------------
    1005 |          2
(1 row)

 post_id | like_count          <- the repeat like changed nothing
---------+------------
(0 rows)
```

`ON CONFLICT DO NOTHING` skips an insert that would violate a unique key, so the CTE returns a row only if the insert happened. `04_instagram.sql` also finds mutual follows (a self-join of `follows` with columns swapped), prints a comment thread indented by depth with a recursive CTE over `parent_id` (Chapter 7), ranks hashtags by posts in the last three days, and ends with a **reconciliation check** recounting likes against the cached `like_count` (0 rows). Every denormalized counter deserves one.

Indexes

Access path	Index
Whom does X follow?	`follows` PK `(follower_id, followee_id)`
Who follows X?	`follows (followee_id)`
X's posts, newest first (profile grid, feed merge)	`posts (user_id, created_at DESC, post_id DESC)`
Has X liked this post? / who liked it	`likes` PK `(post_id, user_id)`
Posts with a tag	`post_hashtags` PK `(tag_id, post_id)`

Design review

The classic defect is modeling follow and like as 1:N: both are M:N and need a uniqueness rule on the pair (the composite PKs), plus `CHECK (follower_id <> followee_id)`. PK order decides which direction each index serves, so `follows (followee_id)` and `group_members (user_id)` exist for the reverse questions. Follows carry `created_at` ("since when?"), comments `parent_id`, and media is a table because a post can hold several.

Scaling notes

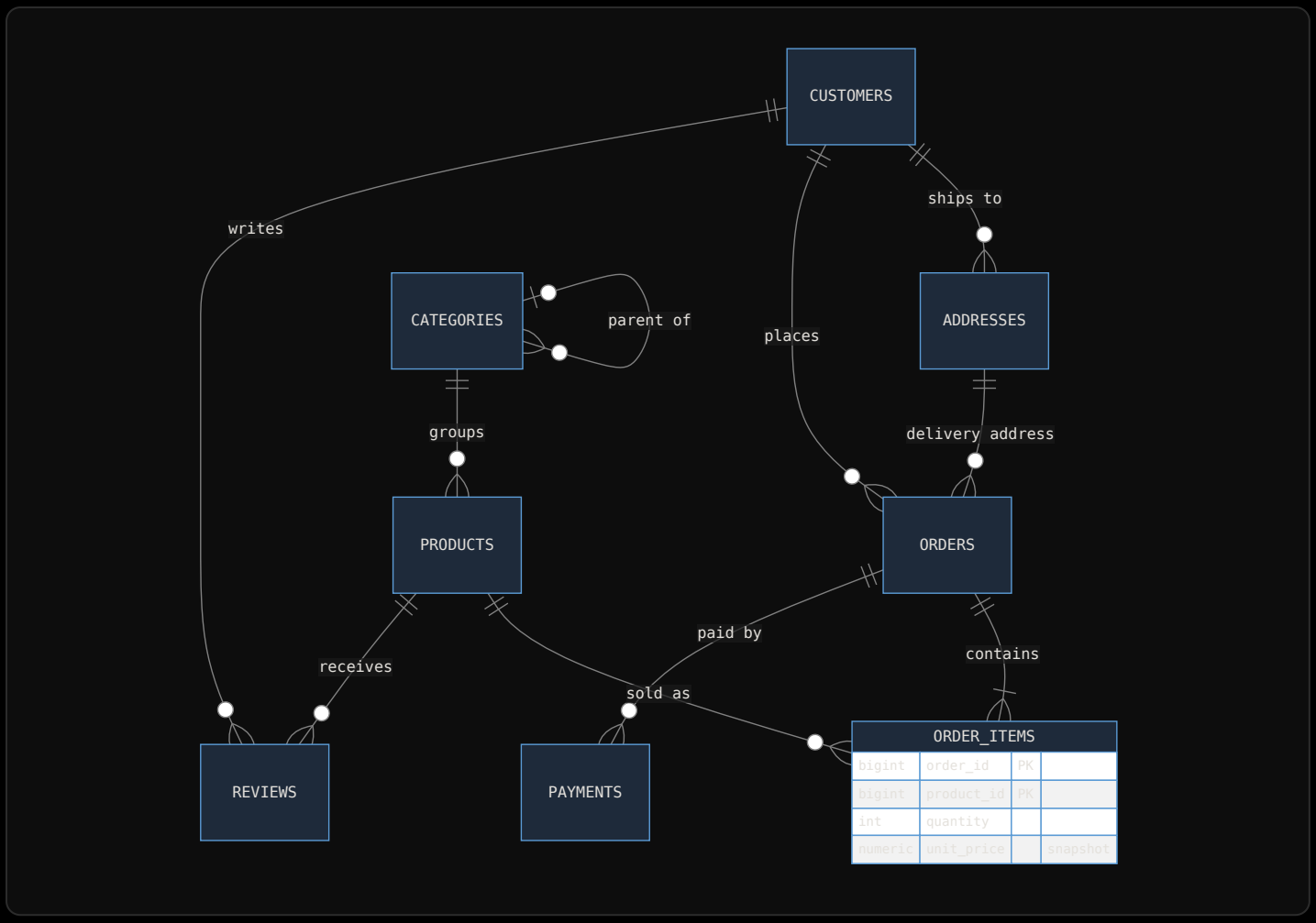
- **Media** in object storage behind a **CDN** (servers worldwide caching files near users); rows keep keys only.
- **Feed:** fan-out on read is fine for small follow counts. At scale **fan-out on write** pushes each new post id into every follower's precomputed feed (Redis or a wide-column store); celebrities with millions of followers stay fan-out on read, merged at request time — the classic hybrid (Chapter 16).
- **Counters** are the hottest rows: a viral post turns `UPDATE posts SET like_count = …` into a lock queue (**lock contention**). Batch increments in a cache and flush periodically, or keep sharded counter rows and sum them.
- **Sharding** by `user_id` keeps a user's posts, likes-given and follows together; ids come from a time-embedding generator (Snowflake-style: timestamp + machine id + counter) so they sort by creation without a central sequence (§15.3).

17.5 E-commerce store (runnable)

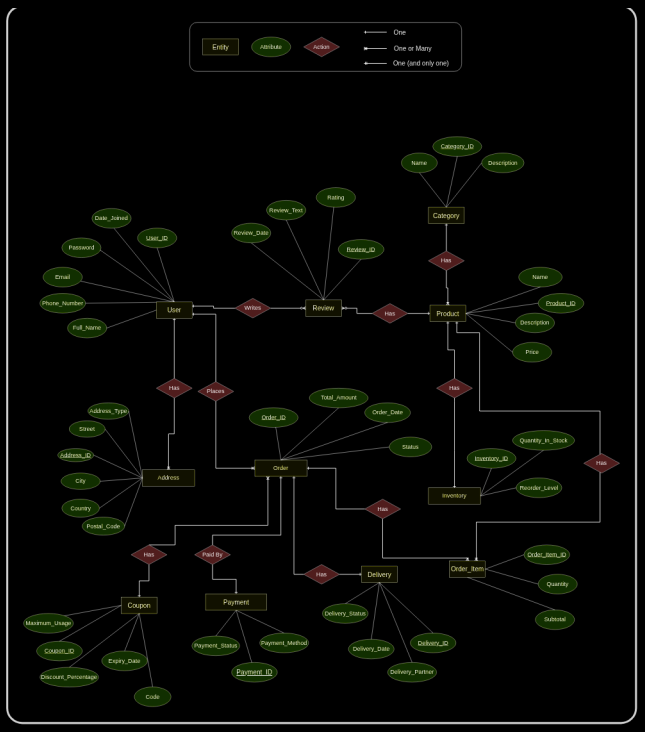
Requirements

- **Customers** register, keep several **addresses**, browse **products** (each with a **SKU**, stock-keeping unit) in a **category tree**, and write **reviews** (one per customer per product).
- An **order** of several products; each **order item** records quantity and price *at purchase time*.
- Stock decrements on purchase and never goes negative. Orders: `pending → paid → shipped → delivered` (or `cancelled`); **payments** attach to orders.
- Discontinued products leave the shop but old orders still show them.
- Each order has one **delivery** record (status, date, partner) and may use one **coupon** (code, discount, expiry, usage limit).
- Non-functional: browsing read-heavy; checkout strictly consistent.
- Questions: a customer's orders newest first, an order's lines and prices, products in a category with stock, a product's average rating, undelivered orders, whether a coupon is valid.

Entities and relationships



`order_items` is the M:N junction with its own attributes; `categories.parent_id` forms a tree (Electronics → Audio → Headphones).



Not in the runnable schema but part of the full design: `deliveries (order_id UNIQUE REFERENCES orders, status, delivered_on, partner)` and `payments` with `order_id UNIQUE` if exactly one payment per order is the rule — the `UNIQUE` makes each 1:1; `coupons (code UNIQUE, discount_pct, expires_on, max_uses)` with a **nullable** `orders.coupon_id` (partial participation); stock on `products` (as here) or in a 1:1 `inventory` table keyed by `product_id` with a reorder level; `password_hash` on customers.

Key tables

```
CREATE TYPE order_status AS ENUM ('pending', 'paid', 'shipped', 'delivered', 'cancelled');

CREATE TABLE products (
  product_id  bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  sku         text NOT NULL UNIQUE,
  name        text NOT NULL,
  category_id int NOT NULL REFERENCES categories,
  price       numeric(10,2) NOT NULL CHECK (price >= 0),
  stock       int NOT NULL CHECK (stock >= 0),     -- overselling becomes impossible
  deleted_at  timestamptz                          -- soft delete
);
CREATE TABLE orders (
  order_id    bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  customer_id bigint NOT NULL REFERENCES customers,
  ship_to     bigint NOT NULL REFERENCES addresses,
  status      order_status NOT NULL DEFAULT 'pending',
  placed_at   timestamptz NOT NULL,
  total       numeric(12,2) NOT NULL DEFAULT 0      -- denormalized sum of the items
);
CREATE TABLE order_items (
  order_id   bigint REFERENCES orders ON DELETE CASCADE,
  product_id bigint REFERENCES products,
  quantity   int NOT NULL CHECK (quantity > 0),
  unit_price numeric(10,2) NOT NULL,
  PRIMARY KEY (order_id, product_id)
);
CREATE INDEX ON orders (customer_id, placed_at DESC);
CREATE INDEX ON products (category_id) WHERE deleted_at IS NULL;   -- partial: live products only
```

Three decisions to say aloud:

- **Price snapshot.** `order_items.unit_price` copies the price at order time — not a normalization violation: the price *paid* differs from the price *now*.
- **Soft delete.** Deleting a product would break old orders' foreign keys (or cascade away the history); `deleted_at` hides it instead, and the **partial index** keeps live-catalogue lookups small.

- **Status enum.** A PostgreSQL `ENUM` (or `CHECK … IN (…)`, or a **lookup table** of allowed values referenced by an FK) limits `status` to known values. Enums are compact but adding a value needs `ALTER TYPE`; a lookup table is most flexible (MySQL has inline `ENUM`; SQL Server and SQLite use `CHECK`).

Key queries

Placing an order is one transaction: create the order, decrement stock **only where enough exists**, copy the prices, set the total (`01_ecommerce.sql`):

```
BEGIN;
INSERT INTO orders (customer_id, ship_to, placed_at)
VALUES (2, 2, '2026-05-05 12:00+00') RETURNING order_id \gset
WITH cart(product_id, qty) AS (VALUES (1::bigint, 2), (3::bigint, 1)),
reserved AS (
  UPDATE products p SET stock = p.stock - c.qty
  FROM cart c
  WHERE p.product_id = c.product_id AND p.stock >= c.qty AND p.deleted_at IS NULL
  RETURNING p.product_id, c.qty, p.price
)
INSERT INTO order_items (order_id, product_id, quantity, unit_price)
SELECT :order_id, product_id, qty, price FROM reserved
RETURNING order_id, product_id, quantity, unit_price;
-- the application compares the row count with the cart size and issues ROLLBACK if short
UPDATE orders SET total = (SELECT sum(quantity * unit_price) FROM order_items WHERE order_id = :order_id)
WHERE order_id = :order_id RETURNING order_id, total;
COMMIT;
```

Output:

```
 order_id | product_id | quantity | unit_price
----------+------------+----------+------------
        5 |          1 |        2 |      89.00
        5 |          3 |        1 |      29.99
(2 rows)

 order_id | total
----------+--------
        5 | 207.99
```

(`\gset` stores the returned `order_id` in a psql variable; application code reads it from `RETURNING`.) Because one `UPDATE` both checks and decrements stock, two concurrent checkouts for the last unit cannot both succeed: the second waits for the first's row lock, then re-checks `stock >= qty` on the new value (§13.7). `CHECK (stock >= 0)` is the last line of defence:

```
psql:01_ecommerce.sql:35: ERROR:  new row for relation "products" violates check constraint "products_stock_check"
DETAIL:  Failing row contains (2, SP-200, Bluetooth speaker, 2, 45.50, -1, null).
```

A best-seller report still counts the soft-deleted earbuds (2 units, 19.98) — history must not change with the catalogue. The file also walks the category tree with a recursive CTE (all live products under Electronics at any depth) and runs a **payment reconciliation** (orders marked paid whose payments don't add up), finding order 3 (total 45.50, paid 30.00) — the mismatch a nightly job should flag.

Indexes

`orders (customer_id, placed_at DESC)` for "my orders"; `order_items (product_id)` for sales per product (the PK covers lookups by `order_id`); the partial index for category pages. Text search belongs in full-text search (§14.6) or a search engine.

Design review

- **Normal forms.** `order_items` and `reviews` have two-column candidate keys with every other column depending on the whole pair (2NF). A stored line `subtotal = quantity × unit_price` would break 3NF (and BCNF) on purpose; `orders.total` is the same redundancy across two tables — kept for fast order lists, checked by reconciliation.

- **Freeze what the order promised.** Snapshot the shipping address too (copy it onto the order), or editing an address rewrites where old orders "went".
- **Carts** look like orders (`carts` + `cart_items`) but read the *live* price every time.
- **Still missing:** counting coupon uses against `max_uses` (concurrent checkouts must lock or count atomically) and an order status history.

Scaling notes

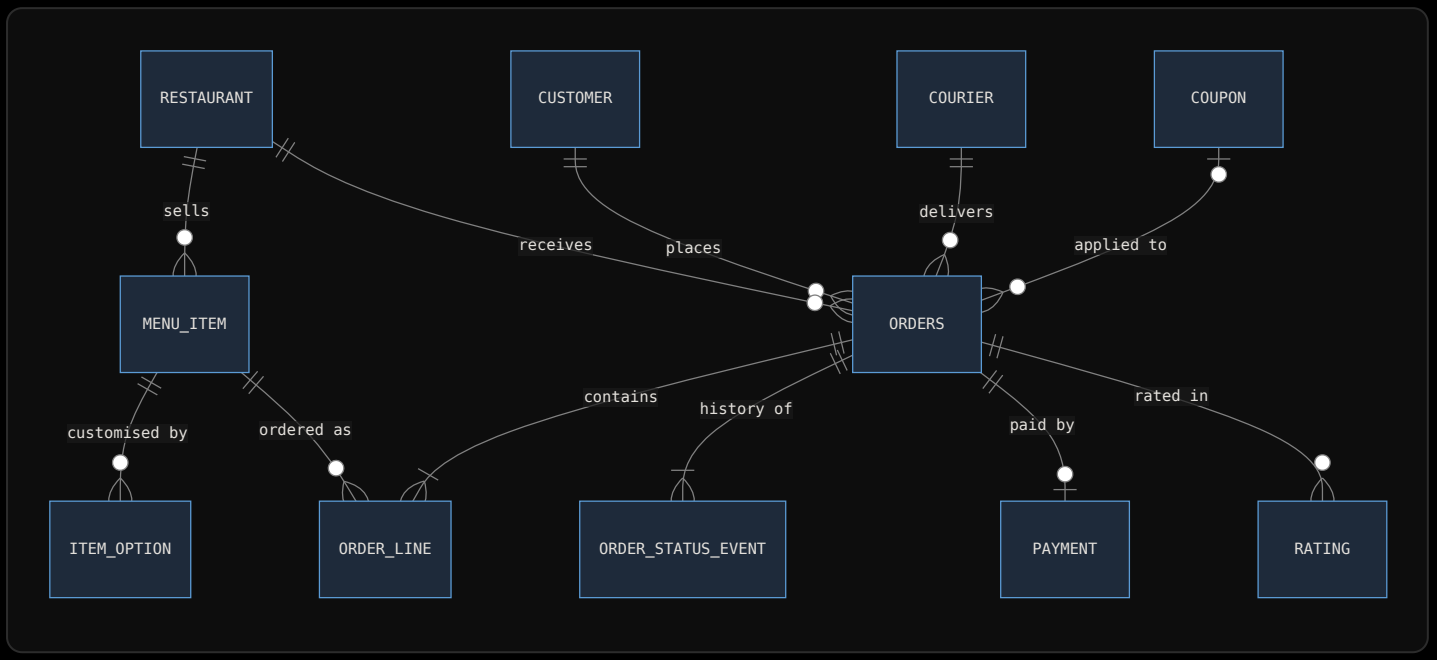
The catalogue is read-mostly: cache it, serve from replicas or a search index. Carts are short-lived and high-churn — a key-value store with expiry (Redis) fits. Orders and payments stay relational and ACID; partition `orders` by `placed_at` month for archiving, shard by `customer_id` when one primary saturates. In flash sales a single hot `stock` row becomes the bottleneck — pre-split inventory into several bucket rows, or reserve stock in a fast counter and settle in the database.

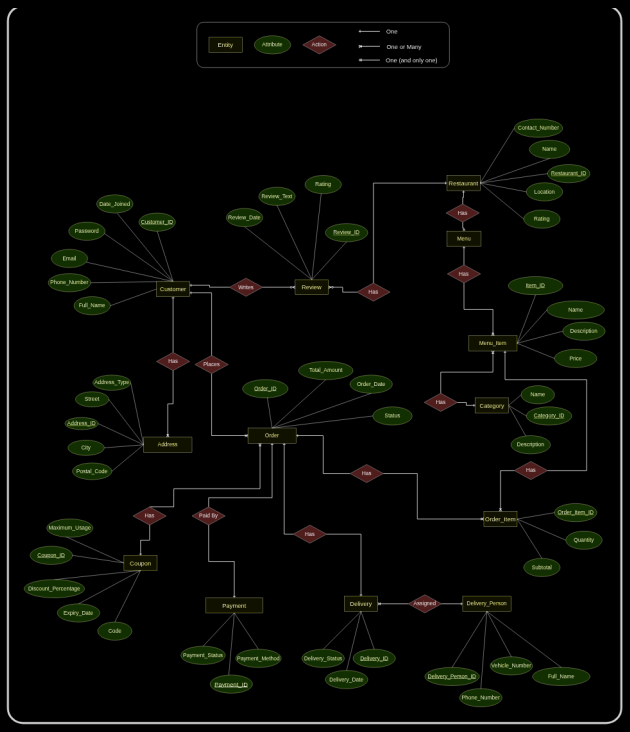
17.6 Food delivery platform

Requirements

- **Customers** order from **restaurants**; each has a **menu** of **items** (options such as size or toppings) and opening hours; items belong to **categories**.
- An **order** holds items from one restaurant and moves `placed → accepted → preparing → picked_up → delivered` (or `cancelled`), every change time-stamped.
- A **courier** is assigned per order; their location updates every few seconds while active.
- **Payments**, **ratings** for restaurant and courier, **promotions/coupons**; customers keep several delivery **addresses**.
- Non-functional: lunch and dinner peaks, live tracking, search by distance.
- Questions: today's menu, a customer's orders this month, an order's lines, where an order is and when it was picked up, a courier's deliveries today, a restaurant's reviews.

Entities and relationships





The course model (thirteen entities; restaurant–menu, order–payment and order–delivery 1:1) reaches the restaurant only through order line → menu item → menu, which cannot stop one order mixing two kitchens; `restaurant_id` on `orders` enforces "one restaurant per order" directly. Its separate 1:1 `menus` table stores nothing but a key — items can point at the restaurant instead. Its **delivery** is a 1:1 table (`deliveries.order_id UNIQUE`) with current status and courier plus a status history — the same pairing as `orders.status` + `order_status_events` here.

Key tables

```
CREATE TABLE orders (
  order_id      bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  customer_id   bigint NOT NULL REFERENCES customers,
  restaurant_id bigint NOT NULL REFERENCES restaurants,
  courier_id    bigint REFERENCES couriers,          -- NULL until assigned
  status        text NOT NULL,                        -- current state (denormalized)
  delivery_addr jsonb NOT NULL,                       -- snapshot of the address + coordinates
  subtotal numeric(10,2) NOT NULL, delivery_fee numeric(10,2) NOT NULL,
  placed_at     timestamptz NOT NULL DEFAULT now()
);
CREATE TABLE order_status_events (                    -- append-only history
  order_id  bigint REFERENCES orders,
  status    text NOT NULL,
  at        timestamptz NOT NULL DEFAULT now(),
  actor     text NOT NULL,                            -- 'customer', 'restaurant', 'courier', 'system'
  PRIMARY KEY (order_id, at)
);
-- order_lines (order_id, line_no, item_id, quantity, unit_price snapshot, options jsonb snapshot)
```

A **current-status column** plus an **append-only event table** is worth remembering: the column answers "what state is it in?" with no join; the events answer "when did each step happen?" (for **SLAs** — promised targets such as "delivered within 40 minutes" — disputes and delivery-time analytics).

Key queries and indexes

1. Open restaurants within 5 km serving "pizza" — a geospatial index (PostGIS `geography` + GiST, §16.8).
2. A restaurant's live queue: `WHERE restaurant_id = $1 AND status IN ('placed','accepted','preparing') ORDER BY placed_at`.
3. Nearest available courier (geo query over latest positions).
4. Average prep time per restaurant: time between the `accepted` and `picked_up` events.

Indexes: `orders (restaurant_id, status)` partial `WHERE status NOT IN ('delivered','cancelled')` keeps the live-queue index tiny; `orders (customer_id, placed_at DESC)`; GiST on restaurant location.

Design review

A stored line subtotal and a stored `restaurants.rating` (instead of `avg()` over reviews) are the deliberate redundancies — name both. Payments carry their own amount (partial payments, refunds, tips); a review should reference an order (or anyone can review a restaurant they never ordered from); a coupon's usage limit needs an atomic count (two simultaneous checks in application code both pass); the delivery address is snapshotted (`delivery_addr jsonb`) so editing an address never moves last month's order.

Scaling notes

Courier locations are high-frequency, short-lived writes — keep them in an in-memory store with geo support (Redis `GEOADD` / `GEOSEARCH`) and write only trip summaries to PostgreSQL. Shard orders by city/region (almost no order crosses a city). Menus are read-heavy and cacheable.

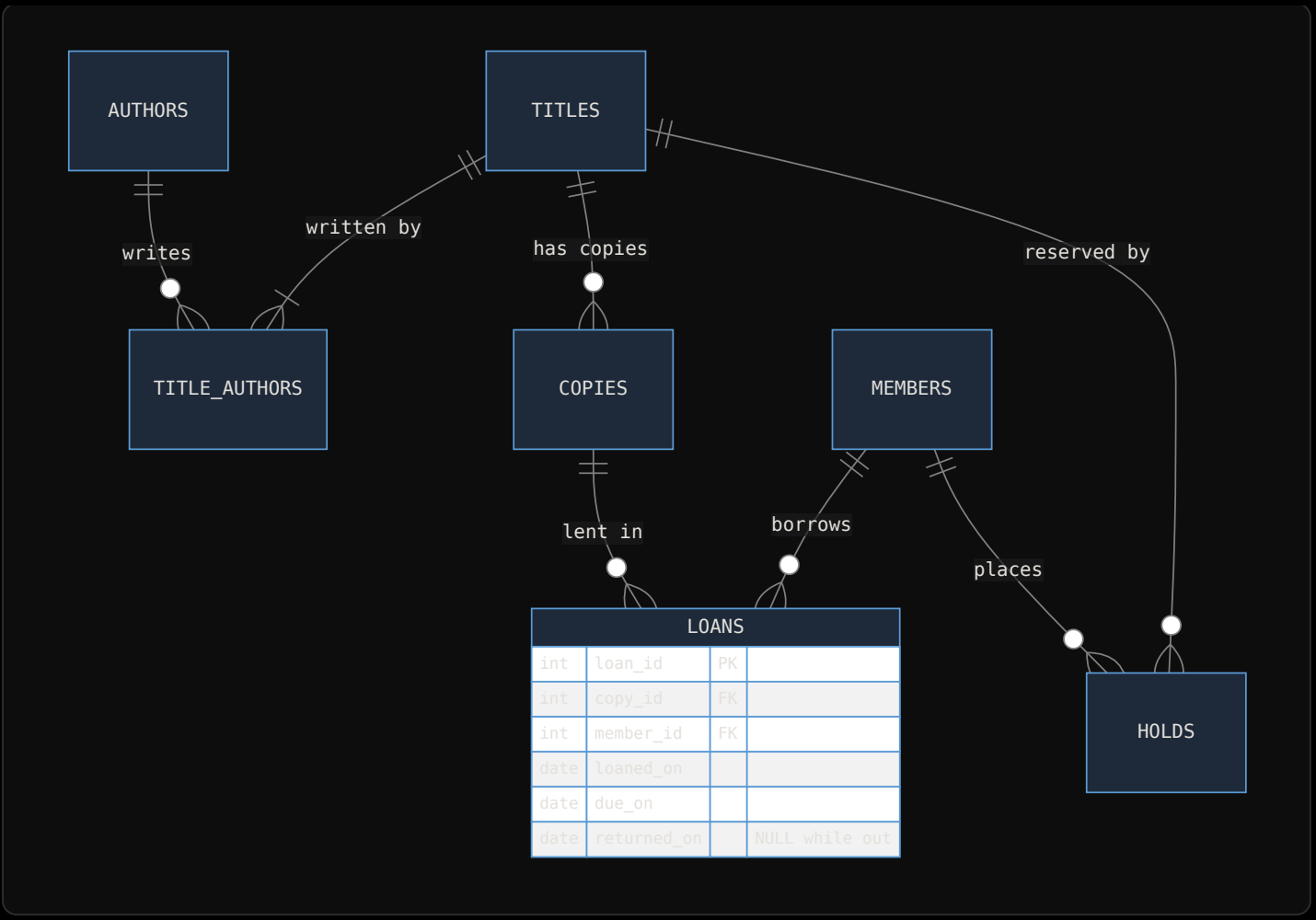
17.7 Library management system (runnable)

Requirements

- The catalogue lists **titles** (by **ISBN**, the international book number) with one or more **authors** in a fixed order; the library owns several physical **copies** per title across **branches**.
- **Members** borrow copies (**loans**) for a fixed period, up to a maximum number of open loans.
- A copy is on at most one open loan. Late returns incur a daily **fine**.
- **Holds** on a title when no copy is free, served first-come, first-served.
- Lost or discarded copies leave the catalogue but keep their loan history.
- Questions: can a member get this title today; which of a member's loans are late; one copy's lending history; the hold queue order; a member's unpaid total.

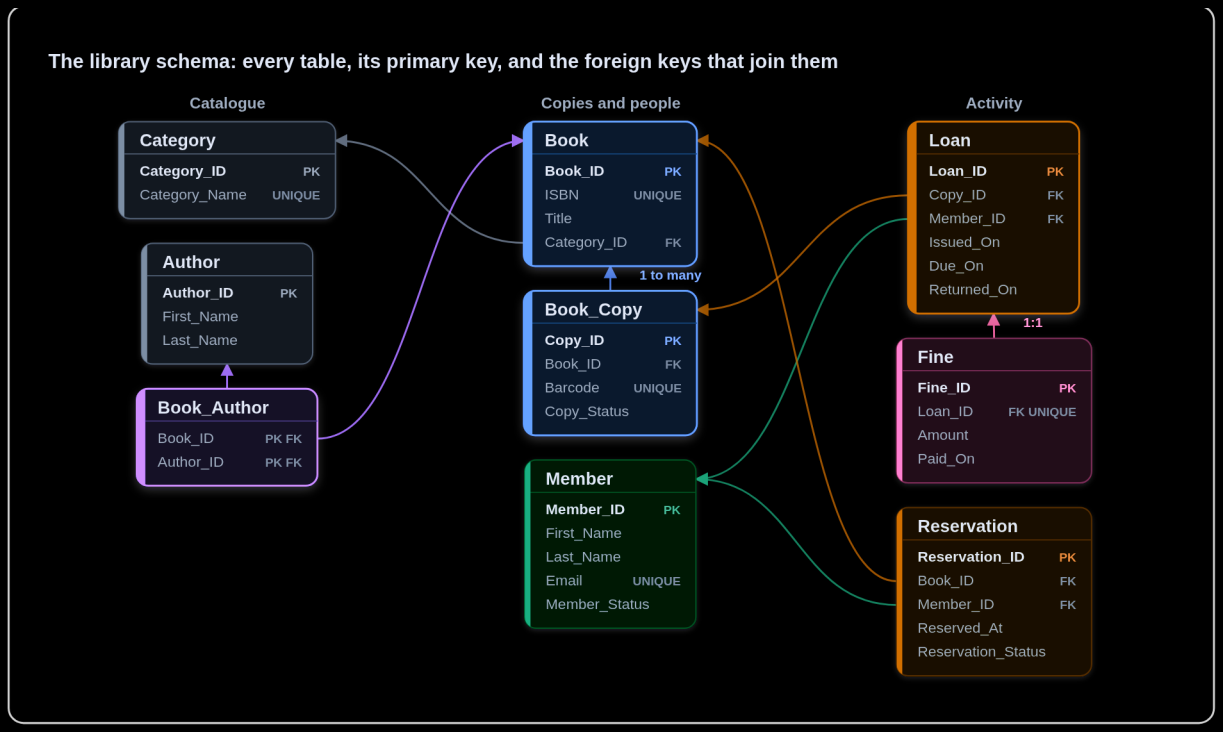
Entities and relationships

The cleanest **type versus instance** example: a *title* is the abstract book, a *copy* the object on the shelf. Loans point at copies; holds point at titles (you want *any* copy).



Key tables

```
CREATE TABLE title_authors (                        -- M:N junction with an ordering column
  isbn char(13) REFERENCES titles, author_id int REFERENCES authors,
  position smallint NOT NULL,
  PRIMARY KEY (isbn, author_id)
);
CREATE TABLE copies (
  copy_id int PRIMARY KEY, isbn char(13) NOT NULL REFERENCES titles,
  branch text NOT NULL, condition text NOT NULL DEFAULT 'good',
  withdrawn_at date                                 -- soft delete
);
CREATE TABLE loans (
  loan_id int GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  copy_id int NOT NULL REFERENCES copies, member_id int NOT NULL REFERENCES members,
  loaned_on date NOT NULL, due_on date NOT NULL, returned_on date,
  CHECK (due_on > loaned_on),
  CHECK (returned_on IS NULL OR returned_on >= loaned_on)
);
-- At most ONE open loan per copy:
CREATE UNIQUE INDEX one_open_loan_per_copy ON loans (copy_id) WHERE returned_on IS NULL;
CREATE TABLE holds (
  isbn char(13) REFERENCES titles, member_id int REFERENCES members,
  placed_at timestamptz NOT NULL,
  PRIMARY KEY (isbn, member_id)                     -- one hold per member per title
);
```



The tell-tale FK pair: `loans.copy_id` **but** `holds.isbn` — borrow one object, reserve a title; `copy_id` plays the barcode. `fines (loan_id PRIMARY KEY REFERENCES loans, amount, paid_on)` uses the loan id as key (1:1, at most one fine per loan), and **stores** the amount although it looks derived, because a later change of the daily rate must not rewrite old fines. The course gives books a **surrogate** `book_id` (no business meaning, unlike the **natural key** ISBN) with `isbn UNIQUE` (shorter FKs; some items lack an ISBN) and a `category` table; authors are a junction table, not a comma-separated `authors` column (which breaks 1NF and makes "books by this author" a text search).

Not stored: a copy's "available" flag — it derives from "no open loan and not withdrawn", so it never disagrees with loans. The **partial unique index** is the star: it enforces a rule about *some* rows (open loans) that a plain `UNIQUE` can't, and doubles as the "is this copy out?" index. (PostgreSQL and SQLite have partial indexes; SQL Server *filtered* indexes; MySQL neither — use a **generated column** that is non-NULL only for open loans, with a unique index on it.)

Key queries

Catalogue view with authors in order and live availability (`03_library.sql`, "today" = 2026-05-10):

```
SELECT t.title,
       string_agg(a.name, ', ' ORDER BY ta.position) AS authors,
       (SELECT count(*) FROM copies c
         WHERE c.isbn = t.isbn AND c.withdrawn_at IS NULL
           AND NOT EXISTS (SELECT 1 FROM loans l WHERE l.copy_id = c.copy_id AND l.returned_on IS NULL)
       ) AS available
FROM titles t JOIN title_authors ta USING (isbn) JOIN authors a USING (author_id)
GROUP BY t.isbn
ORDER BY t.title;
```

Output:

```
        title         |      authors       | available
----------------------+--------------------+-----------
 Joins at Scale       | J. Ortiz, S. Novak |         0
 Query Plans Explained | R. Kumar           |         1
 Tables and Keys      | R. Kumar           |         1
```

(`Query Plans Explained` has two copies, one withdrawn; `Tables and Keys` two, one out.) Overdue loans and fines come from dates (`date - date` = whole days in PostgreSQL):

```
 name |      title      |   due_on   | days_late | fine
------+-----------------+------------+-----------+------
 Omar | Tables and Keys | 2026-05-04 |         6 | 1.50
```

Lending copy 101 while it's still out hits the partial unique index:

```
psql:03_library.sql:32: ERROR:  duplicate key value violates unique constraint "one_open_loan_per_copy"
DETAIL:  Key (copy_id)=(101) already exists.
```

Check-out under the member's limit is a conditional insert:

```sql
INSERT INTO loans (copy_id, member_id, loaned_on, due_on)
SELECT 102, 2, :today, date :today + 14
WHERE (SELECT count(*) FROM loans WHERE member_id = 2 AND returned_on IS NULL)
      < (SELECT max_loans FROM members WHERE member_id = 2)
RETURNING loan_id, copy_id, member_id, due_on;
```

Output: `7 | 102 | 2 | 2026-05-24`. Two concurrent check-outs by one member could both see "1 open loan" — the **write-skew** shape of §13.8 — so lock the member row first (`SELECT … FROM members WHERE member_id = 2 FOR UPDATE`) to serialize them. The hold queue is a window function, `row_number() OVER (PARTITION BY isbn ORDER BY placed_at)`: Eva 1, Omar 2 for *Joins at Scale*.

Indexes and review

Indexes: the partial unique index; `loans (member_id) WHERE returned_on IS NULL` (current loans); `copies (isbn)`; a full-text index on titles/authors. Loans are a *history* table — a `current_member_id` on copies would answer only "who has it now". `title_authors`'s only non-key column `position` depends on the whole key (2NF); ISBN and copy id are candidate keys (BCNF). Still missing: holds that expire (`expires_at` plus a sweep job when a copy is set aside) and a `CHECK` on copy condition so a typo can't become a new status.

Scaling notes

Small — one PostgreSQL instance serves a city's network; partition historical loans by year. The only hot spot is a popular new release with a long hold queue: handle "copy returned → offer to next hold → expire after 3 days" as a small **state machine** (fixed states with allowed transitions) in one transaction.

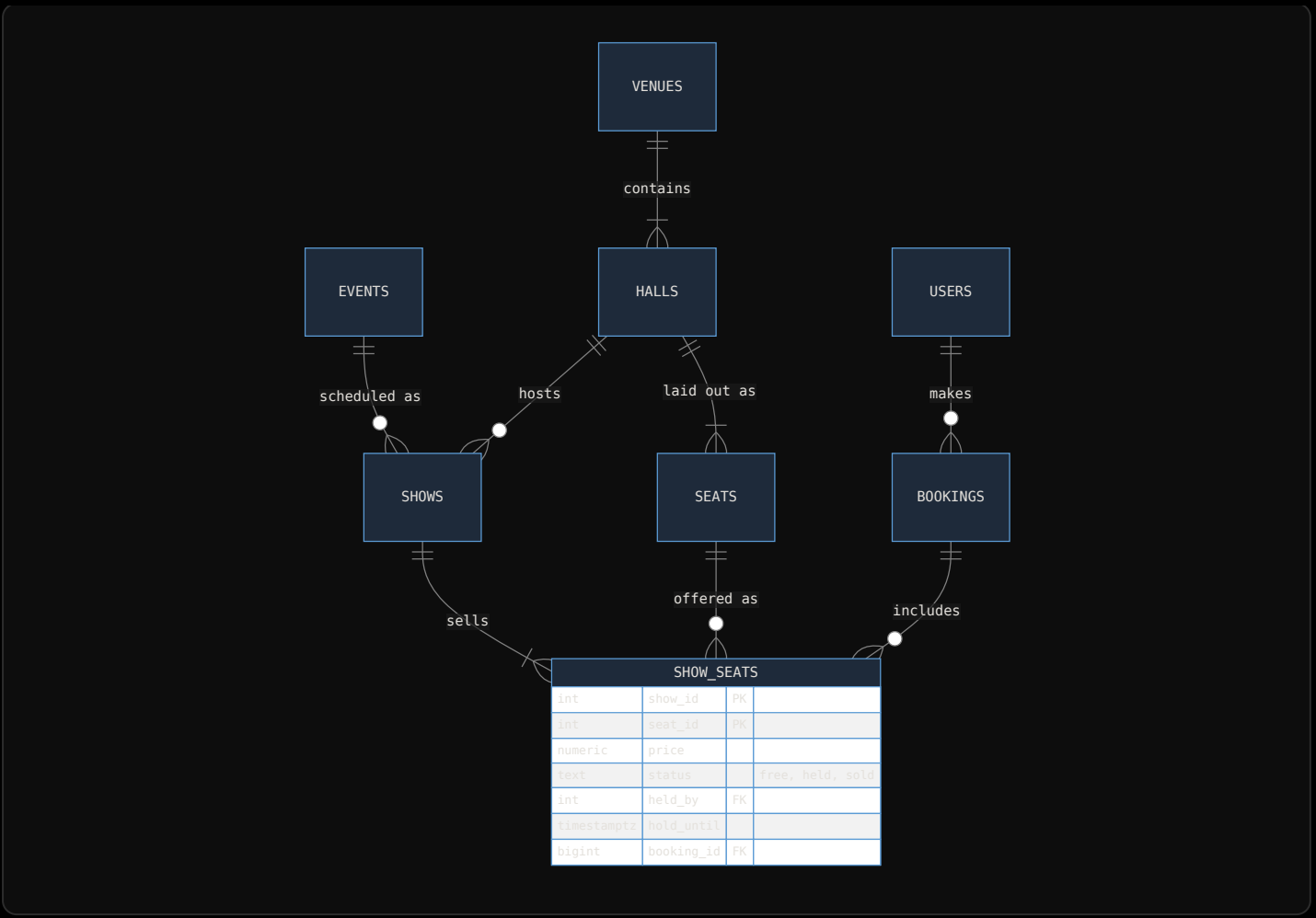
17.8 Ticket booking system (runnable, with seat-hold concurrency)

Requirements

- **Venues** have **halls** with a fixed **seat** layout (row, number, tier).
- **Events** (films, concerts) run as **shows** at a hall and time; each seat of each show has a price.
- A user selects seats; they are **held** for a few minutes during payment. Success → **sold** under a **booking**; expired hold → back on sale.
- **No seat may ever be sold twice**, even with thousands clicking as sales open.
- A booking has at most one **payment**; cancelling releases its seats.
- Questions: free seats for a show, a customer's bookings, what's on in a city today, how full a show was, whether a booking is paid.

Entities and relationships

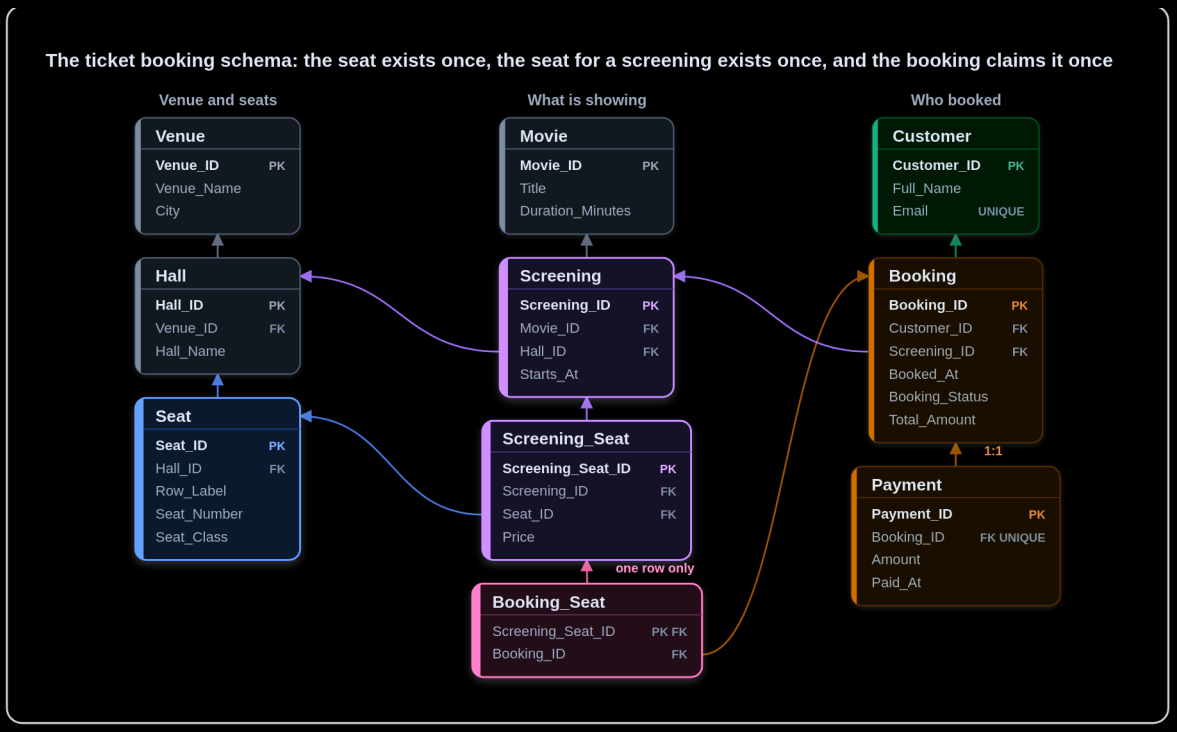
The key table is `show_seats` — one row per seat *per show*. Physical `seats` never change; `show_seats` is the inventory users compete for, and its row lock is the unit of concurrency.



Key tables

```
CREATE TABLE show_seats (
  show_id    int REFERENCES shows,
  seat_id    int REFERENCES seats,
  price      numeric(8,2) NOT NULL,
  status     text NOT NULL DEFAULT 'free' CHECK (status IN ('free', 'held', 'sold')),
  held_by    int REFERENCES users,
  hold_until timestamptz,
  booking_id bigint REFERENCES bookings,
  PRIMARY KEY (show_id, seat_id),               -- a seat appears once per show
  CONSTRAINT held_needs_owner_and_expiry
    CHECK ((status = 'held') = (hold_until IS NOT NULL AND held_by IS NOT NULL)),
  CONSTRAINT sold_needs_booking CHECK ((status = 'sold') = (booking_id IS NOT NULL))
);
CREATE INDEX ON show_seats (show_id) WHERE status = 'free';
```

The named `CHECK`s tie status to its required columns (held → owner and expiry; sold → booking; free → neither): impossible states can't be stored. Three things sound like "seat": the physical seat (row H, number 14), **the seat at one screening** (sold at 20:00, free at 18:00), and a claim on it. An `is_booked` flag on `seats` fails at a hall's second show; the **price** belongs to the seat-at-a-show pairing (weekends cost more; recliners cost more).



The course models the claim as a junction `booking_seats (screening_seat_id PRIMARY KEY, booking_id)`. The trap: a composite PK `(booking_id, screening_seat_id)` only stops *one booking* claiming a seat twice — two bookings could both claim H14. Making `screening_seat_id` the **whole** PK means a seat can be claimed once, ever. That design has no holds and must *delete* claim rows on cancellation (marking the booking cancelled would block the seat forever); the `status` column here gives the same once-only guarantee via the guarded `UPDATE` below and adds holds. The course names the table `screening` because `SHOW` is reserved in MySQL; plural `shows` is safe.

Key queries

Hold seats with a *guarded* update — the `WHERE` matches only free (or expired-hold) seats, and the returned rows say exactly which seats you got (`02_ticket_booking.sql`, "now" = 10:00):

```
UPDATE show_seats ss
SET status = 'held', held_by = 1, hold_until = timestamptz :now + interval '10 minutes'
FROM seats se
WHERE ss.seat_id = se.seat_id AND ss.show_id = 10
  AND se.row_label = 'B' AND se.seat_no IN (2, 3)
  AND (ss.status = 'free' OR (ss.status = 'held' AND ss.hold_until < :now))
RETURNING se.row_label || se.seat_no AS seat, ss.status, ss.hold_until;
```

Output — Asha gets B2 and B3; Ben's request a moment later for B3 and B4 returns only B4, so the app releases it and asks him to choose again:

```
 seat | status |       hold_until
------+--------+------------------------
 B2   | held   | 2026-05-20 10:10:00+00
 B3   | held   | 2026-05-20 10:10:00+00

 seat | held_by          <- Ben asked for B3 and B4
------+---------
 B4   |       2
```

Confirm after payment in one transaction — create the booking, flip only this user's *unexpired* holds to sold:

```
BEGIN;
INSERT INTO bookings (user_id, show_id, status, created_at)
VALUES (1, 10, 'confirmed', :now) RETURNING booking_id \gset
UPDATE show_seats
SET status = 'sold', booking_id = :booking_id, held_by = NULL, hold_until = NULL
WHERE show_id = 10 AND held_by = 1 AND status = 'held' AND hold_until >= :now
RETURNING seat_id, status, booking_id;
COMMIT;
```

The file continues with "any two free premium seats" via `FOR UPDATE SKIP LOCKED` (many buyers get *different* seats without waiting — §13.12), an expiry sweep freeing stale holds, a failed attempt to mark a seat sold without a booking (`violates check constraint "sold_needs_booking"`), and occupancy per show with `count(*) FILTER (…)` (show 10: 2 sold of 12, revenue 400.00).

Proof on two live sessions

`05_seat_race.sql` opens two real connections, A and B, with the `dblink` extension (one session running queries on another connection), and runs the same hold on seat 9 in both at the default `READ COMMITTED`:

```
--- A: holds seat 9 (C1) but has not committed yet
 seat_id | held_by
---------+---------
       9 |       1
--- B: sends the same UPDATE for user 2; it must wait for A's row lock
 b_still_running | b_waits_on
-----------------+------------
               1 | Lock
--- A commits. B wakes up, re-checks status = 'free' on the new row version, and updates nothing
 rows_b_changed
----------------
              0
--- final state of the seat: held by user 1 only
 seat_id | status | held_by
---------+--------+---------
       9 | held   |       1
```

1. A's `UPDATE` locks seat 9's row until A commits.
2. B's `UPDATE` finds the locked row and waits (`wait_event_type = Lock`).
3. On A's commit, PostgreSQL re-evaluates B's `WHERE` against the **newest** row version; `status = 'free'` is now false, so B updates zero rows.
4. The application sees 0 rows and tells user 2 "seat taken".

No `SELECT`-then-`UPDATE` (which *would* race), no application lock, no `SERIALIZABLE`. Alternatives: **optimistic locking** with a `version` column (check unchanged at write time, retry otherwise — good at low contention), or insert-to-claim with a unique constraint on a separate `seat_holds (show_id, seat_id)`.

Indexes and remaining rules

The PK `(show_id, seat_id)` serves the seat map and every hold; the partial index on free seats serves "best available" and "how many left"; `show_seats (held_by) WHERE status = 'held'` would speed confirmation; `shows (event_id, starts_at)` serves listings. Outside these tables: a job must generate `show_seats` for every new show (300 seats × 6 shows = 1,800 rows a day per hall), and two shows mustn't overlap in one hall (an exclusion constraint on `(hall_id, tstzrange(starts_at, ends_at))` or a check in the scheduling transaction).

Scaling notes

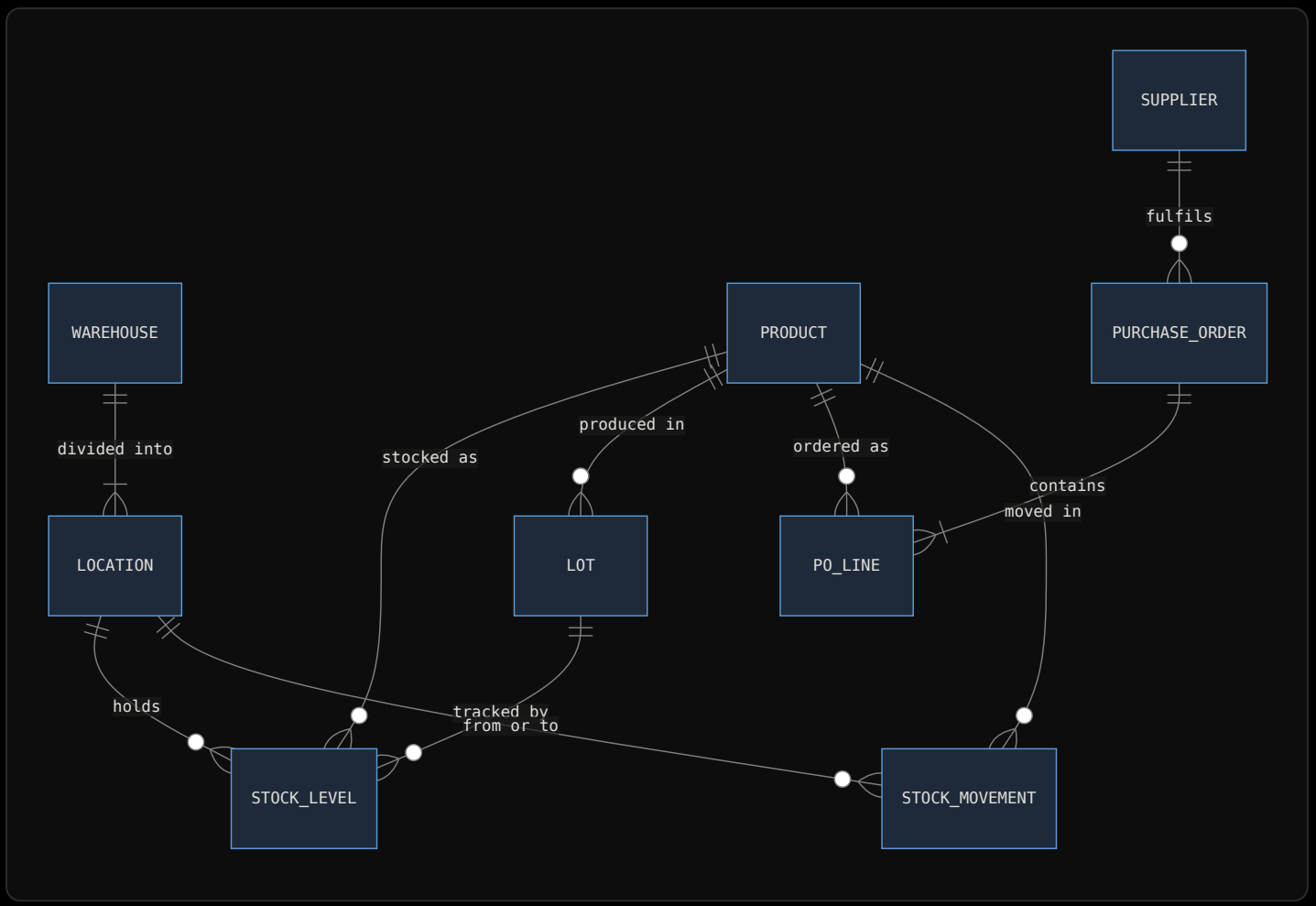
The problem is a **burst of contention on a few rows** (a concert's on-sale minute). The database can hold the rows but not the connections: put a **virtual waiting room / queue** in front (admitting users in batches), let only N users into seat selection at once, keep holds short. Shows are independent, so shard by `show_id` (or event); a hot show can get a dedicated shard. Cache the seat map (a second stale is fine) but never the hold.

17.9 Warehouse inventory system

Requirements

- **Products** (SKUs) stored in several **warehouses**, each divided into **bin locations** (aisle-rack-shelf).
- **Stock on hand** per SKU per location, plus **reserved** quantity for open orders.
- Goods arrive via **purchase orders** from **suppliers** (receipts), leave via **shipments**, move between locations (**transfers**); every movement is recorded.
- **Batches/lots** (units made together, sharing an expiry date) with first-expired-first-out picking; reorder alerts below a **reorder point**.
- Each supplier has its own SKU and **lead time** (days from order to delivery) per product; purchase orders have numbered lines.
- Questions: total stock of a SKU and its bins; a bin's movements this week; the cause of a count mismatch; the quickest supplier for a SKU; open purchase-order quantities.

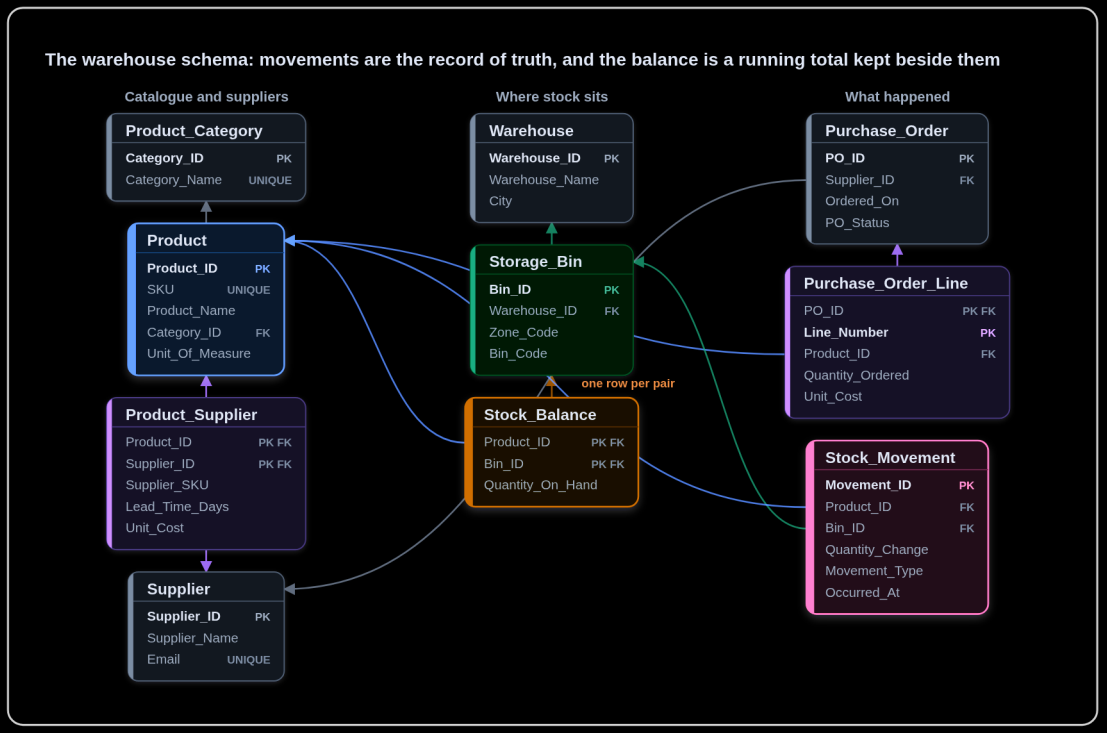
Entities and relationships



Key tables

```
CREATE TABLE stock_levels (                    -- current state, one row per product-lot-location
  product_id  bigint REFERENCES products,
  location_id bigint REFERENCES locations,
  lot_id      bigint REFERENCES lots,
  on_hand     int NOT NULL CHECK (on_hand >= 0),
  reserved    int NOT NULL DEFAULT 0 CHECK (reserved BETWEEN 0 AND on_hand),
  PRIMARY KEY (product_id, location_id, lot_id)
);
CREATE TABLE stock_movements (                 -- append-only ledger of every change
  movement_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  product_id  bigint NOT NULL REFERENCES products,
  lot_id      bigint REFERENCES lots,
  from_location bigint REFERENCES locations,   -- NULL for receipts
  to_location   bigint REFERENCES locations,   -- NULL for shipments / write-offs
  quantity    int NOT NULL CHECK (quantity > 0),
  reason      text NOT NULL CHECK (reason IN ('receipt','shipment','transfer','adjustment','return')),
  reference   text,                            -- PO number, order number ...
  moved_at    timestamptz NOT NULL DEFAULT now(),
  CHECK (from_location IS NOT NULL OR to_location IS NOT NULL)
);
```

The bank's **ledger + cached balance** design again (§17.3): movements are the truth, `stock_levels` the fast answer, both changed in one transaction. A periodic rebuild-and-compare catches bugs — and explains a shelf count that disagrees with the screen by replaying one product's movements in order. Two opposite reasons to store a derivable value: the library fine is kept so it **never** changes; this balance is kept because summing every movement per read is too slow.



The course variant records one row per bin with a **signed** `quantity_change` (+50 received, −3 picked) instead of from/to columns — one table for every movement type; a transfer becomes two rows. Its other tables: `product_suppliers (product_id, supplier_id, supplier_sku, lead_time_days, unit_cost)` — attributes of the *pairing*, so `supplier_name` there would be a 2NF partial dependency; and `purchase_order_lines (po_id, line_no, …)`, a **weak entity** whose key includes its parent's, with `ON DELETE CASCADE`. `unit_cost` appears on both: the agreed price now vs the price paid on that order. A movement's cause is `reference_type` + `reference_id` — a polymorphic reference no FK can check (§17.14). Whether `on_hand` may go negative is a business choice (some like negative stock as an alarm for an unrecorded delivery) — decide and document it.

Key queries and indexes

1. Available to promise per warehouse: `sum(on_hand - reserved)` grouped by warehouse.
2. FEFO pick list: stock rows for the SKU joined to `lots`, ordered by `expires_on`, with a running `sum() OVER (ORDER BY expires_on)` until the ordered quantity is covered.
3. Reorder alerts: products whose total available is below `reorder_point`.
4. Movement history of a lot (for recalls).

Indexes: `stock_levels` PK for picks; `stock_levels (location_id)` for bin counts; `stock_movements (product_id, moved_at DESC)` and `(reference)`; `lots (product_id, expires_on)`.

Scaling notes

Warehouses are mostly independent: shard or run one database per site with a central read model for company-wide stock. Movements grow forever — partition by month. Online-order reservations are the contention point; a reservation row per order line (with expiry, like ticket holds) is safer than decrementing a shared counter.

The last four cases are large consumer products. The relational core resembles the earlier designs, but **no single database fits every part**, so each ends with a "which store for which data" table (§16.10). At this scale the template gains three steps before the tables:

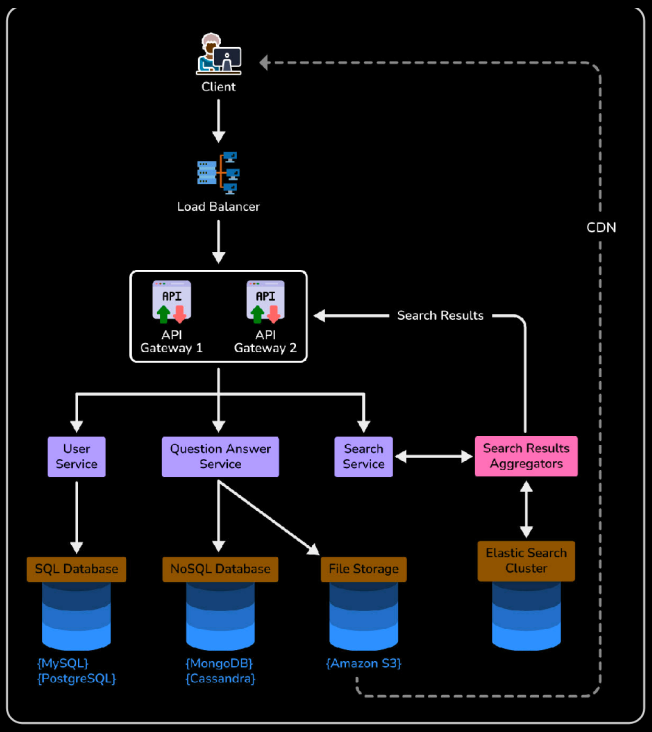
1. **Non-functional targets** — high availability, low latency (a few hundred ms), horizontal scalability, and *where eventual consistency is acceptable* (vote and view counts) versus not (money, bookings, identity).
2. **A back-of-the-envelope storage estimate** — users × bytes per row, items per day × size. Multiply carefully (1 KB × 10 million = 10 GB, not 10 TB); usually rows are small and **media dominates** by orders of magnitude — why media goes to object storage.
3. **A high-level architecture** — clients → **load balancer** (spreads requests over servers) → **API gateway** (single entry point for routing and authentication) → a few **microservices** (small, separately deployed services: user, content, search …), each owning its data store, with a **message queue** (Kafka, RabbitMQ) for **asynchronous** work (search indexing, notifications, video processing).

After the schema come **sharding** (a key keeping each request on one shard, with consistent hashing so adding a shard moves little data) and **replication** (primary–replica with automatic failover for the relational part, leaderless replication for Cassandra-style stores, replicas in several regions) — mechanics in Chapter 15.

17.10 Quora (questions and answers)

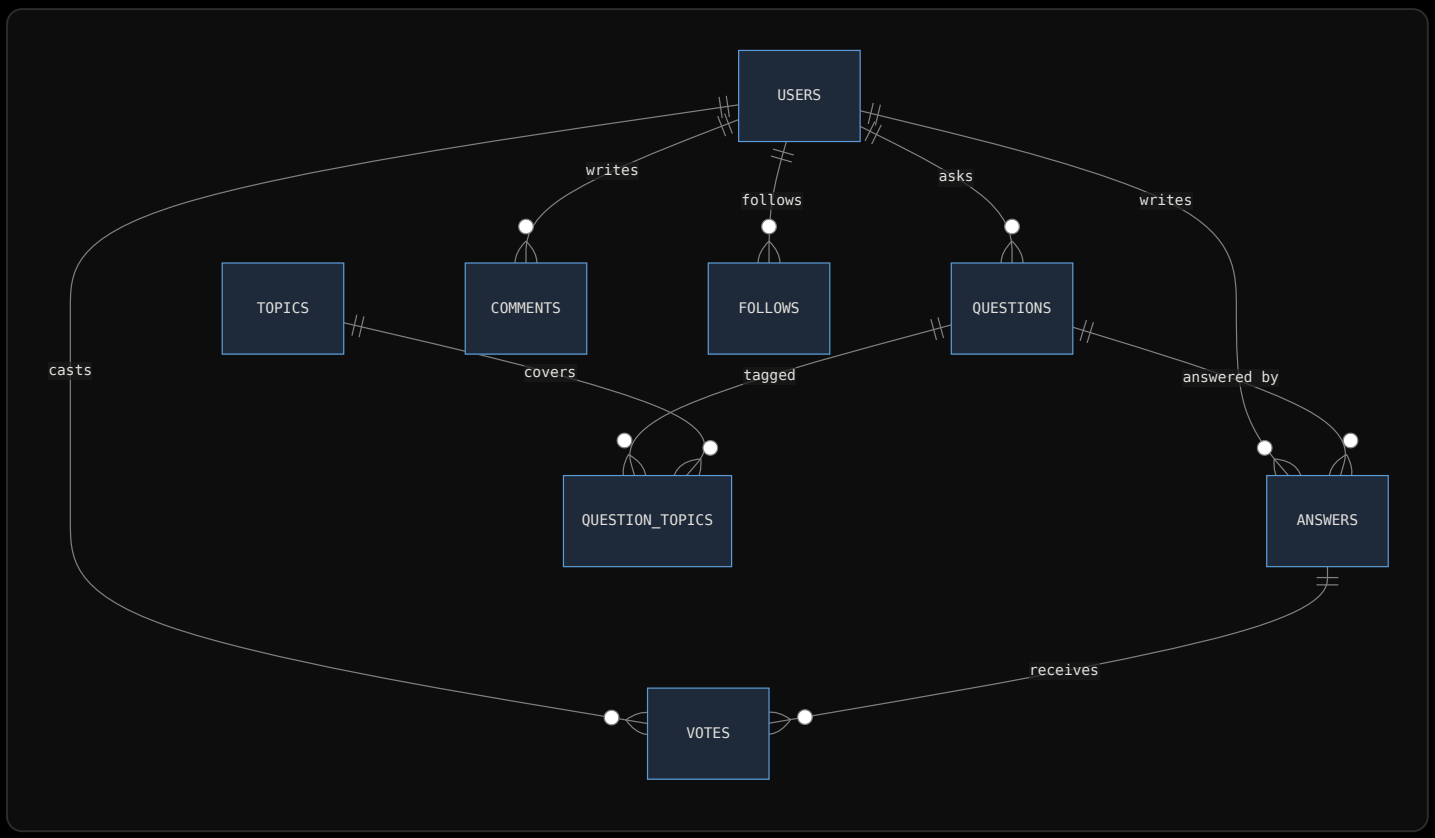
Requirements

- Users ask **questions**, write **answers**, **comment** on both; questions carry **topics**.
- **Upvote/downvote** answers; answers ranked by votes and quality.
- **Follow** people, topics and questions; a personalized feed of new questions and answers.
- Full-text **search** over questions; similar-question suggestions while typing.
- Scale: hundreds of millions of monthly users, reads ≫ writes; vote counts may lag (eventual consistency fine).
- Estimate (100 M users at 2 KB; per year 10 M questions at 1 KB, 50 M answers at 2 KB, 500 M votes at 0.5 KB, 10 M media files at 500 KB): ~200 GB of user rows, then ~10 GB + 100 GB + 250 GB of text and **5 TB of media** per year — relational data fits comfortably; media and replicas drive storage.



The course's Quora HLD: clients, load balancer, API gateway, user / question-answer / search services, a relational DB, a NoSQL store, Elasticsearch, a message queue and file storage.

Entities and relationships



Key tables

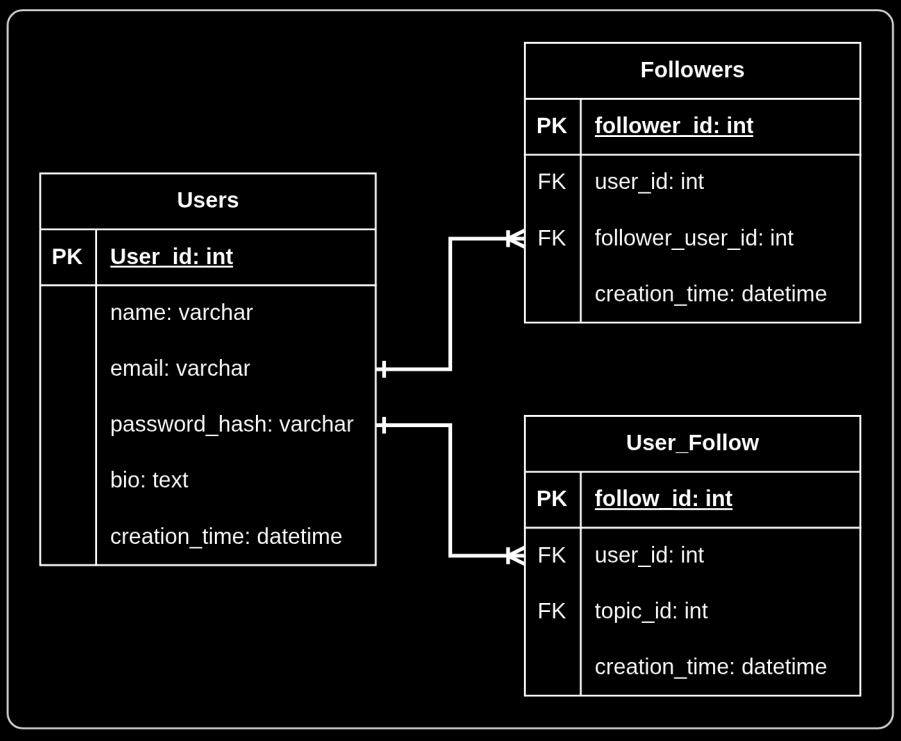
```sql
CREATE TABLE answers (
  answer_id   bigint PRIMARY KEY,
  question_id bigint NOT NULL REFERENCES questions,
  author_id   bigint NOT NULL REFERENCES users,
  body        text NOT NULL,
  upvotes     int NOT NULL DEFAULT 0, downvotes int NOT NULL DEFAULT 0,   -- denormalized
  created_at  timestamptz NOT NULL, updated_at timestamptz,
  UNIQUE (question_id, author_id)                -- one answer per user per question
);
-- votes (answer_id, user_id, value CHECK (value IN (-1, 1)), PRIMARY KEY (answer_id, user_id))
CREATE TABLE comments (                          -- polymorphic target: a question OR an answer
  comment_id  bigint PRIMARY KEY,
  question_id bigint REFERENCES questions,
  answer_id   bigint REFERENCES answers,
  author_id   bigint NOT NULL REFERENCES users,
  body text NOT NULL, created_at timestamptz NOT NULL,
  CHECK (num_nonnulls(question_id, answer_id) = 1)
);
CREATE TABLE follows (                           -- polymorphic: follow a user, topic or question
  follower_id bigint REFERENCES users,
  target_type text NOT NULL CHECK (target_type IN ('user','topic','question')),
  target_id   bigint NOT NULL,                   -- no FK possible: it points at 3 tables
  PRIMARY KEY (follower_id, target_type, target_id)
);
```

The two ways to model a **polymorphic association** (one row pointing to different parent tables): `comments` uses **one nullable FK per possible parent** plus a `CHECK` that exactly one is set — FKs still work; `follows` uses a **type + id pair** — compact and extensible but without referential integrity (nothing checks topic 42 exists). Prefer the first when parent types are few and fixed.

Key queries and indexes

1. Answers ranked: `ORDER BY upvotes - downvotes DESC, created_at` (or a stored `score` also weighing author credibility).
2. Feed: new answers in followed topics/questions/people since the last visit.
3. Voting: `INSERT … ON CONFLICT (answer_id, user_id) DO UPDATE SET value = EXCLUDED.value`, adjusting counters by the difference.

Indexes: `answers (question_id, upvotes DESC)`, `question_topics (topic_id, question_id)`, `follows (target_type, target_id)` ("who follows this topic"), full-text (`tsvector` GIN) or an external search engine.



The document alternative. The course stores each question as one MongoDB-style document with answers (and their comments) **embedded**, so a question page is one read; a mirrored user document carries follower counts and followed topics. Embedding is denormalization: great for "show this question", awkward for "all answers by this user" or a 10,000-answer question (MongoDB caps documents at 16 MB) — the usual compromise embeds the top answers and keeps the rest in their own collection.

Sharding. By `user_id`, a user's content is local but celebrities become **hot spots**; by hashed `question_id`, load spreads but a user's timeline needs **scatter-gather** across shards. Recommended mix: question data by a hash of `question_id`, user data (profiles, follows) by a hash of `user_id`, consistent hashing, and a shard-manager service mapping keys to shards.

Which store for which data

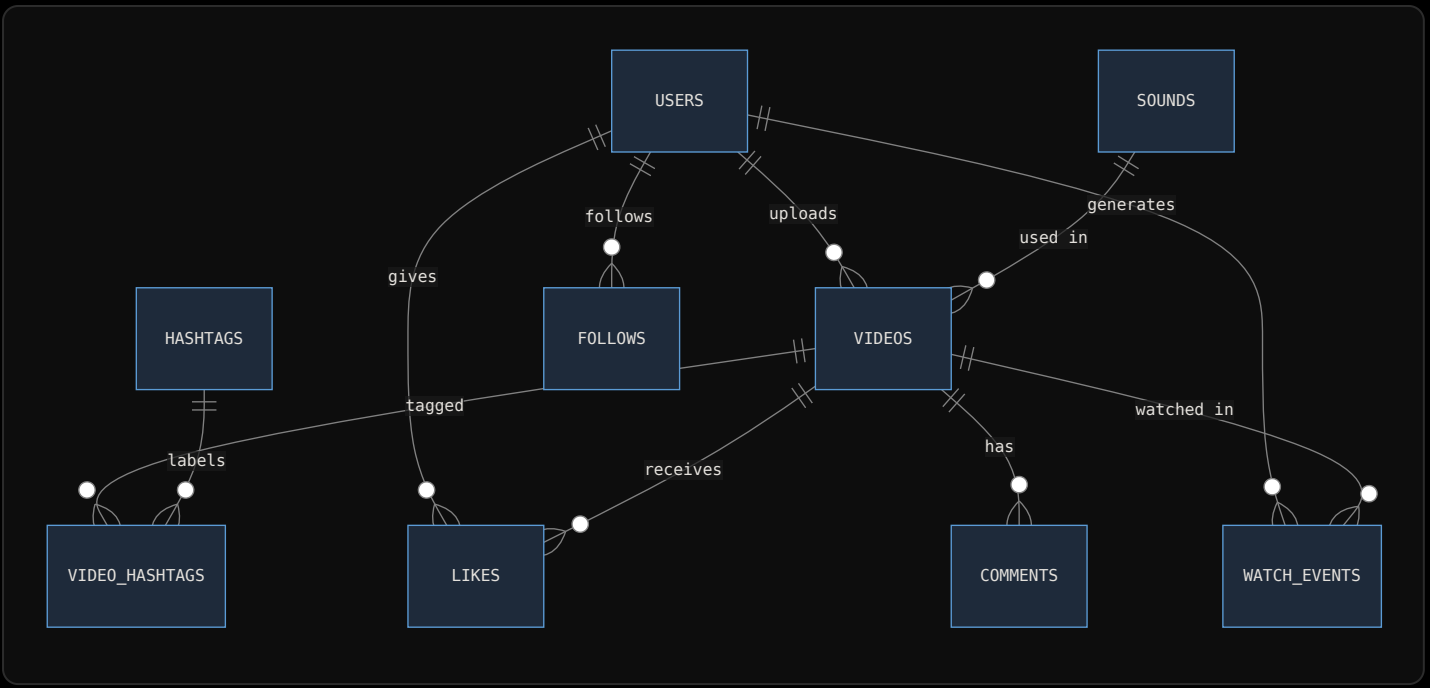
Data	Store
Users, questions, answers, votes	Sharded relational DB (Quora itself runs sharded MySQL — tens of terabytes and hundreds of thousands of queries per second, per its 2020 engineering blog) — or questions/answers in a document or wide-column store, as in the course design
Answer rankings, feed lists, counters	In-memory cache (Redis/Memcached)
Search and "similar question" suggestions	Search engine (Elasticsearch/OpenSearch)
Images in answers	Object storage + CDN

17.11 TikTok (short-video platform)

Requirements

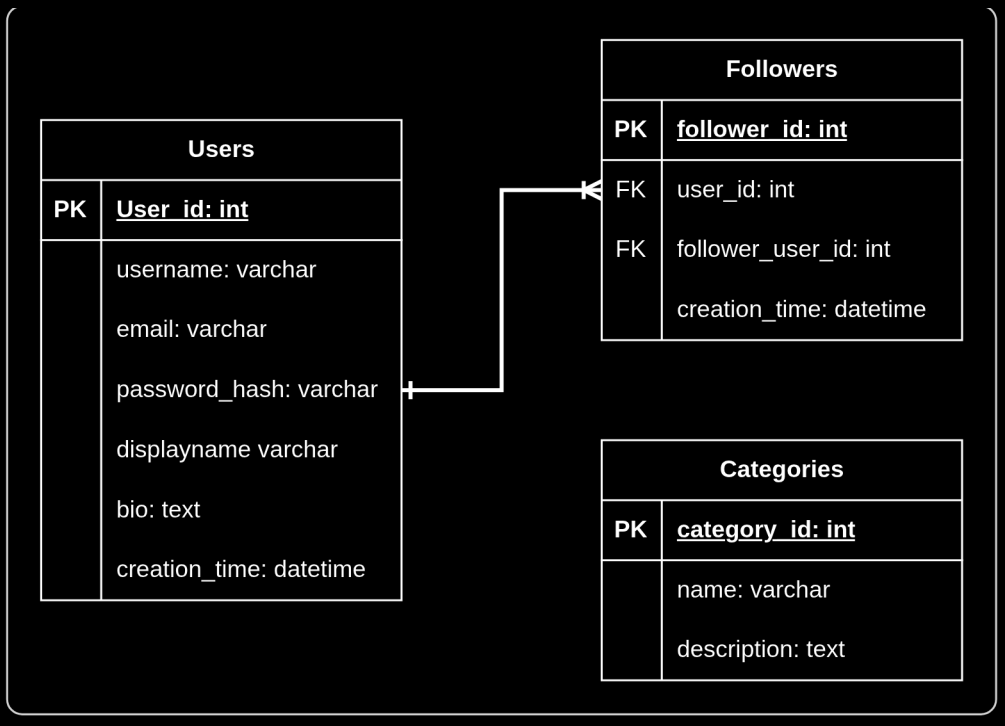
- Users upload short **videos** with captions, **sounds** (music reusable across videos) and **hashtags**.
- Viewers **like**, **comment**, **share**, **follow**; view counts on every video.
- The main surface is a recommendation-driven **"For You" feed**, fed by **watch events** (what was watched, how long, skipped, replayed).
- Real-time **notifications**; **trending** videos and hashtags; **categories** (dance, comedy).
- Scale: billions of views per day; view/watch events vastly outnumber everything else.
- Estimate (1 B daily users each uploading 2 videos of 50 MB): **100 PB of video a day**, against ~4 TB of metadata (2 B videos × 2 KB) and ~2 TB of comments (2 B × 1 KB). Video outweighs the rest by more than 10,000 to 1, so it lives in object storage with **lifecycle rules** (policies moving old files to cheaper storage or deleting them), never in a database.

Entities and relationships



Key tables

```
CREATE TABLE videos (
  video_id    bigint PRIMARY KEY,                 -- time-ordered id (Snowflake-style)
  creator_id  bigint NOT NULL REFERENCES users,
  sound_id    bigint REFERENCES sounds,
  caption     text,
  storage_key text NOT NULL,                      -- original upload in object storage
  duration_ms int NOT NULL,
  status      text NOT NULL CHECK (status IN ('processing','public','private','removed')),
  created_at  timestamptz NOT NULL
);
-- video_renditions (video_id, resolution, bitrate_kbps, storage_key): transcoded copies
CREATE TABLE video_stats (                        -- hot counters kept apart from the video row
  video_id bigint PRIMARY KEY REFERENCES videos,
  views bigint NOT NULL DEFAULT 0, likes bigint NOT NULL DEFAULT 0,
  comments bigint NOT NULL DEFAULT 0, shares bigint NOT NULL DEFAULT 0
);
```



The course puts high-volume tables in **Cassandra**, modeled around the query rather than the entity. A correct CQL sketch (Cassandra has no `ENUM`; **counter** columns — increment/decrement only — need a table of their own; a `timeuuid` is a unique id embedding a timestamp):

```
-- CQL (Cassandra) — not run
CREATE TABLE interactions_by_video (
  video_id uuid, created_at timeuuid, user_id bigint,
  kind text,                      -- 'like' | 'comment' | 'share'
  comment_text text, parent_id timeuuid,
  PRIMARY KEY ((video_id), created_at)          -- partition key, then clustering column
) WITH CLUSTERING ORDER BY (created_at DESC);
CREATE TABLE video_counters (
  video_id uuid PRIMARY KEY, views counter, likes counter, comments counter, shares counter
);
CREATE TABLE user_activity (
  user_id bigint, activity_time timestamp, activity_type text, details text,
  PRIMARY KEY ((user_id), activity_time)
) WITH CLUSTERING ORDER BY (activity_time DESC);
```

Relational tables keep what needs strong consistency (accounts, follows, categories); Cassandra keeps what needs write throughput and tolerates eventual consistency. `video_renditions` holds **transcoded** copies (re-encoded at several resolutions and bitrates for different devices). Splitting `video_stats` from `videos` is a **vertical partition** (§15.2): counters change constantly, metadata almost never, so metadata rows and their cache entries stay stable. Watch events aren't a primary-database table at all but an **event stream** (an append-only ordered flow, e.g. Kafka) consumed by other systems.

Key queries and indexes

1. Profile grid: `videos WHERE creator_id = $1 AND status = 'public' ORDER BY video_id DESC` (time-ordered ids sort by upload time).
2. Videos using a sound ("use this sound" page).
3. Trending hashtags in the last hour — computed by a **stream processor**, not SQL over billions of rows.
4. For You feed — candidates generated and ranked by a recommendation service; the database only fetches metadata for chosen ids.

Indexes: `videos (creator_id, video_id DESC)`, `videos (sound_id)`, `video_hashtags (hashtag_id, video_id)`, `likes (video_id, user_id)` PK plus `likes (user_id, created_at)` for "videos I liked".

Which store for which data

Data	Store
Users, video metadata, follows	Sharded relational or distributed SQL
Video files and renditions	Object storage + CDN (edge caches near viewers)
Views, watch time, skips	Event log (Kafka) → stream processing → analytics warehouse / wide-column store
Counters	In-memory counters, flushed periodically; displayed counts may lag a little
Recommendation features, embeddings (number vectors placing similar items close together)	Feature store (a database of precomputed model inputs) + vector index (finds nearest embeddings; FAISS library, Milvus) fed by training pipelines
Notifications, processing jobs	Message queue → notification and transcoding workers

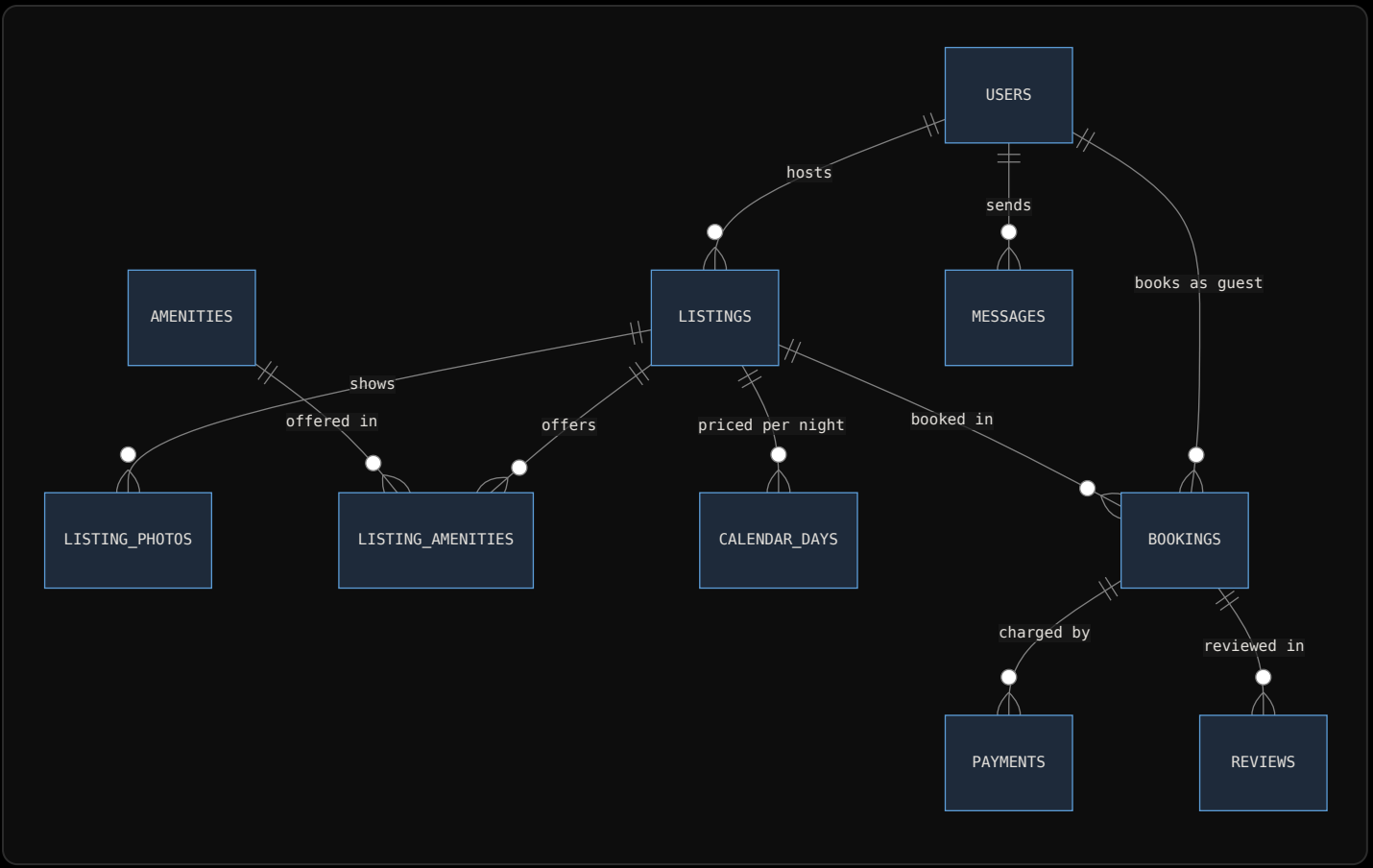
---

17.12 Airbnb (short-term rentals)

Requirements

- **Hosts** publish **listings** (address, geo-location, capacity, amenities, photos, house rules) with a **calendar** of availability and nightly prices.
- **Guests** search by location, dates and guest count, filter by price and amenities, then **book** (instant or on host approval).
- No listing booked twice for overlapping nights.
- **Payments** via an external **payment gateway** (charge the guest, pay out the host later, refunds, deposits), **reviews** in both directions after a stay, guest–host **messaging**; hosts accept or decline requests.
- Estimate (per day: 1 M listings created/updated at ~5 KB with 10 photos of ~1 MB; 10 M reservations and 10 M payments at 1 KB; 5 M reviews at 0.5 KB): photos ≈ **10 TB/day**, reservations + payments + reviews only ≈ 22 GB/day — the transactional core fits a relational cluster.

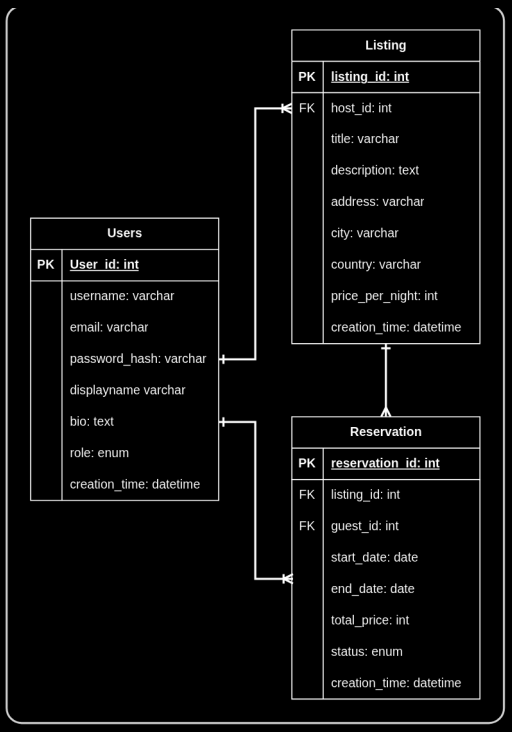
Entities and relationships



A user can be both **host** and **guest**: one `users` table with two roles, not two tables.

Key tables

```
-- needs the PostGIS extension for geography (not run here)
CREATE TABLE listings (
  listing_id bigint PRIMARY KEY,
  host_id    bigint NOT NULL REFERENCES users,
  title      text NOT NULL,
  location   geography(Point, 4326) NOT NULL,     -- PostGIS; lat/long for radius search
  max_guests smallint NOT NULL,
  base_price numeric(10,2) NOT NULL,
  status     text NOT NULL CHECK (status IN ('draft','active','snoozed','deleted'))
);
CREATE TABLE calendar_days (                      -- per-night availability and price overrides
  listing_id bigint REFERENCES listings,
  night      date,
  price      numeric(10,2),                       -- NULL = base price
  blocked    boolean NOT NULL DEFAULT false,      -- host blocked it
  PRIMARY KEY (listing_id, night)
);
CREATE TABLE bookings (
  booking_id bigint PRIMARY KEY,
  listing_id bigint NOT NULL REFERENCES listings,
  guest_id   bigint NOT NULL REFERENCES users,
  stay       daterange NOT NULL,
  guests     smallint NOT NULL,
  total      numeric(12,2) NOT NULL,
  status     text NOT NULL CHECK (status IN ('requested','confirmed','cancelled','completed')),
  EXCLUDE USING gist (listing_id WITH =, stay WITH &&) WHERE (status IN ('requested','confirmed'))
);
-- reviews (booking_id, author_role 'guest' | 'host', rating, body,
--          PRIMARY KEY (booking_id, author_role))  -> each side reviews once per stay
```



The course splits the work into **user, listing, booking and payment services**: users, listings, reservations and payments relational (atomic booking and money); reviews, activity logs and `property_images (listing_id, image_id, url)` — partitioned by `listing_id` so one read returns a listing's photos — in Cassandra; photo bytes in object storage. Its `users.role` column (host or guest) works only if nobody is both; `listings.host_id` is the more flexible way to express "host". The exclusion constraint on `(listing_id, stay)` is the hospital-appointment tool again; unlike a hotel, each listing is a single unit, so no inventory counter is needed.

Key queries and indexes

1. Search: active listings within 10 km (`ST_DWithin(location, $point, 10000)`) with enough capacity, **no** overlapping active booking and no blocked night in range, plus price filtering.
2. Book: insert; the exclusion constraint rejects a conflicting stay, even under concurrency.
3. Host dashboard: upcoming bookings per listing; payouts due.
4. Listing rating: `avg(rating)` of guest reviews, usually cached on the listing.

Indexes: GiST on `listings.location`; the exclusion constraint's GiST on `(listing_id, stay)`; `bookings (guest_id, lower(stay))` for "my trips"; `calendar_days` PK.

Which store for which data

Data	Store
Users, listings, bookings, payments	Relational (strong consistency for bookings and money)
Search with geo + filters + ranking	Search engine fed by change-data-capture from the DB
Photos	Object storage + CDN
Messages	Relational or wide-column, partitioned by conversation
Pricing suggestions, analytics	Data warehouse (replicated nodes plus regular backups)

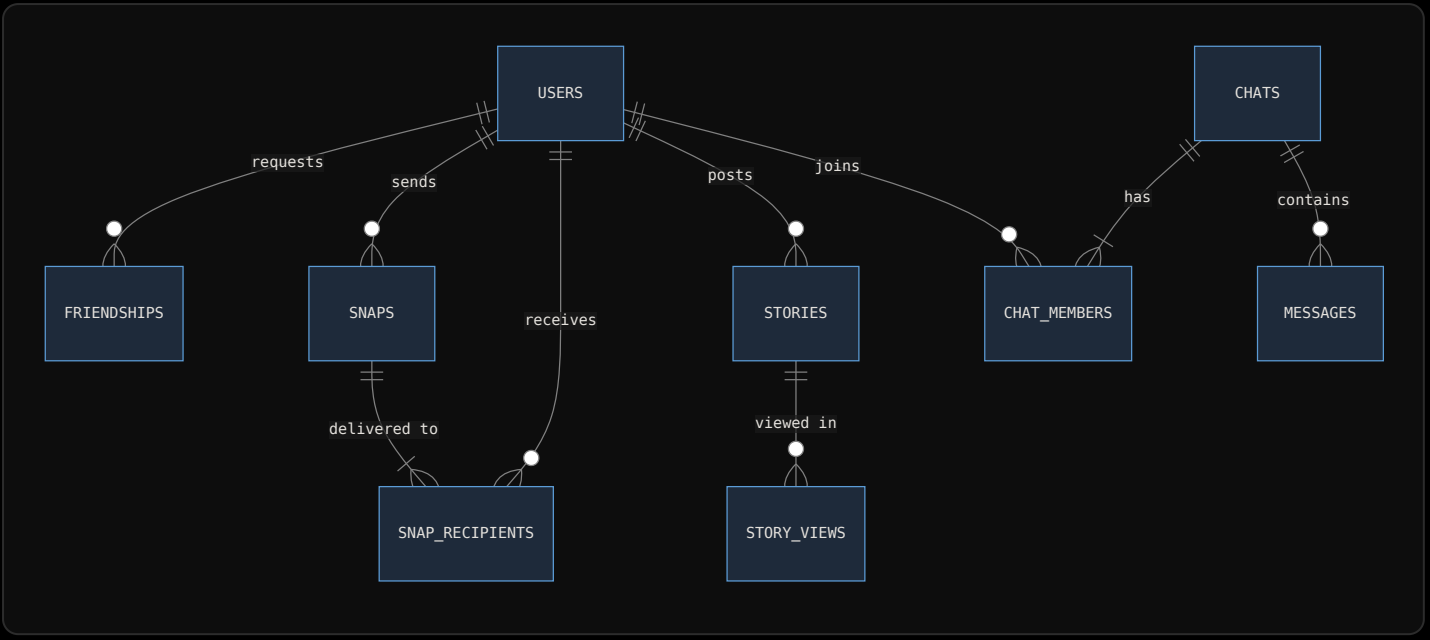
Replication: a primary with read replicas in several regions and automatic failover for the relational core; multi-node eventually consistent replication for Cassandra.

17.13 Snapchat (ephemeral messaging)

Requirements

- Users add **friends** (mutual, request/accept), send **snaps** (photo/video) to one or more friends, and **chat**.
- A snap **disappears** after every recipient opened it, or after a maximum time (e.g. 30 days unopened); a **story** is visible to friends for 24 hours.
- Senders see delivery and "opened" status per recipient; screenshots are reported.
- **Filters/lenses** on snaps and stories; users **searched** by name.
- Scale: billions of snaps a day; almost all data short-lived.
- Estimate (500 M daily users, 10 snaps each — 80 % photos of 1 MB, 20 % videos of 5 MB — plus one 10 MB story each): 4 PB + 5 PB of snaps and 5 PB of stories, **~14 PB of new media a day**. Since snaps vanish on viewing and stories after 24 hours, what is *held* stays bounded — deletion is as important a workload as writing.

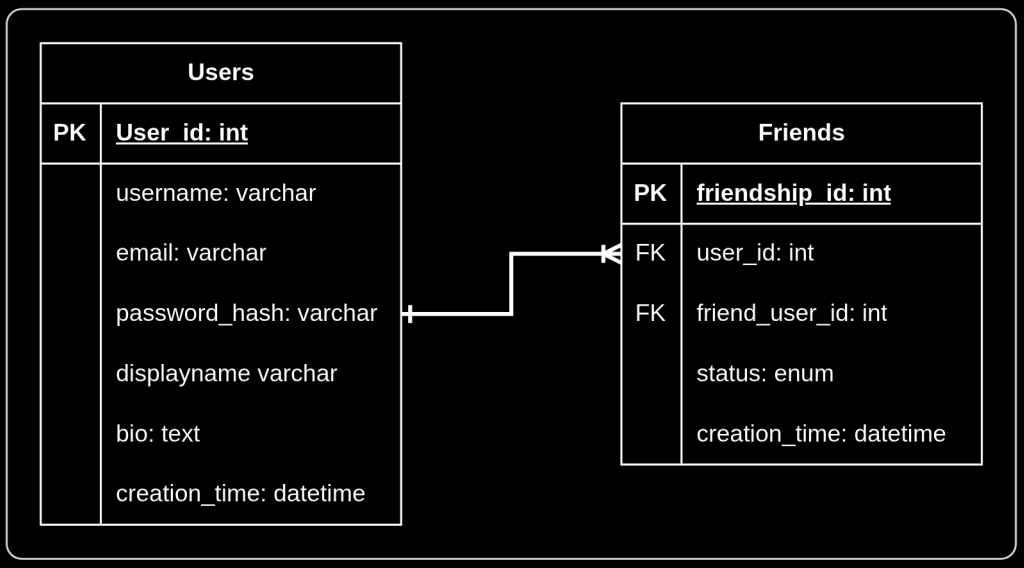
Entities and relationships



Key tables

```
CREATE TABLE friendships (                        -- one row per pair, stored in canonical order
  user_low   bigint REFERENCES users,
  user_high  bigint REFERENCES users,
  status     text NOT NULL CHECK (status IN ('pending','accepted','blocked')),
  requested_by bigint NOT NULL,
  PRIMARY KEY (user_low, user_high),
  CHECK (user_low < user_high)
);
CREATE TABLE snaps (
  snap_id    bigint PRIMARY KEY,
  sender_id  bigint NOT NULL REFERENCES users,
  media_key  text NOT NULL,                       -- encrypted blob in object storage
  created_at timestamptz NOT NULL,
  expires_at timestamptz NOT NULL                 -- hard deadline even if never opened
);
CREATE TABLE snap_recipients (
  snap_id      bigint REFERENCES snaps ON DELETE CASCADE,
  recipient_id bigint REFERENCES users,
  delivered_at timestamptz, opened_at timestamptz, screenshot_at timestamptz,
  PRIMARY KEY (recipient_id, snap_id)             -- "my inbox" is a PK range scan
);
-- stories (story_id, user_id, media_key, created_at, expires_at = created_at + 24 h, set on insert)
```

(`expires_at` can't be `GENERATED … STORED`: `timestamptz + interval` depends on the session time zone, so PostgreSQL rejects it as not **immutable** — a generated column's expression must always give the same output for the same input.)



The course's Cassandra `snaps` table is keyed by `snap_id` with a secondary index on the receiver; Cassandra works best when the **partition key is what you query by**, so an inbox partitioned by recipient (as `snap_recipients` does relationally) avoids the secondary index. Elasticsearch holds a user index for "find friends", sharded by the configured number of primary shards. The **canonical pair** trick (`user_low < user_high`) stores a symmetric relationship once, so "are A and B friends?" is one lookup on `(least(a,b), greatest(a,b))` — unlike Instagram's directional `follows`, one row per direction.

Key queries and indexes

1. Inbox: unopened snaps for a recipient, newest first.
2. Open a snap: set `opened_at`; when every recipient row has it, delete the media and the snap.
3. Friends' stories: stories of accepted friends where `expires_at > now()`.
4. Expiry sweep: delete snaps and stories past `expires_at`, in batches.

Indexes: `snap_recipients` PK `(recipient_id, snap_id)`, `snaps (expires_at)` for the sweeper, `stories (user_id, created_at DESC)`, `friendships (user_high)` for the reverse lookup.

Scaling notes and stores

Ephemeral data suits row-by-row `DELETE` poorly at this volume (dead tuples, vacuum work, index bloat — §13.14). Better: **time partitions** (partition `snaps` by day and `DROP` whole partitions — instant and bloat-free, §15.2) or a store with native **TTL** (Cassandra/ScyllaDB, DynamoDB) where rows expire by themselves. Snap's 2017 IPO filing disclosed five-year cloud commitments of $2 billion with Google Cloud and $1 billion with AWS — at this scale managed cloud databases are the usual choice over self-run servers.

Data	Store
Accounts, friend graph	Relational or key-value, sharded by user
Snaps, recipients, chat messages	Wide-column / key-value with TTL, partitioned by recipient or chat
Media	Encrypted object storage + CDN, lifecycle rules delete it
Presence ("online now"), typing indicators	In-memory store
User search	Search engine index

Partitioning at three levels: **horizontal** (rows by `user_id` or `snap_id` — users by range or consistent hashing, keeping friend lookups on one shard; Cassandra hashes partition keys itself), **vertical** (user data and media in separate stores), **functional** (each microservice — user, media, messaging, search — owns its data).

17.14 Recurring modeling patterns

Across the thirteen cases the same few patterns did most of the work — the chapter in one page:

Pattern	What it is	Seen in	Use it when	Watch out for
Junction table	A table whose PK is two FKs, resolving an M:N relationship, often with its own attributes	`order_items`, `likes`, `title_authors`, `post_hashtags`, `reservation_rooms`	Any M:N relationship; the PK doubles as the uniqueness rule ("like once")	Add an index on the *second* column for the reverse lookup
Type vs instance	Separate the kind of thing from the physical thing	`titles` / `copies`, `room_types` / `rooms`, `seats` / `show_seats`	Users choose a kind but you track individual items	Forgetting it forces one table to do two jobs
Status enum + state history	A constrained `status` column for the current state; optionally an append-only table of transitions	orders, bookings, reservations, food-delivery events	Every entity with a life cycle	Enforce allowed values (`ENUM`, `CHECK`, or lookup table); `CHECK`s can tie a status to required columns (ticket seats)
Soft delete	A `deleted_at` / `withdrawn_at` column instead of `DELETE`	products, library copies, listings	History or FKs must survive removal	Every query must filter it — use partial indexes and views; unique constraints need `WHERE deleted_at IS NULL`; privacy laws may still require a hard delete
Audit columns / ledger	`created_at`, `updated_at`, `created_by`; or a full append-only ledger of changes	hospital records, bank ledger, stock movements	Legal, financial or medical data; debugging "who changed this?"	Maintain with defaults/triggers, not by hand; ledgers need a cached balance for speed and a reconciliation query
Polymorphic association	One row points to one of several parent types	Quora comments (nullable FKs + `CHECK`), follows (`target_type` + `target_id`)	A comment/like/follow can target several entity kinds	The type+id form loses foreign keys; prefer one nullable FK per parent, or a shared supertype table
Denormalized counter / copy	Store a derived value (count, total, current status) to avoid recomputing	`like_count`, `orders.total`, `room_inventory`, `answers.upvotes`, `video_stats`	A measured hot read	Update it in the same transaction (or trigger), add a reconciliation check, and beware lock contention on hot rows
Snapshot of volatile data	Copy a value that may later change	`unit_price`, delivery address, `nightly_price`	The fact is "what it was at the time"	It is not a normalization bug — document it as intentional
Self-reference	A FK to the same table	category tree, comment replies, `follows`	Hierarchies and graphs	Query with recursive CTEs; guard against cycles
Overlap prevention	Exclusion constraint on a range, or partial unique index on "open" rows	appointments, Airbnb stays, hotel rooms, open library loans	"No two X at the same time"	PostgreSQL-specific; elsewhere lock and check in a transaction
Guarded update / claim	`UPDATE … WHERE state = 'free' RETURNING …`	seat holds, stock decrement	Many users compete for the same item	Check the returned row count; hold short; `SKIP LOCKED` for "any available"

Pattern	What it is	Seen in	Use it when	Watch out for
Time partitioning / TTL	Split big tables by time; drop old partitions or let rows expire	ledgers, movements, snaps, stories	Append-heavy data with a retention rule	Queries should include the partition key in `WHERE`

Judgement calls — cover the answer, say it aloud

1. *`order_items.unit_price` duplicates `products.price` — a 3NF violation?* → No: price paid on that order and today's list price are different facts.
2. *Why no stored `is_available` flag on copies or seats?* → Availability is derived from loans / seat status; a second copy of the truth drifts.
3. *A foreign key that should be 1:1 — what else does it need?* → `UNIQUE` (or make it the primary key); otherwise it is 1:N.
4. *Where does the agreed rate, a seat's price, or a medicine's dose go?* → On the junction row: it describes the pairing, not either side.
5. *Hotel search counts per night; Airbnb uses an exclusion constraint. Why?* → A room type has many interchangeable units; a listing is one unit.
6. *When is `target_type + target_id` acceptable?* → When target kinds change often and you accept checking integrity outside the database.
7. *The one-sentence recipe for "no double booking"?* → Make the conflicting state impossible to store (unique / exclusion / check constraint) and claim the item with one guarded statement in a short transaction.

Chapter 17 TL;DR

- Use one template every time: **requirements & questions → entities, cardinality & participation → ER diagram → tables & constraints → key queries → indexes → design review → scaling** (the review checks normal forms, names deliberate redundancy, and lists what the design cannot do).
- Put business rules **in the schema** where possible: `CHECK` for ranges and state/column consistency, partial unique indexes for "one open X", exclusion constraints for "no overlapping X", unique keys for "once per user".
- Separate **type from instance** (titles/copies, room types/rooms, seats/show seats) and **current state from history** (status column + events or ledger).
- **Snapshot** values that must not change later (prices paid, addresses shipped to); **soft-delete** rows that history still references.
- Denormalize **deliberately** (counters, totals, inventory per night) — update in the same transaction and keep a reconciliation query.
- Concurrency for scarce items: a **guarded** `UPDATE … WHERE status = 'free'` in a short transaction is enough — demonstrated on two live sessions; `SKIP LOCKED` for "any available".
- At scale, the relational core stays for money, bookings and identity; media goes to object storage + CDN, feeds and counters to caches, events to logs, search to a search engine, and short-lived data to TTL stores or dropped time partitions.

Next: Chapter 18 — SQL interview problems, part 1, which practises the query skills from Chapters 2–8 on interview-style problems.

Chapter 18 — SQL Interview Problems I: Method and 42 Warm-Up to Medium Problems

Chapters 2–8 taught the tools one at a time; interviews mix them. Given a short business question and a schema, you decide what to filter, group and join and what could go wrong, saying it aloud as you type. This chapter gives the method, then 42 practice problems (easy to lower-medium) grouped by the **pattern** they train, so you learn to spot patterns rather than memorise answers.

#	Section	Patterns
18.1	How to approach a SQL interview question	clarify → sample → build → edge cases → verify
18.2	Filtering (P1–P8)	`WHERE`, `AND` / `OR`, `%`, `IS DISTINCT FROM`, `char_length`, regex, row-level `CASE`
18.3	Aggregation (P9–P16)	`GROUP BY`, `HAVING`, `MIN` / `MAX` / `COUNT`, `string_agg`
18.4	Joins (P17–P22)	inner, self, cross + left, "top 1 with ties"
18.5	Anti-joins (P23–P26)	`NOT EXISTS`, `LEFT JOIN … IS NULL`, the `NOT IN` + NULL trap
18.6	Conditional aggregation (P27–P28)	`FILTER`, `SUM(CASE …)`, `UNION ALL` then pivot
18.7	Date logic (P29–P33)	half-open month ranges, date arithmetic, gaps and islands (runs of consecutive values), "value as of a date"
18.8	Ratios and percentages (P34–P38)	integer division, `NULLIF`, weighted averages, retention (share of users who come back)
18.9	Window and ranking basics (P39–P42)	second highest, first row per group, running total, compare to group average
18.10	Pattern table	every problem → pattern → key SQL feature

The harder problems (Nth highest, top-N per group, streaks, pivots, trips and users) are in Chapter 19.

How to run the examples. Each problem has a file in `code/` that creates small invented tables and runs the solution plus an edge case or alternative, each on a fresh PostgreSQL 14 database. Output is real `psql` output; `(null)` displays `NULL`.

```
bash tools_run_sql.sh sql '18-*'                    # all 42 files, fresh DB each
bash tools_pg.sh psql -d postgres -c 'CREATE DATABASE scratch'
bash tools_pg.sh psql -d scratch -f sql/chapters/18-sql-interview-problems-1/code/26_reps_without_red.sql
```

Every statement, dataset and explanation is written for this book; the problems are classic interview exercises, many also on public practice sites under similar names (the question bank maps LeetCode titles to them).

18.1 How to approach a SQL interview question

What it is

A repeatable routine for turning a vague question ("which sellers were idle last year?") into a correct query while the interviewer watches — dull steps that catch the mistakes nerves cause.

Why it matters

Most wrong answers come from **misreading the question** or **missing an edge case** (an empty table, a NULL), not syntax: counting rows instead of distinct users, losing zero-activity people, integer division

turning 0.67 into 0, `NOT IN` silently returning nothing. Interviewers also score your *process*: questions asked, ideas tested on a few rows, trade-offs explained.

How it works — the five steps



1. **Clarify.** Repeat the question in your own words, then ask:
   - What is one **row** of each table (the *grain*)? What is the primary key? Can there be **duplicate rows**?
   - Which columns can be **NULL**? What should happen to rows with NULLs?
   - Should entities with **no activity** appear (with 0) or be left out?
   - How should **ties** be handled: return all of them, or any one?
   - Output: which columns, what names, what **order**, how much **rounding**?
   - Are dates **inclusive**? Which time zone? Is "consecutive" by calendar day or by record?
2. **Sketch sample data.** Write 3–5 rows, including one awkward row (a NULL, a duplicate, a tie), plus the output you expect. This settles misunderstandings before you write code.
3. **Build in the logical order** the database evaluates (§4.5): `FROM` / `JOIN` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `ORDER BY` → `LIMIT`, with CTEs (`WITH step1 AS (…)`, Chapter 7) so each step can be checked alone.
4. **Attack it with edge cases**, using the checklist below.
5. **Verify and discuss.** Run the query against your sample (in your head or for real), then mention a helpful index and one alternative formulation.

The edge-case checklist

Risk	Typical symptom	Usual fix
NULL in a comparison	rows vanish from `<>`, `NOT IN` returns nothing	`IS DISTINCT FROM`, `NOT EXISTS`, `COALESCE`
Entities with no rows	people with zero orders missing	`LEFT JOIN` from the entity table, `count(col)` not `count(*)`
Duplicates	counts too high, repeated ids	`DISTINCT`, `count(DISTINCT …)`, check the join's grain
Join fan-out (rows multiply)	sums doubled after joining two child tables	aggregate before joining, or `count(DISTINCT)`
Ties	"the winner" is really two people	`RANK` / `DENSE_RANK` (ties share a rank), `FETCH FIRST … WITH TIES`
Integer division	`1/3 = 0`	multiply by `100.0` or cast to `numeric` first
Division by zero	runtime error	`NULLIF(denominator, 0)`
Empty result vs NULL	"return NULL if none" gives zero rows	wrap in an aggregate or a scalar subquery
Date boundaries	last day of the month (or 29 Feb) lost	half-open ranges: `>= start AND < next_start`

Common mistakes

- Starting to type before knowing the grain of each table.
- `SELECT`ing a column that is neither grouped nor aggregated (PostgreSQL rejects it; MySQL with the `ONLY_FULL_GROUP_BY` setting off returns an arbitrary value).
- Filtering the right-hand table of a `LEFT JOIN` in `WHERE`, which turns it back into an inner join (Chapter 5).
- Leaving `ORDER BY` out when the question asks for an order. Without it, SQL guarantees **no** order.

In the interview

> **30-second answer to "how do you approach a SQL problem?"** "I first pin down the grain and keys of each table, what NULLs and duplicates mean, and whether zero-activity rows and ties should appear. I sketch a few rows with the expected output, then build the query in evaluation order (joins, filters, grouping, having, select, sort) using CTEs so I can check each step. Finally I test NULLs, duplicates, ties, empty groups and date boundaries, and mention an index or an alternative."

- *"Your query returns nothing for a new customer — is that right?"* It depends on the spec. If they should appear with 0, start from the customer table with a `LEFT JOIN` and `COALESCE` the aggregate.
- *"Would this be fast on 100 M rows?"* A matching index on filter/join columns, **sargable** predicates (no function around the indexed column — `WHERE year(d) = 2020` works but can't use an index on `d`; write `d >= '2020-01-01' AND d < '2021-01-01'`), and **pre-aggregation** (summarise before joining, or store summaries) — §12.13.
- *Why ask about the grain first?* It says what one row means: count rows or distinct ids, and whether a join can multiply rows.
- *`count(*)` vs `count(col)` after a `LEFT JOIN`?* `count(*)` counts the NULL-padded row as 1; `count(col)` as 0.

TL;DR

Clarify (grain, NULLs, zeros, ties, output), sketch sample rows, build in evaluation order with CTEs, attack with the edge-case checklist, then verify and discuss performance.

18.2 Filtering problems (P1–P8)

The simplest pattern: keep or drop **individual rows** with `WHERE`. The traps are `NULL` (three-valued logic: comparisons with NULL give `UNKNOWN`, and `WHERE` keeps only `TRUE`), boundaries (`>` vs `>=`) and string rules. Background: Chapter 2 and Chapter 3.

P1 · Big cities — `OR` filter

Problem. A `city` table stores name, country, area in km² and population. List the cities that are *either* larger than 3,000 km² *or* home to more than 5 million people.

Data (`01_big_cities.sql`): six invented towns. Coralbay sits exactly on both limits (3000 km², 5,000,000). Eastvale and Fernhill have an unknown (NULL) area.

```
SELECT name FROM city
WHERE area_km2 > 3000 OR population > 5000000
ORDER BY name;
```

Output:

```
   name
-----------
 Aldport
 Brimstead
 Eastvale
(3 rows)
```

Why it works. Each row is tested on its own. Coralbay fails because "more than" is strict. Eastvale's area test gives `UNKNOWN`, but `UNKNOWN OR TRUE` is `TRUE`, so its population is enough. Fernhill gives `UNKNOWN OR FALSE` = `UNKNOWN`, and `WHERE` keeps only `TRUE`.

Alternative. `SELECT … WHERE area_km2 > 3000 UNION SELECT … WHERE population > 5000000` returns the same set; on some engines (notably older MySQL) each branch then uses its own index. PostgreSQL can instead combine two indexes with a *BitmapOr* for the plain `OR`.

P2 · Odd-numbered, not-boring movies — modulo and `<>`

Problem. From `movie(id, title, description, rating)`, return movies whose id is odd and whose description isn't `'boring'`, highest rating first.

Data (`02_odd_movies.sql`): ids 1–5 and 7, one of them described as boring, one with a NULL description, plus id −3.

```
SELECT * FROM movie
WHERE id % 2 = 1 AND description <> 'boring'
ORDER BY rating DESC;
```

Output:

```
 id |    title    | description | rating
----+-------------+-------------+--------
  5 | Deep Blue   | documentary |   8.90
  1 | Night Train | thriller    |   8.10
(2 rows)
```

Why it works. `%` is the remainder. Odd ids leave remainder 1. The `<>` drops the boring one.

Edge cases. Two rows went missing silently. In PostgreSQL `-3 % 2` is `-1` (the sign of the dividend), so `= 1` misses negative odd ids. And `NULL <> 'boring'` is `UNKNOWN`. If an unknown description should count as "not boring", say so explicitly:

```
SELECT * FROM movie
WHERE id % 2 <> 0 AND description IS DISTINCT FROM 'boring'
ORDER BY rating DESC;
```

```
 id |    title    | description | rating
----+-------------+-------------+--------
  5 | Deep Blue   | documentary |   8.90
  1 | Night Train | thriller    |   8.10
 -3 | Rewind      | comedy      |   6.00
  7 | Untitled    | (null)      |   5.00
(4 rows)
```

MySQL has `MOD(id, 2)` as well as `%`; its NULL-safe equality is `<=>`, so "is distinct from" is `NOT (a <=> b)`.

P3 · Green products — `AND` on flag columns

Problem. `inventory(product_id, organic, compostable)` holds `'Y'`/`'N'` flags. Return the products that have both flags set.

```
SELECT product_id FROM inventory
WHERE organic = 'Y' AND compostable = 'Y'
ORDER BY product_id;
```

```
 product_id
------------
         10
         13
(2 rows)
```

Why it works. Both conditions must be `TRUE` for the same row.

Edge case / design note. Flags stored as text invite dirty values such as `'y'` or `' Y'`. The file puts a `CHECK (organic IN ('Y','N'))` on each column, and inserting `'y'` fails with *violates check constraint "inventory_organic_check"*. In PostgreSQL a `boolean NOT NULL` column is cleaner still: `WHERE organic AND compostable`. MySQL's source table often uses `ENUM('Y','N')`.

P4 · Everyone outside department 3 — NULL-safe "not equal"

Problem. From `staff(id, name, dept_id)`, list people who are not in department 3. One person has no department yet.

```
SELECT name FROM staff WHERE dept_id <> 3 ORDER BY name;              -- naive
```

```
 name
------
 Ana
 Chen
(2 rows)
```

```
SELECT name FROM staff WHERE dept_id IS DISTINCT FROM 3 ORDER BY name; -- NULL-safe
```

```
 name
------
 Ana
 Chen
 Dara
(3 rows)
```

Why it matters. `NULL <> 3` is `UNKNOWN`, so Dara disappears from the naive version. Whether Dara *should* appear is a clarifying question. Most people would say "not in department 3" includes "in no department". `IS DISTINCT FROM` treats NULL as an ordinary comparable value. The portable spelling is `dept_id <> 3 OR dept_id IS NULL`.

P5 · Self-assigned tasks — comparing two columns, removing duplicates

Problem. `task_log(task_id, assigner_id, assignee_id, done_on)` may contain duplicate rows. Return, in ascending order, each person who completed at least one task they had given to themselves.

```
SELECT DISTINCT assigner_id AS employee_id
FROM task_log
WHERE assigner_id = assignee_id
ORDER BY employee_id;
```

```
 employee_id
-------------
           7
           9
(2 rows)
```

Why it works. The `WHERE` compares two columns of the same row. `DISTINCT` collapses repeats: without it the output is `7, 7, 9, 9` (one duplicate row for person 7, and two self-assigned tasks for person 9). `GROUP BY assigner_id` gives the same result. "At least one" nearly always means you need `DISTINCT`, `GROUP BY` or `EXISTS`.

P6 · Long comments — length in characters

Problem. `comment(id, body)`: return ids of comments longer than 100 characters (spaces and punctuation count).

```
SELECT id FROM comment WHERE char_length(body) > 100 ORDER BY id;
```

```
 id
----
  2
(1 row)
```

Why it works / edge cases. Comment 3 is exactly 100 characters, so not "more than" 100. Comment 4 is 60 copies of `é`: 60 **characters** but 120 **bytes** in UTF-8 (accented and non-Latin characters take 2–4 bytes).

```sql
SELECT id, char_length(body) AS chars, octet_length(body) AS bytes FROM comment ORDER BY id;
```

```
 id | chars | bytes
----+-------+-------
  1 |    10 |    10
  2 |   102 |   102
  3 |   100 |   100
  4 |    60 |   120
(4 rows)
```

Engine	Characters	Bytes
PostgreSQL	`char_length(s)`, `length(s)`	`octet_length(s)`
MySQL	`CHAR_LENGTH(s)`	`LENGTH(s)` (a classic trap)
SQL Server	`LEN(s)` (ignores trailing spaces)	`DATALENGTH(s)`
SQLite	`length(s)`	`length(CAST(s AS BLOB))`

P7 · Valid company e-mails — regular expressions

Problem. `employee(id, name, email)`. An address is valid when the part before `@` starts with a letter and continues with letters, digits, `_`, `.` or `-`, and the domain is exactly `northwind.dev`. Return the valid rows.

Data (`07_valid_emails.sql`): eight addresses. They include one starting with `_`, one starting with a digit, `northwindXdev`, `northwind.dev.evil.com`, an upper-case domain and one containing `#`.

```sql
SELECT id, name, email FROM employee
WHERE email ~ '^[A-Za-z][A-Za-z0-9_.-]*@northwind\.dev$'
ORDER BY id;
```

```
 id | name |        email
----+------+---------------------
  1 | Ana  | ana.k@northwind.dev
  5 | Eve  | Eve-2@northwind.dev
(2 rows)
```

Why it works. `~` tests a string against a **regular expression** (a text-pattern language). `^` and `$` **anchor** the pattern to the start and end, so it must match the whole string. `[A-Za-z]` forces a letter first, and the class after it allows the other characters (inside `[...]` a `.` is literal, and a `-` placed last is literal). `\.` is a literal dot.

Edge cases. Remove the anchors and the backslash, and six rows match. That includes `dee@northwindXdev`, because an unescaped `.` matches any character, and `fay@northwind.dev.evil.com`, because the end is not anchored. `~` is case-sensitive, so `gus@NORTHWIND.DEV` fails. Use `~*` if the domain should be case-insensitive (it then matches ids 1, 5, 7). Dialects: MySQL `REGEXP` / `REGEXP_LIKE()`, Oracle `REGEXP_LIKE()`. SQL Server gained `REGEXP_LIKE()` only in SQL Server 2025 (database compatibility level 170) and Azure SQL; older versions fall back to `LIKE` patterns. In MySQL string literals the backslash itself must be doubled (`'\\.'`).

P8 · Triangle test — a `CASE` per row

Problem. `segment(a, b, c)` holds three lengths per row. Add a column saying `'Yes'` if they can form a triangle and `'No'` otherwise.

```sql
SELECT a, b, c,
       CASE WHEN a + b > c AND a + c > b AND b + c > a THEN 'Yes' ELSE 'No' END AS triangle
FROM segment;
```

```
 a  |   b    | c | triangle
----+--------+---+----------
  3 |      4 | 5 | Yes
 10 |      2 | 3 | No
  5 |      5 | 5 | Yes
  1 |      2 | 3 | No
  7 | (null) | 4 | No
(5 rows)
```

Why it works. The **triangle inequality** says every pair of sides must be longer than the third side. `(1,2,3)` is *degenerate*: it is flat, because 1 + 2 is not *greater* than 3.

Alternative / edge case. It is enough to check the longest side: `2 * greatest(a,b,c) < a + b + c`. A missing side makes the condition `UNKNOWN`, and `CASE` falls through to `ELSE 'No'`, which quietly claims something we don't know. Adding `WHEN a IS NULL OR b IS NULL OR c IS NULL THEN 'Unknown'` makes that row say `Unknown` (see the file). PostgreSQL's `greatest()` ignores NULLs, whereas MySQL's returns NULL if any argument is NULL.

18.3 Aggregation problems (P9–P16)

Collapse many rows into one per group. The traps: which rows exist before grouping, how aggregates treat NULLs, and what an **empty** input returns. Background: Chapter 4.

P9 · Largest number that appears once — `HAVING` then `MAX`

Problem. `nums(n)` has no key and contains repeats. Return the biggest value that occurs exactly once, or NULL if no value is unique.

```sql
SELECT max(n) AS num
FROM (SELECT n FROM nums GROUP BY n HAVING count(*) = 1) AS singles;
```

```
 num
-----
   5
(1 row)
```

Why it works. The inner query groups equal values and keeps groups of size 1: {1, 4, 5}. The outer `max` picks 5.

Edge case — empty input. After inserting 1, 4, 5 again, no value is unique. An aggregate over zero rows still returns **one row** (`max` → NULL), which is exactly what "report NULL" asks for:

```
  num
--------
 (null)
(1 row)
```

The tempting `… HAVING count(*) = 1 ORDER BY n DESC LIMIT 1` returns **0 rows** in the same situation. That is a different answer, and graders that expect a NULL row mark it wrong.

P10 · Units sold per product — `SUM` with `GROUP BY`

Problem. `sale(sale_id, product_id, yr, qty, unit_price)` references `product`. Report total quantity per product.

```sql
SELECT product_id, sum(qty) AS total_quantity
FROM sale
GROUP BY product_id
ORDER BY product_id;
```

```
 product_id | total_quantity
------------+----------------
        100 |             25
        200 |             15
(2 rows)
```

Edge case — zero sellers. Product 300 never sold, so it has no group. If the business wants it listed, start from `product`:

```sql
SELECT p.product_id, COALESCE(sum(s.qty), 0) AS total_quantity
FROM product p LEFT JOIN sale s USING (product_id)
GROUP BY p.product_id ORDER BY p.product_id;
```

```
 product_id | total_quantity
------------+----------------
        100 |             25
        200 |             15
        300 |              0
(3 rows)
```

`sum` over only NULLs is NULL, not 0, which is why the `COALESCE` is needed.

P11 · First login per player — `MIN` on dates

Problem. `activity(player_id, device_id, played_on, games)` has one row per player per day. Give each player's first login date.

```sql
SELECT player_id, min(played_on) AS first_login
FROM activity GROUP BY player_id ORDER BY player_id;
```

```
 player_id | first_login
-----------+-------------
         1 | 2024-02-27
         2 | 2024-06-25
         3 | 2024-03-02
(3 rows)
```

Why it works. `min` works on anything sortable. Dates sort chronologically, and player 1's earliest row was inserted last, which makes no difference. **Follow-up trap:** "and the device used that day?" cannot be answered by adding `device_id` to this query, because it is neither grouped nor aggregated. That is P40.

P12 · Follower count — `COUNT(*)` vs `COUNT(col)`

Problem. `follows(user_id, follower_id)` has one row per "follower follows user". Count followers per user, ordered by user id.

```sql
SELECT user_id, count(*) AS followers_count
FROM follows GROUP BY user_id ORDER BY user_id;
```

```
 user_id | followers_count
---------+-----------------
       1 |               2
       2 |               1
       3 |               3
(3 rows)
```

Edge case — users with zero followers. User 4 exists in `app_user` but nobody follows them. Joining from the user table fixes the missing row, but only if you count a column from the right-hand side:

```sql
SELECT u.user_id, count(f.follower_id) AS good, count(*) AS wrong
FROM app_user u LEFT JOIN follows f USING (user_id)
GROUP BY u.user_id ORDER BY u.user_id;
```

```
 user_id | good | wrong
---------+------+-------
       1 |    2 |     2
       2 |    1 |     1
       3 |    3 |     3
       4 |    0 |     1
(4 rows)
```

The LEFT JOIN produces one NULL-padded row for user 4. `count(*)` counts it, while `count(f.follower_id)` skips the NULL.

P13 · When was each bike last used — `MAX` and sorting by an aggregate

Problem. `ride(ride_id, bike_no, started_at, ended_at)`. Report the latest end time per bike, with the most recently used bike first.

```sql
SELECT bike_no, max(ended_at) AS last_used
FROM ride GROUP BY bike_no ORDER BY last_used DESC;
```

```
 bike_no |      last_used
---------+---------------------
 B-10    | 2024-05-03 17:45:00
 B-20    | 2024-05-01 09:50:00
 B-30    | 2024-04-28 12:20:00
(3 rows)
```

Edge case. Bike B-20 is out on a ride right now (`ended_at` is NULL). `max` skips NULLs, so B-20 reports its previous ride. Ask whether "in use now" should count as most recent. The file adds `bool_or(ended_at IS NULL) AS out_now` to flag it (`t` for B-20). Also note that in PostgreSQL `ORDER BY … DESC` puts NULLs **first**. Add `NULLS LAST` if some bike could have only unfinished rides.

P14 · Missed appointments — filter, then group

Problem. `appointment(id, patient_id, doctor_id, appt_date, status)` with status Completed / Cancelled / No-Show. Count no-shows per patient and leave out patients who never missed one.

```sql
SELECT patient_id, count(*) AS no_shows
FROM appointment
WHERE status = 'No-Show'
GROUP BY patient_id
ORDER BY patient_id;
```

```
 patient_id | no_shows
------------+----------
         11 |        2
         12 |        2
(2 rows)
```

Why it works. `WHERE` removes the other statuses *before* grouping, so patient 13 (only a cancellation) never forms a group. This matches the spec that such patients are left out.

Alternative. When you need several counts at once, keep all rows and count conditionally: `count(*) FILTER (WHERE status = 'No-Show')`, with `HAVING` that count `> 0`. In the same pass it can also report `count(*) AS booked` (patient 11: 2 of 3). That idea is developed in 18.6.

P15 · Products sold per day — `COUNT(DISTINCT)` and `string_agg`

Problem. `sales_log(sold_on, product)` has no key and contains duplicates. For each date, return how many *different* products were sold and a comma-separated, alphabetical list of them, ordered by date.

```
SELECT sold_on,
       count(DISTINCT product)                          AS num_sold,
       string_agg(DISTINCT product, ',' ORDER BY product) AS products
FROM sales_log
GROUP BY sold_on
ORDER BY sold_on;
```

```
  sold_on   | num_sold |   products
------------+----------+-------------
 2024-06-01 |        2 | Ink,Pen
 2024-06-02 |        1 | Stapler
 2024-06-03 |        3 | Clip,Ink,Pen
(3 rows)
```

Why it works / edge cases. `DISTINCT` inside both aggregates removes the second "Pen" on 1 June. A NULL product on 3 June is ignored by both aggregates. With `DISTINCT`, PostgreSQL requires the `ORDER BY` inside `string_agg` to use the aggregated expression itself.

Engine	Ordered, de-duplicated list
PostgreSQL	`string_agg(DISTINCT p, ',' ORDER BY p)`
MySQL	`GROUP_CONCAT(DISTINCT p ORDER BY p SEPARATOR ',')` (output capped by `group_concat_max_len`, default 1024 bytes)
SQL Server 2017+	`STRING_AGG(p, ',') WITHIN GROUP (ORDER BY p)`: no `DISTINCT`, so de-duplicate in a subquery first
SQLite	`group_concat(DISTINCT p)`: order not guaranteed before 3.44

P16 · Customer basket summary — the same pattern, per customer

Problem. `purchase(bought_on, customer_id, product)` may repeat. For each customer, show the number of distinct products and their alphabetical list.

```
SELECT customer_id,
       count(DISTINCT product) AS num_products,
       string_agg(DISTINCT product, ',' ORDER BY product) AS products
FROM purchase GROUP BY customer_id ORDER BY customer_id;
```

```
 customer_id | num_products |    products
-------------+--------------+----------------
           1 |            2 | Mug,Tea
           2 |            1 | Coffee
           3 |            3 | Coffee,Mug,Tea
(3 rows)
```

Alternative. If the result goes to application code, return a real array rather than a string you'll have to split again: `array_agg(DISTINCT product ORDER BY product)` → `{Mug,Tea}`. The lesson of P15 and P16 is that "one row per X with a list of Y" is a `GROUP BY X` plus a string or array aggregate.

18.4 Join problems (P17–P22)

Combine rows from two tables, or from one table with itself. The traps: rows that have no partner, rows that get multiplied, and ties. Background: Chapter 5.

P17 · Product name for every sale — basic inner join

Problem. `sale(sale_id, product_id, yr, qty, unit_price)` and `product(product_id, name)`. For every sale, show the product name, year and unit price, ordered by year.

```sql
SELECT p.name AS product_name, s.yr AS year, s.unit_price AS price
FROM sale s
JOIN product p ON p.product_id = s.product_id
ORDER BY s.yr, s.sale_id;
```

```
 product_name | year | price
--------------+------+-------
 Kettle       | 2022 |    27
 Kettle       | 2023 |    25
 Toaster      | 2023 |    40
(3 rows)
```

Why it works. Each sale finds its single matching product (`product_id` is the product PK), so the join neither loses nor duplicates sales. **Edge case:** with a foreign key every sale has a product; without one an **orphan** sale (no parent row) vanishes from an inner join — use `LEFT JOIN` to show a NULL name. Break sort ties (`, s.sale_id`) so the order is **deterministic** (same every run).

P18 · Average process time per machine — self-join start to end

Problem. `machine_event(machine_id, process_id, kind, ts)` logs a `'start'` row and an `'end'` row per process. For each machine, report the average of (end − start), rounded to 3 decimals.

Data (`18_process_time.sql`): three machines with two finished processes each; machine 2 also has a third process that never ended.

```sql
SELECT s.machine_id, round(avg(e.ts - s.ts), 3) AS processing_time
FROM machine_event s
JOIN machine_event e
  ON e.machine_id = s.machine_id AND e.process_id = s.process_id
 AND s.kind = 'start' AND e.kind = 'end'
GROUP BY s.machine_id
ORDER BY s.machine_id;
```

```
 machine_id | processing_time
------------+-----------------
          0 |           0.894
          1 |           0.995
          2 |           1.456
(3 rows)
```

Why it works. A **self-join**: aliases `s` and `e` give two views of the same table. The join puts each process's start and end on one row, and `avg` then averages the differences. The unfinished process has no `end` row, so the inner join drops it.

Alternative / edge case. You can pivot each process to one row with `max(ts) FILTER (WHERE kind = 'start')` and `… 'end'`, and then average the differences. That gives the same numbers, because the unfinished process's difference is NULL and `avg` skips it. A tempting shortcut is `(sum of end ts − sum of start ts) / count(DISTINCT process_id)`, and it would be wrong here, because it adds the unfinished start into the sum. **PostgreSQL note:** `round(x, 3)` exists only for `numeric` (here `ts` is `numeric`); for `double precision` write `round((…)::numeric, 3)`.

P19 · Fiction books per author — join, filter, group, two-key sort

Problem. `book(book_id, title, genre)` and `book_author(author_id, book_id, author_name)`. Count each author's Fiction titles, leave out authors with none, and sort by count (highest first) and then by name.

```sql
SELECT a.author_id, a.author_name, count(*) AS fiction_books_count
FROM book_author a
JOIN book b ON b.book_id = a.book_id
WHERE b.genre = 'Fiction'
GROUP BY a.author_id, a.author_name
ORDER BY fiction_books_count DESC, a.author_name, a.author_id;
```

```
 author_id | author_name | fiction_books_count
-----------+-------------+---------------------
        10 | Iris Vale   |                   2
        13 | Iris Vale   |                   1
        11 | Omar Reyes  |                   1
(3 rows)
```

Why it works. The inner join plus `WHERE` keeps only fiction rows, so authors with zero fiction form no group, which is what the spec wants. **Edge case — duplicate names:** two different people are called Iris Vale. Grouping by `author_name` alone merges them into one row showing `3`, which is wrong. Always group by the **key** and add the name for display.

P20 · Exam sittings per student and subject — CROSS JOIN + LEFT JOIN

Problem. `student`, `subject`, and `exam_sitting(student_id, subject_name)` (duplicates allowed, one row per sitting). For every student × subject pair, report how many times the student sat that exam (0 included), ordered by student id and subject.

```sql
SELECT st.student_id, st.student_name, su.subject_name,
       count(e.subject_name) AS attended_exams
FROM student st
CROSS JOIN subject su
LEFT JOIN exam_sitting e
       ON e.student_id = st.student_id AND e.subject_name = su.subject_name
GROUP BY st.student_id, st.student_name, su.subject_name
ORDER BY st.student_id, su.subject_name;
```

```
 student_id | student_name | subject_name | attended_exams
------------+--------------+--------------+----------------
          1 | Noor         | Art          |              0
          1 | Noor         | Math         |              3
          1 | Noor         | Physics      |              1
          2 | Kofi         | Art          |              1
          2 | Kofi         | Math         |              1
          2 | Kofi         | Physics      |              0
          6 | Lin          | Art          |              0
          6 | Lin          | Math         |              0
          6 | Lin          | Physics      |              0
(9 rows)
```

Why it works. "Every pair, including zeros" means you must *create* the pairs, because the **fact table** (recording events — sittings) only has pairs that happened. `CROSS JOIN` builds the full 3 × 3 grid, the `LEFT JOIN` attaches matching sittings (both columns in `ON`), and `count(e.subject_name)` turns missing matches into 0. This **grid + left join** pattern also fills missing days in time series (with `generate_series`).

P21 · Election winner — ORDER BY an aggregate, and ties

Problem. `candidate(id, name)` and `vote(id, candidate_id)` (one row per ballot). Return the name of the candidate with the most votes (the data guarantees a single winner).

```sql
SELECT c.name
FROM candidate c
JOIN vote v ON v.candidate_id = c.id
GROUP BY c.id, c.name
ORDER BY count(*) DESC
LIMIT 1;
```

```
 name
-------
 Tariq
(1 row)
```

Why it works. Group ballots per candidate, sort by the count, and keep the top row. Grouping by `c.id` (not just name) keeps namesakes apart.

Edge case — ties. The guarantee rarely holds in real life. After one more vote for Victor, both have 3 and `LIMIT 1` would pick one arbitrarily. The standard `FETCH FIRST 1 ROWS WITH TIES` (PostgreSQL 13+, Oracle 12c+; SQL Server uses `TOP 1 WITH TIES`) returns every row tied with the last one kept:

```
 name  | votes
-------+-------
 Tariq |     3
 Victor |     3
(2 rows)
```

MySQL has no `WITH TIES`. Use `RANK() OVER (ORDER BY count(*) DESC)` in a subquery and keep rank 1 (Chapter 8).

P22 · "Middle" users in a follow graph — self-join or EXISTS

Problem. `follow(followee, follower)`: the follower follows the followee, and nobody follows themselves. Find users who follow at least one person *and* have at least one follower. Report each one's follower count, alphabetically.

Data: amy←bo, bo←cy, bo←dee, cy←eve, cy←bo, dee←amy (read "amy is followed by bo").

```sql
SELECT f.followee AS follower, count(*) AS num
FROM follow f
WHERE EXISTS (SELECT 1 FROM follow g WHERE g.follower = f.followee)
GROUP BY f.followee
ORDER BY f.followee;
```

```
 follower | num
----------+-----
 amy      |   1
 bo       |   2
 cy       |   2
 dee      |   1
(4 rows)
```

Why it works. Each row of `f` is one follower of `f.followee`. `EXISTS` keeps the row only if that same user also appears as somebody's *follower*. Eve is followed by no one, so she is excluded.

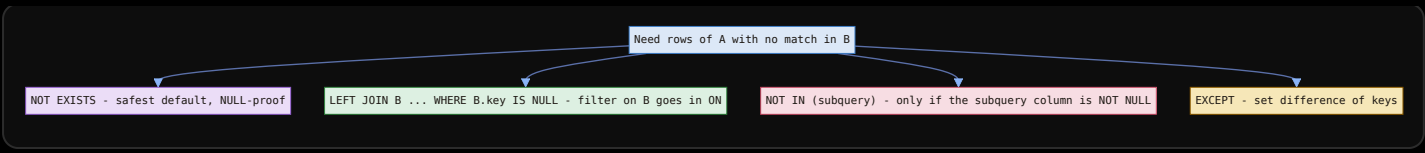
Alternative — and the fan-out trap. The self-join `follow a JOIN follow b ON b.followee = a.follower` also works. However, each of bo's followers pairs with each person bo follows, so the rows multiply:

```
 follower | num | inflated
----------+-----+----------
 amy      |   1 |        1
 bo       |   2 |        4
 cy       |   2 |        2
 dee      |   1 |        1
(4 rows)
```

Only `count(DISTINCT b.follower)` is correct there. When a join can match many rows on **both** sides, de-duplicate or use `EXISTS` (a *semi-join*, which never multiplies).

18.5 Anti-join problems (P23–P26)

"Find X that has **no** Y" is an **anti-join**. There are three common spellings. They behave identically, except that `NOT IN` breaks when the subquery contains a NULL. Background: Chapter 5 §5.8 and Chapter 6 §6.4.



P23 · Sellers with no 2020 sales — NOT EXISTS with a date range

Problem. `seller(seller_id, seller_name)` and `orders(order_id, sold_on, cost, customer_id, seller_id)`. List sellers who made no sale at all in 2020, by name.

```
SELECT s.seller_name
FROM seller s
WHERE NOT EXISTS (
  SELECT 1 FROM orders o
  WHERE o.seller_id = s.seller_id
    AND o.sold_on >= DATE '2020-01-01' AND o.sold_on < DATE '2021-01-01')
ORDER BY s.seller_name;
```

```
 seller_name
-------------
 Reed Ltd
 Tern LLC
(2 rows)
```

Why it works. For each seller, the subquery asks "is there any 2020 order?". `NOT EXISTS` keeps sellers where the answer is no: Reed sold only in 2019, and Tern never sold. Quill's 31 December order counts, and Sable's 1 January 2021 order doesn't. The **half-open** range `>= 1 Jan AND < next 1 Jan` is safe for `date` and `timestamp` alike. `BETWEEN '2020-01-01' AND '2020-12-31'` would miss 31 December after midnight if the column were a timestamp. PostgreSQL executes `NOT EXISTS` as a hash or merge **anti-join** (§12.15); an index on `orders(seller_id, sold_on)` makes each probe cheap.

P24 · Dormant accounts — LEFT JOIN … IS NULL

Problem. `account(account_id, account_name)` and `txn(txn_id, account_id, txn_date)`. Return accounts that made no transaction in 2020, by name.

```
SELECT a.account_name
FROM account a
LEFT JOIN txn t
       ON t.account_id = a.account_id
      AND t.txn_date >= DATE '2020-01-01' AND t.txn_date < DATE '2021-01-01'
WHERE t.txn_id IS NULL
ORDER BY a.account_name;
```

```
 account_name
--------------
 Juniper
 Kestrel
(2 rows)
```

Why it works. The LEFT JOIN attaches only *2020* transactions. Accounts with none get a NULL-padded row, and `WHERE t.txn_id IS NULL` keeps exactly those. Test a column that can't be NULL in a real match, such as the primary key.

The classic bug. Move the date test into `WHERE` and you ask for rows where `t` is NULL **and** `t.txn_date` is in 2020. No row can be both, so the result is `(0 rows)`. **Alternative:** a set difference of ids, `SELECT account_id FROM account EXCEPT SELECT account_id FROM txn WHERE …2020…`, returns the same Juniper and Kestrel.

P25 · Sessions with no ads — anti-join on a range condition

Problem. `playback(session_id, customer_id, start_t, end_t)` and `ad(ad_id, customer_id, shown_at)`. A session is ad-free if that customer was shown no ad between `start_t` and `end_t` (inclusive). List ad-free sessions.

Data: customer 1 has sessions [1,5] and [15,23] and ads at 5 and 9. Customer 2 has [10,12] and [17,28] and ads at 17 and 20. Customer 3 has [2,8] and an ad at 30.

```
SELECT p.session_id
FROM playback p
WHERE NOT EXISTS (
  SELECT 1 FROM ad a
  WHERE a.customer_id = p.customer_id
    AND a.shown_at BETWEEN p.start_t AND p.end_t)
ORDER BY p.session_id;
```

```
 session_id
------------
          2
          3
          5
(3 rows)
```

Why it works / edge cases. The match condition is not plain equality. It is "same customer **and** timestamp inside the interval". Session 1 is excluded because the ad at 5 sits exactly on its (inclusive) end, and session 4 because of the ad at 17. The ad at 9 falls between customer 1's sessions, so it affects neither. Forgetting `a.customer_id = p.customer_id` would let another viewer's ads disqualify a session.

P26 · Reps who never sold to RED — NOT EXISTS vs NOT IN

Problem. `sales_rep(rep_id, name)`, `company(com_id, name, city)`, `orders(order_id, order_date, com_id, rep_id, amount)`. Return the reps with no order from the company named `'RED'`.

```
SELECT r.name
FROM sales_rep r
WHERE NOT EXISTS (
  SELECT 1 FROM orders o JOIN company c ON c.com_id = o.com_id
  WHERE o.rep_id = r.rep_id AND c.name = 'RED')
ORDER BY r.name;
```

```
 name
------
 Kai
 Lu
 Ned
(3 rows)
```

Why it works. The subquery joins orders to companies to filter by company *name*, and is **correlated** on the rep (it refers to the outer row `r`).

Edge case — the NOT IN trap. `WHERE r.rep_id NOT IN (SELECT o.rep_id FROM … WHERE c.name = 'RED')` returns the same three names. Then we insert one RED order with no rep (`rep_id` NULL), and the `NOT IN` version returns:

```
 name
------
(0 rows)
```

`x NOT IN (1, 4, NULL)` means `x <> 1 AND x <> 4 AND x <> NULL`. The last test is `UNKNOWN` for every `x`, so no row qualifies. `NOT EXISTS` still returns Kai, Lu, Ned. Default to `NOT EXISTS`. If you must use `NOT IN`, add `WHERE o.rep_id IS NOT NULL` inside.

18.6 Conditional aggregation (P27–P28)

Conditional aggregation (§4.4) computes several filtered aggregates over the same groups in one pass, turning row values (statuses, types) into columns — a simple **pivot**: `agg(...) FILTER (WHERE cond)` in

PostgreSQL and standard SQL, `SUM(CASE WHEN cond THEN x ELSE 0 END)` everywhere.

P27 · Attendance summary — one column per status

Problem. `employee(employee_id, employee_name)` and `attendance(employee_id, att_date, status)` with statuses Present / Absent / Late (duplicates possible). Show, for every employee, the number of rows of each status, ordered by id.

```
SELECT e.employee_id, e.employee_name,
       count(*) FILTER (WHERE a.status = 'Present') AS days_present,
       count(*) FILTER (WHERE a.status = 'Absent')  AS days_absent,
       count(*) FILTER (WHERE a.status = 'Late')    AS days_late
FROM employee e
LEFT JOIN attendance a ON a.employee_id = e.employee_id
GROUP BY e.employee_id, e.employee_name
ORDER BY e.employee_id;
```

```
 employee_id | employee_name | days_present | days_absent | days_late
-------------+---------------+--------------+-------------+-----------
           1 | Aiko          |            2 |           0 |         1
           2 | Bram          |            2 |           1 |         0
           3 | Cleo          |            0 |           0 |         0
(3 rows)
```

Why it works. The LEFT JOIN keeps Cleo, who has no records. Each `count(*) FILTER` counts only the rows passing its condition, and `count` never returns NULL. Bram's duplicated "Present" row counts twice, as the spec says to count *rows*. If it meant *days*, use `count(DISTINCT a.att_date) FILTER (…)`.

Portable form and a trap. The `sum(CASE …)` spelling with `THEN 1 ELSE 0` also gives Cleo 0 (column `present_else0`). Drop the `ELSE 0` and the `CASE` yields NULL for her one padded row, so `sum` returns **NULL**:

```
 employee_id | present_else0 | present_no_else
-------------+---------------+-----------------
           1 |             2 |               2
           2 |             2 |               2
           3 |             0 |          (null)
(3 rows)
```

P28 · Monthly approvals and chargebacks — UNION ALL, then pivot

Problem. `txn(id, country, state, amount, trans_date)` with state approved/declined, and `chargeback(trans_id, cb_date)`. For each month and country, report the approved count and amount, and the chargeback count and amount. A chargeback belongs to the month it **arrived** and carries the original transaction's amount. Leave out month/country pairs where everything is zero.

```
WITH events AS (
  SELECT to_char(trans_date, 'YYYY-MM') AS month, country, 'approved' AS kind, amount
  FROM txn WHERE state = 'approved'
  UNION ALL
  SELECT to_char(c.cb_date, 'YYYY-MM'), t.country, 'chargeback', t.amount
  FROM chargeback c JOIN txn t ON t.id = c.trans_id
)
SELECT month, country,
       count(*) FILTER (WHERE kind = 'approved')                    AS approved_count,
       COALESCE(sum(amount) FILTER (WHERE kind = 'approved'), 0)    AS approved_amount,
       count(*) FILTER (WHERE kind = 'chargeback')                  AS chargeback_count,
       COALESCE(sum(amount) FILTER (WHERE kind = 'chargeback'), 0) AS chargeback_amount
FROM events
GROUP BY month, country
ORDER BY month, country;
```

```
  month  | country | approved_count | approved_amount | chargeback_count | chargeback_amount
---------+---------+----------------+-----------------+------------------+-------------------
 2024-05 | US      |              1 |            1000 |                1 |              2000
 2024-06 | US      |              2 |            8000 |                1 |              1000
 2024-09 | US      |              0 |               0 |                1 |              5000
(3 rows)
```

Why it works. The two event kinds have **different dates**, so a single join can't bucket them. The trick is to stack them into one long list of `(month, country, kind, amount)` with `UNION ALL`, which keeps duplicates, unlike `UNION`. After that, one `GROUP BY` plus four `FILTER`s pivots the list. Notice what this data exercises. The declined May transaction still produces a May chargeback. Chargebacks land in later months (June, September). The DE row, which is only a declined transaction, produces no event, so the all-zero rows disappear with no `HAVING` needed. `sum … FILTER` over no rows is NULL, hence the `COALESCE`. The `to_char(d, 'YYYY-MM')` month label is `DATE_FORMAT(d, '%Y-%m')` in MySQL and `FORMAT(d, 'yyyy-MM')` in SQL Server.

18.7 Date-logic problems (P29–P33)

Date questions test ranges, arithmetic and "what was true on day X". Useful PostgreSQL facts: `date - date` is an **integer** number of days; `date + 1` is the next day; `date_trunc('month', ts)` gives the first instant of the month. A **half-open** range includes its start but excludes its end (`>= start AND < next_start`). Background: Chapter 3.

P29 · Weather class per country in November 2019 — month range + CASE on an average

Problem. `country(country_id, country_name)` and `weather(country_id, weather_state, day)`. Using only November 2019 readings, label each country Cold if its average is ≤ 15, Hot if ≥ 25, and Warm otherwise.

```sql
SELECT c.country_name,
       CASE WHEN avg(w.weather_state) <= 15 THEN 'Cold'
            WHEN avg(w.weather_state) >= 25 THEN 'Hot'
            ELSE 'Warm' END AS weather_type
FROM weather w
JOIN country c ON c.country_id = w.country_id
WHERE w.day >= DATE '2019-11-01' AND w.day < DATE '2019-12-01'
GROUP BY c.country_id, c.country_name
ORDER BY c.country_name;
```

```
 country_name | weather_type
--------------+--------------
 Arland       | Cold
 Borea        | Cold
 Calidra      | Warm
 Dune         | Hot
(4 rows)
```

Why it works. `WHERE` keeps November rows. Arland's October readings and Dune's December readings are ignored, so Dune's single November reading of 25 makes it Hot. `CASE` can test an aggregate because it runs after grouping. `CASE` stops at the first true branch, so order the thresholds carefully. PostgreSQL's `avg(int)` returns `numeric`, so there is no truncation. **Edge case:** a country with no November data simply doesn't appear. If it must appear, `LEFT JOIN` from `country` with the date filter in `ON`, and decide what label an unknown average gets.

P30 · Engagement level for February 2020 — leap-year boundary

Problem. `app_user(user_id, user_name)` and `engagement(user_id, score, day)`. For February 2020, classify each user's average score: below 20 Low, 20–60 inclusive Medium, above 60 High.

```sql
SELECT u.user_name,
       CASE WHEN avg(e.score) < 20  THEN 'Low'
            WHEN avg(e.score) <= 60 THEN 'Medium'
            ELSE 'High' END AS engagement_level
FROM app_user u
JOIN engagement e ON e.user_id = u.user_id
WHERE e.day >= DATE '2020-02-01' AND e.day < DATE '2020-03-01'
GROUP BY u.user_id, u.user_name
ORDER BY u.user_name;
```

```
 user_name | engagement_level
-----------+-----------------
 Ravi      | Medium
 Sofia     | Medium
 Theo      | High
(3 rows)
```

Why it works / edge case. Same shape as P29, but this data hits a boundary. 2020 is a leap year, and Ravi's 90 was recorded on **29 February**. A hand-written `BETWEEN '2020-02-01' AND '2020-02-28'` drops it, and Ravi's average falls from 50 (Medium) to 10 (Low). The half-open `< DATE '2020-03-01'` never needs to know how long the month is. Sofia's 1 March reading is correctly excluded. (A MySQL-style `YEAR(d) = 2020 AND MONTH(d) = 2` is also correct but can't use a plain index on `d`.)

P31 · Absences longer than three days — date subtraction, inclusive counting

Problem. `absence(employee_id, employee_name, start_date, end_date)` with start ≤ end. Return absences that last more than three days, where both the first and the last day count as days off.

```sql
SELECT employee_name, start_date, end_date, end_date - start_date + 1 AS days_off
FROM absence
WHERE end_date - start_date + 1 > 3
ORDER BY employee_name;
```

```
 employee_name | start_date |  end_date  | days_off
---------------+------------+------------+----------
 Hana          | 2024-03-01 | 2024-03-05 |        5
 Jade          | 2024-02-27 | 2024-03-01 |        4
(2 rows)
```

Why it works / edge cases. `end - start` counts the *gaps* between days, and `+ 1` counts the *days*. Ivo (10–12 March) is exactly 3 days, so it is not "more than three". Jade's absence crosses 29 February 2024 and is still counted correctly as 4 days. **Clarify** calendar vs working days; for working days count `generate_series(start, end, '1 day')` rows whose `extract(isodow …) < 6` (Monday = 1 … Sunday = 7). Dialects: MySQL `DATEDIFF(end, start)` (days), SQL Server `DATEDIFF(day, start, end)`, SQLite `julianday(end) - julianday(start)`.

P32 · Absence streaks of 3+ days — gaps and islands (first taste)

Problem. `attendance(student_id, att_date, status)` with one row per student per day, status Present/Absent. Find every run of **three or more consecutive calendar days** of absence, and report the student and the run's first day.

Data: student 1 is absent 1–3 Jan, present 4 Jan, then absent 5–8 Jan. Student 2 is absent 1–2 and 4–5 Jan (no run of 3). Student 3 is absent 30 Jan – 1 Feb, across a month boundary.

The trick: within a run of consecutive dates, `date − row_number()` is constant (`row_number()` numbers rows 1, 2, 3… within each `PARTITION BY` group, Chapter 8).

```
SELECT student_id, att_date,
       row_number() OVER w AS rn,
       att_date - (row_number() OVER w)::int AS grp
FROM attendance
WHERE status = 'Absent' AND student_id = 1
WINDOW w AS (PARTITION BY student_id ORDER BY att_date)
ORDER BY att_date;
```

```
 student_id |  att_date  | rn |    grp
------------+------------+----+------------
          1 | 2024-01-01 |  1 | 2023-12-31
          1 | 2024-01-02 |  2 | 2023-12-31
          1 | 2024-01-03 |  3 | 2023-12-31
          1 | 2024-01-05 |  4 | 2024-01-01
          1 | 2024-01-06 |  5 | 2024-01-01
          1 | 2024-01-07 |  6 | 2024-01-01
          1 | 2024-01-08 |  7 | 2024-01-01
(7 rows)
```

```
WITH absent AS (
  SELECT student_id, att_date,
         att_date - (row_number() OVER (PARTITION BY student_id ORDER BY att_date))::int AS grp
  FROM attendance
  WHERE status = 'Absent'
)
SELECT student_id, min(att_date) AS start_date, count(*) AS days
FROM absent
GROUP BY student_id, grp
HAVING count(*) >= 3
ORDER BY student_id, start_date;
```

```
 student_id | start_date | days
------------+------------+------
          1 | 2024-01-01 |    3
          1 | 2024-01-05 |    4
          3 | 2024-01-30 |    3
(3 rows)
```

Why it works. When the date goes up by 1 and the row number also goes up by 1, their difference stays the same. A gap in the dates bumps the difference, which starts a new group (an *island*). Grouping by `(student, grp)` then yields one row per run. `min` gives its start and `count` its length.

Common mistake. Subtracting two `row_number()` s that use the **same** partition and order always gives 0, so every absence lands in one group. The "two row numbers" variant only works when the two numberings differ: one over *all* rows and one over *absent* rows. That variant is what you want when "consecutive" means consecutive **records** (for example school days, skipping weekends) rather than calendar days. Full treatment: Chapter 8 §8.9 and Chapter 19.

P33 · Price on a given date — latest row on or before a date

Problem. `price_change(product_id, new_price, effective_on)`. Each product costs 10 until its first change. Report every product's price on 15 March 2024.

```
WITH latest AS (
  SELECT DISTINCT ON (product_id) product_id, new_price
  FROM price_change
  WHERE effective_on <= DATE '2024-03-15'
  ORDER BY product_id, effective_on DESC
)
SELECT p.product_id, COALESCE(l.new_price, 10) AS price
FROM (SELECT DISTINCT product_id FROM price_change) AS p
LEFT JOIN latest l USING (product_id)
ORDER BY p.product_id;
```

```
 product_id | price
------------+-------
          1 |    30
          2 |    50
          3 |    10
(3 rows)
```

Why it works. The CTE throws away future changes and then keeps, per product, the row with the newest `effective_on` (`DISTINCT ON` keeps the first row of each group in the `ORDER BY`). Product 1 changed on the 14th and again on the 15th, and `<=` picks the 15th (30). Product 3's first change is on the 18th, so it has no row in `latest`, and the LEFT JOIN plus `COALESCE` gives the default 10.

Alternatives. A correlated scalar subquery, `COALESCE((SELECT new_price … ORDER BY effective_on DESC LIMIT 1), 10)`, gives the same result and is fast with an index on `(product_id, effective_on)`. `ROW_NUMBER() … = 1` works in every engine. In a real schema, the list of products should come from a `product` table, because products that never changed price would be missing from `price_change`.

18.8 Ratios and percentages (P34–P38)

Ratio questions are mostly about **arithmetic types**. In PostgreSQL, SQL Server and SQLite, `int / int` is integer division (`2/3 = 0`), whereas MySQL's `/` returns a decimal. Guard every denominator that can be 0 with `NULLIF(den, 0)`, and round at the end, not in the middle.

P34 · Search query quality — averaging a ratio, share of "poor" rows

Problem. `search_log(query_name, result, position, rating)` (duplicates allowed). Per query name: *quality* is the average of `rating / position`, and *poor %* is the percentage of rows with rating below 3. Round both to 2 decimals.

```sql
SELECT query_name,
       round(avg(rating::numeric / position), 2)                      AS quality,
       round(100.0 * count(*) FILTER (WHERE rating < 3) / count(*), 2) AS poor_query_percentage
FROM search_log
GROUP BY query_name
ORDER BY query_name;
```

```
 query_name | quality | poor_query_percentage
------------+---------+----------------------
 Cat        |    0.66 |                 33.33
 Dog        |    2.50 |                 33.33
(2 rows)
```

Why it works. The cast makes each ratio exact (Dog: 5, 2.5, 0.005 → average 2.50). `100.0 * …` forces decimal arithmetic before the division.

The bug to mention. Without the casts, every `rating / position` truncates (5/2 → 2, 1/200 → 0) and the poor share becomes `1 / 3 * 100 = 0`:

```
 query_name |      int_quality       | int_poor_pct
------------+------------------------+--------------
 Cat        | 0.3333333333333333333 |            0
 Dog        |     2.3333333333333333 |            0
(2 rows)
```

MySQL gets these right without casts (its `/` is decimal), which is why solutions ported from MySQL often break on PostgreSQL. Also note that `round(double precision, int)` doesn't exist in PostgreSQL. Keep the arithmetic in `numeric`.

P35 · Ad click-through rate — divide by a possibly-zero count

Problem. `ad_action(ad_id, user_id, action)` with actions Clicked / Viewed / Ignored. **CTR** (click-through rate) = clicks × 100 / (clicks + views), or 0 when an ad has neither. Round to 2 decimals and sort by

CTR (highest first), then by ad id.

```
SELECT ad_id,
       round(COALESCE(100.0 * count(*) FILTER (WHERE action = 'Clicked')
                      / NULLIF(count(*) FILTER (WHERE action IN ('Clicked','Viewed')), 0), 0), 2) AS ctr
FROM ad_action
GROUP BY ad_id
ORDER BY ctr DESC, ad_id;
```

```
 ad_id |  ctr
-------+-------
     1 | 66.67
     3 | 50.00
     2 | 33.33
     5 |  0.00
(4 rows)
```

Why it works. Two conditional counts give clicks and clicks + views. Ad 5 was only ignored, so its denominator is 0. `NULLIF(0, 0)` turns it into NULL, the division yields NULL instead of an error, and `COALESCE(…, 0)` applies the spec's rule. "Ignored" rows are in the group but excluded from both counts. The secondary sort key `ad_id` makes ties deterministic.

P36 · Each book's share of all loans — ratio against a grand total

Problem. `book(book_id, title)` and `loan(loan_id, book_id, user_id, loan_date)`. A book's loan share is (its loans) / (all loans). Show every book's share rounded to 2 decimals, highest first, ties by id.

```
SELECT b.book_id,
       round(count(l.loan_id)::numeric / NULLIF((SELECT count(*) FROM loan), 0), 2) AS lf
FROM book b
LEFT JOIN loan l ON l.book_id = b.book_id
GROUP BY b.book_id
ORDER BY lf DESC, b.book_id;
```

```
 book_id |  lf
---------+------
       1 | 0.50
       2 | 0.33
       3 | 0.17
       4 | 0.00
(4 rows)
```

Why it works. The numerator is per group; the denominator is the whole-table count from an uncorrelated scalar subquery, which runs once. Book 4 was never lent, so the LEFT JOIN keeps it and `count(l.loan_id)` is 0. Without `::numeric`, 3/6 would be 0 in PostgreSQL.

Alternative. A window over the grouped result avoids the second scan: `count(l.loan_id)::numeric / NULLIF(sum(count(l.loan_id)) OVER (), 0)`. Window functions run after `GROUP BY`, so they can aggregate the aggregates. This gives the same output here. Keep the `NULLIF`: with an empty `loan` table the window sum is 0 and the division would fail. The two denominators differ only if `loan` could hold book ids missing from `book`, which the foreign key rules out.

P37 · Average selling price — weighted average with a range join

Problem. `price(product_id, start_d, end_d, price)` holds non-overlapping price periods. `units_sold(product_id, bought_on, units)` holds sales. For each product, the average selling price is total revenue / total units, using the price valid on each sale's date. Products that never sold show 0. Round to 2 decimals.

```sql
SELECT p.product_id,
       round(COALESCE(sum(u.units * p.price)::numeric / sum(u.units), 0), 2) AS average_price
FROM price p
LEFT JOIN units_sold u
       ON u.product_id = p.product_id
      AND u.bought_on BETWEEN p.start_d AND p.end_d
GROUP BY p.product_id
ORDER BY p.product_id;
```

```
 product_id | average_price
------------+---------------
          1 |          6.96
          2 |         16.96
          3 |          0.00
(3 rows)
```

Why it works. A **range join** (condition "falls inside an interval", not equality) matches each sale to exactly one price period (same product, date inside the period). Product 1 sold 100 units at 5 and 15 at 20: (500 + 300) / 115 = 6.96. Product 3 has a price but no sales. Its sums are NULL, NULL/NULL is NULL (not an error), and `COALESCE` makes it 0.

The bug. `avg(p.price)` averages the *price rows* (5 and 20 → 12.50) and ignores that most units sold at 5. Whenever rows carry a quantity, use a **weighted** average: `sum(value × weight) / sum(weight)`. If a sale could fall outside every period, it would silently vanish, so check with an anti-join.

P38 · Came back the next day — retention fraction

Problem. `activity(player_id, device_id, played_on, games)`. What share of players showed up again exactly one day after their first-ever login? Round to 2 decimals.

```sql
WITH first_day AS (
  SELECT player_id, min(played_on) AS first_on FROM activity GROUP BY player_id
)
SELECT round(count(a.player_id)::numeric / count(*), 2) AS fraction
FROM first_day f
LEFT JOIN activity a ON a.player_id = f.player_id AND a.played_on = f.first_on + 1;
```

```
 fraction
----------
     0.50
(1 row)
```

Why it works. The CTE gives one row per player (the denominator). The LEFT JOIN looks for a row exactly one day later (`date + 1`). `count(a.player_id)` counts only players who matched (the numerator). Players 1 and 4 returned (player 4 on 29 February 2024, which `+ 1` handles). Player 3 came back two days later, which doesn't count, and player 2 never came back. The result is 2/4. Without the cast, `count / count` is `0`. This "day-1 retention" metric is a staple of product-analytics interviews; variants ask for the fraction *per install date* (group by `first_on`) or for day-7 retention (`+ 7`).

18.9 Window and ranking basics (P39–P42)

Problems easiest with **window functions** (Chapter 8); harder variants are in Chapter 19.

P39 · Second-highest salary — and "return NULL if none"

Problem. `emp(id, salary)`. Return the second-highest **distinct** salary, or NULL when there isn't one.

Data: salaries 300, 300, 200, 100 (a tie at the top).

```sql
SELECT (SELECT DISTINCT salary FROM emp ORDER BY salary DESC OFFSET 1 LIMIT 1)
       AS second_highest_salary;
```

```
 second_highest_salary
----------------------
                   200
(1 row)
```

Why it works. `DISTINCT` collapses the two 300s, so `OFFSET 1` skips exactly one value. Wrapping it in an outer `SELECT (…)` makes a **scalar subquery**, which yields NULL when the inner query finds nothing. After deleting all but the 300s, the wrapped query returns `(null)` in one row, while an unwrapped query (the file shows the bare `dense_rank` version) returns 0 rows.

Alternative. `dense_rank() OVER (ORDER BY salary DESC)` numbers distinct values 1, 1, 2, 3. Keep `dr = 2`, and wrap the result in `max(salary)` so an empty result still becomes one NULL row. Without `DISTINCT`, `OFFSET 1` would return the duplicate 300, the most common wrong answer. Another classic is `SELECT max(salary) FROM emp WHERE salary < (SELECT max(salary) FROM emp)`. The general N-th version is in Chapter 19.

P40 · Device used at first login — first row per group

Problem. Using `activity(player_id, device_id, played_on, games)`, report the device each player used on their earliest day.

```sql
SELECT DISTINCT ON (player_id) player_id, device_id
FROM activity
ORDER BY player_id, played_on;
```

```
 player_id | device_id
-----------+-----------
         1 |         2
         2 |         3
         3 |         1
(3 rows)
```

Why it works. `DISTINCT ON (player_id)` (PostgreSQL only) keeps the first row per player in `ORDER BY` order, so the earliest day wins, and `device_id` comes along with it. That's exactly what `min(played_on)` alone (P11) could not give.

Portable alternatives (same output in the file):

- `row_number() OVER (PARTITION BY player_id ORDER BY played_on) = 1` in a subquery. Use `rank()` if two rows could share the first date and you want both.
- `WHERE (player_id, played_on) IN (SELECT player_id, min(played_on) … GROUP BY player_id)`, a **row-value** `IN` (several columns compared as a tuple); SQL Server lacks it, so join to the grouped subquery instead.

P41 · Games played so far — running total

Problem. For every `(player, day)` in `activity`, show the total games that player has played up to and including that day.

```sql
SELECT player_id, played_on,
       sum(games) OVER (PARTITION BY player_id ORDER BY played_on) AS games_played_so_far
FROM activity
ORDER BY player_id, played_on;
```

```
 player_id | played_on  | games_played_so_far
-----------+------------+---------------------
         1 | 2024-03-01 |                   5
         1 | 2024-05-02 |                  11
         1 | 2024-06-25 |                  12
         3 | 2024-03-02 |                   0
         3 | 2024-07-03 |                   5
(5 rows)
```

Why it works. `PARTITION BY` restarts the sum per player. `ORDER BY` inside `OVER` makes the default **frame** (§8.3) "first row up to the current row, including sort-key ties" (`RANGE UNBOUNDED PRECEDING`). Here

`(player_id, played_on)` is unique, so there are no ties. If two rows could share a date and you want a strict row-by-row total, write `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`.

Alternative. Before window functions (MySQL < 8.0), you'd self-join each row to all earlier-or-same rows of the same player and `sum`. It gives the same output but does O(n²) work per player (10× the rows, 100× the work).

P42 · Active businesses — compare each row to its group's average

Problem. `event(business_id, event_type, occurrences)`. The average for an event type is the mean of `occurrences` over the businesses that have it. A business is **active** if, on *more than one* event type, its occurrences are strictly above that type's average. List active businesses.

First attach each type's average to every row:

```
 business_id | event_type | occurrences | type_avg
-------------+------------+-------------+----------
           1 | ads        |          11 |     8.00
           2 | ads        |           7 |     8.00
           3 | ads        |           6 |     8.00
           4 | coupons    |           9 |     9.00
           1 | page views |           3 |     7.50
           2 | page views |          12 |     7.50
           1 | reviews    |           7 |     5.00
           3 | reviews    |           3 |     5.00
(8 rows)
```

```sql
SELECT business_id
FROM (SELECT business_id, occurrences,
             avg(occurrences) OVER (PARTITION BY event_type) AS type_avg
      FROM event) AS e
WHERE occurrences > type_avg
GROUP BY business_id
HAVING count(*) > 1
ORDER BY business_id;
```

```
 business_id
-------------
           1
(1 row)
```

Why it works. The window `avg` keeps every row and adds the type average next to it. You can't write `WHERE occurrences > avg(...)`, because windows are evaluated after `WHERE`, hence the subquery. Business 1 beats the average on ads and reviews (2 types), and business 2 only on page views. Business 4 is alone in "coupons", so it equals the average and never beats it. **Alternative without windows:** join `event` to `SELECT event_type, avg(occurrences) … GROUP BY event_type` on `event_type`. **Edge case:** "more than one" means `> 1`, not `>= 1`, and "strictly greater" means `>`. Read comparison words carefully.

18.10 Pattern table

#	Problem	Pattern	Key SQL feature
P1	Big cities	Filtering	`OR`, three-valued logic with NULL
P2	Odd non-boring movies	Filtering	`%`, `<>`, `IS DISTINCT FROM`, `ORDER BY DESC`
P3	Green products	Filtering	`AND`, `CHECK` constraint
P4	Outside department 3	Filtering	NULL-safe `IS DISTINCT FROM`
P5	Self-assigned tasks	Filtering	column-to-column compare, `DISTINCT`
P6	Long comments	Filtering	`char_length` vs `octet_length`
P7	Valid e-mails	Filtering	regex `~`, anchors, escaping
P8	Triangle test	Row-level logic	`CASE`, `greatest`
P9	Largest unique number	Aggregation	`HAVING count(*) = 1`, aggregate over empty set → NULL
P10	Units per product	Aggregation	`SUM` + `GROUP BY`, `LEFT JOIN` + `COALESCE`
P11	First login	Aggregation	`MIN` on dates
P12	Follower count	Aggregation	`count(*)` vs `count(col)`
P13	Bike last used	Aggregation	`MAX`, sort by aggregate, NULL handling
P14	Missed appointments	Aggregation	`WHERE` before `GROUP BY`; `FILTER` + `HAVING`
P15	Products per day	Aggregation	`count(DISTINCT)`, `string_agg(DISTINCT … ORDER BY)`
P16	Customer basket	Aggregation	`string_agg` / `array_agg`
P17	Sale product names	Joins	`INNER JOIN` on a key
P18	Process time	Joins	self-join; `FILTER` pivot alternative
P19	Fiction per author	Joins	join + filter + group by key, two-key sort
P20	Exam sittings	Joins	`CROSS JOIN` grid + `LEFT JOIN` + `count(col)`
P21	Election winner	Joins / top-1	`ORDER BY count(*) DESC LIMIT 1`, `WITH TIES`
P22	Middle followers	Joins / semi-join	`EXISTS`; self-join needs `count(DISTINCT)`
P23	Idle sellers	Anti-join	`NOT EXISTS`, half-open date range
P24	Dormant accounts	Anti-join	`LEFT JOIN … IS NULL`, filter in `ON`; `EXCEPT`
P25	Ad-free sessions	Anti-join	`NOT EXISTS` with `BETWEEN` range condition
P26	Reps without RED	Anti-join	`NOT EXISTS` vs `NOT IN` + NULL
P27	Attendance summary	Conditional aggregation	`count(*) FILTER`, `SUM(CASE … ELSE 0)`
P28	Monthly chargebacks	Conditional aggregation	`UNION ALL` then `FILTER`, `to_char` month
P29	Weather class	Date logic	month range + `CASE` on `avg`
P30	Engagement level	Date logic	half-open range (leap day), `CASE` buckets
P31	Long absences	Date logic	`date - date`, inclusive counting
P32	Absence streaks	Date logic / gaps & islands	`date - row_number()` grouping
P33	Price on a date	Date logic / as-of	`DISTINCT ON` or `LIMIT 1` subquery + `COALESCE` default

#	Problem	Pattern	Key SQL feature
P34	Query quality	Ratios	cast before dividing, `FILTER` share
P35	Ad CTR	Ratios	`NULLIF` + `COALESCE` for zero denominators
P36	Loan share	Ratios	ratio to grand total (scalar subquery or `sum() OVER ()`)
P37	Average selling price	Ratios	range join, weighted average
P38	Next-day return	Ratios / retention	`min` CTE + `LEFT JOIN` on `date + 1`
P39	Second-highest salary	Ranking	`DISTINCT … OFFSET 1` in scalar subquery, `dense_rank`
P40	First-login device	Ranking / first per group	`DISTINCT ON`, `row_number() = 1`
P41	Games so far	Window	running `sum() OVER (PARTITION BY … ORDER BY …)`
P42	Active businesses	Window	`avg() OVER (PARTITION BY)` then `HAVING count(*) > 1`

Chapter 18 TL;DR

- **Method:** clarify grain, keys, NULLs, zero rows, ties and output. Sketch rows, build in evaluation order with CTEs, attack edge cases, then verify and discuss indexes.
- **Filtering:** `WHERE` keeps only `TRUE`. `NULL` comparisons give `UNKNOWN`, so use `IS DISTINCT FROM` when NULLs must count. Anchor and escape regexes. Count characters, not bytes.
- **Aggregation:** filter before grouping. `count(col)` skips NULLs, which is what makes zero-counts work after a LEFT JOIN. An aggregate over nothing returns one NULL row, while `LIMIT 1` over nothing returns no row.
- **Joins:** group by keys, not names. Build "every pair" grids with `CROSS JOIN` + `LEFT JOIN`. Watch for fan-out and prefer `EXISTS` for "has at least one". Use `WITH TIES` or `RANK` for fair top-1 answers.
- **Anti-joins:** `NOT EXISTS` is the safe default. With `LEFT JOIN … IS NULL`, the right-table filters belong in `ON`. `NOT IN` returns nothing if the subquery holds a NULL.
- **Conditional aggregation:** `agg(...) FILTER (WHERE …)` or `SUM(CASE … ELSE 0)` pivots values into columns. Stack differently-dated events with `UNION ALL` first.
- **Dates:** half-open ranges (`>= start AND < next_start`); `date - date` is days; `+ 1` for "inclusive" counts; `date - row_number()` finds consecutive runs; "as of a date" = latest row `<=` the date + a default.
- **Ratios:** cast to `numeric` (or `* 100.0`) before dividing, `NULLIF` the denominator, round last, and use weighted averages when rows carry quantities.
- **Windows:** `dense_rank`/`DISTINCT … OFFSET` for N-th highest (wrap for NULL), `DISTINCT ON`/`row_number` for the first row per group, `sum() OVER (… ORDER BY …)` for running totals, `avg() OVER (PARTITION BY …)` to compare a row with its group.

Next: Chapter 19 — SQL interview problems, part 2, with harder problems: top-N per group, streaks, pivots and relational division.

Chapter 19 — SQL Interview Problems 2: Medium and Hard Patterns

The previous chapter practised the basics (filters, grouping, joins, anti-joins, simple ratios). This chapter covers the medium and hard problems: ranking inside groups, running sums, streaks in ordered data, rows into columns (even when the columns are unknown), tables as small graphs, date buckets, and strings built from rows.

Each problem has a statement in our own words, a self-contained file in `code/` (tables, tiny invented sample, solution, often an alternative), real `psql` output (`(null)` marks NULL), why it works, and the trap or alternative worth mentioning. All 41 files run on PostgreSQL 14 with `bash tools_run_sql.sh sql '19-*'`.

What you'll learn: the patterns behind most medium/hard questions, with a template each (19.11, which also lists the wording that signals each pattern) · `ROW_NUMBER` vs `RANK` vs `DENSE_RANK` and why ties change the answer · the *gaps-and-islands* trick (`value - ROW_NUMBER()`) for streaks and "N in a row" · static pivots with `FILTER`, dynamic pivots with PL/pgSQL + `format()`, unpivot via JSON · PostgreSQL traps: integer division (`1/3 = 0`), division by zero, `NOT IN` with NULLs, `COUNT(*)` after a `LEFT JOIN`.

Background you may want open: window functions (ranking, frames, gaps and islands), subqueries and set operations, CTEs, and the interview method in Chapter 18.

19.1 How to attack a medium or hard problem

The Chapter 18 method still applies; three habits matter most here:

Habit	What it means in practice
Build in layers	One CTE per idea ("one row per user-day", then "rank within the day", then "keep rank 1"); run each layer before adding the next.
Say the tie rule out loud	"If two invoices have the same total, which one wins?" decides between `ROW_NUMBER`, `RANK` and `DENSE_RANK`.
Hunt the empty case	Players with no wins, orders with no deliveries, days with no visits: decide whether they appear with 0, with NULL, or not at all.

Run any file on its own with `bash tools_pg.sh psql -d <empty_db> -f code/NN_name.sql`.

19.2 Warm-ups: joins, counts and grids

Three short problems that hide a classic trap each: NULL in a comparison, `COUNT(*)` after an outer join, and "every combination" questions.

19.2.1 Staff rated above 8

Problem. Two tables: `staff` (id, name, team) and `reviews` (one score per staff member). List the names of people whose score is strictly greater than 8.

`code/01_high_rated_staff.sql`

```
CREATE TABLE staff   (staff_id int PRIMARY KEY, full_name text, team text);
CREATE TABLE reviews (staff_id int PRIMARY KEY REFERENCES staff, score int);
INSERT INTO staff VALUES (1,'Asha','Web'),(2,'Bruno','Web'),(3,'Chen','Data'),(4,'Dana','Data');
INSERT INTO reviews VALUES (1,9),(2,8),(3,10),(4,NULL);

SELECT s.full_name
FROM   staff s
JOIN   reviews r ON r.staff_id = s.staff_id
WHERE  r.score > 8
ORDER  BY s.full_name;

-- Edge case: NULL score is neither > 8 nor <= 8
SELECT count(*) FILTER (WHERE score > 8)      AS above,
       count(*) FILTER (WHERE NOT score > 8)  AS not_above,
       count(*)                               AS all_rows
FROM   reviews;
```

Output:

```
 full_name
-----------
 Asha
 Chen
(2 rows)

 above | not_above | all_rows
-------+-----------+----------
     2 |         1 |        4
(1 row)
```

Why it works. The inner join pairs each person with their review; `WHERE r.score > 8` keeps the high ones. Anyone without a review row disappears in the join, which is what "has a rating above 8" means.

Edge case. Dana's score is NULL: `NULL > 8` and `NOT (NULL > 8)` are both *unknown*, so she lands in neither bucket (2 + 1 ≠ 4). "Everyone not above 8" needs `OR score IS NULL`.

19.2.2 Likes per post

Problem. `posts` lists posts; `post_likes` has one row per (post, user who liked it). Show every post with its number of likes, including posts nobody liked, in post order.

`code/02_likes_per_post.sql`

```
CREATE TABLE posts      (post_id int PRIMARY KEY, author text);
CREATE TABLE post_likes (post_id int REFERENCES posts, liker_id int, PRIMARY KEY (post_id, liker_id));
INSERT INTO posts VALUES (10,'ana'),(11,'ben'),(12,'cy');
INSERT INTO post_likes VALUES (10,1),(10,2),(10,3),(11,1);

SELECT p.post_id, count(l.liker_id) AS likes
FROM   posts p
LEFT   JOIN post_likes l ON l.post_id = p.post_id
GROUP  BY p.post_id
ORDER  BY p.post_id;
```

Output:

```
 post_id | likes
---------+-------
      10 |     3
      11 |     1
      12 |     0
(3 rows)
```

Why it works. The `LEFT JOIN` keeps post 12 even though it has no likes, with NULL join columns (a **NULL-padded** row). `count(l.liker_id)` counts only non-NULL values, so the padded row contributes 0.

> **Common mistake.** `count(*)` counts the NULL-padded row too, so post 12 would show 1 (the file runs it; the same trap as Chapter 18, P12). After an outer join, count a column from the *optional* side.

19.2.3 Average mark for every learner × module

Problem. For every learner and every module in the catalogue (even ones the learner has no marks in), show the learner's average mark in that module, rounded to 2 decimals; NULL when there are no marks. Sort by learner, then module.

`code/03_course_grid_averages.sql`

```
CREATE TABLE learners (learner_id int PRIMARY KEY, name text);
CREATE TABLE modules  (module text PRIMARY KEY);
CREATE TABLE marks    (learner_id int, module text, mark int);
INSERT INTO learners VALUES (1,'Ivy'),(2,'Omar');
INSERT INTO modules  VALUES ('SQL'),('Stats');
INSERT INTO marks    VALUES (1,'SQL',70),(1,'SQL',85),(1,'Stats',60),(2,'SQL',90);

SELECT l.learner_id, l.name, m.module,
       round(avg(k.mark), 2) AS avg_mark
FROM   learners l
CROSS  JOIN modules m
LEFT   JOIN marks k ON k.learner_id = l.learner_id AND k.module = m.module
GROUP  BY l.learner_id, l.name, m.module
ORDER  BY l.learner_id, m.module;
```

Output:

```
 learner_id | name | module | avg_mark
------------+------+--------+----------
          1 | Ivy  | SQL    |    77.50
          1 | Ivy  | Stats  |    60.00
          2 | Omar | SQL    |    90.00
          2 | Omar | Stats  |   (null)
(4 rows)
```

Why it works. "Every A with every B" is a `CROSS JOIN` — it builds the full grid of 2 × 2 pairs first. The `LEFT JOIN` then attaches marks where they exist, and `avg()` of only NULLs is NULL, which is exactly the "no marks" answer (Omar/Stats).

Alternative. If the question only wants modules the learner actually took, drop the cross join and group the `marks` table directly. Always ask which one is meant.

19.3 Nth highest, top-N per group and "first row per group"

The ranking family. The three ranking functions differ only on ties:

Scores	ROW_NUMBER	RANK	DENSE_RANK
95, 95, 90	1, 2, 3	1, 1, 3	1, 1, 2

Use `ROW_NUMBER` when exactly one row must win, `RANK` when "all tied for first" should be kept, and `DENSE_RANK` when the question says "top N *distinct* values". PostgreSQL's `DISTINCT ON (cols)` is a compact shortcut for "first row per group": it keeps the first row of each group of equal `cols` values, in `ORDER BY` order.

19.3.1 Second-fastest lap time

Problem. Each runner has one lap time, in seconds. Return the second-smallest *distinct* time as one row, or a single NULL row when there is no second value.

`code/04_second_fastest.sql`

```sql
CREATE TABLE laps (runner_id int PRIMARY KEY, seconds int NOT NULL);
INSERT INTO laps VALUES (1,62),(2,58),(3,58),(4,65),(5,61);

-- Solution: scalar subquery -> NULL when OFFSET skips past the end
SELECT (SELECT DISTINCT seconds FROM laps
        ORDER  BY seconds
        OFFSET 1 LIMIT 1) AS second_fastest;

-- Alternative: DENSE_RANK generalises to the N-th
SELECT max(seconds) FILTER (WHERE dr = 2) AS second_fastest
FROM  (SELECT seconds, dense_rank() OVER (ORDER BY seconds) AS dr FROM laps) t;

-- Edge case: only one distinct time left
DELETE FROM laps WHERE seconds <> 58;
SELECT (SELECT DISTINCT seconds FROM laps ORDER BY seconds OFFSET 1 LIMIT 1) AS second_fastest;
```

Output (both formulations give 61; after the `DELETE` the answer is a NULL row):

```
 second_fastest
----------------
             61
(1 row)
...
 second_fastest
----------------
         (null)
(1 row)
```

Why it works. Same idea as Chapter 18, P39: `DISTINCT` collapses the two 58s, `OFFSET 1 LIMIT 1` takes the next value, and the scalar-subquery wrapper turns "no row" into one NULL row.

Alternatives. `min(seconds) WHERE seconds > (SELECT min(seconds) …)` (also in the file) works only for N = 2. `DENSE_RANK` handles any N (`dr = N`), and wrapping it in `max(...) FILTER (...)` again turns "no row" into NULL.

19.3.2 Top three distinct scores per subject

Problem. Pupils' scores per subject. For each subject, list every pupil whose score is one of the three highest *different* scores in that subject (ties included).

`code/05_top3_per_subject.sql`

```sql
CREATE TABLE subjects (subject_id int PRIMARY KEY, title text);
CREATE TABLE results  (result_id int PRIMARY KEY, pupil text, score int,
                       subject_id int REFERENCES subjects);
INSERT INTO subjects VALUES (1,'Maths'),(2,'History');
INSERT INTO results VALUES
 (1,'Ann',95,1),(2,'Bob',95,1),(3,'Cal',90,1),(4,'Dee',82,1),(5,'Eve',70,1),
 (6,'Fay',88,2),(7,'Gus',75,2);

WITH ranked AS (
  SELECT s.title, r.pupil, r.score,
         dense_rank() OVER (PARTITION BY r.subject_id ORDER BY r.score DESC) AS dr
  FROM   results r
  JOIN   subjects s USING (subject_id)
)
SELECT title AS subject, pupil, score
FROM   ranked
WHERE  dr <= 3
ORDER  BY subject, score DESC, pupil;

-- Alternative: correlated subquery "fewer than 3 distinct higher scores"
SELECT s.title AS subject, r.pupil, r.score
FROM   results r
JOIN   subjects s USING (subject_id)
WHERE  (SELECT count(DISTINCT r2.score) FROM results r2
        WHERE r2.subject_id = r.subject_id AND r2.score > r.score) < 3
ORDER  BY subject, score DESC, pupil;
```

Output (all queries return these rows):

```
 subject | pupil | score
---------+-------+-------
 History | Fay   |    88
 History | Gus   |    75
 Maths   | Ann   |    95
 Maths   | Bob   |    95
 Maths   | Cal   |    90
 Maths   | Dee   |    82
(6 rows)
```

Why it works. `DENSE_RANK` gives Ann and Bob rank 1, Cal rank 2, Dee rank 3, so four pupils appear for Maths — ties don't use up places. History has only two pupils, so both appear.

Alternative. The correlated version keeps a row if fewer than three distinct scores in its subject beat it: portable to any SQL database, but O(n²) per group. With `RANK`, Dee would get rank 4 and vanish — a common wrong answer.

19.3.3 One home office per employee

Problem. Employees can be assigned to several offices; exactly one of them is flagged `'Y'` as the home office. Someone with only one office has that office as home, whatever the flag says. Return each employee with their home office.

`code/06_home_office.sql`

```
CREATE TABLE desk_assignments (emp_id int, office_id int, is_home char(1) NOT NULL CHECK (is_home IN ('Y','N')),
                               PRIMARY KEY (emp_id, office_id));
INSERT INTO desk_assignments VALUES
 (1,100,'N'),(1,200,'Y'),(1,300,'N'),   -- several offices, one flagged
 (2,100,'N'),                            -- single office, flag ignored
 (3,300,'Y');

-- Solution: DISTINCT ON keeps the first row of each emp_id in ORDER BY order
SELECT DISTINCT ON (emp_id) emp_id, office_id
FROM   desk_assignments
ORDER  BY emp_id, is_home DESC;      -- 'Y' sorts after 'N', so DESC puts it first

-- Alternative: the rule written out literally
SELECT emp_id, office_id FROM desk_assignments WHERE is_home = 'Y'
UNION
SELECT emp_id, min(office_id) FROM desk_assignments
GROUP  BY emp_id HAVING count(*) = 1
ORDER  BY emp_id;
```

Output (all queries return these rows):

```
 emp_id | office_id
--------+-----------
      1 |       200
      2 |       100
      3 |       300
(3 rows)
```

Why it works. `DISTINCT ON (emp_id)` keeps each employee's first row under `ORDER BY emp_id, is_home DESC`; `'Y' > 'N'`, so a flagged office sorts first, and a single office is kept as is. The flag is `NOT NULL` on purpose: PostgreSQL sorts NULLs *first* under `DESC`, so a NULL flag would beat `'Y'` (add `NULLS LAST` if NULLs are possible). Portable form: `ROW_NUMBER() OVER (PARTITION BY emp_id ORDER BY is_home DESC) = 1`.

Alternative / edge case. The `UNION` version states the rule literally. If the data breaks the rule (several offices, none flagged), the ranking version silently picks *some* office and the `UNION` version returns nothing for that person — choose your failure.

19.3.4 The most expensive invoice's lines

Problem. `items` holds unit prices; `line_items` holds (invoice, item, quantity). Find the invoice with the largest total — smallest invoice id wins a tie — and list its lines with quantity and line price.

`code/07_largest_invoice.sql`

```
CREATE TABLE items      (item_id int PRIMARY KEY, unit_price int);
CREATE TABLE line_items (invoice_id int, item_id int REFERENCES items, qty int,
                         PRIMARY KEY (invoice_id, item_id));
INSERT INTO items VALUES (1,10),(2,25),(3,4);
INSERT INTO line_items VALUES
 (501,1,3),(501,3,5),        -- 30 + 20 = 50
 (502,2,2),                  -- 50  (tie with 501)
 (503,1,1),(503,3,2);        -- 18

WITH priced AS (
  SELECT l.invoice_id, l.item_id, l.qty, l.qty * i.unit_price AS line_total,
         sum(l.qty * i.unit_price) OVER (PARTITION BY l.invoice_id) AS invoice_total
  FROM   line_items l JOIN items i USING (item_id)
), winner AS (
  SELECT invoice_id FROM priced
  ORDER  BY invoice_total DESC, invoice_id
  LIMIT  1
)
SELECT item_id, qty, line_total
FROM   priced
WHERE  invoice_id = (SELECT invoice_id FROM winner)
ORDER  BY item_id;
```

Output:

```
 item_id | qty | line_total
---------+-----+------------
       1 |   3 |         30
       3 |   5 |         20
(2 rows)
```

Why it works. A window `sum(...) OVER (PARTITION BY invoice_id)` puts each invoice's total next to each of its lines without collapsing them. The `winner` CTE sorts by total descending, then id ascending, and keeps one row. Invoices 501 and 502 both total 50, so 501 wins on id.

Alternative. Group `line_items` by invoice to get totals, pick the winner with `ORDER BY ... LIMIT 1`, then join back — the same logic with one more pass. `FETCH FIRST 1 ROW WITH TIES` is the tool if the question instead wanted *all* tied invoices.

19.3.5 Confirmed or waitlisted

Problem. Flights have a seat count; bookings have a passenger, a flight and a unique booking time. A booking is confirmed if, in booking-time order, it is within the flight's seat count; otherwise it is waitlisted.

`code/08_seat_status.sql`

```
CREATE TABLE flights  (flight_id int PRIMARY KEY, seats int);
CREATE TABLE bookings (passenger_id int PRIMARY KEY, flight_id int, booked_at timestamp UNIQUE);
INSERT INTO flights VALUES (7,2),(8,1),(9,0);
INSERT INTO bookings VALUES
 (1,7,'2026-03-01 09:00'),(2,7,'2026-03-01 08:00'),(3,7,'2026-03-02 10:00'),
 (4,8,'2026-03-03 12:00'),(5,8,'2026-03-01 12:00'),
 (6,9,'2026-03-01 07:00');

SELECT b.passenger_id,
       CASE WHEN row_number() OVER (PARTITION BY b.flight_id ORDER BY b.booked_at) <= f.seats
            THEN 'Confirmed' ELSE 'Waitlist' END AS status
FROM   bookings b
JOIN   flights f USING (flight_id)
ORDER  BY b.passenger_id;
```

Output:

```
 passenger_id |  status
--------------+-----------
            1 | Confirmed
            2 | Confirmed
            3 | Waitlist
            4 | Waitlist
            5 | Confirmed
            6 | Waitlist
(6 rows)
```

Why it works. `row_number()` numbers the bookings of each flight in time order; comparing that number with `seats` gives the status in one pass. On flight 7, passenger 2 booked first, then 1, then 3 (waitlisted). Flight 9 has zero seats, so even the first booking waits.

Edge case. Booking times are unique here, so `RANK` and `ROW_NUMBER` agree. If two bookings could share a timestamp, `RANK` would confirm both (possibly overbooking); `ROW_NUMBER` with a tie-breaker such as `passenger_id` keeps the count exact.

19.3.6 Pupils who are never top or bottom

Problem. A "middle-of-the-pack" pupil sat at least one test and never had the highest or the lowest score in any test they sat (ties for top or bottom count as top or bottom). List them by id.

`code/09_middle_of_the_pack.sql`

```
CREATE TABLE pupils (pupil_id int PRIMARY KEY, name text);
CREATE TABLE tests  (test_id int, pupil_id int, score int, PRIMARY KEY (test_id, pupil_id));
INSERT INTO pupils VALUES (1,'Ada'),(2,'Ben'),(3,'Cleo'),(4,'Dev'),(5,'Eli'),(6,'Fin');
INSERT INTO tests VALUES
 (10,1,60),(10,2,75),(10,3,90),(10,5,70),
 (20,1,55),(20,2,80),(20,4,80),(20,5,62);   -- Ben and Dev tie for top of test 20; Fin sat nothing

WITH marked AS (
  SELECT pupil_id,
         score = max(score) OVER (PARTITION BY test_id)
      OR score = min(score) OVER (PARTITION BY test_id) AS at_extreme
  FROM   tests
)
SELECT p.pupil_id, p.name
FROM   pupils p
JOIN   marked m USING (pupil_id)
GROUP  BY p.pupil_id, p.name
HAVING NOT bool_or(m.at_extreme)
ORDER  BY p.pupil_id;
```

Output:

```
 pupil_id | name
----------+------
        5 | Eli
(1 row)
```

Why it works. Two window aggregates put each test's max and min on every row, so `at_extreme` is true when a pupil matches either. Grouping per pupil and keeping `NOT bool_or(at_extreme)` (`bool_or` is true if any value in the group is true) means "no test where they were extreme". The inner join to `marked` drops Fin, who sat nothing.

Alternative. Mark extremes with `RANK() ... DESC = 1 OR RANK() ... ASC = 1` and exclude those pupils — with `NOT EXISTS`, not `NOT IN`, which returns nothing if the subquery yields a NULL.

19.3.7 Viewers who became streamers

Problem. A sessions table records each user's sessions as `viewer` or `streamer`. For users whose earliest session was as a viewer and who streamed at least once, return how many streaming sessions they have. Sort by that count, then user id, both descending.

code/10_viewers_then_streamers.sql

```
CREATE TABLE sessions (session_id int PRIMARY KEY, user_id int, started_at timestamp,
                       kind text CHECK (kind IN ('viewer','streamer')));
INSERT INTO sessions VALUES
 (1,101,'2026-01-01 10:00','viewer'),(2,101,'2026-01-02 10:00','streamer'),(3,101,'2026-01-05 10:00','streamer'),
 (4,102,'2026-01-01 09:00','streamer'),(5,102,'2026-01-03 09:00','viewer'),
 (6,103,'2026-01-02 08:00','viewer'),(7,103,'2026-01-04 08:00','streamer'),
 (8,104,'2026-01-02 08:00','viewer');                                  -- never streamed

WITH firsts AS (
  SELECT DISTINCT ON (user_id) user_id, kind AS first_kind
  FROM   sessions
  ORDER  BY user_id, started_at
)
SELECT s.user_id, count(*) FILTER (WHERE s.kind = 'streamer') AS streams
FROM   sessions s
JOIN   firsts f USING (user_id)
WHERE  f.first_kind = 'viewer'
GROUP  BY s.user_id
HAVING count(*) FILTER (WHERE s.kind = 'streamer') > 0
ORDER  BY streams DESC, s.user_id DESC;
```

Output:

```
 user_id | streams
---------+---------
     101 |       2
     103 |       1
(2 rows)
```

Why it works. `DISTINCT ON` finds each user's first session kind (user 102 started as a streamer, so is out). Joining back and counting with `FILTER (WHERE kind = 'streamer')` gives the streams; `HAVING ... > 0` drops user 104, who never streamed.

Alternative. `first_value(kind) OVER (PARTITION BY user_id ORDER BY started_at)` puts the first kind on every row (the file prints it: viewer, streamer, viewer, viewer).

19.3.8 First year on loan (keep the ties)

Problem. A library keeps yearly loan statistics per branch and book: number of loans and fee per loan. For each book, report its first year on loan with the loans and fee from that year — if the first year has several rows, report all of them.

code/11_first_year_loans.sql

```
CREATE TABLE loan_stats (branch_id int, book_id int, yr int, loans int, fee int,
                         PRIMARY KEY (branch_id, book_id, yr));
INSERT INTO loan_stats VALUES
 (1,100,2023,12,2),(1,100,2024,20,2),
 (2,100,2023, 5,3),                    -- second row for book 100 in its first year
 (1,200,2025, 7,1);

-- Solution: RANK keeps every row tied for the earliest year
SELECT book_id, yr AS first_year, loans, fee
FROM  (SELECT *, rank() OVER (PARTITION BY book_id ORDER BY yr) AS rk FROM loan_stats) t
WHERE  rk = 1
ORDER  BY book_id, loans DESC;

-- Pitfall: ROW_NUMBER (or DISTINCT ON) silently drops one of the 2023 rows of book 100
SELECT DISTINCT ON (book_id) book_id, yr AS first_year, loans, fee
FROM   loan_stats
ORDER  BY book_id, yr;
```

Output:

```
 book_id | first_year | loans | fee
---------+------------+-------+-----
     100 |       2023 |    12 |   2
     100 |       2023 |     5 |   3
     200 |       2025 |     7 |   1
(3 rows)

 book_id | first_year | loans | fee
---------+------------+-------+-----
     100 |       2023 |    12 |   2
     200 |       2025 |     7 |   1
(2 rows)
```

Why it works. `rank() OVER (PARTITION BY book_id ORDER BY yr)` gives rank 1 to *every* row in the earliest year, so both 2023 rows of book 100 survive. The classic equivalent is "find `min(yr)` per book, then join back on (book, year)".

Edge case. The second query is the tempting shortcut and it is wrong for this data: `DISTINCT ON` (like `ROW_NUMBER`) keeps exactly one row per book, losing branch 2's 2023 figures. Whenever "first" can tie, prefer `RANK` or the join-back.

19.3.9 Busiest collaborators (all ties)

Problem. Each row says one person invited another to a project and it was accepted. Count, for every person, the distinct projects they were part of (as inviter *or* invitee) and return everyone tied for the highest count.

`code/12_busiest_collaborators.sql`

```
CREATE TABLE collabs (inviter int, invitee int, project_id int, accepted_on date,
                      PRIMARY KEY (inviter, invitee, project_id));
INSERT INTO collabs VALUES
 (1,2,900,'2026-02-01'),(1,3,900,'2026-02-02'),(2,3,901,'2026-02-03'),
 (4,1,902,'2026-02-04'),(3,4,903,'2026-02-05'),(1,4,903,'2026-02-06');

WITH involvement AS (
  SELECT inviter AS person, project_id FROM collabs
  UNION                                    -- UNION also removes duplicate (person, project)
  SELECT invitee, project_id FROM collabs
), counts AS (
  SELECT person, count(*) AS projects,
         rank() OVER (ORDER BY count(*) DESC) AS rk
  FROM   involvement
  GROUP  BY person
)
SELECT person, projects FROM counts WHERE rk = 1 ORDER BY person;
```

Output:

```
 person | projects
--------+----------
      1 |        3
      3 |        3
(2 rows)
```

Why it works. Two `SELECT`s turn each row into two "person took part in project" facts; `UNION` removes repeats (person 1 twice in project 900). A window `rank()` over the grouped counts marks every top scorer with 1 — persons 1 and 3, three projects each.

Alternative. `WHERE projects = (SELECT max(projects) FROM counts)` is equally correct and reads well; `ORDER BY ... LIMIT 1` would be wrong because it drops ties.

19.4 Running totals and row-to-row comparisons

A running total is `SUM(...) OVER (PARTITION BY group ORDER BY time)`; "compared with last time" is `LAG` (previous row's value; `LEAD` the next row's). Frames: §8.4, running totals: §8.7.

19.4.1 Running total per team

Problem. Each row is one player's points for a team on a day ((team, day) is unique). Show, for each team and day, the team's cumulative points up to and including that day.

`code/13_running_total_by_team.sql`

```
CREATE TABLE team_points (player text, team char(1), day date, points int, PRIMARY KEY (team, day));
INSERT INTO team_points VALUES
 ('Lia','A','2026-05-01',10),('Max','B','2026-05-01',7),
 ('Noa','A','2026-05-03',4), ('Oli','B','2026-05-02',9),
 ('Pia','A','2026-05-06',6);

SELECT team, day,
       sum(points) OVER (PARTITION BY team ORDER BY day) AS running_total
FROM   team_points
ORDER  BY team, day;
```

Output (the file's alternative returns the same rows):

```
 team |    day     | running_total
------+------------+---------------
 A    | 2026-05-01 |            10
 A    | 2026-05-03 |            14
 A    | 2026-05-06 |            20
 B    | 2026-05-01 |             7
 B    | 2026-05-02 |            16
(5 rows)
```

Why it works. `PARTITION BY team` restarts the sum per team; `ORDER BY day` makes it cumulative. With an `ORDER BY` and no explicit frame the default is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`.

Alternative / edge case. A self-join summing rows of the same team with `day <=` this day (in the file) works on old databases but is O(n²). If days could repeat, the default `RANGE` frame adds all same-day peers at once; use `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` for row-by-row totals.

19.4.2 Last person who fits in the lift

Problem. People queue for a lift with a 500 kg limit; each has a weight and a queue position (`turn`). People board strictly in order. Return the last person who can board without the total exceeding the limit (assume the first person always fits).

`code/14_lift_capacity.sql`

```
CREATE TABLE lift_queue (person_id int PRIMARY KEY, name text, kg int, turn int UNIQUE);
INSERT INTO lift_queue VALUES
 (1,'Rae',90,3),(2,'Sid',120,1),(3,'Tom',200,2),(4,'Uma',60,5),(5,'Vik',85,4);

WITH boarding AS (
  SELECT name, turn, sum(kg) OVER (ORDER BY turn) AS load_kg
  FROM   lift_queue
)
SELECT name, load_kg
FROM   boarding
WHERE  load_kg <= 500
ORDER  BY turn DESC
LIMIT  1;
```

Output:

```
 name | load_kg
------+---------
 Vik  |     495
(1 row)
```

Why it works. The running sum in queue order gives the load after each person boards: 120, 320, 410, 495, 555. Keep loads ≤ 500 and take the latest in the queue — Vik. Weights are positive, so the sum only grows and "latest turn" equals "largest load".

Alternative. A correlated subquery (in the file) recomputes each prefix sum — correct, O(n²), and the answer to "how would you do this without window functions?".

19.4.3 Performance and improvement rate

Problem. Review rounds per person (round number, score 1–100). Report, per person, (a) *performance* = average of score ÷ round, and (b) *improvement rate* = percentage of that person's reviews where the score beat the previous round's score **and** is above 50. Round both to 2 decimals.

`code/15_review_trends.sql`

```
CREATE TABLE review_rounds (person text, rnd int, score int);
INSERT INTO review_rounds VALUES
 ('Kim',1,40),('Kim',2,55),('Kim',3,70),('Kim',4,65),
 ('Lou',1,80),('Lou',2,45);

WITH prev AS (
  SELECT person, rnd, score,
         lag(score) OVER (PARTITION BY person ORDER BY rnd) AS prev_score
  FROM   review_rounds
)
SELECT person,
       round(avg(score::numeric / rnd), 2)                              AS performance,
       round(100.0 * count(*) FILTER (WHERE score > 50 AND score > prev_score)
             / count(*), 2)                                             AS improvement_pct
FROM   prev
GROUP  BY person
ORDER  BY person;
```

Output:

```
 person | performance | improvement_pct
--------+-------------+-----------------
 Kim    |       26.77 |           50.00
 Lou    |       51.25 |            0.00
(2 rows)
```

Why it works. `lag(score)` puts the previous round's score on each row (NULL for round 1, and `score > NULL` is not true, so round 1 never counts). `count(*) FILTER (...)` counts qualifying rows; multiplying by `100.0` first keeps the maths in `numeric`.

> **Common mistake.** In PostgreSQL `55 / 2` is `27`: integer division truncates, and `avg(score / rnd)` gives Kim 26.50 instead of 26.77 (shown in the file). Cast one side first, as in Chapter 18, P34.

19.5 Gaps and islands: consecutive rows and streaks

An *island* is a maximal run of rows that belong together in order (consecutive ids, days or wins); the *gaps* are the breaks. If values rise by exactly 1 inside a run, `value - ROW_NUMBER()` is **constant** inside it and jumps at every gap, so grouping by it gives one group per island.

id	visitors ≥ 100?	row_number among busy rows	id − row_number
1	yes	1	0
3	yes	2	1
4	yes	3	1
5	yes	4	1
7	yes	5	2

19.5.1 Runs of busy days at the gate

Problem. Daily visitor counts with an increasing id. Return every row that belongs to a run of three or more *consecutive ids* where each row has at least 100 visitors, ordered by date.

`code/16_busy_gate_runs.sql`

```
CREATE TABLE gate_counts (id int PRIMARY KEY, visit_day date UNIQUE, visitors int);
INSERT INTO gate_counts VALUES
 (1,'2026-07-01',150),(2,'2026-07-02', 90),(3,'2026-07-03',120),(4,'2026-07-04',130),
 (5,'2026-07-05',101),(6,'2026-07-06', 20),(7,'2026-07-07',300),(8,'2026-07-08',310);

WITH busy AS (
  SELECT id, visit_day, visitors,
         id - row_number() OVER (ORDER BY id) AS island
  FROM   gate_counts
  WHERE  visitors >= 100
), sized AS (
  SELECT *, count(*) OVER (PARTITION BY island) AS run_len FROM busy
)
SELECT id, visit_day, visitors, island, run_len
FROM   sized
WHERE  run_len >= 3
ORDER  BY visit_day;
```

Output:

```
 id | visit_day  | visitors | island | run_len
----+------------+----------+--------+---------
  3 | 2026-07-03 |      120 |      1 |       3
  4 | 2026-07-04 |      130 |      1 |       3
  5 | 2026-07-05 |      101 |      1 |       3
(3 rows)
```

Why it works. Filtering first leaves only busy rows; `id - row_number()` labels ids 3, 4, 5 with island 1, while id 1 (island 0) and ids 7–8 (island 2) are too short. `count(*) OVER (PARTITION BY island)` attaches each island's length to its rows, so no join-back is needed.

Alternative. Group by `island` with `HAVING count(*) >= 3` and join those islands back to the rows — the same idea in two steps. A `LAG`/`LEAD` solution needs three cases (row is first, middle or last of a run) and gets messy.

19.5.2 Values repeated three times in a row

Problem. A sequence of readings ordered by id (ids may have gaps). Return every value that occurs at least three times consecutively.

`code/17_triple_repeats.sql`

```sql
CREATE TABLE readings (id int PRIMARY KEY, val int);
INSERT INTO readings VALUES (1,5),(2,5),(3,5),(4,2),(5,7),(7,7),(9,7),(10,5),(11,5);

-- Solution: compare each row with its neighbours (ids have gaps, order still defines "in a row")
SELECT DISTINCT val
FROM  (SELECT val,
              lag(val)  OVER (ORDER BY id) AS before,
              lead(val) OVER (ORDER BY id) AS after
       FROM   readings) t
WHERE  val = before AND val = after
ORDER  BY val;

-- Alternative: islands via the difference of two ROW_NUMBERs, works for any run length k
SELECT DISTINCT val
FROM  (SELECT val,
              row_number() OVER (ORDER BY id)
            - row_number() OVER (PARTITION BY val ORDER BY id) AS island
       FROM   readings) t
GROUP  BY val, island
HAVING count(*) >= 3
ORDER  BY val;
```

Output (all queries return these rows):

```
 val
-----
   5
   7
(2 rows)
```

Why it works. `lag` and `lead` expose each row's neighbours in id order; a row equal to both is the middle of a run of three or more. The 7s sit at ids 5, 7, 9 — id gaps don't matter, because "consecutive" means consecutive *in the ordering*.

Alternative. Overall position minus position among equal values is constant within each run; `GROUP BY` it with `HAVING count(*) >= k` works for any k. The two versions disagree on NULLs: three NULLs in a row fail `val = before` but form one island (`PARTITION BY val` groups NULLs), so decide whether a missing reading can repeat.

19.5.3 Members active five days in a row

Problem. A login log (duplicates possible, several logins per day) and a members table. Return members who logged in on five or more consecutive calendar days, ordered by id.

`code/18_five_day_streaks.sql`

```sql
CREATE TABLE members (member_id int PRIMARY KEY, name text);
CREATE TABLE logins  (member_id int, login_day date);          -- duplicates allowed
INSERT INTO members VALUES (1,'Ana'),(2,'Bo'),(3,'Cy');
INSERT INTO logins VALUES
 (1,'2026-06-01'),(1,'2026-06-02'),(1,'2026-06-02'),(1,'2026-06-03'),(1,'2026-06-04'),(1,'2026-06-05'),
 (2,'2026-06-01'),(2,'2026-06-02'),(2,'2026-06-04'),(2,'2026-06-05'),(2,'2026-06-06'),
 (3,'2026-06-28'),(3,'2026-06-29'),(3,'2026-06-30'),(3,'2026-07-01'),(3,'2026-07-02');

WITH days AS (SELECT DISTINCT member_id, login_day FROM logins),
islands AS (
  SELECT member_id, login_day,
         login_day - (row_number() OVER (PARTITION BY member_id ORDER BY login_day))::int AS anchor
  FROM   days
)
SELECT m.member_id, m.name
FROM   islands i JOIN members m USING (member_id)
GROUP  BY m.member_id, m.name, i.anchor
HAVING count(*) >= 5
ORDER  BY m.member_id;

-- Alternative: the 5th distinct day after this one is exactly 4 days later
SELECT DISTINCT member_id
FROM  (SELECT member_id, login_day,
              lead(login_day, 4) OVER (PARTITION BY member_id ORDER BY login_day) AS day5
       FROM  (SELECT DISTINCT member_id, login_day FROM logins) d) t
WHERE  day5 = login_day + 4
ORDER  BY member_id;
```

Output:

```
 member_id | name
-----------+------
         1 | Ana
         3 | Cy
(2 rows)

 member_id
-----------
         1
         3
(2 rows)
```

Why it works. First reduce to *distinct* days — duplicates would break the numbering. Then `login_day - row_number` (a date minus an integer is a date in PostgreSQL) gives the same "anchor" date for every day in a streak. Cy's streak crosses June → July and still works, because date arithmetic knows month lengths.

Alternative. `lead(login_day, 4)` looks four distinct days ahead; if that day is exactly `login_day + 4`, the five days in between must all be present. It only handles a fixed length; islands also give each streak's length.

19.5.4 Interview candidates from contest medals

Problem. Each contest row stores the gold, silver and bronze winners; contest ids are consecutive. A coder qualifies if they reached the podium (any colour) in at least three back-to-back contests, **or** took gold at least three times overall, in any contests. Return candidates' names and emails.

`code/19_medal_candidates.sql`

```sql
CREATE TABLE contests (contest_id int PRIMARY KEY, gold int, silver int, bronze int);
CREATE TABLE coders   (coder_id int PRIMARY KEY, name text, email text);
INSERT INTO coders VALUES (1,'Mia','mia@example.com'),(2,'Ned','ned@example.com'),
                          (3,'Oz','oz@example.com'),(4,'Pat','pat@example.com');
INSERT INTO contests VALUES
 (51,1,2,3),(52,2,1,4),(53,3,4,1),   -- Mia medals in 51,52,53
 (54,4,3,2),(55,4,2,3),(56,4,3,1);   -- Pat wins gold 3 times

WITH medals AS (
  SELECT contest_id, m.coder_id, m.kind
  FROM   contests c
  CROSS  JOIN LATERAL (VALUES (c.gold,'gold'),(c.silver,'silver'),(c.bronze,'bronze')) AS m(coder_id, kind)
), runs AS (
  SELECT coder_id
  FROM  (SELECT coder_id, contest_id - row_number() OVER (PARTITION BY coder_id ORDER BY contest_id) AS island
         FROM medals) t
  GROUP  BY coder_id, island
  HAVING count(*) >= 3
), golds AS (
  SELECT coder_id FROM medals WHERE kind = 'gold'
  GROUP  BY coder_id HAVING count(*) >= 3
)
SELECT name, email
FROM   coders
WHERE  coder_id IN (SELECT coder_id FROM runs UNION SELECT coder_id FROM golds)
ORDER  BY name;
```

Output:

```
 name |      email
------+-----------------
 Mia  | mia@example.com
 Oz   | oz@example.com
 Pat  | pat@example.com
(3 rows)
```

Why it works. `CROSS JOIN LATERAL (VALUES ...)` (`VALUES` builds a small inline table; `LATERAL` lets it refer to columns of the current row `c`) unpivots the medal columns into one row per (contest, coder, kind). Rule 1 is an island on `contest_id - row_number()` per coder; rule 2 is `GROUP BY ... HAVING count(*) >= 3` on golds; `IN (... UNION ...)` combines them. Mia medals in 51–53, Oz in 53–56, Pat has three golds; Ned's medals (51, 52, 54, 55) break at 53.

Edge case. A coder cannot win two medals in the same contest here; if they could, deduplicate `(coder_id, contest_id)` before numbering, or the island arithmetic breaks.

19.5.5 Longest winning streak

Problem. Match results per player (`win`, `draw`, `loss`) by date. For every player, return the length of their longest run of consecutive wins — 0 for a player who never won.

`code/20_longest_win_streak.sql`

```sql
CREATE TABLE games (player_id int, played_on date, outcome text CHECK (outcome IN ('win','draw','loss')),
                    PRIMARY KEY (player_id, played_on));
INSERT INTO games VALUES
 (1,'2026-01-01','win'),(1,'2026-01-02','win'),(1,'2026-01-03','loss'),
 (1,'2026-01-04','win'),(1,'2026-01-05','win'),(1,'2026-01-06','win'),
 (2,'2026-01-01','draw'),(2,'2026-01-02','win'),
 (3,'2026-01-01','loss');

WITH tagged AS (
  SELECT player_id, outcome,
         row_number() OVER (PARTITION BY player_id ORDER BY played_on)
       - row_number() OVER (PARTITION BY player_id, outcome ORDER BY played_on) AS island
  FROM   games
), win_runs AS (
  SELECT player_id, count(*) AS run_len
  FROM   tagged
  WHERE  outcome = 'win'
  GROUP  BY player_id, island
)
SELECT p.player_id, coalesce(max(w.run_len), 0) AS longest_streak
FROM  (SELECT DISTINCT player_id FROM games) p
LEFT  JOIN win_runs w USING (player_id)
GROUP  BY p.player_id
ORDER  BY p.player_id;

-- Alternative: a running count of non-wins gives every streak its own label
SELECT player_id, max(len) AS longest_streak
FROM  (SELECT player_id, breaks, count(*) FILTER (WHERE outcome = 'win') AS len
       FROM  (SELECT player_id, outcome,
                     count(*) FILTER (WHERE outcome <> 'win')
                       OVER (PARTITION BY player_id ORDER BY played_on) AS breaks
              FROM games) g
       GROUP  BY player_id, breaks) s
GROUP  BY player_id
ORDER  BY player_id;
```

Output (all queries return these rows):

```
 player_id | longest_streak
-----------+----------------
         1 |              3
         2 |              1
         3 |              0
(3 rows)
```

Why it works. Row number over all games minus row number within the same outcome is constant inside each run of equal outcomes. Counting `win` rows per (player, island) gives each streak's length and `max` the longest; the `LEFT JOIN` from all players plus `coalesce(..., 0)` keeps player 3, who never won.

Alternative. A running count of non-wins (`count(*) FILTER (WHERE outcome <> 'win') OVER (...)`) stays the same across a winning streak and bumps at each draw or loss, so it also labels streaks. Grouping by it and counting wins returns 0 naturally for groups without wins.

19.5.6 Deposits over the limit two months running

Problem. Accounts have a monthly income limit. A movements table holds deposits (`in`) and withdrawals (`out`). Flag accounts whose total deposits in a calendar month exceed the limit in at least two consecutive months.

`code/21_income_over_limit.sql`

```
CREATE TABLE accounts  (account_id int PRIMARY KEY, monthly_limit int);
CREATE TABLE movements (movement_id int PRIMARY KEY, account_id int, direction text, amount int, at timestamp);
INSERT INTO accounts VALUES (1,1000),(2,500),(3,800);
INSERT INTO movements VALUES
 (1,1,'in', 700,'2026-01-05'),(2,1,'in', 400,'2026-01-20'),   -- Jan 1100 > 1000
 (3,1,'in',1200,'2026-02-11'),                                -- Feb 1200 > 1000 -> flagged
 (4,2,'in', 600,'2026-01-03'),(5,2,'in', 700,'2026-03-03'),   -- Jan, Mar: not consecutive
 (6,2,'out',900,'2026-02-03'),                                -- withdrawals don't count
 (7,3,'in', 900,'2025-12-30'),(8,3,'in', 950,'2026-01-02');   -- Dec -> Jan crosses a year

WITH monthly AS (
  SELECT m.account_id, date_trunc('month', m.at)::date AS mon
  FROM   movements m JOIN accounts a USING (account_id)
  WHERE  m.direction = 'in'
  GROUP  BY m.account_id, date_trunc('month', m.at), a.monthly_limit
  HAVING sum(m.amount) > a.monthly_limit
)
SELECT DISTINCT account_id
FROM  (SELECT account_id, mon,
              lag(mon) OVER (PARTITION BY account_id ORDER BY mon) AS prev_mon
       FROM monthly) t
WHERE  prev_mon = mon - interval '1 month'
ORDER  BY account_id;
```

Output:

```
 account_id
------------
          1
          3
(2 rows)
```

Why it works. `date_trunc('month', at)` buckets movements by month and `HAVING` keeps over-limit months. `lag(mon)` brings the previous over-limit month alongside, so "consecutive" is `prev_mon = mon - interval '1 month'`. Account 3 is caught across December → January, which a naive "month number minus one" would miss.

Edge case. Account 2 is over in January and March but not February, so it is not flagged; its February withdrawal is ignored by `direction = 'in'`. For "N consecutive months" use the island trick on month numbers (`year * 12 + month`).

19.6 Pivot and unpivot (static and dynamic)

Pivot = turn distinct values of a column into separate columns; *unpivot* = the reverse. SQL needs the output columns fixed when a query is planned, so a pivot whose columns depend on the data requires **dynamic SQL**: a query that writes another query, which is then executed.

19.6.1 Names by region, side by side

Problem. A list of attendees (name, region; duplicates possible) from America, Asia and Europe. Produce three columns, `america`, `asia`, `europe`, each listing that region's names alphabetically from the top, with NULL where a column runs out.

`code/22_names_by_region.sql`

```sql
CREATE TABLE attendees (name text, region text);
INSERT INTO attendees VALUES
 ('Zoe','Europe'),('Ali','Asia'),('Ben','America'),('Kai','Asia'),
 ('Ada','Europe'),('Liv','Europe'),('Ben','America');     -- duplicate row kept

WITH numbered AS (
  SELECT name, region,
         row_number() OVER (PARTITION BY region ORDER BY name) AS rn
  FROM   attendees
)
SELECT max(name) FILTER (WHERE region = 'America') AS america,
       max(name) FILTER (WHERE region = 'Asia')    AS asia,
       max(name) FILTER (WHERE region = 'Europe')  AS europe
FROM   numbered
GROUP  BY rn
ORDER  BY rn;
```

Output:

```
 america |  asia  | europe
---------+--------+--------
 Ben     | Ali    | Ada
 Ben     | Kai    | Liv
 (null)  | (null) | Zoe
(3 rows)
```

Why it works. `row_number()` per region gives each name its line in its column. Grouping by that number puts every region's 1st name on row 1, 2nd names on row 2, and so on; `max(name) FILTER (...)` picks the single name per region on that line.

Alternative. `max(CASE WHEN region = 'Asia' THEN name END)` is the portable form of `FILTER` (MySQL and SQL Server lack `FILTER`). Duplicate rows stay duplicates (Ben twice); add `DISTINCT` in the inner query if the question wants unique names.

19.6.2 Dynamic pivot: one column per store

Problem. `shelf_prices(product_id, store, price)`. Produce one row per product with one column per store — columns sorted by store name, NULL where the product isn't sold — for whatever stores exist when the code runs.

`code/23_dynamic_pivot.sql`

```
SET client_min_messages = warning;   -- hide "view does not exist, skipping" notices
CREATE TABLE shelf_prices (product_id int, store text, price int, PRIMARY KEY (product_id, store));
INSERT INTO shelf_prices VALUES
 (1,'north',10),(1,'west',12),
 (2,'east', 7),(2,'north',8),(2,'west',9),
 (3,'west', 30);

-- Approach A: a procedure writes the SELECT with format() and (re)creates a view
CREATE PROCEDURE pivot_shelf_prices()
LANGUAGE plpgsql AS $$
DECLARE
  col_list text;
BEGIN
  SELECT string_agg(format('max(price) FILTER (WHERE store = %L) AS %I', store, store),
                    ', ' ORDER BY store)
  INTO   col_list
  FROM  (SELECT DISTINCT store FROM shelf_prices) s;

  DROP VIEW IF EXISTS shelf_prices_wide;       -- column set may change, so rebuild
  EXECUTE format('CREATE VIEW shelf_prices_wide AS
                  SELECT product_id, %s FROM shelf_prices GROUP BY product_id', col_list);
END $$;

CALL pivot_shelf_prices();
SELECT * FROM shelf_prices_wide ORDER BY product_id;

-- A new store appears: re-run the procedure and the view grows a column
INSERT INTO shelf_prices VALUES (3,'central',28);
CALL pivot_shelf_prices();
SELECT * FROM shelf_prices_wide ORDER BY product_id;

-- Approach B (psql only): build the query text, then \gexec runs it
SELECT format('SELECT product_id, %s FROM shelf_prices GROUP BY product_id ORDER BY product_id',
              string_agg(format('max(price) FILTER (WHERE store = %L) AS %I', store, store),
                         ', ' ORDER BY store))
FROM  (SELECT DISTINCT store FROM shelf_prices) s
\gexec
```

Output (first call, then after `central` appears; Approach B and the `crosstab` version in the file print the same four-store table):

```
 product_id |  east  | north  | west
------------+--------+--------+------
          1 | (null) |     10 |   12
          2 |      7 |      8 |    9
          3 | (null) | (null) |   30
(3 rows)

 product_id | central |  east  | north  | west
------------+---------+--------+--------+------
          1 |  (null) | (null) |     10 |   12
          2 |  (null) |      7 |      8 |    9
          3 |      28 | (null) | (null) |   30
(3 rows)
```

Why it works.

1. `string_agg(format(...), ', ' ORDER BY store)` builds the column list, e.g. `max(price) FILTER (WHERE store = 'east') AS east, ...`. `%L` quotes a **literal** (a data value such as `'east'`) and `%I` an **identifier** (a name such as a column), so a store named `O'Neil's` or `Store 1` cannot break the SQL or inject code (§14.9).
2. `EXECUTE` runs the generated `CREATE VIEW`. A PL/pgSQL function must declare its result columns in advance — exactly what we don't know — so the procedure publishes a view that a plain `SELECT *` reads.
3. When `central` appears, calling the procedure again rebuilds the view with a new column. `DROP VIEW` first, because `CREATE OR REPLACE VIEW` can only *append* columns, not insert one in the middle. On an

empty table `string_agg` returns NULL and the generated `SELECT product_id, FROM …` is invalid, so real code should check for that first.

Alternatives. In `psql`, `\gexec` runs each value the previous query returns as SQL — the quickest ad-hoc dynamic pivot. `crosstab()` (from the `tablefunc` extension, in the file) still needs the output columns spelled out in `AS ct(...)`. In MySQL the equivalent is `GROUP_CONCAT` to build the column list plus `PREPARE` / `EXECUTE` (watch `group_concat_max_len`); SQL Server has `PIVOT` plus `sp_executesql`.

19.6.3 Dynamic unpivot: store columns back to rows

Problem. `price_grid(product_id, <one int column per store>)`, where the store columns can change and NULL means "not sold". Produce rows `(product_id, store, price)` with no rows for NULL prices.

`code/24_dynamic_unpivot.sql`

```
SET client_min_messages = warning;   -- hide "view does not exist, skipping" notices
CREATE TABLE price_grid (product_id int PRIMARY KEY, east int, north int, west int);
INSERT INTO price_grid VALUES (1,NULL,10,12),(2,7,8,9),(3,NULL,NULL,30);

-- Approach A: no dynamic SQL at all -- turn each row into JSON and explode its keys
SELECT g.product_id, kv.key AS store, kv.value::int AS price
FROM   price_grid g
CROSS  JOIN LATERAL jsonb_each_text(to_jsonb(g) - 'product_id') AS kv
WHERE  kv.value IS NOT NULL
ORDER  BY g.product_id, store;

-- Approach B: read the column names from the catalog and generate UNION ALL branches
CREATE PROCEDURE unpivot_price_grid()
LANGUAGE plpgsql AS $$
DECLARE
  branches text;
BEGIN
  SELECT string_agg(format('SELECT product_id, %L AS store, %I AS price FROM price_grid WHERE %I IS NOT NULL',
                           column_name, column_name, column_name),
                    ' UNION ALL ' ORDER BY column_name)
  INTO   branches
  FROM   information_schema.columns
  WHERE  table_schema = current_schema()
    AND  table_name   = 'price_grid'
    AND  column_name <> 'product_id';

  DROP VIEW IF EXISTS price_grid_long;
  EXECUTE 'CREATE VIEW price_grid_long AS ' || branches;
END $$;

CALL unpivot_price_grid();
SELECT * FROM price_grid_long ORDER BY product_id, store;
```

Output (all queries return these rows):

```
 product_id | store | price
------------+-------+-------
          1 | north |    10
          1 | west  |    12
          2 | east  |     7
          2 | north |     8
          2 | west  |     9
          3 | west  |    30
(6 rows)
```

Why it works.

- **Approach A** needs no dynamic SQL: `to_jsonb(g)` turns the row into `{"product_id":1,"east":null,...}`, `- 'product_id'` drops that key, and `jsonb_each_text` explodes the rest into (key, value) text pairs (hence `::int`). New columns are picked up automatically.
- **Approach B** reads column names from `information_schema.columns` (the standard catalog views), generates one `SELECT ... WHERE col IS NOT NULL` per column, joins them with `UNION ALL`, and `EXECUTE`s the result.

Edge case. With a fixed, known set of columns, the simplest unpivot is `CROSS JOIN LATERAL (VALUES ('east', east), ('north', north), ('west', west)) AS v(store, price)` — as used for medals in 19.5.4. Use `UNION ALL` (stack results, keeping duplicates), not `UNION` (stack and remove duplicates): `UNION` would wrongly merge identical rows and costs a sort.

19.7 Self-joins, "both directions" and graph-like questions

Friendship, calls and matches store a *pair* in one row, but the question asks about *each person*. The standard move is to "unfold" the pair with `UNION ALL` (or `UNION`) so each participant gets their own row, then aggregate normally. Relational division ("took *all* courses"; defined in 19.7.6) also lives here.

19.7.1 Page suggestions from friends' likes

Problem. `friendships(a, b)` stores each friendship once, in either column order. `page_likes(user_id, page_id)`. Suggest to user 1 every page liked by at least one friend and not already liked by user 1, without duplicates.

`code/25_page_suggestions.sql`

```sql
CREATE TABLE friendships (a int, b int, PRIMARY KEY (a, b));      -- stored once, either direction
CREATE TABLE page_likes  (user_id int, page_id int, PRIMARY KEY (user_id, page_id));
INSERT INTO friendships VALUES (1,2),(3,1),(1,4),(5,6);
INSERT INTO page_likes VALUES
 (1,77),(2,77),(2,88),(3,88),(3,99),(4,55),(5,66),(6,11);

WITH my_friends AS (
  SELECT b AS friend FROM friendships WHERE a = 1
  UNION
  SELECT a FROM friendships WHERE b = 1
)
SELECT DISTINCT l.page_id
FROM   page_likes l
JOIN   my_friends f ON f.friend = l.user_id
WHERE  NOT EXISTS (SELECT 1 FROM page_likes me
                   WHERE me.user_id = 1 AND me.page_id = l.page_id)
ORDER  BY l.page_id;

-- Alternative: set difference
SELECT page_id FROM page_likes
WHERE  user_id IN (SELECT b FROM friendships WHERE a = 1 UNION SELECT a FROM friendships WHERE b = 1)
EXCEPT
SELECT page_id FROM page_likes WHERE user_id = 1
ORDER  BY page_id;
```

Output (all queries return these rows):

```
 page_id
---------
      55
      88
      99
(3 rows)
```

Why it works. A friendship may be stored as (1, x) or (x, 1), so `my_friends` looks both ways (2, 3, 4). Joining likes to friends gives candidates; `NOT EXISTS` drops page 77, which user 1 already likes, and `DISTINCT` collapses page 88 (liked by two friends).

Alternative. `EXCEPT` (set difference) expresses "their pages minus my pages" directly and also removes duplicates. Avoid `page_id NOT IN (SELECT ...)` if the subquery column can be NULL — one NULL makes the whole condition unknown and returns nothing.

19.7.2 Popularity percentage

Problem. Friend pairs, stored once each. For every user, popularity = number of friends ÷ number of distinct users on the platform × 100, rounded to 2 decimals. Order by user.

`code/26_friend_share.sql`

```sql
CREATE TABLE friend_pairs (u1 int, u2 int, PRIMARY KEY (u1, u2));
INSERT INTO friend_pairs VALUES (1,2),(1,3),(2,3),(4,1),(5,6);

WITH both_ways AS (
  SELECT u1 AS person, u2 AS friend FROM friend_pairs
  UNION
  SELECT u2, u1 FROM friend_pairs
)
SELECT person,
       round(100.0 * count(*) / (SELECT count(DISTINCT person) FROM both_ways), 2) AS popularity_pct
FROM   both_ways
GROUP  BY person
ORDER  BY person;
```

Output:

```
 person | popularity_pct
--------+----------------
      1 |          50.00
      2 |          33.33
      3 |          33.33
      4 |          16.67
      5 |          16.67
      6 |          16.67
(6 rows)
```

Why it works. Unfolding each pair both ways makes "my friends" a simple `GROUP BY person` count. The denominator is the count of distinct people in the unfolded set (users 1–6). `100.0 *` at the front keeps everything numeric.

> **Common mistake.** `3 / 6 * 100` is `0` in PostgreSQL (the file shows it), because `3 / 6` truncates first; put `100.0` first. If some users have no friends, take the denominator from a `users` table instead.

19.7.3 Same person first and last in the day

Problem. Calls store caller, callee and time. For each user and each day, look at the first and last call they took part in (as caller *or* callee). Return users for whom, on at least one day, both of those calls were with the same other person.

`code/27_same_first_last_contact.sql`

```
CREATE TABLE phone_calls (caller int, callee int, called_at timestamp, PRIMARY KEY (caller, callee, called_at));
INSERT INTO phone_calls VALUES
 (1,2,'2026-04-01 08:00'),(3,1,'2026-04-01 12:00'),(2,1,'2026-04-01 20:00'),  -- user 1: 2 ... 2
 (4,5,'2026-04-02 09:00'),(4,6,'2026-04-02 18:00'),                          -- user 4: 5 ... 6
 (7,8,'2026-04-03 10:00');                                                    -- one call: first = last

WITH legs AS (                          -- every call seen from both participants
  SELECT caller AS person, callee AS other, called_at FROM phone_calls
  UNION ALL
  SELECT callee, caller, called_at FROM phone_calls
), ends AS (
  SELECT person, called_at::date AS day, other,
         row_number() OVER (PARTITION BY person, called_at::date ORDER BY called_at)      AS from_start,
         row_number() OVER (PARTITION BY person, called_at::date ORDER BY called_at DESC) AS from_end
  FROM   legs
)
SELECT DISTINCT person
FROM   ends
GROUP  BY person, day
HAVING max(other) FILTER (WHERE from_start = 1) = max(other) FILTER (WHERE from_end = 1)
ORDER  BY person;
```

Output:

```
 person
--------
      1
      2
      3
      5
      6
      7
      8
(7 rows)
```

Why it works. `legs` shows every call from both sides, so `other` is always the other party. Ascending and descending `row_number()`s per (person, day) tag the first and last call; the two `max(other) FILTER (...)` extract those partners and `HAVING` compares them. User 1 talked to 2 first and last on 1 April (with 3 in between), and so did user 2 (both calls with 1); user 4 did not.

Edge case. A person with a single call that day has the same call as first and last, so they qualify — that is why 3, 5, 6, 7 and 8 appear. Confirm with the interviewer whether that counts; if not, add `HAVING count(*) > 1`.

19.7.4 League table

Problem. Clubs, plus fixtures with home/away clubs and goals. Award 3 points for a win, 1 for a draw, 0 for a loss. List every club with its points, highest first, ties by club id.

`code/28_league_table.sql`

```
CREATE TABLE clubs    (club_id int PRIMARY KEY, club text);
CREATE TABLE fixtures (fixture_id int PRIMARY KEY, home int, away int, home_goals int, away_goals int);
INSERT INTO clubs VALUES (1,'Rovers'),(2,'United'),(3,'Athletic'),(4,'Wanderers');
INSERT INTO fixtures VALUES (1,1,2,2,0),(2,3,1,1,1),(3,2,3,3,1),(4,1,3,0,1);

WITH per_club AS (                    -- one row per (club, match), from that club's side
  SELECT home AS club_id, home_goals AS gf, away_goals AS ga FROM fixtures
  UNION ALL
  SELECT away, away_goals, home_goals FROM fixtures
)
SELECT c.club_id, c.club,
       coalesce(sum(CASE WHEN p.gf > p.ga THEN 3 WHEN p.gf = p.ga THEN 1 ELSE 0 END), 0) AS points
FROM   clubs c
LEFT   JOIN per_club p USING (club_id)
GROUP  BY c.club_id, c.club
ORDER  BY points DESC, c.club_id;
```

Output:

```
 club_id |   club    | points
---------+-----------+--------
       1 | Rovers    |      4
       3 | Athletic  |      4
       2 | United    |      3
       4 | Wanderers |      0
(4 rows)
```

Why it works. `per_club` rewrites each match twice, once from each club's point of view (goals for, goals against), so one `CASE` scores every row. `LEFT JOIN` from `clubs` plus `coalesce(..., 0)` keeps Wanderers, who played nothing.

Alternative. Joining with `ON club_id = home OR club_id = away` and a bigger `CASE` (which side is the club?) also works, but `OR` joins use indexes poorly. Unfolding first is cleaner and extends to goal difference.

19.7.5 Departments with two above-average earners

Problem. Payroll rows (id, name, department, salary). Return each department that has two or more people paid above that department's mean salary.

`code/29_depts_with_high_earners.sql`

```
CREATE TABLE payroll (emp_id int PRIMARY KEY, name text, dept text, salary int);
INSERT INTO payroll VALUES
 (1,'A','Ops',50),(2,'B','Ops',60),(3,'C','Ops',70),(4,'D','Ops',80),   -- avg 65: C, D above
 (5,'E','Law',90),(6,'F','Law',40),(7,'G','Law',41),                    -- avg 57: only E
 (8,'H','Art',30);

SELECT dept
FROM  (SELECT dept, salary, avg(salary) OVER (PARTITION BY dept) AS dept_avg FROM payroll) t
WHERE  salary > dept_avg
GROUP  BY dept
HAVING count(*) >= 2
ORDER  BY dept;
```

Output (the file's alternative returns the same rows):

```
 dept
------
 Ops
(1 row)
```

Why it works. `avg(salary) OVER (PARTITION BY dept)` attaches each department's average to every employee without grouping them away. Filter rows above that average, then group by department and keep those with two or more. Ops (avg 65) has C and D; Law (avg 57) only has E.

Alternative. A correlated subquery (in the file) computes the department average per row; a third route joins `payroll` to itself on department, groups by employee and keeps `HAVING p.salary > avg(q.salary)`. The window version reads the table once.

19.7.6 Students enrolled in every course

Problem. A course catalogue and an enrolment log (duplicates possible). Return students enrolled in every course in the catalogue. This is *relational division*: "which X are related to **all** Y?".

`code/30_took_every_course.sql`

```sql
CREATE TABLE catalogue   (course_id int PRIMARY KEY);
CREATE TABLE enrolments  (student_id int NOT NULL, course_id int REFERENCES catalogue);   -- duplicates possible
INSERT INTO catalogue VALUES (1),(2),(3);
INSERT INTO enrolments VALUES (10,1),(10,2),(10,3),(11,1),(11,1),(11,2),(12,3),(12,2),(12,1),(12,1);

SELECT student_id
FROM   enrolments
GROUP  BY student_id
HAVING count(DISTINCT course_id) = (SELECT count(*) FROM catalogue)
ORDER  BY student_id;

-- Alternative: "no course exists that this student did not take" (double NOT EXISTS)
SELECT DISTINCT e.student_id
FROM   enrolments e
WHERE  NOT EXISTS (SELECT 1 FROM catalogue c
                   WHERE NOT EXISTS (SELECT 1 FROM enrolments x
                                     WHERE x.student_id = e.student_id AND x.course_id = c.course_id))
ORDER  BY e.student_id;
```

Output (all queries return these rows):

```
 student_id
------------
         10
         12
(2 rows)
```

Why it works. Count each student's *distinct* courses and compare with the catalogue size. `DISTINCT` matters: student 11 has three enrolment rows but only two courses. The foreign key guarantees no course outside the catalogue can inflate the count.

Alternative. The double `NOT EXISTS` is the textbook formulation: "there is no course that this student has not taken". It does not depend on counting, so it stays correct even without the foreign key (enrolments in unknown courses would break the count version).

19.7.7 Shop revenue under two uniqueness rules

Problem. Shops with 2024 revenue, 2025 revenue and coordinates. Total the 2025 revenue of shops whose 2024 revenue equals at least one *other* shop's 2024 revenue **and** whose (lat, lon) is shared with *no* other shop. Round to 2 decimals.

`code/31_shop_revenue_filter.sql`

```sql
CREATE TABLE shops (shop_id int PRIMARY KEY, rev_2024 numeric(10,2), rev_2025 numeric(10,2),
                    lat numeric(8,5), lon numeric(8,5));
INSERT INTO shops VALUES
 (1,100.00,120.50,51.50000,-0.12000),
 (2,100.00,140.25,48.85000, 2.35000),
 (3,200.00,210.00,40.71000,-74.00000),
 (4,100.00,  90.00,40.71000,-74.00000),   -- same place as shop 3
 (5,300.00,330.00,35.68000,139.69000);

SELECT round(sum(rev_2025), 2) AS total_2025
FROM  (SELECT rev_2025,
              count(*) OVER (PARTITION BY rev_2024) AS same_rev,
              count(*) OVER (PARTITION BY lat, lon) AS same_spot
       FROM   shops) t
WHERE  same_rev > 1 AND same_spot = 1;
```

Output:

```
 total_2025
------------
     260.75
(1 row)
```

Why it works. Two window counts label each shop at once: how many shops share its 2024 revenue, and how many share its location. Keep `same_rev > 1 AND same_spot = 1`: shops 1 and 2 (shop 4 shares revenue but also shares a location with shop 3). 120.50 + 140.25 = 260.75.

Edge case. Equality on **floating-point** columns (approximate binary number types such as `float`) is fragile (0.1 + 0.2 ≠ 0.3 in binary floats), so values compared for equality belong in `numeric`. Grouping twice with `HAVING` and joining back gives the same answer in more steps.

19.8 Date bucketing and retention

Group by a *derived* date — the first day a user appeared, the calendar month, the booking day — and measure what happens inside each bucket. `generate_series` fills missing buckets. A **cohort** is a group of users who started in the same period; **retention** is the share of a cohort that comes back later.

19.8.1 Day-1 retention

Problem. Game plays per player per day. A player's *install day* is their first play day. For each install day, report the number of installs and *day-1 retention* = share of those players who also played the next day, rounded to 2 decimals.

`code/32_day1_retention.sql`

```
CREATE TABLE plays (player_id int, device_id int, play_day date, games int, PRIMARY KEY (player_id, play_day));
INSERT INTO plays VALUES
 (1,9,'2026-03-01',3),(1,9,'2026-03-02',1),
 (2,8,'2026-03-01',0),(2,8,'2026-03-04',2),
 (3,7,'2026-03-02',5),(3,7,'2026-03-03',1),
 (4,6,'2026-03-02',1);

WITH firsts AS (
  SELECT player_id, min(play_day) AS install_day FROM plays GROUP BY player_id
)
SELECT f.install_day,
       count(*) AS installs,
       round(avg((EXISTS (SELECT 1 FROM plays p
                          WHERE p.player_id = f.player_id
                            AND p.play_day = f.install_day + 1))::int), 2) AS day1_retention
FROM   firsts f
GROUP  BY f.install_day
ORDER  BY f.install_day;

-- Alternative: LEFT JOIN to the next day's row
SELECT f.install_day, count(*) AS installs,
       round(count(p.player_id)::numeric / count(*), 2) AS day1_retention
FROM  (SELECT player_id, min(play_day) AS install_day FROM plays GROUP BY player_id) f
LEFT  JOIN plays p ON p.player_id = f.player_id AND p.play_day = f.install_day + 1
GROUP  BY f.install_day
ORDER  BY f.install_day;
```

Output (all queries return these rows):

```
 install_day | installs | day1_retention
-------------+----------+----------------
 2026-03-01  |        2 |           0.50
 2026-03-02  |        2 |           0.50
(2 rows)
```

Why it works. `firsts` gives each player one install day. `EXISTS` is true when a play exists on `install_day + 1`; averaging the boolean cast `::int` gives the fraction retained (true → 1, false → 0). Player 2 returned on 4 March, not the next day, so does not count.

Alternative. A `LEFT JOIN` to the next-day row and `count(p.player_id) / count(*)` does the same (the primary key on (player, day) guarantees at most one match). Note the `::numeric` cast — without it the integer division returns 0.

19.8.2 Transactions per visit histogram

Problem. Bank visits (customer, day) and transactions (customer, day, amount; duplicates allowed; every transaction happens on a visit day). Build a **histogram** (a count of items per value or range): for each number k from 0 up to the largest per-visit count, how many visits had exactly k transactions. Include k values with zero visits.

`code/33_transactions_per_visit.sql`

```
CREATE TABLE branch_visits (customer_id int, visit_day date, PRIMARY KEY (customer_id, visit_day));
CREATE TABLE branch_txns   (customer_id int, txn_day date, amount int);   -- duplicates possible
INSERT INTO branch_visits VALUES (1,'2026-02-01'),(1,'2026-02-05'),(2,'2026-02-01'),(3,'2026-02-02'),(4,'2026-02-03');
INSERT INTO branch_txns VALUES
 (1,'2026-02-01',50),(1,'2026-02-01',50),(1,'2026-02-01',20),   -- 3 in one visit
 (2,'2026-02-01',10),                                           -- 1
 (3,'2026-02-02',99);                                           -- 1

WITH per_visit AS (
  SELECT v.customer_id, v.visit_day, count(t.txn_day) AS txns
  FROM   branch_visits v
  LEFT   JOIN branch_txns t ON t.customer_id = v.customer_id AND t.txn_day = v.visit_day
  GROUP  BY v.customer_id, v.visit_day
)
SELECT n AS transactions_count, count(p.txns) AS visits_count
FROM   generate_series(0, (SELECT max(txns) FROM per_visit)) AS n
LEFT   JOIN per_visit p ON p.txns = n
GROUP  BY n
ORDER  BY n;
```

Output:

```
 transactions_count | visits_count
--------------------+--------------
                  0 |            2
                  1 |            2
                  2 |            0
                  3 |            1
(4 rows)
```

Why it works. `per_visit` counts each visit's transactions (`count(t.txn_day)` makes visits with none count 0 after the `LEFT JOIN`). `generate_series(0, max)` produces every k, including 2, which no visit has; a `LEFT JOIN` then counts matching visits, giving 0 for the empty bucket.

Alternative. Without `generate_series` (e.g. MySQL), use a recursive CTE counting from 0 to the max, or `ROW_NUMBER()` over any big-enough table.

19.8.3 Titles borrowed per day

Problem. A borrowing log (date, title; duplicates possible). For each date, report how many distinct titles were borrowed and a comma-separated list of them in alphabetical order.

`code/34_titles_per_day.sql`

```
CREATE TABLE borrow_log (borrowed_on date, title text);          -- no key, duplicates allowed
INSERT INTO borrow_log VALUES
 ('2026-08-01','Dune'),('2026-08-01','Emma'),('2026-08-01','Dune'),
 ('2026-08-02','Ulysses'),('2026-08-02','Beloved'),('2026-08-03','Emma');

SELECT borrowed_on,
       count(DISTINCT title)                          AS titles,
       string_agg(DISTINCT title, ',' ORDER BY title)    AS title_list
FROM   borrow_log
GROUP  BY borrowed_on
ORDER  BY borrowed_on;
```

Output:

```
 borrowed_on | titles |   title_list
-------------+--------+----------------
 2026-08-01  |      2 | Dune,Emma
 2026-08-02  |      2 | Beloved,Ulysses
 2026-08-03  |      1 | Emma
(3 rows)
```

Why it works. `count(DISTINCT title)` ignores the second "Dune". `string_agg(DISTINCT title, ',' ORDER BY title)` concatenates unique titles in sorted order; PostgreSQL allows `DISTINCT` together with `ORDER BY` only when the ordering expression is the aggregated one, which it is here.

Dialect note. MySQL: `GROUP_CONCAT(DISTINCT title ORDER BY title SEPARATOR ',')` (limited by `group_concat_max_len`, default 1024 bytes). SQL Server: `STRING_AGG(title, ',') WITHIN GROUP (ORDER BY title)`, with no `DISTINCT` — deduplicate in a subquery first.

19.8.4 Same-day first bookings

Problem. Clinic bookings record when a booking was made and the day the patient wanted. A booking is *same-day* when those dates match. Considering only each patient's first booking (earliest booking date), what percentage were same-day? Round to 2 decimals.

`code/35_first_booking_same_day.sql`

```sql
CREATE TABLE bookings_clinic (booking_id int PRIMARY KEY, patient_id int, booked_on date, wanted_on date);
INSERT INTO bookings_clinic VALUES
 (1,1,'2026-09-01','2026-09-01'),(2,1,'2026-09-05','2026-09-09'),
 (3,2,'2026-09-02','2026-09-04'),
 (4,3,'2026-09-03','2026-09-03'),
 (5,4,'2026-09-10','2026-09-12'),(6,4,'2026-09-01','2026-09-01');

SELECT round(100.0 * avg((booked_on = wanted_on)::int), 2) AS same_day_pct
FROM  (SELECT DISTINCT ON (patient_id) booked_on, wanted_on
       FROM   bookings_clinic
       ORDER  BY patient_id, booked_on) first_bookings;
```

Output:

```
 same_day_pct
--------------
        75.00
(1 row)
```

Why it works. `DISTINCT ON (patient_id) ... ORDER BY patient_id, booked_on` keeps each patient's earliest booking (patient 4's is 1 September, although inserted last). `(booked_on = wanted_on)::int` is 1 or 0, so 100 × its average is the same-day share: 75.00.

Alternative. The "min then join back" version (`min(booked_on)` per patient, join on patient and date) works everywhere; if a patient could have two bookings on their first day it would count both, whereas `DISTINCT ON` picks one — know which the question wants.

19.9 Ratios and cancellation rates

Rates are `count(condition) / count(all)`. The details that go wrong: filtering *which rows are in the denominator*, integer division, and division by zero. `avg(boolean::int)` is a compact way to compute a share.

19.9.1 Ride cancellation rate (the "trips and users" classic)

Problem. Rides (rider, driver, status, request date) and users (with a `banned` flag). For each day from 1 to 3 October 2026, compute the share of requests that were cancelled (by rider or driver), counting only rides where **both** the rider and the driver are not banned. Round to 2 decimals.

`code/36_ride_cancellation_rate.sql`

```sql
CREATE TABLE app_users (user_id int PRIMARY KEY, banned boolean, role text);
CREATE TABLE rides     (ride_id int PRIMARY KEY, rider_id int, driver_id int, city_id int,
                        status text CHECK (status IN ('completed','cancelled_by_driver','cancelled_by_rider')),
                        requested_on date);
INSERT INTO app_users VALUES
 (1,false,'rider'),(2,true,'rider'),(3,false,'rider'),
 (10,false,'driver'),(11,false,'driver'),(12,true,'driver');
INSERT INTO rides VALUES
 (1,1,10,1,'completed',          '2026-10-01'),
 (2,2,10,1,'cancelled_by_driver','2026-10-01'),   -- banned rider: ignored
 (3,3,11,1,'cancelled_by_rider', '2026-10-01'),
 (4,1,12,1,'completed',          '2026-10-02'),   -- banned driver: ignored
 (5,3,10,1,'cancelled_by_driver','2026-10-02'),
 (6,1,11,1,'completed',          '2026-10-03'),
 (7,3,11,1,'cancelled_by_rider', '2026-10-04');   -- outside the window

SELECT r.requested_on AS day,
       round(avg((r.status <> 'completed')::int), 2) AS cancellation_rate
FROM   rides r
JOIN   app_users ru ON ru.user_id = r.rider_id  AND NOT ru.banned
JOIN   app_users du ON du.user_id = r.driver_id AND NOT du.banned
WHERE  r.requested_on BETWEEN '2026-10-01' AND '2026-10-03'
GROUP  BY r.requested_on
ORDER  BY day;
```

Output (the file's alternative returns the same rows):

```
    day     | cancellation_rate
------------+-------------------
 2026-10-01 |              0.50
 2026-10-02 |              1.00
 2026-10-03 |              0.00
(3 rows)
```

Why it works. Joining `app_users` once as rider and once as driver, with `NOT banned` in each `ON`, removes rides with any banned party from numerator and denominator alike. `avg((status <> 'completed')::int)` is the cancelled share. Ride 2 (banned rider) and ride 4 (banned driver) disappear, so 1 October is 1 of 2 cancelled.

Alternative / edge cases. The file's `NOT IN` + `FILTER` version is safe because `user_id` is a non-NULL primary key. Days with no eligible rides produce no row; drive the query from `generate_series` if they must appear. On a `timestamp` column `BETWEEN` would miss most of the last day — use `>= start AND < end + 1 day`.

19.9.2 Delivery fulfilment rate

Problem. Orders (order, customer) and delivery attempts (order, time, `delivered` or `failed`). An order's rate is delivered attempts ÷ all attempts; an order with no attempts has rate 0. Report each customer's average order rate, rounded to 2 decimals.

`code/37_delivery_success_rate.sql`

```
CREATE TABLE shop_orders (order_id int PRIMARY KEY, customer_id int, placed_at timestamp);
CREATE TABLE attempts    (order_id int REFERENCES shop_orders, tried_at timestamp,
                          outcome text CHECK (outcome IN ('delivered','failed')), PRIMARY KEY (order_id,
tried_at));
INSERT INTO shop_orders VALUES (1,100,'2026-05-01'),(2,100,'2026-05-02'),(3,200,'2026-05-03'),(4,200,'2026-05-
04');
INSERT INTO attempts VALUES
 (1,'2026-05-02 10:00','failed'),(1,'2026-05-03 10:00','delivered'),   -- order 1: 0.5
 (2,'2026-05-03 11:00','delivered'),                                   -- order 2: 1.0
 (3,'2026-05-04 09:00','failed');                                      -- order 3: 0.0, order 4: no attempts

WITH per_order AS (
  SELECT o.order_id, o.customer_id,
         coalesce(count(*) FILTER (WHERE a.outcome = 'delivered')::numeric
                  / nullif(count(a.order_id), 0), 0) AS rate
  FROM   shop_orders o
  LEFT   JOIN attempts a USING (order_id)
  GROUP  BY o.order_id, o.customer_id
)
SELECT customer_id, round(avg(rate), 2) AS fulfilment_rate
FROM   per_order
GROUP  BY customer_id
ORDER  BY customer_id;
```

Output:

```
 customer_id | fulfilment_rate
-------------+-----------------
         100 |            0.75
         200 |            0.00
(2 rows)
```

Why it works. `LEFT JOIN` keeps order 4, which has no attempts. For it, `count(a.order_id)` is 0; `nullif(0, 0)` turns the divisor into NULL, the division yields NULL instead of an error, and `coalesce(..., 0)` applies the "no attempts = 0" rule. Customer 100 averages (0.5 + 1.0) / 2 = 0.75.

Dialect trap. In a MySQL `SELECT`, `x / 0` yields NULL (plus a warning under the default **SQL mode** — MySQL's setting for how strictly it treats questionable data; strict mode turns it into an error only inside `INSERT` / `UPDATE`), so `IFNULL(x / 0, 0)` "works" there. PostgreSQL raises `division by zero` and aborts the query — `NULLIF(divisor, 0)` is the portable guard.

19.9.3 Share of members who borrowed each book

Problem. Library members and a borrowings table ((book, member) unique). For each book, the percentage of all members who borrowed it, rounded to 2 decimals; order by percentage descending, then book id.

`code/38_borrow_share_per_book.sql`

```
CREATE TABLE members_lib (member_id int PRIMARY KEY, name text);
CREATE TABLE borrowings  (book_id int, member_id int, PRIMARY KEY (book_id, member_id));
INSERT INTO members_lib VALUES (1,'Al'),(2,'Bea'),(3,'Cas');
INSERT INTO borrowings VALUES (500,1),(500,2),(500,3),(501,2),(502,1),(502,3);

SELECT book_id,
       round(100.0 * count(DISTINCT member_id) / (SELECT count(*) FROM members_lib), 2) AS pct_of_members
FROM   borrowings
GROUP  BY book_id
ORDER  BY pct_of_members DESC, book_id;
```

Output:

```
 book_id | pct_of_members
---------+----------------
     500 |         100.00
     502 |          66.67
     501 |          33.33
(3 rows)
```

Why it works. The numerator counts distinct borrowers per book; the denominator is a scalar subquery over *all* members, including non-borrowers. `100.0 *` first avoids truncation, and `ORDER BY` can use the alias because it runs after `SELECT`.

Edge case. If there were no members, the denominator would be 0 and PostgreSQL would raise an error; `nullif(..., 0)` makes the result NULL instead.

19.10 String building and expression evaluation

Some questions treat SQL as a small **interpreter** (reading and evaluating expressions): evaluate stored comparisons, render a formula, or validate shapes. The tools are `CASE`, `||` concatenation, `format()` and `string_agg`.

19.10.1 Evaluate stored comparisons

Problem. `vars(name, val)` holds integer variables; `exprs(lhs, op, rhs)` holds comparisons such as `x < y`, with `op` in `<`, `>`, `=`. Return every expression with its truth value.

`code/39_compare_variables.sql`

```
CREATE TABLE vars  (name text PRIMARY KEY, val int);
CREATE TABLE exprs (lhs text REFERENCES vars, op text CHECK (op IN ('<','>','=')), rhs text REFERENCES vars,
                    PRIMARY KEY (lhs, op, rhs));
INSERT INTO vars  VALUES ('x',4),('y',9),('z',4);
INSERT INTO exprs VALUES ('x','<','y'),('x','>','y'),('x','=','z'),('y','=','z');

SELECT e.lhs, e.op, e.rhs,
       CASE e.op WHEN '<' THEN l.val < r.val
                 WHEN '>' THEN l.val > r.val
                 WHEN '=' THEN l.val = r.val END AS result
FROM   exprs e
JOIN   vars l ON l.name = e.lhs
JOIN   vars r ON r.name = e.rhs
ORDER  BY e.lhs, e.op, e.rhs;
```

Output:

```
 lhs | op | rhs | result
-----+----+-----+--------
 x   | <  | y   | t
 x   | =  | z   | t
 x   | >  | y   | f
 y   | =  | z   | f
(4 rows)
```

Why it works. Joining `vars` twice puts both operand values on one row, and `CASE e.op WHEN ...` maps the symbol to the real comparison. The `boolean` result prints as `t`/`f`; add `::text` if the answer must read `true`/`false`.

Alternative. Dynamic SQL (`EXECUTE format(...)`) could evaluate any operator, but it lets data inject code and is overkill for three operators.

19.10.2 Build a polynomial equation string

Problem. Terms of a polynomial: `power` (0–100, unique) and non-zero integer `coef` (−100…100). Render the left-hand side as one string, highest power first, each term as sign + absolute coefficient + `X^power` (just `X` for power 1, nothing for power 0), followed by `=0`.

`code/40_polynomial_string.sql`

```sql
CREATE TABLE poly_terms (power int PRIMARY KEY CHECK (power BETWEEN 0 AND 100),
                         coef int CHECK (coef BETWEEN -100 AND 100 AND coef <> 0));
INSERT INTO poly_terms VALUES (0,-6),(1,5),(3,2),(2,-1);

SELECT string_agg(
         CASE WHEN coef > 0 THEN '+' ELSE '-' END
         || abs(coef)
         || CASE WHEN power = 0 THEN ''
                 WHEN power = 1 THEN 'X'
                 ELSE 'X^' || power END,
         '' ORDER BY power DESC) || '=0' AS equation
FROM   poly_terms;
```

Output:

```
    equation
------------------
 +2X^3-1X^2+5X-6=0
(1 row)
```

Why it works. Each term is sign `||` `abs(coef)` `||` power suffix. `string_agg(..., '' ORDER BY power DESC)` joins them with no separator; the `ORDER BY` inside the aggregate sets the order, not the table.

Edge case. `||` accepts an integer when the other side is text (it is cast); a NULL piece would make the whole term NULL, whereas `concat()` treats NULL as empty. MySQL: `GROUP_CONCAT(... ORDER BY power DESC SEPARATOR '')`.

19.10.3 Can four sides form a rectangle?

Problem. Rows of four side lengths `a, b, c, d`, taken in order around the shape. Report `Yes` if opposite sides are equal (a = c and b = d), otherwise `No`.

`code/41_rectangle_check.sql`

```sql
CREATE TABLE side_sets (a int, b int, c int, d int, PRIMARY KEY (a, b, c, d));
INSERT INTO side_sets VALUES (3,5,3,5),(4,4,4,4),(3,3,5,5),(2,7,2,8);

SELECT a, b, c, d,
       CASE WHEN a = c AND b = d THEN 'Yes' ELSE 'No' END AS rectangle
FROM   side_sets;
```

Output:

```
 a | b | c | d | rectangle
---+---+---+---+-----------
 3 | 5 | 3 | 5 | Yes
 4 | 4 | 4 | 4 | Yes
 3 | 3 | 5 | 5 | No
 2 | 7 | 2 | 8 | No
(4 rows)
```

Why it works. A row-level `CASE` needs no grouping. Order matters: 3, 3, 5, 5 has equal sides but *adjacent* ones, so it fails (if sides could be rearranged, sort the four values and compare the two smallest and the two largest).

Edge case. If any side could be NULL, `a = c` is unknown and the `CASE` falls to `ELSE 'No'`; mention that choice to the interviewer.

19.11 Pattern catalogue

Pattern	Recognise it by	Template query (PostgreSQL)
Nth highest value	"second highest", "Nth largest", "NULL if none"	`SELECT (SELECT DISTINCT v FROM t ORDER BY v DESC OFFSET n-1 LIMIT 1)`
Top-N per group	"top 3 in each", "highest per department"	`WHERE dense_rank() OVER (PARTITION BY g ORDER BY v DESC) <= n` (in a CTE)
First/last row per group	"first session", "latest status", "earliest booking"	`SELECT DISTINCT ON (g) * FROM t ORDER BY g, ts`
All tied for the max	"most ... include ties"	`WHERE cnt = (SELECT max(cnt) FROM c)` or `rank() = 1`
Capacity cut-off	"confirmed vs waitlist", "fits in the lift"	`row_number()` or `sum(w) OVER (ORDER BY pos)` compared with the limit
Running total	"cumulative", "so far", "to date"	`sum(v) OVER (PARTITION BY g ORDER BY ts)`
Compare with previous row	"improved over last", "change since"	`v - lag(v) OVER (PARTITION BY g ORDER BY ts)`
Consecutive ids/days (islands)	"N days in a row", "3+ consecutive"	`GROUP BY g, day - row_number() OVER (PARTITION BY g ORDER BY day)::int HAVING count(*) >= n`
Runs of equal values / streaks	"longest winning streak", "repeated 3 times in a row"	`row_number() OVER (PARTITION BY g ORDER BY ts) - row_number() OVER (PARTITION BY g, val ORDER BY ts)` as island id
Static pivot	fixed list of output columns	`max(v) FILTER (WHERE k = 'a') AS a, ... GROUP BY id`
Pivot of lists	"names under each continent"	`row_number() OVER (PARTITION BY k ORDER BY v)` then `GROUP BY rn` with `FILTER`
Dynamic pivot	"one column per store", columns unknown	`string_agg(format('... %L ... %I', k, k))` then `EXECUTE` (procedure + view) or `\gexec`
Unpivot	wide columns to rows	`CROSS JOIN LATERAL (VALUES ('a', a), ('b', b))` or `jsonb_each_text(to_jsonb(t) - 'id')`
Pair stored once	friendships, calls, matches	`SELECT a AS p, b AS o FROM t UNION ALL SELECT b, a FROM t`
Relational division	"took all", "bought every"	`HAVING count(DISTINCT y) = (SELECT count(*) FROM all_y)`
Compare with group average	"above department average"	`v > avg(v) OVER (PARTITION BY g)`
Uniqueness checks	"shared with another", "unique location"	`count(*) OVER (PARTITION BY k) > 1` / `= 1`
Cohort / retention	"install date", "came back next day"	`min(day)` per user, then `avg((EXISTS (... day = first + 1))::int)`
Histogram with empty buckets	"from 0 to max", "include zero"	`generate_series(0, max) n LEFT JOIN counts ON c = n`
Ratio / rate	"rate", "percentage", "share"	`round(100.0 * count(*) FILTER (WHERE cond) / nullif(count(*), 0), 2)`
String from rows	"list of titles", "build the equation"	`string_agg(expr, sep ORDER BY k)`

Chapter 19 TL;DR

- **Ranking**: `ROW_NUMBER` = exactly one winner, `RANK` = keep ties, `DENSE_RANK` = top N *distinct* values. `DISTINCT ON` is PostgreSQL's shortcut for "first row per group". Use a scalar subquery or aggregate when "no answer" must be a NULL row.
- **Running windows**: `SUM(...) OVER (PARTITION BY ... ORDER BY ...)` for totals so far, `LAG` for "vs previous"; mind the `RANGE` default when the order key ties.
- **Gaps and islands**: `value - ROW_NUMBER()` (or the difference of two row numbers) is constant inside a run; group by it. Deduplicate first.
- **Pivots**: `MAX(...) FILTER (WHERE ...)` per column; for data-driven columns generate SQL with `format()` (`%I` / `%L`) and `EXECUTE` it into a view, or `\gexec`. Unpivot with `LATERAL VALUES` or `jsonb_each_text`.
- **Pairs**: unfold a pair stored in one row with `UNION ALL` so each side gets a row; "all of them" is relational division.
- **Dates**: bucket with `date_trunc` or `min()` per user; fill empty buckets with `generate_series`.
- **Ratios**: `100.0` first, `NULLIF(x, 0)` on the divisor, and a deliberate choice of which rows form the denominator.
- **PostgreSQL traps**: integer division truncates, division by zero is an error, `NOT IN` with a NULL returns nothing, `COUNT(*)` after a `LEFT JOIN` counts the padding row, and `DESC` sorts NULLs first.

Next: Chapter 20 — Cheat sheets and glossary, which condenses the whole book into quick-reference tables, a pattern catalogue and an A–Z glossary.

Chapter 20 — Cheat Sheets, Interview Q&A and Glossary

The book on a few pages: no new topics, just the syntax, rules, decision tables and vocabulary of Chapters 1–19 in one place for quick revision or lookup. Every entry links to the section that explains it with examples and real output.

- **Syntax cheat sheets** per area (20.2–20.14), most ending with the traps that matter most; **the logical query-processing order** (20.1), which answers most "why does this error happen" questions.
- **Decision tables**: which join (20.5), subquery form (20.6), window function (20.8), index (20.11), isolation level (20.12), scaling tool (20.14), database (20.15).
- **A dialect table** for PostgreSQL, MySQL, SQL Server, SQLite and Oracle (20.16).
- **Verified behaviours** (20.17): about a hundred claims from these sheets (`7/2 = 3`, `NULL` logic, rank ties, frame defaults…) asserted by a runnable script on PostgreSQL 14, plus a script checking the SQLite column of the dialect table.
- **The interview pattern catalogue** (20.18), linking every pattern to solved problems in Chapters 18–19, and an **index of concept questions** (20.19) answered in the Chapter 21 question bank.
- **A glossary** (20.20), from *1NF* to *ZooKeeper*: every technical term, acronym and product name the book uses, each defined once in plain English — look a word up there the first time you meet it.

How to use it: read a sheet, cover the right-hand column and say the answer aloud; follow the link when it doesn't come easily. All SQL is PostgreSQL unless a row says otherwise; other databases' syntax in 20.16 comes from their documentation and was not run, except the SQLite column.

20.1 The logical query-processing order

You **write** `SELECT … FROM … WHERE … GROUP BY … HAVING … ORDER BY … LIMIT`, but the database **evaluates** it (logically) starting from `FROM`, each step receiving the previous step's table. The optimizer may physically reorder the work, but the result is always as if these steps ran in this order.

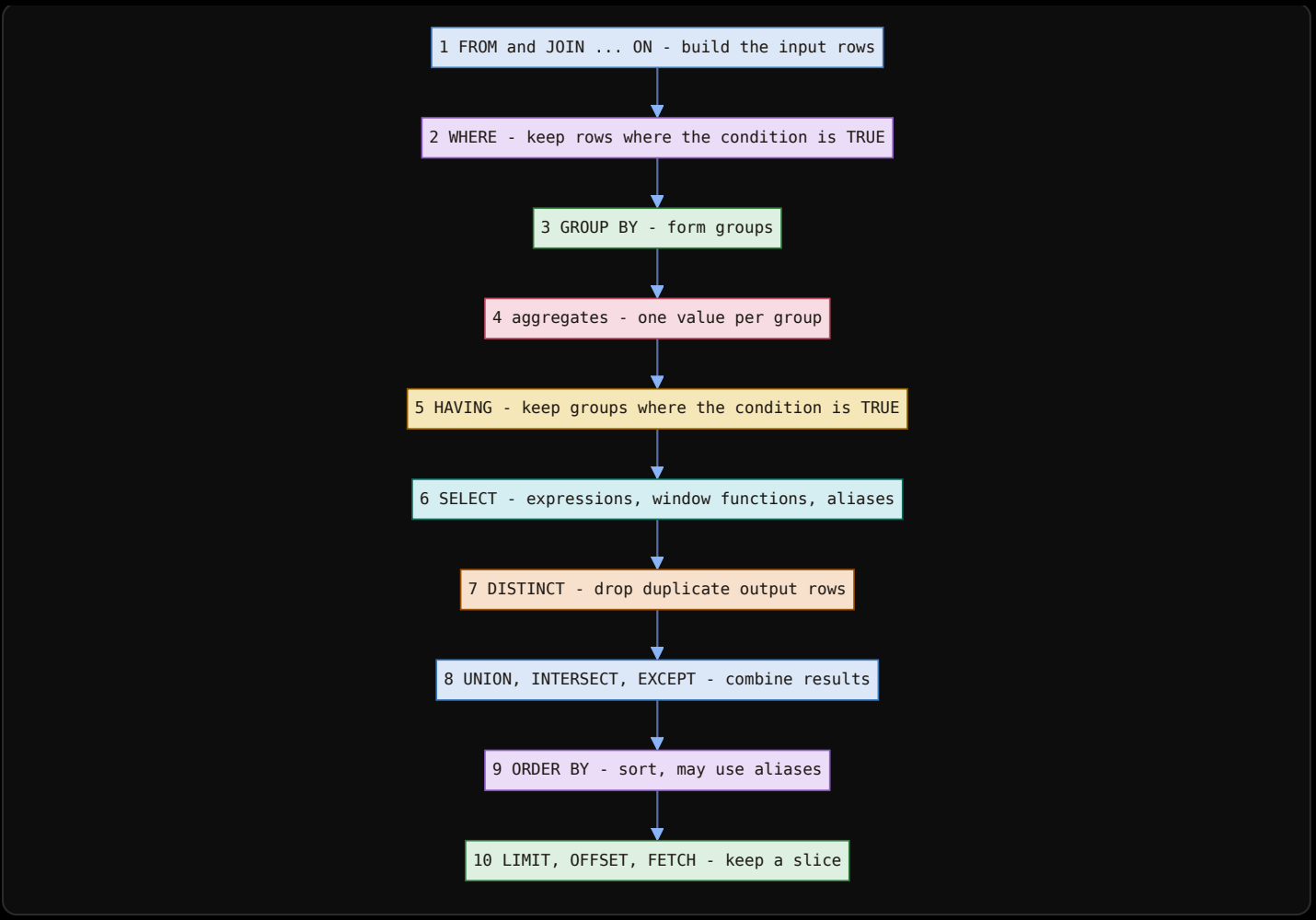


Figure 20.1 — Logical query-processing order (full explanation in §4.5 and §2.9).

Consequence	Because
`WHERE` cannot use a `SELECT` alias	aliases are created at step 6, `WHERE` is step 2
`WHERE` cannot contain `count(*)`, use `HAVING`	groups do not exist before step 3
A window function cannot appear in `WHERE` or `HAVING`	windows are computed at step 6. Filter them in an outer query or CTE (§8.10)
`ORDER BY` can use aliases and column positions	it runs after `SELECT`
`LIMIT` without `ORDER BY` returns arbitrary rows	only step 9 gives order
A filter in `WHERE` on the right table of a `LEFT JOIN` removes the padded rows	`WHERE` runs after the join has built them (§5.9)
With `SELECT DISTINCT`, `ORDER BY` may only use output columns	sorting happens after de-duplication

Two PostgreSQL relaxations: `GROUP BY` may refer to output aliases and positions (`GROUP BY 1`), and grouping by a table's primary key lets you select that table's other columns without listing them.

20.2 SELECT, filtering, NULL, sorting and paging

Covers Chapter 2.

```sql
SELECT DISTINCT e.dept_id, e.salary * 12 AS yearly     -- expressions + aliases
FROM   employees AS e                                  -- table alias
WHERE  e.salary BETWEEN 1000 AND 5000                  -- inclusive on both ends
  AND  e.dept_id IN (1, 2, 3)                          -- same as a chain of ORs
  AND  e.name ILIKE 'a%'                               -- LIKE is case-sensitive, ILIKE is not
  AND  e.manager_id IS NOT NULL                        -- never "= NULL"
ORDER  BY yearly DESC NULLS LAST, e.dept_id            -- add a tie-breaker
LIMIT  10 OFFSET 20;                                   -- or OFFSET 20 ROWS FETCH FIRST 10 ROWS ONLY
```

Need	Syntax	Remember
Rename a column	`expr AS name`	double-quote a name with spaces or capitals: `AS "Total"`
Pattern match	`LIKE 'a_%'`	`%` = any run of characters, `_` = exactly one. `ESCAPE` for literal `%`. `~` / `~*` for regular expressions
Range	`x BETWEEN a AND b`	inclusive. For timestamps use half-open `x >= a AND x < b`
NULL test	`IS NULL`, `IS NOT NULL`	`x = NULL` is never TRUE
NULL-safe compare	`a IS [NOT] DISTINCT FROM b`	treats two NULLs as equal
Default for NULL	`COALESCE(x, 0)`	returns the first non-NULL argument
Unique rows	`SELECT DISTINCT a, b`	applies to the whole output row, and NULLs count as equal
First row per group	`SELECT DISTINCT ON (g) … ORDER BY g, ts`	PostgreSQL only. The leading `ORDER BY` keys must match `g`
Sort	`ORDER BY a DESC NULLS LAST, b`	PostgreSQL puts NULLs **last** in ascending order and **first** in descending order
Page	`LIMIT n OFFSET k` / `OFFSET k ROWS FETCH FIRST n ROWS ONLY`	always with `ORDER BY`. Deep pages → keyset pagination (§12.14)
Keep ties at the cut-off	`FETCH FIRST n ROWS WITH TIES`	needs `ORDER BY` (PostgreSQL 13+)

Three-valued logic. A comparison with `NULL` gives `UNKNOWN` (shown as NULL). `WHERE` and `HAVING` keep only rows where the condition is TRUE; `CHECK` constraints reject only FALSE.

AND	TRUE	FALSE	UNKNOWN
TRUE	TRUE	FALSE	UNKNOWN
FALSE	FALSE	FALSE	FALSE
UNKNOWN	UNKNOWN	FALSE	UNKNOWN

OR	TRUE	FALSE	UNKNOWN
TRUE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	UNKNOWN
UNKNOWN	TRUE	UNKNOWN	UNKNOWN

`NOT UNKNOWN` is UNKNOWN. Precedence is `NOT` > `AND` > `OR`, so parenthesise whenever you mix `AND` and `OR`.

Traps: `x <> 5` silently drops rows where `x` is NULL. `NOT IN (list containing NULL)` returns no rows (§2.5, §6.2). `BETWEEN '2024-01-01' AND '2024-01-31'` on a timestamp misses everything after midnight on the 31st. `OFFSET 100000` still reads and throws away 100,000 rows.

20.3 Functions and expressions

Covers Chapter 3.

Area	PostgreSQL	Notes
Cast	`CAST(x AS numeric(10,2))`, `x::int`	`::` is PostgreSQL-only. Bad input raises an error instead of guessing
Division	`7 / 2` → `3`, `7 / 2.0` → `3.5`, `7::numeric / 2` → `3.5`	integer ÷ integer truncates toward zero (`-7 / 2` → `-3`)
Remainder	`7 % 3`, `mod(7, 3)`	takes the sign of the dividend (`-7 % 2` → `-1`)
Rounding	`round(x, 2)`, `trunc(x)`, `ceil`, `floor`	`round(2.5)` → 3 on `numeric` (half away from zero), `round(2.5::float8)` → 2 (half to even)
Logs and powers	`ln(x)`, `log(x)` (base 10), `log(b, x)`, `power(a, b)`, `sqrt`	MySQL's `LOG(x)` is the natural log
Safe divide	`a / NULLIF(b, 0)`	`1 / 0` is an error (SQLSTATE 22012); the `NULLIF` version returns NULL. MySQL and SQLite return NULL for `x / 0` instead of failing (20.16)
Concatenate	`a || b`, `concat(a, b)`, `concat_ws(', ', a, b)`	`||` with a NULL gives NULL. `concat` skips NULLs
Length	`length(s)` / `char_length(s)` = characters, `octet_length(s)` = bytes	`'é'` is 1 character, 2 bytes in UTF-8
Pieces	`substring(s FROM 2 FOR 3)`, `left`, `right`, `split_part(s, ',', 2)`	positions are 1-based
Clean up	`trim`, `lower`, `upper`, `initcap`, `replace`, `translate`, `lpad`, `regexp_replace`	
Search	`position('x' IN s)`, `strpos(s, 'x')`	0 means "not found"
Now	`now()` = `current_timestamp` (fixed for the transaction), `clock_timestamp()` (real time)	`current_date` for today
Date arithmetic	`d + 7`, `ts + interval '2 hours'`, `date '2024-03-01' - date '2024-02-01'` → `29`	`date - date` is an integer number of days. Month addition clamps (Jan 31 + 1 month = Feb 29 in 2024)
Parts	`extract(year FROM ts)`, `date_part('dow', d)` (Sunday = 0), `date_trunc('month', ts)`, `age(a, b)`	
Format and parse	`to_char(ts, 'YYYY-MM')`, `to_date('03/05/2024', 'DD/MM/YYYY')`	
Time zones	`timestamptz` stores an instant. `ts AT TIME ZONE 'Europe/Paris'` converts	store `timestamptz`, not `timestamp`
Conditionals	`CASE WHEN … THEN … ELSE … END`, `COALESCE`, `NULLIF`, `GREATEST`, `LEAST`	no `ELSE` → NULL. PostgreSQL's `GREATEST`/`LEAST` ignore NULLs

Traps: integer averages and percentages (`100 * a / b` with integers truncates — write `100.0 * a / b`), `float` for money (use `numeric`), `timestamp` without time zone for events in several regions, and assuming `LOG` means the same everywhere.

20.4 Aggregation

Covers Chapter 4.

```sql
SELECT   dept_id,
         count(*)                                  AS staff,        -- rows
         count(manager_id)                         AS with_manager, -- non-NULL values
         count(DISTINCT job)                       AS jobs,
         round(avg(salary), 2)                     AS avg_salary,   -- NULLs ignored
         count(*) FILTER (WHERE salary > 5000)     AS high_paid,    -- conditional aggregate
         sum(CASE WHEN remote THEN 1 ELSE 0 END)   AS remote_staff, -- portable version
         string_agg(name, ', ' ORDER BY name)      AS names
FROM     employees
WHERE    hired >= date '2020-01-01'                -- filters rows before grouping
GROUP BY dept_id
HAVING   count(*) >= 3                             -- filters groups after aggregation
ORDER BY staff DESC;
```

Function	Returns	NULL behaviour / empty input
`count(*)`	number of rows	0 for no rows
`count(x)`, `count(DISTINCT x)`	non-NULL values (distinct)	0 for no rows
`sum`, `avg`, `min`, `max`	as named	skip NULLs. **`NULL`** for no rows (use `COALESCE(sum(x), 0)`)
`avg(int_col)`	`numeric` in PostgreSQL	SQL Server returns an integer
`string_agg(x, sep ORDER BY …)`, `array_agg`, `json_agg`	concatenated value	`string_agg` skips NULLs
`bool_and`, `bool_or`	boolean	
`percentile_cont(0.5) WITHIN GROUP (ORDER BY x)`	median (interpolated)	`percentile_disc` returns an actual value; `mode()` the most common
`GROUPING SETS ((a), (b), ())`, `ROLLUP (a, b)`, `CUBE (a, b)`	subtotals in one query	`grouping(a)` = 1 marks a rolled-up row. `ROLLUP (a, b)` = (a,b), (a), ()

Rules: every selected column must be grouped or aggregated (or depend functionally on a grouped primary key). NULLs form **one** group. `GROUP BY` does not sort. An aggregate query without `GROUP BY` always returns exactly one row, even over zero rows — why "return NULL if there is no answer" questions use an aggregate or scalar subquery (P39).

Traps: `WHERE count(*) > 1` (use `HAVING`). `count(*)` after a `LEFT JOIN` counts the padding row — use `count(right_table.id)` to get zero (§5.9). Averages of averages. Joining two child tables before aggregating multiplies the rows (fan-out).

20.5 Joins

Covers Chapter 5.

```sql
SELECT c.name, o.id, o.total
FROM   customers c
LEFT   JOIN orders o
       ON  o.customer_id = c.id
       AND o.placed_at >= date '2024-01-01';   -- right-table filter belongs in ON
```

Join	Returns	Typical question
`CROSS JOIN`	every pair, m × n rows	"every student × every subject" grid
`[INNER] JOIN … ON`	only pairs that match	"orders with their customer"
`LEFT [OUTER] JOIN`	every left row, with matches or NULL padding	"all customers, with their orders if any"
`RIGHT [OUTER] JOIN`	mirror of `LEFT`	rewrite as `LEFT` for readability
`FULL [OUTER] JOIN`	all rows of both sides	reconciling two lists
Self join	a table joined to itself under two aliases	employee → manager, pairs, previous row
Semi-join: `EXISTS`, `IN`	left rows that have at least one match, each once	"customers who ordered"
Anti-join: `NOT EXISTS`, `LEFT JOIN … WHERE r.id IS NULL`, `EXCEPT`	left rows with no match	"customers who never ordered"
`JOIN … USING (col)`	equality on same-named columns, one output column	fine. `NATURAL JOIN` is not (it silently joins on every shared name)
`JOIN LATERAL (subquery) ON true`	per-row subquery that may reference the left row	"top 3 orders per customer" (§6.7)

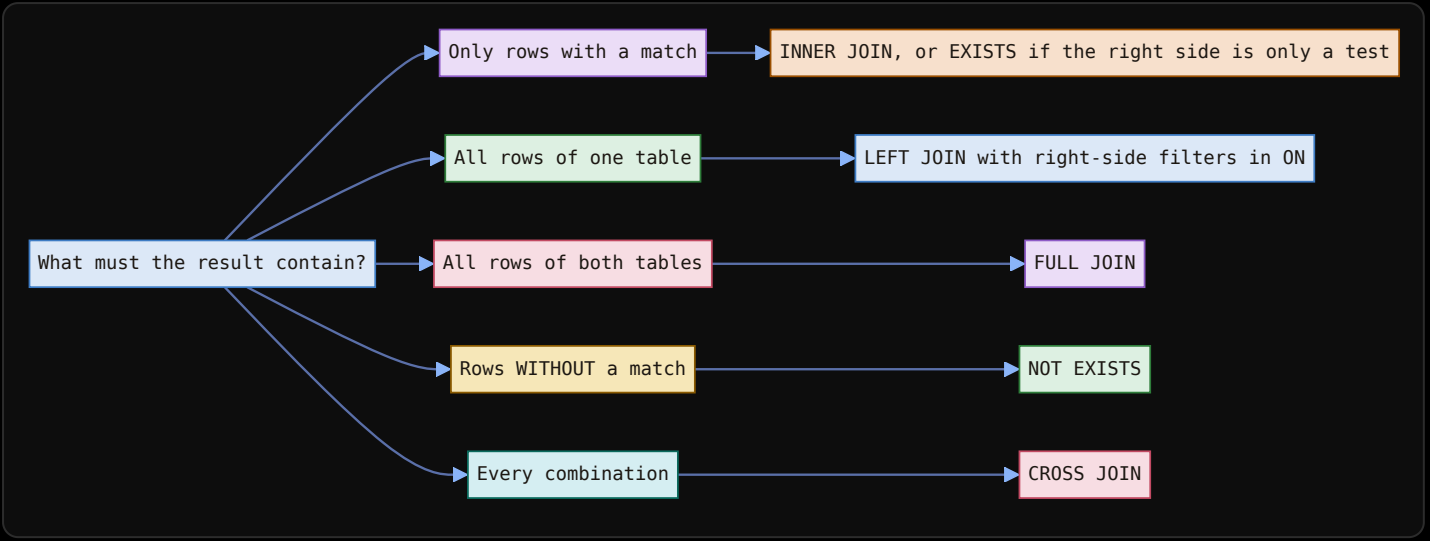


Figure 20.2 — Choosing a join.

Pitfalls (§5.9): fan-out when joining two one-to-many children, right-table conditions in `WHERE` after a `LEFT JOIN`, NULL join keys (never match), `count(*)` of padded rows, joining on names instead of keys, `NOT IN` with NULLs. **How joins run:** nested loop (few outer rows, indexed inner key), hash join (large, equality only), merge join (both inputs sorted) (§12.15).

20.6 Subqueries and set operations

Covers Chapter 6.

Form	Example	Watch out for
Scalar subquery	`SELECT name, (SELECT max(total) FROM orders o WHERE o.cid = c.id) FROM customers c`	0 rows → NULL. 2+ rows → error
`IN` / `NOT IN`	`WHERE id IN (SELECT cid FROM orders)`	`NOT IN` returns nothing if the subquery yields a NULL
Correlated	inner query uses a column of the outer row	conceptually runs once per outer row. Qualify every column
`EXISTS` / `NOT EXISTS`	`WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.cid = c.id)`	the safe anti-join. The select list inside does not matter
`ANY` / `ALL`	`salary > ALL (SELECT …)`	`> ALL (empty)` is TRUE. `> ANY (empty)` is FALSE. `= ANY` = `IN`, `<> ALL` = `NOT IN`
Derived table	`FROM (SELECT …) AS t`	alias required before PostgreSQL 16
Row subquery	`WHERE (a, b) = (SELECT x, y …)`	also `(a, b) IN (…)`
`LATERAL`	`CROSS JOIN LATERAL (SELECT … WHERE o.cid = c.id LIMIT 3) t`	`LEFT JOIN LATERAL … ON true` keeps rows with no match

Set operator	Result	Notes
`UNION`	rows in either query, duplicates removed	costs a sort or hash
`UNION ALL`	everything, duplicates kept	the default choice when duplicates are impossible or wanted
`INTERSECT [ALL]`	rows in both	NULLs compare as equal here
`EXCEPT [ALL]`	rows in the first but not the second	not commutative. Oracle calls it `MINUS`

Rules: same number of columns, compatible types, column names from the first query, `INTERSECT` binds tighter than `UNION` / `EXCEPT`, and `ORDER BY` / `LIMIT` apply to the whole combined result (§6.11).

20.7 CTEs, recursion and views

Covers Chapter 7.

```sql
WITH RECURSIVE chain AS (
    SELECT id, manager_id, name, 1 AS depth, ARRAY[id] AS path   -- anchor: the start rows
    FROM   employees WHERE manager_id IS NULL
  UNION ALL
    SELECT e.id, e.manager_id, e.name, c.depth + 1, c.path || e.id   -- recursive term
    FROM   employees e JOIN chain c ON e.manager_id = c.id
    WHERE  NOT e.id = ANY (c.path)                                    -- cycle guard
)
SELECT repeat('  ', depth - 1) || name AS org_chart FROM chain ORDER BY path;
```

Tool	What it is	Use it for	Remember
CTE `WITH x AS (…)`	a named subquery for one statement	readable step-by-step queries, reuse by name	PostgreSQL 12+ inlines single-use CTEs. `AS MATERIALIZED` / `NOT MATERIALIZED` override
Data-modifying CTE	`WITH moved AS (DELETE … RETURNING *) INSERT … SELECT * FROM moved`	several changes in one atomic statement	PostgreSQL only. Every part sees the same snapshot
Recursive CTE	anchor `UNION [ALL]` recursive term	trees, graphs, number and date series	the recursive term sees only the previous round. Stops when a round adds no rows. Needs a stop condition
`CYCLE` / `SEARCH` clauses	built-in cycle detection and ordering	graphs	PostgreSQL 14+
`generate_series(a, b, step)`	a set-returning function	calendars, filling gaps with zeros	inclusive of both ends. With `date` arguments and an `interval` step it returns `timestamptz`, so cast with `::date`
View	a stored query, no data	simplicity, security, stable interfaces	always current. Simple one-table views are updatable. `WITH CHECK OPTION`
Materialized view	a stored result	expensive reports	stale until `REFRESH MATERIALIZED VIEW [CONCURRENTLY]` (needs a unique index)

20.8 Window functions and frames

Covers Chapter 8.

```
function(args) OVER (
    PARTITION BY dept_id                -- independent groups; rows are NOT collapsed
    ORDER BY     hired, id              -- order inside each partition
    ROWS BETWEEN 2 PRECEDING AND CURRENT ROW   -- frame: which rows the function sees
)
```

Function	Result on scores 30, 20, 20, 10 (descending)	Uses the frame?
`row_number()`	1, 2, 3, 4 (ties broken arbitrarily unless you add a tie-breaker)	no
`rank()`	1, 2, 2, 4 (gaps after ties)	no
`dense_rank()`	1, 2, 2, 3 (no gaps)	no
`ntile(n)`	bucket numbers 1..n. Larger buckets first (7 rows into 3 → 3, 2, 2)	no
`percent_rank()` / `cume_dist()`	(rank − 1)/(n − 1) / (rows sorted before the current row + its peers)/n	no
`lag(x, k, default)` / `lead(…)`	value k rows before / after (NULL or `default` past the edge)	no
`first_value`, `last_value`, `nth_value`	value at a position **in the frame**	**yes**
`sum`, `avg`, `count`, `min`, `max` … `OVER`	aggregate over the frame	**yes**

Frame	Meaning
no `ORDER BY` in `OVER`	frame = whole partition (`sum(x) OVER ()` = grand total on every row)
`ORDER BY` without a frame clause	`RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, which includes all peers (ties) of the current row
`ROWS BETWEEN n PRECEDING AND CURRENT ROW`	physical rows. Use for moving averages and strict running totals
`RANGE BETWEEN interval '7 days' PRECEDING AND CURRENT ROW`	by value distance on the single `ORDER BY` key. Ties included
`GROUPS BETWEEN 1 PRECEDING AND CURRENT ROW`	by peer groups (PostgreSQL 11+)
`EXCLUDE CURRENT ROW / GROUP / TIES`	remove rows from the frame (PostgreSQL 11+)

Patterns: running total `sum(x) OVER (PARTITION BY g ORDER BY t)`, share of total `x / sum(x) OVER ()`, change vs previous `x - lag(x) OVER (…)`, 7-row moving average with `ROWS 6 PRECEDING`, top-N per group (rank in a CTE, filter outside), de-duplication (`row_number() = 1`), gaps and islands (`value − row_number()` is constant inside a run) (§8.7–§8.9).

Traps: `last_value` with the default frame returns the current row (give it `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`). The default `RANGE` frame gives tied rows the same running total. Windows cannot be used in `WHERE`, and PostgreSQL has no `QUALIFY`. `IGNORE NULLS` arrives only in PostgreSQL 19 (in beta as of September 2026), so on 14–18 emulate it (§8.6). Each distinct window specification may need its own sort (§8.10).

20.9 DDL, DML, constraints and upsert

Covers Chapter 9.

```
CREATE TABLE orders (
    id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    customer_id  bigint NOT NULL REFERENCES customers (id) ON DELETE RESTRICT,
    status       text   NOT NULL DEFAULT 'new'
                 CONSTRAINT orders_status_ck CHECK (status IN ('new', 'paid', 'shipped')),
    total        numeric(12, 2) NOT NULL CHECK (total >= 0),
    placed_at    timestamptz NOT NULL DEFAULT now(),
    UNIQUE (customer_id, placed_at)
);
CREATE INDEX ON orders (customer_id);                -- FKs are not indexed automatically

INSERT INTO orders (customer_id, total) VALUES (1, 20), (2, 35) RETURNING id;
UPDATE orders SET status = 'paid' WHERE id = 1 RETURNING *;
DELETE FROM orders o USING customers c WHERE o.customer_id = c.id AND c.banned;

INSERT INTO stock (sku, qty) VALUES ('A-1', 5)
ON CONFLICT (sku) DO UPDATE SET qty = stock.qty + EXCLUDED.qty;   -- upsert
```

Statement	Does	Remember
`CREATE TABLE [IF NOT EXISTS]`	defines a table	`CREATE TABLE t AS SELECT …` copies data but no constraints. `LIKE src INCLUDING ALL` copies structure
`ALTER TABLE … ADD / DROP / RENAME COLUMN`, `ALTER COLUMN … TYPE t USING expr`, `SET NOT NULL`, `ADD CONSTRAINT … NOT VALID` + `VALIDATE CONSTRAINT`	changes a table	many forms take an exclusive lock on the whole table. Backfill before `SET NOT NULL`
`DROP TABLE [IF EXISTS] … [CASCADE]`	removes the table itself	`CASCADE` also drops dependent views and FKs
`TRUNCATE t [RESTART IDENTITY] [CASCADE]`	empties the table fast	no per-row triggers. Transactional in PostgreSQL
`DELETE FROM t WHERE …`	removes chosen rows	fires triggers, logs every row. Forget `WHERE` and every row goes
`INSERT … SELECT`	copies rows	list the target columns explicitly
`UPDATE … SET a = b, b = a`	swaps values	every `SET` expression sees the **old** row
`… RETURNING`	returns affected rows	PostgreSQL, SQLite 3.35+. SQL Server uses `OUTPUT`
`INSERT … ON CONFLICT (key) DO NOTHING / DO UPDATE SET c = EXCLUDED.c`	upsert	needs a unique index or constraint on the conflict target
`MERGE INTO t USING s ON … WHEN MATCHED … WHEN NOT MATCHED …`	standard upsert and sync	PostgreSQL 15+ (not on the book's server, 14)

Constraint	Rejects	Notes
`NOT NULL`	NULL	the most useful constraint
`PRIMARY KEY`	duplicates and NULLs	one per table. Creates a unique index
`UNIQUE`	duplicate non-NULL values	several NULLs allowed (SQL Server: one). `NULLS NOT DISTINCT` in PostgreSQL 15+
`CHECK (expr)`	rows where `expr` is FALSE	NULL passes. Cannot look at other rows
`FOREIGN KEY … REFERENCES`	child values with no parent	`ON DELETE CASCADE / SET NULL / SET DEFAULT / RESTRICT / NO ACTION` (default). NO ACTION can be deferred, RESTRICT cannot
`DEFAULT`	—	used when the column is omitted
`EXCLUDE USING gist (…)`	overlapping values	"no two bookings of one room overlap" (§17.2)

Key generation	Use
`GENERATED ALWAYS AS IDENTITY`	the standard choice. `serial` is the legacy form
Sequence (`nextval`)	shared or custom counters. Gaps are normal (rollbacks, caching)
`uuid` (`gen_random_uuid()`, PostgreSQL 13+)	ids made on many machines. Random UUIDs scatter B-tree inserts
`GENERATED ALWAYS AS (expr) STORED`	derived columns kept in sync automatically. PostgreSQL 18 adds `VIRTUAL` (computed on read), which becomes the default when neither keyword is given

Backups (§9.12): `pg_dump` makes a logical copy (plain SQL, or `-Fc` custom / `-Fd` directory formats for `pg_restore`, which can restore selected objects in parallel). Physical base backups + archived WAL allow

point-in-time recovery (PITR). Plan by RPO (data you may lose) and RTO (time to recover), keep 3 copies on 2 media with 1 off-site, and test restores.

20.10 Data modeling and normalization quick rules

Covers Chapter 10, Chapter 11 and Chapter 17.

Why databases exist (§1.1). File-based systems suffered redundancy and inconsistency (one fact in several files, drifting apart), isolated data in different formats, no general way to ask a new question, integrity rules buried in program code, lost updates when two users wrote at once, no atomic recovery after a crash, and all-or-nothing security. A DBMS answers each by mediating every access.

The three levels of a model (§10.2).

Level	Audience	Contains	Does not contain
Conceptual	business people	entities and relationships	data types, keys, DBMS details
Logical	designers	attributes, keys, cardinalities, normalized tables	a particular product's types or indexes
Physical	the DBMS	exact types, indexes, partitions, storage	—

ER notation (§10.5).

Concept	Chen notation	Crow's foot
Entity / weak entity	rectangle / double rectangle	box
Attribute	ellipse (underlined = key, double = multivalued, dashed = derived)	listed inside the box
Relationship / identifying relationship	diamond / double diamond	a line / a solid line to a weak entity
Cardinality	`1`, `N`, `M` on the lines	line ends: `||` exactly one, `o|` zero or one, `|{` one or many, `o{` zero or many
Participation	double line = total (mandatory)	inner mark: `|` mandatory, `o` optional

Mapping ER → tables (§10.15). Entity → table. Composite attribute → its parts. Multivalued attribute → its own table. Derived attribute → a view or generated column. 1:N → FK on the "many" side. M:N → junction table with a composite PK. 1:1 → FK + `UNIQUE`. Weak entity → owner's key + partial key as a composite PK, with `ON DELETE CASCADE`. Total participation → `NOT NULL` on the FK.

Keys (§10.13). Superkey ⊇ candidate key ⊇ the chosen **primary key**; other candidates are **alternate keys** (enforce with `UNIQUE`). **Natural key** (from the data: email, ISBN) vs **surrogate key** (made up: identity, UUID). A good default: surrogate PK + `UNIQUE` on the natural key.

Functional dependencies (§11.1–§11.5). `X → Y` is a business rule: data can disprove it but never prove it. **Closure X^+**: start with X, fire every FD whose left side is fully inside the set, repeat until nothing changes; X is a superkey when X^+ contains every attribute. **Candidate keys**: attributes never on a right side must be in every key, so start from them. **Armstrong's axioms**: reflexivity, augmentation, transitivity (plus derived union, decomposition, pseudotransitivity). **Minimal cover**: single-attribute right sides, no extraneous left-side attributes, no redundant FDs.

Normal forms (§11.6–§11.13).

Form	Rule (informal)	Removes
1NF	every cell holds one atomic value, no repeating groups	lists in a column, `phone1, phone2, phone3`
2NF	1NF + no non-prime attribute depends on **part** of a composite key	partial dependencies
3NF	2NF + no non-prime attribute depends on another non-key attribute. Formally: for each FD X → A, X is a superkey or A is prime	transitive dependencies
BCNF	for every non-trivial FD X → Y, X is a superkey	the remaining FD redundancy
4NF	BCNF + no non-trivial multivalued dependency unless its left side is a superkey	independent lists stored together
5NF (project-join NF)	every join dependency follows from the keys	facts that only make sense as three-way combinations

Two 1NF misunderstandings: atomicity depends on how data is **used** (a full name is atomic if you never query first names separately), and one string per cell is not enough — a comma-separated list in one string still breaks 1NF.

Anomalies (insertion, update, deletion) come from storing one fact in several rows. **Every decomposition must be lossless** (shared columns are a key of at least one piece) and ideally **dependency-preserving**; 3NF can always have both, BCNF sometimes loses an FD, so most designs stop at 3NF (§11.10).

Design judgement calls (§17.14).

Decision	Rule of thumb
Type vs instance	separate the catalogue row from the physical item (title vs copy, room type vs room, seat vs seat-for-a-show)
Current state vs history	a status column for "now" plus an append-only events or ledger table for "what happened"
Snapshot vs reference	copy values that must not change later (price paid, shipping address) into the order row. That is a separate fact, not redundancy
Delete vs soft-delete	soft-delete (`deleted_at`) rows that history still references
Enforce where	rules in the schema (`CHECK` , `UNIQUE` , partial unique index, `EXCLUDE`) rather than only in application code
Denormalize when	only for a measured read problem, updated in the same transaction, with a reconciliation query (§11.16)
Scarce items	a guarded `UPDATE … WHERE status = 'free'` in a short transaction, or `FOR UPDATE SKIP LOCKED`

20.11 Indexes and reading EXPLAIN

Covers Chapter 12.

```
CREATE INDEX CONCURRENTLY orders_cust_date_ix ON orders (customer_id, placed_at DESC); -- composite
CREATE INDEX ON orders (customer_id) INCLUDE (total);              -- covering (index-only scans)
CREATE INDEX ON orders (placed_at) WHERE status = 'new';           -- partial
CREATE INDEX ON users (lower(email));                              -- expression
CREATE INDEX ON docs USING gin (body_tsv);                         -- full-text / jsonb / arrays
CREATE INDEX ON readings USING brin (taken_at);                    -- huge, append-only, time-ordered
EXPLAIN (ANALYZE, BUFFERS) SELECT … ;                              -- runs the query and shows the plan
```

Which index?

Query shape	Index	Why
`=`, `<`, `>`, `BETWEEN`, `ORDER BY`, `LIKE 'abc%'` (with `text_pattern_ops` or C collation)	**B-tree** (default)	sorted, supports ranges and order
`WHERE a = ? AND b > ?`	composite B-tree `(a, b)`	equality columns first, range column last
`WHERE a = ?` returning only a few columns	`(a) INCLUDE (x, y)`	index-only scan, no table visits
Only a small hot subset is queried (`status = 'new'`)	partial index `WHERE status = 'new'`	smaller, cheaper to maintain
`WHERE lower(email) = ?`	expression index on `lower(email)`	the index must match the expression
Equality only, very large keys	hash	cannot do ranges or ordering
JSONB `@>`, arrays `&&` / `@>`, full-text `@@`, trigram `LIKE '%x%'`	**GIN** (`pg_trgm` for trigrams)	inverted index: value → rows
Ranges overlapping, geometry, nearest neighbour, `EXCLUDE` constraints	**GiST**	tree of bounding boxes
Billions of rows physically ordered by time	**BRIN**	a few bytes per block range
Low-selectivity column (`is_active` true for 90 %)	usually **none**	a sequential scan is cheaper

Reading a plan (§12.10). Read from the innermost node outward. Each node shows `cost=startup..total` (planner units), estimated `rows`, and with `ANALYZE` `actual time`, actual `rows` and `loops` (multiply per-loop numbers by loops). `BUFFERS` shows pages hit in memory vs read.

Node	Means	Chosen when
Seq Scan	reads every page	a large fraction of the table is needed, or no usable index
Index Scan	walks the index, then fetches each row	few rows, or rows needed in index order
Index Only Scan	answers from the index alone	all needed columns are in the index and pages are all-visible (vacuumed)
Bitmap Index Scan + Bitmap Heap Scan	collects row locations, then reads pages in order	a medium number of scattered rows. Can combine indexes with AND/OR
Nested Loop	for each outer row, probe the inner	small outer side + index on the inner join key
Hash Join	builds a hash table on the smaller input	large inputs, equality joins
Merge Join	merges two sorted inputs	both sides already sorted (or cheap to sort)
Sort / Incremental Sort, HashAggregate / GroupAggregate	sorting and grouping	watch `Sort Method: external merge` (spilled to disk)

The big warning sign: estimated rows far from actual rows. Refresh statistics with `ANALYZE`; consider extended statistics for correlated columns (§12.16).

Anti-patterns that disable an index (§12.13): a function or arithmetic on the indexed column (`WHERE date(ts) = …`, `WHERE price * 1.2 > …`), a leading wildcard (`LIKE '%x'`), implicit casts, `OR` across different columns (sometimes fixable with `UNION` or a bitmap), a composite index whose leading column

is not filtered (PostgreSQL 18+, like Oracle and MySQL 8.0.13+, can sometimes skip-scan it when that column has few distinct values, but don't design around it), deep `OFFSET` paging (keyset: `WHERE (ts, id) < (:last_ts, :last_id) ORDER BY ts DESC, id DESC LIMIT 20`), and N+1 query loops in application code. Every index slows writes and uses space, so index real query patterns, not columns.

20.12 Transactions, isolation and concurrency

Covers Chapter 13.

```
BEGIN ISOLATION LEVEL REPEATABLE READ;
  SELECT balance FROM accounts WHERE id = 1 FOR UPDATE;   -- lock the row you will change
  UPDATE accounts SET balance = balance - 100 WHERE id = 1;
  SAVEPOINT s1;
  UPDATE accounts SET balance = balance + 100 WHERE id = 2;
  -- ROLLBACK TO SAVEPOINT s1;   -- undo only the part after s1
COMMIT;                          -- or ROLLBACK
```

ACID. *Atomicity*: all or nothing (undo via log or, in PostgreSQL, marking the transaction aborted). *Consistency*: constraints and application rules hold before and after. *Isolation*: concurrent transactions don't see each other's partial work (locks, MVCC, SSI). *Durability*: a committed change survives a crash (WAL flushed at commit, checkpoints, replicas).

Anomalies (§13.4).

Anomaly	What happens	Example
Dirty write	overwrite another transaction's uncommitted change	two uncommitted updates on one row
Dirty read	read another transaction's uncommitted change	see a balance that is later rolled back
Non-repeatable read (read skew)	the same row read twice gives different values. Read skew: different rows seen at different moments	a report sees an account before a transfer and the other account after it
Phantom read	the same query returns a different **set** of rows	a count rises because someone inserted a matching row
Lost update	two read-modify-write cycles, the second overwrites the first	two sales both set stock 10 → 9
Write skew	two transactions read the same rows, then each updates a **different** row, breaking a rule both checked	two doctors both go off call, leaving nobody on call

Isolation levels — standard minimum vs PostgreSQL.

Level	Standard forbids	PostgreSQL actually prevents	Still possible in PostgreSQL
Read Uncommitted	—	dirty reads (it behaves as Read Committed)	as Read Committed
Read Committed (PostgreSQL default)	dirty read	dirty read and write	non-repeatable read, phantom, lost update, write skew
Repeatable Read (snapshot isolation)	+ non-repeatable read	+ phantoms. Lost updates abort with SQLSTATE 40001	write skew
Serializable (SSI)	+ phantom	everything. One transaction aborts with 40001	nothing, but you must retry

Which isolation level (and lock)?

Situation	Choose
Ordinary OLTP, single-statement changes (`SET qty = qty - 1`)	Read Committed
A consistent multi-query report or export	Repeatable Read, `READ ONLY`
Read a row, decide, write it back	`SELECT … FOR UPDATE` at Read Committed, or an atomic `UPDATE … WHERE`, or Repeatable Read + retry
Optimistic editing across user think-time	a `version` column: `UPDATE … SET version = version + 1 WHERE id = ? AND version = ?`
A rule spanning several rows (no double booking, one on call)	Serializable + retry, an `EXCLUDE`/unique constraint, or explicit locks
Many workers taking jobs from a queue	`SELECT … FOR UPDATE SKIP LOCKED LIMIT 1`
Must not wait at all	`FOR UPDATE NOWAIT` (errors immediately if locked)

Concurrency control families (§13.9). Pessimistic locking (shared/exclusive locks; two-phase locking: grow then shrink; strict 2PL holds write locks to commit). Optimistic (work freely, validate at commit, abort on conflict). Timestamp ordering (operations out of timestamp order are rejected). **MVCC** (writers create new row versions, readers see a snapshot, so readers never block writers; PostgreSQL marks versions with `xmin`/`xmax` and cleans dead ones with VACUUM).

Deadlocks (§13.11): two transactions each wait for a lock the other holds. PostgreSQL detects it after `deadlock_timeout` and aborts one (SQLSTATE 40P01). Prevent by locking rows in a consistent order and keeping transactions short. Application code should **retry on 40001 and 40P01**.

20.13 Advanced PostgreSQL: PL/pgSQL, JSONB, full-text search, security

Covers Chapter 14.

```
CREATE FUNCTION order_total(p_id bigint) RETURNS numeric
LANGUAGE sql STABLE AS $$ SELECT sum(qty * price) FROM order_lines WHERE order_id = p_id $$;

CREATE PROCEDURE archive_old() LANGUAGE plpgsql AS $$
BEGIN
  LOOP
    DELETE FROM events WHERE id IN (SELECT id FROM events WHERE ts < now() - interval '1 year' LIMIT 10000);
    EXIT WHEN NOT FOUND;
    COMMIT;                                  -- procedures may commit (when CALLed outside a transaction)
  END LOOP;
END $$;
CALL archive_old();

CREATE FUNCTION touch() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN NEW.updated_at := now(); RETURN NEW; END $$;
CREATE TRIGGER t_touch BEFORE UPDATE ON products FOR EACH ROW EXECUTE FUNCTION touch();
```

Feature	Syntax	Remember
Function	`CREATE FUNCTION f(args) RETURNS t LANGUAGE sql / plpgsql`	declare volatility honestly: `IMMUTABLE`, `STABLE`, `VOLATILE` (default)
Procedure	`CREATE PROCEDURE`, `CALL p()`	can `COMMIT` / `ROLLBACK` inside. `INOUT` parameters
Trigger	`BEFORE` / `AFTER` / `INSTEAD OF`, `FOR EACH ROW` / `STATEMENT`	BEFORE can modify `NEW` or reject (return NULL). AFTER reacts. Keep them fast
JSONB	`->` (jsonb), `->>` (text), `#>> '{a,b}'`, `@>` (contains), `?` (key exists), `jsonb_set`, `jsonb_each`, `jsonb_array_elements`	binary, keys de-duplicated (last wins) and reordered. GIN index for `@>` and `?`. In a quick demo, `VACUUM` after bulk updates before building the index, or older snapshots may ignore it
Arrays	`ARRAY[1,2]`, `a[1]` (1-based), `= ANY(a)`, `@>`, `&&`, `unnest`, `array_agg`	out-of-range subscript returns NULL
Full-text search	`to_tsvector('english', body) @@ websearch_to_tsquery('english', 'fast db')`, `ts_rank`	stemming and stop words. Store a generated `tsvector` column with a GIN index
Pivot	`max(v) FILTER (WHERE k = 'a') AS a … GROUP BY id`, or `crosstab` (tablefunc)	unpivot with `CROSS JOIN LATERAL (VALUES ('a', a), ('b', b))`
Roles and privileges	`CREATE ROLE app LOGIN`, `GRANT SELECT, INSERT ON t TO app`, `REVOKE`	group roles, least privilege
Row-level security	`ALTER TABLE t ENABLE ROW LEVEL SECURITY`, `CREATE POLICY p ON t USING (tenant_id = current_setting('app.tenant')::int)`	`USING` filters reads, `WITH CHECK` filters writes. Superusers and `BYPASSRLS` roles always bypass it; the table owner does too unless `FORCE ROW LEVEL SECURITY`
SQL injection defence	bind parameters: psycopg `%s`, sqlite3 `?`, JDBC `PreparedStatement` `?`	never concatenate user input. Identifiers: allow-list or `format('%I', …)`
From code	explicit transactions, `Decimal` / `BigDecimal` for money, batching or `COPY` for bulk loads, stream large results	a small, reused connection pool, or PgBouncer in front of many clients
ORMs	fast for CRUD	watch for N+1 queries. Hand-write complex and hot queries (§14.13)

20.14 Scaling: partitioning, sharding, replication

Covers Chapter 15.

```
CREATE TABLE events (id bigint, ts timestamptz NOT NULL, payload jsonb)
PARTITION BY RANGE (ts);
CREATE TABLE events_2024_06 PARTITION OF events
  FOR VALUES FROM ('2024-06-01') TO ('2024-07-01');       -- upper bound is exclusive
-- also PARTITION BY LIST (region) and PARTITION BY HASH (id) … FOR VALUES WITH (MODULUS 4, REMAINDER 0)
```

Tool	What it does	Choose it when	Cost
Scale up	a bigger server	first, almost always	a ceiling, one failure domain
Read replicas	copies that serve reads	read-heavy workloads, reporting	replication lag, stale reads
Partitioning (one server)	split a table by range, list or hash	huge tables, time-based retention (drop a partition), pruning	unique keys must include the partition key
Sharding (many servers)	partitions on different machines	writes or data exceed one server	cross-shard joins and transactions, rebalancing, operations
Caching	keep hot results in memory	repeated reads of the same data	invalidation
Denormalized / summary tables	precomputed answers	expensive repeated aggregates	keeping copies correct

Concept	One-line rule
Shard key	high cardinality, spreads load evenly, matches the most common query filter
Consistent hashing	only about 1/n of keys move when a node joins. Virtual nodes smooth the load
Synchronous vs asynchronous replication	sync = no committed data lost but slower writes. Async = fast, with lag and possible loss on failover
Single-leader / multi-leader / leaderless	one writer (simple) / several writers (conflicts to resolve) / quorums with R + W > N
Consistency models	strong (linearizable) > sequential > causal > session (read-your-writes) > eventual
CAP	during a network partition choose consistency or availability
PACELC	if Partitioned choose A or C, Else choose Latency or Consistency
Consensus (Raft, Paxos)	a majority of 2f + 1 nodes tolerates f crashes. Byzantine faults (PBFT) need 3f + 1
Two-phase commit	prepare, then commit. Blocks if the coordinator dies after prepare
Recovery	WAL first, checkpoints bound replay, redo committed work, undo uncommitted work
High availability	heartbeat → quorum decides → promote a replica → fence the old leader → redirect clients
Backups vs replicas	a replica copies your mistakes too. Only backups (with PITR) undo a bad `DELETE`

20.15 Which database?

Covers Chapter 16. Default to PostgreSQL for transactional data and add a specialised store only when a requirement clearly exceeds it (§16.10).

Family	Data shape	Strength	Weakness	Examples
Relational (OLTP)	tables, keys, constraints	joins, ACID, ad-hoc SQL	scaling writes beyond one node takes work	PostgreSQL, MySQL, SQL Server, Oracle
Key-value	key → opaque value	fastest get/put, TTL, simple scale-out	no queries by value	Redis, DynamoDB (also document-like), Memcached
Document	nested JSON per record	flexible schema, whole-object reads	cross-document joins and transactions are limited or costly	MongoDB, Couchbase, Firestore
Wide-column	partition key + sorted clustering columns	huge write rates, time-ordered data per key	must model around queries, no joins	Cassandra, ScyllaDB, HBase, Bigtable
Graph	nodes and edges	multi-hop traversals, paths	bulk analytics over all data, sharding	Neo4j, Amazon Neptune
Time-series	timestamped measurements	compression, retention, downsampling	general-purpose queries	TimescaleDB, InfluxDB, Prometheus
Search engine	inverted index of documents	relevance ranking, facets, fuzzy text	not a system of record	Elasticsearch, OpenSearch
Columnar warehouse	column-oriented tables	scans and aggregates over billions of rows	slow single-row writes	BigQuery, Snowflake, Redshift, ClickHouse
NewSQL / distributed SQL	SQL tables on consensus-replicated shards	SQL + ACID across nodes and regions	latency of consensus, operational complexity	Spanner, CockroachDB, YugabyteDB, TiDB
In-memory	data kept in RAM, persisted by snapshot + log	microsecond latency	cost per GB, durability settings	Redis, SAP HANA, VoltDB

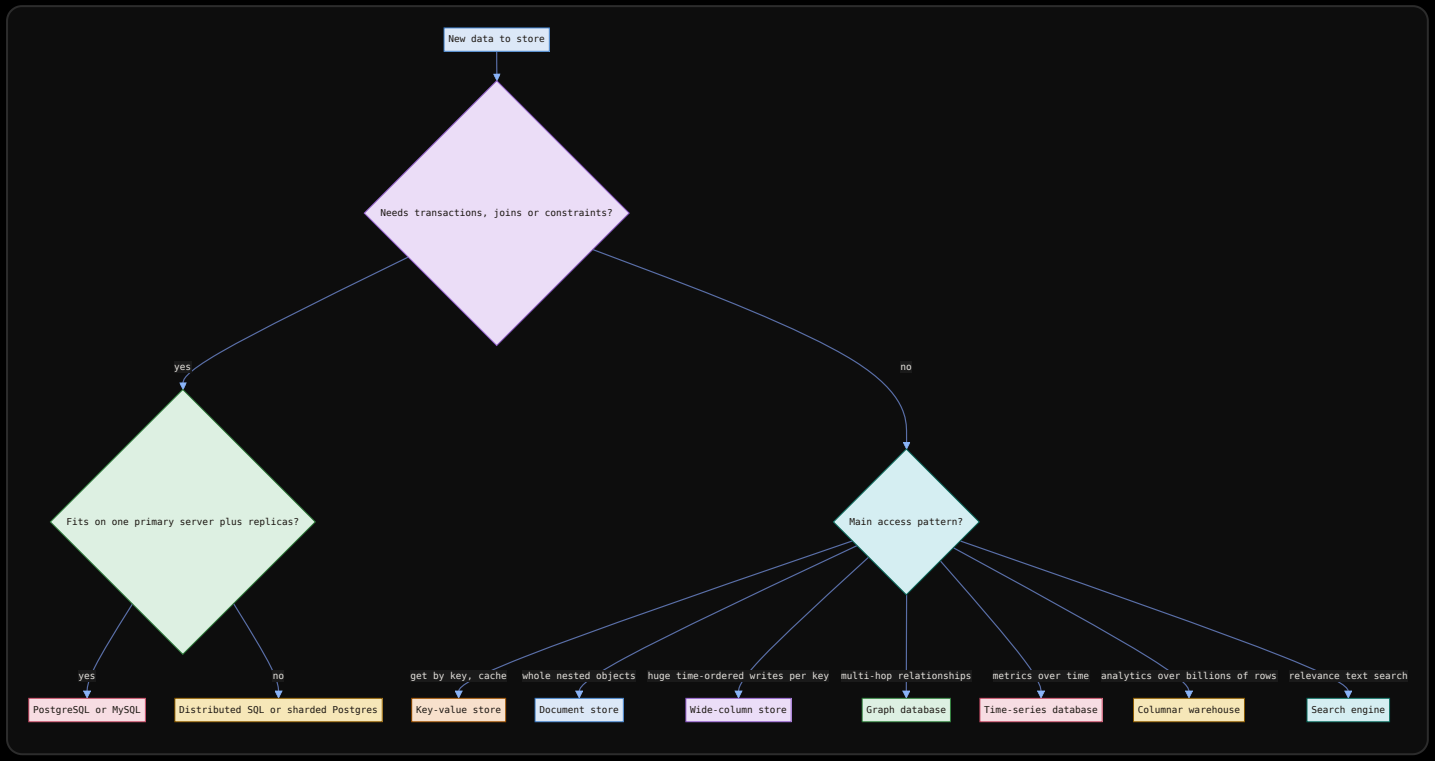


Figure 20.3 — A first-pass database choice. Real systems often combine several (polyglot persistence), fed from one source of truth by change data capture.

ACID vs BASE. BASE = *Basically Available, Soft state, Eventually consistent*: stay up and accept writes, let replicas converge later. A spectrum, not a switch: many NoSQL stores offer tunable or per-operation consistency.

20.16 Dialect differences

The differences you are most likely to hit when moving a query off PostgreSQL 14. Versions in brackets are the first release with the feature; "—" means no built-in equivalent; Oracle means 19c, with 21c or 23ai (renamed 26ai) features marked. The SQLite column was checked on SQLite 3.43 by `code/02_sqlite_dialect.sqlite.sql` (20.17); the other columns come from vendor documentation and were not run.

Syntax and basics

Feature	PostgreSQL	MySQL 8	SQL Server	SQLite	Oracle
First n rows	`LIMIT n` / `FETCH FIRST n ROWS ONLY`	`LIMIT n`	`TOP (n)` / `OFFSET 0 ROWS FETCH NEXT n ROWS ONLY` (2012, needs `ORDER BY`)	`LIMIT n`	`FETCH FIRST n ROWS ONLY` (12c). Older: `ROWNUM`
Quote an identifier	`"Name"`	backticks around the name (`"` only in ANSI_QUOTES mode)	`[Name]` or `"Name"`	`"Name"` (backticks and `[Name]` also accepted)	`"Name"`
Unquoted names are folded to	lower case	case of table names depends on the OS. Column names are case-insensitive	kept. Case-sensitivity follows the collation (insensitive by default)	kept, compared case-insensitively	UPPER case
String concatenation	`a || b` (NULL if either is NULL), `concat()`	`CONCAT(a, b)` (NULL if any is NULL). `||` means OR by default	`a + b`, `CONCAT()` (NULL treated as '')	`a || b`	`a || b` (NULL treated as '')
Empty string `''`	a value, not NULL	a value	a value	a value	**is NULL**
`7 / 2`	`3`	`3.5000` (`7 DIV 2` = 3)	`3`	`3`	`3.5`
`1 / 0` in a `SELECT`	error (22012)	NULL + warning (strict mode errors only in `INSERT`/`UPDATE`)	error (8134)	NULL	error (ORA-01476)
Cast	`CAST(x AS t)`, `x::t`	`CAST`, `CONVERT(x, t)`	`CAST`, `CONVERT(t, x)`, `TRY_CAST`	`CAST`	`CAST`, `TO_NUMBER`, `TO_DATE`
Boolean type	`boolean`	`BOOLEAN` = `TINYINT(1)`	`BIT`	integers 0/1 (keywords `TRUE`/`FALSE` accepted)	`BOOLEAN` (23ai). Before: none in SQL
`SELECT` without `FROM`	yes	yes (or `FROM DUAL`)	yes	yes	`FROM DUAL` required before 23ai
Type checking	strict	lenient unless strict SQL mode (the default in 8.0)	strict	dynamic typing (type affinity). `STRICT` tables (3.37)	strict
Auto-increment	`GENERATED … AS IDENTITY`, `serial`	`AUTO_INCREMENT`	`IDENTITY(1,1)`	`INTEGER PRIMARY KEY` (optionally `AUTOINCREMENT`)	`GENERATED … AS IDENTITY` (12c), sequences
Sequences	yes	—	yes (2012)	—	yes
`DROP TABLE IF EXISTS`	yes	yes	yes (2016)	yes	23ai

NULLs, comparison and sorting

Feature	PostgreSQL	MySQL 8	SQL Server	SQLite	Oracle
NULL-safe equality	`IS NOT DISTINCT FROM`	`<=>`	`IS NOT DISTINCT FROM` (2022)	`IS` (and `IS NOT DISTINCT FROM`, 3.39)	– (use `DECODE(a, b, 1, 0) = 1`)
NULLs in `ORDER BY … ASC`	last	first	first	first	last
`NULLS FIRST / LAST`	yes	– (sort by `x IS NULL` first)	– (sort by a `CASE` first)	yes (3.30)	yes
First non-NULL	`COALESCE`	`COALESCE`, `IFNULL`	`COALESCE`, `ISNULL`	`COALESCE`, `IFNULL`	`COALESCE`, `NVL`
`GREATEST(1, NULL)`	1 (NULLs ignored)	NULL	1 (2022)	use `max(a, b)` (NULL if any NULL)	NULL
`UNIQUE` column with several NULLs	allowed (`NULLS NOT DISTINCT` 15+)	allowed	**one** NULL only (use a filtered index)	allowed	allowed (all-NULL keys are not indexed)
`LIKE` case-sensitive?	yes (`ILIKE` for no)	no with the default collation	no with the default collation	no for ASCII	yes
Regular expressions	`~`, `~*`, `regexp_replace`	`REGEXP`, `REGEXP_LIKE` (8.0)	`REGEXP_LIKE`, `REGEXP_REPLACE` (2025). Before: `LIKE` patterns only	`REGEXP` only if a function is loaded (the shell has one)	`REGEXP_LIKE`, `REGEXP_REPLACE`

Joins, subqueries and set operations

Feature	PostgreSQL	MySQL 8	SQL Server	SQLite	Oracle
`FULL OUTER JOIN`	yes	– (emulate with `UNION`)	yes	yes (3.39)	yes
`RIGHT JOIN`	yes	yes	yes	yes (3.39)	yes
`USING`, `NATURAL JOIN`	yes	yes	–	yes	yes
`LATERAL`	yes	yes (8.0.14)	`CROSS APPLY` / `OUTER APPLY`	–	`LATERAL`, `CROSS/OUTER APPLY` (12c)
`INTERSECT`, `EXCEPT`	yes	yes (8.0.31)	yes	yes	`INTERSECT`, `MINUS` (`EXCEPT` 21c)
Row values `(a, b) IN (SELECT …)`	yes	yes	– (use `EXISTS`)	yes (3.15)	yes
`UPDATE` with a join	`UPDATE t SET … FROM s WHERE …`	`UPDATE t JOIN s ON … SET …`	`UPDATE t SET … FROM t JOIN s ON …`	`UPDATE … FROM` (3.33)	`MERGE`, or a correlated subquery (direct joins 23ai)

Aggregation, CTEs and windows

Feature	PostgreSQL	MySQL 8	SQL Server	SQLite	Oracle
String aggregation	`string_agg(x, ',' ORDER BY x)`	`GROUP_CONCAT(x ORDER BY x SEPARATOR ',')`	`STRING_AGG(x, ',') WITHIN GROUP (ORDER BY x)` (2017)	`group_concat(x, ',')` (`string_agg` 3.44)	`LISTAGG(x, ',') WITHIN GROUP (ORDER BY x)`
`FILTER (WHERE …)` on aggregates	yes	— (use `CASE`)	— (use `CASE`)	yes (3.30)	— (use `CASE`)
`avg()` of an integer column	exact `numeric`	decimal	**integer** (truncated)	floating point	exact `NUMBER`
Median	`percentile_cont(0.5) WITHIN GROUP (ORDER BY x)`	—	`PERCENTILE_CONT` as a window function only	—	`MEDIAN(x)`, `PERCENTILE_CONT`
`ROLLUP` / `CUBE` / `GROUPING SETS`	all three	`WITH ROLLUP` only	all three	—	all three
Recursive CTE keyword	`WITH RECURSIVE`	`WITH RECURSIVE`	plain `WITH`	`WITH RECURSIVE`	plain `WITH` (column list required)
Window functions	yes (8.4)	yes (8.0)	ranking 2005, frames 2012	yes (3.25)	yes (8i)
`GROUPS` frames, `EXCLUDE`	yes (11)	—	—	yes (3.28)	yes (21c)
`WINDOW w AS (…)` clause	yes	yes	yes (2022)	yes	yes (21c)
`IGNORE NULLS` in `lag` / `first_value`	— (19, in beta as of September 2026)	—	yes (2022)	—	yes
`QUALIFY`	—	—	—	—	26ai
`DISTINCT ON`	yes	—	—	—	—

DDL, DML and transactions

Feature	PostgreSQL	MySQL 8	SQL Server	SQLite	Oracle
Upsert	`INSERT … ON CONFLICT`, `MERGE` (15)	`INSERT … ON DUPLICATE KEY UPDATE`, `REPLACE`, `INSERT IGNORE`	`MERGE`	`INSERT … ON CONFLICT` (3.24), `INSERT OR REPLACE`	`MERGE`
Return changed rows	`RETURNING`	—	`OUTPUT inserted.*`	`RETURNING` (3.35)	`RETURNING … INTO` (PL/SQL)
DDL inside a transaction	yes, can be rolled back	no, causes an implicit commit	mostly yes	yes	no, DDL commits
`CHECK` enforced	yes	yes (8.0.16; parsed but ignored before)	yes	yes	yes
Default isolation level	Read Committed	Repeatable Read	Read Committed (locking)	Serializable	Read Committed
"Serializable" means	SSI (true serializability)	2PL: plain reads become locking reads	2PL (or `SNAPSHOT` level)	one writer at a time	snapshot isolation (write skew possible)
Stored-procedure language	PL/pgSQL (and others)	SQL/PSM-style	T-SQL	— (host language)	PL/SQL
Materialized views	yes, manual `REFRESH`	—	indexed views	—	yes, with incremental (fast) refresh

Indexes and plans

Feature	PostgreSQL	MySQL 8 (InnoDB)	SQL Server	SQLite	Oracle
Table storage	heap + separate indexes	clustered on the primary key	heap or clustered index	B-tree keyed by rowid	heap (index-organized tables optional)
Page size	8 KB	16 KB	8 KB	4 KB default	(not covered)
Partial index	yes	—	filtered index	yes	— (function-based index returning NULL)
Covering `INCLUDE`	yes (11)	— (add to the key)	yes	—	—
Expression index	yes	functional key parts (8.0.13)	index a computed column	yes	function-based index
Hash index	yes (crash-safe since 10)	MEMORY engine only; InnoDB builds an adaptive hash index itself	memory-optimized tables	—	hash clusters
Show the plan	`EXPLAIN (ANALYZE, BUFFERS)`	`EXPLAIN ANALYZE` (8.0.18), `EXPLAIN FORMAT=TREE`	actual execution plan, `SET STATISTICS IO ON`	`EXPLAIN QUERY PLAN`	`EXPLAIN PLAN FOR` + `DBMS_XPLAN.DISPLAY`

Dates and functions

Task	PostgreSQL	MySQL 8	SQL Server	SQLite	Oracle
Now	`now()`	`NOW()`	`SYSDATETIME()`, `GETDATE()`	`datetime('now')`	`SYSTIMESTAMP`, `SYSDATE`
Add 3 days	`d + 3`, `ts + interval '3 days'`	`DATE_ADD(d, INTERVAL 3 DAY)`	`DATEADD(day, 3, d)`	`date(d, '+3 days')`	`d + 3`, `d + INTERVAL '3' DAY`
Days between	`d2 - d1`	`DATEDIFF(d2, d1)`	`DATEDIFF(day, d1, d2)`	`julianday(d2) - julianday(d1)`	`d2 - d1`
Truncate to month	`date_trunc('month', d)`	`DATE_FORMAT(d, '%Y-%m-01')`	`DATETRUNC(month, d)` (2022)	`date(d, 'start of month')`	`TRUNC(d, 'MM')`
Year of a date	`extract(year FROM d)`	`YEAR(d)`	`YEAR(d)`	`strftime('%Y', d)`	`EXTRACT(YEAR FROM d)`
Characters / bytes	`length` / `octet_length`	`CHAR_LENGTH` / `LENGTH`	`LEN` (ignores trailing spaces) / `DATALENGTH`	`length` / `length(CAST(s AS BLOB))`	`LENGTH` / `LENGTHB`
If-then-else shortcut	`CASE`	`IF(c, a, b)`	`IIF(c, a, b)`	`iif(c, a, b)` (3.32)	`DECODE`, `CASE`
Random number 0–1	`random()`	`RAND()`	`RAND()`	`random() / 18446744073709551616.0 + 0.5`	`DBMS_RANDOM.VALUE`
JSON	`json`, `jsonb` (+ GIN)	`JSON` (binary)	`nvarchar` + `JSON_VALUE` etc. (native `json` type 2025)	JSON functions on text (built in since 3.38)	`JSON` type (21c)
Arrays	yes	–	–	–	collection types (`VARRAY`) in PL/SQL

20.17 Verified behaviours

Every concrete behaviour quoted on these sheets – `7 / 2 = 3`, `NULL = NULL` being unknown, rank ties, the default window frame, `NOT IN` with a NULL, upsert, `->>` returning text – is asserted by `code/01_cheatsheet_checks.sql`. Fixtures and a small checker live in `code/00_schema.sql`:

```
CREATE FUNCTION chk(p_area text, p_claim text, p_got text, p_expected text)
RETURNS void LANGUAGE plpgsql AS $$
BEGIN
  IF p_got IS DISTINCT FROM p_expected THEN
    RAISE EXCEPTION 'cheat-sheet claim FAILED [%] %: got %, expected %',
      p_area, p_claim, coalesce(p_got, 'NULL'), coalesce(p_expected, 'NULL');
  END IF;
  INSERT INTO check_log (area, claim, got, expected)
  VALUES (p_area, p_claim, coalesce(p_got, 'NULL'), coalesce(p_expected, 'NULL'));
END $$;
```

`chk` compares with `IS DISTINCT FROM`, so an expected NULL is checked properly (a plain `<>` would let a NULL result slip through – the trap of 20.2). Checks are grouped into `DO` blocks by area, for example:

```
DO $$ BEGIN
  PERFORM chk('NULL', '2 NOT IN (1, NULL) is NULL, not TRUE',     (2 NOT IN (1, NULL))::text, NULL);
  -- ...
END $$;

DO $$ BEGIN
  PERFORM chk('Windows', 'default frame with ORDER BY is RANGE: peers share the running total',
              (SELECT string_agg(s::text, ',' ORDER BY day, amount)
                 FROM (SELECT day, amount, sum(amount) OVER (ORDER BY day) AS s FROM sales) t),
              '10,40,40,45');
  -- ... (rank ties; later blocks: transactional TRUNCATE, JSONB, arrays, FTS — see the file)
END $$;
```

A wrong claim stops the file with, e.g., `ERROR: cheat-sheet claim FAILED [Functions] …: got 2/2, expected 1/2` — that exact message appeared once while writing this chapter, when the scratch database was accidentally created with `SQL_ASCII` encoding, where `length('é')` counts bytes (the runner always creates UTF-8 databases). Output (`bash tools_run_sql.sh sql '20-*'` → `OK`): claims per area — NULL 18, Functions 18, Aggregation 12, Joins 5, Subqueries 3, Set ops 5, DISTINCT 1, ORDER BY 4, WHERE 4, Windows 12, CTEs 3, Constraints 3, Upsert 2, DML 1, DDL 1, Transactions 2, JSONB 5, Arrays 3, FTS 2 — `total_claims_verified` **104**.

The SQLite column of the dialect table is checked by `code/02_sqlite_dialect.sqlite.sql`. SQLite has no `DO` blocks, so each claim is inserted into a table whose `CHECK (ok = 1)` fails the script when a claim is false:

```
CREATE TABLE claim (what TEXT, ok INTEGER CHECK (ok = 1));
INSERT INTO claim VALUES
 ('7/2 = 3 (integer division)',              7 / 2 = 3),
 ('a || NULL is NULL',                       ('a' || NULL) IS NULL),
 -- ...
 ('UNIQUE allows several NULLs',             (SELECT count(*) FROM u) = 2);
BEGIN; CREATE TABLE gone (x); ROLLBACK;        -- DDL is transactional in SQLite too
INSERT INTO claim SELECT 'DDL rolls back', (SELECT count(*) FROM sqlite_master WHERE name = 'gone') = 0;
```

Output: `28 SQLite claims verified`.

20.18 Interview pattern catalogue

Most SQL interview questions are one of about twenty patterns. Recognise the pattern from the wording, write the template, then adapt it. Method and edge-case checklist: §18.1 and §19.1. Full catalogues: §18.10 and §19.11.

Pattern	Wording that gives it away	Core technique	Solved examples
Plain filter	"find rows where …", "not equal to"	`WHERE` with NULL-aware logic (`IS DISTINCT FROM`)	P1, P2, P4, 19.2.1
String and regex filter	"valid e-mail", "longer than N characters"	`~` with anchors, `char_length`	P6, P7
Row-level classification	"label each row", "is it a triangle"	`CASE`	P8, 19.10.3
Group and count	"per", "for each", "how many"	`GROUP BY` + aggregate, `HAVING`	P10, P12, P14, 19.2.2
Zero counts included	"including those with none"	`LEFT JOIN` + `count(right.col)` or `CROSS JOIN` grid	P20, 19.2.3
Distinct lists per group	"list the products sold each day"	`count(DISTINCT)` , `string_agg(DISTINCT … ORDER BY …)`	P15, P16, 19.10.2
Semi-join	"who has at least one"	`EXISTS`	P22
Anti-join	"never", "without", "no sales"	`NOT EXISTS` (not `NOT IN`)	P23, P24, P25, P26
Conditional aggregation / static pivot	"one column per status"	`count(*) FILTER (WHERE …)` , `SUM(CASE …)`	P27, P28, 19.6.1
Dynamic pivot / unpivot	"one column per store", columns unknown	`format('%I')` + `EXECUTE` , `LATERAL (VALUES …)` , `jsonb_each_text`	19.6.2, 19.6.3
Date ranges	"in February 2020", "last 30 days"	half-open `>= start AND < next_start`	P23, P29, P30, P31
As-of value	"price on that date"	latest row `<=` the date (`DISTINCT ON` or `LIMIT 1`) + default	P33
Ratio / rate	"percentage", "rate", "share"	`100.0 * …` , `NULLIF(denominator, 0)` , round last	P34, P35, P36, P37, 19.9.1, 19.9.2, 19.9.3
Nth highest	"second highest", "NULL if none"	`DISTINCT … OFFSET n-1 LIMIT 1` in a scalar subquery, or `dense_rank`	P39, 19.3.1
Top-N per group	"top 3 in each department"	`dense_rank() OVER (PARTITION BY …)` in a CTE, filter outside	19.3.2
First / last row per group	"first login device", "latest status"	`DISTINCT ON` , `row_number() = 1`	P40, 19.3.3
Ties must be kept	"all winners", "include ties"	`rank() = 1` , `WITH TIES` , `= (SELECT max …)`	P21, 19.3.8, 19.3.9
Running total / cut-off	"so far", "cumulative", "who fits"	`sum() OVER (ORDER BY …)`	P41, 19.4.1, 19.4.2
Compare with previous row	"improved", "change since last"	`lag()`	19.4.3, 19.5.6
Compare with group	"above the department average"	`avg() OVER (PARTITION BY …)`	P42, 19.7.5
Gaps and islands / streaks	"N in a row", "consecutive", "longest streak"	`value − row_number()` , or difference of two row numbers	P32, 19.5.1, 19.5.2, 19.5.3, 19.5.5

Pattern	Wording that gives it away	Core technique	Solved examples
Pairs stored once	friends, calls, matches	`UNION ALL` both directions	19.7.1, 19.7.2, 19.7.4
Relational division	"took every course"	`HAVING count(DISTINCT x) = (SELECT count(*) …)`	19.7.6
Retention / cohorts	"came back the next day"	first date per user, then join on `first + 1`	P38, 19.8.1
Histogram with empty buckets	"include zero"	`generate_series` + `LEFT JOIN`	19.8.2
Largest unique value	"largest number that appears once"	`HAVING count(*) = 1`, then `max` (NULL when none)	P9

20.19 Interview questions and answers

The concept questions that used to be answered here are answered once, in 30–60-second form with a link to the full explanation, in Chapter 21 Part A. Index by topic (Q-numbers are Chapter 21's):

Topic	Question → Chapter 21
Fundamentals and keys	database vs DBMS vs SQL → Q1 · DDL/DML/DQL/DCL/TCL → Q2 · PK vs unique vs FK → Q4 · superkey vs candidate key → Q5 · natural vs surrogate key → Q6 · `DELETE` vs `TRUNCATE` vs `DROP` → Q9
SELECT, NULL logic and query order	evaluation order of a `SELECT` → Q13 · NULL and `x = NULL` → Q14 · why `NOT IN` returns nothing → Q16 · `WHERE` vs `HAVING` → Q18 · `count(*)` vs `count(col)` vs `count(DISTINCT col)` → Q19 · why `OFFSET` paging is slow → Q22
Joins, subqueries and set operations	join types → Q25 · `LEFT JOIN` behaving like inner → Q27 · anti-join → Q28 (semi-join: Q31) · correlated subquery → Q30 · `EXISTS` vs `IN` → Q31 · `UNION` vs `UNION ALL` → Q32
Aggregation, CTEs, views and window functions	CTE vs subquery → Q36 · recursive CTE → Q37 · view vs materialized view → Q38 · window vs `GROUP BY` → Q40 · `ROW_NUMBER` / `RANK` / `DENSE_RANK` → Q41 · frames, repeated running totals, `LAST_VALUE` → Q43 · gaps and islands → Q45 · Nth highest salary → Q172
Normalization and data modeling	anomalies / why normalize → Q49 · 1NF–BCNF → Q50 · denormalization → Q53 · many-to-many → Q54
Indexes, EXPLAIN and query optimization	B-tree → Q60 · clustered vs non-clustered → Q61 · composite column order → Q63 · covering index → Q64 · index not used → Q65 · reading `EXPLAIN ANALYZE` → Q67 · join algorithms → Q69 · N+1 → Q71
Transactions, isolation and locking	ACID → Q74 · anomalies → Q75 · isolation levels → Q76 · PostgreSQL's Repeatable Read → Q77 · lost updates → Q78 · write skew → Q79 · optimistic vs pessimistic → Q80 · deadlocks → Q82 · job queue → Q84 · MVCC → Q85
Procedures, security, scaling, NoSQL	SQL injection → Q93 · partitioning vs sharding vs replication → Q99 · sync vs async replication → Q101 · CAP and PACELC → Q104 · relational vs NoSQL → Q106 · backup vs replica (PITR) → Q109

One design question not in the bank:

- **What is a weak entity?** An entity that cannot be identified without its owner, such as an order line within an order. Its key is the owner's key + a partial key. → §10.9

20.20 Glossary (A–Z)

Every term the book uses, defined once in plain words; the last column links to the section that explains it with examples. Terms defined in a table on the cheat sheets above (join types, normal forms, isolation anomalies, plan nodes…) point there instead of repeating it. The course's key-term review cards (problems that made databases necessary, ER notation, keys, FDs, normal forms, modeling levels, case-study judgement calls) are included; the judgement calls are the 20.10 table.

601 terms. Jump to: 0–9 · A · B · C · D · E · F · G · H · I · J · K · L · M · N · O · P · Q · R · S · T · U · V · W · X · Y · Z

0–9

Term	Meaning	See
1NF (first normal form)	Defined in the 20.10 sheet.	§11.7
2NF (second normal form)	Defined in the 20.10 sheet.	§11.8
2PC (two-phase commit)	Distributed commit protocol: the coordinator asks every participant to prepare, then tells all to commit or all to abort. Blocks if the coordinator fails after prepare.	§15.9
2PL (two-phase locking)	A transaction acquires locks in a growing phase and releases them in a shrinking phase, never acquiring after releasing. Guarantees serializable schedules.	§13.10
3-2-1 backup rule	Keep three copies of the data on two different kinds of storage, with one copy off-site.	§9.12
3NF synthesis / BCNF decomposition	The two standard normalization algorithms: synthesis builds 3NF tables from a minimal cover (always lossless and dependency-preserving); BCNF decomposition splits on violating FDs (lossless, may lose an FD).	§11.14
3NF (third normal form)	Defined in the 20.10 sheet.	§11.9
4NF (fourth normal form)	Defined in the 20.10 sheet.	§11.12
5NF (fifth normal form)	Defined in the 20.10 sheet.	§11.13

A

Term	Meaning	See
ACID	Atomicity, consistency, isolation, durability: the four guarantees of a database transaction.	§13.1
Adjacency list	Storing a tree or graph as rows that each point to their parent (a `parent_id` column).	§7.4
Advisory lock	A PostgreSQL lock on a number the application chooses (`pg_advisory_lock`), not tied to any row or table.	§13.10
Aggregate function	A function that turns many input rows into one value, such as `count`, `sum`, `avg`, `min`, `max`.	§4.1
Aggregate push-down	Computing partial aggregates close to the data (on each shard, or before a join) so that fewer rows travel.	§15.4
Alias	A temporary name for a column (`AS total`) or a table (`FROM orders o`) within one query.	§2.1
Alternate key	Any candidate key other than the one picked as primary key. Enforced with `UNIQUE`.	§10.13
ALTER TABLE	DDL statement that changes an existing table: add, drop, rename or retype columns, add constraints.	§9.5
Analytic function	Another name for a window function (common in Oracle documentation).	§8.1
ANALYZE	Command that samples a table and refreshes the planner's statistics.	§12.16
Anchor (recursive CTE)	The non-recursive first part of a recursive CTE, which produces the starting rows.	§7.3
Anomaly (data)	An insertion, update or deletion problem caused by storing the same fact in more than one row.	§11.6
Anomaly (isolation)	A result that could not happen if transactions ran one at a time, such as a dirty read or write skew.	§13.4
ANSI / ISO SQL standard	The official SQL specification that every vendor implements, each with its own extensions and gaps.	§1.9
Anti-join	Defined in the 20.5 sheet.	§5.8
ANY / SOME / ALL	Defined in the 20.6 sheet.	§6.5
AOF (append-only file)	Redis persistence mode that logs every write command so the data can be rebuilt after a restart.	§16.11
API	Application programming interface: the set of calls one program offers to others.	§1.8
Append-only table	A table that only ever gets new rows, never updates in place, such as an event log or a ledger.	§17.14
ARIES	A classic crash-recovery algorithm: analyse the log, redo all logged history, then undo transactions that never committed.	§15.9
Armstrong's axioms	Reflexivity, augmentation and transitivity: sound and complete rules for deriving functional dependencies.	§11.3
Array (PostgreSQL)	A column type holding an ordered list of values of one type, indexed from 1.	§14.5
Asynchronous replication	The primary commits without waiting for replicas. Fast, but a failover can lose the newest writes.	§15.5
Atomicity	A transaction's changes happen completely or not at all.	§13.2
Attribute	A named property of an entity or a column of a relation.	§10.6

Term	Meaning	See
Attribute closure (X^+)	Every attribute functionally determined by the set X under a given set of FDs.	§11.4
Audit columns	Columns such as `created_at`, `updated_at` and `created_by` that record when and by whom a row was changed.	§17.14
Augmentation (FD rule)	Armstrong's rule: if X → Y, then XZ → YZ for any attributes Z.	§11.3
Autocommit	Mode in which every statement is its own transaction and commits immediately.	§13.1
Autovacuum	PostgreSQL's background worker that runs VACUUM and ANALYZE automatically as tables change.	§12.16
Availability	The share of time a system answers requests. In CAP: every request to a working node gets an answer, possibly stale.	§15.1

B

Term	Meaning	See
Backfill	Filling in values for existing rows after adding a new column or rule.	§9.5
BASE	Defined in the 20.15 sheet.	§16.1
Base backup	A physical copy of a PostgreSQL server's whole data directory: the starting point for PITR or a new replica.	§9.12
BCNF (Boyce–Codd normal form)	Defined in the 20.10 sheet.	§11.11
BEGIN / COMMIT / ROLLBACK	Start a transaction, make its changes permanent, or discard them.	§13.1
BETWEEN	Inclusive range test: `x BETWEEN a AND b` means `x >= a AND x <= b`.	§2.4
Big data	Data too large, too fast or too varied for one ordinary database server (volume, velocity, variety).	§16.9
BigQuery / Snowflake / Redshift	Managed cloud columnar data warehouses for analytical SQL over very large tables.	§16.8
Binary search	Finding a value in sorted data by repeatedly halving the search range: about $\log_2 n$ steps.	§12.5
Bind parameter (placeholder)	A marker such as `$1`, `%s` or `?` in the SQL text whose value the driver sends separately.	§14.9
BitmapAnd / BitmapOr	Plan nodes that combine the row bitmaps of several indexes with AND or OR before reading the table.	§12.10
Bitmap scan	Defined in the 20.11 sheet.	§12.10
Bloat	Space wasted in a table or index by dead row versions and half-empty pages.	§13.14
Block	Another word for page: the fixed-size unit in which the disk and the DBMS read and write data.	§12.1
Blocking factor	How many records fit in one disk block (block size ÷ record size).	§12.5
Bloom filter	Compact probabilistic set that answers "definitely not present" or "probably present".	§12.8
BM25	The standard relevance score used by search engines such as Elasticsearch: rewards rare query words that appear often in a short document.	§16.11
BRIN index	Defined in the 20.11 sheet.	§12.9
BSON	MongoDB's binary form of JSON, with extra types such as dates and 64-bit integers.	§16.3
B-tree / B+ tree	Balanced, shallow, sorted tree index. In a B+ tree all keys live in linked leaf pages, which makes range scans cheap. PostgreSQL's default index.	§12.6
Bucket (hashing)	A slot, usually a page, that a hash function sends keys to.	§12.3
Buffer pool (shared buffers)	Memory area where the DBMS caches data pages read from disk.	§1.4
Bully algorithm	A leader-election method in which the live node with the highest id takes over.	§15.8
Byzantine fault	A failure in which a node keeps running but sends wrong or malicious messages, rather than simply stopping.	§15.8

C

Term	Meaning	See
Cache	A fast store, usually memory, that holds copies of often-read data so the slower source is asked less often.	§1.4
Cache invalidation	Removing or refreshing cached copies when the underlying data changes.	§7.8
Calendar table	A table (or `generate_series`) with one row per day, joined to data so that days with no activity still appear.	§7.6
Candidate key	A minimal set of attributes that uniquely identifies every row.	§10.13
Canonical pair	Storing an undirected pair (two friends) once in a fixed order, such as `user_a < user_b`.	§17.13
CAP theorem	Defined in the 20.14 sheet.	§15.7
Cardinality (ER)	The maximum number of instances on each side of a relationship: 1:1, 1:N or M:N.	§10.7
Cardinality estimate	The planner's guess of how many rows a plan node will produce.	§12.16
Cardinality (relation)	The number of rows in a relation.	§10.12
Cartesian product	Every row of one table paired with every row of another.	§5.1
CASCADE	Referential action: deleting or updating a parent row does the same to its child rows.	§9.3
CASE	Conditional expression returning the first `THEN` value whose `WHEN` is true, else the `ELSE` value (or NULL).	§3.5
Cassandra / ScyllaDB	Leaderless wide-column databases built for very high write rates spread over many nodes.	§16.4
Cast	Converting a value to another type: `CAST(x AS int)` or, in PostgreSQL, `x::int`.	§3.1
Catalog (data dictionary)	System tables describing all database objects: tables, columns, types, indexes, privileges.	§1.2
Causal consistency	Reads respect cause and effect: nobody sees a reply before the message it answers.	§15.6
CDC (change data capture)	Streaming committed row changes out of a database, usually by reading its log, into other systems.	§16.10
CHECK constraint	A row-level rule that rejects any row for which the expression is FALSE.	§9.3
CHECK OPTION (views)	`WITH CHECK OPTION` makes an updatable view reject inserts and updates whose rows the view would not show.	§7.7
Checkpoint	Point at which all dirty pages are flushed to disk, bounding how much log must be replayed after a crash.	§15.9
Chen notation	ER notation that draws entities as rectangles, attributes as ellipses and relationships as diamonds.	§10.5
Citus	A PostgreSQL extension that shards tables across several PostgreSQL servers and runs queries on them in parallel.	§15.3
ClickHouse	An open-source columnar database for fast analytical queries.	§12.1
Click-through rate (CTR)	Clicks divided by impressions (times the ad was shown), usually as a percentage.	§18.8
Closure of an FD set (F^+)	All functional dependencies implied by a given set F.	§11.3

Term	Meaning	See
Closure table	A tree-storage pattern with one row for every ancestor–descendant pair, so subtree queries need no recursion.	§7.4
Clustered index	An index whose order is the physical order of the table rows (InnoDB primary key, SQL Server clustered index).	§12.4
Clustering column	In a wide-column store, the column that sorts the rows inside one partition.	§16.4
Clustering index	An index on a non-key field by which the data file is physically sorted.	§12.4
COALESCE	Returns the first non-NULL argument.	§3.5
CockroachDB	A distributed SQL database that speaks the PostgreSQL protocol and replicates data ranges with Raft.	§16.7
Cohort	A group of users who started in the same period, followed over time to measure retention.	§19.8
Collation	Rules that decide how text is compared and sorted, including case sensitivity.	§2.7
Co-location	Keeping related rows (one customer's orders and items) on the same shard so joins stay on one node.	§15.3
Column family	A group of columns stored together in wide-column stores such as HBase and Cassandra.	§16.4
Column store (columnar)	Storage that keeps each column's values together, making scans and compression of a few columns efficient.	§12.2
Compaction	Background merging of sorted files in an LSM-tree store, which throws away overwritten and deleted values.	§16.4
Compare-and-set (CAS)	An atomic "change it only if it still has the value I read" operation.	§13.1
Composite attribute	An attribute made of parts, such as an address made of street, city and postcode.	§10.6
Composite index	An index on several columns. Usable for filters on a leftmost prefix of its columns.	§12.11
Composite key	A key made of two or more columns.	§10.13
Compression	Storing data in fewer bytes. Column stores compress well because one column's values are alike.	§12.2
Conceptual model	Defined in the 20.10 sheet.	§10.2
Concurrency control	The mechanisms (locking, timestamps, MVCC, validation) that keep concurrent transactions correct.	§13.9
Conditional aggregation	Aggregating only the rows that meet a condition, with `FILTER (WHERE …)` or `sum(CASE …)`.	§4.4
Connection pool	A set of reusable open database connections shared by application threads.	§14.12
Consensus	Agreement of a group of nodes on one value or log order despite failures, as in Raft and Paxos.	§15.8
Conservative 2PL	A two-phase-locking variant that takes every lock a transaction needs before it starts, so it cannot deadlock.	§13.11
Consistency (ACID)	Every committed transaction leaves all constraints and business rules satisfied.	§13.3
Consistency (CAP)	In CAP: every read sees the most recent write, as if there were a single copy. Not the same as the C in ACID.	§15.7
Consistency model	The guarantee a distributed store gives about what reads can see: strong, causal, eventual and others.	§15.6

Term	Meaning	See
Consistent hashing	Defined in the 20.14 sheet.	§15.3
Constraint	A rule enforced by the DBMS on the data: NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY, CHECK, EXCLUDE.	§9.3
COPY	PostgreSQL's bulk command for loading or exporting many rows at once, far faster than row-by-row `INSERT`s.	§14.10
Correlated subquery	Defined in the 20.6 sheet.	§6.3
Cost (planner)	The planner's estimate of the work of a plan, in arbitrary units based on page reads and CPU.	§12.10
Covering index	An index containing every column a query needs, allowing an index-only scan.	§12.12
CPU	The processor that executes instructions. Sorting, hashing and expression evaluation cost CPU time.	§12.1
CRDT	Conflict-free replicated data type: a structure (counter, set) whose replicas can always be merged without conflicts.	§15.5
CROSS JOIN	Defined in the 20.5 sheet.	§5.1
Crosstab	A function from PostgreSQL's `tablefunc` extension that pivots rows into columns.	§14.7
Crow's foot notation	ER notation that draws cardinality and optionality with symbols at the ends of relationship lines.	§10.5
CRUD	Create, read, update, delete: the four basic operations on stored data.	§14.13
CSV	Comma-separated values: a plain-text table format with one row per line.	§1.1
CTE (common table expression)	Defined in the 20.7 sheet.	§7.1
CUBE	Grouping extension that produces subtotals for every combination of the listed columns.	§4.6
Cursor	A pointer for fetching a query result a batch of rows at a time.	§14.10
Cypher	Neo4j's graph query language, which writes patterns like `(a)-[:FOLLOWS]->(b)`.	§16.5

D

Term	Meaning	See
Database	An organised, persistent, self-describing collection of related data.	§1.2
Data independence	Changing how data is stored (physical) or parts of the schema (logical) without changing applications.	§1.3
Data integrity	The data is accurate and obeys its rules (types, keys, constraints).	§1.3
Data model	A description of the data's entities, attributes, relationships and rules.	§10.1
Data-modifying CTE	Defined in the 20.7 sheet.	§7.2
Data type	The kind of value a column holds (integer, text, date …), which decides what operations are allowed.	§3.1
Data warehouse	A database built for analysis, loaded from operational systems and tuned for large scans and aggregates.	§11.16
DBMS	Software that defines, stores, queries, protects and shares a database.	§1.3
DCL	Data control language: `GRANT` and `REVOKE`.	§1.9
DDL	Data definition language: `CREATE`, `ALTER`, `DROP`, `TRUNCATE`.	§1.9
Deadlock	Two or more transactions each waiting for a lock held by another, so none can proceed.	§13.11
Dead tuple	An old row version that no transaction can see any more, waiting to be removed by VACUUM.	§13.14
Declarative	Saying what result you want and letting the system decide how to compute it. SQL is declarative.	§1.4
Decomposition	Splitting a relation into smaller ones during normalization.	§11.10
Deduplication	Keeping one row per group of duplicates, often with `row_number() = 1` or `DISTINCT ON`.	§8.8
Deferrable constraint	A constraint whose check can wait until `COMMIT` (`DEFERRABLE INITIALLY DEFERRED`).	§9.3
Degree (ER)	The number of entity types taking part in a relationship: unary, binary, ternary.	§10.7
Degree (relation)	The number of attributes (columns) of a relation.	§10.12
DELETE	DML statement that removes the rows matching its `WHERE` clause.	§9.10
Delta encoding	Storing each value as the difference from the previous one, giving small numbers for sorted ids or timestamps.	§12.2
Denormalization	Deliberately storing redundant data to speed up reads, with a mechanism to keep it correct.	§11.16
Dense index	An index with one entry for every record or key value.	§12.4
DENSE_RANK	Defined in the 20.8 sheet.	§8.5
Dependency preservation	Every functional dependency can be checked within a single table after decomposition.	§11.10
Dependent (FD)	The right side of a functional dependency.	§11.1
Derived attribute	An attribute that can be computed from others, such as age from birth date.	§10.6
Derived table	Defined in the 20.6 sheet.	§6.6
Determinant	The left side of a functional dependency.	§11.1

Term	Meaning	See
Dialect	One vendor's version of SQL, with its own syntax and functions.	§1.9
Dictionary encoding	Replacing repeated values with small integer codes plus one lookup table of the distinct values.	§12.2
Differential / incremental backup	A differential backup copies everything changed since the last full backup. An incremental copies changes since the last backup of any kind.	§9.12
Dirty page	A page changed in memory but not yet written to disk.	§15.9
Dirty read	Defined in the 20.12 sheet.	§13.4
Dirty write	Defined in the 20.12 sheet.	§13.4
Disaster recovery	The plans and off-site copies that let you restore service after losing a whole server, site or data set.	§15.11
DISTINCT	Removes duplicate rows from a result. `DISTINCT ON (cols)` keeps the first row per group (PostgreSQL).	§2.6
Distributed SQL	Another name for NewSQL: SQL and ACID transactions over data sharded and replicated across many nodes.	§16.7
Distributed transaction	A transaction that changes data on more than one node, so it needs a protocol such as two-phase commit.	§15.9
DML	Data manipulation language: `INSERT`, `UPDATE`, `DELETE`, `MERGE`.	§1.9
Document database	Defined in the 20.15 sheet.	§16.3
Dollar quoting	PostgreSQL's `$$ … $$` string syntax, used for function bodies so inner quotes need no escaping.	§14.1
Domain	The set of allowed values for an attribute.	§10.12
Double-entry bookkeeping	Recording each money movement as a debit and an equal credit, so all entries always sum to zero.	§17.3
Downsampling	Replacing old detailed measurements with coarser summaries, such as hourly averages.	§16.8
DQL	Data query language: `SELECT`.	§1.9
Driver	The library a program uses to talk to a particular database, such as psycopg or a JDBC driver.	§14.10
DROP	DDL statement that removes a database object entirely.	§9.6
Dual writes	An application writing the same change separately to two systems, which drift apart if one write fails.	§16.11
Durability	Once committed, a transaction's changes survive crashes.	§13.5
Dynamic SQL	SQL text built at run time and executed with `EXECUTE`. Identifiers must be quoted safely with `format('%I', …)`.	§19.6
DynamoDB	Amazon's managed key-value and document database.	§16.2

E

Term	Meaning	See
Elasticsearch / OpenSearch	Distributed search engines built on Lucene that index JSON documents for full-text search and log analytics.	§16.10
ELK stack	Elasticsearch, Logstash and Kibana: a common set of tools to collect, search and chart logs.	§16.11
Embedded database	A database library that runs inside the application process, with no separate server. SQLite is the usual example.	§1.7
Entity	A distinguishable thing the database stores facts about, such as a customer or an order.	§10.6
Entity integrity	No primary-key value may be NULL.	§10.14
Entity set / entity type	The entity type is the kind of thing (Employee). The entity set is all its instances stored right now.	§10.6
Enum type	A type whose values must come from a fixed list, such as `'new', 'paid', 'shipped'`.	§3.1
Equi-join	A join whose condition is equality between columns.	§5.3
Erasure coding	Storing data plus computed parity pieces so lost pieces can be rebuilt, using less space than full copies.	§15.11
ER diagram	A diagram of entities, attributes and relationships.	§10.5
ETL	Extract, transform, load: copying data from source systems, reshaping it, and loading it into a warehouse.	§11.16
Eventual consistency	If writes stop, all replicas eventually return the same value.	§15.6
Eviction policy	The rule a cache uses to decide what to drop when it is full, such as LRU or LFU.	§16.2
Exactly-once	A processing guarantee that each message affects the result exactly one time, even when it is retried.	§16.9
EXCEPT	Defined in the 20.6 sheet.	§6.10
EXCLUDE constraint	PostgreSQL constraint that forbids two rows whose values conflict under given operators, such as overlapping ranges.	§17.2
EXCLUDED (ON CONFLICT)	In `INSERT … ON CONFLICT DO UPDATE`, the pseudo-row holding the values that were proposed for insertion.	§9.11
Exclusive lock (X)	A lock that only one transaction can hold. It blocks every other lock on the same item.	§13.10
Executor	The DBMS component that runs the chosen plan and produces the result rows.	§1.4
Existence dependency	An entity that cannot exist without another one, such as an order line without its order.	§10.9
EXISTS	True if the subquery returns at least one row.	§6.4
EXPLAIN / EXPLAIN ANALYZE	Show the chosen query plan, and with `ANALYZE` run it and report actual rows and times.	§12.10
Expression index	An index on the result of an expression, such as `lower(email)`.	§12.12
Extended statistics	PostgreSQL statistics on combinations of columns (`CREATE STATISTICS`), which help estimates for correlated columns.	§12.16
Extendible hashing	A dynamic hashing scheme that grows by doubling a directory of pointers and splitting only the bucket that overflowed.	§12.7

Term	Meaning	See
Extension (PostgreSQL)	An add-on package installed with `CREATE EXTENSION`, such as PostGIS or pg_trgm.	§15.2

F

Term	Meaning	See
Failover	Promoting a replica to primary when the primary fails.	§15.10
False positive	A "yes" that is actually wrong. A Bloom filter can give false positives but never false negatives.	§12.8
Fan-out	Row multiplication when a row joins to several matching rows, often inflating sums.	§5.9
Fan-out on write / on read	Two feed designs: push each new post into every follower's feed when it is written, or build the feed when it is read.	§17.4
Fault tolerance	Continuing to work correctly when some parts fail.	§15.1
Fencing	Preventing a deposed leader from continuing to write after failover.	§15.8
FETCH FIRST	Standard syntax for limiting rows: `FETCH FIRST n ROWS ONLY` or `WITH TIES`.	§2.8
File organization	How records are arranged in a file: heap, sorted, hashed or clustered.	§12.3
Fillfactor	How full PostgreSQL packs table or index pages, leaving the rest free for later updates.	§12.3
FILTER clause	Restricts the rows fed to one aggregate: `count(*) FILTER (WHERE …)`.	§4.4
Floating-point number	An approximate binary number (`real`, `double precision`) that cannot hold most decimal fractions exactly. Avoid it for money.	§3.1
Foreign data wrapper	A PostgreSQL extension that lets you query another database or a file as if it were a local table.	§15.4
Foreign key	Columns whose values must match a key in a referenced table (or be NULL).	§9.3
Frame (window frame)	The subset of the partition that an aggregate or value window function sees for the current row.	§8.4
fsync	The operating-system call that forces written data out of caches onto durable storage.	§15.9
Full functional dependency	An FD that needs its whole composite determinant: remove any column and it no longer holds.	§11.2
FULL OUTER JOIN	Defined in the 20.5 sheet.	§5.6
Full-text search	Searching documents by words with stemming and ranking. `tsvector @@ tsquery` in PostgreSQL.	§14.6
Functional dependency	`X → Y`: any two rows with equal X values have equal Y values.	§11.1
Fuzzy checkpoint	A checkpoint that writes dirty pages out gradually while transactions keep running, instead of pausing all work.	§15.9

G

Term	Meaning	See
Gaps and islands	Problems about runs of consecutive values or dates, solved by grouping on `value - row_number()`.	§8.9
Generated column	A column computed from other columns of the same row (`GENERATED ALWAYS AS (expr) STORED`).	§9.4
generate_series	PostgreSQL function that returns a series of numbers or timestamps as rows.	§7.6
Geohash	A short string that encodes a map cell from latitude and longitude. Nearby places share a prefix.	§16.8
GIN index	Defined in the 20.11 sheet.	§12.9
GiST index	Defined in the 20.11 sheet.	§12.9
Gossip protocol	Nodes spread cluster state by repeatedly swapping what they know with random peers.	§16.11
GQL	The ISO-standard graph query language (2024), close to Cypher.	§16.5
Graph database	Defined in the 20.15 sheet.	§16.5
Gremlin	A step-by-step graph traversal language used by Apache TinkerPop databases such as Amazon Neptune.	§16.5
GROUP BY	Groups rows that share values so aggregates are computed per group.	§4.2
GROUPING()	A function that returns 1 on a subtotal row where the column was rolled up, and 0 otherwise.	§4.6
GROUPING SETS	Computes several groupings in one query, such as per region, per product and a grand total.	§4.6
GROUPS frame	Defined in the 20.8 sheet.	§8.4

H

Term	Meaning	See
Half-open interval	A range that includes its start but not its end (`>= start AND < end`), so neighbouring ranges never overlap.	§2.4
HashAggregate / GroupAggregate	Plan nodes that group rows, either with an in-memory hash table or by reading input already sorted on the group key.	§12.15
Hash function	A function that turns a key into a fixed-size number. The same key always gives the same number.	§12.3
Hash index	Index that hashes keys into buckets. Supports equality lookups only.	§12.7
Hash join	Defined in the 20.11 sheet.	§12.15
Hash partitioning	Assigning rows to partitions by a hash of the key, spreading them evenly.	§15.2
Hash table	A structure that stores items in buckets chosen by a hash of the key, giving lookups in about constant time.	§12.7
HAVING	Filters groups after aggregation.	§4.3
HBase	A wide-column store on top of Hadoop's HDFS, modelled on Google's Bigtable.	§16.1
HDFS	Hadoop Distributed File System: stores huge files as large blocks copied to several machines.	§16.4
Heap	Unordered table storage: new rows go wherever there is space.	§12.1
Heartbeat	A periodic "I am alive" message. Missing heartbeats suggest that a node has failed.	§15.8
High availability	Keeping a service running through failures, usually with replicas and automatic failover.	§15.10
Hinted handoff	A node temporarily keeps writes meant for an unreachable replica and hands them over when it comes back.	§15.5
Histogram (statistics)	Buckets of a column's value distribution that the planner uses to estimate how many rows a condition keeps.	§12.16
Horizontal scaling	Adding more machines (scale out) rather than a bigger one.	§15.1
Hot partition (hotspot)	A partition or shard that receives far more traffic than the others and becomes the bottleneck.	§15.2
HTAP	Hybrid transactional/analytical processing: one system that serves OLTP and analytics together.	§16.6
Hybrid logical clock	A timestamp made of physical time plus a logical counter, used to order events across machines.	§16.7

I

Term	Meaning	See
IaaS / PaaS / SaaS	Cloud service models: rent machines (infrastructure), a managed platform, or finished software.	§1.5
Idempotent	Having the same effect whether it runs once or several times, which makes retries safe.	§17.3
Identifier	The name of a table, column or other object. Double quotes make it case-sensitive (`"Name"`).	§2.1
Identifying relationship	The relationship that connects a weak entity to its owner and supplies part of its key.	§10.9
Identity column	A column whose values are generated automatically: `GENERATED ALWAYS AS IDENTITY`.	§9.4
Impedance mismatch	The friction between objects in code (nested, linked by references) and flat relational tables.	§14.13
IN	True if a value equals any element of a list or subquery result.	§6.2
Index	An auxiliary structure that finds rows by key without scanning the whole table.	§12.4
Index-only scan	Defined in the 20.11 sheet.	§12.10
Index scan	Defined in the 20.11 sheet.	§12.10
InfluxDB	A time-series database for metrics and sensor data.	§16.8
information_schema	The SQL-standard set of views that describe the database's tables, columns and constraints.	§1.2
Inlining (CTE)	The planner folding a CTE into the main query so it can be optimised together with it (PostgreSQL 12+).	§7.1
In-memory database	Defined in the 20.15 sheet.	§16.6
INNER JOIN	Defined in the 20.5 sheet.	§5.3
InnoDB	MySQL's default storage engine: transactional, with each table stored clustered on its primary key.	§1.4
INSERT	DML statement that adds rows.	§9.8
Integer division	Dividing two integers drops the fraction: in PostgreSQL `7 / 2` is `3`.	§2.3
Intention lock (IS, IX)	A table-level lock announcing that the transaction will lock rows inside the table, so table and row locks can be checked quickly.	§13.10
INTERSECT	Defined in the 20.6 sheet.	§6.9
Interval	A PostgreSQL type for a length of time, such as `interval '2 hours'`.	§3.4
Inverted index	An index that maps each word or element to the list of rows or documents that contain it.	§12.9
I/O	Input/output: reading from or writing to disk or the network, usually the slowest part of running a query.	§12.1
IoT	Internet of Things: networked sensors and devices that send streams of measurements.	§16.4
ISAM	Indexed sequential access method: an older file organization of sorted records with a static multilevel index.	§12.3
Isolation	Concurrent transactions do not observe each other's intermediate states.	§13.4

Term	Meaning	See
Isolation level	How strongly a transaction is isolated: Read Uncommitted, Read Committed, Repeatable Read or Serializable.	§13.4

J

Term	Meaning	See
JDBC	Java Database Connectivity: Java's standard API for running SQL through a database driver.	§14.11
Join	Combining rows of two tables according to a condition.	§5.2
Join dependency	A relation equals the join of some of its projections.	§11.13
JPA / Hibernate	Java's standard ORM interface and its most common implementation.	§12.13
JSON	JavaScript Object Notation: a text format of nested objects, arrays, strings, numbers, booleans and null.	§14.4
JSONB	PostgreSQL's binary JSON type, which supports indexing and containment queries.	§14.4
Junction table	A table that implements a many-to-many relationship with foreign keys to both sides.	§10.15

K

Term	Meaning	See
Kafka	Apache Kafka: a distributed, replicated log used to stream events between systems.	§16.9
Keyset pagination	Paging by remembering the last key seen (`WHERE key > :last`) instead of using `OFFSET` .	§12.14
Key-value store	Defined in the 20.15 sheet.	§16.2

L

Term	Meaning	See
LAG / LEAD	Defined in the 20.8 sheet.	§8.6
Last-write-wins (LWW)	A conflict rule that keeps the value with the latest timestamp and silently drops the others.	§15.5
Latency	The time a single operation takes, from request to response.	§1.5
LATERAL	Defined in the 20.5 sheet.	§6.7
Lazy / eager loading	ORM strategies: fetch related objects only when they are first used (lazy, which risks N+1) or up front with a join (eager).	§12.13
Leader election	Choosing one node to coordinate writes, typically via a consensus system.	§15.8
Leader / follower	In replication, the leader (primary) accepts writes and the followers (replicas, standbys) copy its changes.	§15.5
Leaderless replication	Any replica accepts writes. Clients write to and read from several replicas and rely on quorums.	§15.5
Lease	A lock or leadership right that expires automatically after a set time unless renewed.	§15.8
Ledger	An append-only table of money or stock movements. Balances are sums over it.	§17.3
Leftmost prefix rule	A composite index on (a, b, c) helps filters on a, (a, b) or (a, b, c), not on b alone.	§12.11
LEFT OUTER JOIN	Defined in the 20.5 sheet.	§5.4
Lexeme	A normalised word (after stemming) stored in a `tsvector`.	§14.6
LIKE / ILIKE	Pattern matching with `%` and `_`. `ILIKE` is case-insensitive (PostgreSQL).	§2.4
LIMIT / OFFSET	Return at most n rows, after skipping k rows.	§2.8
Linear hashing	A dynamic hashing scheme that splits buckets one at a time in a fixed order as the file grows.	§12.7
Linearizability	The strongest consistency model: each operation appears to take effect at one instant, in real-time order. Also called strong consistency.	§15.6
List partitioning	Assigning rows to partitions by a list of discrete values, such as region codes.	§15.2
Lock	A marker that restricts other transactions' access to a row or table until released.	§13.10
Lock manager	The DBMS component that grants, queues and releases locks.	§1.4
Logical model	Defined in the 20.10 sheet.	§10.2
Logical query processing order	The conceptual order in which the clauses of a SELECT are evaluated, starting with FROM.	§4.5
Logical replication	Replicating row changes (insert, update, delete) for chosen tables, rather than copying raw WAL bytes.	§15.5
Lossless join	A decomposition whose pieces join back to exactly the original rows.	§11.10
Lost update	Defined in the 20.12 sheet.	§13.7
LRU / LFU	Cache eviction policies: drop the least recently used, or the least frequently used, item first.	§16.2
LSM tree	Log-structured merge tree: writes go to memory and sorted files that are merged in the background.	§16.4

Term	Meaning	See
LSN (log sequence number)	A position in the write-ahead log, used to order changes and to track recovery and replication progress.	§13.5
Lucene	The Java search library inside Elasticsearch and OpenSearch.	§16.11

M

Term	Meaning	See
MapReduce	A batch-processing model: map each record to key-value pairs, group them by key, then reduce each group to a result.	§16.9
MariaDB	An open-source fork of MySQL.	§1.7
Materialized path	Storing each tree node's full ancestor path, such as `/1/4/9/`, in a column.	§7.4
Materialized view	Defined in the 20.7 sheet.	§7.8
Memcached	A simple distributed in-memory key-value cache.	§16.6
MERGE	Standard statement that inserts, updates or deletes target rows based on a source (PostgreSQL 15+).	§9.11
Merge join	Defined in the 20.11 sheet.	§12.15
Merkle tree	A tree of hashes in which each parent hashes its children, so two replicas can find differences by comparing a few hashes.	§15.5
Metadata	Data about data, such as table and column definitions.	§1.2
Minimal cover	An equivalent set of FDs with single-attribute right sides, no extraneous attributes and no redundant FDs.	§11.5
Mirroring	Keeping an exact, continuously updated copy of a disk or database on another device.	§15.11
MongoDB	A widely used document database that stores BSON documents in collections.	§16.3
Monotonic reads	Once you have seen a value, later reads never show you an older one.	§15.6
Moving average	The average of the last n values, recomputed for every row.	§8.7
Multi-leader replication	Several nodes accept writes and replicate to each other, so conflicting writes must be resolved.	§15.5
Multivalued attribute	An attribute that can hold several values for one entity, such as phone numbers.	§10.6
Multivalued dependency	`X ↠ Y`: for each X, the set of Y values is independent of the other attributes.	§11.12
MVCC	Multiversion concurrency control: keep several row versions so readers see a snapshot and do not block writers.	§13.14
MySQL	A widely used open-source relational database.	§1.7

N

Term	Meaning	See
N+1 query problem	Running one query for a list, then one more per item, instead of a single set-based query.	§12.13
Natural join	Defined in the 20.5 sheet.	§5.10
Natural key	A key taken from real-world data, such as an ISBN or email.	§10.13
Neo4j	A widely used graph database, queried with Cypher.	§16.10
Nested loop join	Defined in the 20.11 sheet.	§12.15
Network partition	A network failure that splits the nodes into groups that cannot reach each other.	§15.1
NewSQL	Distributed databases that keep SQL and ACID while scaling out, such as Spanner and CockroachDB.	§16.7
Non-equi join	A join whose condition is not plain equality, such as `BETWEEN` or `<`.	§5.1
Non-repeatable read	Defined in the 20.12 sheet.	§13.4
Normal form	A level of design quality (1NF, 2NF, 3NF, BCNF …) defined by which dependencies a table may contain.	§11.6
Normalization	Organising tables so each fact is stored once, guided by functional dependencies and normal forms.	§11.6
NoSQL	Non-relational databases: key-value, document, wide-column and graph stores.	§16.1
NOT NULL	Constraint forbidding NULL in a column.	§9.3
NOWAIT	A locking option that raises an error at once instead of waiting for a locked row.	§13.12
NTILE	Defined in the 20.8 sheet.	§8.5
NULL	A marker for a missing or unknown value. Comparisons with it yield UNKNOWN.	§2.5
NULLIF	Returns NULL if its two arguments are equal, otherwise the first.	§3.5
NULL padding	The NULLs an outer join fills into the columns of the side that had no match.	§5.4
numeric (decimal)	An exact decimal type with a chosen precision and scale, such as `numeric(12, 2)`. The right choice for money.	§3.1

O

Term	Meaning	See
OLAP	Online analytical processing: large scans and aggregates over historical data.	§11.16
OLTP	Online transaction processing: many small reads and writes of current data.	§11.16
ON CONFLICT	PostgreSQL clause that turns an `INSERT` into an upsert or skips conflicting rows.	§9.11
ON DELETE action	What happens to child rows when a parent is deleted: CASCADE, SET NULL, SET DEFAULT, RESTRICT, NO ACTION.	§9.3
Operator precedence	The order in which operators bind: `NOT` before `AND` before `OR`, so add parentheses when you mix them.	§2.2
Optimistic concurrency control	Proceed without locks, validate at commit, and abort or retry on conflict.	§13.9
Optimizer (planner)	The DBMS component that chooses an efficient execution plan for a query.	§1.4
Oracle Database	A commercial relational database. Its procedural language is PL/SQL.	§1.7
ORDER BY	Sorts the result. The only way to guarantee row order.	§2.7
Ordered-set aggregate	An aggregate that needs sorted input, given with `WITHIN GROUP (ORDER BY …)`, such as `percentile_cont`.	§4.1
ORM	Object-relational mapper: a library that maps classes to tables and generates SQL.	§14.13
Orphan row	A child row whose foreign key points to a parent that does not exist.	§10.14
Outbox pattern	Writing an event row to an outbox table in the same transaction as the data change, then publishing it from there. Avoids dual writes.	§14.3
OUTER JOIN	LEFT, RIGHT or FULL join, which keeps unmatched rows padded with NULLs.	§5.2
Overflow page	An extra page chained to a full bucket or page.	§12.7

P

Term	Meaning	See
PACELC	Defined in the 20.14 sheet.	§15.7
Page	Fixed-size unit of storage and I/O (8 KB in PostgreSQL).	§12.1
Pagination	Returning a result one page at a time, with `LIMIT` / `OFFSET` or keyset paging.	§12.14
Parameterized query	A query whose values are sent separately from the SQL text as bind parameters, preventing injection.	§14.9
Parser	The DBMS component that checks a statement's syntax and turns it into an internal tree for the planner.	§1.4
Partial dependency	A non-prime attribute determined by a proper subset of a composite key.	§11.8
Partial index	An index that covers only rows matching a `WHERE` condition.	§12.12
Partial key (discriminator)	The attribute that identifies a weak entity among the other entities of the same owner.	§10.9
Participation	Whether every entity instance must take part in a relationship (total) or may not (partial).	§10.7
PARTITION BY (window)	Splits rows into independent groups for a window function without collapsing them.	§8.2
Partitioning	Splitting one table into smaller physical pieces by range, list or hash.	§15.2
Partition key	The column whose value decides which partition, shard or node stores a row.	§15.2
Partition pruning	Skipping partitions that cannot contain matching rows.	§15.2
Partition tolerance	In CAP: the system keeps working even when messages between nodes are lost.	§15.7
Patroni	A tool that runs PostgreSQL high availability: it watches the primary and performs automatic failover using a consensus store.	§15.10
Paxos	A classic consensus algorithm in which proposers get a majority of acceptors to agree on a single value.	§15.8
PBFT	Practical Byzantine Fault Tolerance: consensus that survives f lying nodes using 3f + 1 nodes.	§15.8
Peer rows	Rows with equal values of the window `ORDER BY` key.	§8.3
Percentile / median	The value below which a given share of the data falls. The median is the 50th percentile.	§4.1
Pessimistic concurrency control	Lock data before using it so conflicting transactions wait.	§13.9
PgBouncer	A lightweight connection pooler placed between applications and PostgreSQL.	§14.12
pg_dump / pg_restore	PostgreSQL's tools to export a logical backup and to load it back.	§9.12
pg_trgm (trigrams)	A PostgreSQL extension that splits text into three-character pieces so indexes can speed up `LIKE '%x%'` and similarity search.	§12.13
Phantom read	Defined in the 20.12 sheet.	§13.4
Physical model	Defined in the 20.10 sheet.	§10.2
PITR (point-in-time recovery)	Restoring a base backup and replaying archived WAL up to a chosen moment.	§9.12
Pivot	Turning row values into columns, usually with conditional aggregation.	§14.7

Term	Meaning	See
PL/pgSQL	PostgreSQL's procedural language for functions, procedures and triggers.	§14.1
Polyglot persistence	Using several kinds of database in one system, each for what it does best.	§16.10
Polymorphic association	One column that may point to rows of several different tables (a type + id pair). No foreign key can enforce it.	§17.10
PostGIS	A PostgreSQL extension that adds geometry and geography types, spatial functions and spatial indexes.	§16.8
PostgreSQL	The open-source object-relational database used throughout this book (version 14).	§1.7
Posting list	In an inverted index, the list of rows or documents that contain one term.	§16.11
Predicate lock	A lock on a condition rather than on existing rows, used to prevent phantoms.	§13.10
Predicate push-down	Applying filters as early as possible (inside a scan, a subquery or a remote shard) so fewer rows flow on.	§15.4
Prepared statement	A statement parsed once and executed many times with different parameters.	§14.9
Primary index	An index on the field by which a sorted file is ordered, usually with one entry per block (sparse).	§12.4
Primary key	The chosen candidate key: unique and not NULL, one per table.	§9.3
Prime attribute	An attribute appearing in some candidate key.	§10.13
Principle of least privilege	Give each user or program only the rights it needs, and nothing more.	§14.8
Privilege	A permission such as SELECT or INSERT on an object, granted to a role.	§14.8
Procedure (stored)	Named server-side routine called with `CALL`. Can control transactions.	§14.2
Prometheus	A monitoring system with a built-in time-series database for metrics.	§16.8
Pseudotransitivity	A derived FD rule: if X → Y and WY → Z, then WX → Z.	§11.3
psql	PostgreSQL's command-line client.	§1.10

Q

Term	Meaning	See
Quadtree	A spatial index that recursively divides an area into four squares.	§16.8
Query builder	A library that builds SQL from code calls, without the full object mapping of an ORM.	§14.13
Query plan	The tree of operations (scans, joins, sorts) the executor runs for a query.	§12.10
Quorum	The number of replicas that must acknowledge an operation. `R + W > N` makes reads see the latest write.	§15.5

R

Term	Meaning	See
Raft	A consensus algorithm with an elected leader that replicates a log to a majority.	§15.8
RAID	Combining several disks for safety or speed: RAID 1 mirrors, RAID 5 and 6 add parity, RAID 10 mirrors and stripes.	§15.11
RAM	The computer's main memory: much faster than disk, but its contents are lost when the power goes.	§12.1
RANGE frame	Defined in the 20.8 sheet.	§8.4
Range partitioning	Assigning rows to partitions by ranges of a key, usually dates.	§15.2
Range type	A PostgreSQL type holding an interval of values, such as `daterange` or `tstzrange`, with operators like `&&` (overlaps).	§17.1
RANK	Defined in the 20.8 sheet.	§8.5
RDBMS	Relational database management system: a DBMS based on the relational model, such as PostgreSQL.	§1.7
RDB snapshot	Redis's point-in-time dump of its whole data set to a file.	§16.11
Read Committed	Isolation level where each statement sees only data committed before it began. PostgreSQL's default.	§13.4
Read replica	A copy of the database that serves read queries.	§15.5
Read skew	Defined in the 20.12 sheet.	§13.4
Read-your-writes	A guarantee that a user always sees their own earlier writes.	§15.6
Rebalancing	Moving data between nodes when nodes are added or removed, to keep the load even.	§15.3
Recovery	Restoring a consistent state after a crash, using the log to redo committed and undo uncommitted work.	§15.9
Recursive CTE	Defined in the 20.7 sheet.	§7.3
Redis	An in-memory data-structure store (strings, hashes, lists, sets, sorted sets, streams) used as a cache, queue or database.	§16.2
Redo / undo	Redo re-applies committed changes from the log. Undo reverses changes made by transactions that did not commit.	§15.9
Redundancy	The same fact stored more than once.	§11.6
Referential integrity	Every foreign-key value matches an existing referenced key (or is NULL).	§10.14
Reflexivity (FD rule)	Armstrong's rule: if Y is a subset of X, then X → Y.	§11.3
Regular expression	A pattern language for text, such as `^[a-z]+@`. PostgreSQL matches with `~` and `~*`.	§2.4
Relation	The formal name for a table: a set of tuples over a set of attributes.	§10.12
Relational algebra	The formal operations on relations (selection, projection, join, union, difference) that SQL is built on.	§10.11
Relational division	Finding rows related to every member of another set, such as students who took all courses.	§19.7
Relational model	Data as relations of atomic values, linked by values and queried declaratively (Codd, 1970).	§1.6

Term	Meaning	See
Relation schema / relation instance	The schema is a relation's name and attributes. The instance is the set of tuples it holds at a moment.	§10.12
Repeatable Read	Isolation level where rows read once keep their values. In PostgreSQL, snapshot isolation.	§13.4
Replication	Keeping copies of the same data on several servers.	§15.5
Replication lag	The delay before a change made on the primary appears on a replica.	§15.5
Result set	The rows a query returns.	§2.1
Retention policy	A rule for how long data is kept before it is deleted or summarised.	§16.8
RETURNING	Clause that returns the rows affected by `INSERT`, `UPDATE` or `DELETE`.	§9.8
RIGHT OUTER JOIN	Defined in the 20.5 sheet.	§5.5
Rigorous 2PL	Two-phase locking that holds every lock, read and write, until the transaction ends.	§13.10
Role	A PostgreSQL user or group that owns objects and holds privileges.	§14.8
ROLLUP	Grouping extension that adds hierarchical subtotals and a grand total.	§4.6
Round-robin	Handing items to nodes in turn: 1, 2, 3, 1, 2, 3 …	§15.2
Row-level security	Policies that filter which rows a role may read or write.	§14.8
ROW_NUMBER	Defined in the 20.8 sheet.	§8.5
Row-oriented storage	Storage that keeps all the columns of a row together. Good for OLTP reads and writes of whole rows.	§12.1
ROWS frame	Defined in the 20.8 sheet.	§8.4
Row value (row constructor)	Several values written as one, `(a, b)`, compared left to right: `(ts, id) < (:ts, :id)`.	§6.6
RPO / RTO	Recovery point objective (data you may lose) and recovery time objective (downtime you may have).	§9.12
R-tree	A spatial index that groups nearby objects into nested bounding rectangles.	§16.8
Run-length encoding (RLE)	Compressing runs of a repeated value as (value, count) pairs.	§12.2
Running total	A cumulative sum from the first row up to the current row.	§8.3

S

Term	Meaning	See
Saga	A long business process split into local transactions, each with a compensating step that undoes it if a later step fails.	§15.9
Sargable	A predicate the database can answer with an index seek, such as `col >= value` with no function on `col`.	§12.13
Savepoint	A marker inside a transaction you can roll back to without abandoning the whole transaction.	§13.2
Scalar function	A function that returns one value per input row, such as `upper` or `round`.	§4.1
Scalar subquery	Defined in the 20.6 sheet.	§6.1
Scatter-gather	Sending a query to every shard and merging their partial results.	§15.4
Schema	The structure of a database (tables, columns, constraints), or a PostgreSQL namespace that groups objects.	§9.2
Schema-on-read / on-write	Structure checked when data is read (typical of NoSQL) or when it is written (relational).	§1.8
Secondary index	An index on columns other than the file's ordering key, with one entry per record (dense).	§12.4
Selectivity	The fraction of rows a predicate keeps. Low fractions favour index scans.	§12.16
SELF JOIN	Defined in the 20.5 sheet.	§5.7
Semi-join	Defined in the 20.5 sheet.	§5.8
Semi-structured data	Data that carries its own flexible structure, such as JSON or XML.	§14.4
Semi-synchronous replication	The primary waits until at least one replica has received the change before confirming the commit.	§15.5
Sequence	A database object that hands out increasing numbers.	§9.4
Sequential scan	Defined in the 20.11 sheet.	§12.10
serial / bigserial	PostgreSQL's older auto-increment shorthand: an integer column with a sequence as its default.	§9.4
Serializable	The strongest isolation level: the outcome equals some serial order of the transactions.	§13.8
Serialization failure	Error 40001: the database aborted a transaction to preserve isolation. Retry it.	§13.7
Session consistency	Guarantees such as read-your-writes and monotonic reads that hold within one client session.	§15.6
Sessionization	Grouping a user's events into sessions separated by gaps of inactivity.	§8.9
Set operation	UNION, INTERSECT or EXCEPT, which combine the results of two queries.	§6.8
Set-returning function	A function that returns many rows, such as `generate_series` or `unnest`, used in `FROM`.	§7.6
Sharding	Placing partitions of the data on different servers.	§15.3
Shard key	The column that decides which shard stores a row.	§15.3
Shared lock (S)	A lock several transactions can hold at once for reading. It blocks exclusive locks.	§13.10
Single-leader replication	One node takes all the writes and streams them to its followers.	§15.5

Term	Meaning	See
SKIP LOCKED	A locking option that skips rows other transactions have locked, so many workers can take different jobs from one queue.	§13.12
Slotted page	A page layout with an array of line pointers in the header pointing to variable-length rows stored from the end of the page.	§12.1
Snapshot	The set of committed data a transaction or statement is allowed to see.	§13.4
Snapshot isolation	Each transaction reads from a consistent snapshot taken at its start. Allows write skew.	§13.4
Soft delete	Marking a row as deleted (`deleted_at`) instead of removing it.	§17.14
Spanner	Google's globally distributed SQL database, which uses TrueTime clocks to order transactions.	§16.7
Spark	Apache Spark: distributed in-memory processing of large data sets, with SQL and DataFrame APIs.	§16.9
Sparse index	An index with entries for only some records, typically one per block.	§12.4
Split brain	Two nodes both believe they are the leader and accept writes, so the data diverges.	§15.8
Spurious tuple	A row invented by joining the pieces of a lossy decomposition.	§11.10
SQL	Structured Query Language: the declarative standard language of relational databases.	§1.9
SQLAlchemy	Python's widely used SQL toolkit and ORM.	§14.12
SQL injection	An attack that smuggles SQL into a query through unescaped input.	§14.9
SQLite	An embedded relational database stored in a single file and run inside the application.	§1.7
SQL Server	Microsoft's relational database. Its dialect and procedural language is T-SQL.	§1.7
SQLSTATE	The standard five-character error code, such as `23505` (unique violation) or `40001` (serialization failure).	§13.6
SSI	Serializable snapshot isolation: snapshot isolation plus tracking of read-write conflicts, aborting when a cycle is possible.	§13.8
SSTable	Sorted string table: an immutable file of sorted key-value pairs, written by LSM-tree stores.	§12.3
Star schema	A warehouse design: one central fact table of events joined to surrounding dimension tables (date, product, store).	§11.16
Starvation	A transaction waits forever because others keep getting the resource first.	§13.13
Statistics	Per-column summaries (distinct counts, common values, histograms) used by the planner.	§12.16
Stemming	Reducing words to a common root (running → run) so a search matches every form.	§14.6
Stop word	A very common word, such as "the" or "and", that full-text search ignores.	§14.6
Storage engine	The DBMS layer that stores pages and rows on disk.	§1.4
Streaming replication	PostgreSQL's physical replication: replicas receive WAL records continuously and replay them.	§15.5
Strict 2PL	Two-phase locking that holds write locks until commit or abort, so no one reads uncommitted data.	§13.10
Subquery	A query nested inside another statement.	§6.1

Term	Meaning	See
Superkey	Any set of attributes that uniquely identifies rows.	§10.13
Superuser	A role that bypasses all permission checks.	§14.8
Surrogate key	An artificial key with no business meaning, such as an identity number or UUID.	§10.13
Synchronous replication	The primary waits for a replica to confirm before reporting commit.	§15.5

T

Term	Meaning	See
Table	The SQL form of a relation: rows and named, typed columns.	§1.6
Tablespace	A directory on disk where PostgreSQL stores chosen tables and indexes.	§9.12
TCL	Transaction control language: `BEGIN`, `COMMIT`, `ROLLBACK`, `SAVEPOINT`.	§1.9
Temporary table	A table that exists only for the session (or transaction) that created it.	§7.1
Thomas write rule	In timestamp ordering, a write that is already outdated is skipped instead of aborting the transaction.	§13.13
Three-valued logic	SQL logic with TRUE, FALSE and UNKNOWN.	§2.5
Throughput	How much work a system completes per unit of time, such as transactions per second.	§16.2
TiDB	A MySQL-compatible distributed SQL database built on the TiKV key-value store.	§16.7
TimescaleDB	A PostgreSQL extension for time-series data that partitions tables by time automatically.	§16.8
Time-series database	Defined in the 20.15 sheet.	§16.8
Timestamp ordering	Concurrency control that orders conflicting operations by transaction timestamps.	§13.13
timestamptz vs timestamp	`timestamptz` stores an exact instant, shown in the session's time zone. `timestamp` stores a wall-clock reading with no zone.	§3.1
TOAST	PostgreSQL's mechanism for compressing or moving large values out of the main table page.	§12.1
Tombstone	A marker saying a key was deleted, so the deletion reaches every replica and survives compaction.	§16.4
Top-N per group	Returning the N best rows inside each group, usually by ranking in a CTE and filtering outside.	§8.8
Transaction	A unit of work of one or more statements that succeeds or fails as a whole.	§13.1
Transaction manager	The DBMS component that starts, commits and aborts transactions and coordinates locks and logging.	§1.4
Transitive dependency	A → B and B → C where B is not a key, so C depends on the key only through B.	§11.9
Transitivity (FD rule)	Armstrong's rule: if X → Y and Y → Z, then X → Z.	§11.3
Trigger	A function run automatically before or after changes to a table.	§14.3
Trivial FD	An FD whose right side is contained in its left side.	§11.2
TrueTime	Google's clock service that returns the current time as an interval with bounded uncertainty, used by Spanner.	§16.7
TRUNCATE	Quickly removes all rows from a table.	§9.6
T-SQL	Transact-SQL: SQL Server's dialect and procedural language.	§1.7
tsvector / tsquery	PostgreSQL full-text types: a document as a list of lexemes, and a search condition matched against it with `@@`.	§14.6
TTL (time to live)	An expiry time after which a key or row is deleted automatically.	§16.2
Tuple	The formal name for a row.	§10.12

Term	Meaning	See
Type vs instance	Keeping the catalogue description (room type, book title) separate from the physical item (room, copy).	§17.14

U

Term	Meaning	See
Undo log	A log of old values used to roll back the changes of transactions that do not commit.	§13.14
Union / decomposition (FD rules)	Derived Armstrong rules: X → Y and X → Z give X → YZ; X → YZ gives X → Y and X → Z.	§11.3
UNION / UNION ALL	Defined in the 20.6 sheet.	§6.8
UNIQUE constraint	Forbids duplicate non-NULL values in a column or column set.	§9.3
Unique index	An index that also rejects duplicate keys. It is how UNIQUE and PRIMARY KEY are enforced.	§9.3
unnest	A function that expands an array into a set of rows.	§14.5
Unpivot	Turning columns into rows.	§14.7
Updatable view	A simple view that accepts `INSERT`, `UPDATE` and `DELETE`, passing them to the underlying table.	§7.7
UPDATE	DML statement that changes column values in matching rows.	§9.9
Update anomaly	A change to a fact must be repeated in many rows, and missing one leaves the data inconsistent.	§11.6
Upsert	Insert a row, or update it if it already exists.	§9.11
UTC	Coordinated Universal Time: the zone-free reference clock that servers usually store times in.	§3.4
UTF-8	The common text encoding that stores each character in 1 to 4 bytes.	§1.10
UUID	128-bit identifier that can be generated anywhere without coordination.	§9.4
UUIDv7	A UUID version that starts with a timestamp, so new ids are roughly in order and kinder to B-tree indexes.	§9.4

V

Term	Meaning	See
VACUUM	PostgreSQL maintenance that reclaims dead row versions and updates the visibility map.	§13.14
Vector clock	A list of per-node counters attached to a value, used to tell whether two versions are ordered or concurrent.	§15.5
Version column	A counter increased on every update and checked in the `WHERE` clause, which implements optimistic locking.	§13.7
Vertical scaling	Moving to a bigger machine (scale up).	§15.1
View	Defined in the 20.7 sheet.	§7.7
Virtual node	One of many small positions a physical node owns on a hash ring, so load spreads evenly.	§15.3
Visibility map	A PostgreSQL bitmap marking pages whose rows are visible to every transaction, which makes index-only scans possible.	§12.10
Volatility (function)	How stable a function's result is: `IMMUTABLE` (never changes), `STABLE` (fixed within a statement) or `VOLATILE` (may change on every call).	§14.1

W

Term	Meaning	See
Wait-die / wound-wait	Deadlock-prevention schemes that use transaction age to decide whether a transaction waits for a lock or is aborted.	§13.11
WAL (write-ahead log)	A log of changes written and flushed before the data pages, used for durability, recovery and replication.	§13.5
Weak entity	An entity identified only together with its owner entity.	§10.9
Weighted average	An average in which each value counts in proportion to a weight, such as units sold.	§18.8
WHERE	Filters rows before grouping. Keeps only rows where the condition is TRUE.	§2.2
Wide-column store	Defined in the 20.15 sheet.	§16.4
Wildcard	A pattern character: in `LIKE`, `%` matches any run of characters and `_` exactly one.	§2.4
Window function	A function computed over a set of rows related to the current row, without collapsing them.	§8.1
WITH TIES	Extends `FETCH FIRST n` to include rows tied with the last one.	§2.8
work_mem	PostgreSQL setting for the memory one sort or hash may use before it spills to disk.	§12.15
Write amplification	One logical write causing several physical writes (log, page, index, compaction).	§12.4
Write skew	Defined in the 20.12 sheet.	§13.8

X

Term	Meaning	See
xmin / xmax	Hidden PostgreSQL row columns that hold the ids of the transactions that created and deleted that row version.	§13.14

Y

Term	Meaning	See
YugabyteDB	A distributed SQL database with a PostgreSQL-compatible interface.	§16.7

Z

Term	Meaning	See
ZooKeeper	Apache ZooKeeper: a replicated coordination service (using the ZAB protocol) for configuration, locks and leader election.	§15.8

Chapter 20 TL;DR

- **Evaluation order** explains most errors: FROM → WHERE → GROUP BY → HAVING → SELECT (windows) → DISTINCT → set operations → ORDER BY → LIMIT (20.1).
- **NULL** is unknown: `WHERE` keeps only TRUE, `NOT IN` with a NULL returns nothing, `count(col)` and other aggregates skip NULLs, and `sum` over no rows is NULL (20.2, 20.4).
- **PostgreSQL arithmetic**: `7 / 2 = 3`, so write `100.0 *` before dividing and guard with `NULLIF(x, 0)` (20.3).
- **Joins**: right-table filters of a `LEFT JOIN` go in `ON`. `NOT EXISTS` is the safe anti-join. Watch fan-out (20.5, 20.6).
- **Windows**: `row_number` / `rank` / `dense_rank` differ only on ties. The default frame with `ORDER BY` is `RANGE … CURRENT ROW`, so give `last_value` a full frame (20.8).
- **Design**: model entities, keys and relationships first, normalize to 3NF/BCNF, denormalize only with a reason and a way to keep copies right (20.10).
- **Indexes**: B-tree by default, equality columns before range columns, no functions on indexed columns, keyset instead of deep `OFFSET`, and compare estimated with actual rows in `EXPLAIN ANALYZE` (20.11).
- **Transactions**: PostgreSQL defaults to Read Committed. Repeatable Read is snapshot isolation (write skew still possible). Serializable (SSI) prevents everything but needs retries on 40001 (20.12).
- **Dialects** differ most on paging, concatenation, NULL sorting, upsert, `RETURNING`, integer division and transactional DDL (20.16).
- About 100 PostgreSQL claims and 28 SQLite claims on these sheets are checked by runnable scripts (20.17). The pattern catalogue (20.18) and the question index (20.19) link to the full solutions and answers.

This is the end of the SQL Handbook. For more on the design side of data systems, see the DDIA notes, and for how databases fit into whole systems, the case studies in Chapter 17.

Chapter 21 — Interview Question Bank

How to use this bank

This chapter collects, in one list, the questions that SQL and database interviews keep asking, worded the way they usually appear on LeetCode, in interview-experience posts and in the big public question lists. It has two parts:

- **Part A — Concept questions (Q1–Q120).** "Explain X", "X vs Y", "what does this query return" and scenario questions on keys, joins, NULL logic, normalization, indexes and `EXPLAIN`, transactions, isolation and locking, views, CTEs, window functions, procedures and triggers, SQL injection, partitioning, replication, sharding and SQL vs NoSQL.
- **Part B — Coding problems (Q121–Q250).** Every problem of LeetCode's **SQL 50** study plan, in the plan's order, followed by the other database problems that are asked most often (Nth Highest Salary, Rank Scores, Trips and Users, Human Traffic of Stadium, the medians, the Game Play Analysis series and more). Each is listed by its LeetCode title and number.

How to read an entry:

- ⋆ marks a question that appears in many sources and is asked very often. Learn those first.
- Each concept answer is short enough to say out loud in 30–60 seconds. The *More* link points to the full explanation in this book.
- Each coding entry gives the approach in 1–3 sentences and a PostgreSQL solution. When the book already solves the same problem (Chapters 18–19 restate it with their own table names and data), the entry gives the approach and a link instead of repeating the solution.
- All SQL is PostgreSQL and was run on a real server. Features newer than PostgreSQL 14 are marked (15+, 16+, …).

LeetCode's PostgreSQL judge. LeetCode creates its tables with unquoted names, so `Employee` and `managerId` are stored as `employee` and `managerid`, and you can type them in any case. When the expected output has a mixed-case column name such as `SecondHighestSalary`, the solutions quote the alias (`AS "SecondHighestSalary"`) so the case is kept. Most rounding questions need `round(x::numeric, 2)` in PostgreSQL, because `round(double precision, int)` does not exist.

How to drill. Cover the answer, say yours out loud (or write the query on paper), then check. For coding problems, first say the pattern (anti-join, top-N per group, gaps and islands, conditional aggregation…), then the query. Mark the ones you missed and repeat them two days later.

Part A — Concept questions

Fundamentals and keys

Q1. ⋆ What is the difference between a database, a DBMS and SQL?

A **database** is an organised, persistent collection of related data. A **DBMS** (database management system) is the software that stores it, answers queries, enforces rules, controls concurrent access and recovers after crashes; PostgreSQL, MySQL and Oracle are DBMSs. **SQL** (Structured Query Language) is the declarative language you use to talk to a relational DBMS: you say *what* you want, and the DBMS decides *how* to get it. → *More:* Ch 1 — What a DBMS does

Q2. What are DDL, DML, DQL, DCL and TCL? Give examples.

They are SQL's sub-languages. **DDL** (data definition) changes structure: `CREATE`, `ALTER`, `DROP`, `TRUNCATE`. **DML** (data manipulation) changes rows: `INSERT`, `UPDATE`, `DELETE`, `MERGE`. **DQL** (data query) reads: `SELECT`. **DCL** (data control) manages permissions: `GRANT`, `REVOKE`. **TCL** (transaction control) groups work: `BEGIN`, `COMMIT`, `ROLLBACK`, `SAVEPOINT`. In PostgreSQL even DDL is transactional, so a `CREATE TABLE` can be rolled back. → *More:* Ch 1 — What SQL is

Q3. ⋆ What is a relational database? Define table, row, column, relation and tuple.

A relational database stores data in **relations** (tables). Each **row** (tuple) is one fact, each **column** (attribute) has a name and a data type, and rows are identified by keys rather than by their position. Tables are connected through matching key values, not pointers, so any question can be answered by combining tables with joins. → *More:* Ch 1 — The relational model

Q4. ⋆ Primary key vs unique key vs foreign key?

A **primary key** (PK) is the identifier chosen for the table: unique, never NULL, one per table. A **unique key** (unique constraint) also forbids duplicates but is an alternative identifier, and it allows NULLs (in PostgreSQL several NULLs, unless you declare `NULLS NOT DISTINCT`, 15+). A **foreign key** (FK) is a column whose values must exist in another table's PK or unique key; it enforces **referential integrity**, which means no row points at a parent that does not exist. → *More:* Ch 9 — Keys and constraints

Q5. Superkey vs candidate key vs alternate key vs composite key?

A **superkey** is any set of columns that identifies a row uniquely. A **candidate key** is a minimal superkey: remove any column and it stops being unique. The candidate key you pick becomes the primary key, and the others are **alternate keys**. A **composite key** is any key made of two or more columns, such as `(order_id, line_no)`. → *More:* Ch 10 — Keys in relational databases

Q6. ⋆ Natural key vs surrogate key — which would you use?

A **natural key** comes from the business data (an e-mail, an ISBN). A **surrogate key** is a generated value with no meaning (`bigint GENERATED ALWAYS AS IDENTITY` or a UUID). Natural keys can change and are often long; surrogate keys never change and keep foreign keys small. The usual design is a surrogate PK plus a `UNIQUE` constraint on the natural key, so you get both stability and the business rule. → *More:* Ch 9 — Generated keys

Q7. What constraints does SQL offer?

`NOT NULL` (a value is required), `UNIQUE`, `PRIMARY KEY` (unique + not null), `FOREIGN KEY` (value must exist in the parent), `CHECK` (a boolean rule per row, such as `price >= 0`) and `DEFAULT` (value used when none is given). PostgreSQL adds `EXCLUDE` constraints, which forbid overlapping rows such as two bookings of the same room at overlapping times. Constraints are the last line of defence: application checks can be bypassed, constraints cannot. → *More:* Ch 9 — Keys and constraints

Q8. What do `ON DELETE CASCADE`, `SET NULL` and `RESTRICT` do on a foreign key?

They say what happens to child rows when the parent row is deleted (or its key updated). `CASCADE` deletes (or updates) the children too, `SET NULL` / `SET DEFAULT` clear the reference, and `NO ACTION` (the default) and `RESTRICT` refuse the delete while children exist (`RESTRICT` checks at once; `NO ACTION` checks at the end of the statement and can be deferred to commit). Use `CASCADE` only for true ownership (order → order lines), never for shared reference data. → *More:* Ch 9 — Keys and constraints

Q9. ⋆ `DELETE` vs `TRUNCATE` vs `DROP`?

`DELETE` removes the rows that match a `WHERE`, row by row, fires row triggers and can be filtered. `TRUNCATE` empties the whole table at once by replacing its files, which is much faster, does not fire `DELETE` triggers (only `TRUNCATE` triggers) and can reset identity columns. `DROP TABLE` removes the table itself (structure, indexes, privileges). In PostgreSQL all three are transactional and can be rolled back; in MySQL and Oracle `TRUNCATE` commits implicitly. → *More:* Ch 9 — DROP and TRUNCATE

Q10. `CHAR` vs `VARCHAR` vs `TEXT` in PostgreSQL?

`char(n)` pads with spaces to length n, `varchar(n)` stores up to n characters, and `text` has no limit. In PostgreSQL they are stored the same way and perform the same, so `text` (or `varchar(n)` when a length limit is a real business rule) is the usual choice and `char(n)` is almost never right. In other databases the trade-offs differ (for example index key size limits in MySQL). → *More:* Ch 9 — Data types

Q11. Why should money be stored as `numeric` and not `float`?

`float` / `double precision` are binary approximations, so `0.1 + 0.2` is not exactly `0.3` and sums drift. `numeric(p, s)` stores exact decimal digits, which is what accounting needs. A common alternative is an integer number of cents. → *More:* Ch 3 — Data types and conversions

Q12. How do you implement an "upsert" (insert, or update if it exists)?

In PostgreSQL use `INSERT … ON CONFLICT (key) DO UPDATE SET col = EXCLUDED.col`, where `EXCLUDED` is the row you tried to insert; it needs a unique index on the conflict columns and is safe under concurrency. `ON`

`CONFLICT DO NOTHING` skips duplicates. PostgreSQL 15+ also has the standard `MERGE`, which is more flexible but, unlike `ON CONFLICT`, can still fail with a unique violation under concurrent inserts.

```
INSERT INTO stock (sku, qty) VALUES ('A1', 5)
ON CONFLICT (sku) DO UPDATE SET qty = stock.qty + EXCLUDED.qty;
```

→ *More:* Ch 9 — Upsert

SELECT, NULL logic and query order

Q13. ⋆ In what order is a `SELECT` statement logically evaluated?

`FROM` / `JOIN` → `WHERE` → `GROUP BY` → aggregates → `HAVING` → window functions → `SELECT` list → `DISTINCT` → `UNION` / `INTERSECT` / `EXCEPT` → `ORDER BY` → `LIMIT` / `OFFSET`. This explains many errors: a column alias from `SELECT` cannot be used in `WHERE` (it does not exist yet), but it can be used in `ORDER BY`. The planner may execute in a different physical order as long as the result is the same. → *More:* Ch 4 — The logical order of query processing

Q14. ⋆ What is NULL, and why does `WHERE col = NULL` return nothing?

NULL means "unknown or missing", not zero or empty string. Any comparison with NULL gives **UNKNOWN**, the third truth value, and `WHERE` keeps only rows where the condition is TRUE. So test with `IS NULL` / `IS NOT NULL`, or use `IS DISTINCT FROM` for a NULL-safe "not equal". → *More:* Ch 2 — NULL and three-valued logic

Q15. ⋆ What does this return: `SELECT NULL = NULL, NULL AND FALSE, NULL OR TRUE, 1 + NULL;`?

`NULL` (unknown), `false`, `true` and `NULL`. `FALSE AND anything` is FALSE and `TRUE OR anything` is TRUE, because the unknown value cannot change the outcome; arithmetic with NULL is NULL.

Q16. ⋆ Why can `NOT IN (subquery)` return no rows at all?

`x NOT IN (1, 2, NULL)` means `x <> 1 AND x <> 2 AND x <> NULL`. The last part is UNKNOWN, so the whole condition is never TRUE and every row is dropped. Use `NOT EXISTS`, which only asks whether a matching row exists and is not affected by NULLs. → *More:* Ch 6 — Multi-row subqueries: IN and NOT IN

Q17. `COALESCE` vs `NULLIF` vs `CASE`?

`COALESCE(a, b, c)` returns the first non-NULL argument (a default for missing values). `NULLIF(a, b)` returns NULL when `a = b`, otherwise `a`; the classic use is `x / NULLIF(y, 0)` to avoid division by zero. `CASE WHEN … THEN … ELSE … END` is the general if/else, and both functions are shorthands for a `CASE`. → *More:* Ch 3 — Conditional expressions

Q18. ⋆ `WHERE` vs `HAVING`?

`WHERE` filters individual rows **before** grouping and cannot use aggregate functions. `HAVING` filters **groups** after `GROUP BY` and can use aggregates such as `HAVING count(*) > 5`. Put row conditions in `WHERE`: it removes rows early, which is faster. → *More:* Ch 4 — HAVING

Q19. ⋆ `COUNT(*)` vs `COUNT(col)` vs `COUNT(DISTINCT col)`?

`count(*)` counts rows. `count(col)` counts rows where `col` is not NULL. `count(DISTINCT col)` counts different non-NULL values. After a `LEFT JOIN`, count a column of the right table, not `*`, otherwise unmatched rows count as 1 instead of 0. → *More:* Ch 4 — Aggregate functions

Q20. Why does `SELECT dept, name, max(salary) FROM emp GROUP BY dept` fail?

After grouping, each output row stands for a whole department, and `name` has many values in it; the database does not know which one to show. Every selected column must be grouped or aggregated (PostgreSQL allows ungrouped columns only when the primary key is grouped). To get the person with the highest salary per department, use a window function or `DISTINCT ON` (see Q177 and the top-N problems). → *More:* Ch 4 — GROUP BY

Q21. How does `DISTINCT` differ from `GROUP BY`, and what is `DISTINCT ON`?

`SELECT DISTINCT a, b` and `GROUP BY a, b` without aggregates return the same rows; `GROUP BY` is needed when you also compute aggregates. PostgreSQL's `DISTINCT ON (a) … ORDER BY a, b DESC` keeps the **first row of each group** in the given order, which is the shortest way to write "latest row per customer". → *More:* Ch 2 — DISTINCT

Q22. ⋆ How do you paginate, and why is a large `OFFSET` slow?

`LIMIT 20 OFFSET 40000` still reads and throws away 40,000 rows, so deep pages get slower and rows can repeat or be skipped when data changes between requests. **Keyset pagination** remembers the last key seen and asks for the next ones, `WHERE (created_at, id) < ($1, $2) ORDER BY created_at DESC, id DESC LIMIT 20`, which an index answers at the same cost however deep the page is. → *More:* Ch 12 — OFFSET vs keyset pagination

Q23. `LIKE` vs `ILIKE` vs regular expressions? Can `LIKE '%abc'` use an index?

`LIKE` matches simple patterns (`%` any string, `_` one character) and is case-sensitive; `ILIKE` is PostgreSQL's case-insensitive version; `~` and `~*` use POSIX regular expressions. A B-tree index can serve a prefix pattern (`'abc%'`, with the `C` collation or a `text_pattern_ops` index), but a leading wildcard (`'%abc'`) cannot use it; a trigram GIN index (`pg_trgm`) can. → *More:* Ch 2 — Ranges, lists and patterns

Q24. Why does `SELECT 7 / 2` return 3 in PostgreSQL?

Both operands are integers, so it is integer division and the fraction is truncated. Cast one side, `7 / 2.0` or `7::numeric / 2`, to get 3.5. This bites ratio questions (Q134, Q232), which is why the solutions multiply by `1.0` or cast to `numeric`. → *More:* Ch 3 — Data types and conversions

Joins, subqueries and set operations

Q25. ⋆ Explain the types of joins.

INNER JOIN returns only pairs of rows that match. **LEFT JOIN** returns every left row plus matching right rows, with NULLs where none match; **RIGHT JOIN** is the mirror image. **FULL JOIN** returns all rows from both sides. **CROSS JOIN** returns every combination (a Cartesian product), and a **self join** joins a table to itself, such as employees to their managers. → *More:* Ch 5 — The four main join types at a glance

Q26. ⋆ Table A has 3 rows and B has 4 rows. How many rows can `A JOIN B` return?

An inner join returns between 0 and 12 rows (3 × 4 when every row matches every row, for example when all join keys are equal). A `LEFT JOIN` returns at least 3. A `CROSS JOIN` returns exactly 12. Duplicated join keys multiply rows, which is the "fan-out" problem behind inflated sums. → *More:* Ch 5 — Join pitfalls

Q27. ⋆ Why did my `LEFT JOIN` turn into an inner join?

A condition on the right table in `WHERE` (for example `WHERE o.status = 'paid'`) is FALSE or UNKNOWN for the NULL-padded rows, so they are filtered out. Put conditions on the right table into the `ON` clause, and keep `WHERE` for the left table. → *More:* Ch 5 — Join pitfalls

Q28. ⋆ How do you find rows in A that have no match in B (an anti-join)?

Three ways: `WHERE NOT EXISTS (SELECT 1 FROM b WHERE b.a_id = a.id)`, `LEFT JOIN b … WHERE b.a_id IS NULL`, and `NOT IN` (unsafe if `b.a_id` can be NULL). `NOT EXISTS` is the safest and PostgreSQL plans it as a hash or merge anti-join. → *More:* Ch 5 — Semi-joins and anti-joins

Q29. What is a self join? Give an example.

A table joined to itself under two aliases. Typical uses: employee and manager in the same table, comparing a day with the previous day, or finding pairs of rows (`a.id < b.id`) with something in common.

```
SELECT e.name AS employee, m.name AS manager
FROM employee e LEFT JOIN employee m ON m.id = e.manager_id;
```

→ *More:* Ch 5 — SELF JOIN

Q30. What is a correlated subquery, and how is it different from a regular one?

A regular (non-correlated) subquery can run once on its own. A **correlated** subquery refers to a column of the outer row, so it is logically evaluated once per outer row, for example "employees paid more than their department's average". PostgreSQL often rewrites these into joins, but a correlated scalar subquery in the `SELECT` list can still be slow on large tables. → *More:* Ch 6 — Correlated subqueries

Q31. ⋆ `IN` vs `EXISTS` — which is faster?

For positive tests (`IN (subquery)` vs `EXISTS`), PostgreSQL turns both into a semi-join and they usually have the same plan. The real difference is in the negative form: `NOT IN` breaks on NULLs (Q16) and is harder to optimise, so prefer `NOT EXISTS`. → *More:* Ch 6 — EXISTS and NOT EXISTS

Q32. ⋆ `UNION` vs `UNION ALL`?

`UNION` removes duplicate rows from the combined result, which costs a sort or hash. `UNION ALL` keeps every row and is cheaper. Use `UNION ALL` unless you really need de-duplication. Both need the same number of columns with compatible types. → *More:* Ch 6 — Set operations and UNION

Q33. What do `INTERSECT` and `EXCEPT` do?

`INTERSECT` returns rows present in both queries; `EXCEPT` (Oracle: `MINUS`) returns rows of the first query that are not in the second. Both remove duplicates unless you write `ALL`. Unlike `=`, they treat two NULLs as the same. → *More:* Ch 6 — EXCEPT

Q34. What is a `LATERAL` join?

A subquery in `FROM` that may refer to columns of tables listed before it, so it runs once per left row. It is the natural tool for "top 3 orders per customer" with a `LIMIT` inside, and for calling set-returning functions per row (`CROSS JOIN LATERAL unnest(…)`). → *More:* Ch 6 — LATERAL subqueries

Q35. Equi-join vs non-equi join vs natural join?

An **equi-join** matches on equality (`a.id = b.a_id`). A **non-equi join** uses another comparison, such as a sale date `BETWEEN` a price period's start and end. A **natural join** joins automatically on all same-named columns, which silently changes when someone adds a column, so avoid it and write `ON` or `USING`. → *More:* Ch 5 — USING, NATURAL JOIN and multi-table joins

Aggregation, CTEs, views and window functions

Q36. ⋆ What is a CTE, and when would you use one instead of a subquery?

A **CTE** (common table expression) is a named subquery written with `WITH name AS (…)` before the main query. It makes multi-step logic readable, lets you reference the same result more than once, and is the only way to write recursion. Since PostgreSQL 12 a CTE used once is inlined like a subquery, so it costs nothing extra; write `AS MATERIALIZED` to force it to be computed once. → *More:* Ch 7 — Common table expressions

Q37. ⋆ How does a recursive CTE work? Write one that walks an org chart.

The **anchor** query produces the starting rows; the **recursive** part joins the previous round's rows back to the table, and rounds repeat until one adds no rows. To stop cycles, use PostgreSQL 14's `CYCLE` clause (or track the visited path); `UNION` instead of `UNION ALL` helps only when a repeated row is identical, so it does not stop a cycle once you carry a changing column such as `depth`.

```
WITH RECURSIVE chain AS (
  SELECT id, name, manager_id, 1 AS depth FROM employee WHERE manager_id IS NULL
  UNION ALL
  SELECT e.id, e.name, e.manager_id, c.depth + 1
  FROM employee e JOIN chain c ON e.manager_id = c.id
)
SELECT * FROM chain ORDER BY depth, id;
```

→ *More:* Ch 7 — Hierarchies

Q38. ⋆ What is a view? What is a materialized view, and when do you use each?

A **view** is a stored query with a name; it holds no data and runs its query each time, so it is always current. A **materialized view** stores the query's result like a table, so reads are fast but the data is only as fresh as the last `REFRESH MATERIALIZED VIEW` (add `CONCURRENTLY`, which needs a unique index, to avoid blocking readers). Use views for reuse and access control, materialized views for expensive reports that may be slightly stale. → *More:* Ch 7 — Views · Materialized views

Q39. Can you `INSERT` or `UPDATE` through a view?

Yes, if the view is simple: one table, no aggregates, `DISTINCT`, `GROUP BY` or set operations. PostgreSQL makes such views updatable automatically. `WITH CHECK OPTION` refuses changes that would make the row disappear from the view, and `INSTEAD OF` triggers make complex views writable. → *More:* Ch 7 — Views

Q40. ⋆ Window function vs `GROUP BY`?

`GROUP BY` collapses each group into one output row. A **window function** (`… OVER (PARTITION BY … ORDER BY …)`) computes a value over a set of related rows but keeps every row, so you can show each employee next to the department average, a rank or a running total. → *More:* Ch 8 — What a window function is

Q41. ★ `ROW_NUMBER` vs `RANK` vs `DENSE_RANK`?

For scores 100, 90, 90, 80: `row_number()` gives 1, 2, 3, 4 (ties broken arbitrarily), `rank()` gives 1, 2, 2, 4 (gaps after ties) and `dense_rank()` gives 1, 2, 2, 3 (no gaps). Use `dense_rank` for "Nth highest distinct value", `rank` for competition placing, and `row_number` for de-duplication or exactly one row per group. → *More:* Ch 8 — Ranking functions

Q42. ★ What do `LAG` and `LEAD` do?

`lag(col, n, default)` returns the value from n rows **before** the current row in the window's order, and `lead` from n rows **after**. They replace self joins for "compare with the previous day", "time since last login" and "month-over-month growth". → *More:* Ch 8 — Value (offset) functions

Q43. What is a window frame? Why does my running total repeat on tied dates, and why does `LAST_VALUE` return the current row?

The **frame** is the subset of the partition that an aggregate window function sees. With `ORDER BY` and no frame clause, the default is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, which includes all rows tied with the current one (so ties get the same running total) and stops at the current row (so `last_value` sees only up to here). Write `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` or `… AND UNBOUNDED FOLLOWING` when you mean that. → *More:* Ch 8 — Frames

Q44. Why can't I put a window function in `WHERE`?

Window functions are computed after `WHERE`, `GROUP BY` and `HAVING` (Q13), so they do not exist yet when rows are filtered. Compute them in a subquery or CTE, then filter outside: `SELECT * FROM (SELECT …, rank() OVER (…) AS r FROM t) x WHERE r <= 3`. → *More:* Ch 8 — Top-N per group

Q45. ★ What is the "gaps and islands" problem, and the trick that solves it?

Islands are runs of consecutive values (days, ids) and **gaps** are the holes between them. Within a run, `value − row_number()` (ordered by value) is constant, so grouping by that difference puts each run in its own group; then `min`, `max` and `count` describe each island. Used for login streaks, consecutive numbers and contiguous date ranges (Q153, Q199, Q200). → *More:* Ch 8 — Gaps and islands

Q46. ★ How do you remove duplicate rows from a table, keeping one?

Number the copies with `row_number() OVER (PARTITION BY the_duplicate_columns ORDER BY id)` and delete those numbered above 1, or use a self join: `DELETE FROM t a USING t b WHERE a.email = b.email AND a.id > b.id`. Then add a unique constraint so it cannot happen again. → *More:* Ch 8 — Top-N per group and de-duplication

Q47. What are `GROUPING SETS`, `ROLLUP` and `CUBE`?

They compute several groupings in one query. `ROLLUP (year, month)` gives per-month rows, per-year subtotals and a grand total; `CUBE (a, b)` gives every combination of subtotals; `GROUPING SETS` lists exactly the groupings you want. `grouping(col)` tells you which rows are subtotals. → *More:* Ch 4 — GROUPING SETS, ROLLUP and CUBE

Q48. ★ How do you pivot rows into columns in PostgreSQL?

Use conditional aggregation: `sum(amount) FILTER (WHERE month = 'Jan') AS jan` (or `sum(CASE WHEN … THEN amount END)` in other databases), one expression per output column. The `crosstab` function from the `tablefunc` extension also works. A dynamic set of columns needs dynamic SQL, because a query's columns must be known before it runs. → *More:* Ch 14 — Pivot and unpivot

Normalization and data modeling

Q49. ★ What is normalization, and why do we normalize?

Normalization organises columns into tables so that each fact is stored in exactly one place. It removes redundancy and the **anomalies** that come with it: an *update anomaly* (a change made in one copy but not another), an *insertion anomaly* (you cannot record a fact without an unrelated one) and a *deletion anomaly* (deleting one fact loses another). → *More:* Ch 11 — Anomalies

Q50. ★ Explain 1NF, 2NF, 3NF and BCNF with an example.

1NF: every column holds one atomic value (no lists like `'red,blue'`, no repeating groups). **2NF**: 1NF, and no non-key column depends on only part of a composite key (in `order_line(order_id, product_id, product_name)`, `product_name` depends only on `product_id`, so it moves to `product`). **3NF**: 2NF, and no non-

key column depends on another non-key column (`zip → city`). **BCNF**: for every functional dependency X → Y, X is a superkey. → *More:* Ch 11 — 1NF · 2NF · 3NF · BCNF

Q51. What is a functional dependency?

`X → Y` means that rows with the same X always have the same Y; for example `employee_id → department_id`. Keys and normal forms are all defined in terms of functional dependencies. → *More:* Ch 11 — Functional dependencies

Q52. What are 4NF and 5NF in one sentence each?

4NF removes independent multi-valued facts stored in the same table (a teacher's subjects and languages in one table make every combination appear). **5NF** splits a table that can be rebuilt without loss only from three or more smaller tables (a join dependency). They are rarely needed in practice but come up in theory rounds. → *More:* Ch 11 — 4NF · 5NF

Q53. ⋆ What is denormalization, and when would you do it?

Deliberately storing redundant data (a cached `order_total`, a copied customer name, a summary table) to make reads cheaper. Do it only for a measured read bottleneck, and decide how the copies stay correct: the same transaction, a trigger, or a materialized view. Analytics schemas (star schemas) are denormalized by design. → *More:* Ch 11 — Denormalization

Q54. ⋆ How do you model a many-to-many relationship?

With a **junction** (association) table holding two foreign keys, for example `enrolment(student_id, course_id)`, with the pair as the primary key (or a surrogate PK plus `UNIQUE` on the pair). Attributes of the relationship itself, such as `enrolled_on` or `grade`, live in the junction table. → *More:* Ch 10 — Relationships

Q55. What is an ER diagram? What are cardinality and participation?

An **ER (entity–relationship) diagram** draws the entities (things you store), their attributes and the relationships between them. **Cardinality** says how many instances can be related (1:1, 1:N, M:N); **participation** says whether every instance must take part (total) or not (partial), which becomes `NOT NULL` on the foreign key. → *More:* Ch 10 — The ER model and its notations

Q56. Star schema vs snowflake schema? OLTP vs OLAP?

OLTP (online transaction processing) systems run many small reads and writes on normalized tables. **OLAP** (online analytical processing) systems run big scans and aggregations, usually on a **star schema**: a central **fact** table (one row per sale) with foreign keys to denormalized **dimension** tables (date, product, store). A **snowflake schema** normalizes the dimensions into sub-tables, which saves space but costs more joins. → *More:* Ch 11 — Denormalization · Ch 16 — Big data and distributed processing

Q57. How would you store a tree (categories, comments, org chart) in SQL?

The simplest is an **adjacency list**: a `parent_id` column, queried with a recursive CTE (Q37). Alternatives trade write cost for read speed: a **materialized path** (`'1/4/9'`, or PostgreSQL's `ltree`), **nested sets** (left/right numbers, fast subtree reads, slow inserts) and a **closure table** (one row per ancestor–descendant pair). → *More:* Ch 7 — Hierarchies

Q58. Design the schema for a simple e-commerce site (or a library, or Instagram).

Start from the questions the system must answer, list entities (customer, product, order, order_line, payment), pick keys, draw relationships (an order has many lines; a line references one product), normalize to 3NF, then add constraints and indexes for the main queries. Keep the price paid on `order_line` (a historical fact), not only on `product`. Be ready to explain one denormalization and one index choice. → *More:* Ch 17 — Schema design case studies

Indexes, EXPLAIN and query optimization

Q59. ⋆ What is an index, and how does it speed up queries?

An index is a separate, sorted data structure that maps column values to row locations, like a book's index. Instead of reading every row (a sequential scan), the database walks the index to the few matching rows. The price is extra storage and slower `INSERT` / `UPDATE` / `DELETE`, because every index must be updated too. → *More:* Ch 12 — What an index is

Q60. ⋆ How does a B-tree (B+ tree) index work?

It is a balanced tree whose pages hold many sorted keys, so even millions of rows need only 3–4 page reads from root to leaf. Leaf pages are linked in order, which makes range queries (`BETWEEN`, `>`), `ORDER BY` and prefix `LIKE` efficient. It is PostgreSQL's default index type. → *More:* Ch 12 — B-trees and B+ trees

Q61. ⋆ Clustered vs non-clustered index?

A **clustered** index stores the table's rows themselves in index order, so there can be only one (InnoDB's primary key, SQL Server's clustered index). A **non-clustered** (secondary) index is separate and points to the rows. PostgreSQL tables are heaps, so every index is non-clustered; the `CLUSTER` command reorders a table once but the order is not maintained afterwards. → *More:* Ch 9 — Indexes (in brief)

Q62. What index types does PostgreSQL have, and when do you use each?

B-tree: equality, ranges and sorting (the default). **Hash**: equality only. **GIN**: values that contain many items, such as JSONB, arrays and full-text search. **GiST** and **SP-GiST**: geometric data, ranges, nearest-neighbour searches. **BRIN**: very large tables whose values follow the physical order, such as an append-only log by time; tiny but coarse. → *More:* Ch 12 — GIN, GiST, BRIN

Q63. ⋆ What is a composite index, and does column order matter?

An index on several columns, sorted by the first column, then the second within it, and so on. Order matters: an index on `(a, b)` helps `WHERE a = ?` and `WHERE a = ? AND b > ?`, but not `WHERE b = ?` alone (the **leftmost-prefix** rule). Put equality columns first, then the range or sort column. PostgreSQL 18 can sometimes "skip scan" a missing leading column, but do not rely on it. → *More:* Ch 12 — Composite indexes and column order

Q64. What are covering, partial and expression indexes?

A **covering** index holds every column a query needs (extra columns can be added with `INCLUDE`), so PostgreSQL can answer with an **index-only scan** without visiting the table. A **partial** index indexes only rows matching a `WHERE` (`WHERE status = 'pending'`), so it stays small. An **expression** index indexes a computed value such as `lower(email)`, so queries filtering on that expression can use it. → *More:* Ch 12 — Covering, partial and expression indexes

Q65. ⋆ Why is my query not using the index?

Common causes: a function or cast on the column (`WHERE date(created_at) = …`, `WHERE lower(email) = …` without an expression index), a leading wildcard `LIKE '%x'`, a type mismatch, a filter on a non-leading column of a composite index, `OR` across different columns, or a filter that matches so many rows that a sequential scan is cheaper. Also check that statistics are current (`ANALYZE`). → *More:* Ch 12 — Anti-patterns that disable indexes

Q66. Should you index every column? What is selectivity?

No. Each index slows writes, takes space and memory, and may never be used. **Selectivity** is the fraction of rows a condition keeps; an index helps when it keeps few rows (an e-mail lookup) and not when it keeps many (`is_active = true` on 95% of rows). Index the columns used in frequent `WHERE`, `JOIN` and `ORDER BY` clauses, and foreign keys that are joined or whose parents are deleted. → *More:* Ch 12 — What an index is

Q67. ⋆ How do you read `EXPLAIN` / `EXPLAIN ANALYZE`?

`EXPLAIN` shows the plan the optimizer chose as a tree of nodes with estimated costs and row counts; `EXPLAIN (ANALYZE, BUFFERS)` also runs the query and adds real times, real row counts, loops and pages read. Read from the innermost node outwards, look for the nodes with the largest actual time, and compare estimated with actual rows: a big difference means bad statistics and usually a bad join choice. → *More:* Ch 12 — Reading EXPLAIN

Q68. Seq scan vs index scan vs index-only scan vs bitmap scan?

A **sequential scan** reads the whole table. An **index scan** walks the index and fetches each matching row from the table. An **index-only scan** answers from the index alone (needs a covering index and an up-to-date visibility map). A **bitmap scan** first collects matching row locations from one or more indexes, sorts them by page, then reads each page once; it suits medium selectivity and `OR` / `AND` of indexes. → *More:* Ch 12 — Reading EXPLAIN

Q69. Nested loop vs hash join vs merge join?

A **nested loop** looks up each outer row in the inner input, which is best when the outer side is small and the inner side is indexed. A **hash join** builds a hash table on the smaller input and probes it with the other, which is best for large equality joins. A **merge join** walks two inputs already sorted on the key, which suits large inputs that are already sorted (or cheap to sort); like a hash join, it handles only equality conditions. → *More:* Ch 12 — Join algorithms

Q70. ⋆ A query that used to be fast is now slow. How do you troubleshoot it?

1. Get the real plan with `EXPLAIN (ANALYZE, BUFFERS)` and compare with the old one. 2) Check estimates vs actual rows; run `ANALYZE` if statistics are stale. 3) Look for sequential scans on big tables, sorts or hashes spilling to disk (raise `work_mem` or add an index), and nested loops with huge loop counts. 4) Check for lock waits (`pg_stat_activity`, `pg_locks`), table bloat and data growth. 5) Fix the query or index, then measure again; `pg_stat_statements` finds the worst queries overall. → *More:* Ch 12 — A tuning workflow

Q71. ⋆ What is the N+1 query problem?

Code runs one query to load a list, then one more query per item (for each of 100 orders, fetch its customer), which makes 101 round trips. Fix it with a join, one batched query (`WHERE id = ANY($1)`), or the ORM's eager loading. → *More:* Ch 14 — ORMs vs raw SQL

Q72. How does the optimizer estimate rows? What do `ANALYZE` and `VACUUM` do?

The planner uses per-column statistics (row counts, NULL fraction, distinct values, most common values, histograms) gathered by `ANALYZE`. `VACUUM` removes dead row versions left by MVCC (Q85) so space can be reused, and updates the visibility map that index-only scans need. **Autovacuum** runs both automatically; tables with very high churn may need tuned thresholds. → *More:* Ch 12 — Statistics

Q73. Give five ways to make a slow SQL query faster.

Add the right index (composite, covering or partial); remove functions from indexed columns in `WHERE`; select only needed columns instead of `SELECT *`; replace `OFFSET` paging with keyset paging; pre-aggregate with a materialized view or summary table; batch N+1 queries into one; and for huge tables, partition so queries touch fewer rows. Always confirm with `EXPLAIN ANALYZE`.

Transactions, isolation and locking

Q74. ⋆ What is a transaction? Explain ACID.

A **transaction** is a group of statements that succeed or fail as one unit. **Atomicity**: all of its changes happen or none do. **Consistency**: it moves the database from one valid state to another, respecting constraints. **Isolation**: concurrent transactions do not see each other's half-done work (to the degree the isolation level promises). **Durability**: once committed, changes survive a crash, because they were first written to the write-ahead log. → *More:* Ch 13 — What a transaction is

Q75. ⋆ What are dirty reads, non-repeatable reads and phantom reads?

A **dirty read** sees another transaction's uncommitted change. A **non-repeatable read** reads the same row twice and gets different values because another transaction committed an update in between. A **phantom read** runs the same `WHERE` query twice and gets a different *set* of rows because another transaction inserted or deleted matching rows. A **lost update** and **write skew** are two more anomalies (Q78, Q79). → *More:* Ch 13 — Isolation

Q76. ⋆ Name the four isolation levels and what each prevents.

Level	Dirty read	Non-repeatable read	Phantom
Read Uncommitted	possible (not in PostgreSQL)	possible	possible
Read Committed	prevented	possible	possible
Repeatable Read	prevented	prevented	possible in the standard, prevented in PostgreSQL
Serializable	prevented	prevented	prevented

Serializable also prevents write skew: the result equals some one-at-a-time order of the transactions. → *More:* Ch 13 — Isolation

Q77. ⋆ What is PostgreSQL's default isolation level, and how does PostgreSQL implement the levels?

Read Committed is the default (MySQL InnoDB defaults to Repeatable Read). Each statement sees a snapshot taken when the statement starts. Repeatable Read takes one snapshot for the whole transaction (snapshot isolation), so phantoms cannot appear and concurrent updates of the same row fail with a serialization error. Serializable adds **SSI** (serializable snapshot isolation), which tracks read/write dependencies and aborts one transaction with SQLSTATE `40001` when an anomaly is possible; the application must retry. Read Uncommitted behaves like Read Committed. → *More:* Ch 13 — Watching isolation anomalies

Q78. ⋆ What is a lost update, and how do you prevent it?

Two transactions read the same value, both compute a new value from it and both write, so the first write is lost (two clerks both read stock 10 and both write 9). Prevent it with an atomic update (`UPDATE … SET stock = stock - 1 WHERE stock > 0`), a row lock (`SELECT … FOR UPDATE`), optimistic locking with a version column (`UPDATE … WHERE id = ? AND version = ?`, then check the row count), or Repeatable Read/Serializable with retry. → *More:* Ch 13 — Lost updates

Q79. What is write skew? Give an example.

Two transactions read the same data, each checks a rule, then each updates a *different* row, and together they break the rule. Example: the rule is "at least one doctor on call"; two doctors on call both see two on call and both go off call. Row locks on the rows you write do not stop it; use Serializable isolation, or lock the rows you read with `SELECT … FOR UPDATE`. → *More:* Ch 13 — Write skew and Serializable isolation

Q80. ⋆ Optimistic vs pessimistic locking?

Pessimistic locking takes locks before working (`SELECT … FOR UPDATE`), so others wait; it is safe under high contention but reduces concurrency and can deadlock. **Optimistic** locking works without locks and checks at write time whether someone else changed the row (a `version` column or timestamp), retrying on conflict; it is best when conflicts are rare, such as web forms. → *More:* Ch 13 — Lost updates

Q81. Shared vs exclusive locks? Row-level vs table-level locks?

A **shared** lock lets others read but not modify; an **exclusive** lock blocks other writers (in PostgreSQL readers are never blocked by row locks, thanks to MVCC). **Row-level** locks (`FOR UPDATE`, `FOR SHARE`, and those taken by `UPDATE` / `DELETE`) protect single rows; **table-level** locks (eight modes, from `ACCESS SHARE` taken by `SELECT` up to `ACCESS EXCLUSIVE` taken by most `ALTER TABLE` and `DROP`) protect the table's structure. → *More:* Ch 13 — Lock-based concurrency control

Q82. ⋆ What is a deadlock, and how do you avoid one?

Two transactions each hold a lock the other needs, so both wait forever. PostgreSQL checks for cycles after `deadlock_timeout` (1 s by default) and aborts one transaction with error `40P01`. Avoid them by locking rows in a consistent order (for example by primary key), keeping transactions short, and retrying the aborted one. → *More:* Ch 13 — Deadlocks

Q83. What is two-phase locking (2PL)?

A locking protocol in which a transaction first only acquires locks (growing phase) and then only releases them (shrinking phase); strict 2PL holds write locks until commit. It guarantees serializable schedules, at the cost of blocking and deadlocks. MySQL's and SQL Server's Serializable levels use locking like this, while PostgreSQL uses SSI. → *More:* Ch 13 — Lock-based concurrency control

Q84. ⋆ How do you build a job queue in SQL so workers don't take the same job?

Each worker runs, inside a transaction, `SELECT id FROM jobs WHERE status = 'ready' ORDER BY id LIMIT 1 FOR UPDATE SKIP LOCKED`, processes the job, marks it done and commits. `SKIP LOCKED` makes each worker skip rows already locked by others instead of waiting, so workers never block each other. `NOWAIT` instead raises an error at once. → *More:* Ch 13 — Row locks for work queues

Q85. ⋆ What is MVCC?

Multi-version concurrency control: an `UPDATE` writes a new version of the row instead of overwriting it, and each transaction reads the versions visible in its snapshot. So readers never block writers and

writers never block readers. The cost is dead versions that `VACUUM` must clean up, and table bloat if it falls behind (for example because of a long-running transaction). → *More:* Ch 13 — MVCC

Q86. How is durability guaranteed? What is the WAL?

Before a change is applied to the data files, it is written to the **write-ahead log** (WAL) and the WAL is flushed to disk at commit. After a crash, the database replays the WAL from the last checkpoint to redo committed work. The same WAL stream is what replicas receive (Q101) and what point-in-time recovery replays. → *More:* Ch 13 — Durability

Q87. What are savepoints?

Named points inside a transaction: `SAVEPOINT s1; … ROLLBACK TO SAVEPOINT s1;` undoes only the work after `s1` and keeps the transaction open. In PostgreSQL an error aborts the whole transaction unless you roll back to a savepoint, which is how drivers implement "continue after an error". → *More:* Ch 13 — Atomicity

Q88. A bank transfer: write it so money is never lost or created.

Do both updates in one transaction, update rows in a fixed order (lower account id first) to avoid deadlocks, and let a `CHECK (balance >= 0)` constraint reject overdrafts.

```
BEGIN;
UPDATE account SET balance = balance - 100 WHERE id = 1;
UPDATE account SET balance = balance + 100 WHERE id = 2;
COMMIT;   -- any error before this: ROLLBACK, nothing changed
```

→ *More:* Ch 13 — Putting it together

Stored procedures, triggers, security and SQL injection

Q89. ⋆ Function vs stored procedure in PostgreSQL?

A **function** (`CREATE FUNCTION`) returns a value or a set of rows and is called inside a query (`SELECT f(x)`); it runs inside the caller's transaction and cannot commit. A **procedure** (`CREATE PROCEDURE`, PostgreSQL 11+) is invoked with `CALL`, returns no value (only `OUT` parameters, 14+) and can `COMMIT` or `ROLLBACK` inside its body, which suits batch jobs. Both can be written in SQL or PL/pgSQL. → *More:* Ch 14 — Stored procedures

Q90. What are the pros and cons of stored procedures?

Pros: fewer network round trips, logic close to the data, one place for rules shared by several applications, and permissions granted on the procedure rather than the tables. Cons: harder to version, test, debug and deploy than application code, tied to one database's language, and they can hide business logic where developers do not look. → *More:* Ch 14 — Stored procedures

Q91. ⋆ What is a trigger? `BEFORE` vs `AFTER` vs `INSTEAD OF`, row vs statement level?

A trigger runs a function automatically when a table is modified. `BEFORE` triggers can change or cancel the row (fill `updated_at`, normalise data); `AFTER` triggers see the final row and suit audit logs; `INSTEAD OF` triggers replace the operation on a view. A **row-level** trigger runs once per affected row; a **statement-level** trigger runs once per statement, however many rows. → *More:* Ch 14 — Triggers

Q92. When should you avoid triggers?

When the logic is business logic that developers need to see and test; triggers are invisible from the application, can cascade into other triggers, slow down bulk writes and make debugging hard. They are a good fit for audit trails, `updated_at` columns and enforcing invariants that constraints cannot express. → *More:* Ch 14 — Triggers

Q93. ⋆ What is SQL injection, and how do you prevent it?

An attacker puts SQL syntax into input that the application concatenates into a query, for example a name of `' OR '1'='1` that turns `WHERE name = '…'` into a condition that is always true, or one that adds `; DROP TABLE …`. Prevent it with **parameterized queries** (prepared statements with placeholders), where the value is sent separately and never parsed as SQL. Also allow-list identifiers such as sort columns (they cannot be parameters), give the application role only the privileges it needs, and never build dynamic SQL from raw input (in PL/pgSQL use `format('%I', …)` and `USING`). → *More:* Ch 14 — SQL injection and parameterized queries

Q94. Does an ORM protect you from SQL injection?

Mostly, because ORMs generate parameterized queries. You are exposed again as soon as you use raw SQL or string-built fragments (`order_by(user_input)`, string-formatted native queries). Escaping by hand is not a substitute for parameters. → *More:* Ch 14 — ORMs vs raw SQL

Q95. What are `GRANT`, roles and row-level security?

A **role** is a user or group; `GRANT SELECT ON orders TO analyst` gives it a privilege and `REVOKE` takes it away. Grant the least privilege needed. **Row-level security** (RLS) adds policies that filter which rows a role can see or change, for example `USING (tenant_id = current_setting('app.tenant')::int)` for multi-tenant apps. → *More:* Ch 14 — Security

Q96. What is connection pooling, and why does PostgreSQL need it?

Each PostgreSQL connection is a separate server process using memory, so thousands of short-lived connections waste resources. A **pool** (in the application, or PgBouncer in front of the database) keeps a small set of open connections and lends them out per request or per transaction. → *More:* Ch 14 — Connection pooling

Q97. JSON vs JSONB in PostgreSQL? When would you store JSON in a relational database?

`json` stores the text as given; `jsonb` stores a parsed binary form that drops whitespace and duplicate keys, is faster to query and can be indexed with GIN. Use JSONB for truly variable attributes (product specs, event payloads), but keep fields you filter, join or constrain on as real columns. → *More:* Ch 14 — JSON and JSONB

Partitioning, replication, sharding and SQL vs NoSQL

Q98. ⋆ What is partitioning? Range vs list vs hash partitioning?

Partitioning splits one large table into smaller physical tables (partitions) that queries still see as one. **Range** partitioning splits by intervals (one partition per month), **list** by explicit values (per country), and **hash** by a hash of the key (even spread). Benefits: **partition pruning** (queries touch only relevant partitions), and dropping old data with `DROP` / `DETACH PARTITION` instead of a huge `DELETE`. PostgreSQL has declarative partitioning since version 10. → *More:* Ch 15 — Partitioning

Q99. ⋆ Partitioning vs sharding vs replication?

Partitioning splits a table into pieces on the same server. **Sharding** puts those pieces on different servers, so writes and storage scale out. **Replication** keeps full copies of the same data on several servers for read scaling and high availability. They are combined in practice: each shard is usually replicated. → *More:* Ch 15 — Sharding

Q100. ⋆ How do you choose a shard key? What problems does sharding bring?

Pick a key that spreads load evenly and keeps most queries on one shard, often the tenant or user id. Avoid monotonically increasing keys with range sharding (all new writes hit one shard) and low-cardinality keys (hot spots). Costs: cross-shard queries and joins, distributed transactions, resharding when shards fill up (consistent hashing helps), and global uniqueness. Citus is the usual way to shard PostgreSQL. → *More:* Ch 15 — Sharding

Q101. ⋆ Synchronous vs asynchronous replication? What is replication lag?

Synchronous replication makes the primary wait until a replica confirms before reporting commit, so no committed data is lost on failover but every write is slower. **Asynchronous** replication commits first and ships changes afterwards: faster, but recent commits can be lost if the primary dies. **Replication lag** is how far a replica is behind; reading from a lagging replica can show stale data, so read-your-own-writes queries should go to the primary. → *More:* Ch 15 — Replication

Q102. Physical (streaming) vs logical replication in PostgreSQL?

Physical streaming replication ships WAL bytes, making an exact copy of the whole cluster; the replica is read-only and must run the same major version. **Logical replication** (publications and subscriptions, PostgreSQL 10+) sends row changes for chosen tables, works across major versions and lets the subscriber have its own writes and indexes; it suits upgrades and feeding other systems. → *More:* Ch 15 — Replication

Q103. Single-leader vs multi-leader vs leaderless replication?

Single-leader: one primary takes all writes and followers copy it (PostgreSQL, MySQL); simple, no write conflicts. **Multi-leader**: several nodes accept writes (multi-region setups) and must resolve conflicts. **Leaderless**: clients write to several replicas and read with quorums (`W + R > N`), as in Cassandra and DynamoDB. → *More:* Ch 15 — Replication

Q104. ⋆ Explain the CAP theorem (and PACELC).

When a network **partition** separates nodes, a distributed store must choose between **consistency** (every read sees the latest write, so some requests are refused) and **availability** (every request gets an answer, possibly stale). Without a partition there is no forced choice. **PACELC** adds: Else (no partition) the trade-off is **latency** vs **consistency**. → *More:* Ch 15 — The CAP theorem

Q105. Strong vs eventual consistency?

Strong (linearizable) consistency: once a write is acknowledged, every later read sees it. **Eventual** consistency: replicas converge if writes stop, but a read may return old data meanwhile. In between are read-your-writes, monotonic reads and causal consistency. → *More:* Ch 15 — Consistency models

Q106. ⋆ SQL vs NoSQL — how do you choose?

Choose relational (SQL) for structured data with relationships, ad-hoc queries, joins, constraints and multi-row transactions (orders, payments, inventory). Choose NoSQL for a specific access pattern at very large scale or with a flexible shape: key-value for caches and sessions, document stores for self-contained aggregates, wide-column for huge write-heavy time-ordered data, graph for many-hop relationships. Many systems use both; PostgreSQL with JSONB covers many "we need a document store" cases. → *More:* Ch 1 — Relational vs non-relational · Ch 16 — Choosing a database

Q107. ACID vs BASE?

ACID (Q74) is the transactional guarantee of relational databases. **BASE** (Basically Available, Soft state, Eventually consistent) describes many distributed NoSQL stores that stay available and accept temporary inconsistency. The line is blurred today: MongoDB has multi-document transactions, and NewSQL databases (CockroachDB, Spanner, YugabyteDB) give ACID across nodes. → *More:* Ch 1 — Relational vs non-relational

Q108. What is two-phase commit (2PC)?

A protocol that commits one transaction across several databases: the coordinator asks every participant to **prepare** (make the change durable and promise to commit), then sends **commit** only if all said yes, otherwise **abort**. It is blocking: if the coordinator fails after prepare, participants hold locks until it returns. Microservices usually prefer sagas with compensating actions. → *More:* Ch 15 — Consensus and leader election

Q109. How do you achieve high availability for PostgreSQL?

A primary with one or more streaming replicas, an automatic failover manager (Patroni, or a managed service), a connection endpoint that follows the primary, synchronous replication if no data loss is acceptable, and tested backups with WAL archiving for point-in-time recovery. A replica is not a backup: it copies mistakes such as a `DROP TABLE` within seconds. → *More:* Ch 15 — High availability

Q110. What are read replicas, and what problems do they bring?

Copies that serve read-only queries so the primary handles fewer reads. Problems: stale reads because of lag, routing logic in the application, and queries on a replica can be cancelled when they conflict with changes being replayed. They do not help with write load; sharding does. → *More:* Ch 15 — Replication

Scenario and senior-level questions

Q111. ⋆ How would you scale a database that is getting slow as traffic grows?

In order: measure (top queries from `pg_stat_statements`); fix queries and indexes; add caching (Redis) for hot reads; add read replicas; partition the biggest tables; move analytics to a separate store; scale the server vertically; and only then shard. Each step adds complexity, so stop when the bottleneck is gone. → *More:* Ch 15 — Distributed databases: the big picture

Q112. How do you add a column or index to a huge table without downtime?

`ALTER TABLE … ADD COLUMN` with no default, or with a constant default (PostgreSQL 11+), is instant. Create indexes with `CREATE INDEX CONCURRENTLY` (no write lock, cannot run inside a transaction). Add

constraints as `NOT VALID` first, then `VALIDATE CONSTRAINT` separately. Set a short `lock_timeout` so a blocked `ALTER` does not queue every other query behind it, and backfill data in small batches. → *More:* Ch 9 — ALTER

Q113. How do you delete 100 million old rows safely?

If the table is partitioned by time, drop or detach old partitions, which is instant. Otherwise delete in batches (for example 10,000 rows per transaction by key range), pausing between batches so replicas and autovacuum keep up, then let `VACUUM` reclaim the space. One huge `DELETE` holds locks for a long time, bloats the table and floods the WAL. → *More:* Ch 15 — Partitioning

Q114. UUID vs auto-increment primary key?

`bigint` identity keys are small, fast to index and ordered, but reveal counts and need a central generator. UUIDs can be generated anywhere (useful for distributed systems and offline clients) but are 16 bytes, and random UUIDv4 values scatter inserts across the index. Time-ordered UUIDv7 (`uuidv7()` built in from PostgreSQL 18) keeps inserts local. → *More:* Ch 9 — Generated keys

Q115. How do you store and query time zones correctly?

Store instants as `timestamptz` (PostgreSQL stores UTC and converts on display using the session's `TimeZone`), and convert with `AT TIME ZONE 'Asia/Kolkata'` when you need local dates, for example "orders per local day". Use plain `timestamp` only for wall-clock values that are not instants, such as "store opens at 09:00". → *More:* Ch 3 — Date and time functions

Q116. How do you prevent double booking (two people booking the same seat or room)?

Make the database enforce it: a `UNIQUE (show_id, seat_no)` constraint for seats, or an exclusion constraint for time ranges, `EXCLUDE USING gist (room_id WITH =, during WITH &&)` (needs the `btree_gist` extension). A concurrent second booking then fails with a constraint error, whatever the application did. `SELECT … FOR UPDATE` on the seat row is the locking alternative. → *More:* Ch 12 — GIN, GiST, BRIN · Ch 13 — Write skew

Q117. How would you implement soft deletes, and what are the pitfalls?

Add `deleted_at timestamptz` and filter `WHERE deleted_at IS NULL` everywhere (a view helps). Pitfalls: every query must remember the filter, unique constraints must become partial (`UNIQUE (email) WHERE deleted_at IS NULL`), foreign keys still point at "deleted" rows, and the table keeps growing. Archiving deleted rows to another table is the alternative. → *More:* Ch 9 — DELETE · Ch 12 — Covering, partial and expression indexes

Q118. How do you keep a search index or cache in sync with the database?

Write to the database only, and publish changes from it: either the **transactional outbox** (insert an event row in the same transaction, a relay publishes it) or **change data capture** (CDC) that reads the WAL through logical decoding (Debezium). Dual writes from the application (database, then cache) drift when one write fails. → *More:* Ch 16 — Elasticsearch and Redis

Q119. What happens, step by step, when you run a `SELECT`?

The client sends the text; the **parser** checks syntax and builds a parse tree; the **analyzer/rewriter** resolves names and expands views; the **planner/optimizer** estimates the cost of alternative plans using statistics and picks the cheapest; the **executor** runs the plan, reading pages through the **buffer pool** (shared buffers) or from disk, and streams rows back. → *More:* Ch 1 — Inside a DBMS

Q120. Which PostgreSQL features after version 14 are worth knowing?

15: `MERGE`, `UNIQUE NULLS NOT DISTINCT`, `security_invoker` views. 16: `ANY_VALUE()`, logical replication from a standby, more parallelism. 17: `JSON_TABLE` and SQL/JSON functions, `MERGE … RETURNING`, incremental backups. 18: asynchronous I/O, B-tree skip scan, `uuidv7()`, virtual generated columns (the default), `OLD` / `NEW` in `RETURNING`, temporal `WITHOUT OVERLAPS` keys. → *More:* Ch 20 — Dialect differences

Part B — Coding problems

How to answer any of these in an interview: restate the output (one row per *what*?), name the pattern, write the query in small steps (a CTE per step), then check the edge cases: NULLs, ties, empty groups that must show 0, duplicates and date boundaries. The method is in §18.1 and the pattern catalogue in §19.11.

SQL 50 — Select

Q121. ⋆ Recyclable and Low Fat Products · LeetCode 1757 · Easy

Filter on both flag columns with `AND`: `WHERE low_fats = 'Y' AND recyclable = 'Y'`. → *Solved in the book:* Ch 18 — P3 · Green products

Q122. ⋆ Find Customer Referee · LeetCode 584 · Easy

`referee_id <> 2` is UNKNOWN for customers with no referee, so they vanish; add `OR referee_id IS NULL` (or write `referee_id IS DISTINCT FROM 2`). → *Solved in the book:* Ch 18 — P4 · Everyone outside department 3

Q123. Big Countries · LeetCode 595 · Easy

One `OR` filter: `WHERE area >= 3000000 OR population >= 25000000`, returning `name, population, area`. → *Solved in the book:* Ch 18 — P1 · Big cities

Q124. Article Views I · LeetCode 1148 · Easy

Keep rows where `author_id = viewer_id`, then `SELECT DISTINCT author_id AS id … ORDER BY id`, because one author can view their own articles many times. → *Solved in the book:* Ch 18 — P5 · Self-assigned tasks

Q125. Invalid Tweets · LeetCode 1683 · Easy

`WHERE char_length(content) > 15`. `length` / `char_length` count characters; `octet_length` would count bytes. → *Solved in the book:* Ch 18 — P6 · Long comments

SQL 50 — Basic joins

Q126. Replace Employee ID With The Unique Identifier · LeetCode 1378 · Easy

`LEFT JOIN`, because employees without a unique id must still appear (with NULL).

```sql
SELECT u.unique_id, e.name
FROM Employees e
LEFT JOIN EmployeeUNI u ON u.id = e.id;
```

Q127. Product Sales Analysis I · LeetCode 1068 · Easy

Plain inner join of `Sales` to `Product` on `product_id`, selecting `product_name, year, price`. → *Solved in the book:* Ch 18 — P17 · Product name for every sale

Q128. ⋆ Customer Who Visited but Did Not Make Any Transactions · LeetCode 1581 · Easy

Anti-join visits to transactions, then count the remaining visits per customer.

```sql
SELECT v.customer_id, count(*) AS count_no_trans
FROM Visits v
WHERE NOT EXISTS (SELECT 1 FROM Transactions t WHERE t.visit_id = v.visit_id)
GROUP BY v.customer_id;
```

Q129. ⋆ Rising Temperature · LeetCode 197 · Easy

Self-join each day to the calendar day before (`date - 1`), not to the previous row, because days can be missing. (`lag()` would compare with the previous *row*, which is wrong when a day is missing.)

```sql
SELECT w.id
FROM Weather w
JOIN Weather y ON y.recordDate = w.recordDate - 1
WHERE w.temperature > y.temperature;
```

Q130. Average Time of Process per Machine · LeetCode 1661 · Easy

Pair each process's `start` and `end` rows (self-join or conditional aggregation), subtract, then average per machine; cast to `numeric` before `round(…, 3)` because `timestamp` is a float column. → *Solved in the book:* Ch 18 — P18 · Average process time per machine

Q131. ⋆ Employee Bonus · LeetCode 577 · Easy

`LEFT JOIN` the bonus and keep `bonus < 1000 OR bonus IS NULL`; forgetting the NULL case drops employees with no bonus.

```sql
SELECT e.name, b.bonus
FROM Employee e
LEFT JOIN Bonus b ON b.empId = e.empId
WHERE b.bonus < 1000 OR b.bonus IS NULL;
```

Q132. ⋆ Students and Examinations · LeetCode 1280 · Easy

`CROSS JOIN` students × subjects to build every pair, `LEFT JOIN` the exams, and `count(e.subject_name)` so missing exams count 0. → *Solved in the book:* Ch 18 — P20 · Exam sittings per student and subject

Q133. ⋆ Managers with at Least 5 Direct Reports · LeetCode 570 · Medium

Self-join manager to employees, group by the manager, keep groups of 5 or more.

```sql
SELECT m.name
FROM Employee m
JOIN Employee e ON e.managerId = m.id
GROUP BY m.id, m.name
HAVING count(*) >= 5;
```

Q134. ⋆ Confirmation Rate · LeetCode 1934 · Medium

`LEFT JOIN` confirmations to every signup and average a 0/1 flag; a user with no messages gets NULL, which `COALESCE` turns into 0.

```sql
SELECT s.user_id,
       round(COALESCE(avg((c.action = 'confirmed')::int), 0), 2) AS confirmation_rate
FROM Signups s
LEFT JOIN Confirmations c ON c.user_id = s.user_id
GROUP BY s.user_id;
```

SQL 50 — Basic aggregate functions

Q135. Not Boring Movies · LeetCode 620 · Easy

`WHERE id % 2 = 1 AND description <> 'boring' ORDER BY rating DESC`. → *Solved in the book:* Ch 18 — P2 · Odd-numbered, not-boring movies

Q136. ⋆ Average Selling Price · LeetCode 1251 · Easy

Range join each sale to the price valid on its date (`purchase_date BETWEEN start_date AND end_date`), then revenue ÷ units; `LEFT JOIN` from `Prices` and `COALESCE(…, 0)` for products never sold. → *Solved in the book:* Ch 18 — P37 · Average selling price

Q137. Project Employees I · LeetCode 1075 · Easy

Join and average per project; `avg` of an integer is `numeric` in PostgreSQL, so `round(…, 2)` works directly.

```sql
SELECT p.project_id, round(avg(e.experience_years), 2) AS average_years
FROM Project p
JOIN Employee e ON e.employee_id = p.employee_id
GROUP BY p.project_id;
```

Q138. Percentage of Users Attended a Contest · LeetCode 1633 · Easy

Registrations per contest × 100 ÷ total users (a scalar subquery), rounded, sorted by percentage desc then `contest_id`. → *Solved in the book:* Ch 19 — 19.9.3 Share of members who borrowed each book

Q139. Queries Quality and Percentage · LeetCode 1211 · Easy

`avg(rating::numeric / position)` for quality and `100.0 * avg((rating < 3)::int)` for the poor percentage, both rounded to 2. LeetCode's test data includes rows with a NULL `query_name`, which the expected output leaves out, so add `WHERE query_name IS NOT NULL`. → *Solved in the book:* Ch 18 — P34 · Search query quality

Q140. ⋆ Monthly Transactions I · LeetCode 1193 · Medium

Group by month and country; conditional aggregation with `FILTER` gives the approved counts and amounts.

```sql
SELECT to_char(trans_date, 'YYYY-MM') AS month, country,
       count(*)                                          AS trans_count,
       count(*) FILTER (WHERE state = 'approved')        AS approved_count,
       sum(amount)                                       AS trans_total_amount,
       COALESCE(sum(amount) FILTER (WHERE state = 'approved'), 0) AS approved_total_amount
FROM Transactions
GROUP BY 1, 2;
```

Q141. ⋆ Immediate Food Delivery II · LeetCode 1174 · Medium

Keep each customer's first order (`DISTINCT ON` or `min(order_date)`), then the percentage where order date = preferred date. → *Solved in the book:* Ch 19 — 19.8.4 Same-day first bookings

Q142. ⋆ Game Play Analysis IV · LeetCode 550 · Medium

Find each player's first login, check whether a row exists for the next day, and divide by the number of players (rounded to 2). → *Solved in the book:* Ch 18 — P38 · Came back the next day

SQL 50 — Sorting and grouping

Q143. Number of Unique Subjects Taught by Each Teacher · LeetCode 2356 · Easy

The same subject can appear in several departments, so count distinct subjects.

```sql
SELECT teacher_id, count(DISTINCT subject_id) AS cnt
FROM Teacher
GROUP BY teacher_id;
```

Q144. User Activity for the Past 30 Days I · LeetCode 1141 · Easy

The 30 days ending 2019-07-27 *inclusive* start on 2019-06-28; count distinct users per day.

```sql
SELECT activity_date AS day, count(DISTINCT user_id) AS active_users
FROM Activity
WHERE activity_date BETWEEN DATE '2019-07-27' - 29 AND DATE '2019-07-27'
GROUP BY activity_date;
```

Q145. ⋆ Product Sales Analysis III · LeetCode 1070 · Medium

Find each product's first year, then return **all** sales rows from that year (several rows can tie), e.g. `WHERE (product_id, year) IN (SELECT product_id, min(year) … GROUP BY product_id)` or `rank() = 1`. → *Solved in the book:* Ch 19 — 19.3.8 First year on loan (keep the ties)

Q146. Classes More Than 5 Students · LeetCode 596 · Easy

Group by class and keep classes with at least 5 students (`(student, class)` is the key, so no `DISTINCT` is needed).

```sql
SELECT class FROM Courses GROUP BY class HAVING count(*) >= 5;
```

Q147. Find Followers Count · LeetCode 1729 · Easy

`GROUP BY user_id`, `count(follower_id)`, `ORDER BY user_id`. → *Solved in the book:* Ch 18 — P12 · Follower count

Q148. Biggest Single Number · LeetCode 619 · Easy

Keep numbers that occur once (`GROUP BY num HAVING count(*) = 1`), then take `max(num)` in an outer query, which also returns NULL when there is none. → *Solved in the book:* Ch 18 — P9 · Largest number that appears once

Q149. ⋆ Customers Who Bought All Products · LeetCode 1045 · Medium

Relational division: a customer qualifies when their count of *distinct* products equals the number of products, `HAVING count(DISTINCT product_key) = (SELECT count(*) FROM Product)`. → *Solved in the book:* Ch 19 — 19.7.6 Students enrolled in every course

SQL 50 — Advanced select and joins

Q150. The Number of Employees Which Report to Each Employee · LeetCode 1731 · Easy

Self-join manager to reports, count and average the reports' ages, rounded to an integer.

```sql
SELECT m.employee_id, m.name, count(*) AS reports_count, round(avg(e.age)) AS average_age
FROM Employees m
JOIN Employees e ON e.reports_to = m.employee_id
GROUP BY m.employee_id, m.name
ORDER BY m.employee_id;
```

Q151. Primary Department for Each Employee · LeetCode 1789 · Easy

Keep the row flagged `'Y'`, or the only row of employees with one department: `WHERE primary_flag = 'Y' OR employee_id IN (… HAVING count(*) = 1)`. → *Solved in the book:* Ch 19 — 19.3.3 One home office per employee

Q152. Triangle Judgement · LeetCode 610 · Easy

`CASE WHEN x + y > z AND x + z > y AND y + z > x THEN 'Yes' ELSE 'No' END AS triangle`. → *Solved in the book:* Ch 18 — P8 · Triangle test

Q153. ⋆ Consecutive Numbers · LeetCode 180 · Medium

Compare each row with the next two by id order using `lead(num, 1)` and `lead(num, 2)` and return `DISTINCT num` where all three match. Ids can have gaps, so use window order, not `id + 1`. → *Solved in the book:* Ch 19 — 19.5.2 Values repeated three times in a row

Q154. ⋆ Product Price at a Given Date · LeetCode 1164 · Medium

Latest change on or before 2019-08-16 per product (`DISTINCT ON (product_id) … ORDER BY product_id, change_date DESC`), and price 10 for products with no change by then. → *Solved in the book:* Ch 18 — P33 · Price on a given date

Q155. ⋆ Last Person to Fit in the Bus · LeetCode 1204 · Medium

Running total of weight in `turn` order; return the person with the largest turn whose running total is ≤ 1000. → *Solved in the book:* Ch 19 — 19.4.2 Last person who fits in the lift

Q156. Count Salary Categories · LeetCode 1907 · Medium

All three categories must appear even with 0 accounts, so start from a fixed list of categories and `LEFT JOIN` the classified accounts.

```sql
SELECT c.category, count(a.account_id) AS accounts_count
FROM (VALUES ('Low Salary', 1), ('Average Salary', 2), ('High Salary', 3)) AS c(category, k)
LEFT JOIN Accounts a
       ON c.k = CASE WHEN a.income < 20000 THEN 1 WHEN a.income <= 50000 THEN 2 ELSE 3 END
GROUP BY c.category, c.k;
```

SQL 50 — Subqueries

Q157. Employees Whose Manager Left the Company · LeetCode 1978 · Easy

Salary below 30000, a non-NULL manager, and no row for that manager any more (anti-join).

```sql
SELECT e.employee_id
FROM Employees e
WHERE e.salary < 30000 AND e.manager_id IS NOT NULL
  AND NOT EXISTS (SELECT 1 FROM Employees m WHERE m.employee_id = e.manager_id)
ORDER BY e.employee_id;
```

Q158. ⋆ Exchange Seats · LeetCode 626 · Medium

Odd ids take the next student, even ids the previous one; the last odd id keeps its own student when there is no next row.

```sql
SELECT id,
       CASE WHEN id % 2 = 1 THEN COALESCE(lead(student) OVER (ORDER BY id), student)
            ELSE lag(student) OVER (ORDER BY id) END AS student
FROM Seat
ORDER BY id;
```

Q159. Movie Rating · LeetCode 1341 · Medium

Two independent top-1 queries (most ratings by user; best February 2020 average by movie), each breaking ties alphabetically, stacked with `UNION ALL` (not `UNION`: a user name can equal a movie title). Each branch needs parentheses to have its own `ORDER BY` / `LIMIT`.

```
(SELECT u.name AS results
 FROM MovieRating r JOIN Users u ON u.user_id = r.user_id
 GROUP BY u.user_id, u.name ORDER BY count(*) DESC, u.name LIMIT 1)
UNION ALL
(SELECT m.title
 FROM MovieRating r JOIN Movies m ON m.movie_id = r.movie_id
 WHERE r.created_at >= DATE '2020-02-01' AND r.created_at < DATE '2020-03-01'
 GROUP BY m.movie_id, m.title ORDER BY avg(r.rating) DESC, m.title LIMIT 1);
```

Q160. ⋆ Restaurant Growth · LeetCode 1321 · Medium

Total per day first (several customers per day), then a 7-day moving sum with a `RANGE` frame of 6 days, and keep only days that have a full 7-day window.

```
WITH daily AS (SELECT visited_on, sum(amount) AS amount FROM Customer GROUP BY visited_on),
     w AS (SELECT visited_on,
                  sum(amount) OVER (ORDER BY visited_on
                                    RANGE BETWEEN INTERVAL '6 days' PRECEDING AND CURRENT ROW) AS amount,
                  min(visited_on) OVER () AS first_day
           FROM daily)
SELECT visited_on, amount, round(amount / 7.0, 2) AS average_amount
FROM w WHERE visited_on >= first_day + 6
ORDER BY visited_on;
```

Q161. ⋆ Friend Requests II: Who Has the Most Friends · LeetCode 602 · Medium

A friendship counts for both people, so stack requester ids and accepter ids with `UNION ALL`, count per id and take the top one. (The follow-up "return all ties" uses `rank() = 1`, as in the book's version.)

```
WITH ids AS (SELECT requester_id AS id FROM RequestAccepted
             UNION ALL
             SELECT accepter_id FROM RequestAccepted)
SELECT id, count(*) AS num FROM ids GROUP BY id ORDER BY num DESC LIMIT 1;
```

→ *Ties variant in the book:* Ch 19 — 19.3.9 Busiest collaborators (all ties)

Q162. Investments in 2016 · LeetCode 585 · Medium

Two window counts per row: rows sharing `tiv_2015` (must be > 1) and rows sharing `(lat, lon)` (must be 1); sum `tiv_2016` of the survivors. The columns are floats, so write `round(sum(tiv_2016)::numeric, 2)`. → *Solved in the book:* Ch 19 — 19.7.7 Shop revenue under two uniqueness rules

Q163. ⋆ Department Top Three Salaries · LeetCode 185 · Hard

`dense_rank() OVER (PARTITION BY departmentId ORDER BY salary DESC)` in a subquery, keep rank ≤ 3 (top three *distinct* salaries, ties included), join the department name. → *Solved in the book:* Ch 19 — 19.3.2 Top three distinct scores per subject

SQL 50 — Advanced string functions, regex and clauses

Q164. Fix Names in a Table · LeetCode 1667 · Easy

Upper-case the first letter, lower-case the rest. (`initcap` would also capitalise letters after spaces or hyphens.)

```
SELECT user_id, upper(left(name, 1)) || lower(substring(name FROM 2)) AS name
FROM Users
ORDER BY user_id;
```

Q165. Patients With a Condition · LeetCode 1527 · Easy

A code starting with `DIAB1` is at the start of the string or after a space; a plain `LIKE '%DIAB1%'` would wrongly match `ACNE +DIAB100`.

```sql
SELECT patient_id, patient_name, conditions
FROM Patients
WHERE conditions ~ '(^| )DIAB1';
```

Q166. ★ Delete Duplicate Emails · LeetCode 196 · Easy

A `DELETE` (not a `SELECT`) that removes every row for which a row with the same e-mail and a smaller id exists.

```sql
DELETE FROM Person p
USING Person q
WHERE p.email = q.email AND p.id > q.id;
```

Q167. ★ Second Highest Salary · LeetCode 176 · Medium

Second distinct value with `OFFSET 1`, wrapped in a scalar subquery so that "no second salary" returns one row with NULL instead of no rows: `SELECT (SELECT DISTINCT salary FROM Employee ORDER BY salary DESC OFFSET 1 LIMIT 1) AS "SecondHighestSalary"`. → *Solved in the book:* Ch 18 — P39 · Second-highest salary · Ch 19 — 19.3.1 Second-fastest lap time

Q168. ★ Group Sold Products By The Date · LeetCode 1484 · Easy

Per date: `count(DISTINCT product)` and `string_agg(DISTINCT product, ',' ORDER BY product)`. → *Solved in the book:* Ch 18 — P15 · Products sold per day · Ch 19 — 19.8.3 Titles borrowed per day

Q169. List the Products Ordered in a Period · LeetCode 1327 · Easy

Filter to February 2020 with a half-open range, total per product, keep totals ≥ 100.

```sql
SELECT p.product_name, sum(o.unit) AS unit
FROM Products p
JOIN Orders o ON o.product_id = p.product_id
WHERE o.order_date >= DATE '2020-02-01' AND o.order_date < DATE '2020-03-01'
GROUP BY p.product_id, p.product_name
HAVING sum(o.unit) >= 100;
```

Q170. Find Users With Valid E-Mails · LeetCode 1517 · Easy

Anchor a regular expression at both ends and escape the dot: `WHERE mail ~ '^[A-Za-z][A-Za-z0-9_.-]*@leetcode[.]com$'` (`~` is case-sensitive, so `@LEETCODE.COM` is rejected). → *Solved in the book:* Ch 18 — P7 · Valid company e-mails

Most-asked classics beyond SQL 50

Q171. ★ Combine Two Tables · LeetCode 175 · Easy

Every person must appear, with NULL city and state when there is no address: a `LEFT JOIN`.

```sql
SELECT p.firstName, p.lastName, a.city, a.state
FROM Person p
LEFT JOIN Address a ON a.personId = p.personId;
```

Q172. ★ Nth Highest Salary · LeetCode 177 · Medium

Inside a function, rank distinct salaries with `dense_rank` and pick rank N, wrapped in a scalar subquery so a missing Nth salary (or N < 1) gives NULL. Qualify columns (`e.salary`), because the output column `Salary` is also a PL/pgSQL variable and an unqualified `salary` is ambiguous.

```sql
CREATE OR REPLACE FUNCTION NthHighestSalary(N INT) RETURNS TABLE (Salary INT) AS $$
BEGIN
  RETURN QUERY
  SELECT (SELECT DISTINCT r.salary
          FROM (SELECT e.salary, dense_rank() OVER (ORDER BY e.salary DESC) AS dr
                FROM Employee e) r
          WHERE r.dr = N);
END;
$$ LANGUAGE plpgsql;
```

Q173. ★ Rank Scores · LeetCode 178 · Medium

`dense_rank()` gives consecutive ranks without gaps after ties.

```sql
SELECT score, dense_rank() OVER (ORDER BY score DESC) AS "rank"
FROM Scores
ORDER BY score DESC;
```

Q174. ⋆ Employees Earning More Than Their Managers · LeetCode 181 · Easy

Self-join each employee to their manager's row and compare salaries.

```sql
SELECT e.name AS "Employee"
FROM Employee e
JOIN Employee m ON m.id = e.managerId
WHERE e.salary > m.salary;
```

Q175. ⋆ Duplicate Emails · LeetCode 182 · Easy

Group by e-mail and keep groups with more than one row.

```sql
SELECT email AS "Email" FROM Person GROUP BY email HAVING count(*) > 1;
```

Q176. ⋆ Customers Who Never Order · LeetCode 183 · Easy

Anti-join. `NOT EXISTS` is safe even if `customerId` had NULLs, unlike `NOT IN`.

```sql
SELECT c.name AS "Customers"
FROM Customers c
WHERE NOT EXISTS (SELECT 1 FROM Orders o WHERE o.customerId = c.id);
```

Q177. ⋆ Department Highest Salary · LeetCode 184 · Medium

`rank() = 1` per department keeps everyone tied for the top salary.

```sql
SELECT d.name AS "Department", e.name AS "Employee", e.salary AS "Salary"
FROM (SELECT *, rank() OVER (PARTITION BY departmentId ORDER BY salary DESC) AS r
      FROM Employee) e
JOIN Department d ON d.id = e.departmentId
WHERE e.r = 1;
```

Q178. ⋆ Trips and Users · LeetCode 262 · Hard

Join trips to users twice (client and driver), keep only trips where both are unbanned and the date is 2013-10-01 to 2013-10-03, then per day `round(avg((status <> 'completed')::int), 2)` as the cancellation rate. → *Solved in the book:* Ch 19 — 19.9.1 Ride cancellation rate (the "trips and users" classic)

Q179. ⋆ Human Traffic of Stadium · LeetCode 601 · Hard

Keep rows with `people >= 100`, group consecutive ids with `id - row_number()` (gaps and islands), and return rows whose island has 3 or more rows, ordered by `visit_date`. → *Solved in the book:* Ch 19 — 19.5.1 Runs of busy days at the gate

Q180. ⋆ Tree Node · LeetCode 608 · Medium

Root has no parent; Inner has at least one child; everything else is a Leaf. Check "has a child" with `EXISTS` (a `NOT IN` on `p_id` would hit the NULL of the root).

```sql
SELECT id,
       CASE WHEN p_id IS NULL THEN 'Root'
            WHEN EXISTS (SELECT 1 FROM Tree c WHERE c.p_id = t.id) THEN 'Inner'
            ELSE 'Leaf' END AS type
FROM Tree t;
```

Q181. Swap Salary · LeetCode 627 · Easy

One `UPDATE` with a `CASE`; no temporary table is needed, because every row is computed from its old value.

```sql
UPDATE Salary SET sex = CASE sex WHEN 'm' THEN 'f' ELSE 'm' END;
```

Q182. Customer Placing the Largest Number of Orders · LeetCode 586 · Easy

Group by customer, order by the count and take one row (use `rank() = 1` for the follow-up "several customers tie").

```sql
SELECT customer_number
FROM Orders
GROUP BY customer_number
ORDER BY count(*) DESC
LIMIT 1;
```

Q183. Count Student Number in Departments · LeetCode 580 · Medium

Start from departments so empty ones show 0, and count a student column, not `*`.

```sql
SELECT d.dept_name, count(s.student_id) AS student_number
FROM Department d
LEFT JOIN Student s ON s.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_name
ORDER BY student_number DESC, d.dept_name;
```

Game Play Analysis series

Q184. ⋆ Game Play Analysis I · LeetCode 511 · Easy

`SELECT player_id, min(event_date) AS first_login FROM Activity GROUP BY player_id`. → *Solved in the book:* Ch 18 — P11 · First login per player

Q185. Game Play Analysis II · LeetCode 512 · Easy

First row per player: `DISTINCT ON (player_id) player_id, device_id … ORDER BY player_id, event_date`. → *Solved in the book:* Ch 18 — P40 · Device used at first login

Q186. ⋆ Game Play Analysis III · LeetCode 534 · Medium

Running total: `sum(games_played) OVER (PARTITION BY player_id ORDER BY event_date)`. → *Solved in the book:* Ch 18 — P41 · Games played so far

Q187. Game Play Analysis V · LeetCode 1097 · Hard

Install date = first login; per install date, count installs and the share who also played on install date + 1 (day-1 retention), rounded to 2. → *Solved in the book:* Ch 19 — 19.8.1 Day-1 retention

(Game Play Analysis IV is Q142.)

Ranking, top-N and medians

Q188. ⋆ Median Employee Salary · LeetCode 569 · Hard

Number rows by salary within each company and count them; the median rows are those whose position is between n/2 and n/2 + 1 (one row for odd n, two for even n).

```sql
SELECT id, company, salary
FROM (SELECT *, row_number() OVER (PARTITION BY company ORDER BY salary, id) AS rn,
              count(*)     OVER (PARTITION BY company)                      AS n
      FROM Employee) t
WHERE rn BETWEEN n / 2.0 AND n / 2.0 + 1;
```

Q189. ⋆ Find Median Given Frequency of Numbers · LeetCode 571 · Hard

A cumulative sum of frequencies gives each number's range of positions; a number is a median value when its range covers the middle position (total/2). Average those numbers.

```sql
WITH c AS (SELECT num, frequency,
                  sum(frequency) OVER (ORDER BY num) AS upto,
                  sum(frequency) OVER ()             AS total
           FROM Numbers)
SELECT round(avg(num)::numeric, 1) AS median
FROM c
WHERE upto >= total / 2.0 AND upto - frequency <= total / 2.0;
```

For a plain column, PostgreSQL's `percentile_cont(0.5) WITHIN GROUP (ORDER BY x)` computes the median directly.

Q190. Project Employees III · LeetCode 1077 · Medium

Most experienced employee(s) per project, ties kept: `rank() = 1`.

```sql
SELECT project_id, employee_id
FROM (SELECT p.project_id, p.employee_id,
             rank() OVER (PARTITION BY p.project_id ORDER BY e.experience_years DESC) AS r
      FROM Project p JOIN Employee e ON e.employee_id = p.employee_id) t
WHERE r = 1;
```

Q191. Sales Analysis I · LeetCode 1082 · Easy

Total per seller, then every seller tied for the maximum. A window function can rank aggregated values in the same query level.

```sql
SELECT seller_id
FROM (SELECT seller_id, rank() OVER (ORDER BY sum(price) DESC) AS r
      FROM Sales GROUP BY seller_id) t
WHERE r = 1;
```

Q192. ⋆ Highest Grade For Each Student · LeetCode 1112 · Medium

First row per student by grade descending, then the smallest course id.

```sql
SELECT DISTINCT ON (student_id) student_id, course_id, grade
FROM Enrollments
ORDER BY student_id, grade DESC, course_id;
```

Q193. Tournament Winners · LeetCode 1194 · Hard

Unpivot matches into (player, points) rows with `UNION ALL`, total per player, then the first player per group by total desc and `player_id` asc.

```sql
WITH pts AS (SELECT first_player AS player_id, first_score AS score FROM Matches
             UNION ALL
             SELECT second_player, second_score FROM Matches)
SELECT DISTINCT ON (p.group_id) p.group_id, p.player_id
FROM Players p
LEFT JOIN pts ON pts.player_id = p.player_id
GROUP BY p.group_id, p.player_id
ORDER BY p.group_id, COALESCE(sum(pts.score), 0) DESC, p.player_id;
```

Q194. Get the Second Most Recent Activity · LeetCode 1369 · Hard

Number each user's activities newest first and count them; keep row 2, or the only row when a user has one activity.

```sql
SELECT username, activity, startDate AS "startDate", endDate AS "endDate"
FROM (SELECT *, row_number() OVER (PARTITION BY username ORDER BY startDate DESC) AS rn,
                count(*)     OVER (PARTITION BY username)                          AS n
      FROM UserActivity) t
WHERE rn = 2 OR n = 1;
```

Q195. Average Salary: Departments VS Company · LeetCode 615 · Hard

Attach the company-wide monthly average to every salary row with a window, then group by month and department and compare.

```sql
WITH t AS (SELECT to_char(s.pay_date, 'YYYY-MM') AS pay_month, e.department_id, s.amount,
                  avg(s.amount) OVER (PARTITION BY to_char(s.pay_date, 'YYYY-MM')) AS company_avg
           FROM Salary s JOIN Employee e ON e.employee_id = s.employee_id)
SELECT pay_month, department_id,
       CASE WHEN avg(amount) > max(company_avg) THEN 'higher'
            WHEN avg(amount) < max(company_avg) THEN 'lower' ELSE 'same' END AS comparison
FROM t
GROUP BY pay_month, department_id;
```

Q196. Find Cumulative Salary of an Employee · LeetCode 579 · Hard

A `RANGE BETWEEN 2 PRECEDING AND CURRENT ROW` frame on the month number sums this month and the two calendar months before it (a missing month simply contributes nothing); then drop each employee's latest month.

```sql
SELECT id, month, "Salary"
FROM (SELECT id, month,
             sum(salary) OVER (PARTITION BY id ORDER BY month
                               RANGE BETWEEN 2 PRECEDING AND CURRENT ROW) AS "Salary",
             max(month) OVER (PARTITION BY id) AS last_month
      FROM Employee) t
WHERE month < last_month
ORDER BY id, month DESC;
```

Q197. The Most Recent Three Orders · LeetCode 1532 · Medium

Top-N per group: `row_number()` per customer by date descending, keep ≤ 3.

```sql
SELECT customer_name, customer_id, order_id, order_date
FROM (SELECT c.name AS customer_name, c.customer_id, o.order_id, o.order_date,
             row_number() OVER (PARTITION BY c.customer_id ORDER BY o.order_date DESC) AS rn
      FROM Customers c JOIN Orders o ON o.customer_id = c.customer_id) t
WHERE rn <= 3
ORDER BY customer_name, customer_id, order_date DESC;
```

Q198. Find the Quiet Students in All Exams · LeetCode 1412 · Hard

Rank each exam's scores both ascending and descending; a student is quiet if they took an exam and never had rank 1 in either direction. → *Solved in the book:* Ch 19 — 19.3.6 Pupils who are never top or bottom

Consecutive rows, streaks and ranges

Q199. ⋆ Report Contiguous Dates · LeetCode 1225 · Hard

Stack failed and succeeded days with their state, keep 2019, and apply gaps and islands per state (`date - row_number()`); each island becomes one period.

```sql
WITH days AS (SELECT fail_date AS d, 'failed' AS period_state FROM Failed
              UNION ALL
              SELECT success_date, 'succeeded' FROM Succeeded),
     g AS (SELECT d, period_state,
                  d - (row_number() OVER (PARTITION BY period_state ORDER BY d))::int AS grp
           FROM days
           WHERE d BETWEEN DATE '2019-01-01' AND DATE '2019-12-31')
SELECT period_state, min(d) AS start_date, max(d) AS end_date
FROM g
GROUP BY period_state, grp
ORDER BY start_date;
```

Q200. ⋆ Find the Start and End Number of Continuous Ranges · LeetCode 1285 · Medium

In a run of consecutive integers, `log_id - row_number()` is constant, so group by it.

```sql
SELECT min(log_id) AS start_id, max(log_id) AS end_id
FROM (SELECT log_id, log_id - row_number() OVER (ORDER BY log_id) AS grp FROM Logs) t
GROUP BY grp
ORDER BY start_id;
```

Q201. Consecutive Available Seats · LeetCode 603 · Easy

A free seat qualifies when the seat right before or after it is also free. The `::int = 1` cast works whether `free` is a boolean or a 0/1 integer.

```sql
SELECT DISTINCT a.seat_id
FROM Cinema a
JOIN Cinema b ON abs(a.seat_id - b.seat_id) = 1
WHERE a.free::int = 1 AND b.free::int = 1
ORDER BY a.seat_id;
```

Q202. ⋆ Active Users · LeetCode 1454 · Medium

De-duplicate to one row per user per day, then gaps and islands on the dates (`login_date - row_number()`) and keep islands of 5 or more days. → *Solved in the book:* Ch 19 — 19.5.3 Members active five days in a row

Q203. Longest Winning Streak · LeetCode 2173 · Hard

Mark each non-win as the start of a new group with a running count of losses/draws, count wins per group, take each player's maximum, and `COALESCE` to 0 for players who never won. → *Solved in the book:* Ch 19 — 19.5.5 Longest winning streak

Q204. Suspicious Bank Accounts · LeetCode 1843 · Medium

Total deposits per account per month, keep months over the limit, then check whether some month and the next calendar month both qualify (self-join on `month + interval '1 month'` or `lag`). → *Solved in the book:* Ch 19 — 19.5.6 Deposits over the limit two months running

Q205. Find Interview Candidates · LeetCode 1811 · Medium

Unpivot each contest into (contest_id, user) medal rows; a user qualifies with 3 golds overall or medals in 3 consecutive contest ids (gaps and islands on `contest_id`). → *Solved in the book:* Ch 19 — 19.5.4 Interview candidates from contest medals

Joins, anti-joins and hierarchies

Q206. ⋆ Market Analysis I · LeetCode 1158 · Medium

Put the 2019 date filter in the `ON` clause of the `LEFT JOIN`; in `WHERE` it would drop users with no 2019 orders.

```sql
SELECT u.user_id AS buyer_id, u.join_date, count(o.order_id) AS orders_in_2019
FROM Users u
LEFT JOIN Orders o ON o.buyer_id = u.user_id
                  AND o.order_date >= DATE '2019-01-01' AND o.order_date < DATE '2020-01-01'
GROUP BY u.user_id, u.join_date;
```

Q207. Top Travellers · LeetCode 1407 · Easy

`LEFT JOIN` rides, total with `COALESCE(…, 0)`, and group by the user id (two users can share a name).

```sql
SELECT u.name, COALESCE(sum(r.distance), 0) AS travelled_distance
FROM Users u
LEFT JOIN Rides r ON r.user_id = u.id
GROUP BY u.id, u.name
ORDER BY travelled_distance DESC, u.name;
```

Q208. Employees With Missing Information · LeetCode 1965 · Easy

`FULL JOIN` both tables and keep ids missing a name or a salary; with `USING`, `employee_id` is the merged non-NULL key.

```sql
SELECT employee_id
FROM Employees FULL JOIN Salaries USING (employee_id)
WHERE name IS NULL OR salary IS NULL
ORDER BY employee_id;
```

Q209. Students With Invalid Departments · LeetCode 1350 · Easy

Anti-join students to departments.

```sql
SELECT s.id, s.name
FROM Students s
WHERE NOT EXISTS (SELECT 1 FROM Departments d WHERE d.id = s.department_id);
```

Q210. Sales Analysis II · LeetCode 1083 · Easy

Per buyer, `bool_or` asks "did any of this buyer's rows match?": bought S8 and never an iPhone.

```sql
SELECT s.buyer_id
FROM Sales s JOIN Product p ON p.product_id = s.product_id
GROUP BY s.buyer_id
HAVING bool_or(p.product_name = 'S8') AND NOT bool_or(p.product_name = 'iPhone');
```

Q211. ⋆ Sales Analysis III · LeetCode 1084 · Easy

"Sold only in Q1 2019" means the earliest and latest sale dates are both inside the quarter.

```sql
SELECT p.product_id, p.product_name
FROM Product p JOIN Sales s ON s.product_id = p.product_id
GROUP BY p.product_id, p.product_name
HAVING min(s.sale_date) >= DATE '2019-01-01' AND max(s.sale_date) <= DATE '2019-03-31';
```

Q212. Customers Who Bought Products A and B but Not C · LeetCode 1398 · Medium

Same "has / has not" pattern with one `bool_or` per product.

```sql
SELECT c.customer_id, c.customer_name
FROM Customers c JOIN Orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.customer_name
HAVING bool_or(o.product_name = 'A') AND bool_or(o.product_name = 'B')
   AND NOT bool_or(o.product_name = 'C')
ORDER BY c.customer_id;
```

Q213. All People Report to the Given Manager · LeetCode 1270 · Medium

Walk down from the head (id 1, who is their own manager) with a recursive CTE; excluding id 1 in the anchor stops the self-loop. (The problem caps the depth at 3, so three self-joins also work.)

```sql
WITH RECURSIVE r AS (
  SELECT employee_id FROM Employees WHERE manager_id = 1 AND employee_id <> 1
  UNION
  SELECT e.employee_id FROM Employees e JOIN r ON e.manager_id = r.employee_id
)
SELECT employee_id FROM r;
```

Q214. Find the Subtasks That Did Not Execute · LeetCode 1767 · Hard

Generate every expected subtask with `generate_series` per task, then anti-join the executed ones.

```sql
SELECT t.task_id, s.subtask_id
FROM Tasks t
CROSS JOIN LATERAL generate_series(1, t.subtasks_count) AS s(subtask_id)
WHERE NOT EXISTS (SELECT 1 FROM Executed e
                  WHERE e.task_id = t.task_id AND e.subtask_id = s.subtask_id);
```

Q215. Second Degree Follower · LeetCode 614 · Medium

A second-degree follower both follows someone and is followed by someone: keep followees who also appear as followers, and count their followers. → *Solved in the book:* Ch 18 — P22 · "Middle" users in a follow graph

Q216. ⋆ Sellers With No Sales · LeetCode 1607 · Easy

Anti-join sellers to 2020 orders (`NOT EXISTS` with a half-open date range), ordered by name. → *Solved in the book:* Ch 18 — P23 · Sellers with no 2020 sales · Ch 18 — P24 · Dormant accounts

Q217. Ad-Free Sessions · LeetCode 1809 · Easy

Anti-join on a range: no ad for the same customer with `timestamp BETWEEN start_time AND end_time`. → *Solved in the book:* Ch 18 — P25 · Sessions with no ads

Q218. ⋆ Sales Person · LeetCode 607 · Easy

Sales people with no order for the company named `'RED'`: `NOT EXISTS` over orders joined to company. → *Solved in the book:* Ch 18 — P26 · Reps who never sold to RED

Q219. Winning Candidate · LeetCode 574 · Medium

Count votes per candidate, order by the count descending, `LIMIT 1`, and join the name. → *Solved in the book:* Ch 18 — P21 · Election winner

Q220. Page Recommendations · LeetCode 1264 · Medium

Friends of user 1 come from both columns of `Friendship`; suggest their liked pages except pages user 1 already likes, `DISTINCT`. → *Solved in the book:* Ch 19 — 19.7.1 Page suggestions from friends' likes

Aggregation, rates and reports

Q221. Actors and Directors Who Cooperated At Least Three Times · LeetCode 1050 · Easy

```sql
SELECT actor_id, director_id
FROM ActorDirector
GROUP BY actor_id, director_id
HAVING count(*) >= 3;
```

Q222. Capital Gain/Loss · LeetCode 1393 · Medium

Sells add the price, buys subtract it; a signed `CASE` inside `sum`.

```sql
SELECT stock_name,
       sum(CASE operation WHEN 'Sell' THEN price ELSE -price END) AS capital_gain_loss
FROM Stocks
GROUP BY stock_name;
```

Q223. Bank Account Summary II · LeetCode 1587 · Easy

```sql
SELECT u.name, sum(t.amount) AS balance
FROM Users u JOIN Transactions t ON t.account = u.account
GROUP BY u.account, u.name
HAVING sum(t.amount) > 10000;
```

Q224. Daily Leads and Partners · LeetCode 1693 · Easy

```sql
SELECT date_id, make_name,
       count(DISTINCT lead_id) AS unique_leads, count(DISTINCT partner_id) AS unique_partners
FROM DailySales
GROUP BY date_id, make_name;
```

Q225. Find Total Time Spent by Each Employee · LeetCode 1741 · Easy

```sql
SELECT event_day AS day, emp_id, sum(out_time - in_time) AS total_time
FROM Employees
GROUP BY event_day, emp_id;
```

Q226. The Latest Login in 2020 · LeetCode 1890 · Easy

A half-open range on the timestamp (`< 2021-01-01`) keeps logins late on 31 December, which `<= '2020-12-31'` would miss.

```sql
SELECT user_id, max(time_stamp) AS last_stamp
FROM Logins
WHERE time_stamp >= '2020-01-01' AND time_stamp < '2021-01-01'
GROUP BY user_id;
```

Q227. Calculate Special Bonus · LeetCode 1873 · Easy

```sql
SELECT employee_id,
       CASE WHEN employee_id % 2 = 1 AND name NOT LIKE 'M%' THEN salary ELSE 0 END AS bonus
FROM Employees
ORDER BY employee_id;
```

Q228. Customer Order Frequency · LeetCode 1511 · Easy

Two conditional sums, one per month, both required in `HAVING`.

```sql
SELECT c.customer_id, c.name
FROM Customers c
JOIN Orders o  ON o.customer_id = c.customer_id
JOIN Product p ON p.product_id = o.product_id
GROUP BY c.customer_id, c.name
HAVING sum(o.quantity * p.price) FILTER (WHERE o.order_date >= '2020-06-01' AND o.order_date < '2020-07-01') >=
100
   AND sum(o.quantity * p.price) FILTER (WHERE o.order_date >= '2020-07-01' AND o.order_date < '2020-08-01') >=
100;
```

Q229. Warehouse Manager · LeetCode 1571 · Easy

```sql
SELECT w.name AS warehouse_name, sum(w.units * p.width * p.length * p.height) AS volume
FROM Warehouse w JOIN Products p ON p.product_id = w.product_id
GROUP BY w.name;
```

Q230. Number of Calls Between Two Persons · LeetCode 1699 · Medium

Normalise each pair with `least`/`greatest` so (1, 2) and (2, 1) fall into the same group.

```sql
SELECT least(from_id, to_id) AS person1, greatest(from_id, to_id) AS person2,
       count(*) AS call_count, sum(duration) AS total_duration
FROM Calls
GROUP BY 1, 2;
```

Q231. Reported Posts · LeetCode 1113 · Easy

"Yesterday" is 2019-07-04; a post reported twice for the same reason counts once.

```sql
SELECT extra AS report_reason, count(DISTINCT post_id) AS report_count
FROM Actions
WHERE action = 'report' AND action_date = DATE '2019-07-04'
GROUP BY extra;
```

Q232. Immediate Food Delivery I · LeetCode 1173 · Easy

The average of a 0/1 flag is a fraction; × 100 for a percentage.

```sql
SELECT round(100.0 * avg((order_date = customer_pref_delivery_date)::int), 2) AS immediate_percentage
FROM Delivery;
```

Q233. Friend Requests I: Overall Acceptance Rate · LeetCode 597 · Easy

Distinct accepted pairs ÷ distinct requested pairs, 0.00 when there are no requests (`NULLIF` avoids division by zero, `COALESCE` turns the NULL into 0).

```sql
SELECT round(COALESCE(
        (SELECT count(*) FROM (SELECT DISTINCT requester_id, accepter_id FROM RequestAccepted) a)::numeric
        / NULLIF((SELECT count(*) FROM (SELECT DISTINCT sender_id, send_to_id FROM FriendRequest) r), 0),
      0), 2) AS accept_rate;
```

Q234. Get Highest Answer Rate Question · LeetCode 578 · Medium

Answer rate = answers ÷ shows per question; order by it (NULLs last for questions never shown) and take the smallest id on ties.

```sql
SELECT question_id AS survey_log
FROM SurveyLog
GROUP BY question_id
ORDER BY count(*) FILTER (WHERE action = 'answer')::numeric
         / NULLIF(count(*) FILTER (WHERE action = 'show'), 0) DESC NULLS LAST,
         question_id
LIMIT 1;
```

Q235. Monthly Transactions II · LeetCode 1205 · Medium

Stack approved transactions and chargebacks (chargebacks dated by their own date, with the original amount) using `UNION ALL`, then pivot per month and country with conditional sums. → *Solved in the book:* Ch 18 — P28 · Monthly approvals and chargebacks

Q236. Weather Type in Each Country · LeetCode 1294 · Easy

Filter November 2019, average per country, then `CASE` on the average (≤ 15 Cold, ≥ 25 Hot, else Warm). → *Solved in the book:* Ch 18 — P29 · Weather class per country in November 2019

Q237. Ads Performance · LeetCode 1322 · Easy

CTR = clicks ÷ (clicks + views) × 100, and 0 when both are zero (`COALESCE(… / NULLIF(…, 0), 0)`), sorted by CTR desc then `ad_id`. → *Solved in the book:* Ch 18 — P35 · Ad click-through rate

Q238. Active Businesses · LeetCode 1126 · Medium

Attach each event type's average with `avg(occurences) OVER (PARTITION BY event_type)`, keep rows above it, and return businesses with more than one such row. → *Solved in the book:* Ch 18 — P42 · Active businesses

Q239. Running Total for Different Genders · LeetCode 1308 · Medium

`sum(score_points) OVER (PARTITION BY gender ORDER BY day)`, ordered by gender and day. → *Solved in the book:* Ch 19 — 19.4.1 Running total per team

Q240. Popularity Percentage · LeetCode 2720 · Hard

Count each user's friends from both columns (`UNION ALL` of the two directions), divide by the number of distinct users × 100, round to 2. → *Solved in the book:* Ch 19 — 19.7.2 Popularity percentage

Q241. Team Scores in Football Tournament · LeetCode 1212 · Medium

For every team (`LEFT JOIN` so teams with no matches get 0), sum 3 / 1 / 0 points for wins, draws and losses as host or guest; sort by points desc, then `team_id`. → *Solved in the book:* Ch 19 — 19.7.4 League table

Q242. Number of Transactions per Visit · LeetCode 1336 · Hard

Count transactions per visit (`LEFT JOIN`, so visits with none count 0), build every k from 0 to the maximum with `generate_series`, and count visits per k with a `LEFT JOIN` so empty buckets show 0. → *Solved in the book:* Ch 19 — 19.8.2 Transactions per visit histogram

Pivot, unpivot and string building

Q243. ⋆ Reformat Department Table · LeetCode 1179 · Easy

Static pivot: one conditional aggregate per month.

```sql
SELECT id,
  sum(revenue) FILTER (WHERE month = 'Jan') AS "Jan_Revenue", sum(revenue) FILTER (WHERE month = 'Feb') AS
"Feb_Revenue",
  sum(revenue) FILTER (WHERE month = 'Mar') AS "Mar_Revenue", sum(revenue) FILTER (WHERE month = 'Apr') AS
"Apr_Revenue",
  sum(revenue) FILTER (WHERE month = 'May') AS "May_Revenue", sum(revenue) FILTER (WHERE month = 'Jun') AS
"Jun_Revenue",
  sum(revenue) FILTER (WHERE month = 'Jul') AS "Jul_Revenue", sum(revenue) FILTER (WHERE month = 'Aug') AS
"Aug_Revenue",
  sum(revenue) FILTER (WHERE month = 'Sep') AS "Sep_Revenue", sum(revenue) FILTER (WHERE month = 'Oct') AS
"Oct_Revenue",
  sum(revenue) FILTER (WHERE month = 'Nov') AS "Nov_Revenue", sum(revenue) FILTER (WHERE month = 'Dec') AS
"Dec_Revenue"
FROM Department
GROUP BY id;
```

Q244. ⋆ Rearrange Products Table · LeetCode 1795 · Easy

Unpivot: a `LATERAL VALUES` list turns the three store columns into rows; drop NULL prices. (`UNION ALL` of three `SELECT`s also works.)

```sql
SELECT p.product_id, v.store, v.price
FROM Products p
CROSS JOIN LATERAL (VALUES ('store1', p.store1), ('store2', p.store2), ('store3', p.store3))
     AS v(store, price)
WHERE v.price IS NOT NULL;
```

Q245. Grand Slam Titles · LeetCode 1783 · Medium

Unpivot the four tournament columns into one column of winner ids, then count per player.

```sql
SELECT p.player_id, p.player_name, count(*) AS grand_slams_count
FROM Championships c
CROSS JOIN LATERAL (VALUES (c.wimbledon), (c.fr_open), (c.us_open), (c.au_open)) AS w(player_id)
JOIN Players p ON p.player_id = w.player_id
GROUP BY p.player_id, p.player_name;
```

Q246. Students Report By Geography · LeetCode 618 · Hard

Number students alphabetically within each continent (`row_number`), then pivot on that number with one `max(name) FILTER (WHERE continent = …)` per continent. → *Solved in the book:* Ch 19 — 19.6.1 Names by region, side by side

Q247. Dynamic Pivoting of a Table · LeetCode 2252 · Hard

The column list is not known in advance, so a procedure reads the distinct stores, builds the pivot query as text with `format('%I')` for identifiers, and runs it with `EXECUTE`. → *Solved in the book:* Ch 19 — 19.6.2 Dynamic pivot: one column per store

Q248. Dynamic Unpivoting of a Table · LeetCode 2253 · Hard

Read the store column names from the catalog (`information_schema.columns`), build one `SELECT … UNION ALL` branch per column with `format`, and `EXECUTE` it. → *Solved in the book:* Ch 19 — 19.6.3 Dynamic unpivot: store columns back to rows

Q249. Evaluate Boolean Expression · LeetCode 1440 · Medium

Join `Variables` twice (left and right operand), then `CASE operator WHEN '<' THEN l.value < r.value … END`, turned into `'true'`/`'false'`. → *Solved in the book:* Ch 19 — 19.10.1 Evaluate stored comparisons

Q250. Build the Equation · LeetCode 2118 · Hard

Format each term (sign, absolute coefficient, `X^power`, `X` or nothing) and join them with `string_agg(term, '' ORDER BY power DESC) || '=0'`. → *Solved in the book:* Ch 19 — 19.10.2 Build a polynomial equation string

Sources of the question lists

Question titles and wording only were taken from these lists; every answer and solution above is written for this book.

- LeetCode — *SQL 50* study plan (leetcode.com/studyplan/top-sql-50) and the LeetCode Database problem set (problem titles and numbers).
- LeetCode Discuss — "SQL Interview Questions Bootcamp" study guide and SQL interview-experience posts.
- GitHub mirrors of the SQL 50 plan: Karthik-S-R/Leetcode-SQL-50, HarshitLaloriya/Leetcode-SQL-50 (section sizes 5/9/8/7/7/7/7), HamzaHassanain/Leetcode_SQL_50_Study_Plan.
- devinterview-io/databases-interview-questions (GitHub) — concept topics: ACID, isolation levels, indexes, normalization.
- "Top 10 frequently asked SQL coding interview questions" (Medium) and "10 questions you must know to ace any SQL interviews" (Towards Data Science).
- Public "Top SQL questions LeetCode" lists (Scribd uploads), used only to confirm which classic problems recur.